

SAS[®] Certification Prep Guide: Base Programming for SAS[®] 9, Fourth Edition

The correct bibliographic citation for this manual is as follows: SAS Institute Inc. 2017. *SAS® Certification Prep Guide: Base Programming for SAS®9, Fourth Edition*. Cary, NC: SAS Institute Inc.

SAS® Certification Prep Guide: Base Programming for SAS®9, Fourth Edition

Copyright © 2017, SAS Institute Inc., Cary, NC, USA

ISBN 978-1-63526-373-2 (Hard copy)

ISBN 978-1-63526-468-5 (Epub)

ISBN 978-1-63526-469-2 (Mobi)

ISBN 978-1-63526-470-8 (PDF)

All Rights Reserved. Produced in the United States of America.

For a hard copy book: No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, or otherwise, without the prior written permission of the publisher, SAS Institute Inc.

For a web download or e-book: Your use of this publication shall be governed by the terms established by the vendor at the time you acquire this publication.

The scanning, uploading, and distribution of this book via the Internet or any other means without the permission of the publisher is illegal and punishable by law. Please purchase only authorized electronic editions and do not participate in or encourage electronic piracy of copyrighted materials. Your support of others' rights is appreciated.

U.S. Government License Rights; Restricted Rights: The Software and its documentation is commercial computer software developed at private expense and is provided with RESTRICTED RIGHTS to the United States Government. Use, duplication, or disclosure of the Software by the United States Government is subject to the license terms of this Agreement pursuant to, as applicable, FAR 12.212, DFAR 227.7202-1(a), DFAR 227.7202-3(a), and DFAR 227.7202-4, and, to the extent required under U.S. federal law, the minimum restricted rights as set out in FAR 52.227-19 (DEC 2007). If FAR 52.227-19 is applicable, this provision serves as notice under clause (c) thereof and no other notice is required to be affixed to the Software or documentation. The Government's rights in Software and documentation shall be only those set forth in this Agreement.

SAS Institute Inc., SAS Campus Drive, Cary, NC 27513-2414

January 2018

SAS® and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.

Other brand and product names are trademarks of their respective companies.

P2:certpgbp

Contents

<i>How to Prepare for the SAS Base Programming for SAS®9 Exam</i>	<i>ix</i>
<i>Accessibility Features of the SAS Certification Prep Guide</i>	<i>xiii</i>
Chapter 1 • Setting Up Practice Data	1
Determine What SAS Solution or Environment You Are Using	1
Determine Whether You Have Write Access	1
SAS Windowing Environment	2
SAS Studio and SAS University Edition	2
SAS Enterprise Guide	3
Chapter 2 • Basic Concepts	5
Getting Started	6
The Basics of the SAS Language	6
SAS Libraries	11
Referencing SAS Files	13
SAS Data Sets	15
Chapter Quiz	22
Chapter 3 • Setting Up Your SAS Session	27
SAS Libraries	28
Viewing SAS Libraries	31
Specifying Results Formats	36
Setting System Options	41
Chapter Quiz	48
Chapter 4 • Identifying and Correcting Errors	51
Error Messages	51
Correcting Common Errors	53
Chapter Quiz	62
Chapter 5 • Creating List Reports	67
Creating a Basic Report	68
Selecting Variables	69
Identifying Observations	70
Sorting Data	74
Generating Column Totals	76
Double Spacing LISTING Output	82
Specifying Titles and Footnotes in Procedure Output	82
Assigning Descriptive Labels	88
Formatting Data Values	90
Using Permanently Assigned Labels and Formats	93
Chapter Quiz	94
Chapter 6 • Creating SAS Data Sets from External Files	99
Raw Data Files	100
Creating a SAS Data Set from a Raw Data File	101
Referencing a Raw Data File	102
DATA Step Statements for Reading Data	104
Reading and Verifying the Data	108

Creating and Modifying Variables	111
Subsetting Data	115
Reading Instream Data	116
Creating a Raw Data File	118
Reading Microsoft Excel Data	121
LIBNAME Statement Options	129
Creating Excel Worksheets	130
The IMPORT Procedure	130
Chapter Quiz	134
Chapter 7 • Understanding DATA Step Processing	139
How SAS Processes Programs	139
Compilation Phase	141
Execution Phase	144
Debugging a DATA Step	152
Testing Your Programs	158
Chapter Quiz	161
Chapter 8 • Creating and Applying User-Defined Formats	165
The FORMAT Procedure	165
Defining a Unique Format	167
Associating User-Defined Formats with Variables	169
Chapter Quiz	172
Chapter 9 • Producing Descriptive Statistics	175
Computing Statistics Using PROC MEANS	176
Selecting Statistics	177
Limiting Decimal Places	180
Specifying Variables in PROC MEANS	181
Group Processing Using the CLASS Statement	182
Group Processing Using the BY Statement	183
Creating a Summarized Data Set Using PROC MEANS	184
Creating a Summarized Data Set Using PROC SUMMARY	186
Producing Frequency Tables Using PROC FREQ	187
Specifying Variables in PROC FREQ	189
Creating Two-Way Tables	191
Creating N-Way Tables	192
Creating Tables in List Format	194
Chapter Quiz	198
Chapter 10 • Creating Output with ODS	203
The Output Delivery System (ODS)	204
Creating HTML Output with ODS	206
Creating PDF Output with ODS	215
Creating RTF Output with ODS	220
Creating EXCEL Output with ODS	222
Chapter Quiz	228
Chapter 11 • Creating and Managing Variables	231
Creating and Modifying Variables	232
Assigning Values Conditionally	234
Specifying Lengths for Variables	238
Subsetting Data	240
Assigning Permanent Labels and Formats	243
Assigning Values Conditionally Using SELECT Groups	245
Grouping Statements Using DO Groups	248

Chapter Quiz	250
Chapter 12 • Reading SAS Data Sets	255
Reading a Single Data Set to Create Another	255
Manipulating Data	257
Using BY-Group Processing	259
Writing Observations Explicitly	263
Detecting the End of a Data Set	264
Chapter Quiz	273
Chapter 13 • Combining SAS Data Sets	277
How to Prepare Your Data Sets	278
Methods of Combining SAS Data Sets: The Basics	278
One-to-One Reading: Details	280
Concatenating: Details	284
Appending: Details	286
Interleaving: Details	289
Match-Merging: Details	291
Match-Merge Processing	295
Renaming Variables	300
Excluding Unmatched Observations	302
Selecting Variables	304
Chapter Quiz	305
Chapter 14 • Using Functions to Manipulate Data	313
The Basics of SAS Functions	314
SAS Functions Syntax	315
Converting Data with Functions	316
Manipulating SAS Date Values with Functions	323
Modifying Character Values with Functions	340
Modifying Numeric Values with Functions	360
Nesting SAS Functions	362
Chapter Quiz	362
Chapter 15 • Processing Data with DO Loops	367
The Basics of DO Loops	367
Constructing DO Loops	370
Nesting DO Loops	373
Iteratively Processing Observations from a Data Set	375
Conditionally Executing DO Loops	376
Using Conditional Clauses with the Iterative DO Statement	377
Chapter Quiz	379
Chapter 16 • Processing Data with Arrays	383
The Basics of SAS Arrays	383
Creating One-Dimensional Arrays	384
Expanding Your Use of Arrays	391
Chapter Quiz	397
Chapter 17 • Reading Raw Data in Fixed Fields	401
Identifying the Type of Numeric Data	401
Using Formatted Input	403
Using Informats	406
Using Record Formats	410
Chapter Quiz	413

Chapter 18 • Reading and Creating Raw Data in Free-Format	417
Free-Format Data	417
Reading Standard Data with List Input	418
Specifying the Length of Character Variables	421
Reading Free-Format Data with Non-Blank Delimiters and Missing Values	424
Reading Nonstandard Data with List Input	431
Creating Free-Format Data	436
Write a Comma-Delimited File Using Formats	439
Chapter Quiz	441
Chapter 19 • SAS Date and Time Values	447
Reading Dates and Times with Informats	447
Using Dates and Times in Calculations	451
Displaying Date and Time Values with Formats	456
Chapter Quiz	458
Chapter 20 • Creating a Single Observation from Multiple Raw Data Records	461
A Word About Creating a Single Observation	461
Using Line Pointer Controls	462
Reading Multiple Records Sequentially	462
Reading Multiple Records Non-Sequentially	469
Combining Line Pointer Controls	473
Chapter Quiz	474
Chapter 21 • Creating Multiple Observations from a Single Raw Data Record	479
A Word About Creating Multiple Observations	479
Reading Repeating Blocks of Data	480
Reading the Same Number of Repeating Fields	487
Reading a Varying Number of Repeating Fields	495
Chapter Quiz	501
Chapter 22 • Conditioning Raw Data Files	507
A Word About Conditioning Raw Data Files	507
Creating One Observation per Detail Record	508
Creating One Observation per Header Record	515
Processing a DATA Step That Creates One Observation per Header Record	520
Chapter Quiz	522
Appendix 1 • Quiz Answer Keys	527
Chapter 2: Basic Concepts	527
Chapter 3: Setting Up Your SAS Session	528
Chapter 4: Identifying and Correcting Errors	529
Chapter 5: Creating List Reports	531
Chapter 6: Creating SAS Data Sets from External Files	532
Chapter 7: Understanding DATA Step Processing	533
Chapter 8: Creating and Applying User-Defined Formats	534
Chapter 9: Producing Descriptive Statistics	535
Chapter 10: Creating Output with ODS	536
Chapter 11: Creating and Managing Variables	537
Chapter 12: Reading SAS Data Sets	538
Chapter 13: Combining SAS Data Sets	539
Chapter 14: Using Functions to Manipulate Data	541
Chapter 15: Processing Data with DO Loops	542
Chapter 16: Processing Data with Arrays	542
Chapter 17: Reading Raw Data in Fixed Fields	543
Chapter 18: Reading and Creating Raw Data in Free-Format	544

Chapter 19: SAS Date and Time Values	545
Chapter 20: Creating a Single Observation from Multiple Raw Data Records	546
Chapter 21: Creating Multiple Observations from a Single Raw Data Record	547
Chapter 22: Conditioning Raw Data Files	548
Index	551

How to Prepare for the SAS Base Programming for SAS®9 Exam

Requirements and Details

Requirements

To complete examples in this book, you must have access to Base SAS, SAS Enterprise Guide, or SAS Studio. See [Chapter 1, “Setting Up Practice Data,”](#) to ensure you have proper access.

Exam Objectives and Updates to This Book

The current exam objectives and a list of any updates to this book are available at www.sas.com/certify. Exam objectives are subject to change.

Take a Practice Exam

Practice exams are available for purchase through SAS and Pearson VUE. For more information about practice exams, see www.sas.com/base_programmer_cert.

Registering for the Exam

To register for the official SAS Base Programming for SAS®9 exam, see the SAS Global Certification website at www.sas.com/certify.

Additional Resources for Learning SAS Programming

From SAS Software

Help	• SAS®9: Select Help ⇒ SAS Help and Documentation. • SAS Enterprise Guide: Select Help ⇒ SAS Enterprise Guide Help. • SAS Studio: Select the Help icon .
Documentation	• SAS®9: Select Help ⇒ SAS Help and Documentation. • SAS Enterprise Guide: Access online documentation on the web. • SAS Studio: Select the Help icon  and then click Help.

On the Web	
Base SAS Glossary	support.sas.com/baseglossary
Bookstore	www.sas.com/books
Certification	www.sas.com/certify
Community	communities.sas.com
Knowledge Base	support.sas.com/notes
Learning Center	www.sas.com and click Learn . Then select Get Started Learning SAS .
SAS Documentation	support.sas.com/documentation documentation.sas.com
SAS Global Academic Program	www.sas.com and click Learn . Then select For Students and Educators .
SAS OnDemand	support.sas.com/ondemand/
Training	www.sas.com/training
Technical Support	support.sas.com . Then select Technical Support .

Syntax Conventions

In this book, SAS syntax looks like this example:

```
DATA output-SAS-data-set
    (DROP=variables(s) | KEEP=variables(s));
SET SAS-data-set <options>;
BY variable(s)
RUN;
```

Here are the conventions that are used in the example:

- DATA, DROP=, KEEP=, SET, BY, and RUN are in uppercase bold because they must be spelled as shown.
- *output-SAS-data-set*, *variable(s)*, *SAS-data-set*, and *options* are in italics because each represents a value that you supply.
- <*options*> is enclosed in angle brackets because it is optional syntax.
- DROP= and KEEP= are separated by a vertical bar (|) to indicate that they are mutually exclusive.

The example syntax that is shown in this book includes only what you need to know in order to prepare for the certification exam. For complete syntax, see the appropriate SAS reference guide.

Accessibility Features of the SAS Certification Prep Guide

Overview

The *SAS Certification Prep Guide: Base Programming* is a test preparation document that uses the following environments and products:

- SAS Windowing Environment
- SAS Enterprise Guide
- SAS Studio or SAS University Edition

Accessibility Documentation Help

The following table contains accessibility information for the listed products:

Accessibility Documentation Links

Product or Environment	Where to Find Accessibility Documentation
Base SAS (Microsoft Windows, UNIX, and z/OS)	support.sas.com/baseaccess
SAS Enterprise Guide	support.sas.com/documentation/onlinedoc/guide/index.html
SAS Studio	support.sas.com/studioaccess

Documentation Format

Contact accessibility@sas.com if you need this document in an alternative digital format.

Chapter 1

Setting Up Practice Data

Determine What SAS Solution or Environment You Are Using	1
Determine Whether You Have Write Access	1
SAS Windowing Environment	2
SAS Studio and SAS University Edition	2
SAS Enterprise Guide	3

Determine What SAS Solution or Environment You Are Using

This book assumes you are running SAS windowing environment, SAS Enterprise Guide, SAS University Edition, or SAS Studio. If you are not sure which of these you are using, select **Help** ⇒ **About** from the SAS software main menu.

Determine Whether You Have Write Access

If you are using SAS Studio or SAS University Edition, you might not have Write access to the Sasuser directory where the sample data is stored.

Note: If you are using the SAS windowing environment or SAS Enterprise Guide, you can skip this section.

Submit the following code:

```
proc options option=rsasuser;
run;
```

If the result from the OPTIONS procedure code is NORSASUSER, the Sasuser folder is writable.

If the result from the PROC OPTIONS code is RSASUSER, the Sasuser folder is Read only, and you must redirect the Sasuser folder by using a LIBNAME statement. To set up practice data:

1. In the **Folders** pane, select **My Folders**. Then, right-click and select **New** ⇒ **Folder**.

2. In the **Name** box, enter a folder name. This example uses the name **certprep**. Click **Save**.
3. Redirect your SASUSER library to the new folder as follows:

If you are using SAS University Edition, submit a LIBNAME statement by copying the following code into the **Code** tab:

```
libname sasuser "/folders/myfolders/certprep";
```

Note: You must use the filename of the new directory. These examples use the name **certprep**. If you use another filename, substitute the name that you created.

If you are using SAS Studio, do the following:

1. Right-click the new folder that you created and select **Properties**.
2. Copy the path in the **Location** field.
3. Enter the following code, replacing *location field* with the path that you copied from the **Location** field.

```
libname sasuser "location field";
```
4. Click **Run**.
5. Save the program as `libname_cert.sas`. You must resubmit this LIBNAME statement program every time you work with the sample data.
6. Copy the sample data program into a new Code window in SAS Studio. You can access the sample data at <http://support.sas.com/publishing/cert/sampdata.txt>.
7. Click **Run**.

Now that the sample data is stored in a permanent directory, reissue the LIBNAME statement whenever you want to use the data.

SAS Windowing Environment

To set up practice data in SAS:

1. Copy the sample data program into the Editor. You can access the sample data at <http://support.sas.com/publishing/cert/sampdata.txt>.
2. Click **Run**.

SAS Studio and SAS University Edition

To set up practice data in SAS Studio or SAS University Edition:

1. Copy the sample data program into a new Code window. You can access the sample data at <http://support.sas.com/publishing/cert/sampdata.txt>.
2. Click **Run**.

SAS Enterprise Guide

To download the sample data:

1. Start SAS Enterprise Guide.
2. In the Welcome to SAS Enterprise window, select **New Project**.
3. Select **File** ⇒ **New** ⇒ **Program**.
4. Depending on your network configuration, you might not have Write access to the Sasuser directory where the sample data is stored. To determine the status of the Sasuser directory, submit the following code:

```
proc options option=rsasuser;  
run;
```

5. If the result from the PROC OPTIONS code is RSASUSER, you must redirect the Sasuser folder by creating a new folder. From your server area, open the Files folder, right-click a drive or folder, and select **New Folder**. Enter the new folder name.

Note: If the result from the PROC OPTIONS code is NORSASUSER, the Sasuser folder is writable, and you do not have to redirect the Sasuser folder. Therefore, you can skip this step and the next one.

6. Submit the following code in a Code window:

```
libname sasuser "/folders/myfolders/certprep";
```

Note: You must use the filename of the new folder. These examples use the name **certprep**. If you use another filename, substitute the folder name that you created for **certprep**.

7. Copy the sample data program into the Program window and then run the program. You can access the sample data at <http://support.sas.com/publishing/cert/sampdata.txt>.
8. Because you do not need these shortcuts, delete the **Program** item and all the shortcuts from the project. This action does not delete the data that you created. To delete the item from the project, right-click **Program** and select **Delete**.
9. In the Confirmation window, click **Yes**.

Chapter 2

Basic Concepts

Getting Started	6
The Basics of the SAS Language	6
SAS Statements	6
Global Statements	6
DATA Step	6
PROC Step	7
A Simple SAS Program	7
Processing SAS Programs	8
Log Messages	8
Results of Processing	9
SAS Libraries	11
Definition	11
Predefined SAS Libraries	11
Defining Libraries	11
How SAS Files Are Stored	12
Storing Files Temporarily or Permanently	12
Referencing SAS Files	13
Referencing Permanent SAS Data Sets	13
Referencing Temporary SAS Files	14
Rules for SAS Names	14
SAS Data Sets	15
Overview of Data Sets	15
Descriptor Portion	15
Data Portion	16
Variable Attributes	17
Variable Names	18
Type	19
Length	20
Format	20
Informat	21
Label	21
SAS Indexes	22
Extended Attributes	22
Chapter Quiz	22

Getting Started

In the SAS Base Programming for SAS®9 exam, you are not tested on the details of running SAS software in the various environments. However, you might find such information useful when working with the practice data.

You can access a brief overview on the windows and menus of the SAS windowing environment, Enterprise Guide, and SAS Studio at <http://video.sas.com/>. From **Categories** select **How To Tutorials** ⇒ **Programming**. Select the video for your SAS environment. Other tutorials are available from the SAS website.

The Basics of the SAS Language

SAS Statements

A SAS statement is a type of SAS language element that is used to perform a particular operation in a SAS program or to provide information to a SAS program. SAS statements are free-format. This means that they can begin and end anywhere on a line, that one statement can continue over several lines, and that several statements can be on the same line. Blank or special characters separate words in a SAS statement.

TIP You can specify SAS statements in uppercase or lowercase. In most situations, text that is enclosed in quotation marks is case sensitive.

Here are two important rules for writing SAS programs:

- A SAS statement ends with a semicolon.
- A statement usually begins with a SAS keyword.

There are two types of SAS statements:

- statements that are used in DATA and PROC steps
- statements that are global in scope and can be used anywhere in a SAS program

Global Statements

Global statements are used anywhere in a SAS program and stay in effect until changed or canceled, or until the SAS session ends. Here are some common global statements: TITLE, LIBNAME, OPTIONS, and FOOTNOTE.

DATA Step

The DATA step creates or modifies data. The input for a DATA step can be of several types, such as raw data or a SAS data set. The output from a DATA step can be of several types, such as a SAS data set or a report. A SAS data set is a data file that is formatted in a way that SAS can understand.

For example, you can use DATA steps to do the following:

- put your data into a SAS data set

- compute values
- check for and correct errors in your data
- produce new SAS data sets by subsetting, supersetting, merging, and updating existing data sets

PROC Step

The PROC (procedure) step analyzes data, produces output, or manages SAS files. The input for a PROC step is usually a SAS data set. The output from a PROC step can be of several types, such as a report or an updated SAS data set.

For example, you can use PROC steps to do the following:

- create a report that lists the data
- produce descriptive statistics
- create a summary report
- produce plots and charts

A Simple SAS Program

This program uses an existing SAS data set to create a new SAS data set containing a subset of the original data set. It then prints a listing of the new data set using PROC PRINT.

```
data sasuser.admit2;
    set sasuser.admit;
    where age>39;
run;
proc print data=sasuser.admit2;
run;
```

The sample SAS program contains a DATA step and a PROC step. The DATA step produced a new SAS data set. Only those observations with an age value greater than 39 are written to the new SAS data set.

A DATA step begins with a DATA statement, which begins with the keyword DATA. A PROC step begins with a PROC statement, which begins with the keyword PROC. The sample program contains the following statements:

Table 2.1 SAS Program Statements

Statements	Sample Program Code
DATA statement	data sasuser.admit2;
SET statement	set sasuser.admit;
Additional programming statements	where age>39;
RUN statement	run;
PROC PRINT statement	proc print data=sasuser.admit2;

Statements	Sample Program Code
RUN statement	run;

Processing SAS Programs

When you submit a SAS program, SAS begins reading the statements and checking them for errors.

DATA and PROC statements signal the beginning of a new step. The RUN statement (for most procedures and the DATA step) and the QUIT statement (for some procedures) mark step boundaries. The beginning of a new step (DATA or PROC) also implies the end of the previous step. At a step boundary, SAS executes any statements that have not previously executed and ends the step. In the sample program, each step ends with a RUN statement.

```
data sasuser.admit2;
    set sasuser.admit;
    where age>39;
run;
proc print data=sasuser.admit2;
run;
```

TIP The RUN statement is not required between steps in a SAS program. It is a best practice to use a RUN statement because it can make the SAS program easier to read and the SAS log easier to understand when debugging.

Log Messages

The SAS log collects messages about the processing of SAS programs and about any errors that occur. Each time a step is executed, SAS generates a log of the processing activities and the results of the processing.

When SAS processes the sample program, it produces the log messages shown below. Notice that you get separate sets of messages for each step in the program.

Log 2.1 SAS Log Messages for Each Program Step

```

5  data sasuser.admit2;
6      set sasuser.admit;
7      where age>39;
8  run;

NOTE: There were 10 observations read from the data set SASUSER.ADMIT.
      WHERE age>39;
NOTE: The data set SASUSER.ADMIT2 has 10 observations and 9 variables.
NOTE: DATA statement used (Total process time):
      real time          0.00 seconds
      cpu time           0.00 seconds

9  proc print data=sasuser.admit2;
NOTE: Writing HTML Body file: sashtml.htm
10 run;

NOTE: There were 10 observations read from the data set SASUSER.ADMIT2.
NOTE: PROCEDURE PRINT used (Total process time):
      real time          0.35 seconds
      cpu time           0.24 seconds

```

Results of Processing**The DATA Step**

Suppose you submit the sample program below:

```

data sasuser.admit2;
    set sasuser.admit;
    where age>39;
run;

```

When the program is processed, it creates a new SAS data set (sasuser.admit2) containing only those observations with age values greater than 39. The DATA step creates a new data set and produces messages in the SAS log, but it does not create a report or other output.

The PROC Step

If you add a PROC PRINT step to this same example, the program produces the same new data set as before, but it also creates the following report, which is displayed in HTML:

```

data sasuser.admit2;
    set sasuser.admit;
    where age>39;
run;
proc print data=sasuser.admit2;
run;

```

Note: The default output in SAS Enterprise Guide is SAS Report. To change the default output in SAS Enterprise Guide to HTML, click **Tools** and select **Options** ⇒ **Results** ⇒ **Results General**. Then select HTML. Ensure that you have cleared SAS Report.

Figure 2.1 PRINT Procedure Output

The SAS System									
Obs	ID	Name	Sex	Age	Date	Height	Weight	ActLevel	Fee
1	2523	Johnson, R	F	43	31	63	137	MOD	149.75
2	2539	LaMance, K	M	51	4	71	158	LOW	124.80
3	2568	Eberhardt, S	F	49	27	64	172	LOW	124.80
4	2571	Nunnelly, A	F	44	19	66	140	HIGH	149.75
5	2575	Quigley, M	F	40	8	69	163	HIGH	124.80
6	2578	Cameron, L	M	47	5	72	173	MOD	124.80
7	2579	Underwood, K	M	60	22	71	191	LOW	149.75
8	2584	Takahashi, Y	F	43	29	65	123	MOD	124.80
9	2589	Wilcox, E	F	41	16	67	141	HIGH	149.75
10	2595	Warren, C	M	54	7	71	183	MOD	149.75

Other Procedures

SAS programs often invoke procedures that create output in the form of a report, as is the case with the FREQ procedure:

```
proc freq data=sashelp.cars;
  table origin*DriveTrain;
run;
```

Figure 2.2 FREQ Procedure Output

The FREQ Procedure					
Frequency Percent Row Pct Col Pct	Table of Origin by DriveTrain				
	Origin	DriveTrain			Total
		All	Front	Rear	
	Asia	34	99	25	158
		7.94	23.13	5.84	36.92
		21.52	62.66	15.82	
		36.96	43.81	22.73	
	Europe	36	37	50	123
		8.41	8.64	11.68	28.74
		29.27	30.08	40.65	
		39.13	16.37	45.45	
	USA	22	90	35	147
		5.14	21.03	8.18	34.35
		14.97	61.22	23.81	
		23.91	39.82	31.82	
	Total	92	226	110	428
		21.50	52.80	25.70	100.00

Other SAS programs perform tasks such as sorting and managing data, which have no visible results except for messages in the log. (All SAS programs produce log messages, but some SAS programs produce only log messages.)

```
proc copy in=sasuser out=work;
  select admit;
run;
```

Log 2.2 SAS Log: COPY Procedure Output

```

11  proc copy in=sasuser out=work;
12      select admit;
13  run;

```

```

NOTE: Copying SASUSER.ADMIT to WORK.ADMIT (memtype=DATA).
NOTE: There were 21 observations read from the data set SASUSER.ADMIT.
NOTE: The data set WORK.ADMIT has 21 observations and 9 variables.
NOTE: PROCEDURE COPY used (Total process time):
      real time          0.02 seconds
      cpu time           0.01 seconds

```

SAS Libraries

Definition

A SAS library contains one or more files that are defined, recognized, and accessible by SAS, and that are referenced and stored as a unit. One special type of file is called a *catalog*. In SAS libraries, catalogs function much like subfolders for grouping other members.

Predefined SAS Libraries

By default, SAS defines several libraries for you:

Sashelp

a permanent library that contains sample data and other files that control how SAS works at your site. This is a Read-Only library.

Sasuser

a permanent library that contains SAS files in the Profile catalog and that stores your personal settings. This is also a convenient place to store your own files.

Note: If you are using SAS Studio or SAS University Edition, you might not have Write access to the Sasuser directory. To verify whether you have Write access, see [“Determine Whether You Have Write Access” on page 1](#).

Work

a temporary library for files that do not need to be saved from session to session.

You can also define additional libraries. When you define a library, you indicate the location of your SAS files to SAS. After you define a library, you can manage SAS files within it.

Note: If you are using SAS Studio, you might encounter Webwork library. Webwork is the default output library in interactive mode. For more information about the Webwork library, see *SAS Studio: User's Guide*.

Defining Libraries

To define a library, you assign a library name to it and specify the location of the files, such as a directory path.

You can also specify an engine, which is a set of internal instructions that SAS uses for writing to and reading from files in a library.

You can define SAS libraries using programming statements. For information about how to write LIBNAME statements to define SAS libraries, see [Assigning Librefs on page 28](#).

TIP Depending on your operating environment and the SAS/ACCESS products that you license, you can create libraries with various engines. Each engine enables you to read a different file format, including file formats from other software vendors.

When you delete a SAS library, the pointer to the library is deleted, and SAS no longer has access to the library. However, the contents of the library still exist in your operating environment.

How SAS Files Are Stored

A SAS library is the highest level of organization for information within SAS.

For example, in the Windows and UNIX environments, a library is typically a group of SAS files in the same folder or directory.

The table below summarizes the implementation of SAS libraries in various operating environments.

Table 2.2 *Environments and SAS Libraries*

Environment	Library Definition
Windows, UNIX	a group of SAS files that are stored in the same directory. Other files can be stored in the directory, but only the files that have SAS file extensions are recognized as part of the SAS library.
z/OS	a specially formatted host data set in which only SAS files are stored.

Storing Files Temporarily or Permanently

Depending on the library name that you use when you create a file, you can store SAS files temporarily or permanently.

Table 2.3 *Temporary and Permanent SAS Libraries*

Temporary SAS libraries last only for the current SAS session.	If you do not specify a library name when you create a file (or if you specify the library name Work), the file is stored in the temporary SAS library. When you end the session, the temporary library and all of its files are deleted.
--	---

Permanent SAS libraries are available to you during subsequent SAS sessions.

To store files permanently in a SAS library, specify a library name other than the default library name Work.

For example, by specifying the library name **sasuser** when you create a file, you specify that the file is to be stored in a permanent SAS library until you delete it.

Referencing SAS Files

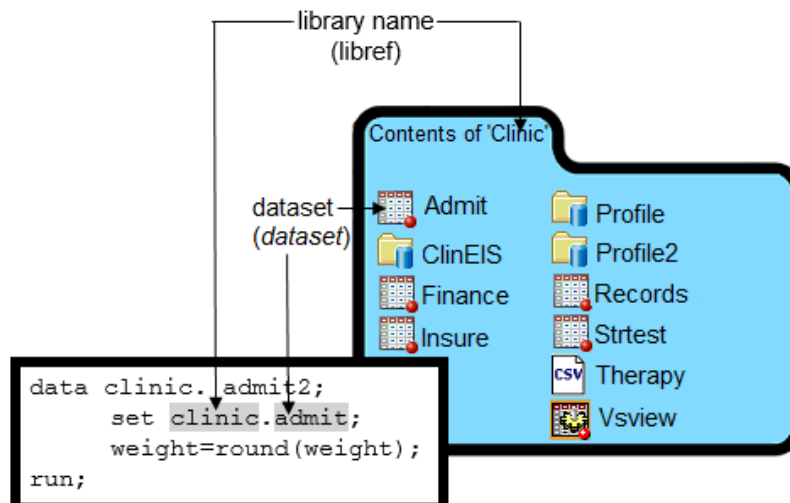
Referencing Permanent SAS Data Sets

To reference a permanent SAS data set in your SAS programs, use a two-level name consisting of the library name and the data set name:

libref.dataset

In the two-level name, *libref* is the name of the SAS library that contains the data set, and *dataset* is the name of the SAS data set. A period separates the libref and data set name.

Figure 2.3 Two-Level Permanent SAS Name



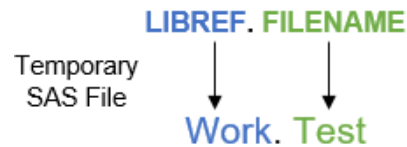
For example, suppose you want to create a new permanent SAS library named Clinic. In the sample program, Clinic.Admit is the two-level name for the SAS data set Admit, which is stored in the library named Clinic.

```
data clinic.admit2;
  set clinic.admit;
  weight =round(weight);
run;
```

Referencing Temporary SAS Files

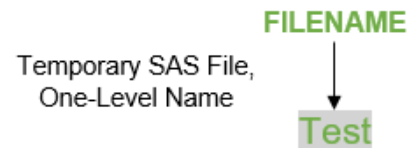
To reference temporary SAS files, you can specify the default libref `Work`, a period, and the data set name. For example, the two-level name, `Work.Test`, references the SAS data set named `Test` that is stored in the temporary SAS library `Work`.

Figure 2.4 Two-Level Temporary SAS Library Name



Alternatively, you can use a one-level name (the data set name only) to reference a file in a temporary SAS library. When you specify a one-level name, the default libref `Work` is assumed. For example, the one-level name `Test` also references the SAS data set named `Test` that is stored in the temporary SAS library `Work`.

Figure 2.5 One-Level Temporary SAS Library Name



Rules for SAS Names

The following rules apply to the names of SAS data sets, variables, and libraries:

- They must begin with a letter (A-Z, either uppercase or lowercase) or an underscore (`_`).
- They can continue with any combination of numbers, letters, or underscores.
- They can be 1 to 32 characters long.
- SAS library names (librefs) can be 1 to 8 characters long.

These are examples of valid data set names and variable names:

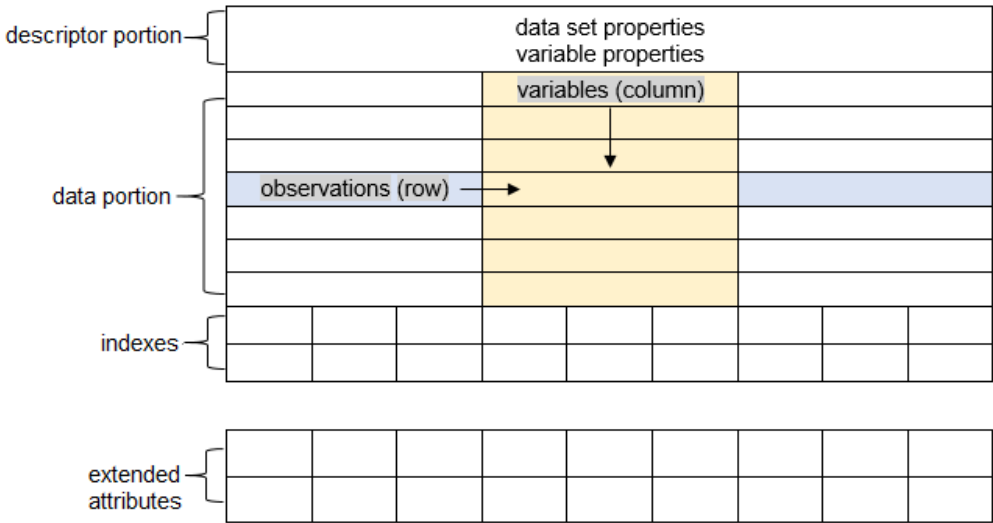
- `Payroll`
- `LABDATA1995_1997`
- `_EstimatedTaxPayments3`

SAS Data Sets

Overview of Data Sets

A SAS data set is a file that consists of two parts: a descriptor portion and a data portion. Sometimes a SAS data set also points to one or more indexes, which enable SAS to locate rows in the data set more efficiently. (The data sets that are shown in this chapter do not contain indexes.) Extended attributes are user-defined attributes that further define a SAS data set.

Figure 2.6 Parts of a SAS Data Set



Descriptor Portion

The descriptor portion of a SAS data set contains information about the data set, including the following:

- the name of the data set
- the date and time that the data set was created
- the number of observations
- the number of variables

The table below lists part of the descriptor portion of the data set sasuser.insure, which contains insurance information for patients who are admitted to a wellness clinic.

Table 2.4 Descriptor Portion of Attributes in a SAS Data Set

Data Set Name:	SASUSER.INSURE
Member Type:	DATA

Engine:	V9
Created:	10:05 Thursday, February 16, 2017
Observations:	21
Variables:	7
Indexes:	0
Observation Length:	64

Data Portion

Data Portion Overview

The data portion of a SAS data set is a collection of data values that are arranged in a rectangular table. In the example below, the company **MUTUALITY** is a data value, Policy **32668** is a data value, and so on.

Figure 2.7 Parts of a SAS Data Set: Data Portion

ID	Name	Policy	Company	PctInsured	Total	BalanceDue
2458	Murray, W	32668	MUTUALITY	100	98.64	0.00
2462	Almers, C	95824	RELIABLE	80	780.23	156.05
2501	Bonaventure, T	87795	A&R	80	47.38	9.48
2523	Johnson, R	39022	ACME	50	122.07	61.04

Observations (Rows)

Observations (also called rows) in the data set are collections of data values that usually relate to a single object. The values **2458**, **Murray W**, **32668**, **MUTUALITY**, **100**, **98.64**, and **0.00** comprise a single observation in the data set shown below.

Figure 2.8 Parts of a SAS Data Set: Observations

	ID	Name	Policy	Company	PctInsured	Total	BalanceDue
Observation	2458	Murray, W	32668	MUTUALITY	100	98.64	0.00
	2462	Almers, C	95824	RELIABLE	80	780.23	156.05
	2501	Bonaventure, T	87795	A&R	80	47.38	9.48
	2523	Johnson, R	39022	ACME	50	122.07	61.04

This data set has 21 observations, each containing information about an individual. To view the full descriptor portion of this data set, see [Table 2.4 on page 15](#). A SAS data set can store any number of observations.

Variables (Columns)

Variables (also called columns) in the data set are collections of values that describe a particular characteristic. The values **2458**, **2462**, **2501**, and **2523** comprise the variable ID in the data set shown below.

Figure 2.9 Parts of a SAS Data Set: Variables

Variables	ID	Name	Policy	Company	PctInsured	Total	BalanceDue
	2458	Murray, W	32668	MUTUALITY	100	98.64	0.00
	2462	Almers, C	95824	RELIABLE	80	780.23	156.05
	2501	Bonaventure, T	87795	A&R	80	47.38	9.48
	2523	Johnson, R	39022	ACME	50	122.07	61.04

This data set contains seven variables: ID, Name, Policy, Company, PctInsured, Total, and BalanceDue. A SAS data set can store thousands of variables.

Missing Values

Every variable and observation in a SAS data set must have a value. If a data value is unknown for a particular observation, a missing value is recorded in the SAS data set. A period (.) is the default value for a missing numeric value, and a blank space is the default value for a missing character value.

Figure 2.10 Parts of a SAS Data Set: Missing Data Values

ID	Name	Policy	Company	PctInsured	Total	BalanceDue
2458	Murray, W	32668	MUTUALITY	100	98.64	0.00
2462	Almers, C	95824	RELIABLE	80	780.23	156.05
2501	Bonaventure, T	87795	A&R	.	47.38	9.48
2523	Johnson, R	39022	ACME	50	122.07	61.04

Missing Value

Variable Attributes

In addition to general information about the data set, the descriptor portion contains information about the properties of each variable in the data set. The properties information includes the variable's name, type, length, format, informat, and label.

When you write SAS programs, it is important to understand the attributes of the variables that you use. For example, you might need to combine SAS data sets that contain same-named variables. In this case, the variables must be the same type (character or numeric). If the same-named variables are both character variables, you still need to check that the variable lengths are the same. Otherwise, some values might be truncated.

Here is a partial listing of the attribute information in the descriptor portion of the SAS data set sasuser.insure.

Table 2.5 Variable Attributes in the Descriptor Portion of a SAS Data Set *sasuser.insure*

Variable	Type	Length	Format	Informat	Label
Policy	Char	8			Policy Number
Total	Num	8	DOLLAR8.2	COMMA10.	Total Balance
Name	Char	20			Patient Name

Variable Names

Rules for Variable Names

Each variable has a name that conforms to SAS naming conventions. Variable names follow the same rules as SAS data set names.

- They can be 1 to 32 characters long.
- They must begin with a letter (A-Z, either uppercase or lowercase) or an underscore (_).
- They can continue with any combination of numbers, letters, or underscores.

Table 2.6 Variable Name Attributes

Variable	Type	Length	Format	Informat	Label
Policy	Char	8			Policy Number
Total	Num	8	DOLLAR8.2	COMMA10.	Total Balance
Name	Char	20			Patient Name

VALIDVARNAME= System Option

Note: If you use characters other than the ones that are valid when the VALIDVARNAME system option is set to V7 (letters of the Latin alphabet, numerals, or underscores), then you must express the variable name as a name literal and you must set VALIDVARNAME=ANY. If the name includes either a percent sign (%) or an ampersand (&), then you must use single quotation marks in the name literal in order to avoid interaction with the SAS macro facility.

VALIDVARNAME specifies the rules for valid SAS variable names that can be created and processed during a SAS session.

Syntax, VALIDVARNAME=

VALIDVARNAME= *V7*[*UPCASE*][*ANY*]

V7 specifies that variable names must follow these rules:

- SAS variable names can be up to 32 characters long.
- The first character must begin with a letter of the Latin alphabet (A - Z, either uppercase or lowercase) or an underscore (_). Subsequent characters can be letters of the Latin alphabet, numerals, or underscores.
- Trailing blanks are ignored. The variable name alignment is left-justified.
- A variable name cannot contain blanks or special characters except for an underscore.
- A variable name can contain mixed-case letters. SAS stores and writes the variable name in the same case that is used in the first reference to the variable. However, when SAS processes a variable name, SAS internally converts it to uppercase. Therefore, you cannot use the same variable name with a different combination of uppercase and lowercase letters to represent different variables. For example, **cat**, **Cat**, and **CAT** all represent the same variable.
- Do not assign variables the names of special SAS automatic variables (such as **_N_** and **_ERROR_**) or variable list names (such as **_NUMERIC_**, **_CHARACTER_**, and **_ALL_**) to variables.

UPCASE specifies that the variable name follows the same rules as *V7*, except that the variable name is uppercase, as in earlier versions of SAS.

ANY specifies that SAS variable names must follow these rules:

- The name can begin with or contain any characters, including blanks, national characters, special characters, and multi-byte characters.
- The name can be up to 32 bytes long.
- The name cannot contain any null bytes.
- Leading blanks are preserved, but trailing blanks are ignored.
- The name must contain at least one character. A name with all blanks is not permitted.
- A variable name can contain mixed-case letters. SAS stores and writes the variable name in the same case that is used in the first reference to the variable. However, when SAS processes a variable name, SAS internally converts it to uppercase. Therefore, you cannot use the same variable name with a different combination of uppercase and lowercase letters to represent different variables. For example, **cat**, **Cat**, and **CAT** all represent the same variable.

CAUTION:

Throughout SAS, using the name literal syntax with SAS member names that exceed the 32-byte limit or have excessive embedded quotation marks might cause unexpected results. The VALIDVARNAME=ANY system option enables compatibility with other DBMS variable (column) naming conventions, such as allowing embedded blanks and national characters.

Type

A variable's type is either character or numeric.

- Character variables, such as Name (shown below), can contain any values.
- Numeric variables, such as Total (shown below), can contain only numeric values (the numerals 0 through 9, +, -, ., and E for scientific notation).

Table 2.7 Type Attribute

Variable	Type	Length	Format	Informat	Label
Policy	Char	8			Policy Number
Total	Num	8	DOLLAR8.2	COMMA10.	Total Balance
Name	Char	20			Patient Name

Length

A variable's length (the number of bytes used to store it) is related to its type.

- Character variables can be up to 32,767 bytes long. In the example below, Name has a length of 20 characters and uses 20 bytes of storage.
- All numeric variables have a default length of 8 bytes. Numeric values are stored as floating-point numbers in 8 bytes of storage.

Table 2.8 Length Attribute

Variable	Type	Length	Format	Informat	Label
Policy	Char	8			Policy Number
Total	Num	8	DOLLAR9.2	COMMA10.	Total Balance
Name	Char	20			Patient Name

Format

Formats are variable attributes that affect how data values are written. Formats do not change the stored value in any way; they merely control how that value is displayed. SAS software offers a variety of character, numeric, and date and time formats. You can also create and store your own formats. To write values out using a particular form, you select the appropriate format.

For example, to display the value 1234 as \$1,234.00 in a report, you can use the DOLLAR9.2 format, as shown for Total below.

Table 2.9 Format Attribute

Variable	Type	Length	Format	Informat	Label
Policy	Char	8			Policy Number

Variable	Type	Length	Format	Informat	Label
Total	Num	8	DOLLAR9.2	COMMA10.	Total Balance
Name	Char	20			Patient Name

Usually you have to specify the maximum width (*w*) of the value to be written. Depending on the particular format, you might also need to specify the number of decimal places (*d*) to be written. For example, to display the value **5678** as 5,678.00 in a report, you can use the COMMA8.2 format, which specifies a width of 8 including 2 decimal places.

TIP You can permanently assign a format to a variable in a SAS data set, or you can temporarily specify a format in a PROC step to determine how the data values appear in output.

Informat

Whereas formats write values out using some particular form, informats read data values in certain forms into standard SAS values. Informats determine how data values are read into a SAS data set. You must use informats to read numeric values that contain letters or other special characters.

For example, the numeric value **\$12,345.00** contains two special characters, a dollar sign (\$) and a comma (.). You can use an informat to read the value while removing the dollar sign and comma, and then store the resulting value as a standard numeric value. For Total below, the COMMA10. informat is specified.

Table 2.10 Informat Attribute

Variable	Type	Length	Format	Informat	Label
Policy	Char	8			Policy Number
Total	Num	8	DOLLAR9.2	COMMA10.	Total Balance
Name	Char	20			Patient Name

Label

A variable can have a label, which consists of descriptive text up to 256 characters long. By default, many reports identify variables by their names. You might want to replace the name with more descriptive information about the variable by assigning a label to the variable.

For example, you can label Policy as Policy Number, Total as Total Balance, and Name as Patient Name to display these labels in reports.

Table 2.11 Label Attribute

Variable	Type	Length	Format	Informat	Label
Policy	Char	8			Policy Number
Total	Num	8	DOLLAR9.2	COMMA10.	Total Balance
Name	Char	20			Patient Name

You can use labels to shorten long variable names in your reports.

SAS Indexes

An index is a separate file that you can create for a SAS data file in order to provide direct access to a specific observation. The index file has the same name as its data file and a member type of INDEX. Indexes can provide faster access to specific observations, particularly when you have a large data set. The purpose of SAS indexes is to optimize WHERE expressions and to facilitate BY-group processing. For more information, see [“Specifying WHERE Expressions” on page 72](#) and see [“Group Processing Using the BY Statement” on page 183](#).

Extended Attributes

Extended attributes are user-defined metadata that is defined on a data set or on a variable (column). Extended attributes are represented as name-value pairs and are created using the DATASETS procedure.

TIP You can use PROC CONTENTS to display data set and variable extended attributes.

Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- How many observations and variables does the data set below contain?

Name	Sex	Age
Picker	M	32
Fletcher		28
Romano	F	.
Choi	M	42

- 3 observations, 4 variables
- 3 observations, 3 variables

- c. 4 observations, 3 variables
- d. cannot tell because some values are missing
2. How many program steps are executed when the program below is processed?

```
data user.tables;
  infile jobs;
  input date yyddmm8. name $ job $;
run;
proc sort data=user.tables;
  by name;
run;
proc print data=user.tables;
run;
```

- a. three
- b. four
- c. five
- d. six
3. What type of variable is the variable AcctNum in the data set below?

AcctNum	Gender
3456_1	M
2451_2	
Romano	F
Choi	M

- a. numeric
- b. character
- c. can be either character or numeric
- d. cannot tell from the data shown
4. What type of variable is the variable Wear in the data set below?

Brand	Wear
Acme	43
Ajax	34
Atlas	.

- a. numeric
- b. character
- c. can be either character or numeric
- d. cannot tell from the data shown
5. With the system option VALIDVARNAME=ANY, which of the following variable names is valid?

- a. 4BirthDate
 - b. \$Cost
 - c. _Items_
 - d. Tax-Rate
 - e. All of the above
6. Which of the following files is a permanent SAS file?
- a. Sashelp.PrdSale
 - b. Sasuser.MySales
 - c. Profits.Quarter1
 - d. all of the above
7. In a DATA step, how can you reference a temporary SAS data set named Forecast?
- a. Forecast
 - b. Work.Forecast
 - c. Sales.Forecast (after assigning the libref Sales)
 - d. only a and b above
8. What is the default length for the numeric variable Balance?

Name	Balance
Adams	105.73
Geller	107.89
Martinez	97.45
Noble	182.50

- a. 5
 - b. 6
 - c. 7
 - d. 8
9. How many statements does the following SAS program contain?
- ```
proc print data=clinic.admit
 label double;
 var ID Name Sex Age; where Sex=F;
 label Sex='Gender'; run;
```
- a. three
  - b. four
  - c. five
  - d. six
10. What is a SAS library?
- a. collection of SAS files, such as SAS data sets and catalogs
  - b. in some operating environments, a physical collection of SAS files

- c. a group of SAS files in the same folder or directory
- d. all of the above



## Chapter 3

# Setting Up Your SAS Session

---

|                                                                                      |           |
|--------------------------------------------------------------------------------------|-----------|
| <b>SAS Libraries</b>                                                                 | <b>28</b> |
| Assigning Librefs                                                                    | 28        |
| Verifying Librefs                                                                    | 29        |
| How Long Librefs Remain in Effect                                                    | 29        |
| Specifying Two-Level Names                                                           | 30        |
| Referencing Third-Party Data                                                         | 30        |
| Accessing Stored Data                                                                | 30        |
| Accessing BMDP, OSIRIS, and SPSS Files                                               | 30        |
| <b>Viewing SAS Libraries</b>                                                         | <b>31</b> |
| Viewing Libraries                                                                    | 31        |
| Viewing Libraries Using PROC CONTENTS                                                | 31        |
| Examples: PROC CONTENTS Output                                                       | 31        |
| Viewing Libraries Using PROC DATASETS                                                | 33        |
| Viewing Data Set Descriptor Information Using PROC<br>DATASETS and the VARNUM Option | 35        |
| <b>Specifying Results Formats</b>                                                    | <b>36</b> |
| HTML Formats                                                                         | 36        |
| The Results Tab                                                                      | 36        |
| <b>Setting System Options</b>                                                        | <b>41</b> |
| The Basics of System Options                                                         | 41        |
| Changing System Options                                                              | 41        |
| Options for Printed Results                                                          | 42        |
| Handling Two-Digit Years                                                             | 42        |
| How the YEARCUTOFF= Option Works                                                     | 43        |
| Handling Four-Digit Years                                                            | 44        |
| Using System Options to Specify Observations                                         | 44        |
| Examples: FIRSTOBS= and OBS= Options                                                 | 45        |
| Using FIRSTOBS= and OBS= for Specific Data Sets                                      | 46        |
| Example: FIRSTOBS= and OBS= as Data Set Options                                      | 46        |
| The SAS System Options Window                                                        | 47        |
| Changing Options                                                                     | 47        |
| Finding Options Quickly                                                              | 47        |
| <b>Chapter Quiz</b>                                                                  | <b>48</b> |

## SAS Libraries

A *SAS library* is a collection of one or more SAS files, including SAS data sets, that are referenced and stored as a unit. In a directory-based operating environment, a *SAS library* is a group of SAS files that are stored in the same directory. In z/OS (OS/390), a *SAS library* is a group of SAS files that are stored in an operating environment file.

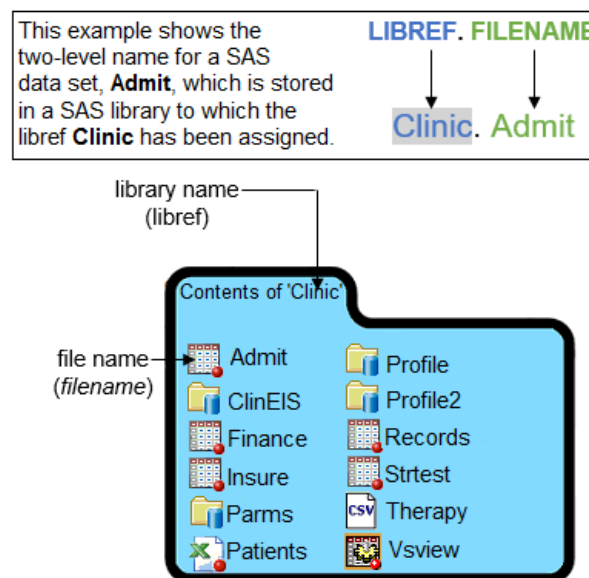
### Assigning Librefs

Often the first step in setting up your SAS session is to define the libraries. You can use programming statements to assign library names.

To reference a permanent SAS file:

1. Assign a name (*libref*) to the SAS library in which the file is stored.
2. Use the libref as the first part of the two-level name (*libref.filename*) to reference the file within the library.

**Figure 3.1** Defining Libraries



A logical name (libref) can be assigned to a SAS library using the LIBNAME statement. You can include the LIBNAME statement with any SAS program so that the SAS library is assigned each time the program is submitted. Using the user interface, LIBNAME statements can be set up to be automatically assigned when SAS starts.

Syntax, LIBNAME statement:

**LIBNAME** *libref* engine '*SAS-data-library*';

- *libref* is 1 to 8 characters long, begins with a letter or underscore, and contains only letters, numbers, or underscores.
- *engine* is the name of a library engine that is supported in your operating environment.  
*Note:* For SAS®9, the default engine is V9, which works in all operating environments.
- *SAS-data-library* is the name of a SAS library in which SAS data files are stored. The specification of the physical name of the library differs by operating environment.

The LIBNAME statement below assigns the libref **Medical** to the SAS library **C:\Users\johnsmith\medical** in the Windows environment. When the default engine is used, you do not have to specify it in the LIBNAME statement.

```
libname medical 'c:\users\johnsmith\medical';
```

The table below gives examples of physical names for SAS libraries in various operating environments.

**Table 3.1** Sample Physical Names for SAS Libraries

| Environment | Sample Physical Name         |
|-------------|------------------------------|
| Windows     | c:\fitness\data              |
| UNIX        | /users/april/fitness/sasdata |
| z/OS)       | april.fitness.sasdata        |

**TIP** You can use multiple LIBNAME statements to assign as many librefs as needed.

## Verifying Librefs

After assigning a libref, it is a good idea to check the log to verify that the libref has been assigned successfully.

**Log 3.1** Output for Clinic Libref

```
1 libname medical 'C:\users\johnsmith\medical';
NOTE: Libref MEDICAL was successfully assigned as follows:
 Engine: V9
 Physical Name: C:\users\johnsmith\medical
```

## How Long Librefs Remain in Effect

The LIBNAME statement is global, which means that the librefs remain in effect until changed or canceled, or until the SAS session ends.

By default, the LIBNAME statement assigns the libref for the current SAS session only. Each time you begin a SAS session, you must assign a libref to each permanent SAS

library that contains files that you want to access in that session. (Remember that Work is the default libref for a temporary SAS library.)

### Specifying Two-Level Names

After you assign a libref, you specify it as the first element in the two-level name for a SAS file.

In order for the PRINT procedure to read clinic.admit, you specify the two-level name of the file as follows:

```
proc print data=clinic.admit;
run;
```

### Referencing Third-Party Data

You can use the LIBNAME statement to reference not only SAS files but also files that were created with other software products, such as database management systems.

A SAS engine is a set of internal instructions that SAS uses for writing to and reading from files in a SAS library. SAS can read or write these files by using the appropriate engine for that file type. For some file types, you need to tell SAS which engine to use. For others, SAS automatically chooses the appropriate engine.

### Accessing Stored Data

If your site licenses SAS/ACCESS software, you can use the LIBNAME statement to access data that is stored in a database management system (DBMS) file. The types of data you can access depend on your operating environment and on which SAS/ACCESS products you have licensed. For more information about SAS/ACCESS engines, see the SAS documentation for your DBMS.

### Accessing BMDP, OSIRIS, and SPSS Files

Interface library engines support Read-Only access to BMDP, OSIRIS, and SPSS files. With these engines, the physical filename that is associated with a libref is an actual filename, not a SAS library name.

**Table 3.2** Engines and Their Descriptions

| Engine | Description                             |
|--------|-----------------------------------------|
| BMDP   | allows Read-Only access to BMDP files   |
| OSIRIS | allows Read-Only access to OSIRIS files |
| SPSS   | allows Read-Only access to SPSS files   |

For example, the LIBNAME statement below specifies the libref Rptdata and the engine SPSS for the file **G:\MySpss.spss** in the Windows operating environment.

```
libname rptdata spss 'g:\myspss.spss';
```



For more information about interface library engines, see the SAS documentation for your operating environment.

---

## Viewing SAS Libraries

### Viewing Libraries

Besides accessing library details with librefs, you can also see libraries in other environments. You can access a brief overview on the windows and menus of the SAS windowing environment, Enterprise Guide, and SAS Studio at <http://video.sas.com/>. From **Categories** select **How To Tutorials** ⇒ **Programming**. Select the video for your SAS environment. Other tutorials are available from the SAS website.

### Viewing Libraries Using PROC CONTENTS

You can use the CONTENTS procedure to create SAS output that describes either of the following:

- the contents of a library
- the descriptor information for an individual SAS data set

---

Syntax, PROC CONTENTS step:

**PROC CONTENTS DATA=SAS-file-specification NODS;  
RUN;**

- *SAS-file-specification* specifies an entire library or a specific SAS data set within a library. *SAS-file-specification* can take one of the following forms:
    - `<libref>SAS-data-set` names one SAS data set to process.
    - `<libref>_ALL_` requests a listing of all files in the library. (Use a period (.) to append `_ALL_` to the libref.)
  - NODS suppresses the printing of detailed information about each file when you specify `_ALL_`. (You can specify NODS *only* when you specify `_ALL_`.)
- 

### Examples: PROC CONTENTS Output

To view the contents of the entire clinic library, you can submit the following PROC CONTENTS step:

```
proc contents data=medical._all_ nods;run;
```

The output from this step lists only the names, types, sizes, and modification dates for the SAS files in the Medical library.

**Figure 3.2** PROC CONTENTS Output: the SAS Library Medical (partial output)

| The SAS System         |                  |  |  |  |
|------------------------|------------------|--|--|--|
| The CONTENTS Procedure |                  |  |  |  |
| Directory              |                  |  |  |  |
| Libref                 | MEDICAL          |  |  |  |
| Engine                 | V9               |  |  |  |
| Physical Name          | C:\Users\medical |  |  |  |
| Filename               | C:\Users\medical |  |  |  |
| Owner Name             | Users            |  |  |  |
| File Size              | 20KB             |  |  |  |
| File Size (bytes)      | 20480            |  |  |  |

| #  | Name         | Member Type | File Size | Last Modified       |
|----|--------------|-------------|-----------|---------------------|
| 1  | AHLID        | DATA        | 128KB     | 11/15/2015 12:38:59 |
| 2  | ANAEROB      | DATA        | 9KB       | 12/02/2014 17:13:50 |
| 3  | ARRESTS      | DATA        | 128KB     | 11/22/2014 07:40:58 |
| 4  | BABIES       | DATA        | 9KB       | 09/10/2007 06:31:34 |
| 5  | BAITING      | DATA        | 128KB     | 11/22/2014 06:43:39 |
| 6  | BLADDER      | DATA        | 128KB     | 03/27/2015 05:07:34 |
| 7  | CACERVIX     | DATA        | 128KB     | 12/01/2014 17:07:48 |
| 8  | CANCER       | DATA        | 5KB       | 01/15/2007 18:56:16 |
| 9  | CASTOMACH    | DATA        | 5KB       | 11/30/2006 18:11:40 |
| 10 | CA_LUNG      | DATA        | 128KB     | 12/01/2014 17:29:55 |
| 11 | CHD          | DATA        | 128KB     | 11/15/2015 12:55:26 |
| 12 | CHOLESES     | DATA        | 5KB       | 01/11/2007 18:42:12 |
| 13 | CLOUD        | DATA        | 128KB     | 02/13/2015 09:47:18 |
| 14 | CRASH        | DATA        | 5KB       | 01/15/2007 18:51:56 |
| 15 | DEPRESSIONIQ | DATA        | 128KB     | 12/07/2014 11:54:20 |
| 16 | DIET         | DATA        | 5KB       | 01/11/2007 18:44:00 |

To view the descriptor information for only the medical.Lesions data set, you can submit the following PROC CONTENTS step:

```
proc contents data=medical.lesions;
run;
```

The output from this step lists information for medical.Lesions, including an alphabetic list of the variables in the data set.

**Figure 3.3** PROC CONTENTS Output: Descriptor Information

|                     |                           |                      |    |
|---------------------|---------------------------|----------------------|----|
| Data Set Name       | MEDICAL.LESIONS           | Observations         | 27 |
| Member Type         | DATA                      | Variables            | 3  |
| Engine              | V9                        | Indexes              | 0  |
| Created             | 12/10/2014 16:12:20       | Observation Length   | 32 |
| Last Modified       | 12/10/2014 16:12:20       | Deleted Observations | 0  |
| Protection          |                           | Compressed           | NO |
| Data Set Type       |                           | Sorted               | NO |
| Label               |                           |                      |    |
| Data Representation | WINDOWS_64                |                      |    |
| Encoding            | wlatin1 Western (Windows) |                      |    |

**Figure 3.4** PROC CONTENTS Output: Engine/Host Dependent Information

| Engine/Host Dependent Information |                                   |
|-----------------------------------|-----------------------------------|
| Data Set Page Size                | 65536                             |
| Number of Data Set Pages          | 1                                 |
| First Data Page                   | 1                                 |
| Max Obs per Page                  | 2039                              |
| Obs in First Data Page            | 27                                |
| Number of Data Set Repairs        | 0                                 |
| ExtendObsCounter                  | YES                               |
| Filename                          | c:\users\medical\lesions.sas7bdat |
| Release Created                   | 9.0401M2                          |
| Host Created                      | X64_7PRO                          |
| Owner Name                        | user                              |
| File Size                         | 128KB                             |
| File Size (bytes)                 | 131072                            |

**Figure 3.5** PROC CONTENTS Output: Alphabetic List of Variables

| Alphabetic List of Variables and Attributes |          |      |     |
|---------------------------------------------|----------|------|-----|
| #                                           | Variable | Type | Len |
| 3                                           | n        | Num  | 8   |
| 1                                           | region   | Char | 8   |
| 2                                           | site     | Char | 16  |

## Viewing Libraries Using PROC DATASETS

In addition to PROC CONTENTS, you can use PROC DATASETS to view the contents of a SAS library or a SAS data set. PROC DATASETS also enables you to perform a number of management tasks such as copying, deleting, or modifying SAS files.

PROC CONTENTS and PROC DATASETS overlap in terms of functionality. Generally, the following are the same:

- the CONTENTS procedure
- the CONTENTS statement in the DATASETS procedure

---

```
PROC CONTENTS<options>;
```

```
RUN;
```

---

```
PROC DATASETS<options>;
```

```
 CONTENTS<options>;
```

```
QUIT;
```

---

Here is the major difference between the CONTENTS procedure and the CONTENTS statement in PROC DATASETS:

- The default value for libref in the DATA= option. For PROC CONTENTS, the default is either Work or User depending on your SAS solution or environment. For the CONTENTS statement, the default is the libref of the procedure input library.

*Note:* PROC DATASETS supports RUN-group processing. It uses a QUIT statement to end the procedure.

However, the options for the PROC CONTENTS statement and the CONTENTS statement in the DATASETS procedure are the same. For example, the following PROC steps produce essentially the same output (with minor formatting differences):

```
proc datasets;
 contents data=medical._all_ nods;

proc contents data=medical._all_ nods;
run;
```

**Figure 3.6** PROC DATASETS Output: Directory Information

| Directory         |                                                                 |
|-------------------|-----------------------------------------------------------------|
| Libref            | WORK                                                            |
| Engine            | V9                                                              |
| Physical Name     | C:\Users\AppData\Local\Temp\SAS Temporary Files\_TD6364_D7C049_ |
| Filename          | C:\Users\AppData\Local\Temp\SAS Temporary Files\_TD6364_D7C049_ |
| Owner Name        | Users                                                           |
| File Size         | 4KB                                                             |
| File Size (bytes) | 4096                                                            |

| # | Name    | Member Type | File Size | Last Modified       |
|---|---------|-------------|-----------|---------------------|
| 1 | SASGOPT | CATALOG     | 5KB       | 05/01/2017 10:15:51 |
| 2 | SASMAC3 | CATALOG     | 13KB      | 05/01/2017 10:15:50 |

### The SAS System

#### The DATASETS Procedure

| Directory         |                  |
|-------------------|------------------|
| Libref            | MEDICAL          |
| Engine            | V9               |
| Physical Name     | C:\Users\medical |
| Filename          | C:\Users\medical |
| Owner Name        | Users            |
| File Size         | 20KB             |
| File Size (bytes) | 20480            |

**Figure 3.7** PROC DATASETS Output: Library Members (partial output)

| #  | Name         | Member Type | File Size | Last Modified       |
|----|--------------|-------------|-----------|---------------------|
| 1  | AHLID        | DATA        | 128KB     | 11/15/2015 12:38:59 |
| 2  | ANAEROB      | DATA        | 9KB       | 12/02/2014 17:13:50 |
| 3  | ARRESTS      | DATA        | 128KB     | 11/22/2014 07:40:58 |
| 4  | BABIES       | DATA        | 9KB       | 09/10/2007 06:31:34 |
| 5  | BAITING      | DATA        | 128KB     | 11/22/2014 06:43:39 |
| 6  | BLADDER      | DATA        | 128KB     | 03/27/2015 05:07:34 |
| 7  | CACERVIX     | DATA        | 128KB     | 12/01/2014 17:07:48 |
| 8  | CANCER       | DATA        | 5KB       | 01/15/2007 18:56:16 |
| 9  | CASTOMACH    | DATA        | 5KB       | 11/30/2006 18:11:40 |
| 10 | CA_LUNG      | DATA        | 128KB     | 12/01/2014 17:29:55 |
| 11 | CHD          | DATA        | 128KB     | 11/15/2015 12:55:26 |
| 12 | CHOLES       | DATA        | 5KB       | 01/11/2007 18:42:12 |
| 13 | CLOUD        | DATA        | 128KB     | 02/13/2015 09:47:18 |
| 14 | CRASH        | DATA        | 5KB       | 01/15/2007 18:51:56 |
| 15 | DEPRESSIONIQ | DATA        | 128KB     | 12/07/2014 11:54:20 |
| 16 | DIET         | DATA        | 5KB       | 01/11/2007 18:44:00 |

**Figure 3.8** PROC CONTENTS Output: Directory Information

| Directory         |                  |
|-------------------|------------------|
| Libref            | MEDICAL          |
| Engine            | V9               |
| Physical Name     | C:\Users\medical |
| Filename          | C:\Users\medical |
| Owner Name        | Users            |
| File Size         | 20KB             |
| File Size (bytes) | 20480            |

**Figure 3.9** PROC CONTENTS Output: Library Members (partial output)

| #  | Name         | Member Type | File Size | Last Modified       |
|----|--------------|-------------|-----------|---------------------|
| 1  | AHLID        | DATA        | 128KB     | 11/15/2015 12:38:59 |
| 2  | ANAEROB      | DATA        | 9KB       | 12/02/2014 17:13:50 |
| 3  | ARRESTS      | DATA        | 128KB     | 11/22/2014 07:40:58 |
| 4  | BABIES       | DATA        | 9KB       | 09/10/2007 06:31:34 |
| 5  | BAITING      | DATA        | 128KB     | 11/22/2014 06:43:39 |
| 6  | BLADDER      | DATA        | 128KB     | 03/27/2015 05:07:34 |
| 7  | CACERVIX     | DATA        | 128KB     | 12/01/2014 17:07:48 |
| 8  | CANCER       | DATA        | 5KB       | 01/15/2007 18:56:16 |
| 9  | CASTOMACH    | DATA        | 5KB       | 11/30/2006 18:11:40 |
| 10 | CA_LUNG      | DATA        | 128KB     | 12/01/2014 17:29:55 |
| 11 | CHD          | DATA        | 128KB     | 11/15/2015 12:55:26 |
| 12 | CHOLE        | DATA        | 5KB       | 01/11/2007 18:42:12 |
| 13 | CLOUD        | DATA        | 128KB     | 02/13/2015 09:47:18 |
| 14 | CRASH        | DATA        | 5KB       | 01/15/2007 18:51:56 |
| 15 | DEPRESSIONIQ | DATA        | 128KB     | 12/07/2014 11:54:20 |
| 16 | DIET         | DATA        | 5KB       | 01/11/2007 18:44:00 |

### Viewing Data Set Descriptor Information Using PROC DATASETS and the VARNUM Option

As with PROC CONTENTS, you can also use PROC DATASETS to display the descriptor information for a specific SAS data set.

By default, PROC CONTENTS and PROC DATASETS list variables alphabetically. To list variable names in the order of their logical position (or creation order) in the data set, specify the VARNUM option in PROC CONTENTS or in the CONTENTS statement in PROC DATASETS.

For example, either of these programs creates output that includes the list of variables shown below:

```
proc datasets;
 contents data=medical.Lesions varnum;

proc contents data=medical.Lesions varnum;
run;
```

**Figure 3.10** Viewing Descriptor Information for a SAS Data Set Using VARNUM

| Variables in Creation Order |          |      |     |
|-----------------------------|----------|------|-----|
| #                           | Variable | Type | Len |
| 1                           | region   | Char | 8   |
| 2                           | site     | Char | 16  |
| 3                           | n        | Num  | 8   |

## Specifying Results Formats

### HTML Formats

In SAS 9.3 and later versions, when running SAS in windowing mode in the Windows and UNIX operating environments, HTML output is created by default. In SAS Studio the default output is HTML, and in Enterprise Guide the default output is SAS Report. In other platforms, you can create HTML output using programming statements. This book shows all output in HTML format.

This is HTML output from the same program:

**Figure 3.11** HTML Output

| Obs | SSN         | Name      | Salary | Date     |
|-----|-------------|-----------|--------|----------|
| 1   | 029-46-9261 | Rudelich  | 35000  | 02/15/95 |
| 2   | 074-53-9892 | Vincent   | 35000  | 05/22/97 |
| 3   | 228-88-9649 | Benito    | 28000  | 03/04/96 |
| 4   | 442-21-8075 | Sirignano | 5000   | 11/22/95 |
| 5   | 446-93-2122 | Harbinger | 33900  | 07/08/96 |
| 6   | 776-84-5391 | Phillipon | 29750  | 12/15/96 |
| 7   | 929-75-0218 | Gunter    | 27500  | 04/30/97 |

**TIP** If you are not running SAS in a desktop operating environment, skip this topic and go to “[The Basics of System Options](#)” on page 41. For more information about creating HTML output using programming statements on any SAS platform, see “[Creating HTML Output with ODS](#)” on page 206.

### The Results Tab

#### SAS Windowing Environment

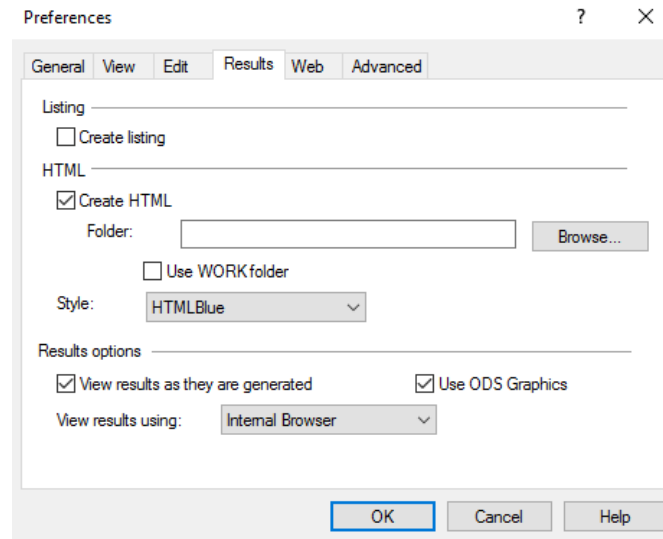
Use the Preferences window to set the result format or formats that you prefer. Your preferences are saved until you modify them, and they apply to all output that you create.

To open this window in desktop operating environments, select **Tools** ⇒ **Options** ⇒ **Preferences**. Then click the **Results** tab. Select either **Listing** or **HTML**, or both. You can also specify options for displaying and storing your results.

**TIP** **Results** tab options might differ somewhat, depending on your operating environment. The example below is from the Windows operating environment.

The following figure shows the SAS **Results** tab with new default settings specified.

**Figure 3.12** SAS Results Tab with New Default Settings



The default settings on the **Results** tab are as follows:

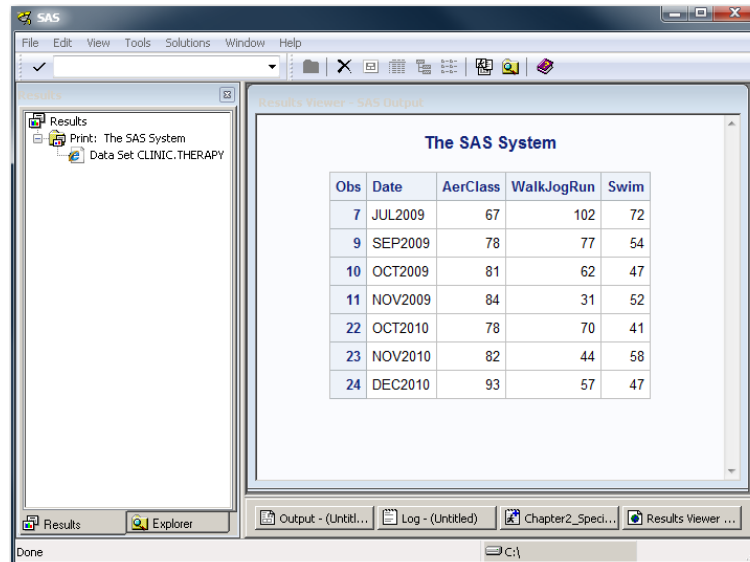
- The **Create listing** check box is not selected, so LISTING output is not created.
- The **Create HTML** check box is selected, so HTML output is created.
- The **Use WORK folder** check box is selected, so both HTML and graph image files are saved in the WORK folder (and not your current directory).
- The default style, HTMLBlue, is selected from the **Style** drop-down list.
- The **Use ODS Graphics** check box is selected, so ODS Graphics is enabled.
- **Internal browser** is selected from the **View results using** drop-down list, so results are viewed in an internal SAS browser.

If you create HTML files, they are stored in the folder that you specify and are by default incrementally named sashtml.htm, sashtml1.htm, sashtml2.htm, and so on, throughout your SAS session.

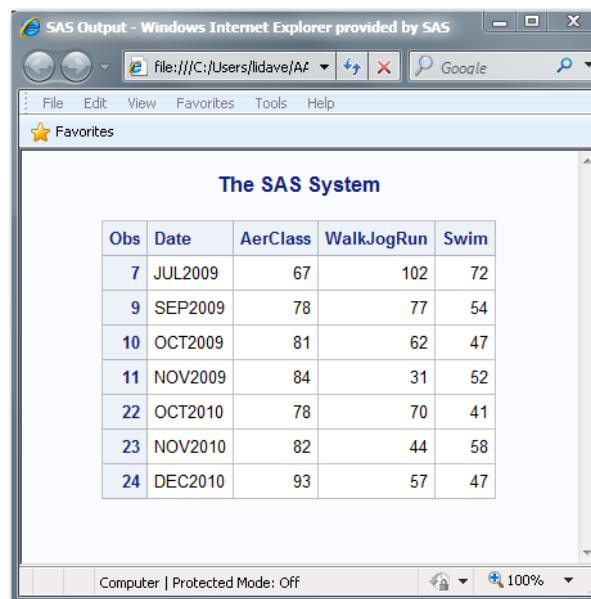
There are two choices for viewing HTML results: Internal Browser and Preferred Web Browser. (These options appear on the **Results** tab only in the Windows operating environment.)

To view HTML output, select either of these options on the **Results** tab of the Preferences window in the Windows operating environment:

- Internal Browser, called the Results Viewer window. SAS provides this browser as part of your SAS installation. The Results Viewer is displayed as a SAS window, as shown below.

**Figure 3.13** Internal Browser Showing HTML Output

- **Preferred Web Browser.** If you select this option, SAS uses the browser that is specified on the **Web** tab of the Preferences window. This is the default browser for your PC.

**Figure 3.14** Preferred Web Browser Showing HTML Output

### SAS Enterprise Guide

Use the Options window to set the result format or formats that you prefer. Your options are saved until you modify them, and they apply to all output that you create.

To open this window in desktop operating environments, select **Tools** ⇒ **Options**. Then select the **Results** ⇒ **Results General** tab. Select **SAS Report**, **HTML**, **PDF**, **RTF**, **Text output**, **PowerPoint**, or **Excel**. You can also specify options for displaying and storing your results.

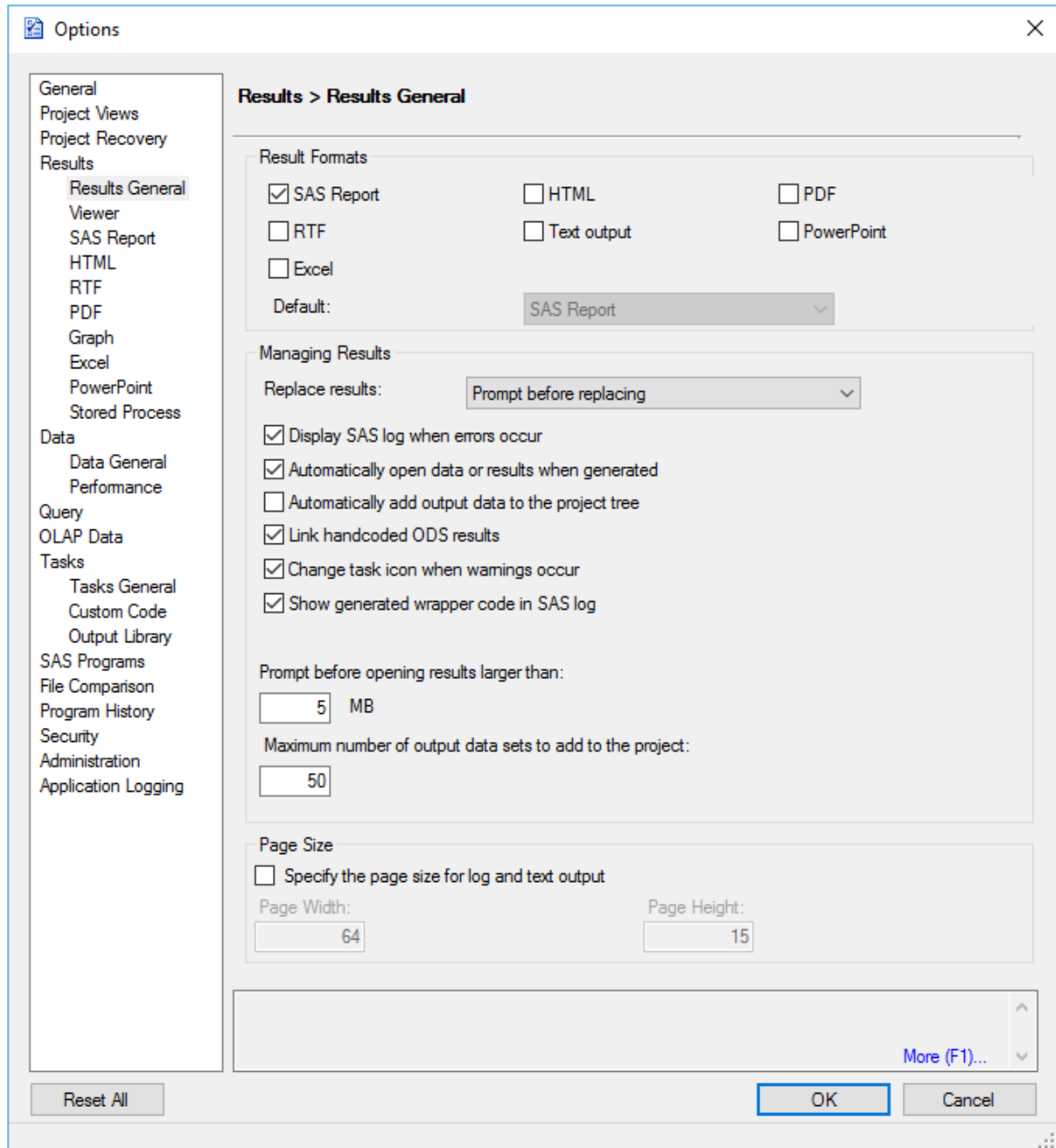
Default output options that are available from the drop-down list include **HTML**, **SAS Report**, or **PDF**.



**TIP** **Results** tab options might differ somewhat, depending on your operating environment. The example below is from the Windows operating environment.

The following display shows the SAS **Results General** tab with new default settings specified.

**Figure 3.15** SAS Results General Tab with New Default Settings



The default settings on the **Results General** tab are as follows:

- The **SAS Report** check box is selected.
- The **Prompt before replacing** is selected from the drop-down menu.
- The **Display SAS log when errors occur** check box is selected.

- The **Automatically open data or results when generated** check box is selected.
- The **Automatically add output data to the project tree** check box is not selected.
- The **Link handcoded ODS results** check box is selected.
- The **Change task icon when warnings occur** check box is selected.
- The **Show generated wrapper code in SAS log** check box is selected.
- The value that is in the field titled **prompt before opening results larger than** is 5.
- The value that is in the field titled **maximum number of output data sets to add to the project** is 50.

### SAS Studio and SAS University Edition

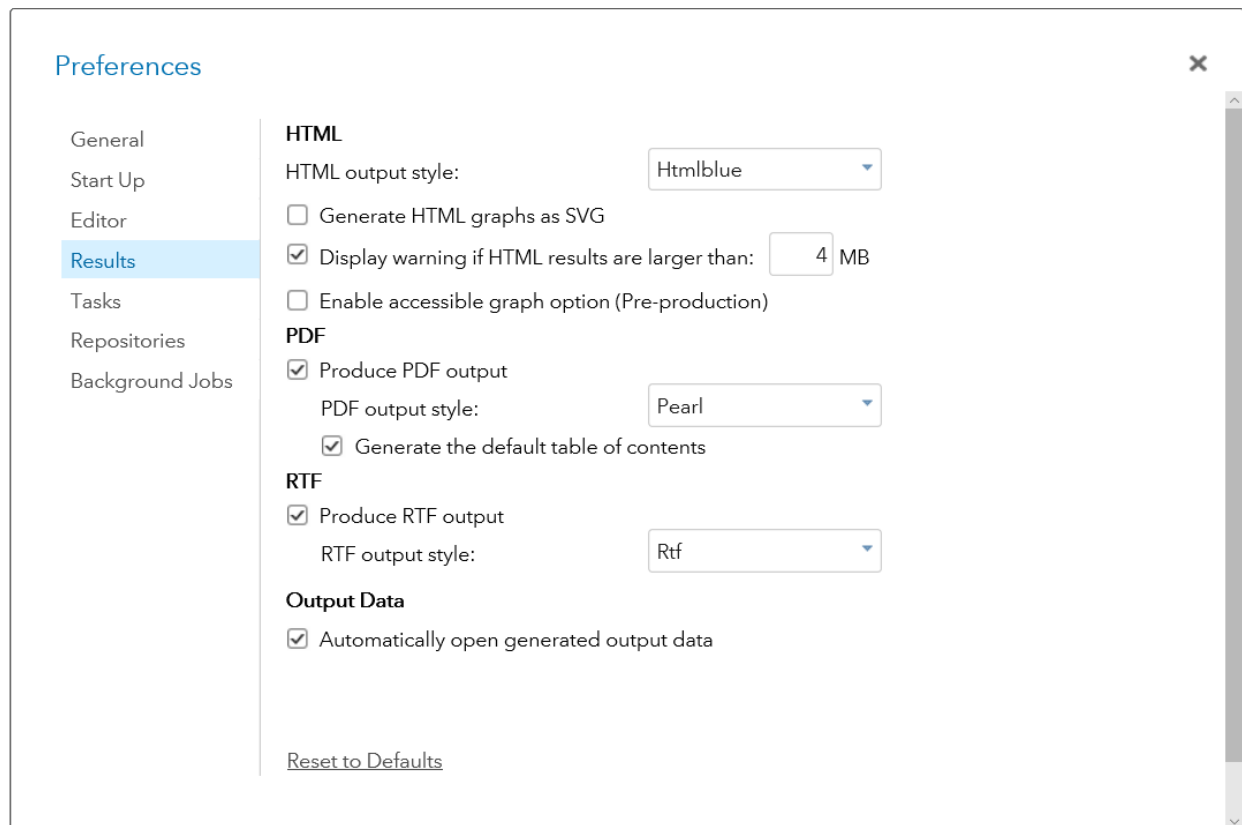
Use the Preferences window to set the result format or formats that you prefer. Your options are saved until you modify them, and they apply to all output that you create.

To open this window in desktop operating environments, click  and select **Preferences**. Then select the **Results** tab. Specify either **HTML**, **PDF**, or **RTF**. You can also specify options for displaying and storing your results.

**TIP** **Results** tab options might differ somewhat, depending on your operating environment. The example below is from the Windows operating environment.

The following display shows the SAS **Results** tab with new default settings specified.

**Figure 3.16** SAS Results Tab with New Default Settings



The default settings on the **Results** tab are as follows:

- The **Display warning if HTML results are larger than: 4MB** check box is selected.
- The **Produce PDF output** check box is selected.
- The **Generate the default table of contents** check box is selected.
- The **Produce RTF output** check box is selected.
- The **Automatically open generated output data** check box is selected.

## Setting System Options

### The Basics of System Options

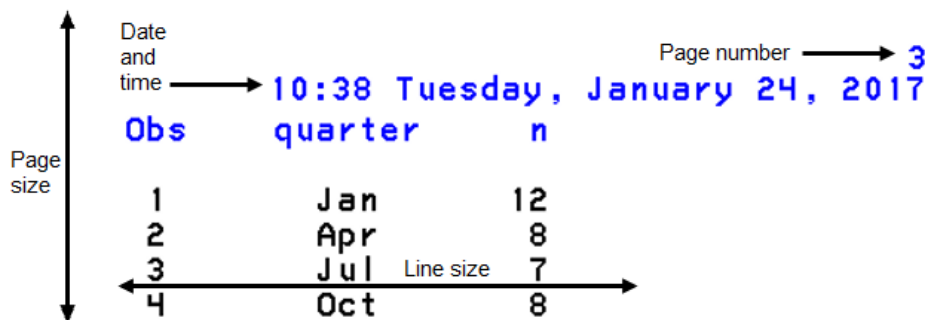
*Note:* The options below do not affect the appearance of HTML output. HTML is the recommended method setting for output.

*System options* are instructions that affect the processing of an entire SAS program or SAS session from the time the option is specified until it is changed. If you create your procedure output as LISTING output, you can also control the appearance of your output by setting these system options:

- line size (the maximum width of the log and output)
- page size (the number of lines per printed page of output)
- the display of page numbers
- the display of date and time

All SAS system options have default settings that are used unless you specify otherwise. For example, page numbers are automatically displayed in LISTING output (unless your site modifies this default).

**Figure 3.17** LISTING Output Showing Default Settings



### Changing System Options

To modify system options in your LISTING output, you submit an **OPTIONS** statement. You can place an **OPTIONS** statement anywhere in a SAS program to change the settings from that point on. However, it is good programming practice to place

OPTIONS statements outside DATA or PROC steps so that your programs are easier to read and debug.

Because the OPTIONS statement is global, the settings remain in effect until changed or canceled, or until the SAS session ends.

---

Syntax, OPTIONS statement:

**OPTIONS** *options*;

*options* specifies one or more system options to be changed. The available system options depend on your host operating system.

---

## Options for Printed Results

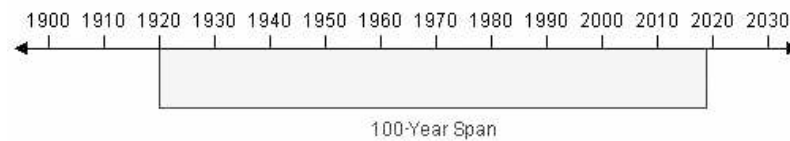
Here are some system options that affect the appearance of LISTING output.

**Table 3.3** Selected System Options and Their Descriptions

|                                 |                                                                                                                 |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------|
| CENTER   NOCENTER               | controls whether output is centered or left-justified. The default is CENTER.                                   |
| DATE   NODATE                   | controls whether or not today's date appears at the top of each page of output. The default is DATE.            |
| NUMBER   NONUMBER               | controls whether or not page numbers appear on each page of SAS output. The default is NUMBER.                  |
| ORIENTATION= <i>orientation</i> | specifies the orientation for printing output, either LANDSCAPE or PORTRAIT. The default is PORTRAIT.           |
| PAGENO= <i>n</i>                | starts numbering output pages with <i>n</i> . The default is 1.                                                 |
| RIGHTMARGIN= <i>n</i>           | specifies the size of the margin (such as 0.75in or 2cm) to be used for printing output. The default is 0.00in. |
| LEFTMARGIN= <i>n</i>            |                                                                                                                 |
| TOPMARGIN= <i>n</i>             |                                                                                                                 |
| BOTTOMMARGIN= <i>n</i>          |                                                                                                                 |

## Handling Two-Digit Years

If you use two-digit year values in your data lines, external files, or programming statements, you should consider another important system option, the YEARCUTOFF= option. This option specifies which 100-year span is used to interpret two-digit year values.

**Figure 3.18** The Default 100 Year Span in SAS

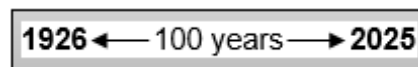
All versions of SAS represent dates correctly from 1582 C.E. to 20,000 C.E. (Leap years, century, and fourth-century adjustments are made automatically. Leap seconds are ignored, and SAS does not adjust for daylight saving time. You use SAS informats to read date, times, and datetime values to account for various time zones. However, you should be aware of the `YEARCUTOFF=` value to ensure that you are properly interpreting two-digit years in data lines.

As with other system options, you specify the `YEARCUTOFF=` option in the `OPTIONS` statement:

```
options yearcutoff=1925;
```

### How the `YEARCUTOFF=` Option Works

When a two-digit year value is read, SAS interprets it based on a 100-year span that starts with the `YEARCUTOFF=` value. The default value of `YEARCUTOFF=` is **1926**.

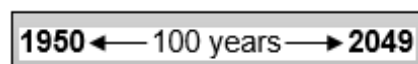
**Figure 3.19** Default `YEARCUTOFF=` Date (1926)**Table 3.4** Date Expressions and How They Are Displayed (`YEARCUTOFF= 1926`)

| Date Expression | Displayed Value |
|-----------------|-----------------|
| 12/07/41        | 12/07/1941      |
| 18Dec15         | 18Dec2015       |
| 04/15/30        | 04/15/1930      |
| 15Apr95         | 15Apr1995       |

However, you can override the default and change the value of `YEARCUTOFF=` to the first year of another 100-year span. For example, if you specify `YEARCUTOFF=1950`, then the 100-year span is from 1950 to 2049.

```
options yearcutoff=1950;
```

Using `YEARCUTOFF=1950`, dates are interpreted as shown below:

**Figure 3.20** Interpreting Dates When `YEARCUTOFF=1950`

**Table 3.5** Date Expressions and How They Are Displayed (YEARCUTOFF= 1950)

| Date Expression | Displayed Value |
|-----------------|-----------------|
| 12/07/41        | 12/07/2041      |
| 18Dec15         | 18Dec2015       |
| 04/15/30        | 04/15/2030      |
| 15Apr95         | 15Apr1995       |

### Handling Four-Digit Years

Remember, the value of the YEARCUTOFF= system option affects only two-digit year values. A date value that contains a four-digit year value is interpreted correctly even if it does not fall within the 100-year span that is set by the YEARCUTOFF= system option.

For more information about reading date values, see [Chapter 19, “SAS Date and Time Values,”](#) on page 447.

### Using System Options to Specify Observations

Besides using SAS system options to change the appearance of output and interpret two-digit year values, you can also use the FIRSTOBS= and OBS= system options to specify the observations to process from SAS data sets.

Specify either or both of these options as needed.

- FIRSTOBS= starts processing at a specific observation.
- OBS= stops processing after a specific observation.

*Note:* Using FIRSTOBS= and OBS= together processes a specific group of observations.

**CAUTION:**

Each of these options applies to every input data set that is used in a program or a SAS process.

---

Syntax, FIRSTOBS=, and OBS= options in an OPTIONS statement:

**FIRSTOBS=*n***

**OBS=*n***

*n* is a positive integer. For FIRSTOBS=, *n* specifies the number of the *first* observation to process. For OBS=, *n* specifies the number of the *last* observation to process. By default, FIRSTOBS=1. The default value for OBS= is MAX, which is the largest signed, 8-byte integer that is representable in your operating environment. The number can vary depending on your operating system.

---

### Examples: **FIRSTOBS=** and **OBS=** Options

The data set clinic.heart contains 20 observations. If you specify FIRSTOBS=10, SAS reads the 10th observation of the data set first and reads through the last observation (for a total of 11 observations).

```
options firstobs=10;
proc print data=clinic.heart;
run;
```

Here is the output:

**Figure 3.21** PROC PRINT Output with FIRSTOBS=10

| Obs | Patient | Sex | Survive | Shock    | Arterial | Heart | Cardiac | Urinary |
|-----|---------|-----|---------|----------|----------|-------|---------|---------|
| 10  | 509     | 2   | SURV    | OTHER    | 79       | 84    | 256     | 90      |
| 11  | 742     | 1   | DIED    | HYPOVOL  | 100      | 54    | 135     | 0       |
| 12  | 609     | 2   | DIED    | NONSHOCK | 93       | 101   | 260     | 90      |
| 13  | 318     | 2   | DIED    | OTHER    | 72       | 81    | 410     | 405     |
| 14  | 412     | 1   | SURV    | BACTER   | 61       | 87    | 296     | 44      |
| 15  | 601     | 1   | DIED    | BACTER   | 84       | 101   | 260     | 377     |
| 16  | 402     | 1   | SURV    | CARDIO   | 88       | 137   | 312     | 75      |
| 17  | 98      | 2   | SURV    | CARDIO   | 84       | 87    | 260     | 377     |
| 18  | 4       | 1   | SURV    | HYPOVOL  | 81       | 149   | 406     | 200     |
| 19  | 50      | 2   | SURV    | HYPOVOL  | 72       | 111   | 332     | 12      |
| 20  | 2       | 2   | DIED    | OTHER    | 101      | 114   | 424     | 97      |

If you specify OBS=10 instead, SAS reads through the 10th observation. In this case, that is for a total of 10 observations. Notice that FIRSTOBS= has been reset to the default value.

```
options firstobs=1 obs=10;
proc print data=clinic.heart;
run;
```

Here is the output:

**Figure 3.22** PROC PRINT Output with FIRSTOBS=1 and Obs=10

| Obs | Patient | Sex | Survive | Shock    | Arterial | Heart | Cardiac | Urinary |
|-----|---------|-----|---------|----------|----------|-------|---------|---------|
| 1   | 203     | 1   | SURV    | NONSHOCK | 88       | 95    | 66      | 110     |
| 2   | 54      | 1   | DIED    | HYPOVOL  | 83       | 183   | 95      | 0       |
| 3   | 664     | 2   | SURV    | CARDIO   | 72       | 111   | 332     | 12      |
| 4   | 210     | 2   | DIED    | BACTER   | 74       | 97    | 369     | 0       |
| 5   | 101     | 2   | DIED    | NEURO    | 80       | 130   | 291     | 0       |
| 6   | 102     | 2   | SURV    | OTHER    | 87       | 107   | 471     | 65      |
| 7   | 529     | 1   | DIED    | CARDIO   | 103      | 106   | 217     | 15      |
| 8   | 524     | 2   | DIED    | CARDIO   | 145      | 99    | 156     | 10      |
| 9   | 426     | 1   | SURV    | OTHER    | 68       | 77    | 410     | 75      |
| 10  | 509     | 2   | SURV    | OTHER    | 79       | 84    | 256     | 90      |

Combining FIRSTOBS= and OBS= processes observations in the middle of the data set. For example, the following program processes only observations 10 through 15, for a total of 6 observations:

```
options firstobs=10 obs=15;
```

```
proc print data=clinic.heart;
run;
```

Here is the output:

**Figure 3.23** PROC PRINT Output with FIRSTOBS=10 and Obs=15

| Obs | Patient | Sex | Survive | Shock    | Arterial | Heart | Cardiac | Urinary |
|-----|---------|-----|---------|----------|----------|-------|---------|---------|
| 10  | 509     | 2   | SURV    | OTHER    | 79       | 84    | 256     | 90      |
| 11  | 742     | 1   | DIED    | HYPOVOL  | 100      | 54    | 135     | 0       |
| 12  | 609     | 2   | DIED    | NONSHOCK | 93       | 101   | 260     | 90      |
| 13  | 318     | 2   | DIED    | OTHER    | 72       | 81    | 410     | 405     |
| 14  | 412     | 1   | SURV    | BACTER   | 61       | 87    | 296     | 44      |
| 15  | 601     | 1   | DIED    | BACTER   | 84       | 101   | 260     | 377     |

To reset the number of the last observation to process, you can specify OBS=MAX in the OPTIONS statement.

```
options obs=max;
```

This instructs any subsequent SAS programs in the SAS session to process through the last observation in the data set that is being read.

### Using FIRSTOBS= and OBS= for Specific Data Sets

Using the FIRSTOBS= or OBS= system options determines the first or last observation, respectively, that is read for all steps for the duration of your current SAS session or until you change the setting. However, you can still do the following:

- override these options for a given data set
- apply these options to a specific data set only

To affect any single file, use FIRSTOBS= or OBS= as data set options instead of as system options. You specify the data set option in parentheses immediately following the input data set name.

**TIP** A FIRSTOBS= or OBS= specification from a data set option overrides the corresponding FIRSTOBS= or OBS= system option.

### Example: FIRSTOBS= and OBS= as Data Set Options

As shown in the following example, this program processes only observations 10 through 15, for a total of 6 observations:

```
options firstobs=10 obs=15;
proc print data=clinic.heart;
run;
```

You can create the same output by specifying FIRSTOBS= and OBS= as data set options, as follows. The data set options override the system options for this instance only.

```
options firstobs=10 obs=15;
proc print data=clinic.heart(firstobs=20 obs=30);
run;
```

To specify FIRSTOBS= or OBS= for this program only, you could omit the OPTIONS statement altogether and simply use the data set options.



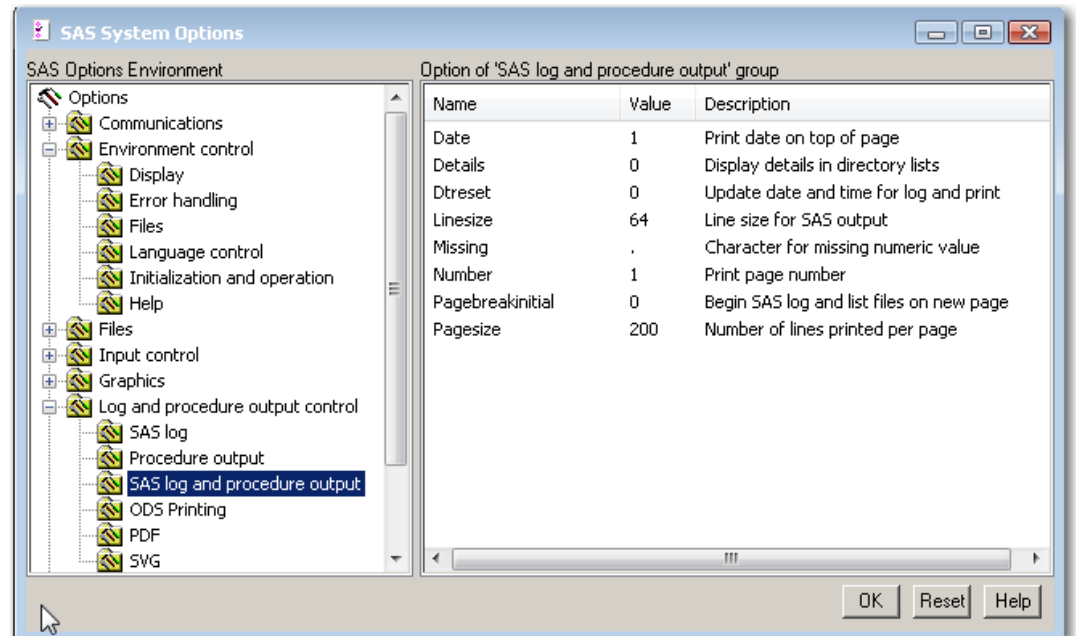
## The SAS System Options Window

*Note:* There is no SAS System Options window in Enterprise Guide or SAS Studio. Use the **OPTIONS** statement to change system options.

You can also set system options by using the SAS System Options window. The changed options are reset to the defaults at the end of your SAS session.

To view the SAS System Options window, select **Tools** ⇒ **Options** ⇒ **System**.

**Figure 3.24** The SAS System Options Window



## Changing Options

To change an option:

1. Expand the groups and subgroups under SAS Options Environment until you find the option that you want to change. Options in subgroups are listed in the right pane of the window.
2. Click the name of the option that you want to change, and display its pop-up menu. Specify either of the following:
  - **Modify Value** opens a window in which you enter or select a new value for the option.
  - **Set to Default** immediately resets the option to its default value.

## Finding Options Quickly

To locate an option in the SAS System Options window:

1. Place your cursor over the name of any option group or subgroup, and display its pop-up menu.

2. Click **Find Option**. The **Find Option** dialog box appears.
3. Enter the name of the option that you want to locate, and click **OK**.

The SAS System Options window expands to the appropriate option subgroup. All subgroup options also appear, and the option that you located is highlighted.

---

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

1. How can you create SAS output in HTML format on any SAS platform?
  - a. by specifying system options
  - b. by using programming statements
  - c. by using SAS windows to specify the result format
  - d. you cannot create HTML output on all SAS platforms
2. In order for the date values **05May1955** and **04Mar2046** to be read correctly, what value must the **YEARCUTOFF=** option have?
  - a. a value between 1947 and 1954, inclusive
  - b. 1955 or higher
  - c. 1946 or higher
  - d. any value
3. When you specify an engine for a library, what are you specifying?
  - a. the file format for files that are stored in the library
  - b. the version of SAS that you are using
  - c. access to other software vendors' files
  - d. instructions for creating temporary SAS files
4. Which statement prints a summary of all the files stored in the library named **Area51**?
  - a. `proc contents data=area51._all_ nods;`
  - b. `proc contents data=area51 _all_ nods;`
  - c. `proc contents data=area51 _all_ noobs;`
  - d. `proc contents data=area51 _all_.nods;`
5. Which of the following programs correctly references a SAS data set named *SalesAnalysis* that is stored in a permanent SAS library?
  - a. 

```
data saleslibrary.salesanalysis;
 set mydata.quarter1sales;
 if sales>100000;
run;
```
  - b. 

```
data mysales.totals;
 set sales_99.salesanalysis;
 if totalsales>50000;
```

- ```
run;
```
- c. `proc print data=salesanalysis.quarter1;`
`var sales salesrep month;`
`run;`
 - d. `proc freq data=1999data.salesanalysis;`
`tables quarter*sales;`
`run;`
6. Which time span is used to interpret two-digit year values if the YEARCUTOFF= option is set to 1950?
 - a. 1950-2049
 - b. 1950-2050
 - c. 1949-2050
 - d. 1950-2000
 7. Assuming you are using SAS code and not the SAS windowing environment, which one of the following statements is *false*?
 - a. LIBNAME statements can be stored with a SAS program to reference the SAS library automatically when you submit the program.
 - b. When you delete a libref, SAS no longer has access to the files in the library. However, the contents of the library still exist on your operating system.
 - c. Librefs can last from one SAS session to another.
 - d. You can access files that were created with other vendors' software by submitting a LIBNAME statement.
 8. What does the following statement do?


```
libname osiris spss 'c:\myfiles\sasdata\data.spss';
```

 - a. defines a library called Spss using the OSIRIS engine
 - b. defines a library called Osiris using the SPSS engine
 - c. defines two libraries called Osiris and Spss using the default engine
 - d. defines the default library using the OSIRIS and SPSS engines

Chapter 4

Identifying and Correcting Errors

Error Messages	51
Types of Errors	51
Syntax Errors	51
Example: Syntax Error Messages	52
Correcting Common Errors	53
The Basics of Error Correction	53
Resubmitting a Revised Program	53
The PUTLOG Statement	55
Missing RUN Statement	57
Correcting the Error: Missing RUN Statement	57
Missing Semicolon	57
Correcting the Error: Missing Semicolon	58
Unbalanced Quotation Marks	58
Correcting the Error in the Windows Operating Environment	59
Correcting the Error in the UNIX Operating Environment	60
Correcting the Error in the z/OS Operating Environment	60
Semantic Error: Invalid Option	61
Correcting the Error: Invalid Option	62
Chapter Quiz	62

Error Messages

Types of Errors

SAS can detect several types of errors. Here are two common ones:

- syntax errors that occur when program statements do not conform to the rules of the SAS language
- semantic errors that occur when you specify a language element that is not valid for a particular usage

Syntax Errors

When you submit a program, SAS scans each statement for syntax errors, and then executes the step (if no syntax errors are found). SAS then goes to the next step and

repeats the process. Syntax errors, such as misspelled keywords, generally prevent SAS from executing the step in which the error occurred.

Notes are written to the Log window when the program finishes executing. When a program that contains an error is submitted, messages about the error appear in the Log window. Here is what SAS does:

- displays the word ERROR
- identifies the possible location of the error
- gives an explanation of the error

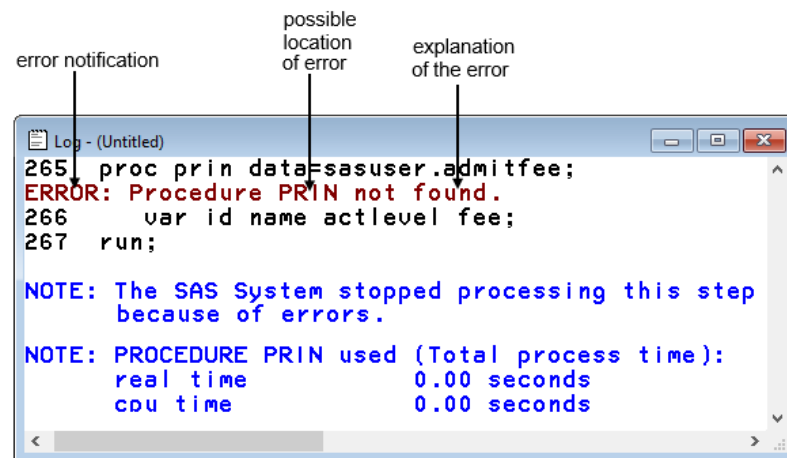
Example: Syntax Error Messages

In the program below, the DATA step copies the SAS data set Clinic.Admit into a new data set named Clinic.Admitfee. The PROC step should print the values for the variables ID, Name, Actlevel, and Fee in the new data set. However, the SAS keyword *print* is misspelled in the PROC PRINT statement.

```
data sasuser.admitfee;
    set sasuser.admit;
run;
proc prin data=sasuser.admitfee;
    var id name actlevel fee;
run;
```

When the program is submitted, messages in the Log window indicate that the procedure PRIN was not found and that SAS stopped processing the PROC step because of errors. No output is produced by the PRINT procedure, because the second step fails to execute.

Figure 4.1 Log Window Displaying Error Message



Note: Errors in your statements or data might not be evident when you look at results in the Results viewer. Review the messages in the Log window each time you submit a SAS program.

In addition to correcting spelling mistakes, you might need to resolve other common syntax errors such as these:

- missing RUN statement
- missing semicolon
- unbalanced quotation mark

You might also need to correct a semantic error such as this:

- invalid option

Correcting Common Errors

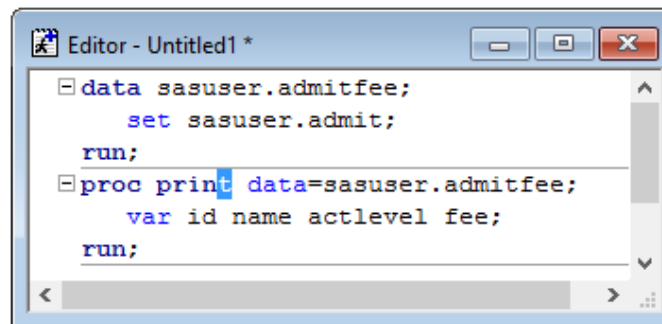
The Basics of Error Correction

To correct errors in programs, edit them in the code editing window. To correct simple errors, such as the spelling error here, type over the incorrect text, delete text, or insert text.

```
data sasuser.admitfee;  
    set sasuser.admit;  
run;  
proc prin data=sasuser.admitfee;  
    var id name actlevel fee;  
run;
```

In the program below, the missing letter *t* has been inserted into the SAS keyword that is specified in the PROC PRINT statement.

Figure 4.2 *Corrected Program*



Resubmitting a Revised Program

After correcting your program, you can resubmit it. If you are working in the SAS windowing environment, it is a good idea to clear the messages from the Log window before resubmitting the program so that you do not confuse the old error messages with the new messages.

Figure 4.3 Correct PROC PRINT Output**The SAS System**

Obs	ID	Name	ActLevel	Fee
1	2458	Murray, W	HIGH	85.20
2	2462	Almers, C	HIGH	124.80
3	2501	Bonaventure, T	LOW	149.75
4	2523	Johnson, R	MOD	149.75
5	2539	LaMance, K	LOW	124.80
6	2544	Jones, M	HIGH	124.80
7	2552	Reberson, P	MOD	149.75
8	2555	King, E	MOD	149.75
9	2563	Pitts, D	LOW	124.80
10	2568	Eberhardt, S	LOW	124.80
11	2571	Nunnelly, A	HIGH	149.75
12	2572	Oberon, M	LOW	85.20
13	2574	Peterson, V	MOD	149.75
14	2575	Quigley, M	HIGH	124.80
15	2578	Cameron, L	MOD	124.80
16	2579	Underwood, K	LOW	149.75
17	2584	Takahashi, Y	MOD	124.80
18	2586	Derber, B	HIGH	85.20
19	2588	Ivan, H	LOW	85.20
20	2589	Wilcox, E	HIGH	149.75
21	2595	Warren, C	MOD	149.75

Remember to check the Log window again to verify that your program ran correctly.

Log 4.1 SAS Log: No Error Messages

```

9231 data sasuser.admitfee;
9232     set sasuser.admit;
9233 run;

NOTE: There were 21 observations read from the data set SASUSER.ADMIT.
NOTE: The data set SASUSER.ADMITFEE has 21 observations and 9 variables.
NOTE: DATA statement used (Total process time):
      real time           0.00 seconds
      cpu time            0.00 seconds

9234 proc print data=sasuser.admitfee;
9235     var id name actlevel fee;
9236 run;

NOTE: There were 21 observations read from the data set SASUSER.ADMITFEE.
NOTE: PROCEDURE PRINT used (Total process time):
      real time           0.01 seconds
      cpu time            0.01 seconds

```

TIP To resubmit a section of code in the Windows operating environment, highlight the selected code in the code editing window. Then press the **F3** key on the keyboard.

CAUTION:

If you are programming in the SAS windowing environment, this and other chapters show you the Editor window only. If you are not using the Editor as a code editing window, be sure to adapt the directions for the Editor window.

The PUTLOG Statement

The Basics of Logic Errors

A *logic error* occurs when the program statements execute, but produce incorrect results. Because no notes are written to the log, logic errors are often difficult to detect. Use the PUTLOG statement in the DATA step to help identify logic errors.

Syntax, PUTLOG statement

PUTLOG 'message';

message specifies the message that you want to write to the SAS log. It can include character literals, variable names, formats, and pointer controls.

Note: You can precede your message text with WARNING, MESSAGE, or NOTE to better identify the output in the log.

The PUTLOG statement can be used to write to the SAS log in both batch and interactive modes. If an external file is open for output, use this statement to ensure that debugging messages are written to the SAS log and not to the external file.

Details

The temporary variables `_N_` and `_ERROR_` can be helpful when you debug a DATA step.

Variable	Description	Debugging Use
<code>_N_</code>	The number of times the DATA step iterated	Displays debugging messages for a specified number of iterations of the DATA step
<code>_ERROR_</code>	Initialized to 0, set to 1 when an error occurs	Displays debugging messages when an error occurs

Example: The DATA Step Produces Wrong Results but There Are No Error Messages

The raw data file contains information from a class. For each student, there are three scores from tests, and one score from homework. The program below is designed to select students whose average score is below 70. Although the program produces incorrect results, there are no error messages in the log.

```
data work.grades;
  infile class;
  input Name $ Score1 Score2 Score3 Homework;
  Homework=Homework*2;
  AverageScore = MEAN(Score1 + Score2 + Score3 + Homework);
  if AverageScore < 70;
run;
```

A glance at the raw data shows that there should be students whose mean scores are below 70. However, the data set Work.Grades has 0 observations and 6 variables.

```
NOTE: The infile CLASS is:
      Filename=C:\Users\My SAS Files\9.4\class.dat,
      RECFM=V,LRECL=32767,File Size (bytes)=328,
      Last Modified=04Dec2017:11:01:52,
      Create Time=04Dec2017:10:55:21

NOTE: 4 records were read from the infile CLASS.
      The minimum record length was 80.
      The maximum record length was 80.

NOTE: The data set WORK.GRADES has 0 observations and 6 variables.
```

Use the PUTLOG statement to determine where the DATA step received incorrect instructions. Place the PUTLOG statement before the subsetting IF.

```
PUTLOG Name= Score1= Score2= Score3= Homework= AverageScore=;
```

Looking at the log, you can see the result of the PUTLOG statement. The data that is listed in the middle of the log shows that the variables were read in properly, and the variable Homework was adjusted to be weighted the same as Scores1–3. However, the values of AverageScore are incorrect. They are above the available maximum grade. There is a syntax error in the line that computes AverageScore: Instead of commas separating the three score variables in the MEAN function, there are plus signs. Since functions can contain arithmetic expressions, SAS simply added the four variables together, as instructed, and computed the mean of a single number. That is why no observations had values of AverageScore below 70.

```
10243 data work.grades;
10244     infile class;
10245     input Name $1-7 Score1 Score2 Score3 Homework;
10246     Homework=Homework*2;
10247     AverageScore = MEAN(Score1 + Score2 + Score3 + Homework);
10248     PUTLOG Name= Score1= Score2= Score3= Homework= AverageScore=;
10249     IF AverageScore < 70;
10250 run;

NOTE: The infile CLASS is:
      Filename=C:\Users\Documents\My SAS Files\9.4\class.dat,
      RECFM=V,LRECL=32767,File Size (bytes)=328,
      Last Modified=04Dec2017:11:01:01,
      Create Time=04Dec2017:10:55:21

Name=LINDA Score1=53 Score2=60 Score3=66 Homework=84 AverageScore=263
Name=DEREK Score1=72 Score2=64 Score3=56 Homework=64 AverageScore=256
Name=KATHY Score1=98 Score2=82 Score3=100 Homework=96 AverageScore=376
Name=MICHAEL Score1=80 Score2=55 Score3=95 Homework=100 AverageScore=330
NOTE: 4 records were read from the infile CLASS.
      The minimum record length was 80.
      The maximum record length was 80.

NOTE: The data set WORK.GRADES has 0 observations and 6 variables.
```

Fix the error by replacing the plus signs in the MEAN function with commas. You can drop the PUTLOG statement and use a PROC PRINT statement to view your results.

```
data work.grades;
    infile class;
    input Name $1-7 Score1 Score2 Score3 Homework;
```

```

Homework=Homework*2;
AverageScore = MEAN(Score1, Score2, Score3, Homework);
if AverageScore < 70;
run;
proc print data=work.grades;
run;

```

The figure below lists the names of students whose average score is below 70.

Figure 4.4 Corrected Program Output

Name	Score1	Score2	Score3	Homework	AverageScore
LINDA	53	60	66	84	65.75
DEREK	72	64	56	64	64.00

Missing RUN Statement

Each step in a SAS program is compiled and executed independently from every other step. As a step is compiled, SAS recognizes the end of the current step when it encounters one of the following errors:

- a DATA or PROC statement, which indicates the beginning of a new step
- a RUN or QUIT statement, which indicates the end of the current step

When the program below is submitted, the DATA step executes, but the PROC step does not. The PROC step does not execute because there is no following DATA or PROC step to indicate the beginning of a new step, nor is there a following RUN statement to indicate the end of the step.

```

data sasuser.admitfee;
  set sasuser.admit;
proc print data=sasuser.admitfee;
  var id name actlevel fee;

```

If you are programming in Enterprise Guide or SAS Studio, the system submits a RUN statement after every program that you submit, so the above program would execute normally. But in the SAS windowing environment, because there is nothing to indicate the end of the PROC step, the PRINT procedure waits before executing, and a PROC PRINT running message appears at the top of the active window.

Note: Although omitting a RUN statement is not technically an error, it can produce unexpected results.

Correcting the Error: Missing RUN Statement

To correct the error, submit a RUN statement to complete the PROC step.

```

run;

```

Missing Semicolon

One of the most common errors is a missing semicolon at the end of a statement. Here is an example.

```

data sasuser.admitfee;
    set sasuser.admit;
run;
proc print data=sasuser.admitfee
    var id name actlevel fee;
run;

```

When you omit a semicolon, SAS reads the statement that lacks the semicolon (along with the following statement) as one long statement.

Log 4.2 SAS Log: Error Messages

```

9237 data sasuser.admitfee;
9238     set sasuser.admit;
9239 run;

NOTE: There were 21 observations read from the data set SASUSER.ADMIT.
NOTE: The data set SASUSER.ADMITFEE has 21 observations and 9 variables.
NOTE: DATA statement used (Total process time):
      real time          0.00 seconds
      cpu time           0.00 seconds

9240 proc print data=sasuser.admitfee
9241     var id name actlevel fee;
      ---
      22
      76
ERROR 22-322: Syntax error, expecting one of the following: ;, (, BLANKLINE,
CONTENTS, DATA,
          DOUBLE, GRANDTOTAL_LABEL, GRANDTOT_LABEL, GRAND_LABEL,
GTOTAL_LABEL, GTOT_LABEL,
          HEADING, LABEL, N, NOOBS, NOSUMLABEL, OBS, ROUND, ROWS, SPLIT,
STYLE, SUMLABEL,
          UNIFORM, WIDTH.
ERROR 76-322: Syntax error, statement will be ignored.
9242 run

```

Correcting the Error: Missing Semicolon

1. Find the statement that lacks a semicolon. You can usually find it by looking at the underscored keywords in the error message and working backward.
2. Add a semicolon in the appropriate location.
3. Resubmit the corrected program.
4. Check the Log window again to make sure there are no other errors.

Unbalanced Quotation Marks

Some syntax errors, such as the missing quotation mark after HIGH in the program below, cause SAS to misinterpret the statements in your program.

```

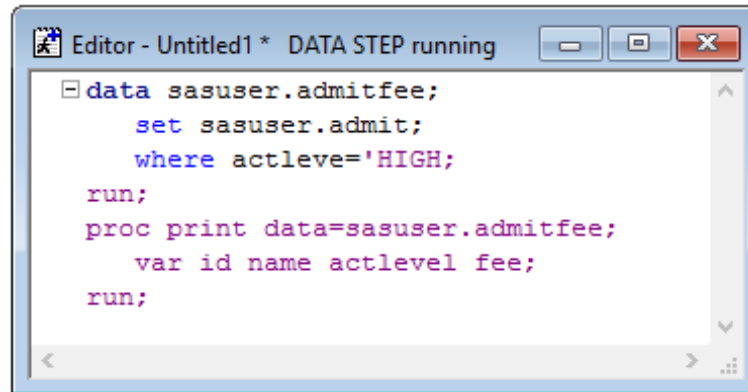
data sasuser.admitfee;
    set sasuser.admitfee;
    where actlevel='HIGH;
run;

```

```
proc print data=sasuser.admitfee;
  var id name actlevel fee;
run;
```

When the program is submitted, SAS is unable to resolve the DATA step, and a DATA STEP running message appears at the top of the active window.

Figure 4.5 SAS Editor Window with Message



TIP Both SAS Enterprise Guide and SAS Studio add a final line of code to stop unbalanced quotation marks.

Sometimes a warning appears in the SAS log that indicates the following:

- A quoted string has become too long.
- A statement that contains quotation marks (such as a TITLE or FOOTNOTE statement) is ambiguous because of invalid options or unquoted text.

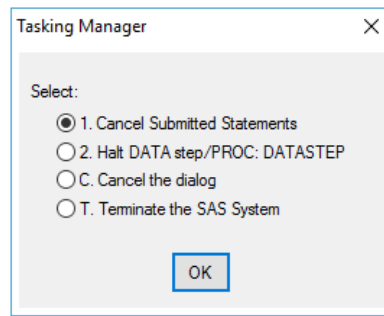
When you have unbalanced quotation marks, SAS is often unable to detect the end of the statement in which it occurs. In Enterprise Guide or SAS Studio, simply add the balancing quotation mark and resubmit the program. However, in some environments, this technique usually does not correct the error. SAS still considers the quotation marks to be unbalanced.

Therefore, you need to resolve the unbalanced quotation mark by canceling the submitted statements (in the Windows and UNIX operating environments) or by submitting a line of SAS code (in the z/OS operating environment) before you recall, correct, and resubmit the program.

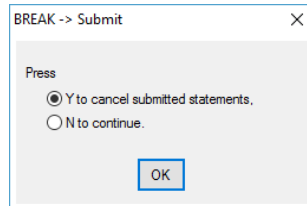
TIP If you do not correct this error when it occurs in the SAS windowing environment, it is likely that any subsequent programs that you submit in the current SAS session will generate errors.

Correcting the Error in the Windows Operating Environment

1. Press the Ctrl and Break keys or click the Break Icon  on the toolbar.
2. Select **1. Cancel Submitted Statements**, and then click **OK**.



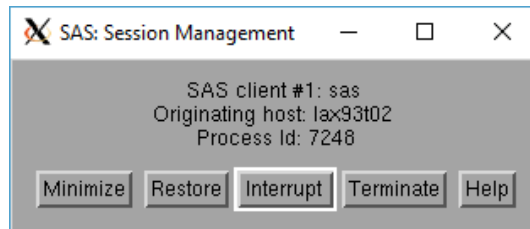
3. Select **Y to cancel submitted statements**, and then click **OK**.



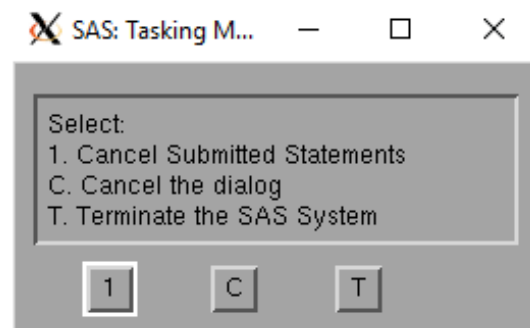
4. Correct the error and resubmit the program.

Correcting the Error in the UNIX Operating Environment

1. Open the Session Management window and click **Interrupt**.



2. Select **1. Cancel Submitted Statements**, and then click **Y**.



3. Correct the error and resubmit the program.

Correcting the Error in the z/OS Operating Environment

1. Submit an asterisk followed by a single quotation mark, a semicolon, and a RUN statement.

```
*'; run;
```

2. Delete the line that contains the asterisk followed by the single quotation mark, the semicolon, and the RUN statement.
3. Insert the missing quotation mark in the appropriate place.
4. Submit the corrected program.

TIP You can also use the above method in the Windows and UNIX operating environments.

Semantic Error: Invalid Option

An invalid option error occurs when you specify an option that is not valid in a particular statement. In the program below, the KEYLABEL option is not valid when used with the PROC PRINT statement.

```
data sasuser.admitfee;
    set sasuser.admit;
    where weight>180 and (actlevel="MOD" or actlevel="LOW");
run;
proc print data=sasuser.admitfee keylabel;
    label actlevel='Activity Level';
run;
```

When a SAS statement that contains an invalid option is submitted, a message appears in the Log window indicating that the option is not valid or not recognized.

Log 4.3 SAS Log: Syntax Error Message

```

9250 data sasuser.admitfee;
9251     set sasuser.admit;
9252     where weight>180 and (actlevel="MOD" or actlevel="LOW");
9253 run;

NOTE: There were 2 observations read from the data set SASUSER.ADMIT.
      WHERE (weight>180) and actlevel in ('LOW', 'MOD');
NOTE: The data set SASUSER.ADMITFEE has 2 observations and 9 variables.
NOTE: DATA statement used (Total process time):
      real time          0.00 seconds
      cpu time           0.00 seconds

9254 proc print data=sasuser.admit keylabel;
                        -----
                        22
                        202
ERROR 22-322: Syntax error, expecting one of the following: ;, (, BLANKLINE,
CONTENTS, DATA,
      DOUBLE, GRANDTOTAL_LABEL, GRANDTOT_LABEL, GRAND_LABEL,
GTOTAL_LABEL, GTOT_LABEL,
      HEADING, LABEL, N, NOOBS, NOSUMLABEL, OBS, ROUND, ROWS, SPLIT,
STYLE, SUMLABEL,
      UNIFORM, WIDTH.
ERROR 202-322: The option or parameter is not recognized and will be ignored.
9255     label actlevel='Activity Level';
9256 run;

NOTE: The SAS System stopped processing this step because of errors.
NOTE: PROCEDURE PRINT used (Total process time):
      real time          0.00 seconds
      cpu time           0.01 seconds

```

Correcting the Error: Invalid Option

1. Remove or replace the invalid option, and check your statement syntax as needed.
2. Resubmit the corrected program.
3. Check the Log window again to make sure there are no other errors.

Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

1. As you write and edit SAS programs, it is a good idea to do the following:
 - a. Begin DATA and PROC steps in column one.
 - b. Indent statements within a step.
 - c. Begin RUN statements in column one.
 - d. Do all of the above.

2. Suppose you have submitted a SAS program that contains spelling errors. Which set of steps should you perform, in the order shown, to revise and resubmit the program?
- Correct the errors.
 - Clear the Log window.
 - Resubmit the program.
 - Check the Log window.
 - Correct the errors.
 - Resubmit the program.
 - Check the Output window.
 - Check the Log window.
 - Correct the errors.
 - Clear the Log window.
 - Resubmit the program.
 - Check the Output window.
 - Correct the errors.
 - Clear the Output window.
 - Resubmit the program.
 - Check the Output window.

3. What happens if you submit the following program?

```
proc sort data=sasuser.stress out=maxrates;
  by maxhr;
run;
proc print data=maxrates label double noobs;
  label rechr='Recovery Heart Rate';
  var resthr maxhr rechr date;
  where toler='I' and resthr>90;
  sum fee;
run;
```

- Log messages indicate that the program ran successfully.
 - A PROC SORT running message appears at the top of the active window, and a log message might indicate an error in a statement that seems to be valid.
 - A log message indicates that an option is not valid or not recognized.
 - A PROC PRINT running message appears at the top of the active window, and a log message might indicate that a quoted string has become too long or that the statement is ambiguous.
4. What generally happens when a syntax error is detected?
- SAS continues processing the step.
 - SAS continues to process the step, and the Log window displays messages about the error.
 - SAS stops processing the step in which the error occurred, and the Log window displays messages about the error.

- d. SAS stops processing the step in which the error occurred, and the Output window displays messages about the error.
5. A syntax error occurs during the following actions:
 - a. Some data values are not appropriate for the SAS statements that are specified in a program.
 - b. The form of the elements in a SAS statement is correct, but the elements are not valid for that usage.
 - c. Program statements do not conform to the rules of the SAS language.
 - d. None of the above.
6. How can you tell whether you have specified an invalid option in a SAS program?
 - a. A log message indicates an error in a statement that seems to be valid.
 - b. A log message indicates that an option is not valid or not recognized.
 - c. The message PROC STEP running or DATA STEP running appears at the top of the active window.
 - d. You cannot tell until you view the output from the program.
7. Which of the following programs contains a syntax error?
 - a.

```
proc sort data=sasuser.mysales;
    by region;
run;
```
 - b.

```
dat sasuser.mysales;
    set mydata.sales99;
    where sales<5000;
run;
```
 - c.

```
proc print data=sasuser.mysales label;
    label region='Sales Region';
run;
```
 - d. None of the above.
8. What should you do after submitting the following program in the Windows or UNIX operating environment?


```
proc print data=mysales;
    where state='NC';
run;
```

 - a. Submit a RUN statement to complete the PROC step.
 - b. Recall the program. Then add a quotation mark and resubmit the corrected program.
 - c. Cancel the submitted statements. Then recall the program, add a quotation mark, and resubmit the corrected program.
 - d. Recall the program. Then replace the invalid option and resubmit the corrected program.
9. Suppose you submit a short, simple DATA step. If the active window displays the message DATA step running for a long time, what probably happened?
 - a. You misspelled a keyword.
 - b. You forgot to end the DATA step with a RUN statement.

- c. You specified an invalid data set option.
 - d. Some data values were not appropriate for the SAS statements that you specified.
10. What types of errors can PUTLOG statement help you resolve?
- a. syntax errors
 - b. semantic errors
 - c. logic errors
 - d. all of the above

Chapter 5

Creating List Reports

Creating a Basic Report	68
Selecting Variables	69
The PROC PRINT Step	69
Removing the OBS Column	70
Identifying Observations	70
Using the ID Statement in PROC PRINT	70
Example: ID Statement and VAR Statement	71
Selecting Observations	71
Specifying WHERE Expressions	72
Using the CONTAINS Operator	73
Specifying Compound WHERE Expressions	73
Examples of WHERE Statements	73
Sorting Data	74
The SORT Procedure	74
Example: PROC SORT	75
Generating Column Totals	76
The SUM Statement	76
Requesting Subtotals	77
Example: SUM Statement	78
Creating a Customized Layout with BY Groups and ID Variables	79
Example: ID, BY, and SUM Statements	79
Requesting Subtotals on Separate Pages	80
Example: PAGEBY Statement	80
Double Spacing LISTING Output	82
Specifying Titles and Footnotes in Procedure Output	82
TITLE and FOOTNOTE Statements	82
Using the TITLES and FOOTNOTES Windows	83
Example: Creating Titles	83
Example: Creating Footnotes	84
Modifying and Canceling Titles and Footnotes	85
Assigning Descriptive Labels	88
Temporarily Assigning Labels to Variables	88
Example: The LABEL Option	88
Using Single or Multiple LABEL Statements	89
Formatting Data Values	90
Temporarily Assigning Formats to Variables	90

Specifying SAS Formats	91
Field Widths	92
Decimal Places	92
Examples: Data Values and Formats	93
Using Permanently Assigned Labels and Formats	93
Chapter Quiz	94

Creating a Basic Report

To produce a simple list report, you first reference the library where your SAS data set is stored. You can also set system options to control the appearance of your reports. Then you submit a PROC PRINT step.

Syntax, PROC PRINT step:

PROC PRINT DATA=SAS-data-set;

RUN;

SAS-data-set is the name of the SAS data set to be printed.

In the program below, the PROC PRINT statement invokes the PRINT procedure and specifies the data set Therapy in the SAS library to which the libref Patients has been assigned.

```
libname patients 'c:\records\patients';
proc print data=patients.therapy;
run;
```

Notice the layout of the resulting report below. These are the default behaviors:

- All observations and variables in the data set are printed.
- A column for observation numbers appears on the far left.
- Variables and observations appear in the order in which they occur in the data set.

Figure 5.1 Patients.Therapy Data Set (partial output)

Obs	Date	AerClass	WalkJogRun	Swim
1	JAN2012	56	78	14
2	FEB2012	32	109	19
3	MAR2012	35	106	22
4	APR2012	47	115	24
5	MAY2012	55	121	31

Selecting Variables

The PROC PRINT Step

By default, a PROC PRINT step lists all the variables in a data set. You can select variables and control the order in which they appear by using a VAR statement in your PROC PRINT step.

Syntax, VAR statement:

VAR *variable(s)*;

variable(s) is one or more variable names, separated by blanks.

For example, the following VAR statement specifies that only the variables Age, Height, Weight, and Fee be printed, in that order:

```
proc print data=clinic.admit;
  var age height weight fee;
run;
```

The procedure output from the PROC PRINT step with the VAR statement lists only the values for those variables.

Figure 5.2 PRINT Procedure Output

The SAS System				
Obs	Age	Height	Weight	Fee
1	27	72	168	85.20
2	34	66	152	124.80
3	31	61	123	149.75
4	43	63	137	149.75
5	51	71	158	124.80
6	29	76	193	124.80
7	32	67	151	149.75
8	35	70	173	149.75
9	34	73	154	124.80
10	49	64	172	124.80
11	44	66	140	149.75
12	28	62	118	85.20
13	30	69	147	149.75
14	40	69	163	124.80
15	47	72	173	124.80
16	60	71	191	149.75
17	43	65	123	124.80
18	25	75	188	85.20
19	22	63	139	85.20
20	41	67	141	149.75
21	54	71	183	149.75

In addition to selecting variables, you can suppress observation numbers.

Removing the OBS Column

To remove the Obs column, specify the NOOBS option in the PROC PRINT statement.

```
proc print data=work.example noobs;
  var age height weight fee;
run;
```

Figure 5.3 PRINT Procedure Output with No Observation Numbers

The SAS System

Age	Height	Weight	Fee
27	72	168	85.20
34	66	152	124.80
31	61	123	149.75
43	63	137	149.75
51	71	158	124.80
29	76	193	124.80
32	67	151	149.75
35	70	173	149.75
34	73	154	124.80
49	64	172	124.80
44	66	140	149.75
28	62	118	85.20
30	69	147	149.75
40	69	163	124.80
47	72	173	124.80
60	71	191	149.75
43	65	123	124.80
25	75	188	85.20
22	63	139	85.20
41	67	141	149.75
54	71	183	149.75

Identifying Observations

Using the ID Statement in PROC PRINT

This statement identifies observations by using the formatted values of the variables that you list instead of by using observation numbers. This statement is particularly useful when observations are too long to print on one line.

Syntax, ID statement in the PRINT procedure:

ID *variable(s)*;

variable(s) specifies one or more variables to print instead of the observation number at the beginning of each row of the report.

Example: ID Statement and VAR Statement

To replace the Obs column and identify observations based on an employee's ID number and last name, submit the following program.

```
proc print data=sasuser.reps;
    id idnum lastname;
run;
```

This is HTML output from the program:

Figure 5.4 HTML Output

The SAS System

IDnum	LastName	FirstName	City	State	Sex	JobCode	Salary	Birth	Hired	HomePhone
1269	CASTON	FRANKLIN	STAMFORD	CT	M	NA1	41690.00	06MAY60	01DEC80	203/781-3335
1935	FERNANDEZ	KATRINA	BRIDGEPORT	CT		NA2	51081.00	31MAR42	19OCT69	203/675-2962
1417	NEWKIRK	WILLIAM	PATERSON	NJ	,	NA2	52270.00	30JUN52	10MAR77	201/732-6611
1839	NORRIS	DIANE	NEW YORK	YN	F	NA1	43433.00	02DEC58	06JUL81	718/384-1767
1111	RHODES	JEREMY	PRINCETON	NJ	M	NA1	40586.00	17JUL61	03NOV80	201/812-1837
1352	RIVERS	SIMON	NEW YORK	NY	M	NA2	5379.80	05DEC48	19OCT74	718/383-3345
1332	STEPHENSON	ADAM	BRIDGEPORT	CT	M	NA1	42178.00	20SEP58	07JUN79	203/675-1497
1443	WELLS	AGNES	STAMFORD	CT	F	NA1	422.74	20NOV56	01SEP79	203/781-5546

If a variable in the ID statement also appears in the VAR statement, the output contains two columns for that variable. In the example below, the variable IDnum appears twice.

```
proc print data=sasuser.reps;
    id idnum lastname;
    var idnum sex jobcode salary;
run;
```

Figure 5.5 IDNUM Output

The SAS System

IDnum	LastName	IDnum	Sex	JobCode	Salary
1269	CASTON	1269	M	NA1	41690.00
1935	FERNANDEZ	1935		NA2	51081.00
1417	NEWKIRK	1417	,	NA2	52270.00
1839	NORRIS	1839	F	NA1	43433.00
1111	RHODES	1111	M	NA1	40586.00
1352	RIVERS	1352	M	NA2	5379.80
1332	STEPHENSON	1332	M	NA1	42178.00
1443	WELLS	1443	F	NA1	422.74

Selecting Observations

By default, a PROC PRINT step lists all the observations in a data set. You can control which observations are printed by adding a WHERE statement to your PROC PRINT step. There should be only one WHERE statement in a step. If multiple WHERE statements are issued, only the last statement is processed.

Syntax, WHERE statement:

WHERE *where-expression*;

where-expression specifies a condition for selecting observations. The *where-expression* can be any valid SAS expression.

For example, the following WHERE statement selects only observations for which the value of Age is greater than 30:

```
proc print data=clinic.admit;
  var age height weight fee;
  where age>30;
run;
```

Here is the output from the PROC PRINT step with the WHERE statement:

Figure 5.6 PROC PRINT Output with WHERE Statement

The SAS System

Obs	Age	Height	Weight	Fee
2	34	66	152	124.80
3	31	61	123	149.75
4	43	63	137	149.75
5	51	71	158	124.80
7	32	67	151	149.75
8	35	70	173	149.75
9	34	73	154	124.80
10	49	64	172	124.80
11	44	66	140	149.75
14	40	69	163	124.80
15	47	72	173	124.80
16	60	71	191	149.75
17	43	65	123	124.80
20	41	67	141	149.75
21	54	71	183	149.75

Specifying WHERE Expressions

In the WHERE statement, you can specify any variable in the SAS data set, not just the variables that are specified in the VAR statement. The WHERE statement works for both character and numeric variables. To specify a condition based on the value of a character variable:

- Enclose the value in quotation marks.
- Write the value with lowercase, uppercase, or mixed case letters exactly as it appears in the data set.

You use the following comparison operators to express a condition in the WHERE statement:

Table 5.1 Comparison Operators in a WHERE Statement

Symbol	Meaning	Sample Program Code
= or eq	equal to	where name='Jones, C.';
^= or ne	not equal to	where temp ne 212;
> or gt	greater than	where income>20000;
< or lt	less than	where partno lt "BG05";
>= or ge	greater than or equal to	where id>='1543';
<= or le	less than or equal to	where pulse le 85;

For more information about valid SAS expressions, see [Chapter 6, “Creating SAS Data Sets from External Files,”](#) on page 99.

Using the CONTAINS Operator

The CONTAINS operator selects observations that include the specified substring. The symbol for the CONTAINS operator is ?. You can use either the CONTAINS keyword or the symbol in your code, as shown below.

```
where firstname CONTAINS 'Jon';
where firstname ? 'Jon';
```

Specifying Compound WHERE Expressions

You can also use WHERE statements to select observations that meet multiple conditions. To link a sequence of expressions into compound expressions, you use logical operators, including the following:

Table 5.2 Compound WHERE Expression Operators

Operator, Symbol		Description
AND	&	and, both. If both expressions are true, then the compound expression is true.
OR		or, either. If either expression is true, then the compound expression is true.

Examples of WHERE Statements

- You can use compound expressions like these in your WHERE statements:

```
where age<=55 and pulse>75;
where area='A' or region='S';
where ID>'1050' and state='NC';
```

- When you test for multiple values of the same variable, you specify the variable name in each expression:

```
where actlevel='LOW' or actlevel='MOD';
where fee=124.80 or fee=178.20;
```

- You can use the IN operator as a convenient alternative:

```
where actlevel in ('LOW','MOD');
where fee in (124.80,178.20);
```

- To control how compound expressions are evaluated, you can use parentheses (expressions in parentheses are evaluated first):

```
where (age<=55 and pulse>75) or area='A';
where age<=55 and (pulse>75 or area='A');
```

Sorting Data

The SORT Procedure

By default, PROC PRINT lists observations in the order in which they appear in your data set. To sort your report based on values of a variable, you must use PROC SORT to sort your data before using the PRINT procedure to create reports from the data.

The SORT procedure does the following:

- rearranges the observations in a SAS data set
- creates a new SAS data set that contains the rearranged observations
- replaces the original SAS data set by default
- can sort on multiple variables
- can sort in ascending or descending order
- treats missing values as the smallest possible values

Note: PROC SORT does not generate printed output.

Syntax, PROC SORT step:

```
PROC SORT DATA=SAS-data-set<OUT=SAS-data-set>;
```

```
    BY <DESCENDING> BY-variable(s);
```

```
RUN;
```

- The DATA= option specifies the data set to be read.
 - The OUT= option creates an output data set that contains the data in sorted order.
 - BY-variable(s) in the required BY statement specifies one or more variables whose values are used to sort the data.
 - The DESCENDING option in the BY statement sorts observations in descending order. If you have more than one variable in the BY statement, DESCENDING applies only to the variable that immediately follows it.
-

CAUTION:

If you do not use the OUT= option, PROC SORT overwrites the data set that is specified in the DATA= option.

Example: PROC SORT

In the following program, the PROC SORT step sorts the permanent SAS data set Clinic.Admit by the values of the variable Age within the values of the variable Weight and creates the temporary SAS data set Wgtadmit. Then the PROC PRINT step prints a subset of the Wgtadmit data set.

```
proc sort data=clinic.admit out=work.wgtadmit;
    by weight age;
run;
proc print data=work.wgtadmit;
    var weight age height fee;
    where age>30;
run;
```

The report displays observations in ascending order of age within weight.

Figure 5.7 Observations Displayed in Ascending Order of Age within Weight

The SAS System

Obs	Weight	Age	Height	Fee
2	123	31	61	149.75
3	123	43	65	124.80
4	137	43	63	149.75
6	140	44	66	149.75
7	141	41	67	149.75
9	151	32	67	149.75
10	152	34	66	124.80
11	154	34	73	124.80
12	158	51	71	124.80
13	163	40	69	124.80
15	172	49	64	124.80
16	173	35	70	149.75
17	173	47	72	124.80
18	183	54	71	149.75
20	191	60	71	149.75

Adding the DESCENDING option to the BY statement sorts observations in ascending order of age within descending order of weight. Notice that DESCENDING applies only to the variable Weight.

```
proc sort data=clinic.admit out=work.wgtadmit;
    by descending weight age;
run;
proc print data=work.wgtadmit;
    var weight age height fee;
    where age>30;
run;
```

Figure 5.8 Observations Displayed in Descending Order**The SAS System**

Obs	Weight	Age	Height	Fee
2	191	60	71	149.75
4	183	54	71	149.75
5	173	35	70	149.75
6	173	47	72	124.80
7	172	49	64	124.80
9	163	40	69	124.80
10	158	51	71	124.80
11	154	34	73	124.80
12	152	34	66	124.80
13	151	32	67	149.75
15	141	41	67	149.75
16	140	44	66	149.75
18	137	43	63	149.75
19	123	31	61	149.75
20	123	43	65	124.80

Generating Column Totals

The SUM Statement

To produce column totals for numeric variables, you can list the variables to be summed in a SUM statement in your PROC PRINT step.

Syntax, SUM statement:

SUM *variable(s)*;

variable(s) is one or more numeric variable names, separated by blanks.

The SUM statement in the following PROC PRINT step requests column totals for the variable BalanceDue:

```
proc print data=clinic.insure;
  var name policy balancedue;
  where pctinsured < 100;
  sum balancedue;
run;
```

Column totals appear at the end of the report in the same format as the values of the variables.

Figure 5.9 Column Totals

The SAS System

Obs	Name	Policy	BalanceDue
2	Almers, C	95824	156.05
3	Bonaventure, T	87795	9.48
4	Johnson, R	39022	61.04
5	LaMance, K	63265	43.68
6	Jones, M	92478	52.42
7	Reberson, P	25530	207.41
8	King, E	18744	27.19
9	Pitts, D	60976	310.82
10	Eberhardt, S	81589	173.17
13	Peterson, V	75986	228.00
14	Quigley, M	97048	99.01
15	Cameron, L	42351	111.41
17	Takahashi, Y	54219	186.58
18	Derber, B	74653	236.11
20	Wilcox, E	94034	212.20
21	Warren, C	20347	164.44
			2279.0

Requesting Subtotals

You might also want to subtotal numeric variables. To produce subtotals, add both a SUM statement and a BY statement to your PROC PRINT step.

Syntax, BY statement in the PRINT procedure:

```
BY <DESCENDING> BY-variable-1
      <...<DESCENDING> <BY-variable-n>>
      <NOTSORTED>;
```

- *BY-variable* specifies a variable that the procedure uses to form BY groups. You can specify more than one variable, separated by blanks.
 - The DESCENDING option specifies that the data set is sorted in descending order by the variable that immediately follows.
 - The NOTSORTED option specifies that the observations in the data set that have the same BY values are grouped together, but are not necessarily sorted in alphabetical or numeric order. For example, the observations might be sorted in chronological order using a date format such as DDMMYY. If observations that have the same values for the BY variables are not contiguous, the procedure treats each contiguous set as a separate BY group.
-

Note: The NOTSORTED option applies to all of the variables in the BY statement. You can specify the NOTSORTED option anywhere within the BY statement. The requirement for ordering or indexing observations according to the values of BY variables is suspended when you use the NOTSORTED option.

Example: SUM Statement

The SUM statement in the following PROC PRINT step requests column totals for the variable Fee, and the BY statement produces a subtotal for each value of ActLevel.

```
proc sort data=clinic.admit out=work.activity;
  by actlevel;
run;
proc print data=work.activity;
  var age height weight fee;
  where age>30;
  sum fee;
  by actlevel;
run;
```

In the output, the BY variable name and value appear before each BY group. The BY variable name and the subtotal appear at the end of each BY group.

Figure 5.10 BY Group Output: High

ActLevel=HIGH				
Obs	Age	Height	Weight	Fee
2	34	66	152	124.80
4	44	66	140	149.75
5	40	69	163	124.80
7	41	67	141	149.75
ActLevel				549.10

Figure 5.11 BY Group Output: Low

ActLevel=LOW				
Obs	Age	Height	Weight	Fee
8	31	61	123	149.75
9	51	71	158	124.80
10	34	73	154	124.80
11	49	64	172	124.80
13	60	71	191	149.75
ActLevel				673.90

Figure 5.12 BY Group Output: Mod

ActLevel=MOD				
Obs	Age	Height	Weight	Fee
15	43	63	137	149.75
16	32	67	151	149.75
17	35	70	173	149.75
19	47	72	173	124.80
20	43	65	123	124.80
21	54	71	183	149.75
ActLevel				848.60
				2071.60

Creating a Customized Layout with BY Groups and ID Variables

In the previous example, you might have noticed the redundant information for the BY variable. For example, in the PROC PRINT output below, the BY variable ActLevel is identified both before the BY group and for the subtotal.

Figure 5.13 Creating a Customized Layout with BY Groups and ID Variables

ActLevel=HIGH

Obs	Age	Height	Weight	Fee
2	34	66	152	124.80
4	44	66	140	149.75
5	40	69	163	124.80
7	41	67	141	149.75
ActLevel				549.10

To show the BY variable heading only once, use an ID statement and a BY statement together with the SUM statement. Here are the results when an ID statement specifies the same variable as the BY statement:

- The Obs column is suppressed.
- The ID or BY variable is printed in the left-most column.
- Each ID or BY value is printed only at the start of each BY group and on the line that contains that group's subtotal.

Example: ID, BY, and SUM Statements

The ID, BY, and SUM statements work together to produce the output shown below. The ID variable is listed only once for each BY group and once for each sum. The BY lines are suppressed. Instead, the value of the ID variable, ActLevel, identifies each BY group.

```
proc sort data=clinic.admit out=work.activity;
    by actlevel;
run;
proc print data=work.activity;
    var age height weight fee;
    where age>30;
    sum fee;
    by actlevel;
    id actlevel;
run;
```

Figure 5.14 Creating Custom Output Example Output**The SAS System**

ActLevel	Age	Height	Weight	Fee
HIGH	34	66	152	124.80
	44	66	140	149.75
	40	69	163	124.80
	41	67	141	149.75
HIGH				549.10

ActLevel	Age	Height	Weight	Fee
LOW	31	61	123	149.75
	51	71	158	124.80
	34	73	154	124.80
	49	64	172	124.80
	60	71	191	149.75
LOW				673.90

ActLevel	Age	Height	Weight	Fee
MOD	43	63	137	149.75
	32	67	151	149.75
	35	70	173	149.75
	47	72	173	124.80
	43	65	123	124.80
	54	71	183	149.75
MOD				848.60
				2071.60

Requesting Subtotals on Separate Pages

As another enhancement to your PROC PRINT report, you can request that each BY group be printed on a separate page by using the PAGEBY statement.

Syntax, PAGEBY statement:

PAGEBY *BY-variable*;

BY-variable identifies a variable that appears in the BY statement in the PROC PRINT step. PROC PRINT begins printing a new page if the value of the BY variable changes, or if the value of any BY variable that precedes it in the BY statement changes.

Note: The variable specified in the PAGEBY statement must also be specified in the BY statement in the PROC PRINT step.

Example: PAGEBY Statement

The PAGEBY statement in the program below prints BY groups for the variable ActLevel separately. The BY groups appear separated by horizontal lines in the HTML output.

```

proc sort data=clinic.admit out=work.activity;
  by actlevel;
run;
proc print data=work.activity;
  var age height weight fee;
  where age>30;
  sum fee;
  by actlevel;
  id actlevel;
  pageby actlevel;
run;

```

Figure 5.15 PAGEBY Example: High

The SAS System

ActLevel	Age	Height	Weight	Fee
HIGH	34	66	152	124.80
	44	66	140	149.75
	40	69	163	124.80
	41	67	141	149.75
HIGH				549.10

Figure 5.16 PAGEBY Example: Low

The SAS System

ActLevel	Age	Height	Weight	Fee
LOW	31	61	123	149.75
	51	71	158	124.80
	34	73	154	124.80
	49	64	172	124.80
	60	71	191	149.75
LOW				673.90

Figure 5.17 PAGEBY Example: Mod

The SAS System

ActLevel	Age	Height	Weight	Fee
MOD	43	63	137	149.75
	32	67	151	149.75
	35	70	173	149.75
	47	72	173	124.80
	43	65	123	124.80
	54	71	183	149.75
MOD				848.60
				2071.60

Double Spacing LISTING Output

If you are generating SAS LISTING output, one way to control the layout is to double space it. To do so, specify the `DOUBLE` option in the `PROC PRINT` statement.

```
proc print data=clinic.stress double;
  var resthr maxhr rechr;
  where tolerance='I';
run;
```

Note: Double spacing does not apply to HTML output.

TIP To generate SAS LISTING output, you must select **Tools** ⇒ **Options** ⇒ **Preferences**. Select the **Results** tab. Select the **Create listing** option.

Figure 5.18 Double-Spaced LISTING Output

The SAS System			
Obs	Rest HR	Max HR	Rec HR
2	68	171	133
3	78	177	139
8	70	167	122
11	65	181	141
14	74	152	113
15	75	158	108
20	78	189	138

Specifying Titles and Footnotes in Procedure Output

TITLE and FOOTNOTE Statements

To make your report more meaningful and self-explanatory, you can assign up to 10 titles with procedure output by using `TITLE` statements before the `PROC` step. Likewise, you can specify up to 10 footnotes by using `FOOTNOTE` statements before the `PROC` step.

TIP Because `TITLE` and `FOOTNOTE` statements are global statements, place them anywhere within or before the `PRINT` procedure. Titles and footnotes are assigned as soon as `TITLE` or `FOOTNOTE` statements are read; they apply to all subsequent output.

Syntax, TITLE, and FOOTNOTE statements:

TITLE<*n*> '*text*';

FOOTNOTE<*n*> '*text*';

n is a number from 1 to 10 that specifies the title or footnote line, and '*text*' is the actual title or footnote to be displayed. The maximum title or footnote length depends on your operating environment and on the value of the LINESIZE= option.

The keyword TITLE is equivalent to title1. Likewise, FOOTNOTE is equivalent to footnote1. If you do not specify a title, the default title is The SAS System. No footnote is printed unless you specify one.

As a best practice make sure to match quotation marks that enclose the title or footnote text.

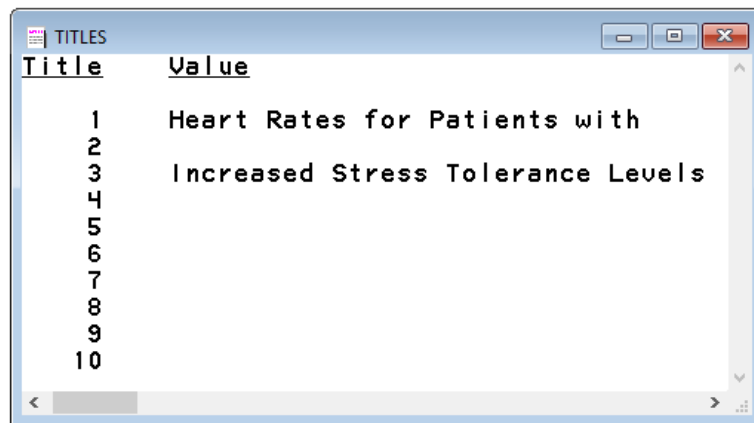
Using the TITLES and FOOTNOTES Windows

You can also specify titles in the TITLES window and footnotes in the FOOTNOTES window. Titles and footnotes that you specify in these windows are not stored with your program, and they remain in effect only during your SAS session.

To open the TITLES window, issue the TITLES command in the command bar. To open the FOOTNOTES window, issue the FOOTNOTES command in the command bar.

To specify a title or footnote, enter the text that you want next to the number of the line where the text should appear. To cancel a title or footnote, delete the existing text. Notice that you do not enclose text in quotation marks in these windows.

Figure 5.19 Titles Window



Example: Creating Titles

The two TITLE statements below, specified for lines 1 and 3, define titles for the PROC PRINT output.

```
title1 'Heart Rates for Patients with';
title3 'Increased Stress Tolerance Levels';
proc print data=clinic.stress;
    var resthr maxhr rech;
    where tolerance='I';
run;
```

Figure 5.20 HTML Output with Titles: SAS®9

**Heart Rates for Patients with
Increased Stress Tolerance Levels**

Obs	RestHR	MaxHR	RecHR
2	68	171	133
3	78	177	139
8	70	167	122
11	65	181	141
14	74	152	113
15	75	158	108
20	78	189	138

In SAS LISTING output for all versions of SAS, title line 2 is blank, as shown below. Titles are centered by default.

Figure 5.21 LISTING Output with Titles: All Versions

**Heart Rates for Patients with
Increased Stress Tolerance Levels**

Obs	Rest HR	Max HR	Rec HR
2	68	171	133
3	78	177	139
8	70	167	122
11	65	181	141
14	74	152	113
15	75	158	108
20	78	189	138

Example: Creating Footnotes

The two FOOTNOTE statements below, specified for lines 1 and 3, define footnotes for the PROC PRINT output. Since there is no footnote2, a blank line is inserted between footnotes 1 and 2 in the output.

```
footnote1 'Data from Treadmill Tests';
footnote3 '1st Quarter Admissions';
proc print data=clinic.stress;
  var resthr maxhr rechr;
  where tolerance='I';
run;
```

Footnotes appear at the bottom of each page of procedure output. Notice that footnote lines are pushed up from the bottom. The FOOTNOTE statement that has the largest number appears on the bottom line.

Figure 5.22 HTML Output with Footnotes

**Heart Rates for Patients with
Increased Stress Tolerance Levels**

Obs	RestHR	MaxHR	RecHR
2	68	171	133
3	78	177	139
8	70	167	122
11	65	181	141
14	74	152	113
15	75	158	108
20	78	189	138

Data from Treadmill Tests

1st Quarter Admissions

In SAS LISTING output, footnote line 2 is blank, as shown below. Footnotes are centered by default.

Figure 5.23 LISTING Output with Footnotes

Obs	Rest HR	Max HR	Rec HR
2	68	171	133
3	78	177	139
8	70	167	122
11	65	181	141
14	74	152	113
15	75	158	108
20	78	189	138
Data from Treadmill Tests			
1st Quarter Admissions			

Modifying and Canceling Titles and Footnotes

TITLE and FOOTNOTE statements are global statements. That is, after you define a title or footnote, it remains in effect until you modify it, cancel it, or end your SAS session.

For example, the footnotes that are assigned in the PROC PRINT step below also appear in the output from the PROC TABULATE step.

```
footnote1 'Data from Treadmill Tests';
footnote3 '1st Quarter Admissions';
proc print data=clinic.stress;
  var resthr maxhr rechhr;
  where tolerance='I';
run;
```

```
proc tabulate data=clinic.stress;
  where tolerance='I';
  var resthr maxhr;
  table mean*(resthr maxhr);
run;
```

Output 5.1 HTML Output with Titles and Footnotes

**Heart Rates for Patients with
Increased Stress Tolerance Levels**

Obs	RestHR	MaxHR	RecHR
2	68	171	133
3	78	177	139
8	70	167	122
11	65	181	141
14	74	152	113
15	75	158	108
20	78	189	138

Data from Treadmill Tests

1st Quarter Admissions

**Heart Rates for Patients with
Increased Stress Tolerance Levels**

Mean	
RestHR	MaxHR
72.57	170.71

Data from Treadmill Tests

1st Quarter Admissions

Redefining a title or footnote line cancels any higher-numbered title or footnote lines, respectively. In the example below, defining a title for line 2 in the second report automatically cancels title line 3.

```
title3 'Participation in Exercise Therapy';
proc print data=clinic.therapy;
  var swim walkjogrun aerclass;
run;
title2 'Report for March';
proc print data=clinic.therapy;
run;
```


Output 5.2 HTML Output with Titles and Footnotes

**Heart Rates for Patients with
Participation in Exercise Therapy**

Obs	Swim	WalkJogRun	AerClass
1	14	78	56
2	19	109	32
20	53	65	63
21	68	49	60
22	41	70	78
23	58	44	82
24	47	57	93

Data from Treadmill Tests

1st Quarter Admissions

**Heart Rates for Patients with
Report for March**

Obs	Date	AerClass	WalkJogRun	Swim
1	JAN2012	56	78	14
2	FEB2012	32	109	19
20	AUG2013	63	65	53
21	SEP2013	60	49	68
22	OCT2013	78	70	41
23	NOV2013	82	44	58
24	DEC2013	93	57	47

Data from Treadmill Tests

1st Quarter Admissions

To cancel all previous titles or footnotes, specify a null TITLE or FOOTNOTE statement (a TITLE or FOOTNOTE statement with no number or text) or a TITLE1 or FOOTNOTE1 statement with no text. This action cancels the default title The SAS System.

For example, in the program below, the null TITLE1 statement cancels all titles that are in effect before either PROC step executes. The null FOOTNOTE statement cancels all footnotes that are in effect after the PROC PRINT step executes. The PROC TABULATE output appears without a title or a footnote.

```

title1;
footnote1 'Data from Treadmill Tests';
footnote3 '1st Quarter Admissions';
proc print data=clinic.stress;
    var resthr maxhr rechr;

```

```

        where tolerance='I';
run;
footnote;
proc tabulate data=clinic.stress;
    var timemin timesec;
    table max*(timemin timesec);
run;

```

Output 5.3 HTML Output with No Titles

Obs	RestHR	MaxHR	RecHR
2	68	171	133
3	78	177	139
8	70	167	122
11	65	181	141
14	74	152	113
15	75	158	108
20	78	189	138

Data from Treadmill Tests
1st Quarter Admissions

Assigning Descriptive Labels

Temporarily Assigning Labels to Variables

You can also enhance your PROC PRINT report by labeling columns with more descriptive text. To label columns:

- Use the LABEL statement to assign a descriptive label to a variable.
- Use the LABEL option in the PROC PRINT statement to specify that the labels be displayed.

Syntax, LABEL statement:

```

LABEL variable1='label1'
        variable2='label2'
        ... ;

```

Labels can be up to 256 characters long. Enclose the label in quotation marks.

Tip: The LABEL statement applies only to the PROC step in which it appears.

Example: The LABEL Option

In the PROC PRINT step below, the variable name WalkJogRun is displayed with the label Walk/Jog/Run. Note the LABEL option in the PROC PRINT statement. Without the LABEL option in the PROC PRINT statement, PROC PRINT would use the name of the column heading, **walkjogrun**, even though you specified a value for the variable.

```
proc print data=clinic.therapy label;
  label walkjogrun='Walk/Jog/Run';
run;
```

Figure 5.24 Output Created with the LABEL Option (partial output)

Obs	Date	AerClass	Walk/Jog/Run	Swim
1	JAN2012	56	78	14
2	FEB2012	32	109	19
20	AUG2013	63	65	53
21	SEP2013	60	49	68
22	OCT2013	78	70	41
23	NOV2013	82	44	58
24	DEC2013	93	57	47

Using Single or Multiple LABEL Statements

You can assign labels in separate LABEL statements.

```
proc print data=clinic.admit label;
  var age height;
  label age='Age of Patient';
  label height='Height in Inches';
run;
```

Output 5.4 HTML Output with Multiple LABEL Statements (partial output)

Obs	Age of Patient	Height in Inches
1	27	72
2	34	66
17	43	65
18	25	75
19	22	63
20	41	67
21	54	71

Alternatively, you can assign any number of labels in a single LABEL statement.

```
proc print data=clinic.admit label;
  var actlevel height weight;
  label actlevel='Activity Level'
        height='Height in Inches'
        weight='Weight in Pounds';
run;
```

Output 5.5 HTML Output with One LABEL Statement (partial output)

Obs	Activity Level	Height in Inches	Weight in Pounds
1	HIGH	72	168
2	HIGH	66	152
17	MOD	65	123
18	HIGH	75	188
19	LOW	63	139
20	HIGH	67	141
21	MOD	71	183

Formatting Data Values

Temporarily Assigning Formats to Variables

In your SAS reports, formats control how the data values are displayed. To make data values more understandable when they are displayed in your procedure output, you can use the FORMAT statement, which associates formats with variables.

Formats affect only how the data values appear in output, not the actual data values as they are stored in the SAS data set.

Syntax, FORMAT statement:

FORMAT *variable(s) format-name;*

- *variable(s)* is the name of one or more variables whose values are to be written according to a particular pattern
- *format-name* specifies a SAS format or a user-defined format that is used to write out the values.

Tip: The FORMAT statement applies only to the PROC step in which it appears.

You can use a separate FORMAT statement for each variable, or you can format several variables (using either the same format or different formats) in a single FORMAT statement.

Table 5.3 Formats That Are Used to Format Data

FORMAT Statement	Description	Example
<code>format date mmddyy8.;</code>	associates the format MMDDYY8. with the variable Date	06/05/03
<code>format net comma5.0 gross comma8.2;</code>	associates the format COMMA5.0 with the variable Net and the format COMMA8.2 with the variable Gross	1,234 5,678.90

FORMAT Statement	Description	Example
<code>format net gross dollar9.2;</code>	associates the format DOLLAR9.2 with both variables, Net, and Gross	\$1,234.00 \$5,678.90

For example, the FORMAT statement below writes values of the variable Fee using dollar signs, commas, and no decimal places.

```
proc print data=clinic.admit;
  var actlevel fee;
  where actlevel='HIGH';
  format fee dollar4.;
run;
```

Figure 5.25 FORMAT Statement Output

Obs	ActLevel	Fee
1	HIGH	\$85
2	HIGH	\$125
6	HIGH	\$125
11	HIGH	\$150
14	HIGH	\$125
18	HIGH	\$85
20	HIGH	\$150

Specifying SAS Formats

The table below describes some SAS formats that are commonly used in reports.

Table 5.4 Commonly Used SAS Formats

Format	Description	Example
COMMA $w.d$	specifies values that contain commas and decimal places	comma8.2
DOLLAR $w.d$	specifies values that contain dollar signs, commas, and decimal places	dollar6.2
MMDDYY $w.$	specifies values as date values of the form 09/12/97 (MMDDYY8.) or 09/12/1997 (MMDDYY10.)	mmddy10.
$w.$	specifies values that are rounded to the nearest integer in w spaces	7.
$w.d$	specifies values that are rounded to d decimal places in w spaces	8.2
\$ $w.$	specifies values as character values in w spaces	\$12.

Format	Description	Example
DATEw.	specifies values as date values of the form 16OCT99 (DATE7.) or 16OCT1999 (DATE9.)	date9.

Field Widths

All SAS formats specify the total field width (*w*) that is used for displaying the values in the output. For example, suppose the longest value for the variable *Net* is a four-digit number, such as 5400. To specify the COMMAw.d format for *Net*, you specify a field width of 5 or more. You must count the comma, because it occupies a position in the output.

Note: When you use a SAS format, specify a field width (*w*) that is wide enough for the largest possible value. Otherwise, values might not be displayed properly.

Figure 5.26 Specifying a Field Width (*w*) with the *FORMAT* Statement

```
format net comma5.0;
5 , 4 0 0
1 2 3 4 5
```

Decimal Places

For numeric variables you can also specify the number of decimal places (*d*), if any, to be displayed in the output. Numbers are rounded to the specified number of decimal places. In the example above, no decimal places are displayed.

Writing the whole number 2030 as 2,030.00 requires eight print positions, including two decimal places and the decimal point.

Figure 5.27 Whole Number Decimal Places

```
format qtr3tax comma8.2;
2 , 0 3 0 . 0 0
1 2 3 4 5 6 7 8
```

Formatting 15374 with a dollar sign, commas, and two decimal places requires 10 print positions.

Figure 5.28 Specifying 10 Decimal Places

```
format totsales dollar10.2;
$ 1 5 , 3 7 4 . 0 0
1 2 3 4 5 6 7 8 9 10
```

Examples: Data Values and Formats

This table shows you how data values are displayed when different format, field width, and decimal place specifications are used.

Table 5.5 *Displaying Data Values with Formats*

Stored Value	Format	Displayed Value
38245.3975	COMMA12.2	38,245.40
38245.3975	12.2	38245.40
38245.3975	DOLLAR12.2	\$38,245.40
38245.3975	DOLLAR9.2	\$38245.40
38245.3975	DOLLAR8.2	38245.40
0	MMDDYY8.	01/01/60
0	MMDDYY10.	01/01/1960
0	DATE7.	01JAN60
0	DATE9.	01JAN1960

TIP If a format is too small, the following message is written to the SAS log:
 “NOTE: At least one W.D format was too small for the number to be printed. The decimal might be shifted by the 'BEST' format.”

Using Permanently Assigned Labels and Formats

When you use a LABEL or FORMAT statement within a PROC step, the label or format applies only to the output from that step.

However, in PROC steps, you can also use permanently assigned labels or formats. Permanent labels and formats can be assigned in the DATA step. These labels and formats are saved with the data set, and they can be reused by procedures that reference the data set.

For example, the DATA step below creates Flights.March and defines a format and label for the variable Date. Because the LABEL and FORMAT statements are inside the DATA step, they are written to the Flights.March data set and are available to the subsequent PRINT procedure.

```
data sasuser.paris;
set sasuser.laguardia;
  where dest="PAR" and (boarded=155 or boarded=146);
  label date='Departure Date';
  format date date9.;
```

```
run;

proc print data=sasuser.paris label date='Departure Date';
  var date dest boarded;
run;
```

Figure 5.29 Using Permanent Labels and Formats

Obs	Departure Date	Dest	Boarded
1	04MAR2012	PAR	146
2	07MAR2012	PAR	155
3	04MAR2012	PAR	146
4	07MAR2012	PAR	155

Notice that the PROC PRINT statement still requires the LABEL option in order to display the permanent labels. Other SAS procedures display permanently assigned labels and formats without additional statements or options.

For more information about permanently assigning labels and formats, see [Chapter 11](#), “Creating and Managing Variables,” on page 231.

Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- Which PROC PRINT step below creates the following output?

Date	Boarded	Transferred	Flight
04MAR12	232	18	219
05MAR12	160	4	219
06MAR12	163	14	219
07MAR12	241	9	219
08MAR12	183	11	219
09MAR12	211	18	219
10MAR12	167	7	219

- ```
proc print data=flights.laguardia noobs;
 var on changed flight;
 where on>=160;
run;
```
- ```
proc print data=flights.laguardia;
  var date on changed flight;
  where changed>3;
run;
```


- c.

```
proc print data=flights.laguardia label;
    id date;
    var boarded transferred flight;
    label boarded='On' transferred='Changed';
    where flight='219';
run;
```
 - d.

```
proc print flights.laguardia noobs;
    id date;
    var date on changed flight;
    where flight='219';
run;
```
2. Which of the following PROC PRINT steps is correct if labels are not stored with the data set?
 - a.

```
proc print data=allsales.totals label;
    label region8='Region 8 Yearly Totals';
run;
```
 - b.

```
proc print data=allsales.totals;
    label region8='Region 8 Yearly Totals';
run;
```
 - c.

```
proc print data allsales.totals label noobs;
run;
```
 - d.

```
proc print allsales.totals label;
run;
```
3. Which of the following statements selects from a data set only those observations for which the value of the variable Style is **RANCH**, **SPLIT**, or **TWOSTORY**?
 - a.

```
where style='RANCH' or 'SPLIT' or 'TWOSTORY';
```
 - b.

```
where style in 'RANCH' or 'SPLIT' or 'TWOSTORY';
```
 - c.

```
where style in (RANCH, SPLIT, TWOSTORY);
```
 - d.

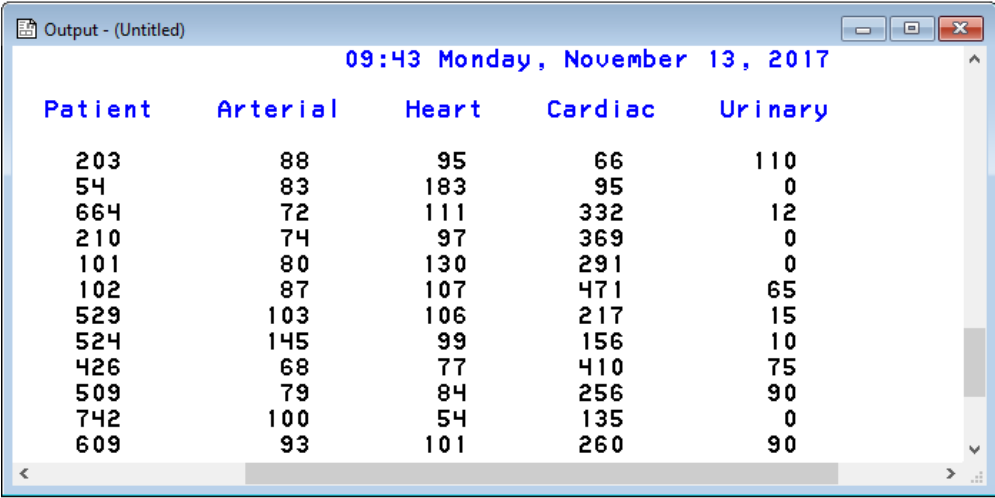
```
where style in ('RANCH','SPLIT','TWOSTORY');
```
4. If you want to sort your data and create a temporary data set named Calc to store the sorted data, which of the following steps should you submit?
 - a.

```
proc sort data=work.calc out=finance.dividend;
run;
```
 - b.

```
proc sort dividend out=calc;
    by account;
run;
```
 - c.

```
proc sort data=finance.dividend out=work.calc;
    by account;
run;
```
 - d.

```
proc sort from finance.dividend to calc;
    by account;
run;
```
5. Which options are used to create the following PROC PRINT output?



Output - (Untitled) 09:43 Monday, November 13, 2017

Patient	Arterial	Heart	Cardiac	Urinary
203	88	95	66	110
54	83	183	95	0
664	72	111	332	12
210	74	97	369	0
101	80	130	291	0
102	87	107	471	65
529	103	106	217	15
524	145	99	156	10
426	68	77	410	75
509	79	84	256	90
742	100	54	135	0
609	93	101	260	90

- the DATE system option and the LABEL option in PROC PRINT
 - the DATE and NONUMBER system options and the DOUBLE and NOOBS options in PROC PRINT
 - the DATE and NONUMBER system options and the DOUBLE option in PROC PRINT
 - the DATE and NONUMBER system options and the NOOBS option in PROC PRINT
6. Which of the following statements can you use in a PROC PRINT step to create this output?

Month	Instructors	AerClass	WalkJogRun	Swim
1	1	37	91	83
2	2	41	102	27
3	1	52	98	19
4	1	61	118	22
5	3	49	88	29
6	2	24	101	54
7	1	45	91	69
8	2	63	65	53
9	1	60	49	68
10	1	78	70	41
11	3	82	44	58
12	2	93	57	47
	20	685	974	570

- ```
var month instructors;
sum instructors aerclass walkjogrun swim;
```
- ```
var month;
sum instructors aerclass walkjogrun swim;
```

- c. `var month instructors aerclass;`
`sum instructors aerclass walkjogrun swim;`

d. all of the above

7. What happens if you submit the following program?

```
proc sort data=clinic.diabetes;
run;
proc print data=clinic.diabetes;
  var age height weight pulse;
  where sex='F';
run;
```

- a. The PROC PRINT step runs successfully, printing observations in their sorted order.
- b. The PROC SORT step permanently sorts the input data set.
- c. The PROC SORT step generates errors and stops processing, but the PROC PRINT step runs successfully, printing observations in their original (unsorted) order.
- d. The PROC SORT step runs successfully, but the PROC PRINT step generates errors and stops processing.

8. If you submit the following program, which output does it create?

```
proc sort data=sasuser.loans out=work.loans;
  by months amount;
run;
proc print data=work.loans noobs;
  var months;
  sum amount payment;
  where months<360;
run;
```

a.

Months	Amount	Payment
12	\$3,500	\$308.52
24	\$8,700	\$403.47
36	\$10,000	\$325.02
48	\$5,000	\$128.02
60	\$18,500	\$393.07
60	\$22,000	\$467.43
	\$67,700	\$2,025.53

b.

Months	Amount	Payment
12	\$3,500	\$308.52
24	\$8,700	\$403.47
36	\$10,000	\$325.02
48	\$5,000	\$128.02
60	\$18,500	\$393.07
60	\$22,000	\$467.43
	67,700	2,025.53

c.

Months	Amount	Payment
12	\$3,500	\$308.52
48	\$5,000	\$128.02
60	\$18,500	\$393.07
24	\$8,700	\$403.47
36	\$10,000	\$325.02
60	\$22,000	\$467.43
	\$67,700	\$2,025.53

d.

Months	Amount	Payment
12	\$3,500	\$308.52
24	\$8,700	\$403.47
36	\$10,000	\$325.02
48	\$5,000	\$128.02
60	\$18,500	\$393.07
60	\$22,000	\$467.43
		\$2,025.53

9. Which statement below selects rows that satisfy both these conditions?

- The amount is less than or equal to \$5000.
- The account is 101-1092 or the rate equals 0.095.

- a. where amount <= 5000 and
account='101-1092' or rate = 0.095;
- b. where (amount le 5000 and account='101-1092')
or rate = 0.095;
- c. where amount <= 5000 and
(account='101-1092' or rate eq 0.095);
- d. where amount <= 5000 or account='101-1092'
and rate = 0.095;

10. What does PROC PRINT display by default?

- a. PROC PRINT does not create a default report; you must specify the rows and columns to be displayed.
- b. PROC PRINT displays all observations and variables in the data set. If you want an additional column for observation numbers, you can request it.
- c. PROC PRINT displays columns in the following order: a column for observation numbers, all character variables, and all numeric variables.
- d. PROC PRINT displays all observations and variables in the data set, a column for observation numbers on the far left, and variables in the order in which they occur in the data set.

Chapter 6

Creating SAS Data Sets from External Files

Raw Data Files	100
Creating a SAS Data Set from a Raw Data File	101
Referencing a Raw Data File	102
Using a FILENAME Statement	102
Defining a Fully Qualified Filename	102
Referencing a Fully Qualified Filename	103
Defining an Aggregate Storage Location	103
Referencing a File in an Aggregate Storage Location	103
DATA Step Statements for Reading Data	104
Naming the Data Set with the DATA Statement	104
Specifying the Raw Data File with the INFILE Statement	104
Column Input	105
Standard and Nonstandard Numeric Data	105
Describing the Data with the INPUT Statement	106
Specifying Variable Names	107
Reading and Verifying the Data	108
Verifying the Code That Reads the Data	108
Checking DATA Step Processing	108
Printing the Data Set	109
Reading the Entire Raw Data File	109
Invalid Data	109
Creating and Modifying Variables	111
Assignment Statements	111
SAS Expressions	112
Using Operators in SAS Expressions	112
Examples: Assignment Statements	113
Date Constants	115
Example: Assignment Statement and Date Value	115
Subsetting Data	115
Using a Subsetting IF Statement	115
Example: Subsetting IF Statement	116
Reading Instream Data	116
Overview	116
Example: Reading Instream Data	117
Creating a Raw Data File	118
Overview	118

Using the <code>_NULL_</code> Keyword	118
Specifying the Raw Data File	118
Describing the Data	119
Reading Microsoft Excel Data	121
Decide How to Reference Your Data	121
Running SAS with Microsoft Excel	122
Steps for Reading Excel Data	122
The SAS/ACCESS LIBNAME Statement	123
Referencing an Excel Workbook	123
Referencing an Excel Workbook in the DATA Step	125
Printing an Excel Worksheet as a SAS Data Set	126
Using PROC CONTENTS	127
Disassociating a Libref	128
LIBNAME Statement Options	129
Creating Excel Worksheets	130
The IMPORT Procedure	130
The Basics of PROC IMPORT	130
The PROC IMPORT Statement	131
Example: Importing a Comma-Delimited File with a CSV Extension	133
Chapter Quiz	134

Raw Data Files

A *raw data file* is an external text file whose records contain data values that are organized in fields. Raw data files are non-proprietary and can be read by a variety of software programs. The sample raw data files in this book are shown with a ruler to help you identify where individual fields begin and end. The ruler is not part of the raw data file.

Figure 6.1 Raw Data File

Raw Data File	
Ruler →	>-----+-----10-----+-----20
Data Organized in Fields →	2810 61 MOD F 2804 38 HIGH F 2807 42 LOW M 2816 26 HIGH M 2833 32 MOD F 2823 29 HIGH M

The table below describes the record layout for a raw data file that contains readings from exercise stress tests that have been performed on patients at a health clinic. Exercise physiologists in the clinic use the test results to prescribe various exercise therapies. The file contains fixed fields. That is, values for each variable are in the same location in all records.

Table 6.1 Record Layout for Raw Data

Field Name	Starting Column	Ending Column	Description of Field
ID	1	4	patient ID number
Name	6	25	patient name
RestHR	27	29	resting heart rate
MaxHR	31	33	maximum heart rate during test
RecHR	35	37	recovery heart rate after test
TimeMin	39	40	time, complete minutes
TimeSec	42	43	time, seconds
Tolerance	45	45	comparison of stress test tolerance between this test and the last test (I=increased, D=decreased, S=same, N=no previous test)

Creating a SAS Data Set from a Raw Data File

To read the raw data file, SAS must receive the following information in the DATA step:

- the location or name of the external text file
- a name for the new SAS data set
- a reference that identifies the external file
- a description of the data values to be read

After using the DATA step to read the raw data, you can use a PROC PRINT step to produce a report that displays the data values that are in the new data set.

The table below outlines the basic statements that are used in a program that reads raw data in fixed fields.

Table 6.2 Statements for Reading Raw Data

Task	SAS Statement
Reference a SAS library	LIBNAME statement
Reference an external file	FILENAME statement
Name a SAS data set	DATA statement
Identify an external file	INFILE statement
Describe data	INPUT statement
Execute the DATA step	RUN statement
Display the data set	PROC PRINT statement
Execute the final program step	RUN statement

Referencing a Raw Data File

Using a FILENAME Statement

Use the FILENAME statement to point to the location of the external file that contains the data.

Filerefs perform the same function as librefs: they temporarily point to a storage location for data. However, librefs reference SAS libraries, whereas filerefs reference external files.

Syntax, FILENAME statement:

FILENAME *fileref* '*filename*';

- *fileref* is a name that you associate with an external file. The name must be 1 to 8 characters long, begin with a letter or underscore, and contain only letters, numbers, or underscores.
 - '*filename*' is the fully qualified name or location of the file.
-

Defining a Fully Qualified Filename

The following FILENAME statement temporarily associates the fileref `Exercise` with the external file that contains the data from the exercise stress tests. The complete filename is specified as `C:\certdata\Tmill.dat` in the Windows operating environment.

```
filename exercise 'c:\certdata\tmill.dat';
```


Figure 6.2 File Location

1	10	20	30	40	
2458	Murray, W	72	185	128	12 38 D
2462	Almers, C	68	171	133	10 5 I
2501	Bonaventure, T	78	177	139	11 13 I
2523	Johnson, R	69	162	114	9 42 S
2539	LaMance, K	75	168	141	11 46 D
2552	Reberson, P	69	158	139	15 41 D

Referencing a Fully Qualified Filename

When you associate a fileref with an individual external file, you specify the fileref in subsequent SAS statements and commands.

Figure 6.3 Referencing a Fully Qualified Filename

fileref Exercise

single
external
file

```
data clinic.therapy;
  infile exercise;
  input Name $ City $
        Age Sex Height;
run;
```

Defining an Aggregate Storage Location

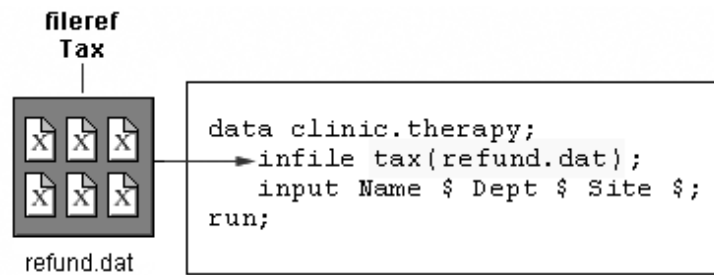
An aggregate storage location contains multiple external files. To define an aggregate storage location, use the FILENAME statement to associate a fileref.

```
filename tax 'c:\clinic\revenue';
```

This FILENAME statement temporarily associates the fileref Tax with the aggregate storage directory C:\clinic\revenue.

Referencing a File in an Aggregate Storage Location

To reference an external file with a fileref that points to an aggregate storage location, specify the fileref followed by the individual filename in parentheses.

Figure 6.4 Referencing a File in an Aggregate Storage Location

TIP If the file extension is omitted, SAS looks for a .dat file.

DATA Step Statements for Reading Data

Naming the Data Set with the DATA Statement

The DATA statement indicates the beginning of the DATA step and names the SAS data set to be created.

Syntax, DATA statement:

DATA *SAS-data-set-1* <...*SAS-data-set-n*>;

SAS-data-set names (in the format *libref.filename*) the data set or data sets to be created.

Remember that a permanent SAS data set name is a two-level name. For example, the two-level name Clinic.Admit specifies that the data set Admit is stored in the permanent SAS library to which the libref Clinic has been assigned.

Specifying the Raw Data File with the INFILE Statement

When reading raw data, use the INFILE statement to indicate which file the data is in.

Syntax, INFILE statement:

INFILE *file-specification* <*options*>;

- *file-specification* can take the form *fileref* to name a previously defined file reference or '*filename*' to point to the actual name and location of the file.
- *options* describes the input file's characteristics and specifies how it is to be read with the INFILE statement.

To read the raw data file to which the fileref Tests has been assigned, you write the following INFILE statement:

```
infile tests;
```

TIP Instead of using a FILENAME statement, you can identify the raw data file by specifying the entire filename and location in the INFILE statement. For example, the following statement points directly to the C:\Irs\Personal\Refund.dat file:

```
infile 'c:\irs\personal\refund.dat';
```

TIP When creating a temporary data set in the Work library, it is permissible to specify only the data set name and omit the Work library name.

Column Input

Using column input is one of the several methods for reading data. Column input specifies actual column locations for values. However, column input is appropriate only in certain situations. When you use column input, your data must meet these conditions:

- It must contain only standard character or numeric values.
- It must be arranged in fixed fields.

The following external file contains data that is arranged in columns or fixed fields. You can specify a beginning and ending column for each field.

Figure 6.5 External File with Columns

1 --- + --- 10 --- + --- 20			
2810	61	MOD	F
2804	38	HIGH	F
2807	42	LOW	M
2816	26	HIGH	M
2833	32	MOD	F
2823	29	HIGH	M

Standard and Nonstandard Numeric Data

Standard numeric data values can contain only the following text and characters:

- numbers
- decimal points
- numbers in scientific or E notation (2.3E4, for example)
- plus or minus signs

Nonstandard numeric data that includes the following text and characters cannot be read by column input:

- values that contain special characters, such as percent signs (%), dollar signs (\$), and commas (,)
- date and time values
- data in fraction, integer binary, real binary, and hexadecimal forms

The following external file contains the personnel information for a technical writing department of a small computer manufacturer. The fields contain values for each employee's last name, first name, job code, and annual salary.

Notice that the salary values contain commas. So, the salary values are considered to be nonstandard numeric values. You cannot use column input to read these values.

Figure 6.6 Raw Data File

Raw Data File Staff			
1 --- + --- 10 --- + --- 20 --- + ---			
EVANS	DONNY	112	29,996.63
HELMS	LISA	105	18,567.23
HIGGINS	JOHN	111	25,309.00
LARSON	AMY	113	32,696.78
MOORE	MARY	112	28,945.89

Describing the Data with the INPUT Statement

The INPUT statement describes the fields of raw data to be read and placed into the SAS data set.

Syntax, INPUT statement using column input:

INPUT *variable* <\$> *startcol-endcol* . . . ;

- *variable* is the SAS name that you assign to the field.
 - The dollar sign (\$) identifies the variable type as character (if the variable is numeric, then nothing appears here).
 - *startcol* represents the starting column for this variable.
 - *endcol* represents the ending column for this variable.
-

Here is a small data file.

Figure 6.7 Raw Data File

1 --- + --- 10 --- + --- 20			
2810	61	MOD	F
2804	38	HIGH	F
2807	42	LOW	M
2816	26	HIGH	M
2833	32	MOD	F
2823	29	HIGH	M

The INPUT statement below assigns the character variable ID to the data in columns 1-4, the numeric variable Age to the data in columns 6-7, the character variable ActLevel to the data in columns 9-12, and the character variable Sex to the data in column 14.

```
filename exer 'Z:\sasuser\exer.dat';
data exercise;
  infile exer;
  input ID $ 1-4 Age 6-7 ActLevel $ 9-12 Sex $ 14;
run;
proc print data=exercise;
run;
```

Figure 6.8 Assigning Column Ranges to Variables

Obs	ID	Age	ActLevel	Sex
1	2810	61	MOD	F
2	2804	38	HIGH	F
3	2807	42	LOW	M
4	2816	26	HIGH	M
5	2833	32	MOD	F
6	2823	29	HIGH	M

When you use column input, you can do the following:

- read any or all fields from the raw data file
- read the fields in any order
- specify only the starting column for values that occupy only one column

```
input ActLevel $ 9-12 Sex $ 14 Age 6-7;
```

TIP Remember that when you name a new variable, you must specify the name in the exact case that you want it stored (for example, NewBalance). Thereafter, you can specify the name in uppercase, lowercase, or mixed case letters.

Specifying Variable Names

Each variable has a name that conforms to SAS naming conventions. Variable names follow these rules for naming:

- They can be 1 to 32 characters long.
- They must begin with a letter (A-Z, either uppercase or lowercase) or an underscore (_)
- They can continue with any combination of numbers, letters, or underscores.

Note: It is a best practice to restrict variable names with the global option `VALIDVARNAME=v7` and to follow the variable naming rules.

The INPUT statement uses column input to read the three data field in the raw data file below.

Figure 6.9 Raw Data File

Raw Data File Admit	
1---	10---+-----20
58MOD M	
29LOW F	
34LOW M	
41HIGHF	
30MOD F	
22HIGHM	

The values for the variable Age are located in columns 1-2. Because Age is a numeric variable, you do not specify a dollar sign (\$) after the variable name.

```
input Age 1-2
```

The values for the variable ActLevel are located in columns 3-6. You specify a \$ to indicate that ActLevel is a character variable.

```
input Age 1-2 ActLevel $ 3-6
```

The values for the character variable Sex are located in column 7. Notice that you specify only a single column.

```
input Age 1-2 ActLevel $ 3-6 Sex $ 7;
```

Reading and Verifying the Data

Verifying the Code That Reads the Data

Before you read a complete external file, you can verify the code that reads the data by limiting the number of observations that SAS reads. Adding `OBS=n` to the `INFILE` statement enables you to process records only 1 through *n*, so you can verify that the correct fields are read before reading the entire data file.

The program below reads the first 10 records in the raw data file that is referenced by the fileref `Tests`. The data is stored in a permanent SAS data set, named `sasuser.stress`. The `RUN` statement tells SAS to execute the previous SAS statements.

```
data sasuser.stress;
  infile tests obs=10;
  input ID $ 1-4 Name $ 6-25
        RestHR 27-29 MaxHR 31-33
        RecHR 35-37 TimeMin 39-40
        TimeSec 42-43 Tolerance $ 45;
run;
```

Checking DATA Step Processing

Messages in the log verify that the raw data file was read correctly. The notes in the log indicate the following:

- 10 records were read from the raw data file.
- The SAS data set `sasuser.stress` was created with 10 observations and 8 variables.

Log 6.1 SAS Log

```
NOTE: The infile TESTS is:
      Filename=Z:\sasuser\tests.dat,
      RECFM=V,LRECL=32767,File Size (bytes)=1722,
      Last Modified=02Feb2017:13:50:22,
      Create Time=19Dec2016:12:49:09

NOTE: 10 records were read from the infile TESTS.
      The minimum record length was 80.
      The maximum record length was 80.

NOTE: The data set SASUSER.STRESS has 10 observations
      and 8 variables.

NOTE: DATA statement used 0.07 seconds
```

Printing the Data Set

The messages in the log seem to indicate that the DATA step program correctly accessed the raw data file. But it is a good idea to look at the 10 observations in the new data set before reading the entire raw data file. You can submit a PROC PRINT step to view the data.

The following PROC PRINT step prints the Sasuser.Stress data set.

```
proc print data=sasuser.stress;
run;
```

The PROC PRINT output indicates that the variables in the Sasuser.Stress data set were read correctly for the first 10 records.

Figure 6.10 PROC Print Output

Obs	ID	Name	RestHR	MaxHR	RecHR	TimeMin	TimeSec	Tolerance
1	2458	Murray, W	72	185	128	12	38	D
2	2462	Almers, C	68	171	133	10	5	I
3	2501	Bonaventure, T	78	177	139	11	13	I
4	2523	Johnson, R	69	162	114	9	42	S
5	2539	LaMance, K	75	168	141	11	46	D
6	2544	Jones, M	79	187	136	12	26	N
7	2552	Reberson, P	69	158	139	15	41	D
8	2555	King, E	70	167	122	13	13	I
9	2563	Pitts, D	71	159	116	10	22	S
10	2568	Eberhardt, S	72	182	122	16	49	N

Reading the Entire Raw Data File

To modify the DATA step to read the entire raw data file, remove the OBS= option from the INFILE statement and resubmit the program.

```
data sasuser.stress;
  infile tests;
  input ID $ 1-4 Name $ 6-25
        RestHR 27-29 MaxHR 31-33
        RecHR 35-37 TimeMin 39-40
        TimeSec 42-43 Tolerance $ 45;
run;
```

Invalid Data

The log includes a note indicating that invalid data appears for the variable RecHR in line 14 of the raw data file, columns 35-37.

This note is followed by a column ruler and the actual data line that contains the invalid value for RecHR.

Output 6.1 SAS Log

```

NOTE: Invalid data for RecHR in line 14 35-37.
RULE:      +---+---1---+---2---+---3---+---4---+---5---+---6---
+---7---+---8---+---
14          2575 Quigley, M          74  152 Q13 11 26
I              80
ID=2575 Name=Quigley, M RestHR=74 MaxHR=152 RecHR=. TimeMin=11 TimeSec=26
Tolerance=I _ERROR_=1
_N_=14
NOTE: 21 records were read from the infile TEST.
      The minimum record length was 80.
      The maximum record length was 80.
NOTE: The data set SASUSER.STRESS has 21 observations and 8 variables.
NOTE: DATA statement used 0.13 seconds

```

The value Q13 is a data-entry error. It was entered incorrectly for the variable RecHR.

RecHR is a numeric variable, but Q13 is not a valid number. So RecHR is assigned a missing value, as indicated in the log. Because RecHR is numeric, the missing value is represented with a period (RecHR=.).

Notice, though, that the DATA step does not fail as a result of the invalid data but continues to execute. Unlike syntax errors, invalid data errors do not cause SAS to stop processing a program.

When you correct the invalid value and rerun the DATA step, the log will then show that the data set Sasuser.Stress was created with 21 observations and 8 variables. There will be no messages about invalid data.

Output 6.2 SAS Log

```

NOTE: The infile TESTS2 is:
      File Name=Z:\sasuser\tests.dat,
      RECFM=V, LRECL=256
NOTE: 21 records were read from the infile TESTS.
      The minimum record length was 80.
      The maximum record length was 80.
NOTE: The data set SASUSER.STRESS has 21 observations
      and 8 variables.
NOTE: DATA statement used 0.14 seconds

```

After correcting the raw data file, you can print the data again to verify that it is correct.

```

proc print data=sasuser.stress;
run;

```


Figure 6.11 PROC Print Output

Obs	ID	Name	RestHR	MaxHR	RecHR	TimeMin	TimeSec	Tolerance
1	2458	Murray, W	72	185	128	12	38	D
2	2462	Almers, C	68	171	133	10	5	I
3	2501	Bonaventure, T	78	177	139	11	13	I
4	2523	Johnson, R	69	162	114	9	42	S
5	2539	LaMance, K	75	168	141	11	46	D
6	2544	Jones, M	79	187	136	12	26	N
7	2552	Reberson, P	69	158	139	15	41	D
8	2555	King, E	70	167	122	13	13	I
9	2563	Pitts, D	71	159	116	10	22	S
10	2568	Eberhardt, S	72	182	122	16	49	N
11	2571	Nunnelly, A	65	181	141	15	2	I
12	2572	Oberon, M	74	177	138	12	11	D
13	2574	Peterson, V	80	164	137	14	9	D
14	2575	Quigley, M	74	152	.	11	26	I
15	2578	Cameron, L	75	158	108	14	27	I
16	2579	Underwood, K	72	165	127	13	19	S
17	2584	Takahashi, Y	76	163	135	16	7	D
18	2586	Derber, B	68	176	119	17	35	N
19	2588	Ivan, H	70	182	126	15	41	N
20	2589	Wilcox, E	78	189	138	14	57	I
21	2595	Warren, C	77	170	136	12	10	S

When you use the DATA step to read raw data, remember the steps that you followed in this chapter. These help you to avoid wasting resources when accessing data:

- write the DATA step using the OBS= option in the INFILE statement
- submit the DATA step
- check the log for messages
- view the resulting data set
- remove the OBS= option and resubmit the DATA step
- check the log again
- view the resulting data set again

Creating and Modifying Variables

Assignment Statements

Use an assignment statement in any DATA step in order to modify existing values or create new variables.

Syntax, assignment statement:

variable=*expression*;

- *variable* names a new or existing variable
- *expression* is any valid SAS expression

Tip: The assignment statement is one of the few SAS statements that do not begin with a keyword.

For example, here is an assignment statement that assigns the character value **Toby Witherspoon** to the variable Name:

Name='Toby Witherspoon';

SAS Expressions

You use SAS expressions in assignment statements and many other SAS programming statements to do the following:

- transform variables
- create new variables
- conditionally process variables
- calculate new values
- assign new values

An expression is a sequence of operands and operators that form a set of instructions. The instructions are performed to produce a new value:

- Operands are variable names or constants. They can be numeric, character, or both.
- Operators are special-character operators, grouping parentheses, or functions.

Using Operators in SAS Expressions

Use the following arithmetic operators to perform a calculation.

Table 6.3 Arithmetic Operators

Operator	Action	Example	Priority
-	negative prefix	negative=-x;	I
**	exponentiation	raise=x**y;	I
*	multiplication	mult=x*y;	II
/	division	divide=x/y;	II
+	addition	sum=x+y;	III
-	subtraction	diff=x-y;	III

The order of operation is determined by the following conditions:

- Operations of priority I are performed before operations of priority II, and so on.
- Consecutive operations that have the same priority are performed in this order:
 - from right to left within priority I
 - from left to right within priority II and III
- You can use parentheses to control the order of operations.

Note: When a value that is used with an arithmetic operator is missing, the result of the expression is missing. The assignment statement assigns a missing value to a variable if the result of the expression is missing.

Use the following comparison operators to express a condition.

Table 6.4 Comparison Operators

Operator	Meaning	Example
= or eq	equal to	name='Jones, C.'
^= or ne	not equal to	temp ne 212
> or gt	greater than	income>20000
< or lt	less than	partno lt "BG05"
>= or ge	greater than or equal to	id>='1543'
<= or le	less than or equal to	pulse le 85

Use logical operators to link a sequence of expressions into compound expressions.

Table 6.5 Logical Operators

Operator, symbol	Description
AND or &	and, both. If both expressions are true, then the compound expression is true.
OR or	or, either. If either expression is true, then the compound expression is true.

Examples: Assignment Statements

The assignment statement in the DATA step below creates a new variable, TotalTime, by multiplying the values of TimeMin by 60 and then adding the values of TimeSec.

```
data sasuser.stress;
  infile tests;
  input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
```

```

RecHR 35-37 TimeMin 39-40 TimeSec 42-43
Tolerance $ 45;
TotalTime=(timemin*60)+timesec;
run;

```

Figure 6.12 Assignment Statement Output

Obs	ID	Name	RestHR	MaxHR	RecHR	TimeMin	TimeSec	Tolerance	TotalTime
1	2458	Murray, W	72	185	128	12	38	D	758
2	2462	Almers, C	68	171	133	10	5	I	605
3	2501	Bonaventure, T	78	177	139	11	13	I	673
4	2523	Johnson, R	69	162	114	9	42	S	582
5	2539	LaMance, K	75	168	141	11	46	D	706
6	2544	Jones, M	79	187	136	12	26	N	746
7	2552	Reberson, P	69	158	139	15	41	D	941
8	2555	King, E	70	167	122	13	13	I	793
9	2563	Pitts, D	71	159	116	10	22	S	622
10	2568	Eberhardt, S	72	182	122	16	49	N	1009
11	2571	Nunnelly, A	65	181	141	15	2	I	902
12	2572	Oberon, M	74	177	138	12	11	D	731
13	2574	Peterson, V	80	164	137	14	9	D	849
14	2575	Quigley, M	74	152	.	11	26	I	686
15	2578	Cameron, L	75	158	108	14	27	I	867
16	2579	Underwood, K	72	165	127	13	19	S	799
17	2584	Takahashi, Y	76	163	135	16	7	D	967
18	2586	Derber, B	68	176	119	17	35	N	1055
19	2588	Ivan, H	70	182	126	15	41	N	941
20	2589	Wilcox, E	78	189	138	14	57	I	897
21	2595	Warren, C	77	170	136	12	10	S	730

The expression can also contain the variable name that is on the left side of the equal sign, as the following assignment statement shows. This statement redefines the values of the variable RestHR as 10% higher.

```

data sasuser.stress;
  infile tests;
  input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
        RecHR 35-37 TimeMin 39-40 TimeSec 42-43
        Tolerance $ 45;
  resthr=resthr+(resthr*.10);
run;

```

When a variable name appears on both sides of the equal sign, the original value on the right side is used to evaluate the expression. The result is assigned to the variable on the left side of the equal sign.

```

data sasuser.stress;
  infile tests;
  input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
        RecHR 35-37 TimeMin 39-40 TimeSec 42-43
        Tolerance $ 45;
  resthr=resthr+(resthr*.10);
run;

```

Date Constants

You can assign date values to variables in assignment statements by using date constants. To represent a constant in SAS date form, specify the date as *'ddmmmyy'* or *'ddmmmyyyy'*, immediately followed by a D.

Syntax, date constant:

'ddmmmyy'd

or

'ddmmmyy'd

- *dd* is a one- or two-digit value for the day.
- *mmm* is a three-letter abbreviation for the month (JAN, FEB, and so on).
- *yy* or *yyyy* is a two- or four-digit value for the year, respectively.

Tip: Be sure to enclose the date in quotation marks.

Example: Assignment Statement and Date Value

In the following program, the second assignment statement assigns a date value to the variable `TestDate`.

```
data sasuser.stress;
  infile tests;
  input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
        RecHR 35-37 TimeMin 39-40 TimeSec 42-43
        Tolerance $ 45;
  TotalTime=(timemin*60)+timesec;
  TestDate='01jan2015'd;
run;
```

TIP You can also use SAS time constants and SAS datetime constants in assignment statements.

```
Time='9:25't;

DateTime='18jan2015:9:27:05'dt;
```

Subsetting Data

As you read your data, you can subset it by processing only those observations that meet a specified condition. To do this, you can use a subsetting IF statement in any DATA step.

Using a Subsetting IF Statement

The subsetting IF statement causes the DATA step to continue processing only those observations that meet the condition of the expression specified in the IF statement. The resulting SAS data set or data sets contain a subset of the original external file or SAS data set.

Syntax, subsetting IF statement:

IF *expression*;

expression is any valid SAS expression.

- If the expression is true, the DATA step continues to process that observation.
 - If the expression is false, no further statements are processed for that observation, and control returns to the top of the DATA step.
-

Example: Subsetting IF Statement

The subsetting IF statement below selects only observations whose values for Tolerance are D. It is positioned in the DATA step for efficiency: other statements do not need to process unwanted observations.

```
data sasuser.stress;
  infile tests;
  input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
        RecHR 35-37 TimeMin 39-40 TimeSec 42-43
        Tolerance $ 45;
  if tolerance='D';
  TotalTime=(timemin*60)+timesec;
run;
```

Because Tolerance is a character variable, the value D must be enclosed in quotation marks, and it must be the same case as in the data set.

Reading Instream Data

Overview

Reading instream data is extremely helpful if you want to create data and test your programming statements on a few observations that you can specify according to your needs.

To read instream data lines that you enter directly in your SAS program, rather than data that is stored in an external file:

- Insert a DATALINES statement as the last statement in the DATA step and immediately preceding the data lines.
- Insert a null statement (a single semicolon) to indicate the end of the input data.

```
data sasuser.stress;
  input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
        RecHR 35-37 TimeMin 39-40 TimeSec 42-43
        Tolerance $ 45;
  datalines;
2458 Murray, W      72 185 128 12 38 D
2462 Almers, C      68 171 133 10 5 I
2501 Bonaventure, T 78 177 139 11 13 I
2523 Johnson, R     69 162 114 9 42 S
2539 LaMance, K     75 168 141 11 46 D
2544 Jones, M       79 187 136 12 26 N
;
```

```

2555 King, E      70 167 122 13 13 I
2563 Pitts, D    71 159 116 10 22 S
2568 Eberhardt, S 72 182 122 16 49 N
2571 Nunnelly, A  65 181 141 15  2 I
2572 Oberon, M   74 177 138 12 11 D
2574 Peterson, V  80 164 137 14  9 D
2575 Quigley, M   74 152 113 11 26 I
2578 Cameron, L   75 158 108 14 27 I
2579 Underwood, K 72 165 127 13 19 S
2584 Takahashi, Y 76 163 135 16  7 D
2586 Derber, B    68 176 119 17 35 N
2588 Ivan, H      70 182 126 15 41 N
2589 Wilcox, E    78 189 138 14 57 I
2595 Warren, C    77 170 136 12 10 S

```

i

Syntax, DATALINES statement:

DATALINES;

TIP

- You can use only one DATALINES statement in a DATA step. Use separate DATA steps to enter multiple sets of data.
- You can also use LINES as the last statement in a DATA step and immediately preceding the data lines. Both LINES and CARDS are aliases for the DATALINES statement.
- If your data contains semicolons, use the DATALINES4 statement plus a null statement that consists of four semicolons (;;;;).

Example: Reading Instream Data

To read the data for the treadmill stress tests as instream data, you can submit the following program:

```

data sasuser.stress;
  input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
        RecHR 35-37 TimeMin 39-40 TimeSec 42-43
        Tolerance $ 45;
  if tolerance='D';
  TotalTime=(timemin*60)+timesec;
  datalines;
2458 Murray, W      72  185 128 12 38 D
2462 Almers, C      68  171 133 10  5 I
2501 Bonaventure, T  78  177 139 11 13 I
2523 Johnson, R      69  162 114  9 42 S
2539 LaMance, K      75  168 141 11 46 D
2544 Jones, M        79  187 136 12 26 N
2552 Reberson, P     69  158 139 15 41 D
2555 King, E         70  167 122 13 13 I
2563 Pitts, D        71  159 116 10 22 S
2568 Eberhardt, S    72  182 122 16 49 N
2571 Nunnelly, A     65  181 141 15  2 I
2572 Oberon, M       74  177 138 12 11 D

```

```

2574 Peterson, V      80  164 137 14  9 D
2575 Quigley, M       74  152 113 11 26 I
2578 Cameron, L       75  158 108 14 27 I
2579 Underwood, K     72  165 127 13 19 S
2584 Takahashi, Y     76  163 135 16  7 D
2586 Derber, B        68  176 119 17 35 N
2588 Ivan, H          70  182 126 15 41 N
2589 Wilcox, E        78  189 138 14 57 I
2595 Warren, C        77  170 136 12 10 S
;

```

TIP Notice that you do not need a RUN statement following the null statement (the semicolon after the data lines). The DATALINES statement functions as a step boundary, so the DATA step is executed as soon as SAS encounters it.

Creating a Raw Data File

Overview

Here are a SAS program and SAS data set that appeared earlier in this chapter.

```

data sasuser.stress;
  infile tests;
  input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
        RecHR 35-37 TimeMin 39-40 TimeSec 42-43
        Tolerance $ 45;
  if tolerance='D';
  TotalTime=(timemin*60)+timesec;
run;

```

Notice that the data set has been modified with SAS statements. If you wanted to write the new observations to a raw data file, you could reverse the process that you have been following and write out the observations from a SAS data set as records or lines to a new raw data file.

Using the `_NULL_` Keyword

Because the goal of your SAS program is to create a raw data file and not a SAS data set, it is inefficient to use a data set name in the DATA statement. Instead, use the keyword `_NULL_`, which enables you to use the DATA step without actually creating a SAS data set. A SET statement specifies the SAS data set that you want to read from.

```

data _null_;
  set sasuser.stress;

```

The next step is to specify the output file.

Specifying the Raw Data File

You use the FILE and PUT statements to write the observations from a SAS data set to a raw data file, just as you used the INFILE and INPUT statements to create a SAS data set. These two sets of statements work almost identically.

When writing observations to a raw data file, use the FILE statement to specify the output file.

Syntax, FILE statement:

FILE *file-specification* <*options*> <*operating-environment-options*>;

- *file-specification* can take the form *fileref* to name a previously defined file reference or '*filename*' to point to the actual name and location of the file
 - *options* names options that are used in creating the output file
 - *operating-environment-options* names options that are specific to an operating environment.
-

For example, if you want to read the Sasuser.Stress data set and write it to a raw data file that is referenced by the fileref Newdat, you would begin your program with the following SAS statements.

```
data _null_;
    set sasuser.stress;
    file newdat;
run;
```

Instead of identifying the raw data file with a SAS fileref, you can specify the entire filename and location in the FILE statement. For example, the following FILE statement points directly to the **Z:\sasuser\tests.dat** file. Note that the path specifying the filename and location must be enclosed in quotation marks.

```
data _null_;
    set sasuser.stress;
    file 'Z:\sasuser\tests.dat';
run;
```

Describing the Data

Whereas the FILE statement specifies the output raw data file, the PUT statement describes the lines to write to the raw data file.

Syntax, PUT statement using column output:

PUT *variable startcol-endcol*...;

- *variable* is the name of the variable whose value is written.
 - *startcol* indicates where in the line to begin writing the value.
 - *endcol* indicates where in the line to end the value.
-

In general, the PUT statement mirrors the capabilities of the INPUT statement. In this case you are working with column output. Therefore, you need to specify the variable name, starting column, and ending column for each field that you want to create. Because you are creating raw text data, the dollar sign (\$) is optional.

```
data _null_;
    set sasuser.stress;
    file 'Z:\sasuser\tests.dat';
    put id $ 1-4 name 6-25 resthr 27-29 maxhr 31-33
        rechr 35-37 timemin 39-40 timesec 42-43
        tolerance 45 totaltime 47-49;
run;
```

Figure 6.13 SAS Data Set *sasuser.stress* Output with PUT Statement

Obs	ID	Name	RestHR	MaxHR	RecHR	TimeMin	TimeSec	Tolerance	TotalTime
1	2458	Murray, W	72	185	128	12	38	D	758
2	2462	Almers, C	68	171	133	10	5	I	605
3	2501	Bonaventure, T	78	177	139	11	13	I	673
4	2523	Johnson, R	69	162	114	9	42	S	582
5	2539	LaMance, K	75	168	141	11	46	D	706
6	2544	Jones, M	79	187	136	12	26	N	746
7	2552	Reberson, P	69	158	139	15	41	D	941
8	2555	King, E	70	167	122	13	13	I	793
9	2563	Pitts, D	71	159	116	10	22	S	622
10	2568	Eberhardt, S	72	182	122	16	49	N	1009
11	2571	Nunnelly, A	65	181	141	15	2	I	902
12	2572	Oberon, M	74	177	138	12	11	D	731
13	2574	Peterson, V	80	164	137	14	9	D	849
14	2575	Quigley, M	74	152	.	11	26	I	686
15	2578	Cameron, L	75	158	108	14	27	I	867
16	2579	Underwood, K	72	165	127	13	19	S	799
17	2584	Takahashi, Y	76	163	135	16	7	D	967
18	2586	Derber, B	68	176	119	17	35	N	1055
19	2588	Ivan, H	70	182	126	15	41	N	941
20	2589	Wilcox, E	78	189	138	14	57	I	897
21	2595	Warren, C	77	170	136	12	10	S	730

The resulting raw data file would look like this.

Figure 6.14 Creating a Raw Data File

1----	10----	20----	30----	40----	50----	+		
2458	Murray, W	72	185	128	12	38	D	758
2462	Almers, C	68	171	133	10	5	I	605
2501	Bonaventure, T	78	177	139	11	13	I	673
2523	Johnson, R	69	162	114	9	42	S	582
2539	LaMance, K	75	168	141	11	46	D	706
2544	Jones, M	79	187	136	12	26	N	746
2552	Reberson, P	69	158	139	15	41	D	941
2555	King, E	70	167	122	13	13	I	793
2563	Pitts, D	71	159	116	10	22	S	622
2568	Eberhardt, S	72	182	122	16	49	N	1E3
2571	Nunnelly, A	65	181	141	15	2	I	902
2572	Oberon, M	74	177	138	12	11	D	731
2574	Peterson, V	80	164	137	14	9	D	849
2575	Quigley, M	74	152	.	11	26	I	686
2578	Cameron, L	75	158	108	14	27	I	867
2579	Underwood, K	72	165	127	13	19	S	799
2584	Takahashi, Y	76	163	135	16	7	D	967
2586	Derber, B	68	176	119	17	35	N	1E3
2588	Ivan, H	70	182	126	15	41	N	941
2589	Wilcox, E	78	189	138	14	57	I	897
2595	Warren, C	77	170	136	12	10	S	730

TIP If you do not execute a FILE statement before a PUT statement in the current iteration of the DATA step, SAS writes the lines to the SAS log.

TIP SAS assigns the fileref Print to the location where procedure output is being written. If you specify **file print** in your DATA step, the PUT statement writes the lines to the procedure output location.

Reading Microsoft Excel Data

Decide How to Reference Your Data

Use the DATA step to create a SAS data set whether the input data source is a SAS data set, a raw data file, or a file from another application. The difference between reading these various types of input is in how you reference the data. You can use one of the following methods to read data:

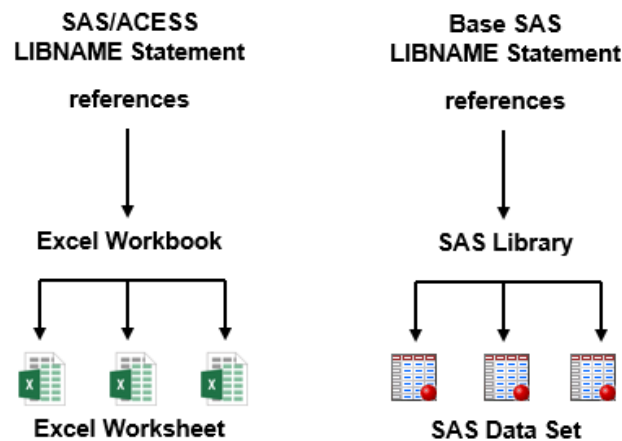
- SAS/ACCESS LIBNAME statement
- PROC IMPORT

The Base SAS LIBNAME statement associates a SAS name (libref) with a SAS data library by pointing to its physical location. But, the SAS/ACCESS LIBNAME statement associates a SAS name with an Excel workbook file by pointing to its location.

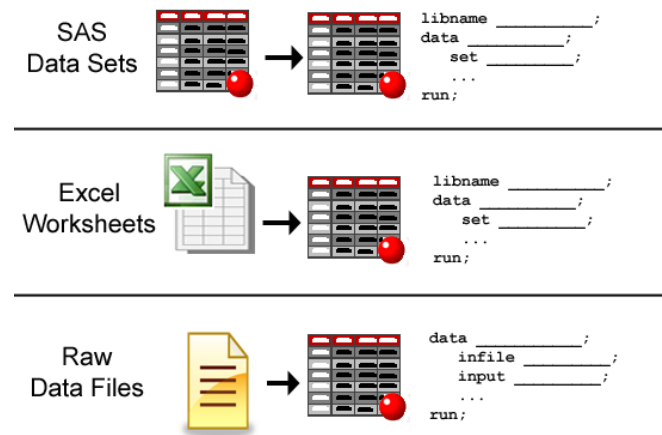
In doing so, the Excel workbook becomes a new library in SAS, and the worksheets in the workbook become the individual SAS data sets in that library.

The figure below illustrates the difference between how the two LIBNAME statements treat the data.

Figure 6.15 Comparing LIBNAME Statements



The next figure shows how the DATA step is used with three types of input data.

Figure 6.16 Using the DATA Step with Different Types of Output

Notice how the INFILE and INPUT statements are used in the DATA step for reading raw data, but the SET statement is used in the DATA step for reading in the Excel worksheets.

Running SAS with Microsoft Excel

- You must have licensed SAS/ACCESS Interface to PC Files to use a SAS/ACCESS LIBNAME statement that references an Excel workbook.
- If you are running SAS 9.1 or earlier and want to read in Microsoft Excel data, you must use Microsoft Excel 2003 or earlier.
- To read Microsoft Excel 2007 or later data, you must be running SAS 9.2 or later.
- The examples in this section are based on SAS 9.4 running with Microsoft Excel 2016.

Steps for Reading Excel Data

To read the Excel workbook file, SAS must receive the following information in the DATA step:

- a libref to reference the Excel workbook to be read
- the name and location (using another libref) of the new SAS data set
- the name of the Excel worksheet that is to be read

The table below outlines the basic statements that are used in a program that reads Excel data and creates a SAS data set from an Excel worksheet. The PROC CONTENTS and PROC PRINT statements are not requirements for reading in Excel data and creating a SAS data set. However, these statements are useful for confirming that your Excel data has successfully been read into SAS.

Table 6.6 Basic Steps for Reading Excel Data into a SAS Data Set

Task	Statement	Example
Reference an Excel workbook file	SAS/ACCESS LIBNAME statement	LIBNAME results 'c:\certdata\exercise.xlsx';

Task	Statement	Example
Write out the contents of the SAS Library	PROC CONTENTS	<code>proc contents data=results._all_;</code>
Execute the PROC CONTENTS statement	RUN statement	<code>run;</code>
Name and create a new SAS data set	DATA statement	<code>data work.stress;</code>
Read in an Excel worksheet (as the input data for the new SAS data set)	SET statement	<code>set results.'ActLevel\$'n;</code>
Execute the DATA step	RUN statement	<code>run;</code>
View the contents of a particular data set	PROC PRINT	<code>proc print data=stress;</code>
Execute the PROC PRINT statement	RUN statement	<code>run;</code>

The SAS/ACCESS LIBNAME Statement

To extend the SAS global LIBNAME statement and assign a libref to a relational DBMS, use the SAS/ACCESS LIBNAME statement. LIBNAME associates a SAS libref with a DBMS database, schema, server, or a group of tables and views.

Syntax, SAS/ACCESS LIBNAME statement:

LIBNAME <libref>XLSX <'physical-path-and-filename.xls'><options>;

- *libref* is a name that you associate with an Excel workbook.
- *XLSX* is the name of the SAS/ACCESS engine.
- *'physical-path-and-filename.xls'* is the physical location of the Excel workbook.

Example:

`libname results XLSX 'c:\certdata\exercise.xlsx';`

Note: The EXCEL engine (also called the XLSX engine) requires single quotation marks for *physical-path-and-filename.xls*.

Referencing an Excel Workbook

Overview

This example uses data similar to the scenario used for the raw data in the previous section. The data shows the readings from exercise stress tests that have been performed on patients at a health clinic.

The stress test data is located in an Excel workbook named `exercise.xlsx` (shown below), which is stored in the location `c:\users`.

Figure 6.17 Excel Workbook

Worksheet: tests

	A	B	C	D	E	F	G	H	I
	ID	Name	RestHR	MaxHR	RecHR	TimeMin	TimeSec	Tolerance	TestDate
2	2458	Murray, W	72	185	128	12	38	D	8/25/2008
3	2462	Almers, C	68	171	133	10	5	I	6/26/2008
4	2501	Bonaventure, T	78	177	139	11	13	I	6/26/2008
5	2523	Johnson, R	69	162	114	9	42	S	7/4/2008
6	2539	LaMance, R	75	168	141	11	46	D	8/25/2008
7	2544	Jones, M	79	187	136	12	26	N	7/14/2008
8	2552	Reberson, P	69	158	139	15	41	D	8/25/2008
9	2555	King, E	70	167	122	13	13	I	7/14/2008
10	2563	Pitts, D	71	159	116	10	22	S	8/25/2008
11	2568	Eberhardt, S	72	182	122	16	49	N	6/26/2008
12	2571	Nunnelly, A	65	181	141	15	2	I	8/9/2008
13	2572	Oberon, M	74	177	138	12	11	D	8/8/2008
14	2574	Peterson, V	80	164	137	14	9	D	7/21/2008
15	2575	Quigley, M	74	152		11	26	I	7/13/2008
16	2578	Cameron, L	75	158	108	14	27	I	8/16/2008
17	2579	Underwood, K	72	165	127	13	19	S	6/27/2008
18	2584	Takahashi, Y	76	163	135	16	7	D	8/16/2008
19	2586	Derber, B	68	176	119	17	35	N	8/17/2008
20	2588	Ivan, H	70	182	126	15	41	N	6/18/2008
21	2589	Wilcox, E	78	189	138	14	57	I	7/19/2008
22	2595	Warren, C	77	170	136	12	10	S	7/20/2008

Ready | Worksheets | Cells formatted as dates

In the sample worksheet above, the date column is defined in Excel as dates. If you right-click the cells and select **Format Cells**, the cells have a category of Date. SAS reads this data just as it is stored in Excel. If the date had been stored as text in Excel, then SAS would have read it as a character string.

To read in this workbook, create a libref to point to the workbook's location:

```
libname results XLSX 'c:\certdata\exercise.xlsx';
```

The LIBNAME statement creates the libref Results, which points to the Excel workbook `exercise.xlsx`. The workbook contains two worksheets, Tests and Adv, which are now available in the new SAS library (Results) as data sets.

Referencing an Excel Workbook in the DATA Step

SET Statement

Use the SET statement to indicate which worksheet in the Excel file you want to read.

```
data work.stress;
    set results.'ActivityLevels'n;
run;
```

In this example, the DATA statement tells SAS to name the new data set, Stress, and store it in the temporary library Work. The SET statement in the DATA step specifies the libref (the reference to the Excel file) and the worksheet name as the input data.

You can use several statements in the DATA step to subset your data as needed. Here, the WHERE statement is used with a variable to include only those participants whose activity level is HIGH.

```
data work.stress;
    set results.'ActivityLevels'n;
    where ActLevel='HIGH';
run;
```

The figure below shows the partial output for this DATA step in table format.

Figure 6.18 DATA Step Output (partial output)

Label changes column heading
↓

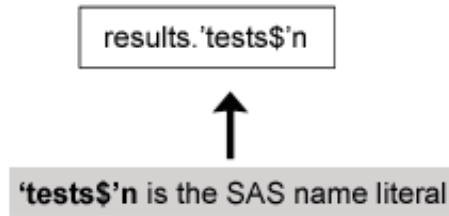
	ID	Age	ActLevel	Sex
1	2462	38	HIGH	F
2	2523	26	HIGH	M
3	2544	29	HIGH	M
4	2568	28	HIGH	M

↑
WHERE statement subsets data to only HIGH

Name Literals

The LIBNAME statement created a permanent library, Results, which is the libref for the workbook file and its location. The new library contains two SAS data sets, which access the data from the Excel worksheets.

The Excel worksheet names have the special character (\$) at the end. All Excel worksheets are designated this way. But remember, special characters such as these are not allowed in SAS data set names by default. So, in order for SAS to allow this character to be included in the data set name, you must assign a name literal to the data set name. A SAS *name literal* is a name token that is expressed as a string within quotation marks, followed by the uppercase or lowercase letter *n*. The name literal tells SAS to allow the special character (\$) in the data set name.

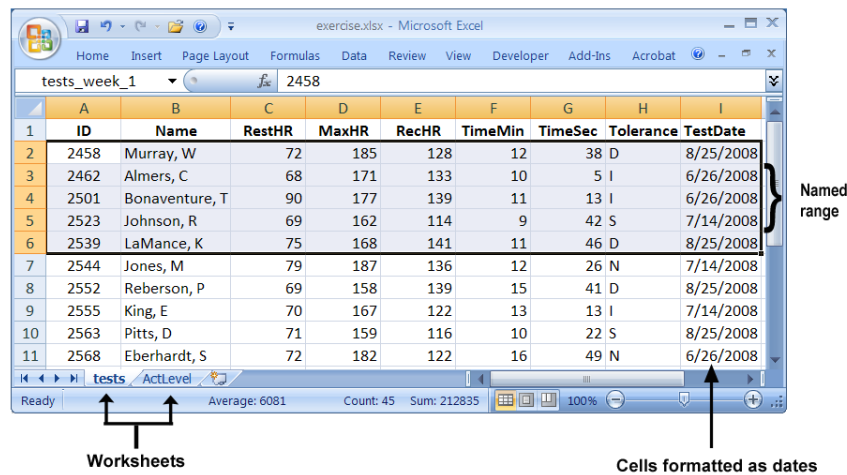
Figure 6.19 Name Literal

Named Ranges

A *named range* is a range of cells within a worksheet that you define in Excel and assign a name to. In the example below, the worksheet contains a named range, `tests_week_1`, which SAS recognizes as a data set.

The named range, `tests_week_1`, and its parent worksheet, `tests`, appear in the SAS windowing environment as separate data sets, except that the data set that was created from the named range has no dollar sign (\$) appended to its name.

For more information about named ranges, see your Microsoft Excel documentation.

Figure 6.20 Named Range

TIP Named ranges are available as data sets even though they do not appear in PROC CONTENTS.

Printing an Excel Worksheet as a SAS Data Set

After using the DATA step to read in the Excel data and create a SAS data set, you can use PROC PRINT to produce a report that displays the data set values.

You can also use the PRINT procedure to refer to a specific worksheet. Remember to use the name literal when referring to a specific Excel worksheet. In the example below, the first PRINT statement displays the data values for the new data set that was created in

the DATA step. The second PRINT statement displays the contents of the Excel worksheet that was referenced by the LIBNAME statement.

```
proc print data=work.stress;
run;
proc print data=results.ActivityLevels'n;
run;
```

Using PROC CONTENTS

In the example below, PROC CONTENTS displays a description of the SAS data set. The example also includes a LABEL= data set option that gives a label for the entire data set while the LABEL statement assigns labels to individual labels. The INFORMAT and FORMAT statements associate informats or formats with variables. When an INFORMAT or FORMAT statement is specified in a DATA step, the name of that statement can be used in PROC steps to customize your report.

```
data funnies (LABEL= 'Comics Character Data');
  input Id Name $ Height Weight DoB MMDDYY8. @@;
  Label Id = 'Identification no.';
      Height = 'Height in inches';
      Weight = 'Weight in pounds';
      DoB = 'Date of Birth';
  INFORMAT DoB MMDDYY8.;
  FORMAT DoB WORDDATE18.;
Datalines;
53 Susie 42 41 07-11-93
54 Charlie 46 55 10-26-90
55 Calvin 40 35 01-10-91
56 Lucy 46 52 01-13-95
;
proc contents data=funnies;
run;
```

TIP Use `_ALL_` keyword to produce information about a data library and its contents.

Figure 6.21 CONTENTS Procedure Output

The SAS System			
The CONTENTS Procedure			
Data Set Name	WORK.FUNNIES	Observations	4
Member Type	DATA	Variables	5
Engine	V9	Indexes	0
Created	05/22/2017 14:04:56	Observation Length	40
Last Modified	05/22/2017 14:04:56	Deleted Observations	0
Protection		Compressed	NO
Data Set Type		Sorted	NO
Label	Comics Character Data		
Data Representation	WINDOWS_64		
Encoding	wlatin1 Western (Windows)		

Alphabetic List of Variables and Attributes						
#	Variable	Type	Len	Format	Informat	Label
5	DoB	Num	8	WORDDATE18.	MMDDYY8.	
3	Height	Num	8			
1	Id	Num	8			Identification no.
2	Name	Char	8			
4	Weight	Num	8			

Note: This example demonstrates reading data that is not aligned in columns. To learn about reading data that is not aligned, see [Chapter 18, “Reading and Creating Raw Data in Free-Format,”](#) on page 417.

Disassociating a Libref

If SAS has assigned a libref to an Excel workbook, the workbook cannot be opened in Excel. To disassociate a libref, use a LIBNAME statement, specify the libref, and use the CLEAR option.

```
libname results XLSX "c:\users\exercise.xlsx";
proc print data=results.'tests'n;
run;
```

```
libname results clear;
```

SAS disconnects from the data source and closes any resources that are associated with that libref's connection.

LIBNAME Statement Options

To associate a SAS libref with a Microsoft Excel workbook, use one of the following LIBNAME statements.

LIBNAME <libref><XLSX> <“*physical-path-and-filename.xlsx*”> <SAS/ACCESS
LIBNAME-options>;

LIBNAME <libref> <CLEAR> | <_ALL_>;

LIBNAME <libref> <LIST> | <_ALL_>;

Note: The XLSX engine accepts either single or double quotation marks for *physical-path-and-filename.xlsx*.

<libref>

is any SAS name that associates SAS with the SAS library where the Microsoft Excel spreadsheet is stored. The association between a libref and a SAS library lasts only for the duration of the SAS session or until you change the libref or discontinue it with another LIBNAME statement.

XLSX

is the SAS LIBNAME engine name for an XLSX file format. The LIBNAME statement associates a libref with an XLSX engine that supports connections to Microsoft Excel 2007, 2010, and later files.

This engine name XLSX is required. Otherwise, SAS/ACCESS assumes the use of the LIBNAME engine for Microsoft Excel for XLSX files.

Note: The XLSX engine enables you to read and write XLSX data directly across UNIX, Linux, and Windows operating environments. If you have stored XLSX files on UNIX or Linux, you do not need to use the SAS PC Files Server (or the PCFILES LIBNAME engine) to access the Excel (.xlsx) files. If you have SAS running on UNIX or Linux and your XLSX files reside remotely in the Windows operating environment, use the PCFILES LIBNAME engine to access them.

The XLSX engine enables you to replace an existing worksheet or to add a new worksheet, but it cannot be used to update the values in individual worksheets.

When importing (reading) XLSX data, the XLSX engine reads mixed data (that is, columns containing numeric and character values) and converts it to character data values.

The XLSX engine allows the sequential reading of data only. That is, it does not support random access. It does not support certain tasks that require random access, such as the RANK procedure, which requires the reading of rows in a random order.

Example This LIBNAME statement specifies Wsinv as a reference (that is, libref) to a SAS library. The XLSX engine specifies an engine that supports the connection to the file type .XLSX.

```
LIBNAME wsinv XLSX 'c:\WestSector\Q1_Invoices.xlsx';
```

<“*physical-path-and-filename.xlsx*”>

is the physical-path and filename of a Microsoft Excel 2007 or later workbook. Using the extension (.xlsx) is required. Enclose the path and workbook name in quotation marks.

<SAS/ACCESS LIBNAME-options>

define how SAS interacts with the external data source (in this case, XLSX files), providing enhanced control of how SAS processes data source objects.

Because the XLSX LIBNAME engine works on any version of Microsoft Excel 2007 or later, it has no engine-specific LIBNAME options, such as VERSION=.

<CLEAR>

clears one libref.

Specify *libref* to disassociate a single libref.

<_ALL_>

specifies that the CLEAR or LIST argument applies to all librefs.

<LIST>

writes the attributes of one or more SAS/ACCESS libraries or SAS libraries to the SAS log.

Specify *libref* to list the attributes of a single SAS/ACCESS library or SAS library.

Specify *_ALL_* to list the attributes of all librefs in your current session.

Examples List the attributes of a single library:

```
LIBNAME LIBREF LIST;
```

List the attributes of all the libraries:

```
LIBNAME _ALL_ LIST;
```

Creating Excel Worksheets

In addition reading Excel data, SAS can also create Excel worksheets from SAS data sets.

To do this, you use the SAS/ACCESS LIBNAME statement. For example, to create a new worksheet named *high_stress* from the temporary SAS data set *work.high_stress* and save this worksheet in the new Excel file *newExcel.xlsx*, you would submit the following LIBNAME statement and DATA step:

```
libname clinic xlsx 'c:\Users\mylaptop\newExcel.xlsx';
data clinic.high_stress;
    set high_stress;
run;
```

The IMPORT Procedure

The Basics of PROC IMPORT

The IMPORT procedure reads data from an external data source and writes it to a SAS data set.

In delimited files, a delimiter (such as a blank, comma, or tab) separates columns of data values. If you license SAS/ACCESS Interface to PC Files, additional external data sources can include Microsoft Access database files, Microsoft Excel files, and Lotus spreadsheets.

When you run the IMPORT procedure, it reads the input file and writes the data to the specified SAS data set. By default, IMPORT procedure expects the variable names to appear in the first row. The procedure scans the first 20 rows to count the variables, and it attempts to determine the correct informat and format for each variable. You can use the IMPORT procedure statements to do the following:

- indicate how many rows SAS scans for variables to determine the type and length (GUESSINGROWS=)
- indicate at which row SAS begins to read the data (DATAROW=)
- modify whether SAS extracts the variable names (GETNAMES=)

You can also use these same statements to change the default values.

When the IMPORT procedure reads a delimited file, it generates a DATA step to import the data. You control the results with options and statements that are specific to the input data source.

The IMPORT procedure generates the specified output SAS data set and writes information about the import to the SAS log. The log displays the DATA step code that is generated by the IMPORT procedure.

If you need to revise your code after the procedure runs, issue the RECALL command (or press F4) to recall the generated DATA step. At this point, you can add or remove options from the INFILE statement and customize the INFORMAT, FORMAT, and INPUT statements to your data.

Note: By default, the IMPORT procedure reads delimited files as varying record-length files. If your external file has a fixed-length format, use a SAS DATA step with an INFILE statement that includes the RECFM=F and LRECL= options

The PROC IMPORT Statement

PROC IMPORT statement imports an external data file to a SAS data set.

PROC IMPORT

```
DATAFILE= "filename" | TABLE= "tablename"
OUT=<libref. SAS data set><SAS data set options>
<DBMS=identifier><REPLACE>;
```

DATAFILE= "filename" | "fileref"

specifies the complete path and filename or fileref for the input PC file, spreadsheet, or delimited external file. A fileref is a SAS name that is associated with the physical location of the output file. To assign a fileref, use the FILENAME statement.

If you specify a fileref or if the complete path and filename does not include special characters such as the backslash in a path, lowercase characters, or spaces, then you can omit the quotation marks.

Restrictions The IMPORT procedure does not support device types or access methods for the FILENAME statement except for DISK. For example, the IMPORT procedure does not support the TEMP device type, which creates a temporary external file.

The IMPORT procedure can import data only if SAS supports the data type. SAS supports numeric and character types of data but not (for example) binary objects. If the data that you want to import is a type that SAS does not support, the IMPORT procedure might not be able to import it correctly. In many cases, the procedure attempts to convert

the data to the best of its ability. However, conversion is not possible for some types.

Interactions By default, the IMPORT procedure reads delimited files as varying record-length files. If your external file has a fixed-length format, use a SAS DATA step with an INFILE statement that includes the RECFM=F and LRECL= options.

When you use a fileref to specify a delimited file to import, the logical record length (LRECL) defaults to 256, unless you specify the LRECL= option in the FILENAME statement. The maximum LRECL value that the IMPORT procedure supports is 32767.

For delimited files, the first 20 rows are scanned to determine the variable attributes. You can increase the number of rows that are scanned by using the GUESSINGROWS= statement. All values are read in as character strings. If a Date and Time format or a numeric informat can be applied to the data value, the type is declared as numeric. Otherwise, the type remains character.

OUT= <libref> SAS data set

identifies the output SAS data set with either a one or two-level SAS name (library and member name). If the specified SAS data set does not exist, the IMPORT procedure creates it. If you specify a one-level name, by default the IMPORT procedure uses either the USER library (if assigned) or the WORK library (if USER is not assigned).

A SAS data set name can contain a single quotation mark when the VALIDMEMNAME=EXTEND system option is also specified. Using VALIDMEMNAME= expands the rules for the names of certain SAS members, such as a SAS data set name.

TABLE= “tablename”

specifies the name of the input DBMS table. If the name does not include special characters (such as question marks), lowercase characters, or spaces, you can omit the quotation marks. Note that the DBMS table name might be case sensitive.

Requirements You must have a license for SAS/ACCESS Interface to PC Files to import to a DBMS table.

When you import a DBMS table, you must specify the DBMS= option.

<DBMS=identifier>

specifies the type of data to import. You can import delimited files or JMP files (DBMS=JMP) in Base SAS. The JMP file format must be JMP 7 or later, and JMP variable names can be up to 255 characters long. SAS supports importing JMP files that have more than 32,767 variables.

To import a tab-delimited file, specify TAB as the identifier. To import any other delimited file that does not end in .CSV, specify DLM as the identifier. For a comma-separated file with a .CSV extension, DBMS= is optional. The IMPORT procedure recognizes .CSV as an extension for a comma-separated file.

<REPLACE>

overwrites an existing SAS data set. If you omit REPLACE, the IMPORT procedure does not overwrite an existing data set.

CAUTION:

Using the IMPORT procedure with the REPLACE option to write to an existing SAS generation data set causes the most recent generation data set or group of generation data sets to be deleted.

Here are two scenarios:

- If you specify the GENMAX= data set option to increase or decrease the number of generations, then all existing generations are deleted and replaced with a single new base generation data set
- If you omit the GENMAX= data set option, then all existing generations are deleted and replaced with a single new data set by the same name, but it is not a generation data set

Instead, use a SAS DATA step with the REPLACE= data set option to replace a permanent SAS data set and to maintain the generation group for that SAS data set.

<SAS data set options>

specifies SAS data set options. For example, to assign a password to the resulting SAS data set, you can use the ALTER=, PW=, READ=, or WRITE= data set options. To import only data that meets a specified condition, you can use the WHERE= data set option.

Restriction You cannot specify data set options when importing delimited, comma-separated, or tab-delimited external files.

Example: Importing a Comma-Delimited File with a CSV Extension

This example imports a comma-delimited file and creates a temporary SAS data set Work.Shoes.

```
proc import datafile="C:\certdata\test.csv";
    out=shoes
    dbms=csv
    replace;
    getnames=no;
run;
proc print data=work.shoes;
run;
```

Output 6.3 HTML Output: Work.Shoes Data Set**The SAS System**

Obs	VAR1	VAR2	VAR3	VAR4	VAR5	VAR6	VAR7
1	Africa	Boot	Addis Ababa	12	\$29,761	\$191,821	\$769
2	Asia	Boot	Bangkok	1	\$1,996	\$9,576	\$80
3	Canada	Boot	Calgary	8	\$17,720	\$63,280	\$472
4	Central America/Caribbean	Boot	Kingston	33	\$102,372	\$393,376	\$4,454
5	Eastern Europe	Boot	Budapest	22	\$74,102	\$317,515	\$3,341
6	Middle East	Boot	Al-Khobar	10	\$15,062	\$44,658	\$765
7	Pacific	Boot	Auckland	12	\$20,141	\$97,919	\$962
8	South America	Boot	Bogota	19	\$15,312	\$35,805	\$1,229
9	United States	Boot	Chicago	16	\$82,483	\$305,061	\$3,735
10	Western Europe	Boot	Copenhagen	2	\$1,663	\$4,657	\$129

Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- Which SAS statement associates the fileref `Crime` with the raw data file `C:\States\Data\crime.dat`?
 - `filename crime 'c:\states\data\crime.dat';`
 - `filename crime c:\states\data\crime.dat;`
 - `fileref crime 'c:\states\data\crime.dat';`
 - `filename 'c:\states\data\crime' crime.dat;`
- How long do filerefs remain in effect?
 - until you change them
 - until you cancel them
 - until you end your SAS session
 - all of the above
- Which statement identifies the name of a raw data file to be read with the fileref `Products` and specifies that the `DATA` step read only records 1-15?
 - `infile products obs 15;`
 - `infile products obs=15;`
 - `input products obs=15;`
 - `input products 1-15;`
- Which of the following programs correctly writes the observations from the data set below to a raw data file?


```

data shoes;
  infile 'c:\states\data\shoes.dat';
  input VAR1 VAR2 VAR3 VAR4 VAR5 VAR6 VAR7;
run;
```


SAS data set Work.Patients

ID	Sex	Age	Height	Weight	Pulse
2304	F	16	61	102	100
1128	M	43	71	218	76
4425	F	48	66	162	80
1387	F	57	64	142	70
9012	F	39	63	157	68
6312	M	52	72	240	77
5438	F	42	62	168	83
3788	M	38	73	234	71
9125	F	56	64	159	70
3438	M	15	66	140	67

- a.

```
data _null_;
    set work.patients;
    infile 'c:\clinic\patients\referrals.dat';
    input id $ 1-4 sex 6 $ age 8-9 height 11-12
          weight 14-16 pulse 18-20;

run;
```
- b.

```
data referrals.dat;
    set work.patients;
    input id $ 1-4 sex $ 6 age 8-9 height 11-12
          weight 14-16 pulse 18-20;

run;
```
- c.

```
data _null_;
    set work.patients;
    file c:\clinic\patients\referrals.dat;
    put id $ 1-4 sex 6 $ age 8-9 height 11-12
        weight 14-16 pulse 18-20;

run;
```
- d.

```
data _null_;
    set work.patients;
    file 'c:\clinic\patients\referrals.dat';
    put id $ 1-4 sex 6 $ age 8-9 height 11-12
        weight 14-16 pulse 18-20;

run;
```

5. Which raw data file can be read using column input?

a.

```
1---+-----10---+-----20---+
Henderson CA 26 ADM
Josephs SC 33 SALES
Williams MN 40 HRD
Rogan NY RECRTN
```

b.

```
1---+-----10---+-----20---+-----30
2803 Deborah Campos 173.97
2912 Bill      Marin 205.14
3015 Helen    Stinson 194.08
3122 Nicole   Terry 187.65
```

c.

1----	10----	20----
Avery John	\$601.23	
Davison Sherrill	\$723.15	
Holbrook Grace	\$489.76	
Jansen Mike	\$638.42	

d. all of the above.

6. Which program creates the output shown below?

1----	10----	20----	30----
3427 Chen	Steve	Raleigh	
1436 Davis	Lee	Atlanta	
2812 King	Vicky	Memphis	
1653 Sanchez	Jack	Atlanta	

Obs	ID	LastName	FirstName	City
1	3427	Chen	Steve	Raleigh
2	1436	Davis	Lee	Atlanta
3	2812	King	Vicky	Memphis
4	1653	Sanchez	Jack	Atlanta

- a.

```
data work.salesrep;
    infile empdata;
    input ID $ 1-4 LastName $ 6-12
           FirstName $ 14-18 City $ 20-29;
run;
proc print data=work.salesrep;
run;
```
- b.

```
data work.salesrep;
    infile empdata;
    input ID $ 1-4 Name $ 6-12
           FirstName $ 14-18 City $ 20-29;
run;
proc print data=work.salesrep;
run;
```
- c.

```
data work.salesrep;
    infile empdata;
    input ID $ 1-4 name1 $ 6-12
           name2 $ 14-18 City $ 20-29;
run;
proc print data=work.salesrep;
run;
```
- d. all of the above.

7. Which statement correctly reads the fields in the following order: StockNumber, Price, Item, Finish, Style?

Field Name	Start Column	End Column	Data Type
StockNumber	1	3	character
Finish	5	9	character

Field Name	Start Column	End Column	Data Type
Style	11	18	character
Item	20	24	character
Price	27	32	numeric

```

1---+----10---+----20---+----30---+
310 oak   pedestal table 329.99
311 maple pedestal table 369.99
312 brass floor   lamp   79.99
313 glass table   lamp   59.99
313 oak   rocking chair 153.99

```

- input StockNumber \$ 1-3 Finish \$ 5-9 Style \$ 11-18
Item \$ 20-24 Price 27-32;
 - input StockNumber \$ 1-3 Price 27-32
Item \$ 20-24 Finish \$ 5-9 Style \$ 11-18;
 - input \$ StockNumber 1-3 Price 27-32 \$
Item 20-24 \$ Finish 5-9 \$ Style 11-18;
 - input StockNumber \$ 1-3 Price \$ 27-32
Item \$ 20-24 Finish \$ 5-9 Style \$ 11-18;
8. Which statement correctly redefines the values of the variable Income as 100% higher?
- income=income*1.00;
 - income=income+(income*2.00);
 - income=income*2;
 - income=*2;
9. Which program correctly reads instream data?
- ```

data finance.newloan;
input datalines;
if country='JAPAN';
MonthAvg=amount/12;
1998 US CARS 194324.12
1998 US TRUCKS 142290.30
1998 CANADA CARS 10483.44
1998 CANADA TRUCKS 93543.64
1998 MEXICO CARS 22500.57
1998 MEXICO TRUCKS 10098.88
1998 JAPAN CARS 15066.43
1998 JAPAN TRUCKS 40700.34
;

```
  - ```

data finance.newloan;
input Year 1-4 Country $ 6-11
Vehicle $ 13-18 Amount 20-28;
if country='JAPAN';
MonthAvg=amount/12;
datalines;

```

```

run;

c. data finance.newloan;
    input Year 1-4 Country 6-11
           Vehicle 13-18 Amount 20-28;
    if country='JAPAN';
    MonthAvg=amount/12;
    datalines;
1998 US      CARS      194324.12
1998 US      TRUCKS    142290.30
1998 CANADA  CARS      10483.44
1998 CANADA  TRUCKS    93543.64
1998 MEXICO  CARS      22500.57
1998 MEXICO  TRUCKS    10098.88
1998 JAPAN   CARS      15066.43
1998 JAPAN   TRUCKS    40700.34
;

d. data finance.newloan;
    input Year 1-4 Country $ 6-11
           Vehicle $ 13-18 Amount 20-28;
    if country='JAPAN';
    MonthAvg=amount/12;
    datalines;
1998 US      CARS      194324.12
1998 US      TRUCKS    142290.30
1998 CANADA  CARS      10483.44
1998 CANADA  TRUCKS    93543.64
1998 MEXICO  CARS      22500.57
1998 MEXICO  TRUCKS    10098.88
1998 JAPAN   CARS      15066.43
1998 JAPAN   TRUCKS    40700.34
;

```

10. Which SAS statement subsets the raw data shown below so that only the observations in which Sex (in the second field) has a value of F are processed?

				10		20
Alfred	M	14	69.0	112.5		
Becka	F	13	65.3	98.0		
Gail	F	14	64.3	90.0		
Jeffrey	M	13	62.5	84.0		
John	M	12	59.0	99.5		
Karen	F	12	56.3	77.0		
Mary	F	15	66.5	112.0		
Philip	M	16	72.0	150.0		
Sandy	F	11	51.3	50.5		
Tammy	F	14	62.8	102.5		
William	M	15	66.5	112.0		

- if sex=f;
- if sex=F;
- if sex='F';
- a or b

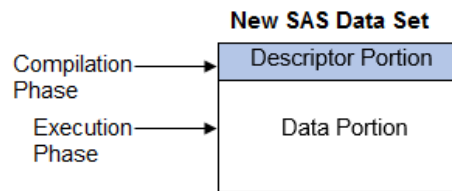
Chapter 7

Understanding DATA Step Processing

How SAS Processes Programs	139
Compilation Phase	141
Input Buffer	141
Program Data Vector (PDV)	141
Syntax Checking	142
Data Set Variables	142
Descriptor Portion of the SAS Data Set	143
Execution Phase	144
Initializing Variables	144
INFILE Statement	144
INPUT Statement	144
End of the DATA Step	146
Iterations of the DATA Step	148
End-of-File Marker	150
End of the Execution Phase	151
Debugging a DATA Step	152
Diagnosing Errors in the Compilation Phase	152
Diagnosing Errors in the Execution Phase	152
Example: Execution Phase Error	153
Validating and Cleaning Data	155
Using an Assignment Statement to Clean the Data	157
Testing Your Programs	158
Limiting Observations	158
PUT Statement	158
Example: Character Strings	159
Example: Data Set Variables	159
Example: Automatic Variables	160
Example: Conditional Processing	161
Chapter Quiz	161

How SAS Processes Programs

When you submit a DATA step, SAS processes the DATA step and creates a new SAS data set. A SAS DATA step is processed in two phases:

Figure 7.1 DATA Step Process

When you submit a DATA step for execution, SAS checks the syntax of the SAS statements and compiles them. In this phase, SAS identifies the type and length of each new variable, and determines whether a variable type conversion is necessary for each subsequent reference to a variable. During the compilation phase, SAS creates the following items:

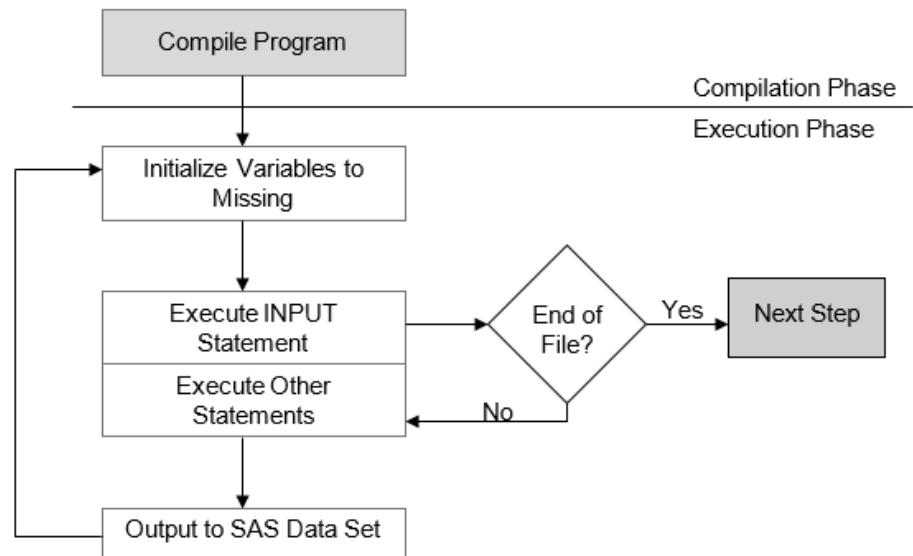
- input buffer
- program data vector (PDV)
- descriptor information

When the compilation phase is complete, the descriptor portion of the new data set is created.

By default, a simple DATA step iterates once for each observation that is being created. The flow of action in the Execution Phase of a simple DATA step is described as follows:

1. The DATA step begins with a DATA statement. Each time the DATA statement executes, a new iteration of the DATA step begins, and the `_N_` automatic variable is incremented by 1. The `_N_` automatic variable represents the number of times the DATA step has iterated.
2. SAS sets the newly created program variables to missing in the program data vector (PDV).
3. SAS reads a data record from a raw data file into the input buffer, or it reads an observation from a SAS data set directly into the program data vector. You can use an INPUT, MERGE, SET, MODIFY, or UPDATE statement to read a record.
4. SAS executes any subsequent programming statements for the current record and updates the PDV.
5. When SAS executes the last statement in the DATA step, all values in the PDV, except those marked to be dropped, are written as a single observation to the data set. Note that variables that you read with a SET, MERGE, MODIFY, or UPDATE statement are not reset to missing here.
6. SAS counts another iteration, reads the next record or observation, and executes the subsequent programming statements for the current observation.
7. The DATA step terminates when SAS encounters the end-of-file in a SAS data set or a raw data file.

Figure 7.2 shows the general flow of DATA step processing for reading raw data.

Figure 7.2 Compilation and Execution Phases of DATA Step Processing

Compilation Phase

Input Buffer

The input buffer is a logical area in memory into which SAS reads each record of a raw data file when SAS executes an INPUT statement. The buffer is created only when the DATA step reads raw data. When the DATA step reads a SAS data set, SAS reads the data directly into the PDV.

Program Data Vector (PDV)

After the input buffer is created, the PDV is created. The PDV is a logical area in memory where SAS builds a data set, one observation at a time. When a program executes, SAS reads data values from the input buffer or creates them by executing SAS language statements. The data values are assigned to the appropriate variables in the program data vector. From here, SAS writes the values to a SAS data set as a single observation.

Along with data set variables and computed variables, the PDV contains these automatic variables:

- the `_N_` variable, which counts the number of times the DATA step iterates.
- the `_ERROR_` variable, which signals the occurrence of an error caused by the data during execution. The value of `_ERROR_` is either 0 (no error exists) or 1 (one or more errors occurred). The default value is 0.

Note: SAS does not write these variables to the output data set.

Syntax Checking

During the compilation phase, SAS scans each statement in the DATA step, looking for syntax errors. Here are examples:

- missing or misspelled keywords
- invalid variable names
- missing or invalid punctuation
- invalid options

Data Set Variables

As the INPUT statement is compiled, a slot is added to the program data vector for each variable in the new data set. Generally, variable attributes such as length and type are determined the first time a variable is encountered.

```
filename invent 'Z:\sasuser\invent.dat';
data work.update;
    infile invent;
    input Item $1-13 IDnum $15-19
    InStock 21-22 BackOrd 24-25;
    Total=instock+backord;
run;
```

Program Data Vector

N	ERROR	Item	IDnum	InStock	BackOrd	

Any variables that are created with an assignment statement in the DATA step are also added to the program data vector. For example, the assignment statement below creates the variable `Total`. As the statement is compiled, the variable is added to the program data vector. The attributes of the variable are determined by the expression in the statement. Because the expression contains an arithmetic operator and produces a numeric value, `Total` is defined as a numeric variable and is assigned the default length of 8.

```
filename invent 'Z:\sasuser\invent.dat';
data work.update;
    infile invent;
    input Item $1-13 IDnum $15-19
    InStock 21-22 BackOrd 24-25;
    Total=instock+backord;
run;
```

Program Data Vector

N	ERROR	Item	IDnum	InStock	BackOrd	Total

Descriptor Portion of the SAS Data Set

The descriptor portion is information that SAS creates and maintains about each SAS data set, including data set attributes and variable attributes. Here are examples:

- the name of the data set and its member type
- the date and time that the data set was created
- the names, data types (character or numeric), and lengths of the variables

The descriptor information also contains information about extended attributes (if defined on a data set). Extended attribute descriptor information includes the name of the attribute, the name of the variable, and the value of the attribute.

Figure 7.3 CONTENTS Procedure Output: Data Set Descriptor Specifics

The SAS System			
The CONTENTS Procedure			
Data Set Name	WORK.UPDATE	Observations	0
Member Type	DATA	Variables	5
Engine	V9	Indexes	0
Created	03/22/2017 16:24:42	Observation Length	48
Last Modified	03/22/2017 16:24:42	Deleted Observations	0
Protection		Compressed	NO
Data Set Type		Sorted	NO
Label			
Data Representation	WINDOWS_64		
Encoding	utf-8 Unicode (UTF-8)		

Engine/Host Dependent Information	
Data Set Page Size	65536
Number of Data Set Pages	1
First Data Page	1
Max Obs per Page	1361
Obs in First Data Page	9
Number of Data Set Repairs	0
ExtendObsCounter	YES
Filename	C:\Users\sasuser\SAS Temporary Files\update.sas7bdat
Release Created	V.0301M0
Host Created	X64_10PRO
Owner Name	sasuser
File Size	128KB
File Size (bytes)	131072

Alphabetic List of Variables and Attributes			
#	Variable	Type	Len
4	BackOrd	Num	8
2	IDnum	Char	5
3	InStock	Num	8
1	Item	Char	13
5	Total	Num	8

At this point, the data set contains the five variables that are defined in the input data set and in the assignment statement. `_N_` and `_ERROR_` are not written to the data set. There are no observations because the DATA step has not yet executed. During execution, each raw data record is processed and is then written to the data set as an observation.

Execution Phase

Initializing Variables

At the beginning of the execution phase, the value of `_N_` is 1. Because there are no data errors, the value of `_ERROR_` is 0.

```
filename invent 'Z:\sasuser\invent.dat';
data work.update;
    infile invent;
    input Item $1-13 IDnum $15-19
          InStock 21-22 BackOrd 24-25;
    Total=instock+backord;
run;
```

Program Data Vector

<code>_N_</code>	<code>_ERROR_</code>	<code>Item</code>	<code>IDnum</code>	<code>InStock</code>	<code>BackOrd</code>	<code>Total</code>
1	0			.	.	.

|----- Initialized to Missing -----|

The remaining variables are initialized to missing. Missing numeric values are represented by periods, and missing character values are represented by blanks.

INFILE Statement

The INFILE statement identifies the location of the raw data.

```
filename invent 'Z:\sasuser\invent.dat';
data work.update;
    infile invent;
    input Item $1-13 IDnum $15-19
          InStock 21-22 BackOrd 24-25;
    Total=instock+backord;
run;
```

INPUT Statement

After that, the INPUT statement reads a record into the input buffer. Then, the raw data in columns 1-13 is read and is assigned to `Item` in the program data vector.

```
data work.update;
    infile invent;
    input Item $ 1-13 IDnum $ 15-19
          InStock 21-22 BackOrd 24-25;
    Total=instock+backord;
run;
```

Raw Data File Invent

1	---	----	10	---	----	20	---	+	-
Bird Feeder			LG088	3		20			
6 Glass Mugs			SB082	6		12			
Glass Tray			BQ049	12		6			
Padded Hangrs			MN256	15		20			
Jewelry Box			AJ498	23		0			
Red Apron			AQ072	9		12			
Crystal Vase			AQ672	27		0			
Picnic Basket			LS930	21		0			
Brass Clock			AN910	2		10			

Program Data Vector

N	ERROR	Item	IDnum	InStock	BackOrd	Total
1	0			•	•	•

Next, the data in columns 15-19 is read and is assigned to IDnum in the program data vector.

```
data work.update;
  infile invent;
  input Item $ 1-13 IDnum $ 15-19
         InStock 21-22 BackOrd 24-25;
  Total=instock+backord;
run;
```

Raw Data File Invent

1	---	----	10	---	----	V0	---	+	-
Bird Feeder			LG088	•		3			20
6 Glass Mugs			SB082	6		12			
Glass Tray			BQ049	12		6			
Padded Hangrs			MN256	15		20			
Jewelry Box			AJ498	23		0			
Red Apron			AQ072	9		12			
Crystal Vase			AQ672	27		0			
Picnic Basket			LS930	21		0			
Brass Clock			AN910	2		10			

Program Data Vector

N	ERROR	Item	IDnum	InStock	BackOrd	Total
1	0	Bird Feeder	LG088	•	•	•

Likewise, the INPUT statement reads the values for InStock from columns 21-22, and it reads the values for BackOrd from columns 24-25.

```
data work.update;
  infile invent;
  input Item $ 1-13 IDnum $ 15-19
         InStock 21-22 BackOrd 24-25;
  Total=instock+backord;
run;
```

Raw Data File Invent

1	---	+	----	10	---	+	----	20	---	+	V
Bird Feeder				LG088				3			20•
6 Glass Mugs				SB082				6			12
Glass Tray				BQ049				12			6
Padded Hangrs				MN256				15			20
Jewelry Box				AJ498				23			0
Red Apron				AQ072				9			12
Crystal Vase				AQ672				27			0
Picnic Basket				LS930				21			0
Brass Clock				AN910				2			10

Program Data Vector

N	ERROR	Item	IDnum	InStock	BackOrd	Total
1	0	Bird Feeder	LG088	3	20	•

Next, the assignment statement executes. The values for InStock and BackOrd are added to produce the values for Total.

```
data work.update;
  infile invent;
  input Item $ 1-13 IDnum $ 15-19
        InStock 21-22 BackOrd 24-25;
  Total=instock+backord;
run;
```

Raw Data File Invent

1	---	+	----	10	---	+	----	20	---	+	V
Bird Feeder				LG088				3			20•
6 Glass Mugs				SB082				6			12
Glass Tray				BQ049				12			6
Padded Hangrs				MN256				15			20
Jewelry Box				AJ498				23			0
Red Apron				AQ072				9			12
Crystal Vase				AQ672				27			0
Picnic Basket				LS930				21			0
Brass Clock				AN910				2			10

Program Data Vector

N	ERROR	Item	IDnum	InStock	BackOrd	Total
1	0	Bird Feeder	LG088	3	20	23

└─ + ─┐
└─ + ─┐ = ─┐

End of the DATA Step

At the end of the DATA step, several actions occur. First, the values in the program data vector are written to the output data set as the first observation.

```
data work.update;
  infile invent;
  input Item $ 1-13 IDnum $ 15-19
        InStock 21-22 BackOrd 24-25;
  Total=instock+backord;
```

```
run;
```

Raw Data File Invent

```
1---+----10---+----20---+V
Bird Feeder   LG088   3 20•
6 Glass Mugs  SB082   6 12
Glass Tray    BQ049  12  6
Padded Hangrs MN256  15 20
Jewelry Box   AJ498   23  0
Red Apron     AQ072   9 12
Crystal Vase  AQ672  27  0
Picnic Basket LS930  21  0
Brass Clock   AN910   2 10
```

Program Data Vector

N	_ERROR	Item	IDnum	InStock	BackOrd	Total
1	0	Bird Feeder	LG088	3	20	23

SAS Data Set Perm.Update

Item	IDnum	InStock	BackOrd	Total
Bird Feeder	LG088	3	20	23

Next, control returns to the top of the DATA step and the value of `_N_` increments from 1 to 2. Finally, the variable values in the program data vector are reset to missing. Notice that the automatic variable `_ERROR_` is reset to 0 if necessary.

```
data work.update;
  infile invent;
  input Item $ 1-13 IDnum $ 15-19
         InStock 21-22 BackOrd 24-25;
  Total=instock+backord;
run;
```

Raw Data File Invent

```
V---+----10---+----20---+--
Bird Feeder   LG088   3 20
6 Glass Mugs  SB082   6 12
Glass Tray    BQ049  12  6
Padded Hangrs MN256  15 20
Jewelry Box   AJ498   23  0
Red Apron     AQ072   9 12
Crystal Vase  AQ672  27  0
Picnic Basket LS930  21  0
Brass Clock   AN910   2 10
```

Program Data Vector

N	_ERROR	Item	IDnum	InStock	BackOrd	Total
2	0			.	.	.

Set to Missing

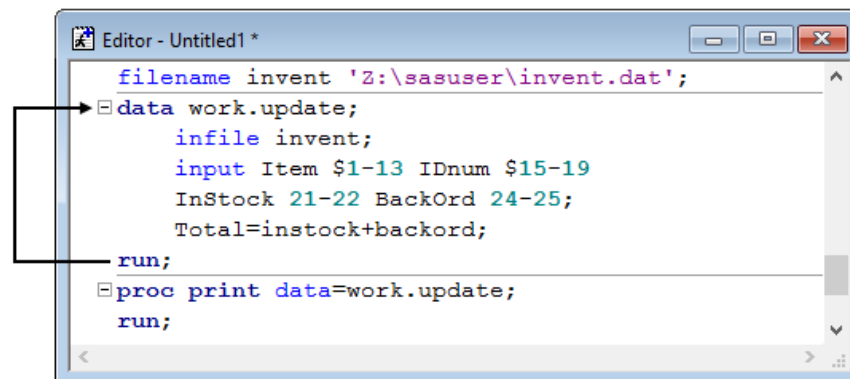
SAS Data Set Perm.Update

Item	IDnum	InStock	BackOrd	Total
Bird Feeder	LG088	3	20	23

Iterations of the DATA Step

You can see that the DATA step works like a loop, repetitively executing statements to read data values and create observations one by one. At the beginning of the second iteration, the value of `_N_` is 2, and `_ERROR_` is still 0. Each loop (or cycle of execution) is called an *iteration*.

Figure 7.4 Iterations of the DATA Step



As the INPUT statement executes for the second time, the values from the second record are read into the input buffer and then into the program data vector.

Raw Data File Invent

1	---	10	---	20	---	V
Bird Feeder	LG088	3	20			
6 Glass Mugs	SB082	6	12			
Glass Tray	BQ049	12	6			
Padded Hangrs	MN256	15	20			
Jewelry Box	AJ498	23	0			
Red Apron	AQ072	9	12			
Crystal Vase	AQ672	27	0			
Picnic Basket	LS930	21	0			
Brass Clock	AN910	2	10			

Program Data Vector

<code>_N_</code>	<code>_ERROR_</code>	Item	IDnum	InStock	BackOrd	Total
2	0	6 Glass Mugs	SB082	6	12	

SAS Data Set Perm.Update

Item	IDnum	InStock	BackOrd	Total
Bird Feeder	LG088	3	20	23

Next, the value for `Total` is calculated based on the current values for `InStock` and `BackOrd`. The `RUN` statement indicates the end of the DATA step loop.

```

data work.update;
    infile invent;
    input Item $ 1-13 IDnum $ 15-19
          InStock 21-22 BackOrd 24-25;
    Total=instock+backord;
  
```

```
run;
```

Raw Data File Invent				
1---+-----10---+-----20---+V				
Bird Feeder	LG088	3	20	
6 Glass Mugs	SB082	6	12	*
Glass Tray	BQ049	12	6	
Padded Hangrs	MN256	15	20	
Jewelry Box	AJ498	23	0	
Red Apron	AQ072	9	12	
Crystal Vase	AQ672	27	0	
Picnic Basket	LS930	21	0	
Brass Clock	AN910	2	10	

Program Data Vector

N	ERROR	Item	IDnum	InStock	BackOrd	Total
2	0	6 Glass Mugs	SB082	6	12	18

$\boxed{+} \quad \boxed{-} \quad \boxed{=} \quad \boxed{\uparrow}$

SAS Data Set Perm.Update

Item	IDnum	InStock	BackOrd	Total
Bird Feeder	LG088	3	20	23

At the bottom of the DATA step, the values in the program data vector are written to the data set as the second observation.

```
data work.update;
  infile invent;
  input Item $ 1-13 IDnum $ 15-19
         InStock 21-22 BackOrd 24-25;
  Total=instock+backord;
run;
```

Next, the value of `_N_` increments from 2 to 3, control returns to the top of the DATA step, and the values for `Item`, `IDnum`, `InStock`, `BackOrd`, and `Total` are reset to missing.

```
data work.update;
  infile invent;
  input Item $ 1-13 IDnum $ 15-19
         InStock 21-22 BackOrd 24-25;
  Total=instock+backord;
run;
```

Raw Data File Invent

V---+-----10---+-----20---+--				
Bird Feeder	LG088	3	20	
6 Glass Mugs	SB082	6	12	
Glass Tray	BQ049	12	6	
Padded Hangrs	MN256	15	20	
Jewelry Box	AJ498	23	0	
Red Apron	AQ072	9	12	
Crystal Vase	AQ672	27	0	
Picnic Basket	LS930	21	0	
Brass Clock	AN910	2	10	

Program Data Vector

N	ERROR	Item	IDnum	InStock	BackOrd	Total
3	0			.	.	.

Reset to Missing

SAS Data Set Perm.Update

Item	IDnum	InStock	BackOrd	Total
Bird Feeder	LG088	3	20	23
6 Glass Mugs	SB082	6	12	18

End-of-File Marker

The execution phase continues in this manner until the end-of-file marker is reached in the raw data file. When there are no more records in the raw data file to be read, the data portion of the new data set is complete and the DATA step stops.

Raw Data File Invent

1---+-----10---+-----20---+--				
Bird Feeder	LG088	3	20	
6 Glass Mugs	SB082	6	12	
Glass Tray	BQ049	12	6	
Padded Hangrs	MN256	15	20	
Jewelry Box	AJ498	23	0	
Red Apron	AQ072	9	12	
Crystal Vase	AQ672	27	0	
Picnic Basket	LS930	21	0	
Brass Clock	AN910	2	10	

This is the output data set that SAS creates:

Figure 7.5 SAS Data Set Work.Update**The SAS System**

Obs	Item	IDnum	InStock	BackOrd	Total
1	Bird Feeder	LG088	3	20	23
2	6 Glass Mugs	SB082	6	12	18
3	Glass Tray	BQ049	12	6	18
4	Padded Hangrs	MN256	15	6	21
5	Jewelry Box	AJ498	23	0	23
6	Red Apron	AQ072	9	12	21
7	Crystal Vase	AQ672	27	0	27
8	Picnic Basket	LS930	21	0	21
9	Brass Clock	AN910	2	10	12

When reading variables from raw data, SAS sets the value of each variable in the DATA step to missing at the beginning of each cycle of execution, with these exceptions:

- variables that are named in a RETAIN statement
- variables that are created in a sum statement
- data elements in a _TEMPORARY_ array
- any variables that are created with options in the FILE or INFILE statements
- automatic variables

In contrast, when reading variables from a SAS data set, SAS sets the values to missing only before the first cycle of execution of the DATA step. Therefore, the variables retain their values until new values become available (for example, through an assignment statement or through the next execution of a SET or MERGE statement). Variables that are created with options in the SET or MERGE statements also retain their values from one cycle of execution to the next.

End of the Execution Phase

At the end of the execution phase, the SAS log confirms that the raw data file was read, and it displays the number of observations and variables in the data set.

Log 7.1 SAS Log

```
NOTE: 9 records were read from the infile INVENT.
      The minimum record length was 80.
      The maximum record length was 80.
NOTE: The data set WORK.UPDATE has 9 observations and 5 variables.
```

Recall that you can display the data set with the PRINT procedure.

```
proc print data=work.update;
run;
```

Figure 7.6 Output from the PRINT Procedure

The SAS System

Obs	Item	IDnum	InStock	BackOrd	Total
1	Bird Feeder	LG088	3	20	23
2	6 Glass Mugs	SB082	6	12	18
3	Glass Tray	BQ049	12	6	18
4	Padded Hangrs	MN256	15	6	21
5	Jewelry Box	AJ498	23	0	23
6	Red Apron	AQ072	9	12	21
7	Crystal Vase	AQ672	27	0	27
8	Picnic Basket	LS930	21	0	21
9	Brass Clock	AN910	2	10	12

Debugging a DATA Step

Diagnosing Errors in the Compilation Phase

Errors that are detected during the compilation phase include these:

- misspelled keywords and data set names
- unbalanced quotation marks
- invalid options

During the compilation phase, SAS can interpret some syntax errors (such as the keyword DATA misspelled as DAAT). If it cannot interpret the error, SAS does the following:

- prints the word ERROR followed by an error message in the log
- compiles but does not execute the step where the error occurred, and prints the following message:

NOTE: The SAS System stopped processing this step because of errors.

Some errors are explained fully by the message that SAS prints; other error messages are not as easy to interpret. For example, because SAS statements are free-format, when you fail to end a SAS statement with a semicolon, SAS cannot detect the error.

Diagnosing Errors in the Execution Phase

When SAS detects an error in the execution phase, the following can occur, depending on the type of error:

- A note, warning, or error message is displayed in the log.
- The values that are stored in the program data vector are displayed in the log.
- The processing of the step either continues or stops.

Example: Execution Phase Error

Suppose you misspelled the fileref in the INFILE statement below. This is not a syntax error, because SAS does not validate the file that you reference until the execution phase. During the compilation phase, the fileref `Invnt` is assumed to reference some external raw data file.

```
data work.update;
  infile invnt;
  input Item $ 1-13 IDnum $ 15-19
         InStock 21-22 BackOrd 24-25;
  Total=instock+backord;
run;
```

This error is not detected until the execution phase begins. Because there is no external file that is referenced by the fileref `Invnt`, the DATA step stops processing.

Log 7.2 SAS Log

```
3486 data work.update;
3487   infile invnt;
3488   input Item $1-13 IDnum $15-19
3489     InStock 21-22 BackOrd 24-25;
3490   Total=instock+backord;
3491 run;

ERROR: No logical assign for filename INVNT.
NOTE: The SAS System topped processing this step because of errors.
WARNING: The data set WORK.UPDATE may be incomplete. When this step was stopped
         there was 0 observations and 5 variables.
WARNING: Data set WORK.UPDATE was not replaced because this step was stopped.
NOTE: DATA statement used (Total process time):
      real time           0.01 seconds
      cpu time            0.01 seconds
```

Because `Invnt` is misspelled, the statement in the DATA step that identifies the raw data is incorrect. Note, however, that the correct number of variables was defined in the descriptor portion of the data set.

Incorrectly identifying a variable's type is another common execution-time error. Recall that the values for `IDnum` are character values. Suppose you forget to place the dollar sign (\$) after the variable's name in your INPUT statement. This is not a compile-time error, because SAS cannot verify `IDnum`'s type until the data values for `IDnum` are read.

Raw Data File `Invnt`

	10	20
Bird Feeder	LG088	3 20
6 Glass Mugs	SB082	6 12
Glass Tray	BQ049	12 6
Padded Hangrs	MN256	15 20
Jewelry Box	AJ498	23 0
Red Apron	AQ072	9 12
Crystal Vase	AQ672	27 0
Picnic Basket	LS930	21 0
Brass Clock	AN910	2 10

```
data work.update;
```

```

infile invent;
input Item $ 1-13 IDnum 15-19
       InStock 21-22 BackOrd 24-25;
Total=instock+backord;
run;

```

In this case, the DATA step completes the execution phase, and the observations are written to the data set. However, several notes appear in the log.

Log 7.3 SAS Log (partial log)

```

NOTE: The infile INVENT is:
      Filename=Z:\sasuser\invent.dat,
      RECFM=V,LRECL=32767,File Size (bytes)=738,
      Last Modified=02Feb2017:13:50:21,
      Create Time=19Dec2016:12:49:08

NOTE: Invalid data for IDnum in line 1 15-19.
RULE:  ----+----1-----2-----3-----4-----5-----6-----
+----7-----8-----+
1      Bird Feeder  LG088  3
20                                           80
Item=Bird Feeder IDnum=. InStock=3 BackOrd=20 Total=23 _ERROR_=1 _N_=1
NOTE: Invalid data for IDnum in line 2 15-19.
2      6 Glass Mugs  SB082  6
12                                           80
Item=6 Glass Mugs IDnum=. InStock=6 BackOrd=12 Total=18 _ERROR_=1 _N_=2
NOTE: Invalid data for IDnum in line 3 15-19.
3      Glass Tray   BQ049 12
6                                           80
Item=Glass Tray IDnum=. InStock=12 BackOrd=6 Total=18 _ERROR_=1 _N_=3
NOTE: Invalid data for IDnum in line 4 15-19.
4      Padded Hangrs MN256 15
6                                           80
Item=Padded Hangrs IDnum=. InStock=15 BackOrd=6 Total=21 _ERROR_=1 _N_=4
NOTE: Invalid data for IDnum in line 5 15-19.

```

Each note identifies the location of the invalid data for each observation. In this example, the invalid data is located in columns 15-19 for all observations.

The second line in each note (excluding the RULE line) displays the raw data record. Notice that the second field displays the values for IDnum, which are obviously character values.

The third and fourth lines display the values that are stored in the program data vector. Here, the values for IDnum are missing, although the other values have been correctly assigned to their respective variables. Notice that `_ERROR_` has a value of 1, indicating that a data error has occurred.

The PRINT procedure displays the data set, showing that the values for IDnum are missing. In this example, the periods indicate that IDnum is a numeric variable, although it should have been defined as a character variable.

```

proc print data=work.update;
run;

```

Figure 7.7 Output from the PRINT Procedure Showing Missing Values for IDnum

The SAS System

Obs	Item	IDnum	InStock	BackOrd	Total
1	Bird Feeder	.	3	20	23
2	6 Glass Mugs	.	6	12	18
3	Glass Tray	.	12	6	18
4	Padded Hangrs	.	15	6	21
5	Jewelry Box	.	23	0	23
6	Red Apron	.	9	12	21
7	Crystal Vase	.	27	0	27
8	Picnic Basket	.	21	0	21
9	Brass Clock	.	2	10	12

Validating and Cleaning Data

Recall that data errors occur when data values are not appropriate for the SAS statements that are specified in a program. SAS detects data errors during program execution. When a data error is detected, SAS continues to execute the program.

In general, SAS procedures analyze data, produce output, or manage SAS files. In addition, SAS procedures can be used to detect invalid data. In addition to the PRINT procedure showing missing values, the following procedures can be used to detect invalid data:

- PROC FREQ
- PROC MEANS

The FREQ procedure detects invalid character and numeric values by looking at distinct values. You can use PROC FREQ to identify any variables that were not given an expected value.

Syntax, FREQ procedure:

PROC FREQ DATA=SAS-data-set <NLEVELS>;

TABLES variable(s);

RUN;

- The NLEVELS option displays a table that provides the number of distinct values for each variable that is named in the TABLES statement.
 - The TABLES statement specifies the frequency tables to produce.
-

In the following example, the data set contains invalid characters for the variables Gender and Age. PROC FREQ displays the distinct values of variables and is therefore useful for finding invalid values in data. You can use PROC FREQ with the TABLES statement to produce a frequency table for specific variables.

```
proc freq data=work.Patients;
    tables Gender Age;
run;
```

In the following figures, notice the valid (M and F) and invalid (G) values for Gender, and the valid and invalid (242) values for Age. In both the Gender and Age FREQ tables, one observation needs data to be cleaned.

Figure 7.8 HTML Output: FREQ Procedure

The FREQ Procedure				
Gender	Frequency	Percent	Cumulative Frequency	Cumulative Percent
F	2	50.00	2	50.00
G	1	25.00	3	75.00
M	1	25.00	4	100.00

Age	Frequency	Percent	Cumulative Frequency	Cumulative Percent
44	1	25.00	1	25.00
61	1	25.00	2	50.00
64	1	25.00	3	75.00
242	1	25.00	4	100.00

The MEANS procedure can also be used to validate data because it produces summary reports that display descriptive statistics. For example, PROC MEANS can show whether the values for a particular variable are within their expected range.

Syntax, MEANS procedure:

PROC MEANS DATA=*SAS-data-set* <*statistics*>;

VAR *variable(s)*;

RUN;

- The statistics to display can be specified as an option in the PROC MEANS statement.
 - The VAR statement specifies the analysis variables and their order in the results.
-

Using the same data set as in the previous example, you can submit PROC MEANS to determine whether the age of all test subjects is within a reasonable range. Notice that the VAR statement is specified with that particular variable (Age) to get the statistical information, or range, of the data values.

```
proc means data=work.Patients;
    var Age;
run;
```

The following figure shows the output for the MEANS procedure. It displays a range of 44 to 242, which clearly indicates that there is invalid data somewhere in the Age column.

Figure 7.9 HTML Output: MEANS Procedure

The MEANS Procedure				
Analysis Variable : Age				
N	Mean	Std Dev	Minimum	Maximum
4	102.7500000	93.2501117	44.0000000	242.0000000

Using an Assignment Statement to Clean the Data

You can use an assignment statement or a conditional clause to programmatically clean invalid data when it is identified. For example, if your input data contains a field called Gender, and that field has an invalid value (a value other than M or F), then you can clean your data by changing the invalid value to a valid value for Gender. To avoid overwriting your original data set, you can use the DATA statement to create a new data set. The new data set contains all of the data from your original data set, along with the correct values for invalid data.

The following example assumes that Gender has an invalid value of G in the input data. This error might be the result of a data entry error. If G should actually be F, it is possible to correct the invalid data for Gender by using an assignment statement along with an IF-THEN statement:

```
data work.clean_data;
    set work.patients;
    gender=upcase(Gender);
    if Gender='G' then Gender='F';
run;

proc print data=work.clean_data;
run;
```

Notice that two observations contain invalid values for Age. These values exceed a maximum value of 110. It is possible to uniquely identify each of the observations by specifying the variable Empid. After checking the date of birth in each of the observations and determining the correct value for Age, you can change the data by inserting an IF-THEN-ELSE statement:

```
data work.clean_data;
    set work.patients;
    if empid=3294 then age=65;
    else if empid=7391 then age=75;
run;

proc print data=work.clean_data;
run;
```

Another way of ensuring that your output data set contains valid data is to programmatically identify invalid data and delete the associated observations from your output data set:

```
data work.clean_data;
```

```

        set work.patients;
        if Age>110 then delete;
run;

proc print data=work.clean_data;
run;

```

Testing Your Programs

Limiting Observations

Remember that you can use the OBS= option in the INFILE statement to limit the number of observations that are read or created during the execution of the DATA step.

```

data work.update;
    infile invent obs=10;
    input Item $ 1-13 IDnum $ 15-19
           InStock 21-22 BackOrd 24-25;
    Total=instock+backord;
run;

```

When processed, this DATA step creates the work.update data set with variables but with only 10 observations.

PUT Statement

When the source of program errors is not apparent, you can use the PUT statement to examine variable values and to print your own message in the log. For diagnostic purposes, you can use IF-THEN/ELSE statements to conditionally check for values. For more information about IF-THEN/ELSE statements, see [Chapter 11, “Creating and Managing Variables,”](#) on page 231.

```

data work.test;
    infile loan;
    input Code $ 1 Amount 3-10 Rate 12-16
           Account $ 18-25 Months 27-28;
    if code='1' then type='variable';
    else if code='2' then type='fixed';
    else type='unknown';
    if type='unknown' then put 'MY NOTE: invalid value:' code=;
run;

```

In this example, if Code does not have the expected values of 1 or 2, the PUT statement writes a message to the log:

Log 7.4 SAS Log

```

MY NOTE: invalid value: Code=V
NOTE: 9 records were read from the infile LOAN.
      The minimum record length was 28.
      The maximum record length was 28.
NOTE: The data set WORK.TEST has 9 observations and 6 variables.
NOTE: DATA statement used (Total process time):
      real time          0.01 seconds
      cpu time           0.03 seconds

```

Syntax, PUT statement:

PUT *specification(s)*;

specification specifies what is written, how it is written, and where it is written. Here are examples:

- a character string
- one or more data set variables
- the automatic variables `_N_` and `_ERROR_`
- the automatic variable `_ALL_`

Example: Character Strings

You can use a PUT statement to specify a character string to identify your message in the log. The character string must be enclosed in quotation marks.

```
put 'MY NOTE: The condition was met.';
```

Log 7.5 SAS Log

```

MY NOTE: The condition was met.
NOTE: 9 records were read from the infile LOAN.
      The minimum record length was 28.
      The maximum record length was 28.
NOTE: The data set WORK.TEST has 9 observations and 6 variables.
NOTE: DATA statement used (Total process time):
      real time          0.01 seconds
      cpu time           0.01 seconds

```

Example: Data Set Variables

You can use a PUT statement to specify one or more data set variables to be examined for that iteration of the DATA step:

```
put 'MY NOTE: invalid value: 'code type;
```

Log 7.6 SAS Log

```

MY NOTE: invalid value: V unknown
NOTE: 9 records were read from the infile LOAN.
      The minimum record length was 28.
      The maximum record length was 28.
NOTE: The data set WORK.TEST has 9 observations and 6 variables.
NOTE: DATA statement used (Total process time):
      real time          0.00 seconds
      cpu time           0.00 seconds

```

Note that when you specify a variable in the PUT statement, only its value is written to the log. To write both the variable name and its value in the log, add an equal sign (=) to the variable name.

```
put 'MY NOTE: invalid value: 'code= type=;
```

Log 7.7 SAS Log

```

MY NOTE: invalid value: Code=V type=unknown
NOTE: 9 records were read from the infile LOAN.
      The minimum record length was 28.
      The maximum record length was 28.
NOTE: The data set WORK.TEST has 9 observations and 6 variables.
NOTE: DATA statement used (Total process time):
      real time          0.01 seconds
      cpu time           0.00 seconds

```

Example: Automatic Variables

You can use a PUT statement to display the values of the automatic variables `_N_` and `_ERROR_`. In some cases, knowing the value of `_N_` can help you locate an observation in the data set:

```
put 'MY NOTE: invalid value: '
    code= _n= _error=;
```

Log 7.8 SAS Log

```

MY NOTE: invalid value: Code=V _N_=3 _ERROR_=0
NOTE: 9 records were read from the infile LOAN.
      The minimum record length was 28.
      The maximum record length was 28.
NOTE: The data set WORK.TEST has 9 observations and 6 variables.
NOTE: DATA statement used (Total process time):
      real time          0.01 seconds
      cpu time           0.01 seconds

```

You can also use a PUT statement to write all variable names and variable values, including automatic variables, to the log. Use the `_ALL_` specification:

```
put 'MY NOTE: invalid value: ' _all_ ;
```

Log 7.9 SAS Log

```

MY NOTE: invalid value: Code=V Amount=10000 Rate=10.5 Account=101-1289 Months=8
type=unknown _ERROR_=0 _N_=3
NOTE: 9 records were read from the infile LOAN.
      The minimum record length was 28.
      The maximum record length was 28.
NOTE: The data set WORK.TEST has 9 observations and 6 variables.
NOTE: DATA statement used (Total process time):
      real time          0.01 seconds
      cpu time           0.01 seconds

```

Example: Conditional Processing

You can use a PUT statement with conditional processing (that is, with IF-THEN/ELSE statements) to flag program errors or data that is out of range. In the example below, the PUT statement is used to flag any missing or zero values for the variable Rate.

```

data finance.newcalc;
  infile newloans;
  input LoanID $ 1-4 Rate 5-8 Amount 9-19;
  if rate>0 then
    Interest=amount*(rate/12);
  else put 'DATA ERROR ' rate= _n_=;
run;

```

Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

1. Which of the following is not created during the compilation phase?
 - a. the data set descriptor
 - b. the first observation
 - c. the program data vector
 - d. the _N_ and _ERROR_ automatic variables
2. During the compilation phase, SAS scans each statement in the DATA step, looking for syntax errors. Which of the following is not considered a syntax error?
 - a. incorrect values and formats
 - b. invalid options or variable names
 - c. missing or invalid punctuation
 - d. missing or misspelled keywords
3. Unless otherwise directed, how does the DATA step execute?
 - a. once for each compilation phase
 - b. once for each DATA step statement
 - c. once for each record in the input file

- d. once for each variable in the input file
- 4. At the beginning of the execution phase, the value of `_N_` is 1, the value of `_ERROR_` is 0, and the values of the remaining variables are set to the following:
 - a. 0
 - b. 1
 - c. undefined
 - d. missing
- 5. Suppose you run a program that causes three DATA step errors. What is the value of the automatic variable `_ERROR_` when the observation that contains the third error is processed?
 - a. 0
 - b. 1
 - c. 2
 - d. 3
- 6. Which of the following actions occurs at the beginning of an iteration of the DATA step?
 - a. The automatic variables `_N_` and `_ERROR_` are incremented by one.
 - b. The DATA step stops execution.
 - c. The descriptor portion of the data set is written.
 - d. The values of variables created in programming statements are reset to missing in the program data vector.
- 7. Consider the following DATA step. Based on the INPUT statement, in what order are the variables stored in the new data set?

```
data work.update;
  infile invent;
  input IDnum $ Item $ 1-13 Instock 21-22
        BackOrd 24-25;
  Total=instock+backord;
run;
```

- a. IDnum Item InStock BackOrd Total
- b. Item IDnum InStock BackOrd Total
- c. Total IDnum Item InStock BackOrd
- d. Total Item IDnum InStock BackOrd
- 8. What happens when SAS cannot interpret syntax errors?
 - a. Data set variables contain missing values.
 - b. The DATA step does not compile.
 - c. The DATA step still compiles, but it does not execute.
 - d. The DATA step still compiles and executes.
- 9. What is wrong with this program?

```
data work.update;
  infile invent
  input Item $ 1-13 IDnum $ 15-19 Instock 21-22
```

```

        BackOrd 24-25;
    total=instock+backord;
run;

```

- a. There is a missing semicolon on the second line.
 - b. There is a missing semicolon on the third line.
 - c. The variables are in incorrect order.
 - d. The variable type is incorrect.
10. Consider this section of a SAS session log. Based on the note, what was the most likely problem with the DATA step?

```

NOTE: Invalid data for IDnum in line 7 15-19.
RULE: ----+----1-----2-----3-----4
7      Bird Feeder LG088 3 20
Item=Bird Feeder IDnum=. InStock=3 BackOrd=20
Total=23 _ERROR_=1 _N_=1

```

- a. A keyword was misspelled in the DATA step.
- b. A semicolon was missing from the INFILE statement.
- c. A variable was misspelled in the INPUT statement.
- d. A dollar sign was missing in the INPUT statement.

Chapter 8

Creating and Applying User-Defined Formats

The FORMAT Procedure	165
A Word About PROC FORMAT	165
The PROC FORMAT Statement	166
Permanently Storing Your Formats	166
Defining a Unique Format	167
The VALUE Statement	167
Specifying Value Ranges	168
Associating User-Defined Formats with Variables	169
How SAS Finds Format Catalogs	169
Assigning Formats to Variables	170
Displaying User-Defined Formats	171
Chapter Quiz	172

The FORMAT Procedure

A Word About PROC FORMAT

The FORMAT procedure stores user-defined informats and formats as entries in SAS catalogs. Formats determine how variable values are printed. For example, when the PRINT procedure displays the data set Work.Carsurvey, notice that the values for Sex, Income, and Color appear as numbers and letters, and therefore they are not easy to interpret.

Figure 8.1 *Work.Carsurvey Data Set (partial output)*

Obs	Age	Sex	Income	Color
1	19	1	14000	Y
2	45	1	65000	G
3	72	2	35000	B
4	31	1	44000	Y
5	58	2	83000	W

This chapter shows how to display more descriptive values for these variables.

The PROC FORMAT Statement

To begin a PROC FORMAT step, you use a PROC FORMAT statement.

Syntax, PROC FORMAT statement:

PROC FORMAT <options>;

options includes the following:

- **LIBRARY=libref** specifies the libref for a SAS library to contain a permanent catalog of user-defined formats
 - **FMTLIB** displays a list of all of the formats in your catalog, along with descriptions of their values.
-

Anytime you use PROC FORMAT to create a format, the format is stored in a format catalog. If the SAS library does not already contain a format catalog, SAS automatically creates one. If you do not specify the LIBRARY= option, the formats are stored in a default format catalog named Work.Formats.

The libref Work signifies that any format that is stored in Work.Formats is a temporary format; it exists only for the current SAS session.

Permanently Storing Your Formats

To store formats in a permanent format catalog named Library.Formats:

- Specify a LIBNAME statement that associates the libref with the permanent SAS library in which the format catalog is to be stored.

```
libname library 'c:\sas\formats\lib';
```

- Specify the LIBRARY= option in the PROC FORMAT statement and specify the libref Library.

```
PROC FORMAT LIBRARY=library;
```

Note: If you omit the LIBRARY=option, then formats and informats are stored in the Work.Formats catalog.

The LIBRARY= option accepts a libref and a catalog in the format *library.format*. When the LIBRARY= option specifies a libref and not a catalog, PROC FORMAT uses the catalog Formats.

When you associate a permanent format with a variable in a subsequent DATA or PROC step, use the Library libref to reference the location of the format catalog.

Any format that you create in this PROC FORMAT step is now stored in a permanent format catalog called Library.Formats.

```
libname library 'c:\sas\formats\lib';
proc format library=library;
    ...more SAS statements...
run;
```

In the program above, the catalog Library.Formats is located in the SAS library **c:\sas\formats\lib**, which is referenced by the libref Library.

Notice that LIB= is an acceptable abbreviation for the LIBRARY= option.

```
proc format lib=library;
```

Defining a Unique Format

The VALUE Statement

Use the VALUE statement to define a format for displaying one or more values.

Syntax, VALUE statement:

```
VALUE format-name
    range1='label1'
    range2='label2'
    ...more format-names...;
```

The following are true about *format-name*:

- A format name must begin with a dollar sign (\$) if the format applies to character data.
- A format name must be a valid SAS name.
- A format name cannot be the name of an existing SAS format.
- A format name cannot end in a number.
- A format name does not end in a period when specified in a VALUE statement.
- A numeric format name can be up to 32 characters long.
- A character format name can be up to 31 characters long.

Tip: If you are running a version of SAS before SAS®9, the format name must be a SAS name up to eight characters long and cannot end in a number.

Notice that the statement begins with the keyword VALUE and ends with a semicolon after all the labels have been defined. The following VALUE statements create the GENDER, AGEGROUP, and \$COL formats to specify descriptive labels that are later assigned to the variables Sex, Age, and Color respectively:

```
proc format;
    value gender
        1 = 'Male'
        2 = 'Female';
    value agegroup
        13 -< 20 = 'Teen'
        20 -< 65 = 'Adult'
        65 - HIGH = 'Senior';
    value $col
        'W' = 'Moon White'
        'B' = 'Sky Blue'
        'Y' = 'Sunburst Yellow'
        'G' = 'Rain Cloud Gray';
run;
```

The VALUE range specifies the following types of values:

- a single value, such as 24 or 'S'
- a range of numeric values, such as 0-1500
- a range of character values enclosed in quotation marks, such as 'A'-'M'

- a list of unique values separated by commas, such as 90,180,270 or 'B', 'D', 'F'. These values can be character values or numeric values, but not a combination of character and numeric values (because formats themselves are either character or numeric).

When the specified values are character values, they must be enclosed in quotation marks and must match the case of the variable's values. The format's name must also start with a dollar sign (\$). For example, the VALUE statement below defines the \$GRADE format, which displays the character values as text labels.

```
proc format lib=library;
  value $grade
    'A'='Good'
    'B'-'D'='Fair'
    'F'='Poor'
    'I','U'='See Instructor';
run;
```

When the specified values are numeric values, they are not enclosed in quotation marks, and the format's name should not begin with a dollar sign (\$).

Specifying Value Ranges

You can specify a non-inclusive range of numeric values by using the less than symbol (<) to avoid any overlapping. In this example, the range of values from 0 to less than 13 is labeled as child. The next range begins at 13, so the label teenager would be assigned to the values 13 to 19.

```
proc format lib=library;
  value agefmt
    0-<13='child'
    13-<20='teenager'
    20-<65='adult'
    65-100='senior citizen';
run;
```

You can also use the keywords LOW and HIGH to specify the lower and upper limits of a variable's value range. The keyword LOW does not include missing numeric values. The keyword OTHER can be used to label missing values as well as any values that are not specifically addressed in a range.

```
proc format lib=library;
  value agefmt
    low-<13='child'
    13-<20='teenager'
    20-<65='adult'
    65-high='senior citizen'
    other='unknown';
run;
```

TIP If applied to a character format, the keyword LOW includes missing character values.

When specifying a label for displaying each range, remember to do the following:

- Enclose the label in quotation marks.
- Limit the label to 32,767 characters.
- Use two single quotation marks if you want an apostrophe to appear in the label:

```
000='employee's jobtitle unknown';
```

To define several formats, you can use multiple VALUE statements in a single PROC FORMAT step. In this example, each VALUE statement defines a different format.

```
proc format;
  value gender
    1 = 'Male'
    2 = 'Female';
  value agegroup
    13 -< 20 = 'Teen'
    20 -< 65 = 'Adult'
    65 - HIGH = 'Senior';
  value $col
    'W' = 'Moon White'
    'B' = 'Sky Blue'
    'Y' = 'Sunburst Yellow'
    'G' = 'Rain Cloud Gray';
run;
```

The SAS log prints notes informing you that the formats have been created.

Log 8.1 SAS Log

```
146 proc format;
147     value gender    1 = 'Male'
148                     2 = 'Female';
NOTE: Format GENDER is already on the library WORK.FORMATS.
NOTE: Format GENDER has been output.
149     value agegroup 13 -< 20 = 'Teen'
150                     20 -< 65 = 'Adult'
151                     65 - HIGH = 'Senior';
NOTE: Format AGEGROUP is already on the library WORK.FORMATS.
NOTE: Format AGEGROUP has been output.
152     value $col     'W' = 'Moon White'
153                     'B' = 'Sky Blue'
154                     'Y' = 'Sunburst Yellow'
155                     'G' = 'Rain Cloud Gray';
NOTE: Format $COL is already on the library WORK.FORMATS.
NOTE: Format $COL has been output.
```

Because you have defined the GENDER, AGEGROUP, and \$COL formats for displaying the values of Sex, Age, and Color, the format can be used with PROC PRINT for more legible output.

Associating User-Defined Formats with Variables

How SAS Finds Format Catalogs

To use the GENDER, AGEGROUP, and \$COL formats in a subsequent SAS session, you must assign the libref Library again.

```
libname library 'c:\sas\formats\lib';
```

SAS searches for the formats GENDER, AGEGROUP, and \$COL in two libraries, in this order:

- the temporary library referenced by the libref Work

- a permanent library referenced by the libref Library

SAS uses the first instance of a specified format that it finds.

TIP You can delete formats using PROC CATALOG.

Assigning Formats to Variables

Just as with SAS formats, you associate a user-defined format with a variable in a FORMAT statement.

```
data carsurvey;
    infile 'Z:\sasuser\cars.dat';
    input Age Sex Income Color $;
    format Sex gender. Age agegroup. Color $col. Income DOLLAR8.;
run;
```

Remember, you can place the FORMAT statement in either a DATA step or a PROC step. By placing the FORMAT statement in a DATA step, you permanently associate a format with a variable. Note that you do not have to specify a width value when using a user-defined format.

When you submit the PRINT procedure, the output for carsurvey now shows descriptive labels instead of the values for Age, Sex, Income, and Color.

```
proc print data=carsurvey;
run;
```

Figure 8.2 Carsurvey Data Set with Formatted Values

Obs	Age	Sex	Income	Color
1	Teen	Male	\$14,000	Sunburst Yellow
2	Adult	Male	\$65,000	Rain Cloud Gray
3	Senior	Female	\$35,000	Sky Blue
4	Adult	Male	\$44,000	Sunburst Yellow
5	Adult	Female	\$83,000	Moon White

When associating a format with a variable, remember to do the following:

- Use the same format name in the FORMAT statement that you specified in the VALUE statement.
- Place a period at the end of the format name when it is used in the FORMAT statement.

If you do not format all of a variable's values, then those that are not listed in the VALUE statement are printed as they appear in the SAS data set, as shown in observation 3 of the figure below.

```
libname library 'c:\sas\formats\lib';
proc format lib=library;
    value gender
        1 = 'Male'
        2 = 'Female';
    value agegroup
        13 -< 20 = 'Teen'
        20 -< 65 = 'Adult';
```

```

value $col
    'W' = 'Moon White'
    'Y' = 'Sunburst Yellow'
    'G' = 'Rain Cloud Gray';

run;
data carsurvey;
    infile 'Z:\sasuser\cars.dat';
    input Age Sex Income Color $;
    format Sex gender. Age agegroup. Color $col. Income DOLLAR8.;
run;
proc print data=work.carsurvey;
run;

```

Figure 8.3 *Work.Carsurvey Data Set*

Obs	Age	Sex	Income	Color
1	Teen	Male	\$14,000	Sunburst Yellow
2	Adult	Male	\$65,000	Rain Cloud Gray
3	72	Female	\$35,000	B
4	Adult	Male	\$44,000	Sunburst Yellow
5	Adult	Female	\$83,000	Moon White

Displaying User-Defined Formats

When you build a large catalog of permanent formats, it can be easy to forget the exact spelling of a specific format name or its range of values. Adding the keyword `FMTLIB` to the `PROC FORMAT` statement displays a list of all the formats in your catalog, along with descriptions of their values.

```

libname library 'c:\sas\formats\lib';
proc format library=library fmtlib;
run;

```

When you submit this `PROC` step, a description of each format in your permanent catalog is displayed as output.

Figure 8.4 SAS Output: Format Description

FORMAT NAME: AGEGROUP LENGTH:6 NUMBER OF VALUES:3 MIN LENGTH:1 MAX LENGTH:40 DEFAULT LENGTH:6 FUZZ: STD		
START	END	LABEL (VER. 07 08 30MAR2017:13:26:59)
13	20	Teen
20	65	Adult
65	100	Senior

FORMAT NAME: GENDER LENGTH:6 NUMBER OF VALUES:2 MIN LENGTH:1 MAX LENGTH:40 DEFAULT LENGTH:6 FUZZ:STD		
START	END	LABEL (VER. 07 08 30MAR2017:13:26:59)
1	1	Male
2	2	Female

FORMAT NAME: \$COL LENGTH:15 NUMBER OF VALUES:4 MIN LENGTH:1 MAX LENGTH:40 DEFAULT LENGTH:15 FUZZ:0		
START	END	LABEL (VER. 07 08 30MAR2017:13:26:59)
B	B	Sky Blue
G	G	Rain Cloud Gray
W	W	Moon White
Y	Y	Sunburst Yellow

In addition to the name, range, and label, the format description includes the following details:

- length of the longest label
- number of values defined by this format
- version of SAS that was used to create the format
- date and time of creation

Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- Suppose you do not specify the LIBRARY= option and your formats are stored in Work.Formats. How long do they exist?
 - only for the current procedure
 - only for the current DATA step
 - only for the current SAS session
 - permanently
- Which of the following statements store your formats in a permanent catalog?
 - libname library 'c:\sas\formats\lib';
proc format lib=library

- ```

...;
b. libname library 'c:\sas\formats\lib';
 format lib=library
 ...;
c. library='c:\sas\formats\lib';
 proc format library
 ...;
d. library='c:\sas\formats\lib';
 proc library
 ...;

```
3. When you create a format with the VALUE statement, recall that the new format's name cannot end with a number, cannot end with a period, and cannot be the name of a SAS format. Which of the following is also true?
    - a. The name cannot be the name of a data set variable.
    - b. The name must be at least two characters long.
    - c. The name must be at least eight characters long.
    - d. The name must begin with a dollar sign (\$) if used with a character variable.
  4. Which of the following FORMAT procedures is written correctly?
    - a. 

```
proc format lib=library
 value colorfmt;
 1='Red'
 2='Green'
 3='Blue'

run;
```
    - b. 

```
proc format lib=library;
 value colorfmt
 1='Red'
 2='Green'
 3='Blue';

run;
```
    - c. 

```
proc format lib=library;
 value colorfmt;
 1='Red'
 2='Green'
 3='Blue'

run;
```
    - d. 

```
proc format lib=library;
 value colorfmt
 1='Red';
 2='Green';
 3='Blue';

run;
```
  5. Which of these statements is false regarding what the ranges in the VALUE statement can specify?
    - a. They can specify a single value, such as 24 or 'S'.
    - b. a range of numeric values, such as 0-1500.
    - c. a range of character values, such as 'A'-'M'.

- d. a list of numeric and character values separated by commas, such as 90,'B',180,'D',270.
6. How many characters can be used in a label?
- a. 96
  - b. 200
  - c. 256
  - d. 32,767
7. Which keyword can be used to label missing numeric values as well as any values that are not specified in a range?
- a. LOW
  - b. MISS
  - c. MISSING
  - d. OTHER
8. You can place the FORMAT statement in either a DATA step or a PROC step. What happens when you place it in a DATA step?
- a. You temporarily associate the formats with variables.
  - b. You permanently associate the formats with variables.
  - c. You replace the original data with the format labels.
  - d. You make the formats available to other data sets.
9. Suppose the format JOBFMT was created in a FORMAT procedure. Which FORMAT statement applies it to the variable JobTitle in the program output?
- a. `format jobtitle jobfmt;`
  - b. `format jobtitle jobfmt.;`
  - c. `format jobtitle=jobfmt;`
  - d. `format jobtitle='jobfmt';`
10. Which keyword, when added to the PROC FORMAT statement, displays all the formats in your catalog?
- a. CATALOG
  - b. LISTFMT
  - c. FMTCAT
  - d. FMTLIB



## Chapter 9

# Producing Descriptive Statistics

---

|                                                                |            |
|----------------------------------------------------------------|------------|
| <b>Computing Statistics Using PROC MEANS</b> .....             | <b>176</b> |
| What Does the MEANS Procedure Do? .....                        | 176        |
| The PROC MEANS Step .....                                      | 176        |
| <b>Selecting Statistics</b> .....                              | <b>177</b> |
| <b>Limiting Decimal Places</b> .....                           | <b>180</b> |
| <b>Specifying Variables in PROC MEANS</b> .....                | <b>181</b> |
| <b>Group Processing Using the CLASS Statement</b> .....        | <b>182</b> |
| <b>Group Processing Using the BY Statement</b> .....           | <b>183</b> |
| <b>Creating a Summarized Data Set Using PROC MEANS</b> .....   | <b>184</b> |
| OUTPUT Statement Syntax .....                                  | 184        |
| Specifying Statistics Using PROC MEANS .....                   | 185        |
| <b>Creating a Summarized Data Set Using PROC SUMMARY</b> ..... | <b>186</b> |
| Example: Creating an Output Data Set .....                     | 186        |
| <b>Producing Frequency Tables Using PROC FREQ</b> .....        | <b>187</b> |
| PROC FREQ Syntax .....                                         | 187        |
| <b>Specifying Variables in PROC FREQ</b> .....                 | <b>189</b> |
| TABLES Statement Syntax .....                                  | 189        |
| Example: Determining the Report Layout .....                   | 189        |
| <b>Creating Two-Way Tables</b> .....                           | <b>191</b> |
| <b>Creating N-Way Tables</b> .....                             | <b>192</b> |
| Overview .....                                                 | 192        |
| Determining the Table Layout .....                             | 193        |
| <b>Creating Tables in List Format</b> .....                    | <b>194</b> |
| LIST Option Syntax .....                                       | 194        |
| Example: Using the LIST Option .....                           | 195        |
| Changing the Table Format .....                                | 195        |
| Suppressing Table Information .....                            | 196        |
| Example: Suppress Percentages .....                            | 197        |
| <b>Chapter Quiz</b> .....                                      | <b>198</b> |

---

## Computing Statistics Using PROC MEANS

### *What Does the MEANS Procedure Do?*

The MEANS procedure provides data summarization tools to compute descriptive statistics for variables across all observations and within groups of observations. For example, PROC MEANS does the following:

- calculates descriptive statistics based on moments
- estimates quantiles, which includes the median
- calculates confidence limits for the mean
- identifies extreme values
- performs a *t* test

By default, PROC MEANS displays output.

### *The PROC MEANS Step*

The MEANS procedure can include many statements and options for specifying statistics.

---

Syntax, MEANS procedure:

```
PROC MEANS <DATA=SAS-data-set>
 <statistic-keyword(s)> <option(s)>;
RUN;
```

- *SAS-data-set* is the name of the data set to be analyzed.
  - *statistic-keyword(s)* specify the statistics to compute.
  - *option(s)* control the content, analysis, and appearance of output.
- 

In its simplest form, PROC MEANS prints the *n*-count (number of nonmissing values), the mean, the standard deviation, and the minimum and maximum values of every numeric variable in a data set.

```
proc means data=perm.survey;
run;
```

**Figure 9.1** Default PROC MEANS Output

| Variable | N | Mean      | Std Dev   | Minimum   | Maximum   |
|----------|---|-----------|-----------|-----------|-----------|
| Item1    | 4 | 3.7500000 | 1.2583057 | 2.0000000 | 5.0000000 |
| Item2    | 4 | 3.0000000 | 1.6329932 | 1.0000000 | 5.0000000 |
| Item3    | 4 | 4.2500000 | 0.5000000 | 4.0000000 | 5.0000000 |
| Item4    | 4 | 3.5000000 | 1.2909944 | 2.0000000 | 5.0000000 |
| Item5    | 4 | 3.0000000 | 1.6329932 | 1.0000000 | 5.0000000 |
| Item6    | 4 | 3.7500000 | 1.2583057 | 2.0000000 | 5.0000000 |
| Item7    | 4 | 3.0000000 | 1.8257419 | 1.0000000 | 5.0000000 |
| Item8    | 4 | 2.7500000 | 1.5000000 | 1.0000000 | 4.0000000 |
| Item9    | 4 | 3.0000000 | 1.4142136 | 2.0000000 | 5.0000000 |
| Item10   | 4 | 3.2500000 | 1.2583057 | 2.0000000 | 5.0000000 |
| Item11   | 4 | 3.0000000 | 1.8257419 | 1.0000000 | 5.0000000 |
| Item12   | 4 | 2.7500000 | 0.5000000 | 2.0000000 | 3.0000000 |
| Item13   | 4 | 2.7500000 | 1.5000000 | 1.0000000 | 4.0000000 |
| Item14   | 4 | 3.0000000 | 1.4142136 | 2.0000000 | 5.0000000 |
| Item15   | 4 | 3.0000000 | 1.6329932 | 1.0000000 | 5.0000000 |
| Item16   | 4 | 2.5000000 | 1.9148542 | 1.0000000 | 5.0000000 |
| Item17   | 4 | 3.0000000 | 1.1547005 | 2.0000000 | 4.0000000 |
| Item18   | 4 | 3.2500000 | 1.2583057 | 2.0000000 | 5.0000000 |

---

## Selecting Statistics

The default statistics that the MEANS procedure produces are not always the ones that you need. You might prefer to limit output to the mean of the values. Or you might need to compute a different statistic, such as the median or range of the values.

To specify statistics, include statistic keywords as options in the PROC MEANS statement. When you specify a statistic in the PROC MEANS statement, default statistics are not produced. For example, to determine the median and range of Perm.Survey numeric values, add the MEDIAN and RANGE keywords as options.

```
proc means data=perm.survey median range;
run;
```

**Figure 9.2** MEANS Procedure Output Displaying Median and Range

| Variable | Median    | Range     |
|----------|-----------|-----------|
| Item1    | 4.0000000 | 3.0000000 |
| Item2    | 3.0000000 | 4.0000000 |
| Item3    | 4.0000000 | 1.0000000 |
| Item4    | 3.5000000 | 3.0000000 |
| Item5    | 3.0000000 | 4.0000000 |
| Item6    | 4.0000000 | 3.0000000 |
| Item7    | 3.0000000 | 4.0000000 |
| Item8    | 3.0000000 | 3.0000000 |
| Item9    | 2.5000000 | 3.0000000 |
| Item10   | 3.0000000 | 3.0000000 |
| Item11   | 3.0000000 | 4.0000000 |
| Item12   | 3.0000000 | 1.0000000 |
| Item13   | 3.0000000 | 3.0000000 |
| Item14   | 2.5000000 | 3.0000000 |
| Item15   | 3.0000000 | 4.0000000 |
| Item16   | 2.0000000 | 4.0000000 |
| Item17   | 3.0000000 | 2.0000000 |
| Item18   | 3.0000000 | 3.0000000 |

Use the following keywords with PROC MEANS to compute statistics:

**Table 9.1** Descriptive Statistics

| Keyword  | Description                                   |
|----------|-----------------------------------------------|
| CLM      | Two-sided confidence limit for the mean       |
| CSS      | Corrected sum of squares                      |
| CV       | Coefficient of variation                      |
| KURTOSIS | Kurtosis                                      |
| LCLM     | One-sided confidence limit below the mean     |
| MAX      | Maximum value                                 |
| MEAN     | Average                                       |
| MIN      | Minimum value                                 |
| MODE     | Value that occurs most frequently             |
| N        | Number of observations with nonmissing values |
| NMISS    | Number of observations with missing values    |
| RANGE    | Range                                         |

| Keyword      | Description                               |
|--------------|-------------------------------------------|
| SKEWNESS     | Skewness                                  |
| STDDEV / STD | Standard deviation                        |
| STDERR       | Standard error of the mean                |
| SUM          | Sum                                       |
| SUMWGT       | Sum of the Weight variable values         |
| UCLM         | One-sided confidence limit above the mean |
| USS          | Uncorrected sum of squares                |
| VAR          | Variance                                  |

**Table 9.2** *Quantile Statistics*

| Keyword      | Description                                                   |
|--------------|---------------------------------------------------------------|
| MEDIAN / P50 | Median or 50th percentile                                     |
| P1           | 1st percentile                                                |
| P5           | 5th percentile                                                |
| P10          | 10th percentile                                               |
| Q1 / P25     | Lower quartile or 25th percentile                             |
| Q3 / P75     | Upper quartile or 75th percentile                             |
| P90          | 90th percentile                                               |
| P95          | 95th percentile                                               |
| P99          | 99th percentile                                               |
| QRANGE       | Difference between upper and lower interquartile range: Q3-Q1 |

**Table 9.3** *Hypothesis Testing*

| Keyword | Description                                                                    |
|---------|--------------------------------------------------------------------------------|
| PROBT   | Probability of a greater absolute value for the t value                        |
| T       | Student's t statistic for testing the hypothesis that the population mean is 0 |

## Limiting Decimal Places

By default, PROC MEANS uses the BESTw. format to display numeric values in the report.

When there is no format specification, SAS chooses the format that provides the most information about the value according to the available field width. At times, this can result in unnecessary decimal places, making your output hard to read.

```
proc means data=clinic.diabetes min max;
run;
```

**Figure 9.3** Variables Formatted with BESTw. Format

**The MEANS Procedure**

| Variable | Minimum     | Maximum     |
|----------|-------------|-------------|
| Age      | 15.0000000  | 63.0000000  |
| Height   | 61.0000000  | 75.0000000  |
| Weight   | 102.0000000 | 240.0000000 |
| Pulse    | 65.0000000  | 100.0000000 |
| FastGluc | 152.0000000 | 568.0000000 |
| PostGluc | 206.0000000 | 625.0000000 |

To limit decimal places, use the MAXDEC= option in the PROC MEANS statement, and set it equal to the length that you prefer.

Syntax, PROC MEANS statement with MAXDEC= option:

```
PROC MEANS <DATA=SAS-data-set>
 <statistic-keyword(s)> MAXDEC=n;
```

*n* specifies the maximum number of decimal places.

```
proc means data=clinic.diabetes min max maxdec=0;
run;
```

**Figure 9.4** Variables Formatted Using the MAXDEC= Option

**The MEANS Procedure**

| Variable | Minimum | Maximum |
|----------|---------|---------|
| Age      | 15      | 63      |
| Height   | 61      | 75      |
| Weight   | 102     | 240     |
| Pulse    | 65      | 100     |
| FastGluc | 152     | 568     |
| PostGluc | 206     | 625     |

## Specifying Variables in PROC MEANS

By default, the MEANS procedure generates statistics for every numeric variable in a data set. But the typical focus is on just a few variables, particularly if the data set is large. It also makes sense to exclude certain types of variables. The values of a numeric identifier variable ID, for example, are unlikely to yield useful statistics.

To specify the variables that PROC MEANS analyzes, add a VAR statement and list the variable names.

Syntax, VAR statement:

**VAR** *variable(s)*;

*variable(s)* lists numeric variables for which to calculate statistics.

```
proc means data=clinic.diabetes min max maxdec=0;
 var age height weight;
run;
```

**Figure 9.5** Output with Selected Variables Age, Height, and Weight

**The MEANS Procedure**

| Variable | Minimum | Maximum |
|----------|---------|---------|
| Age      | 15      | 63      |
| Height   | 61      | 75      |
| Weight   | 102     | 240     |

In addition to listing variables separately, you can use a numbered range of variables.

```
proc means data=perm.survey mean stderr maxdec=2;
 var item1-item5;
run;
```

**Figure 9.6** Output with a Range of Variables Selected

**The MEANS Procedure**

| Variable | Mean | Std Error |
|----------|------|-----------|
| Item1    | 3.75 | 0.63      |
| Item2    | 3.00 | 0.82      |
| Item3    | 4.25 | 0.25      |
| Item4    | 3.50 | 0.65      |
| Item5    | 3.00 | 0.82      |

## Group Processing Using the CLASS Statement

You often want statistics for groups of observations, rather than for the entire data set. For example, census numbers are more useful when grouped by region than when viewed as a national total. To produce separate analyses of grouped observations, add a CLASS statement to the MEANS procedure.

Syntax, CLASS statement:

**CLASS** *variable(s)*;

*variable(s)* specifies category variables for group processing.

CLASS variables are used to categorize data. CLASS variables can be either character or numeric, but they should contain a limited number of discrete values that represent meaningful groupings. If a CLASS statement is used, then the N Obs statistic is calculated. The N Obs statistic is based on the CLASS variables, as shown in the output below.

The output of the program shown below is grouped by values of the variables Survive and Sex. The order of the variables in the CLASS statement determines their order in the output table.

```
proc means data=clinic.heart maxdec=1;
 var arterial heart cardiac urinary;
 class survive sex;
run;
```

**Figure 9.7** Output Grouped by Values of Variables

| The MEANS Procedure |     |       |          |   |       |         |         |         |
|---------------------|-----|-------|----------|---|-------|---------|---------|---------|
| Survive             | Sex | N Obs | Variable | N | Mean  | Std Dev | Minimum | Maximum |
| DIED                | 1   | 4     | Arterial | 4 | 92.5  | 10.5    | 83.0    | 103.0   |
|                     |     |       | Heart    | 4 | 111.0 | 53.4    | 54.0    | 183.0   |
|                     |     |       | Cardiac  | 4 | 176.8 | 75.2    | 95.0    | 260.0   |
|                     |     |       | Urinary  | 4 | 98.0  | 186.1   | 0.0     | 377.0   |
|                     | 2   | 6     | Arterial | 6 | 94.2  | 27.3    | 72.0    | 145.0   |
|                     |     |       | Heart    | 6 | 103.7 | 16.7    | 81.0    | 130.0   |
|                     |     |       | Cardiac  | 6 | 318.3 | 102.6   | 156.0   | 424.0   |
|                     |     |       | Urinary  | 6 | 100.3 | 155.7   | 0.0     | 405.0   |
| SURV                | 1   | 5     | Arterial | 5 | 77.2  | 12.2    | 61.0    | 88.0    |
|                     |     |       | Heart    | 5 | 109.0 | 32.0    | 77.0    | 149.0   |
|                     |     |       | Cardiac  | 5 | 298.0 | 139.8   | 66.0    | 410.0   |
|                     |     |       | Urinary  | 5 | 100.8 | 60.2    | 44.0    | 200.0   |
|                     | 2   | 5     | Arterial | 5 | 78.8  | 6.8     | 72.0    | 87.0    |
|                     |     |       | Heart    | 5 | 100.0 | 13.4    | 84.0    | 111.0   |
|                     |     |       | Cardiac  | 5 | 330.2 | 87.0    | 256.0   | 471.0   |
|                     |     |       | Urinary  | 5 | 111.2 | 152.4   | 12.0    | 377.0   |



---

## Group Processing Using the BY Statement

Like the CLASS statement, the BY statement specifies variables to use for categorizing observations.

---

Syntax, BY statement:

**BY** *variable(s)*;

*variable(s)* specifies category variables for group processing.

---

But BY and CLASS differ in two key ways:

- Unlike CLASS processing, BY-group processing requires that your data already be sorted or indexed in the order of the BY variables. Unless data set observations are already sorted, you must run the SORT procedure before using PROC MEANS with any BY group.

**CAUTION:**

If you do not specify an output data set by using the OUT= option, PROC SORT overwrites the initial data set with newly sorted observations.

- The layout of BY-group results differs from the layout of CLASS group results. Note that the BY statement in the program below creates four small tables; a CLASS statement would produce a single large table.

```
proc sort data=clinic.heart out=work.heartsort;
 by survive sex;
run;
proc means data=work.heartsort maxdec=1;
 var arterial heart cardiac urinary;
 by survive sex;
run;
```

**Figure 9.8** BY Groups Created by PROC MEANS**The MEANS Procedure****Survive=DIED Sex=1**

| Variable | N | Mean  | Std Dev | Minimum | Maximum |
|----------|---|-------|---------|---------|---------|
| Arterial | 4 | 92.5  | 10.5    | 83.0    | 103.0   |
| Heart    | 4 | 111.0 | 53.4    | 54.0    | 183.0   |
| Cardiac  | 4 | 176.8 | 75.2    | 95.0    | 260.0   |
| Urinary  | 4 | 98.0  | 186.1   | 0.0     | 377.0   |

**Survive=DIED Sex=2**

| Variable | N | Mean  | Std Dev | Minimum | Maximum |
|----------|---|-------|---------|---------|---------|
| Arterial | 6 | 94.2  | 27.3    | 72.0    | 145.0   |
| Heart    | 6 | 103.7 | 16.7    | 81.0    | 130.0   |
| Cardiac  | 6 | 318.3 | 102.6   | 156.0   | 424.0   |
| Urinary  | 6 | 100.3 | 155.7   | 0.0     | 405.0   |

**Survive=SURV Sex=1**

| Variable | N | Mean  | Std Dev | Minimum | Maximum |
|----------|---|-------|---------|---------|---------|
| Arterial | 5 | 77.2  | 12.2    | 61.0    | 88.0    |
| Heart    | 5 | 109.0 | 32.0    | 77.0    | 149.0   |
| Cardiac  | 5 | 298.0 | 139.8   | 66.0    | 410.0   |
| Urinary  | 5 | 100.8 | 60.2    | 44.0    | 200.0   |

**Survive=SURV Sex=2**

| Variable | N | Mean  | Std Dev | Minimum | Maximum |
|----------|---|-------|---------|---------|---------|
| Arterial | 5 | 78.8  | 6.8     | 72.0    | 87.0    |
| Heart    | 5 | 100.0 | 13.4    | 84.0    | 111.0   |
| Cardiac  | 5 | 330.2 | 87.0    | 256.0   | 471.0   |
| Urinary  | 5 | 111.2 | 152.4   | 12.0    | 377.0   |

**TIP** The CLASS statement is easier to use than the BY statement because it does not require a sorting step. However, BY-group processing can be more efficient when your categories might contain many levels.

---

## Creating a Summarized Data Set Using PROC MEANS

**OUTPUT Statement Syntax**

To write summary statistics to a new data set, use the OUTPUT statement in the MEANS procedure.

Syntax, OUTPUT statement:

**OUTPUT OUT=***SAS-data-set statistic=variable(s);*

- **OUT=** specifies the name of the output data set.
- **statistic=** specifies which statistic to store in the output data set.
- **variable(s)** specifies the names of the variables to create. These variables represent the statistics for the analysis variables that are listed in the VAR statement.

When you use the OUTPUT statement, the summary statistics N, MEAN, STD, MIN, and MAX are produced for all of the numeric variables or for all of the variables that are listed in a VAR statement by default. To specify specific statistics to produce, use the **statistic=variable(s)** option in the OUTPUT statement.

## Specifying Statistics Using PROC MEANS

To specify specific statistics in the output data set, you must specify the statistic and then list all of the variables names that contain the value for one or more statistics that are specified in the VAR statement. The variables must be listed in the same order as in the VAR statement. You can specify more than one statistic in the OUTPUT statement.

The following program creates a typical PROC MEANS report and also creates a summarized output data set.

```
proc means data=clinic.diabetes;
 var age height weight;
 class sex;
 output out=work.sum_gender
 mean=AvgAge AvgHeight AvgWeight
 min=MinAge MinHeight MinWeight;
run;
```

**Figure 9.9** Report Created by PROC MEANS

| Sex | N Obs | Variable | N  | Mean        | Std Dev    | Minimum     | Maximum     |
|-----|-------|----------|----|-------------|------------|-------------|-------------|
| F   | 11    | Age      | 11 | 48.9090909  | 13.3075508 | 16.0000000  | 63.0000000  |
|     |       | Height   | 11 | 63.9090909  | 2.1191765  | 61.0000000  | 68.0000000  |
|     |       | Weight   | 11 | 150.4545455 | 18.4464828 | 102.0000000 | 168.0000000 |
| M   | 9     | Age      | 9  | 44.0000000  | 12.3895117 | 15.0000000  | 54.0000000  |
|     |       | Height   | 9  | 70.6666667  | 2.6457513  | 66.0000000  | 75.0000000  |
|     |       | Weight   | 9  | 204.2222222 | 30.2893454 | 140.0000000 | 240.0000000 |

To see the contents of the output data set, submit a PROC PRINT step.

```
proc print data=work.sum_gender;
run;
```

**Figure 9.10** Data Set Created by PROC PRINT

| Obs | Sex | _TYPE_ | _FREQ_ | AvgAge  | AvgHeight | AvgWeight | MinAge | MinHeight | MinWeight |
|-----|-----|--------|--------|---------|-----------|-----------|--------|-----------|-----------|
| 1   |     | 0      | 20     | 46.7000 | 66.9500   | 174.650   | 15     | 61        | 102       |
| 2   | F   | 1      | 11     | 48.9091 | 63.9091   | 150.455   | 16     | 61        | 102       |
| 3   | M   | 1      | 9      | 44.0000 | 70.6667   | 204.222   | 15     | 66        | 140       |

**TIP** You can use the NOPRINT option in the PROC MEANS statement to suppress the default report. For example, the following program creates only the output data set:

```
proc means data=clinic.diabetes noprint;
 var age height weight;
 class sex;
 output out=work.sum_gender
 mean=AvgAge AvgHeight AvgWeight;
run;
```

**TIP** In addition to the variables that you specify, PROC MEANS adds the \_TYPE\_ and \_FREQ\_ variables to the output data set. When no statistic keywords are specified, PROC MEANS also adds the variable \_STAT\_.

---

## Creating a Summarized Data Set Using PROC SUMMARY

You can also create a summarized output data set with PROC SUMMARY. When you use PROC SUMMARY, you use the same code to produce the output data set that you would use with PROC MEANS. The difference between the two procedures is that PROC MEANS produces a report by default. By contrast, to produce a report in PROC SUMMARY, you must include a PRINT option in the PROC SUMMARY statement.

### Example: Creating an Output Data Set

The following example creates an output data set but does not create a report:

```
proc summary data=clinic.diabetes;
 var age height weight;
 class sex;
 output out=work.sum_gender
 mean=AvgAge AvgHeight AvgWeight;
run;
```

If you placed a PRINT option in the PROC SUMMARY statement above, this program would produce the same report as though you replaced the word SUMMARY with MEANS.

```
proc summary data=clinic.diabetes print;
 var age height weight;
 class sex;
 output out=work.sum_gender
 mean=AvgAge AvgHeight AvgWeight;
run;
```

**Figure 9.11** Output Created by the SUMMARY Procedure with the PRINT Option

| Sex | N Obs | Variable | N  | Mean        | Std Dev    | Minimum     | Maximum     |
|-----|-------|----------|----|-------------|------------|-------------|-------------|
| F   | 11    | Age      | 11 | 48.9090909  | 13.3075508 | 16.0000000  | 63.0000000  |
|     |       | Height   | 11 | 63.9090909  | 2.1191765  | 61.0000000  | 68.0000000  |
|     |       | Weight   | 11 | 150.4545455 | 18.4464828 | 102.0000000 | 168.0000000 |
| M   | 9     | Age      | 9  | 44.0000000  | 12.3895117 | 15.0000000  | 54.0000000  |
|     |       | Height   | 9  | 70.6666667  | 2.6457513  | 66.0000000  | 75.0000000  |
|     |       | Weight   | 9  | 204.2222222 | 30.2893454 | 140.0000000 | 240.0000000 |

## Producing Frequency Tables Using PROC FREQ

The FREQ procedure is a descriptive procedure as well as a statistical procedure. It produces one-way and  $n$ -way frequency tables, and it concisely describes data by reporting the distribution of variable values. Use the FREQ procedure to create crosstabulation tables that summarize data for two or more categorical variables by showing the number of observations for each combination of variable values.

### PROC FREQ Syntax

The FREQ procedure can include many statements and options for controlling frequency output.

---

Syntax, FREQ procedure:

**PROC FREQ**

*<options>;*

**RUN;**

---

You can specify the following options in the PROC FREQ statement:

#### COMPRESS

begins display of the next one-way frequency table on the same page as the preceding one-way table if there is enough space to begin the table. By default, the next one-way table begins on the current page only if the entire table fits on that page. The COMPRESS option is not valid with PAGE=option.

#### FORMCHAR(1,2,7)= "*formchar-string*"

defines the characters to be used for constructing the outlines and dividers for the cells of crosstabulation table displays. *formchar-string* should be three characters long. The characters are used to draw the vertical separators (1), the horizontal separators (2), and the vertical-horizontal intersections (7). If you do not specify the FORMCHAR= option, PROC FREQ uses FORMCHAR(1,2,7)='|-+' by default.

#### NLEVELS

displays the "Number of Variable Levels" table, which provides the number of levels for each variable named in the TABLES statements.

#### NOPRINT

suppresses the display of all output. Use the NOPRINT option when all you want to create is an output data set.

ORDER=DATA | FORMATTED | FREQ | INTERNAL

specifies the order of the variable levels in the frequency and crosstabulation tables, which you request in the TABLES statement. By default, ORDER=INTERNAL. The FORMATTED and INTERNAL orders are machine-dependent. The ORDER= option does not apply to missing values, which are always ordered first. The ORDER= option can take the following values.

**Table 9.4** ORDER=Option

| Value of ORDER= | How Levels Are Ordered                                                                                                                 |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| DATA            | Order of appearance in the input data set                                                                                              |
| FORMATTED       | External formatted value, except for numeric variables with no explicit format, which are sorted by their unformatted (internal) value |
| FREQ            | Descending frequency count; levels with the most observations come first in the order                                                  |
| INTERNAL        | Unformatted value                                                                                                                      |

By default, PROC FREQ creates a one-way table with the frequency, percent, cumulative frequency, and cumulative for of all variables in a data set. This can produce excessive or inappropriate output. It is recommended that you always use a TABLES statement with PROC FREQ.

**Table 9.5** Default PROC FREQ Output

| Variable | Frequency                             | Percent                                                            | Cumulative Frequency                                                                       | Cumulative Percent                                                            |
|----------|---------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Value    | Number of observations with the value | Frequency of the value divided by the total number of observations | Sum of the frequency counts of the value and all other values listed above it in the table | Cumulative frequency of the value divided by the total number of observations |

For example, the following FREQ procedure creates a frequency table for each variable in the data set Finance.Loans. All the unique values are shown for the variables Department and Job Category.

```
proc freq data=finance.usa;
run;
```

**Figure 9.12** Frequency Table for Department and Job Category (partial HTML output)

**The FREQ Procedure**

| Dept  | Frequency | Percent | Cumulative Frequency | Cumulative Percent |
|-------|-----------|---------|----------------------|--------------------|
| ADM10 | 5         | 33.33   | 5                    | 33.33              |
| ADM20 | 4         | 26.67   | 9                    | 60.00              |
| ADM30 | 2         | 13.33   | 11                   | 73.33              |
| CAM10 | 3         | 20.00   | 14                   | 93.33              |
| CAM20 | 1         | 6.67    | 15                   | 100.00             |

| WageCat | Frequency | Percent | Cumulative Frequency | Cumulative Percent |
|---------|-----------|---------|----------------------|--------------------|
| H       | 1         | 6.67    | 1                    | 6.67               |
| S       | 14        | 93.33   | 15                   | 100.00             |

---

## Specifying Variables in PROC FREQ

### **TABLES Statement Syntax**

By default, the FREQ procedure creates frequency tables for every variable in a data set. But this is not always what you want. A variable that has continuous numeric values (such as DateTime) can result in a lengthy and meaningless table. Likewise, a variable that has a unique value for each observation (such as FullName) is unsuitable for PROC FREQ processing. Frequency distributions work best with variables whose values are categorical, and whose values are better summarized by counts rather than by averages.

To specify the variables to be processed by the FREQ procedure, include a TABLES statement.

---

Syntax, TABLES statement:

**TABLES** *variable(s)*;

*variable(s)* lists the variables to include.

---

### **Example: Determining the Report Layout**

The order in which the variables appear in the TABLES statement determines the order in which they are listed in the PROC FREQ report.

Consider the SAS data set Finance.Loans. The variables Rate and Months are categorical variables, so they are the best choices for frequency tables.

**Figure 9.13** PROC PRINT Report of the Data Set Finance.Loans

| Obs | Account  | Amount    | Rate   | Months | Payment  |
|-----|----------|-----------|--------|--------|----------|
| 1   | 101-1092 | \$22,000  | 10.00% | 60     | \$467.43 |
| 2   | 101-1731 | \$114,000 | 9.50%  | 360    | \$958.57 |
| 3   | 101-1289 | \$10,000  | 10.50% | 36     | \$325.02 |
| 4   | 101-3144 | \$3,500   | 10.50% | 12     | \$308.52 |
| 5   | 103-1135 | \$8,700   | 10.50% | 24     | \$403.47 |
| 6   | 103-1994 | \$18,500  | 10.00% | 60     | \$393.07 |
| 7   | 103-2335 | \$5,000   | 10.50% | 48     | \$128.02 |
| 8   | 103-3864 | \$87,500  | 9.50%  | 360    | \$735.75 |
| 9   | 103-3891 | \$30,000  | 9.75%  | 360    | \$257.75 |

```
proc freq data=finance.loans;
 tables rate months;
run;
```

**Figure 9.14** Frequency Tables for Rate and Months

### The SAS System

#### The FREQ Procedure

| Rate   | Frequency | Percent | Cumulative Frequency | Cumulative Percent |
|--------|-----------|---------|----------------------|--------------------|
| 9.50%  | 2         | 22.22   | 2                    | 22.22              |
| 9.75%  | 1         | 11.11   | 3                    | 33.33              |
| 10.00% | 2         | 22.22   | 5                    | 55.56              |
| 10.50% | 4         | 44.44   | 9                    | 100.00             |

| Months | Frequency | Percent | Cumulative Frequency | Cumulative Percent |
|--------|-----------|---------|----------------------|--------------------|
| 12     | 1         | 11.11   | 1                    | 11.11              |
| 24     | 1         | 11.11   | 2                    | 22.22              |
| 36     | 1         | 11.11   | 3                    | 33.33              |
| 48     | 1         | 11.11   | 4                    | 44.44              |
| 60     | 2         | 22.22   | 6                    | 66.67              |
| 360    | 3         | 33.33   | 9                    | 100.00             |

In addition to listing variables separately, you can use a numbered range of variables.

```
proc freq data=perm.survey;
 tables item1-item3;
run;
```



**Figure 9.15** Frequency Tables for Item1–Item3

| The SAS System     |           |         |                      |                    |
|--------------------|-----------|---------|----------------------|--------------------|
| The FREQ Procedure |           |         |                      |                    |
| Item1              | Frequency | Percent | Cumulative Frequency | Cumulative Percent |
| 2                  | 1         | 25.00   | 1                    | 25.00              |
| 4                  | 2         | 50.00   | 3                    | 75.00              |
| 5                  | 1         | 25.00   | 4                    | 100.00             |

| Item2 | Frequency | Percent | Cumulative Frequency | Cumulative Percent |
|-------|-----------|---------|----------------------|--------------------|
| 1     | 1         | 25.00   | 1                    | 25.00              |
| 3     | 2         | 50.00   | 3                    | 75.00              |
| 5     | 1         | 25.00   | 4                    | 100.00             |

| Item3 | Frequency | Percent | Cumulative Frequency | Cumulative Percent |
|-------|-----------|---------|----------------------|--------------------|
| 4     | 3         | 75.00   | 3                    | 75.00              |
| 5     | 1         | 25.00   | 4                    | 100.00             |

**TIP** To suppress the display of cumulative frequencies and cumulative percentages in one-way frequency tables and in list output, add the NOCUM option to your TABLES statement. Here is the syntax:

**TABLES** *variable(s)* / **NOCUM**;

---

## Creating Two-Way Tables

As noted earlier, you often want statistics for grouped data. It is possible to crosstabulate frequencies of two or more variables. For example, census data is typically crosstabulated with a variable that represents geographical regions.

The simplest crosstabulation is a two-way table. To create a two-way table, join two variables with an asterisk (\*) in the TABLES statement of a PROC FREQ step.

---

Syntax, TABLES statement for crosstabulation:

**TABLES** *variable-1* \**variable-2* < \* ... *variable-n* >;

Here are the options for two-way tables:

- *variable-1* specifies table rows.
- *variable-2* specifies table columns.

---

When crosstabulations are specified, PROC FREQ produces tables with cells that contain the following frequencies:

- cell frequency
- cell percentage of total frequency
- cell percentage of row frequency
- cell percentage of column frequency

For example, the following program creates the two-way table shown below.

```
proc format;
 value wtfmt low-139='< 140'
 140-180='140-180'
 181-high='> 180';
 value htfmt low-64='< 5''5"'
 65-70='5''5-10"'
 71-high='> 5''10"';
run;
proc freq data=clinic.diabetes;
 tables weight*height;
 format weight wtfmt. height htfmt.;
run;
```

**Figure 9.16** Two-Way Table Created by PROC FREQ

| The SAS System                             |                               |                              |                               |              |
|--------------------------------------------|-------------------------------|------------------------------|-------------------------------|--------------|
| The FREQ Procedure                         |                               |                              |                               |              |
| Frequency<br>Percent<br>Row Pct<br>Col Pct | Table of Weight by Height     |                              |                               |              |
|                                            | Height                        |                              |                               |              |
| Weight                                     | <5'5"                         | 5'5-10"                      | >5'10"                        | Total        |
| <140                                       | 2<br>10.00<br>100.00<br>28.57 | 0<br>0.00<br>0.00<br>0.00    | 0<br>0.00<br>0.00<br>0.00     | 2<br>10.00   |
| 140-180                                    | 5<br>25.00<br>50.00<br>71.43  | 5<br>25.00<br>50.00<br>62.50 | 0<br>0.00<br>0.00<br>0.00     | 10<br>50.00  |
| >180                                       | 0<br>0.00<br>0.00<br>0.00     | 3<br>15.00<br>37.50<br>37.50 | 5<br>25.00<br>62.50<br>100.00 | 8<br>40.00   |
| Total                                      | 7<br>35.00                    | 8<br>40.00                   | 5<br>25.00                    | 20<br>100.00 |

Note that the first variable, Weight, forms the table rows, and the second variable, Height, forms the columns. Reversing the order of the variables in the TABLES statement would reverse their positions in the table. Note also that the statistics are listed in the legend box.

## Creating N-Way Tables

### Overview

For a frequency analysis of more than two variables, use PROC FREQ to create *n*-way crosstabulations. A series of two-way tables is produced, with a table for each level of the other variables.

For example, suppose you want to add the variable Sex to your crosstabulation of Weight and Height in the data set Clinic.Diabetes. Add Sex to the TABLES statement, joined to the other variables with an asterisk (\*).

```
tables sex*weight*height;
```

### Determining the Table Layout

The order of the variables is important. In  $n$ -way tables, the last two variables of the TABLES statement become the two-way rows and columns. Variables that precede the last two variables in the TABLES statement stratify the crosstabulation tables.

```

levels
 ↓
tables sex*weight*height;
 ↑ ↑
 rows + columns = two-way tables

```

Notice the structure of the output that is produced by the program shown below.

```

proc format;
 value wtfmt low-139='< 140'
 140-180='140-180'
 181-high='> 180';
 value htfmt low-64='< 5''5"'
 65-70='5''5-10"'
 71-high='> 5''10"';
run;
proc freq data=clinic.diabetes;
 tables sex*weight*height;
 format weight wtfmt. height htfmt.;
run;

```

Figure 9.17 Creating N-Way Tables

**The SAS System**  
**The FREQ Procedure**

| Frequency<br>Percent<br>Row Pct<br>Col Pct | Table 1 of Weight by Height |        |         |        |        |
|--------------------------------------------|-----------------------------|--------|---------|--------|--------|
|                                            | Controlling for Sex=F       |        |         |        |        |
|                                            | Weight                      | Height |         |        | Total  |
|                                            |                             | <5'5"  | 5'5-10" | >5'10" |        |
| <140                                       |                             | 2      | 0       | 0      | 2      |
|                                            |                             | 18.18  | 0.00    | 0.00   | 18.18  |
|                                            |                             | 100.00 | 0.00    | 0.00   |        |
|                                            |                             | 28.57  | 0.00    | .      |        |
| 140-180                                    |                             | 5      | 4       | 0      | 9      |
|                                            |                             | 45.45  | 36.36   | 0.00   | 81.82  |
|                                            |                             | 55.56  | 44.44   | 0.00   |        |
|                                            |                             | 71.43  | 100.00  | .      |        |
| >180                                       |                             | 0      | 0       | 0      | 0      |
|                                            |                             | 0.00   | 0.00    | 0.00   | 0.00   |
|                                            |                             | .      | .       | .      |        |
|                                            |                             | 0.00   | 0.00    | .      |        |
| Total                                      |                             | 7      | 4       | 0      | 11     |
|                                            |                             | 63.64  | 36.36   | 0.00   | 100.00 |
|                                            |                             |        |         |        |        |
|                                            |                             |        |         |        |        |

| Frequency<br>Percent<br>Row Pct<br>Col Pct | Table 2 of Weight by Height |        |         |        |        |
|--------------------------------------------|-----------------------------|--------|---------|--------|--------|
|                                            | Controlling for Sex=M       |        |         |        |        |
|                                            | Weight                      | Height |         |        | Total  |
|                                            |                             | <5'5"  | 5'5-10" | >5'10" |        |
| <140                                       |                             | 0      | 0       | 0      | 0      |
|                                            |                             | 0.00   | 0.00    | 0.00   | 0.00   |
|                                            |                             | .      | 0.00    | 0.00   |        |
|                                            |                             | .      | .       | .      |        |
| 140-180                                    |                             | 0      | 1       | 0      | 1      |
|                                            |                             | 0.00   | 11.11   | 0.00   | 11.11  |
|                                            |                             | 0.00   | 100.00  | 0.00   |        |
|                                            |                             | .      | 25.00   | 0.00   |        |
| >180                                       |                             | 0      | 3       | 5      | 8      |
|                                            |                             | 0.00   | 33.33   | 55.56  | 88.89  |
|                                            |                             | 0.00   | 37.50   | 62.50  |        |
|                                            |                             | .      | 75.00   | 100.00 |        |
| Total                                      |                             | 0      | 4       | 5      | 9      |
|                                            |                             | 0.00   | 44.44   | 55.56  | 100.00 |
|                                            |                             |        |         |        |        |
|                                            |                             |        |         |        |        |

## Creating Tables in List Format

### LIST Option Syntax

When three or more variables are specified, the multiple levels of  $n$ -way tables can produce considerable output. Such bulky, often complex crosstabulations are often easier to read when they are arranged as a continuous list. Although this arrangement eliminates row and column frequencies and percentages, the results are compact and clear.

**TIP** The LIST option is not available when you also specify statistical options.

To generate list output for crosstabulations, add a slash (/) and the LIST option to the TABLES statement in your PROC FREQ step. Here is the syntax:

**TABLES** *variable-1* \**variable-2* <\* ... *variable-n*> / **LIST**;

**Example: Using the LIST Option**

Adding the LIST option to the Clinic.Diabetes program puts its frequencies in a simple, short table.

```
proc format;
 value wtfmt low-139='< 140'
 140-180='140-180'
 181-high='> 180';
 value htfmt low-64='< 5''5"'
 65-70='5''5-10"'
 71-high='> 5''10"';
run;
proc freq data=clinic.diabetes;
 tables sex*weight*height / list;
 format weight wtfmt. height htfmt.;
run;
```

**Figure 9.18** Table Created by Using the LIST Option

**The SAS System**  
**The FREQ Procedure**

| Sex | Weight  | Height  | Frequency | Percent | Cumulative Frequency | Cumulative Percent |
|-----|---------|---------|-----------|---------|----------------------|--------------------|
| F   | <140    | <5'5"   | 2         | 10.00   | 2                    | 10.00              |
| F   | 140-180 | <5'5"   | 5         | 25.00   | 7                    | 35.00              |
| F   | 140-180 | 5'5-10" | 4         | 20.00   | 11                   | 55.00              |
| M   | 140-180 | 5'5-10" | 1         | 5.00    | 12                   | 60.00              |
| M   | >180    | 5'5-10" | 3         | 15.00   | 15                   | 75.00              |
| M   | >180    | >5'10"  | 5         | 25.00   | 20                   | 100.00             |

**Changing the Table Format**

Adding the CROSSLIST option to a TABLES statement displays crosstabulation tables in ODS column format. This option creates a table that has a table definition that you can customize by using the TEMPLATE procedure.

Notice the structure of the output that is produced by the program shown below.

```
proc format;
 value wtfmt low-139='< 140'
 140-180='140-180'
 181-high='> 180';
 value htfmt low-64='< 5''5"'
 65-70='5''5-10"'
 71-high='> 5''10"';
run;
proc freq data=clinic.diabetes;
 tables sex*weight*height / crosstlist;
 format weight wtfmt. height htfmt.;
run;
```

**Figure 9.19** Table Created by Using CROSSLIST Option: Sex=F

**The FREQ Procedure**

| Table of Weight by Height |         |           |         |             |                |
|---------------------------|---------|-----------|---------|-------------|----------------|
| Controlling for Sex=F     |         |           |         |             |                |
| Weight                    | Height  | Frequency | Percent | Row Percent | Column Percent |
| <140                      | <5'5"   | 2         | 18.18   | 100.00      | 28.57          |
|                           | 5'5-10" | 0         | 0.00    | 0.00        | 0.00           |
|                           | >5'10"  | 0         | 0.00    | 0.00        | .              |
|                           | Total   | 2         | 18.18   | 100.00      |                |
| 140-180                   | <5'5"   | 5         | 45.45   | 55.56       | 71.43          |
|                           | 5'5-10" | 4         | 36.36   | 44.44       | 100.00         |
|                           | >5'10"  | 0         | 0.00    | 0.00        | .              |
|                           | Total   | 9         | 81.82   | 100.00      |                |
| >180                      | <5'5"   | 0         | 0.00    | .           | 0.00           |
|                           | 5'5-10" | 0         | 0.00    | .           | 0.00           |
|                           | >5'10"  | 0         | 0.00    | .           | .              |
|                           | Total   | 0         | 0.00    | .           |                |
| Total                     | <5'5"   | 7         | 63.64   |             | 100.00         |
|                           | 5'5-10" | 4         | 36.36   |             | 100.00         |
|                           | >5'10"  | 0         | 0.00    |             | .              |
|                           | Total   | 11        | 100.00  |             |                |

**Figure 9.20** Table Created by Using CROSSLIST Option: Sex=M

| Table of Weight by Height |         |           |         |             |                |
|---------------------------|---------|-----------|---------|-------------|----------------|
| Controlling for Sex=M     |         |           |         |             |                |
| Weight                    | Height  | Frequency | Percent | Row Percent | Column Percent |
| <140                      | <5'5"   | 0         | 0.00    | .           | .              |
|                           | 5'5-10" | 0         | 0.00    | .           | 0.00           |
|                           | >5'10"  | 0         | 0.00    | .           | 0.00           |
|                           | Total   | 0         | 0.00    | .           |                |
| 140-180                   | <5'5"   | 0         | 0.00    | 0.00        | .              |
|                           | 5'5-10" | 1         | 11.11   | 100.00      | 25.00          |
|                           | >5'10"  | 0         | 0.00    | 0.00        | 0.00           |
|                           | Total   | 1         | 11.11   | 100.00      |                |
| >180                      | <5'5"   | 0         | 0.00    | 0.00        | .              |
|                           | 5'5-10" | 3         | 33.33   | 37.50       | 75.00          |
|                           | >5'10"  | 5         | 55.56   | 62.50       | 100.00         |
|                           | Total   | 8         | 88.89   | 100.00      |                |
| Total                     | <5'5"   | 0         | 0.00    |             | .              |
|                           | 5'5-10" | 4         | 44.44   |             | 100.00         |
|                           | >5'10"  | 5         | 55.56   |             | 100.00         |
|                           | Total   | 9         | 100.00  |             |                |

### Suppressing Table Information

Another way to control the format of crosstabulations is to limit the output of the FREQ procedure to a few specific statistics. Remember that when crosstabulations are run, PROC FREQ produces tables with cells that contain these frequencies:

- cell frequency
- cell percentage of total frequency
- cell percentage of row frequency
- cell percentage of column frequency

You can use options to suppress any of these statistics. To control the depth of crosstabulation results, add any combination of the following options to the TABLES statement:

- NOFREQ suppresses cell frequencies
- NOPERCENT suppresses cell percentages
- NOROW suppresses row percentages
- NOCOL suppresses column percentages

### ***Example: Suppress Percentages***

Suppose you want to use only the percentages of Sex and Weight combinations in the data set Clinic.Diabetes. To suppress frequency counts and row and column percentages, add the NOFREQ, NOROW, and NOCOL options to the program's TABLES statement.

```
proc format;
 value wtfmt low-139='< 140'
 140-180='140-180'
 181-high='> 180';
run;
proc freq data=clinic.diabetes;
 tables sex*weight / nofreq norow nocol;
 format weight wtfmt.;
run;
```

**Figure 9.21** Suppressing Table Information

| The SAS System     |                             |            |           |              |
|--------------------|-----------------------------|------------|-----------|--------------|
| The FREQ Procedure |                             |            |           |              |
| Percent            | Table 1 of Weight by Height |            |           |              |
|                    | Controlling for Sex=F       |            |           |              |
|                    | Height                      |            |           |              |
| Weight             | <5'5"                       | 5'5-10"    | >5'10"    | Total        |
| <140               | 18.18                       | 0.00       | 0.00      | 18.18        |
| 140-180            | 45.45                       | 36.36      | 0.00      | 81.82        |
| >180               | 0.00                        | 0.00       | 0.00      | 0.00         |
| Total              | 7<br>63.64                  | 4<br>36.36 | 0<br>0.00 | 11<br>100.00 |

|         |                             |            |            |             |
|---------|-----------------------------|------------|------------|-------------|
| Percent | Table 2 of Weight by Height |            |            |             |
|         | Controlling for Sex=M       |            |            |             |
|         | Height                      |            |            |             |
| Weight  | <5'5"                       | 5'5-10"    | >5'10"     | Total       |
| <140    | 0.00                        | 0.00       | 0.00       | 0.00        |
| 140-180 | 0.00                        | 11.11      | 0.00       | 11.11       |
| >180    | 0.00                        | 33.33      | 55.56      | 88.89       |
| Total   | 0<br>0.00                   | 4<br>44.44 | 5<br>55.56 | 9<br>100.00 |

Notice that Percent is the only statistic that remains in the table's legend box.

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- The default statistics produced by the MEANS procedure are *n*-count, mean, minimum, maximum, and which one of the following statistics:
  - median
  - range
  - standard deviation
  - standard error of the mean
- Which statement limits a PROC MEANS analysis to the variables Boarded, Transfer, and Deplane?
  - by boarded transfer deplane;
  - class boarded transfer deplane;
  - output boarded transfer deplane;
  - var boarded transfer deplane;
- The data set Survey.Health includes the following numeric variables. Which is a poor candidate for PROC MEANS analysis?



- a. IDnum
  - b. Age
  - c. Height
  - d. Weight
4. Which of the following statements is true regarding BY-group processing?
    - a. BY variables must be either indexed or sorted.
    - b. Summary statistics are computed for BY variables.
    - c. BY-group processing is preferred when you are categorizing data that contains few variables.
    - d. BY-group processing overwrites your data set with the newly grouped observations.
  5. Which group processing statement produced the PROC MEANS output shown below?

**The MEANS Procedure**

| Survive | Sex | N Obs | Variable | N | Mean        | Std Dev     | Minimum     | Maximum     |
|---------|-----|-------|----------|---|-------------|-------------|-------------|-------------|
| DIED    | 1   | 4     | Arterial | 4 | 92.5000000  | 10.4721854  | 83.0000000  | 103.0000000 |
|         |     |       | Heart    | 4 | 111.0000000 | 53.4103610  | 54.0000000  | 183.0000000 |
|         |     |       | Cardiac  | 4 | 176.7500000 | 75.2257713  | 95.0000000  | 260.0000000 |
|         |     |       | Urinary  | 4 | 98.0000000  | 186.1343601 | 0           | 377.0000000 |
|         | 2   | 6     | Arterial | 6 | 94.1666667  | 27.3160514  | 72.0000000  | 145.0000000 |
|         |     |       | Heart    | 6 | 103.6666667 | 16.6573307  | 81.0000000  | 130.0000000 |
|         |     |       | Cardiac  | 6 | 318.3333333 | 102.6034437 | 156.0000000 | 424.0000000 |
|         |     |       | Urinary  | 6 | 100.3333333 | 155.7134120 | 0           | 405.0000000 |
| SURV    | 1   | 5     | Arterial | 5 | 77.2000000  | 12.1942609  | 61.0000000  | 88.0000000  |
|         |     |       | Heart    | 5 | 109.0000000 | 31.9687347  | 77.0000000  | 149.0000000 |
|         |     |       | Cardiac  | 5 | 298.0000000 | 139.8499196 | 66.0000000  | 410.0000000 |
|         |     |       | Urinary  | 5 | 100.8000000 | 60.1722527  | 44.0000000  | 200.0000000 |
|         | 2   | 5     | Arterial | 5 | 78.8000000  | 6.8337398   | 72.0000000  | 87.0000000  |
|         |     |       | Heart    | 5 | 100.0000000 | 13.3790882  | 84.0000000  | 111.0000000 |
|         |     |       | Cardiac  | 5 | 330.2000000 | 86.9839066  | 256.0000000 | 471.0000000 |
|         |     |       | Urinary  | 5 | 111.2000000 | 152.4096454 | 12.0000000  | 377.0000000 |

- a. class sex survive;
  - b. class survive sex;
  - c. by sex survive;
  - d. by survive sex;
6. Which program can be used to create the following output?

| Sex | N Obs | Variable | N  | Mean        | Std Dev    | Minimum     | Maximum     |
|-----|-------|----------|----|-------------|------------|-------------|-------------|
| F   | 11    | Age      | 11 | 48.9090909  | 13.3075508 | 16.0000000  | 63.0000000  |
|     |       | Height   | 11 | 63.9090909  | 2.1191765  | 61.0000000  | 68.0000000  |
|     |       | Weight   | 11 | 150.4545455 | 18.4464828 | 102.0000000 | 168.0000000 |
| M   | 9     | Age      | 9  | 44.0000000  | 12.3895117 | 15.0000000  | 54.0000000  |
|     |       | Height   | 9  | 70.6666667  | 2.6457513  | 66.0000000  | 75.0000000  |
|     |       | Weight   | 9  | 204.2222222 | 30.2893454 | 140.0000000 | 240.0000000 |

- a. 

```
proc means data=clinic.diabetes;
 var age height weight;
 class sex;
 output out=work.sum_gender
 mean=AvgAge AvgHeight AvgWeight;
run;
```
  - b. 

```
proc summary data=clinic.diabetes print;
 var age height weight; class sex;
 output out=work.sum_gender
 mean=AvgAge AvgHeight AvgWeight;
run;
```
  - c. 

```
proc means data=clinic.diabetes noprint;
 var age height weight;
 class sex;
 output out=work.sum_gender
 mean=AvgAge AvgHeight AvgWeight;
run;
```
  - d. Both a and b.
7. By default, PROC FREQ creates a table of frequencies and percentages for which data set variables?
    - a. character variables
    - b. numeric variables
    - c. both character and numeric variables
    - d. none: variables must always be specified
  8. Frequency distributions work best with variables that contain which types of values?
    - a. continuous values
    - b. numeric values
    - c. categorical values
    - d. unique values
  9. Which PROC FREQ step produced this two-way table?

## The FREQ Procedure

| Frequency | Table of Weight by Height |        |         |         |
|-----------|---------------------------|--------|---------|---------|
| Percent   | Weight                    | Height |         |         |
| Row Pct   |                           | < 5'5" | 5'5-10" | > 5'10" |
| Col Pct   |                           |        |         | Total   |
| < 140     | 2                         | 0      | 0       | 2       |
|           | 10.00                     | 0.00   | 0.00    | 10.00   |
|           | 100.00                    | 0.00   | 0.00    |         |
|           | 28.57                     | 0.00   | 0.00    |         |
| 140-180   | 5                         | 5      | 0       | 10      |
|           | 25.00                     | 25.00  | 0.00    | 50.00   |
|           | 50.00                     | 50.00  | 0.00    |         |
|           | 71.43                     | 62.50  | 0.00    |         |
| > 180     | 0                         | 3      | 5       | 8       |
|           | 0.00                      | 15.00  | 25.00   | 40.00   |
|           | 0.00                      | 37.50  | 62.50   |         |
|           | 0.00                      | 37.50  | 100.00  |         |
| Total     | 7                         | 8      | 5       | 20      |
|           | 35.00                     | 40.00  | 25.00   | 100.00  |

- a. 

```
proc freq data=clinic.diabetes;
 tables height weight;
 format height htfmt. weight wtfmt.;
run;
```
- b. 

```
proc freq data=clinic.diabetes;
 tables weight height;
 format weight wtfmt. height htfmt.;
run;
```
- c. 

```
proc freq data=clinic.diabetes;
 tables height*weight;
 format height htfmt. weight wtfmt.;
run;
```
- d. 

```
proc freq data=clinic.diabetes;
 tables weight*height;
 format weight wtfmt. height htfmt.;
run;
```

10. Which PROC FREQ step produced this table?

## The FREQ Procedure

| Percent                |        |         |       |        |
|------------------------|--------|---------|-------|--------|
| Table of Sex by Weight |        |         |       |        |
| Sex                    | Weight |         |       |        |
|                        | < 140  | 140-180 | > 180 | Total  |
| F                      | 10.00  | 45.00   | 0.00  | 55.00  |
| M                      | 0.00   | 5.00    | 40.00 | 45.00  |
| Total                  | 2      | 10      | 8     | 20     |
|                        | 10.00  | 50.00   | 40.00 | 100.00 |

- a. 

```
proc freq data=clinic.diabetes;
 tables sex weight / list;
 format weight wtfmt.;
run;
```
- b. 

```
proc freq data=clinic.diabetes;
 tables sex*weight / nocol;
 format weight wtfmt.;
run;
```
- c. 

```
proc freq data=clinic.diabetes;
 tables sex weight / norow nocol;
 format weight wtfmt.;
run;
```
- d. 

```
proc freq data=clinic.diabetes;
 tables sex*weight / nofreq norow nocol;
 format weight wtfmt.;
run;
```

## Chapter 10

# Creating Output with ODS

---

|                                                                         |            |
|-------------------------------------------------------------------------|------------|
| <b>The Output Delivery System (ODS)</b> .....                           | <b>204</b> |
| Overview of ODS .....                                                   | 204        |
| Opening and Closing ODS Destinations .....                              | 204        |
| Using Statements to Open and Close ODS Destinations .....               | 205        |
| <b>Creating HTML Output with ODS</b> .....                              | <b>206</b> |
| The ODS HTML Statement .....                                            | 206        |
| Example: Creating Output with PROC PRINT .....                          | 207        |
| Creating HTML Output with a Table of Contents .....                     | 208        |
| Using Options to Specify Links and Paths .....                          | 211        |
| Changing the Appearance of HTML Output .....                            | 214        |
| <b>Creating PDF Output with ODS</b> .....                               | <b>215</b> |
| The ODS PDF Statement .....                                             | 215        |
| The ODS Printer Family of Statements .....                              | 216        |
| Opening and Closing the PDF Destination .....                           | 216        |
| Working with the Table of Contents .....                                | 216        |
| Example: Creating PDF Output Using the FILE= Option .....               | 217        |
| Example: Creating a Printable Table of Contents .....                   | 218        |
| Changing the Appearance of PDF Output .....                             | 219        |
| <b>Creating RTF Output with ODS</b> .....                               | <b>220</b> |
| The ODS RTF Statement .....                                             | 220        |
| Opening and Closing the RTF Destination .....                           | 221        |
| Understanding How RTF Formats Output .....                              | 221        |
| ODS RTF and Graphics .....                                              | 221        |
| Example: Using the STYLE= Option (FestivalPrinter Style) .....          | 222        |
| <b>Creating EXCEL Output with ODS</b> .....                             | <b>222</b> |
| The ODS EXCEL Statement .....                                           | 222        |
| Details About the ODS Destination .....                                 | 223        |
| Example: Customizing Your Excel Output .....                            | 223        |
| Example: Customizing ODS Excel Output Using TAGATTR Style Attribute ... | 225        |
| Example: Applying a Style Sheet to Excel Output .....                   | 226        |
| <b>Chapter Quiz</b> .....                                               | <b>228</b> |

## The Output Delivery System (ODS)

### Overview of ODS

The SAS Output Delivery System (ODS) gives you greater flexibility in generating, storing, and reproducing SAS procedure and DATA step output along with a wide range of formatting options. ODS provides formatting functionality that is not available when you are using either individual procedures or the DATA step without ODS enhancements.

ODS enables you to create reports for popular software applications. For example, use the ODS PDF statement to create PDF files for viewing with Adobe Acrobat or for printing. With ODS, you can easily create output in a variety of formats, including EXCEL, HTML, PDF, and RTF.

### Opening and Closing ODS Destinations

You use ODS statements to specify destinations for your output. Each destination creates a specific type of formatted output. The following table lists some of the ODS destinations that are currently supported.

| Destination                    | Result                                                                                                                                                   |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Document                       | a hierarchy of output objects that enables you to render multiple ODS output without rerunning procedures.                                               |
| EXCEL                          | writes Excel spreadsheet files that are compatible with Microsoft Office 2010 and later versions.                                                        |
| HTML                           | output that is formatted in Hypertext Markup Language (HTML). You do not have to specify the ODS HTML statement to produce basic HTML output.            |
| LISTING                        | plain text output that is formatted like traditional SAS procedure (LISTING) output.                                                                     |
| Markup Languages Family        | output that is formatted using markup languages such as Extensible Markup Language (XML).                                                                |
| Output                         | SAS data sets.                                                                                                                                           |
| Printer Family (PDF and so on) | output that is formatted for a high-resolution printer such as PostScript (PS), Portable Document Format (PDF), or Printer Control Language (PCL) files. |
| RTF                            | Rich Text Format output.                                                                                                                                 |

This book covers the EXCEL, HTML, PDF, and RTF destinations.

## Using Statements to Open and Close ODS Destinations

### Syntax

For each type of formatted output that you want to create, you use an ODS statement to open the destination. At the end of your program, you use another ODS statement to close the destination so that you can access your output.

---

Syntax, ODS statement to open and close destinations:

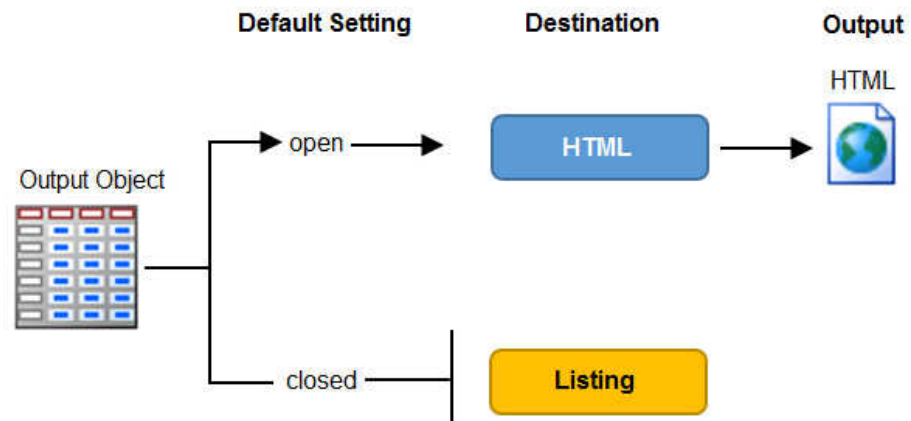
**ODS** *open-destination*;

**ODS** *close-destination* **CLOSE**;

- *open-destination* is a keyword, and any required options for the type of output that you want to create. Here are examples:
    - HTML FILE='html-file-pathname'
    - LISTING
  - *close-destination* is a keyword for the type of output.
- 

You can issue ODS statements in any order, depending on whether you need to open or close the destination. Most ODS destinations are closed by default. You open them at the beginning of your program and close them at the end. The exception is the HTML destination, which is open by default.

**Figure 10.1** Default ODS Destination



### Closing Multiple ODS Destinations at Once

You can produce output in multiple formats at once by opening each ODS destination at the beginning of the program.

When you have more than one open ODS destination, you can use the keyword `_ALL_` in the ODS CLOSE statement to close all open destinations at once.

### Closing the HTML Destination

Because open destinations use system resources, it is a good idea to close the HTML destination at the beginning of your program if you do not want to produce HTML output. Here is an example:

```
ods html close;
```

The HTML destination remains closed until you end your current SAS session or until you re-open the destination. It is good programming practice to reset ODS to HTML output (the default setting) at the end of your programs.

```
ods html path="%qsysfunc(pathname(work))";
```

---

## Creating HTML Output with ODS

### The ODS HTML Statement

To create simple HTML output files in the default location using the default file-naming conventions, you do not have to specify the ODS HTML statement. However, to create HTML output with options specified, you open the HTML destination using the ODS HTML statement.

*Note:* You do not have to specify the ODS HTML statement to produce basic HTML output unless the HTML destination is closed.

---

Syntax, ODS HTML statement:

**ODS HTML BODY** = *file-specification*;

**ODS HTML CLOSE** ;

- *file-specification* identifies the file that contains the HTML output. The specification can be any of the following:
    - a quoted string that contains the HTML filename (use only the filename to write the file to your current working directory, such as **C:\Documents and Settings\username\My Documents\My SAS Files**). Example: **ODS HTML BODY= "myreport.html"**;
    - a quoted string that contains the complete directory path and HTML filename (include the complete pathname if you want to save the HTML file to a specific location other than your working directory). Example: **ODS HTML BODY= "c:\reportdir\myreport.html"**;
    - a fileref (unquoted file shortcut) that has been assigned to an HTML file using the FILENAME statement. Example: **FILENAME MYHTML "c:\reportdir\myreport.html"**;  
**ODS HTML BODY=MYHTML**;
    - a SAS catalog entry in the form *entry-name.html*. Note that the catalog name is specified in the PATH= option and the *entry-name.html* value for the BODY= option is unquoted. Example: **ODS HTML PATH=work.mycat BODY=myentry**  
**BODY=bodyfile.html**;
- 

**TIP** FILE= can also be used to specify the file that contains the HTML output. FILE= is an alias for BODY=.

**TIP** You can also use the PATH= option to explicitly specify a directory path for your file.



**Example: Creating Output with PROC PRINT**

The following program creates PROC PRINT output in an HTML file. The ODS HTML BODY= option specifies the file `c:\certdata\admit.html` in the Windows operating environment as the file that contains the PROC PRINT results.

```
ods html body='c:\certdata\admit.html';
proc print data=clinic.admit label;
 var sex age height weight actlevel;
 label actlevel='Activity Level';
run;
ods html close;
ods html path="%qsysfunc(pathname(work))";
```

The HTML file `admit.html` contains the results of all procedure steps between the ODS HTML statement and ODS HTML CLOSE statement.

**Output 10.1** HTML Output

| Obs | Sex | Age | Height | Weight | Activity Level |
|-----|-----|-----|--------|--------|----------------|
| 1   | M   | 27  | 72     | 168    | HIGH           |
| 2   | F   | 34  | 66     | 152    | HIGH           |
| 3   | F   | 31  | 61     | 123    | LOW            |
| 4   | F   | 43  | 63     | 137    | MOD            |
| 5   | M   | 51  | 71     | 158    | LOW            |
| 6   | M   | 29  | 76     | 193    | HIGH           |
| 7   | F   | 32  | 67     | 151    | MOD            |
| 8   | M   | 35  | 70     | 173    | MOD            |
| 9   | M   | 34  | 73     | 154    | LOW            |
| 10  | F   | 49  | 64     | 172    | LOW            |
| 11  | F   | 44  | 66     | 140    | HIGH           |
| 12  | F   | 28  | 62     | 118    | LOW            |
| 13  | M   | 30  | 69     | 147    | MOD            |
| 14  | F   | 40  | 69     | 163    | HIGH           |
| 15  | M   | 47  | 72     | 173    | MOD            |
| 16  | M   | 60  | 71     | 191    | LOW            |
| 17  | F   | 43  | 65     | 123    | MOD            |
| 18  | M   | 25  | 75     | 188    | HIGH           |
| 19  | F   | 22  | 63     | 139    | LOW            |
| 20  | F   | 41  | 67     | 141    | HIGH           |
| 21  | M   | 54  | 71     | 183    | MOD            |

### Creating HTML Output with a Table of Contents

#### Overview

The BODY= specification is one way to create an HTML file containing procedure output. To create an HTML file that has a table of contents with links to the output of each specific procedure, specify additional files in the ODS HTML statement.

Syntax, ODS HTML statement to create a linked table of contents:

#### ODS HTML

**BODY**=*body-file-specification*  
**CONTENTS**=*contents-file-specification*  
**FRAME**=*frame-file-specification*;

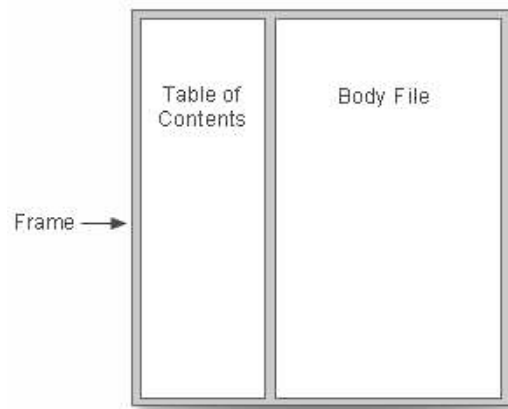
#### ODS HTML CLOSE;

- *body-file-specification* is the name of an HTML file that contains the procedure output.
- *contents-file-specification* is the name of an HTML file that contains a table of contents with links to the procedure output.
- *frame-file-specification* is the name of an HTML file that integrates the table of contents and the body file. If you specify **FRAME**=, you must also specify **CONTENTS**=.

**TIP** To direct the HTML output to a specific storage location, specify the complete pathname of the HTML file in the *file-specification*.

Here is an example that does the following:

- The **BODY**= specification creates the file `data.html` in `c:\certdata` directory. The body file contains the results of the two procedures.
- The **CONTENTS**= specification creates the file `toc.html` in the `c:\certdata` directory. The table of contents file has links to each procedure output in the body file.
- The **FRAME**= specification creates the file `frame.html` in the `c:\certdata` directory. The frame file integrates the table of contents and the body file.



```
ods html body='C:\certdata\data.html'
 contents='C:\certdata\toc.html'
 frame='C:\certdata\frame.html';
proc print data=clinic.admit (obs=10) label;
 var id sex age height weight actlevel;
 label actlevel='Activity Level';
run;
proc print data=clinic.stress2 (obs=10);
 var id resthr maxhr rechr;
run;
ods html close;
ods html path="%qsysfunc(pathname(work))";
```

### Viewing Frame Files

The Results window does not display links to frame files. In the Windows environment, only the body file automatically appears in the internal browser or your preferred web browser.

To view the frame file that integrates the body file and the table of contents, select **File** ⇒ **Open** from within the internal browser or your preferred web browser. Then open the frame file that you specified using `FRAME=`. In the example above, this file is `frame.html`, which is stored in the `Certdata` directory in the Windows environment.

The frame file, `frame.html`, is shown below.

**Figure 10.2** Partial Output: Frame File, `frame.html`

**Table of Contents**

1. The Print Procedure  
[Data Set CLINIC.ADMIT](#)
2. The Print Procedure  
[Data Set CLINIC.STRESS2](#)

**The SAS System**

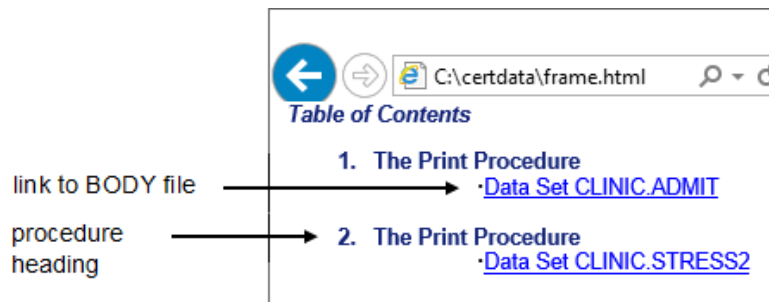
| Obs | ID   | Sex | Age | Height | Weight | Activity Level |
|-----|------|-----|-----|--------|--------|----------------|
| 1   | 2458 | M   | 27  | 72     | 168    | HIGH           |
| 2   | 2462 | F   | 34  | 66     | 152    | HIGH           |
| 3   | 2501 | F   | 31  | 61     | 123    | LOW            |
| 4   | 2523 | F   | 43  | 63     | 137    | MOD            |
| 5   | 2539 | M   | 51  | 71     | 158    | LOW            |
| 6   | 2544 | M   | 29  | 76     | 193    | HIGH           |
| 7   | 2552 | F   | 32  | 67     | 151    | MOD            |
| 8   | 2555 | M   | 35  | 70     | 173    | MOD            |
| 9   | 2563 | M   | 34  | 73     | 154    | LOW            |
| 10  | 2568 | F   | 49  | 64     | 172    | LOW            |

FRAME=frame.html      CONTENTS=toc.html      BODY=data.html

### Using the Table of Contents

The table of contents that was created by the `CONTENTS=` option contains a numbered heading for each procedure that creates output. Below each heading is a link to the output for that procedure.

**TIP** On some browsers, you can select a heading to contract or expand the table of contents.

**Figure 10.3** Table of Contents

## Using Options to Specify Links and Paths

### Overview

When ODS generates HTML files for the body, contents, and frame, it also generates links between the files using HTML filenames that you specify in the ODS HTML statement. If you specify complete pathnames, ODS uses those pathnames in the links that it generates.

The following ODS statement creates a frame file that links to `c:\certdata\toc.html` and `c:\certdata\data.html`, and a contents file that has links to `c:\certdata\data.html`.

```
ods html body='c:\certdata\data.html'
 contents='c:\certdata\toc.html'
 frame='c:\certdata\frame.html';
```

A portion of the source code for the HTML file `frame.html` is shown below. Notice that the links have the complete pathnames from the file specifications for the contents and body files.

### Example Code 10.1 Source Code for the HTML File `Frame.html`

```
<FRAME MARGINWIDTH="4" MARGINHEIGHT="0" SRC="c:\certdata\toc.html"
 NAME="contents" SCROLLING=auto>
<FRAME MARGINWIDTH="9" MARGINHEIGHT="0" SRC="c:\certdata\data.html"
 NAME="body" SCROLLING=auto>
```

These links work when you are viewing the HTML files locally. If you want to place these files on a web server so that others can access them, then the link needs to include either the complete URL for an absolute link or the HTML filename for a relative link.

### The URL= Suboption

To provide a URL that ODS uses in all the links that it creates to the file, specify the URL= suboption in the BODY= or CONTENTS= file specification. You can use the URL= suboption in any ODS file specification except FRAME= (because no ODS file references the frame file).

Syntax, URL= suboption in a file specification:

(URL= "*Uniform-Resource-Locator*";

- *Uniform-Resource-Locator* is the name of an HTML file or the full URL of an HTML file. ODS uses this URL instead of the file specification in all the links and references that it creates that point to the file.

**TIP** The URL= suboption is useful for building HTML files that might be moved from one location to another. If the links from the contents and page files are constructed with a simple URL (one name), they work as long as the contents, page, and body files are all in the same location.

### Example: Relative URLs

In this ODS HTML statement, the URL= suboption specifies only the HTML filename. This is the most common style of linking between files because maintenance is easier. The files can be moved as long as they all remain in the same directory or storage location.

```
ods html body='c:\certdata\data.html' (url='data.html')
 contents='c:\certdata\toc.html' (url='toc.html')
 frame='c:\certdata\frame.html';
```

The source code for frame.html has only the HTML filename as specified in the URL= suboptions for the body and contents files.

#### Example Code 10.2 Source Code for the HTML File Frame.html

```
<FRAME MARGINWIDTH="4" MARGINHEIGHT="0" SRC="toc.html"
 NAME="contents" SCROLLING=auto>
<FRAME MARGINWIDTH="9" MARGINHEIGHT="0" SRC="data.html"
 NAME="body" SCROLLING=auto>
```

### Example: Absolute URLs

Alternatively, in this ODS HTML statement, the URL= suboptions specify complete URLs using HTTP. These files can be stored in the same or different locations.

```
ods html body='c:\certdata\data.html'
 (url='http://mysite.com/certdata/data.html')
 contents='c:\certdata\toc.html'
 (url='http://mysite.com/certdata/toc.html')
 frame='c:\certdata\frame.html';
```

As you would expect, the source code for Frame.html has the entire HTTP addresses that you specified in the URL= suboptions for the body and contents file.

#### Example Code 10.3 Source Code for the HTML File Frame.html

```
<FRAME MARGINWIDTH="4" MARGINHEIGHT="0" SRC="http://mysite.com/certdata/data.html"
 NAME="contents" SCROLLING=auto>
<FRAME MARGINWIDTH="9" MARGINHEIGHT="0" SRC="http://mysite.com/certdata/toc.html"
 NAME="body" SCROLLING=auto>
```

**TIP** When you use the URL= suboption to specify a complete URL, you might need to move your files to that location before you can view them.

### The PATH= Option

Syntax, PATH= option with the URL= suboption:

**PATH=***file-location-specification*<(URL=NONE | “Uniform-Resource-Locator”)>

- *file-location-specification* identifies the location where you want HTML files to be saved. It can be one of the following:
  - the complete pathname to an aggregate storage location, such as a directory or partitioned data set
  - a fileref (file shortcut) that has been assigned to a storage location
  - a SAS catalog (*libname.catalog*)
- *Uniform-Resource-Locator* provides a URL for links in the HTML files that ODS generates. If you specify the keyword NONE, no information from the PATH= option appears in the links or references.

If you do not use the URL= suboption, information from the PATH= option is added to links and references in the files that are created.

*Note:* In the z/OS operating environment, if you store your HTML files as members in a partitioned data set, the PATH= value must be a PDSE, not a PDS. You can allocate a PDSE within SAS as shown in this example:

```
filename pdsehtml '.example.htm'
 dsntype=library dsorg=po
 disp=(new, catlg, delete);
```

You should specify valid member names for the HTML files (without extensions).

### **Example: PATH= Option with URL=NONE**

In the following program, the PATH= option directs the files data.html, toc.html, and frame.html to the **c:\certdata** directory in the Windows operating environment. The links from the frame file to the body and contents files contain only the HTML filenames data.html and toc.html.

```
ods html path='c:\certdata' (url=none)
 body='data.html'
 contents='toc.html'
 frame='frame.html';
proc print data=clinic.admit;
run;
proc print data=clinic.stress2;
run;
ods html close;
ods html path="%qsysfunc(pathname(work))";
```

This program generates the same files and links as the previous example in which you learned how to use the URL= suboption with the BODY= and CONTENTS= file specifications. However, it is simpler to specify the path once in the PATH= option and to specify URL=NONE.

**TIP** If you plan to move your HTML files, you should specify URL=NONE with the PATH= option to prevent information from the PATH= option from creating URLs that are invalid or incorrect.

### **Example: PATH= Option without the URL= Suboption**

In the following program, the PATH= option directs the files data.html, toc.html, and frame.html to the **c:\certdata** directory in the Windows operating environment. The

links from the frame file to the body and contents files contain the complete pathnames, **c:\certdata\data.html** and **c:\certdata\toc.html**:

```
ods html path='c:\certdata'
 body='data.html'
 contents='toc.html'
 frame='frame.html';
proc print data=clinic.admit;
run;
proc print data=clinic.stress2;
run;
ods html close;
ods html path="%qsysfunc(pathname(work))";
```

### **Example: PATH= Option with a Specified URL**

In the following program, the PATH= option directs the files data.html, toc.html, and frame.html to the **c:\certdata** directory in the Windows operating environment. The links from the frame file to the body and contents files contain the specified URLs, **http://mysite.com/certdata/data.html**, and **http://mysite.com/certdata/toc.html**:

```
ods html path='c:\certdata' (url='http://mysite.com/certdata/')
 body='data.html'
 contents='toc.html'
 frame='frame.html';
proc print data=clinic.admit;
run;
proc print data=clinic.stress2;
run;
ods html close;
ods html path="%qsysfunc(pathname(work))";
```

## **Changing the Appearance of HTML Output**

### **Style Templates**

You can change the appearance of your HTML output by specifying a style in the STYLE= option in the ODS HTML statement. Here are some of the style templates that are currently available:

- Banker
- BarrettsBlue
- Default
- HTMLblue
- Minimal
- Statistical

**TIP** To see a list of styles that SAS supplies, submit the following code:

```
proc template;
 list styles/store=sashelp.tmplmst;
run;
```



Syntax, STYLE= option:

**STYLE=***style-name*;

- *style-name* is the name of a valid SAS or user-defined style template.

**TIP** Do not enclose *style-name* in quotation marks.

### **Example: The STYLE= Option (Banker Style)**

In the following program, the STYLE= option applies the Banker style to the output for the PROC PRINT step:

```
ods html body='c:\certdata\data.html'
 style=banker;
proc print data=clinic.admit label;
 var sex age height weight actlevel;
run;
ods html close;
ods html path="%qsysfunc(pathname(work)) ";
```

**Figure 10.4** PROC PRINT Output with Banker Style Applied (partial output)

Obs	Sex	Age	Height	Weight	ActLevel
1	M	27	72	168	HIGH
2	F	34	66	152	HIGH
3	F	31	61	123	LOW
4	F	43	63	137	MOD
5	M	51	71	158	LOW
6	M	29	76	193	HIGH

*Note:* Your site might have its own, customized, style templates.

## Creating PDF Output with ODS

### **The ODS PDF Statement**

To open, manage, or close the PDF destinations that produce PDF output, use the ODS PDF statement:

Syntax, ODS PDF statement:

**ODS PDF** <(<ID=>*identifier*)> <*action*>;

- (<ID=>*identifier*) enables you to open multiple instances of the same destination at the same time. Each instance can have different options.
  - *identifier* can be numeric or can be a series of characters that begin with a letter or an underscore. Subsequent characters can include letters, underscores, and numerals.
- *action* can be one of the following:
  - CLOSE action closes the destination and any files that are associated with it.
  - EXCLUDE *exclusions*| ALL | NONE action excludes one or more output objects from the destination.

*Note:* The default is NONE. A destination must be open for this action to take effect.

- SELECT *selections*| ALL | NONE action selects output objects for the specified destination.

*Note:* The default is ALL. A destination must be open for this action to take effect.

- SHOW action writes the current selection list or exclusion list for the destination to the SAS log.

*Note:* If the selection or exclusion list is the default list (SELECT ALL), then SHOW also writes the entire selection or exclusion list. The destination must be open for this action to take effect.

---

In SAS Studio, the PDF destination is open by default. In SAS Studio, you must use the ODS PDF statement with at least one action or option. When you do this, it opens another instance of a PDF destination and creates PDF output.

## **The ODS Printer Family of Statements**

The ODS PDF statement is part of the ODS printer family of statements. Statements in the printer family open the PCL, PDF, PRINTER, or PS destination, producing output that is suitable for a high-resolution printer. The ODS PCL, ODS PRINTER, and ODS PS statements are also members of the ODS printer family of statements.

## **Opening and Closing the PDF Destination**

You can modify an open PDF destination with many ODS PDF options. However, the FILE= and SAS options perform the following actions on an open PDF destination:

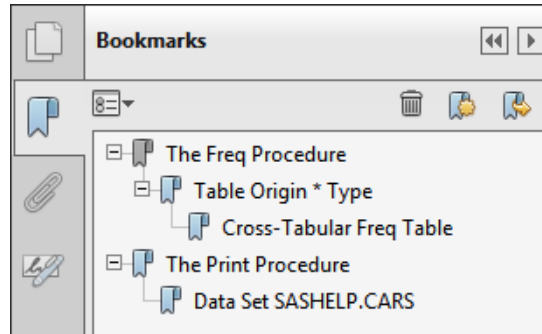
- close the open destination referred to in the ODS PDF statement
- close any files associated with the open PDF destination
- open a new instance of the PDF destination

*Note:* If you use one of these options, you should explicitly close the destination yourself.

## **Working with the Table of Contents**

The ODS PDF destination provides the following navigation tools:

- The default table of contents (TOC), which is a clickable bookmark tree that is not printed.

**Figure 10.5** PDF Output Default Bookmark Tree

- A printable table of contents, which is generated using the CONTENTS=YES option in the ODS PDF FILE= statement. The output that is created this way is static and does not count toward the page count of the PDF file. The text “Table of Contents” is customizable using PROC TEMPLATE, and the text of each of the entries is customizable with the ODS PROCLABEL statement and CONTENTS= options in some of the PROC statements.

**Figure 10.6** PDF Output Default Table of Contents Page

---

Table of Contents	
The Freq Procedure	1
Table Origin * Type	1
Cross-Tabular Freq Table	1
The Print Procedure	2
Data Set SASHELP.CARS	2

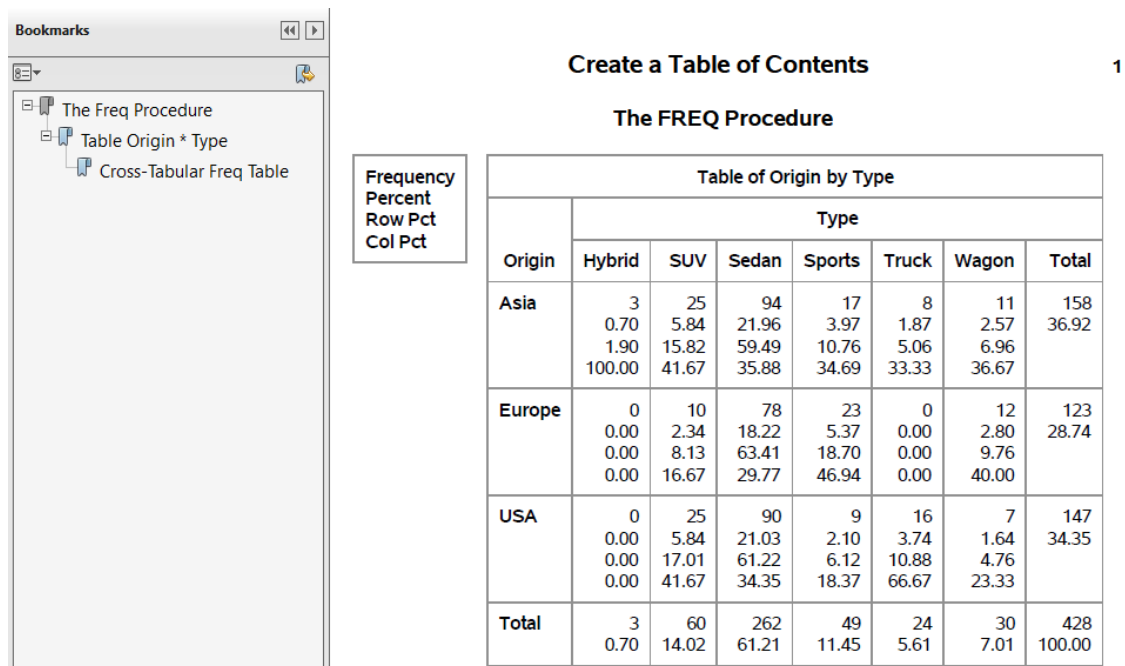
The text displayed by the nodes of each tool is controlled with the following:

- the ODS PROCLABEL statement
- the CONTENTS=, the DESCRIPTION=, and the OBJECTLABEL= options
- the DOCUMENT destination and procedure
- the TEMPLATE procedure

### **Example: Creating PDF Output Using the FILE= Option**

This example opens an instance of the PDF destination to create PDF output. The FILE= option specifies the PDF filename.

```
ods html close;
ods pdf file="SamplePDF";
proc freq data=sashelp.cars;
 tables origin*type;
run;
ods pdf close;
```

**Figure 10.7** PDF Output of FREQ Procedure


**Create a Table of Contents**

**The FREQ Procedure**

Table of Origin by Type							
Origin	Type						Total
	Hybrid	SUV	Sedan	Sports	Truck	Wagon	
Asia	3	25	94	17	8	11	158
	0.70	5.84	21.96	3.97	1.87	2.57	36.92
	1.90	15.82	59.49	10.76	5.06	6.96	
	100.00	41.67	35.88	34.69	33.33	36.67	
Europe	0	10	78	23	0	12	123
	0.00	2.34	18.22	5.37	0.00	2.80	28.74
	0.00	8.13	63.41	18.70	0.00	9.76	
	0.00	16.67	29.77	46.94	0.00	40.00	
USA	0	25	90	9	16	7	147
	0.00	5.84	21.03	2.10	3.74	1.64	34.35
	0.00	17.01	61.22	6.12	10.88	4.76	
	0.00	41.67	34.35	18.37	66.67	23.33	
Total	3	60	262	49	24	30	428
	0.70	14.02	61.21	11.45	5.61	7.01	100.00

**Example: Creating a Printable Table of Contents**

By default, ODS PDF does not create a printable table of contents, only a clickable bookmark tree. This example shows you how to create a printable table of contents.

```
ods html close;
title "Create a Table of Contents";
options nodate;
ods pdf file="MyDefaultToc.pdf" contents=yes bookmarklist=hide;
proc freq data=sashelp.cars;
 tables origin*type;
run;
proc print data=sashelp.cars (obs=15);
run;
ods pdf close;
ods html path="%qsysfunc(pathname(work))";
```

The ODS PDF statement uses the following options:

- The FILE= option specifies the PDF filename.
- The CONTENTS=YES option specifies that a table of contents is created.
- The BOOKMARKLIST=HIDE option specifies that a bookmark tree is created, but hidden.

**Figure 10.8** Printable Table of Contents for PDF Output

Table of Contents	
The Freq Procedure .....	1
Table Origin * Type .....	1
Cross-Tabular Freq Table .....	1
The Print Procedure .....	2
Data Set SASHELP.CARS .....	2

## Changing the Appearance of PDF Output

### Style Templates

You can change the appearance of your PDF output by specifying a style in the `STYLE=` option in the ODS PDF statement. The default style for PDF output is Pearl. Here are the style templates that are currently available:

- FancyPrinter
- FestivalPrinter
- GrayscalePrinter
- Journal
- MeadowPrinter
- MonoChromePrinter
- Monospace
- NormalPrinter
- Pearl
- Printer
- Sapphire
- SasDocPrinter
- SeasidePrinter

### Example: The `STYLE=` Option (FestivalPrinter Style)

In the following program, the `STYLE=` option applies the FestivalPrinter style to the output for the ODS PDF statement:

```
ods html close;
ods pdf file="SamplePDF" style=FestivalPrinter;
proc freq data=sashelp.cars;
 tables origin*type;
run;
ods pdf close;
```

**Figure 10.9** ODS PDF Output with FestivalPrinter Style Applied

Bookmarks

6

Freq  
Table Origin \* Type  
Cross-Tabular  
Freq Table

The SAS System

09:51 Monday, November 27, 2017 1

The FREQ Procedure

Frequency  
Percent  
Row Pct  
Col Pct

Table of Origin by Type							
Origin	Type						
	Hybrid	SUV	Sedan	Sports	Truck	Wagon	Total
Asia	3	25	94	17	8	11	158
	0.70	5.84	21.96	3.97	1.87	2.57	36.92
	1.90	15.82	59.49	10.76	5.06	6.96	
	100.00	41.67	35.88	34.69	33.33	36.67	
Europe	0	10	78	23	0	12	123
	0.00	2.34	18.22	5.37	0.00	2.80	28.74
	0.00	8.13	63.41	18.70	0.00	9.76	
	0.00	16.67	29.77	46.94	0.00	40.00	
USA	0	25	90	9	16	7	147
	0.00	5.84	21.03	2.10	3.74	1.64	34.35
	0.00	17.01	61.22	6.12	10.88	4.76	
	0.00	41.67	34.35	18.37	66.67	23.33	
Total	3	60	262	49	24	30	428
	0.70	14.02	61.21	11.45	5.61	7.01	100.00

8.50 x 11.00 in

<

## Creating RTF Output with ODS

### The ODS RTF Statement

To open, manage, or close the RTF destinations that produces output that is written in Rich Text Format for use with Microsoft Word, use the ODS RTF statement:

Syntax, ODS RTF statement:

**ODS RTF** <(<ID=>identifier)> <action>;

- (<ID=>identifier) enables you to open multiple instances of the same destination at the same time. Each instance can have different options.
  - identifier can be numeric or can be a series of characters that begin with a letter or an underscore. Subsequent characters can include letters, underscores, and numerals.
- action can be one of the following:
  - CLOSE action closes the destination and any files that are associated with it.
  - EXCLUDE *exclusions* | ALL | NONE action excludes one or more output objects from the destination.

*Note:* The default is NONE. A destination must be open for this action to take effect.

- SELECT *selections* | ALL | NONE action selects output objects for the specified destination.

*Note:* The default is ALL. A destination must be open for this action to take effect.

- SHOW action writes the current selection list or exclusion list for the destination to the SAS log.

*Note:* If the selection or exclusion list is the default list (SELECT ALL), then SHOW also writes the entire selection or exclusion list. The destination must be open for this action to take effect.

## Opening and Closing the RTF Destination

You can modify an open RTF destination with many ODS RTF options. However, the FILE= option performs the following actions on an open RTF destination:

- close the open destination referred to in the ODS RTF statement
- close any files associated with the open RTF destination
- open a new instance of the RTF destination

**TIP** If you use the FILE= option, you should explicitly close the destination yourself.

## Understanding How RTF Formats Output

RTF produces output for Microsoft Word. Although other applications can read RTF files, the RTF output might not work successfully with the other applications.

The RTF destination enables you to view and edit the RTF output. ODS does not define the vertical measurement, which means that SAS does not determine the optimal place to position each item on the page. For example, page breaks are not always fixed because you do not want your RTF output tables to split at inappropriate places when you edit your text. Your tables remain intact on one page, or they break where you specify.

However, Microsoft Word requires the widths of table columns, and Microsoft Word cannot adjust tables if they are too wide for the page. Therefore, ODS measures the width of the text and tables (horizontal measurement). All of the column widths can be set properly by SAS, and the table can be divided into panels if it is too wide to fit on a single page.

In short, when producing RTF output for input to Microsoft Word, SAS determines the horizontal measurement, and Microsoft Word controls the vertical measurement. Because Microsoft Word can determine how much room there is on the page, your tables are displayed consistently even after you modify your RTF file.

*Note:* Complex tables that contain a large number of observations can reduce system efficiencies and take longer to process.

## ODS RTF and Graphics

ODS RTF produces output in rich text format, which supports three formats for graphics that Microsoft Word can read.

Format for Graphics	Corresponding SAS Graphics Driver
emfblips	EMF
pngblips	PNG
jpegblips	JPEG

When you do not specify a target device, the default target is EMF.

**Example: Using the STYLE= Option (FestivalPrinter Style)**

In the following program, the STYLE= option applies the FestivalPrinter style to the output for the ODS RTF statement:

```
ods html close;
ods rtf file="SampleRTF" style=FestivalPrinter;
proc freq data=sashelp.cars;
 tables origin*type;
run;
ods rtf close;
```

**Figure 10.10** ODS RTF Output with FestivalPrinter Style Applied

01:41 Friday, December 01, 2017 1

### The SAS System

#### The FREQ Procedure

Table of Origin by Type							
Origin	Type						
Frequency Percent Row Pct Col Pct	Hybrid	SUV	Sedan	Sports	Truck	Wagon	Total
<b>Asia</b>	3 0.70 1.90 100.00	25 5.84 15.82 41.67	94 21.96 59.49 35.88	17 3.97 10.76 34.69	8 1.87 5.06 33.33	11 2.57 6.96 36.67	158 36.92
<b>Europe</b>	0 0.00 0.00 0.00	10 2.34 8.13 16.67	78 18.22 63.41 29.77	23 5.37 18.70 46.94	0 0.00 0.00 0.00	12 2.80 9.76 40.00	123 28.74
<b>USA</b>	0 0.00 0.00 0.00	25 5.84 17.01 41.67	90 21.03 61.22 34.35	9 2.10 6.12 18.37	16 3.74 10.88 66.67	7 1.64 4.76 23.33	147 34.35
<b>Total</b>	3 0.70	60 14.02	262 61.21	49 11.45	24 5.61	30 7.01	428 100.00

## Creating EXCEL Output with ODS

### The ODS EXCEL Statement

To open, manage, or close the Excel destinations that produce Excel spreadsheet files that are compatible with Microsoft 2010 and later versions, use the ODS EXCEL statement:



Syntax, ODS EXCEL statement:

**ODS EXCEL** <(<ID=>*identifier*)> <*action*>;

**ODS EXCEL** <(<ID=>*identifier*)> <*option(s)*>;

- (<ID=>*identifier*) enables you to open multiple instances of the same destination at the same time. Each instance can have different options.
  - *identifier* can be numeric or can be a series of characters that begin with a letter or an underscore. Subsequent characters can include letters, underscores, and numerals.
- *action* can be one of the following:
  - CLOSE action closes the destination and any files that are associated with it.
  - EXCLUDE *exclusions*| ALL | NONE action excludes one or more output objects from the destination.

*Note:* The default is NONE. A destination must be open for this action to take effect.

- SELECT *selections*| ALL | NONE action selects output objects for the specified destination.

*Note:* The default is ALL. A destination must be open for this action to take effect.

- SHOW action writes the current selection list or exclusion list for the destination to the SAS log.

*Note:* If the selection or exclusion list is the default list (SELECT ALL), then SHOW also writes the entire selection or exclusion list. The destination must be open for this action to take effect.

## Details About the ODS Destination

The ODS destination for Excel uses Microsoft Open Office XML Format for Office 2010 and later. This statement produces XML and represents a way to define and format data for easy exchange.

The ODS destination for Excel creates Microsoft spreadsheet in ML XML. Each table is placed in its own worksheet within a workbook. This destination supports ODS styles, trafficlighting, and custom formats. Numbers, currency, and percentages are correctly detected and displayed. Style override, a TAGATTR= style attribute, can be used to create custom formats for the data. By default, titles and footnotes are included in the worksheet, but they are part of the header and footer of the worksheet.

Portrait is the default printing orientation. The orientation can be changed to landscape.

## Example: Customizing Your Excel Output

The following example shows you how to create a customized Excel workbook that contains PROC TABULATE output. The ODS EXCEL statement is used to make the following customizations:

- A new sheet is created for each BY group.
- The BY lines are suppressed.
- The title that is created by the TITLE statement is embedded in the output.
- The worksheet labels are customized.

```
ods excel file="multitablefinal.xlsx"
options(sheet_interval="bygroup"
```

```

suppress_bylines="yes"
sheet_label="country"
embedded_titles="yes"
embed_titles_once="yes");
title "Historical Sales Data";
proc tabulate data=sashelp.prdsale;
 by country;
 var predict actual;
 class region division prodtype year;
 table year=[label=' '],
 region*(division*prodtype all=[label='division total'])
 all=[label='grand total'],
 predict=[label='total predicted sales']*f=dollar10.*sum=[label='']
 actual=[label='total actual sales']*f=dollar10.*sum=[label=''] /
 box=_page_;
run;
ods excel close;

```

Figure 10.11 Customized Excel Output

	A	B	C	D	E
1	Historical Sales Data				
2					
3	1993			TOTAL PREDICTED SALES	TOTAL ACTUAL SALES
4	Region	Division	Product type		
5	EAST	CONSUMER	FURNITURE	\$11,081	\$12,483
6			OFFICE	\$21,939	\$16,991
7		EDUCATION	FURNITURE	\$12,972	\$14,467
8			OFFICE	\$16,434	\$20,189
9		DIVISION TOTAL		\$62,426	\$64,130
10	WEST	Division	Product type		
11		CONSUMER	FURNITURE	\$10,286	\$10,380
12			OFFICE	\$16,042	\$16,371
13		EDUCATION	FURNITURE	\$12,816	\$11,234
14			OFFICE	\$17,759	\$18,905
15	DIVISION TOTAL		\$56,903	\$56,890	
16	GRAND TOTAL			\$119,329	\$121,020
17					
18	1994			TOTAL PREDICTED SALES	TOTAL ACTUAL SALES
19					
20	Country - CANADA		Country - GERMANY	Country - U.S.A.	

The SHEET\_INTERVAL= option specifies that a new sheet is created after each BY group. The SUPPRESS\_BYLINES= option is used to suppress the printing of the BY lines text. The SHEET\_LABEL= option specifies the text “Country” as the first part of the worksheet label. The EMBEDDED\_TITLES= option specifies that the title that was created by the TITLE statement is embedded in the Excel worksheet. The EMBED\_TITLES\_ONCE option specifies that the title is embedded once, at the top of each sheet.

### Example: Customizing ODS Excel Output Using TAGATTR Style Attribute

In the following example, the TAGATTR= style attribute allows you to customize the styles and format of the data in the Excel workbook. Negative values are noted by red font and parentheses. These are created using the TAGATTR= style attribute.

The TAGATTR= style attribute in the first VAR statement specifies the format for the positive values. The negative numbers are printed in red and are enclosed in parentheses. The TAGATTR= style attribute in the second VAR statement specifies the format for negative values. The negative numbers are printed in red and are enclosed in parentheses. The difference is calculated using the formula RC[-1] . The TAGATTR= style attribute in the SUM statement specifies the format that is used to show the sums for Predicted Sales For Area , Actual Sales Amount, and Difference. The first TAGATTR= attribute specifies the format for the positive values, and the second format is for negative values. The negative values are printed in red and are enclosed in parentheses.

```
ods html close;
data prdsale;
 set sashelp.prdsale;
 Difference = actual-predict;
run;
proc sort data=prdsale;
 by country region division year;
run;
ods excel file='tagattr.xlsx';
proc print data=prdsale(obs=15) noobs label split='';
 id country region division;
var prodtype product quarter month year;
var predict actual /
 style={tagattr='format:$#,##0_); [Red] \ ($#,##0\)' };
var difference /
 style={tagattr='format:$#,##0_); [Red] \ ($#,##0\)' formula:RC[-1] -RC[-2] '};
sum predict actual difference /
 style={tagattr='format:$#,##0_); [Red] \ ($#,##0\)' };
label prodtype = 'Product*Type'
 predict = 'Predicted*Sales*For Area'
 actual = 'Actual*Sales*Amount';
run;
ods excel close;
ods html path="%qsysfunc(pathname(work)) ";
```

This output is created using the ODS destination for Excel. It uses the style attribute TAGATTR= to customize the data in the Excel workbook.

**Output 10.2** Customized Data in the Excel Workbook Using the TAGATTR=Style Attribute

	A	B	C	D	E	F	G	H	I	J	K
	Country	Region	Division	Product Type	Product	Quarter	Month	Year	Predicted Sales For Area	Actual Sales Amount	Difference
2	CANADA	EAST	CONSUMER	FURNITURE	SOFA	1	Jan	1993	\$425	\$5	(\$420)
3	CANADA	EAST	CONSUMER	FURNITURE	SOFA	1	Feb	1993	\$215	\$164	(\$51)
4	CANADA	EAST	CONSUMER	FURNITURE	SOFA	1	Mar	1993	\$948	\$422	(\$526)
5	CANADA	EAST	CONSUMER	FURNITURE	SOFA	2	Apr	1993	\$544	\$424	(\$120)
6	CANADA	EAST	CONSUMER	FURNITURE	SOFA	2	May	1993	\$764	\$854	\$90
7	CANADA	EAST	CONSUMER	FURNITURE	SOFA	2	Jun	1993	\$446	\$168	(\$278)
8	CANADA	EAST	CONSUMER	FURNITURE	SOFA	3	Jul	1993	\$957	\$8	(\$949)
9	CANADA	EAST	CONSUMER	FURNITURE	SOFA	3	Aug	1993	\$967	\$748	(\$219)
10	CANADA	EAST	CONSUMER	FURNITURE	SOFA	3	Sep	1993	\$11	\$682	\$671
11	CANADA	EAST	CONSUMER	FURNITURE	SOFA	4	Oct	1993	\$110	\$300	\$190
12	CANADA	EAST	CONSUMER	FURNITURE	SOFA	4	Nov	1993	\$263	\$672	\$409
13	CANADA	EAST	CONSUMER	FURNITURE	SOFA	4	Dec	1993	\$215	\$894	\$679
14	CANADA	EAST	CONSUMER	FURNITURE	BED	1	Jan	1993	\$414	\$284	(\$130)
15	CANADA	EAST	CONSUMER	FURNITURE	BED	1	Feb	1993	\$770	\$705	(\$65)
16	CANADA	EAST	CONSUMER	FURNITURE	BED	1	Mar	1993	\$679	\$737	\$58
17									\$7,728	\$7,067	(\$661)
18											

Print 1 - Data Set WORK.PRDS

### Example: Applying a Style Sheet to Excel Output

The following program applies a style sheet created in a CSS file to the Excel output. In the style sheet, the value for the anchor (the value used with CSS as the ID) is specified. The default ID is #IDX. To choose the name of the ID so that it does not use the default, rename the anchor.

Here is an example of the external CSS file StyleSheet.css. To save the external CSS file, copy and paste into a text editor and save as StyleSheet.css.

```
#Expense .header {
 background-color:green
}

#Reports .header {
 background-color:red
}
```

The following example adds the worksheets on the same sheet using option SHEET\_INTERVAL=. The CSS styles create different background colors for the headers in the first table and the headers in the second table. ID #expense is specified for the first table, and the ID #Reports is specified for the second table using the ANCHOR= option.

```
ods html close;
ods excel file="c:\excelAnchorCss.xlsx"
 cssstyle="file-path\Stylesheet.css"
 options(sheet_interval="none");
ods excel anchor="expense";
proc print data=sashelp.class;
```

```
run;

ods excel anchor="reports" cssstyle="file-path\Stylesheet.css";
proc print data=sashelp.class;
run;

ods excel close;
ods html path="%qsysfunc(pathname(work))";
```

The SHEET\_INTERVAL="NONE" option specifies that all output appear on the same sheet. The CSSSTYLE= option specifies the file StyleSheet.css to provide different background colors to the output.

It is possible to create a different background color for the header in the first table by adding ID #Expense for the first table using the ANCHOR= option. The file StyleSheet.css contains a matching ID. By default, the IDs are #IDX and #IDX1 because an anchor ID was not specified.

It is possible to create a different background colors for the header in the second table by adding ID #Reports using the ANCHOR= option. The file StyleSheet.css contains a matching ID. By default, the ID for the second table is #IDX1 because an anchor ID was not specified.

**Output 10.3** Customized ODS Excel Output by Using a Style Sheet

	A	B	C	D	E	F	G
1	Obs Name	Sex	Age	Height	Weight		
2	1 Alfred	M	14	69.0	112.5		
3	2 Alice	F	13	56.5	84.0		
4	3 Barbara	F	13	65.3	98.0		
5	4 Carol	F	14	62.8	102.5		
6	5 Henry	M	14	63.5	102.5		
7	6 James	M	12	57.3	83.0		
8	7 Jane	F	12	59.8	84.5		
9	8 Janet	F	15	62.5	112.5		
10	9 Jeffrey	M	13	62.5	84.0		
11	10 John	M	12	59.0	99.5		
12	11 Joyce	F	11	51.3	50.5		
13	12 Judy	F	14	64.3	90.0		
14	13 Louise	F	12	56.3	77.0		
15	14 Mary	F	15	66.5	112.0		
16	15 Philip	M	16	72.0	150.0		
17	16 Robert	M	12	64.8	128.0		
18	17 Ronald	M	15	67.0	133.0		
19	18 Thomas	M	11	57.5	85.0		
20	19 William	M	15	66.5	112.0		
21							
22	Obs Name	Sex	Age	Height	Weight		
23	1 Alfred	M	14	69.0	112.5		
24	2 Alice	F	13	56.5	84.0		
25	3 Barbara	F	13	65.3	98.0		
26	4 Carol	F	14	62.8	102.5		
27	5 Henry	M	14	63.5	102.5		
28	6 James	M	12	57.3	83.0		

Sheet 1

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- Using ODS statements, how many types of output can you generate at once?
  - 1 (only LISTING output)
  - 2
  - 3
  - as many as you want
- If ODS is set to its default settings, what types of output are created by the following code?
 

```
ods html file='c:\myhtml.htm';
ods pdf file='c:\mypdf.pdf';
```

- a. HTML and PDF
  - b. PDF only
  - c. HTML, PDF, and LISTING
  - d. No output is created because ODS is closed by default.
3. What is the purpose of closing the HTML destination in the following code?
 

```
ods HTML close;
ods pdf ... ;
```

  - a. It conserves system resources.
  - b. It simplifies your program.
  - c. It makes your program compatible with other hardware platforms.
  - d. It makes your program compatible with previous versions of SAS.
4. When the following code runs, what does the file D:\Output\body.html contain?
 

```
ods html body='d:\output\body.html';
proc print data=work.alpha;
run;
proc print data=work.beta;
run;
ods html close;
```

  - a. The PROC PRINT output for Work.Alpha.
  - b. The PROC PRINT output for Work.Beta.
  - c. The PROC PRINT output for both Work.Alpha and Work.Beta.
  - d. Nothing. No output is written to D:\Output\body.html.
5. When the following code runs, what file is loaded by the links in D:\Output\contents.html?
 

```
ods html body='d:\output\body.html'
 contents='d:\output\contents.html'
 frame='d:\output\frame.html';
```

  - a. D:\Output\body.html
  - b. D:\Output\contents.html
  - c. D:\Output\frame.html
  - d. There are no links from the file D:\Output\contents.html.
6. The table of contents that was created by the CONTENTS= option contains a numbered heading for which of the following?
  - a. each procedure
  - b. each procedure that creates output
  - c. each procedure and DATA step
  - d. each HTML file created by your program
7. When the following code runs, what will the file D:\Output\frame.html display?
 

```
ods html body='d:\output\body.html'
 contents='d:\output\contents.html'
 frame='d:\output\frame.html';
```

- a. The file D:\Output\contents.html .
  - b. The file D:\Output\frame.html.
  - c. The files D:\Output\contents.html and D:\Output\body.html.
  - d. It displays no other files.
8. What is the purpose of the following URL= suboptions?

```
ods html body='d:\output\body.html' (url='body.html')
 contents='d:\output\contents.html'
 (url='contents.html')
 frame='d:\output\frame.html';
```

- a. To create absolute link addresses for loading the files from a server.
  - b. To create relative link addresses for loading the files from a server.
  - c. To allow HTML files to be loaded from a local drive.
  - d. To send HTML output to two locations.
9. Which ODS HTML option was used in creating the following table?

Obs	Sex	Age	Height	Weight	ActLevel
1	M	27	72	168	HIGH
2	F	34	66	152	HIGH
3	F	31	61	123	LOW

- a. format=MeadowPrinter
  - b. format='MeadowPrinter'
  - c. style=MeadowPrinter
  - d. style='MeadowPrinter'
10. What is the purpose of the PATH= option?

```
ods html path='d:\output' (url=none)
 body='body.html'
 contents='contents.html'
 frame='frame.html';
```

- a. It creates absolute link addresses for loading HTML files from a server.
- b. It creates relative link addresses for loading HTML files from a server.
- c. It allows HTML files to be loaded from a local drive.
- d. It specifies the location of HTML file output.



## Chapter 11

# Creating and Managing Variables

---

<b>Creating and Modifying Variables</b>	<b>232</b>
Accumulating Totals	232
Example: Sum Statement	232
Initializing Sum Variables	233
Example: RETAIN Statement	233
<b>Assigning Values Conditionally</b>	<b>234</b>
Categorizing Values	234
Example: IF-THEN Statement	234
Comparison and Logical Operators	235
Examples: Logical Operators	235
Providing an Alternative Action	237
<b>Specifying Lengths for Variables</b>	<b>238</b>
Avoiding Truncated Variable Values	238
Examples: LENGTH Statement	239
<b>Subsetting Data</b>	<b>240</b>
Deleting Unwanted Observations	240
Example: IF-THEN and DELETE Statements	240
Selecting Variables	241
Example: DROP Data Set Option	241
Example: Using the DROP= Data Set Option and DROP Statement	242
<b>Assigning Permanent Labels and Formats</b>	<b>243</b>
Common Tasks and Their Corresponding Statements	243
Example: COMMA6. Format	244
<b>Assigning Values Conditionally Using SELECT Groups</b>	<b>245</b>
The Basics of Using SELECT Groups	245
Example: Basic SELECT Group	245
Example: SELECT Group in a DATA Step	246
Specifying SELECT Statements with Expressions	247
Specifying SELECT Statements without Expressions	247
<b>Grouping Statements Using DO Groups</b>	<b>248</b>
The Basics of Using Grouping Statements and DO Groups	248
Examples: DO and END Statements	248
Indenting and Nesting DO Groups	249
<b>Chapter Quiz</b>	<b>250</b>

## Creating and Modifying Variables

### Accumulating Totals

To add the result of an expression to an accumulator variable, you can use a sum statement in your DATA step.

Syntax, sum statement:

*variable*+*expression*;

- *variable* specifies the name of the accumulator variable. This variable must be numeric. The variable is automatically set to 0 before the first observation is read. The variable's value is retained from one DATA step execution to the next.
- *expression* is any valid SAS expression.

*Note:* If the expression produces a missing value, the sum statement ignores it. The sum statement is one of the few SAS statements that do not begin with a keyword.

The sum statement adds the result of the expression that is on the right side of the plus sign (+) to the numeric variable that is on the left side of the plus sign. The value of the accumulator variable is initialized to 0 instead of missing before the first iteration of the DATA step. Subsequently, the variable's value is retained from one iteration to the next, as if it had appeared in a RETAIN statement.

### Example: Sum Statement

To find the total number of elapsed seconds in treadmill stress tests, you need a variable (in this example, SumSec) whose value begins at 0 and increases by the amount of the total seconds in each observation. To calculate the total number of elapsed seconds in treadmill stress tests, you use the sum statement shown below:

```
data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 TotalTime=(timemin*60)+timesec;
 SumSec+totaltime;
run;
```

The value of the variable on the left side of the plus sign (here, SumSec) begins at 0 and increases by the value of TotalTime with each observation.

SumSec	=	TotalTime	+	Previous total
0				
758	=	758	+	0
1363	=	605	+	758

SumSec	=	TotalTime	+	Previous total
2036	=	673	+	1363
2618	=	582	+	2036
3324	=	706	+	2618

### Initializing Sum Variables

In the previous example, the sum variable SumSec was initialized to 0 by default before the first observation was read. But what if you want to initialize SumSec to a different number, such as the total seconds from previous treadmill stress tests?

You can use the RETAIN statement to assign an initial value other than the default value of 0 to a variable whose value is assigned by a sum statement.

The RETAIN statement serves several purposes:

- It assigns an initial value to a retained variable.
- It prevents variables from being initialized each time the DATA step executes.

---

Syntax, RETAIN statement for initializing sum variables:

**RETAIN** *variable* <initial-value>;

- *variable* is a variable whose values you want to retain.
- *initial-value* specifies an initial value (numeric or character) for the preceding variable.

*Note:* The following statements are true about the RETAIN statement:

- It is a compile-time-only statement that creates variables if they do not already exist.
  - It initializes the retained variable to missing before the first execution of the DATA step if you do not supply an initial value.
  - It has no effect on variables that are read with SET, MERGE, or UPDATE statements.
- 

### Example: RETAIN Statement

Suppose you want to add 5400 seconds (the accumulated total seconds from a previous treadmill stress test) to the variable SumSec in the Clinic.Stress data set when you create the data set. To initialize SumSec with the value 5400, you use the RETAIN statement shown below.

```
data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 TotalTime=(timemin*60)+timesec;
 retain SumSec 5400;
 sumsec+totaltime;
run;
```

Now the value of SumSec begins at 5400 and increases by the value of TotalTime with each observation.

SumSec	=	TotalTime	+	Previous Total
5400				
6158	=	758	+	5400
6763	=	605	+	6158
7436	=	673	+	6763
8018	=	582	+	7436
8724	=	706	+	8018

## Assigning Values Conditionally

### Categorizing Values

Suppose you want to create a variable that categorizes the length of time that a subject spends on the treadmill during a stress test. This new variable, TestLength, is based on the value of the existing variable TotalTime. The value of TestLength is assigned conditionally:

Value for TotalTime	Resulting Value for TestLength
greater than 800	Long
750 - 800	Normal
less than 750	Short

To perform an action conditionally, use an IF-THEN statement. The IF-THEN statement executes a SAS statement when the condition in the IF clause is true.

Syntax, IF-THEN statement:

**IF** *expression* **THEN** *statement*;

- *expression* is any valid SAS expression.
- *statement* is any executable SAS statement.

### Example: IF-THEN Statement

To assign the value Long to the variable TestLength when the value of TotalTime is greater than 800, add the following IF-THEN statement to your DATA step:

```

data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 TotalTime=(timemin*60)+timesec;
 retain SumSec 5400;
 sumsec+totaltime;
 if totaltime>800 then TestLength='Long';
run;

```

SAS executes the assignment statement only when the condition (TotalTime>800) is true. If the condition is false, the value of TestLength is missing.

## Comparison and Logical Operators

When writing IF-THEN statements, you can use any of the following comparison operators:

Operator	Comparison Operation
= or <b>eq</b>	equal to
^= or ~= or <b>ne</b>	not equal to
> or <b>gt</b>	greater than
< or <b>lt</b>	less than
>= or <b>ge</b>	greater than or equal to
<= or <b>le</b>	less than or equal to
<b>in</b>	equal to one of a list

## Examples: Logical Operators

```

if test<85 and time<=20
 then Status='RETEST';
if region in ('NE','NW','SW')
 then Rate=fee-25;
if target gt 300 or sales ge 50000
 then Bonus=salary*.05;

```

You can also use these logical operators:

Operator	Logical Operation
&	and
	or

Operator	Logical Operation
^ or ~	not

Use the AND operator to execute the THEN statement if both expressions that are linked by AND are true.

```
if status='OK' and type=3
 then Count+1;
if (age^=agecheck | time^=3)
 & error=1 then Test=1;
```

Use the OR operator to execute the THEN statement if either expression that is linked by OR is true.

```
if (age^=agecheck || time^=3)
 & error=1 then Test=1;
if status='S' or cond='E'
 then Control='Stop';
```

Use the NOT operator with other operators to reverse the logic of a comparison.

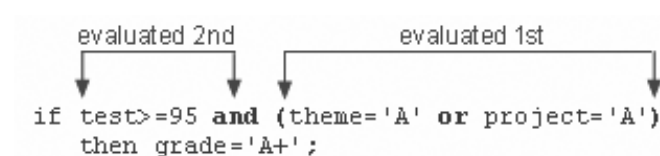
```
if not(loghours<7500)
 then Schedule='Quarterly';
if region not in ('NE','SE')
 then Bonus=200;
```

Character values must be specified in the same case in which they appear in the data set and must be enclosed in quotation marks.

```
if status='OK' and type=3
 then Count+1;
if status='S' or cond='E'
 then Control='Stop';
if not(loghours<7500)
 then Schedule='Quarterly';
if region not in ('NE','SE')
 then Bonus=200;
```

Logical comparisons that are enclosed in parentheses are evaluated as true or false before they are compared to other expressions. In the example below, the OR comparison in parenthesis is evaluated before the first expression and the AND operator are evaluated.

**Figure 11.1** Example of a Logical Comparison



*Note:* In SAS, any numeric value other than 0 or missing is true, and a value of 0 or missing is false. Therefore, a numeric variable or expression can stand alone in a condition. If its value is a number other than 0 or missing, the condition is true. If its value is 0 or missing, the condition is false.

- 0 = False

- . = False
- 1 = True

Therefore, be careful when using the OR operator with a series of comparisons. Remember that only one comparison in a series of OR comparisons must be true to make a condition true, and any nonzero, nonmissing constant is always evaluated as true. Therefore, the following subsetting IF statement is always true:

```
if x=1 or 2;
```

SAS first evaluates `x=1`, and the result can be either true or false. However, since the 2 is evaluated as nonzero and nonmissing (true), the entire expression is true. In this statement, however, the condition is not necessarily true because either comparison can be evaluated as true or false:

```
if x=1 or x=2;
```

*Note:* Both sides of the OR must contain complete expressions.

## Providing an Alternative Action

Now suppose you want to assign a value to `TestLength` based on the other possible values of `TotalTime`. One way to do this is to add IF-THEN statements for the other two conditions.

```
if totaltime>800 then TestLength='Long';
if 750<=totaltime<=800 then TestLength='Normal';
if totaltime<750 then TestLength='Short';
```

However, when the DATA step executes, each IF statement is evaluated in order, even if the first condition is true. This wastes system resources and slows the processing of your program.

Instead of using a series of IF-THEN statements, you can use the ELSE statement to specify an alternative action to be performed when the condition in an IF-THEN statement is false. As shown below, you can write multiple ELSE statements to specify a series of mutually exclusive conditions.

```
if totaltime>800 then TestLength='Long';
else if 750<=totaltime<=800 then TestLength='Normal';
else if totaltime<750 then TestLength='Short';
```

The ELSE statement must immediately follow the IF-THEN statement in your program. An ELSE statement executes only if the previous IF-THEN/ELSE statement is false.

---

Syntax, ELSE statement:

**ELSE** *statement*;

*statement* is any executable SAS statement, including another IF-THEN statement.

---

To assign a value to `TestLength` when the condition in your IF-THEN statement is false, you can add the ELSE statement to your DATA step:

```
data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 TotalTime=(timemin*60)+timesec;
```

```

retain SumSec 5400;
sumsec+totaltime;
if totaltime>800 then TestLeng h='Long';
else if 750<=totaltime<=800 then TestLength='Normal';
else if totaltime<750 then TestLength='Short';
run;

```

**TIP** Using ELSE statements with IF-THEN statements can save resources:

- Using IF-THEN statements without the ELSE statement causes SAS to evaluate all IF-THEN statements.
- Using IF-THEN statements with the ELSE statement causes SAS to execute IF-THEN statements until it encounters the first true statement. Subsequent IF-THEN statements are not evaluated.

For greater efficiency, construct your IF-THEN/ELSE statements with conditions of decreasing probability.

**TIP** You can use PUT statements to test your conditional logic.

```

if totaltime>800 then TestLength='Long';
else if 750<=totaltime<=800 then TestLength='Normal';
else put 'NOTE: Check this Length: ' totaltime=;
run;

```

---

## Specifying Lengths for Variables

### Avoiding Truncated Variable Values

Although it is possible to add IF-THEN and ELSE statements to a DATA step in order to create the variable TestLength, consider what happens when you submit the following program:

```

data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 TotalTime=(timemin*60)+timesec;
 retain SumSec 5400;
 sumsec+totaltime;
 if totaltime>800 then TestLength='Long';
 else if 750<=totaltime<=800 then TestLength='Normal';
 else if totaltime<750 then TestLength='Short';
run;

```

During compilation, when creating a new character variable in an assignment statement, SAS allocates as many bytes of storage space as there are characters in the first value that it encounters for that variable. In this case, the first value for TestLength occurs in the IF-THEN statement, which specifies a four-character value (Long). So, TestLength is assigned a length of 4, and any longer values (Normal and Short) are truncated.



**Figure 11.2** Truncated Variable Values (partial output)

TestLength
Norm
Shor
Shor
Shor
Shor
Shor
Long

The example above assigns a character constant as the value of the new variable. You can also see other examples of the default type and length that SAS assigns when the type and length of a variable are not explicitly set.

**TIP** You can use a LENGTH statement to specify a length (the number of bytes) for TestLength before the first value is referenced elsewhere in the DATA step.

---

Syntax, LENGTH statement:

**LENGTH** *variable(s)* <\$> *length*;

- *variable(s)* names the variable or variables to be assigned a length.
  - \$ is specified if the variable is a character variable.
  - *length* is an integer that specifies the length of the variable.
- 

### Examples: LENGTH Statement

Here is a variable list in which all three variables are assigned a length of \$200.

```
length Type $ 8;
length Address1 Address2 Address3 $200;
length FirstName $12 LastName $16;
```

Within the program, a LENGTH statement is included to assign a length to accommodate the longest value of the variable TestLength. The longest value is **Normal**, which has six characters. Because TestLength is a character variable, you must follow the variable name with a dollar sign (\$).

Make sure the LENGTH statement appears before any other reference to the variable in the DATA step.

```
data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 TotalTime=(timemin*60)+timesec;
 retain SumSec 5400;
 sumsec+totaltime;
 length TestLength $ 6;
 if totaltime>800 then testlength='Long';
 else if 750<=totaltime<=800 then testlength='Normal';
 else if totaltime<750 then TestLength='Short';
run;
```

*Note:* If the variable has been created by another statement, then a later use of the LENGTH statement does not change its length.

Now that the LENGTH statement has been added to the program, the values of TestLength are no longer truncated.

**Figure 11.3** Variable Values That Are Not Truncated (partial output)

TestLength
Normal
Short
Short
Short
Short
Short
Long

---

## Subsetting Data

### Deleting Unwanted Observations

You can specify any executable SAS statement in an IF-THEN statement. For example, you can use an IF-THEN statement with a DELETE statement to determine which observations to omit as you read data.

---

Syntax, DELETE statement:

**DELETE;**

To conditionally execute a DELETE statement, use the following syntax for an IF statement:

**IF expression THEN DELETE;**

The expression is evaluated as follows:

- If it is true, execution stops for that observation. The DELETE statement deletes the observation from the output data set, and control returns to the top of the DATA step.
  - If it is false, the DELETE statement does not execute, and processing continues with the next statement in the DATA step.
- 

### Example: IF-THEN and DELETE Statements

The IF-THEN and DELETE statements below omit any observations whose values for RestHR are lower than 70.

```
data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 if resthr<70 then delete;
 TotalTime=(timemin*60)+timesec;
```

```

retain SumSec 5400;
sumsec+totaltime;
length TestLength $ 6;
if totaltime>800 then testlength='Long';
else if 750<=totaltime<=800 then testlength='Normal';
else if totaltime<750 then TestLength='Short';
run;

```

## Selecting Variables

Sometimes you want to read and process variables that you do not want to keep in your output data set. In this case, use the DROP= and KEEP= data set options to specify the variables to drop or keep.

Use the KEEP= option instead of the DROP= option if more variables are dropped than kept.

---

Syntax, DROP=, and KEEP= data set options:

**(DROP=variable(s))**

**(KEEP=variable(s))**

- The DROP= or KEEP= options, in parentheses, follow the names of the data sets that contain the variables to be dropped or kept.
  - *variable(s)* identifies the variables to drop or keep.
- 

## Example: DROP Data Set Option

Suppose you want to use the TimeMin and TimeSec variables to calculate the total time in the TotalTime variable, but you do not want to keep them in the output data set. You want to keep only the TotalTime variable. When you use the DROP data set option, the TimeMin and TimeSec variables are not written to the output data set:

```

data clinic.stress(drop=timemin timesec);
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 if tolerance='D';
 TotalTime=(timemin*60)+timesec;
 retain SumSec 5400;
 sumsec+totaltime;
 length TestLength $ 6;
 if totaltime>800 then testlength='Long';
 else if 750<=totaltime<=800 then testlength='Normal';
 else if totaltime<750 then TestLength='Short';
run;

```

**Figure 11.4** Stress Data Set with Dropped Variables (partial output)

**The SAS System**

Obs	ID	Name	RestHR	MaxHR	RecHR	Tolerance	TotalTime	SumSec	TestLength
1	2458	Murray, W	72	185	128	D	758	6158	Normal
2	2462	Almers, C	68	171	133	I	605	6763	Short
3	2501	Bonaventure, T	78	177	139	I	673	7436	Short
4	2523	Johnson, R	69	162	114	S	582	8018	Short
5	2539	LaMance, K	75	168	141	D	706	8724	Short
6	2544	Jones, M	79	187	136	N	746	9470	Short
7	2552	Reberson, P	69	158	139	D	941	10411	Long
8	2555	King, E	70	167	122	I	793	11204	Normal

Another way to exclude variables from a data set is to use the DROP statement or the KEEP statement. Like the DROP= and KEEP= data set options, these statements drop or keep variables. However, the DROP and KEEP statements differ from the DROP= and KEEP= data set options in the following ways:

- You cannot use the DROP and KEEP statements in SAS procedure steps.
- The DROP and KEEP statements apply to all output data sets that are named in the DATA statement. To exclude variables from some data sets but not from others, use the DROP= and KEEP= data set options in the DATA statement.

The KEEP statement is similar to the DROP statement, except that the KEEP statement specifies a list of variables to write to output data sets. Use the KEEP statement instead of the DROP statement if the number of variables to keep is smaller than the number to drop.

---

Syntax, DROP, and KEEP statements:

**DROP** *variable(s)*;

**KEEP** *variable(s)*;

*variable(s)* identifies the variables to drop or keep.

---

### **Example: Using the DROP= Data Set Option and DROP Statement**

The two programs below produce the same results. The first example uses the DROP= data set option. The second example uses the DROP statement.

```
data clinic.stress(drop=timemin timesec);
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 if tolerance='D';
 TotalTime=(timemin*60)+timesec;
 retain SumSec 5400;
 sumsec+totaltime;
 length TestLength $ 6;
 if totaltime>800 then testlength='Long';
 else if 750<=totaltime<=800 then testlength='Normal';
 else if totaltime<750 then TestLength='Short';
run;
```

```

data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 if tolerance='D';
 drop timemin timesec;
 TotalTime=(timemin*60)+timesec;
 retain SumSec 5400;
 sumsec+totaltime;
 length TestLength $ 6;
 if totaltime>800 then testlength='Long';
 else if 750<=totaltime<=800 then testlength='Normal';
 else if totaltime<750 then TestLength='Short';
run;

```

## Assigning Permanent Labels and Formats

### Common Tasks and Their Corresponding Statements

You saw how to temporarily assign labels and formats within a PROC step in [Chapter 5, “Creating List Reports,”](#) on page 67. These temporary labels and formats are applicable only for the duration of the step. To permanently assign labels and formats, you use LABEL and FORMAT statements in DATA steps.

*Note:* Labels and formats do not affect how data is stored in the data set, but only how it appears in output.

Task	Statement to Use
Reference a SAS data library	libname clinic 'c:\users\may\data';
Reference an external file	filename tests 'c:\users\tmill.dat';
Name a SAS data set	data clinic.stress;
Identify an external file	infile tests obs=10;
Describe raw data	input ID \$ 1-4 Name \$ 6-25 ...;
Subset data	if resthr<70 then delete; if tolerance='D';
Drop unwanted variables	drop timemin timesec;
Create or modify a variable	TotalTime=(timemin*60)+timesec;
Initialize and retain variable	retain SumSec 5400;
Accumulate values	sumsec+totaltime;
Specify a variable's length	length TestLength \$ 6;

Task	Statement to Use
Execute statements conditionally	<pre>if totaltime&gt;800 then TestLength='Long'; else if 750&lt;=totaltime&lt;=800     then TestLength='Normal'; else if totaltime&lt;750     then TestLength='Short';</pre>
Label a variable	<pre>label sumsec='Cumulative Total Seconds (+5,400)';</pre>
Format a variable	<pre>format sumsec comma6.;</pre>
Execute the DATA step	<pre>run;</pre>
List the data	<pre>proc print data=clinic.stress label;</pre>
Execute the final program step	<pre>run;</pre>

**Example: COMMA6. Format**

To specify the label Cumulative Total Seconds (+5,400) and the format COMMA6. for the variable SumSec, you can submit the following program:

```
data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 if resthr<70 then delete;
 if tolerance='D';
 drop timemin timesec;
 TotalTime=(timemin*60)+timesec;
 retain SumSec 5400;
 sumsec+totaltime;
 length TestLength $ 6;
 if totaltime>800 then testlength='Long';
 else if 750<=totaltime<=800 then testlength='Normal';
 else if totaltime<750 then TestLength='Short';
 label sumsec='Cumulative Total Seconds (+5,400)';
 format sumsec comma6.;
run;
```

When the new data set is displayed, SumSec is labeled and formatted as specified.

```
proc print data=clinic.stress label;
run;
```

Figure 11.5 Completed Clinic.Stress Data Set

The SAS System

Obs	ID	Name	RestHR	MaxHR	RecHR	Tolerance	TotalTime	SumSec	TestLength
1	2458	Murray, W	72	185	128	D	758	6,158	Normal
2	2539	LaMance, K	75	168	141	D	706	6,864	Short
3	2572	Oberon, M	74	177	138	D	731	7,595	Short
4	2574	Peterson, V	80	164	137	D	849	8,444	Long
5	2584	Takahashi, Y	76	163	135	D	967	9,411	Long

**TIP** Most SAS procedures automatically use permanent labels and formats in output, without requiring additional statements or options. If you assign temporary labels or formats within a PROC step, they override any permanent labels or formats that were assigned during the DATA step.

## Assigning Values Conditionally Using SELECT Groups

### The Basics of Using SELECT Groups

You can use SELECT groups in DATA steps to perform conditional processing.

Syntax, SELECT group:

```
SELECT <(select-expression)>;
 WHEN-1 (when-expression-1<..., when-expression-n>) statement;
 <...WHEN-n (when-expression-1 <..., when-expression-n>) statement;>
 <OTHERWISE statement;>
END;
```

- SELECT begins a SELECT group.
- *select-expression* specifies any SAS expression that evaluates to a single value.
- WHEN identifies SAS statements that are executed when a particular condition is true.
- *when-expression* specifies any SAS expression, including a compound expression. You must specify at least one *when-expression*.
- *statement* is any executable SAS statement. You must specify the *statement* argument.
- OTHERWISE specifies a statement to be executed if no WHEN condition is met.
- END ends a SELECT group.

### Example: Basic SELECT Group

Here is an example of a SELECT group. Notice that the variable *a* is specified in the SELECT statement, and various values that are compared are specified in the WHEN statements. Here are variations for the value of the variable *a*:

- When *a* is 1, *x* is multiplied by 10.

- When a is 3, 4, or 5, x is multiplied by 100.
- When a is 2 or any other value, nothing happens.

```
select (a);
 when (1) x=x*10;
 when (3,4,5) x=x*100;
 otherwise;
end
```

### Example: SELECT Group in a DATA Step

In the DATA step below, the SELECT group assign values to the variable Group based on values of the variable JobCode. Most of the assignments are one-to-one correspondences, but ticket agents (the JobCode values TA1, TA2, and TA3) are grouped together, as are values in the Other category.

```
data emps(keep=salary group);
 set sasuser.payrollmaster;
 length Group $ 20;
 select(jobcode);
 when ("FA1") group="Flight Attendant I";
 when ("FA2") group="Flight Attendant II";
 when ("FA3") group="Flight Attendant III";
 when ("ME1") group="Mechanic I";
 when ("ME2") group="Mechanic II";
 when ("ME3") group="Mechanic III";
 when ("NA1") group="Navigator I";
 when ("NA2") group="Navigator II";
 when ("NA3") group="Navigator III";
 when ("NA1") group="Navigator I";
 when ("NA2") group="Navigator II";
 when ("NA3") group="Navigator III";
 when ("PT1") group="Pilot I";
 when ("PT2") group="Pilot II";
 when ("PT3") group="Pilot III";
 when ("TA1","TA2","TA3") group="Ticket Agents";
 otherwise group="Other";
 end;
run;
```

The SELECT statement evaluates the select-expression (jobcode), which is more concise than eliminating the select-expression and repeating the variable in each when-expression, as is the case here: (jobcode="FA1"). Notice that the LENGTH statement in the DATA step above specifies a length of 20 for Group. Remember that without the LENGTH statement, values for Group might be truncated, as the first value for Group (Flight Attendant I) is not the longest possible value.

**TIP** When you are comparing values in the when-expression, be sure to express the values exactly as they appear in the data. For example, the when-expression below would be evaluated as false because the values for JobCode in Sasuser.Payrollmaster are stored in uppercase letters.

```
when ("fa1") group="Flight Attendant I";
```

In this case, in the SELECT group above, Group would be assigned the value Other.



## Specifying SELECT Statements with Expressions

Although you can specify a select-expression in the SELECT statement, the way SAS evaluates a when-expression depends on whether you specify a select-expression.

When you specify a select-expression in the SELECT statement, SAS compares the value of the select-expression with the value of each when-expression. That is, SAS evaluates the select-expression and when-expression, compares the two for equality, and returns a value of true or false.

- When the comparison is true, SAS executes the statement in the WHEN statement.
- When the comparison is false, SAS proceeds either to the next when-expression in the current WHEN statement, or to the next WHEN statement if no more expressions are present. If no WHEN statements remain, execution proceeds to the OTHERWISE statement, if one is present.

In the following SELECT group, SAS determines the value of Toy and compares it to values in each WHEN statement in turn. If a WHEN statement is true compared to the toy value, SAS assigns the related price and continues processing the rest of the DATA step. When none of the comparisons are true, SAS executes the OTHERWISE statement and writes a debugging message to the SAS log.

```
select (toy);
 when ("Bear") price=35.00;
 when ("Violin") price=139.00;
 when ("Top","Whistle","Duck") price=7.99;
 otherwise put "Check unknown toy: " toy=;
end;
```

## Specifying SELECT Statements without Expressions

When you do not specify a select-expression, SAS evaluates each when-expression to produce a result of true or false.

- When the result is true, SAS executes the statement in the WHEN statement.
- When the result is false, SAS proceeds either to the next when-expression in the current WHEN statement, or to the next WHEN statement if no more expressions are present, or to the OTHERWISE statement if one is present. (That is, SAS performs the action that is indicated in the first true WHEN statement.)

When more than one WHEN statement has a true when-expression, only the first WHEN statement is used. After a when-expression is true, no other when-expressions are evaluated.

*Note:* If the result of all when-expressions is false and no OTHERWISE statement is present, SAS issues an error message.

In the example below, the SELECT statement does not specify a select-expression. The WHEN statements are evaluated in order, and only one is used. For example, if the value of toy is Bear and the value of month is FEB, only the second WHEN statement is used, even though the condition in the third WHEN statement is also met. In this case, the variable price is assigned the value 25.00:

```
select;
 when (toy="Bear" and month in ('OCT', 'NOV', 'DEC')) price=45.00;
 when (toy="Bear" and month in ('JAN', 'FEB')) price=25.00;
 when (toy="Bear") price=35.00;
```

```

 otherwise;
 end;

```

---

## Grouping Statements Using DO Groups

### *The Basics of Using Grouping Statements and DO Groups*

You can execute a group of statements as a unit by using DO groups.

To construct a DO group, you use the DO and END statements along with other SAS statements.

---

Syntax, DO group:

**DO;**

*SAS statements*

**END;**

- The DO statement begins DO-group processing.
- *SAS statements* between the DO and END statements are called a DO group and are executed as a unit.
- The END statement terminates DO-group processing.

*Tip:* You can nest DO statements within DO groups.

---

You can use DO groups in IF-THEN/ELSE statements and SELECT groups to execute many statements as part of the conditional action.

### *Examples: DO and END Statements*

In this simple DO group, the statements between DO and END are performed only when TotalTime is greater than 800. If TotalTime is less than or equal to 800, statements in the DO group are not executed, and the program continues with the assignment statement that follows the appropriate ELSE statement.

```

data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 TotalTime=(timemin*60)+timesec;
 retain SumSec 5400;
 sumsec+totaltime;
 length TestLength $ 6 Message $ 20;
 if totaltime>800 then
 do;
 testlength='Long';
 message='Run blood panel';
 end;
 else if 750<=totaltime<=800 then testlength='Normal';
 else if totaltime<750 then TestLength='Short';
run;

```

In the SELECT group below, the statements between DO and END are performed only when the value of Payclass is hourly. Notice that an IF-THEN statement appears in the DO group; the PUT statement executes only when Hours is greater than 40. The second END statement in the program closes the SELECT group.

```
data payroll;
 set salaries;
 select (payclass);
 when ('monthly') amt=salary;
 when ('hourly')
 do;
 amt=hrlywage*min(hrs,40);
 if hrs>40 then put 'CHECK TIMECARD';
 end;
 otherwise put 'PROBLEM OBSERVATION';
end;
run;
```

### Indenting and Nesting DO Groups

You can nest DO groups to any level, just like you nest IF-THEN/ELSE statements.

*Note:* The memory capabilities of your system might limit the number of nested DO statements that you can use.

Here is an example of nested DO groups:

```
do;
 statements;
 do;
 statements;
 do;
 statements;
 end;
 end;
end;
```

**TIP** It is good practice to indent the statements in DO groups, as shown in the preceding statements, so that their position indicates the levels of nesting.

*Note:* There are three other forms of the DO statement:

- The iterative DO statement executes statements between DO and END statements repetitively based on the value of an index variable. The iterative DO statement can contain a WHILE or UNTIL clause.
- The DO UNTIL statement executes statements in a DO loop repetitively until a condition is true, checking the condition after each iteration of the DO loop.
- The DO WHILE statement executes statements in a DO loop repetitively while a condition is true, checking the condition before each iteration of the DO loop.

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

1. Given the following data set, which program creates the output shown below?

	StockNum	Finish	Style	Item	Price
1	310	oak	pedestal	table	229.99
2	311	maple	pedestal	table	369.99
3	312	brass	floor	lamp	79.99
4	313	glass	table	lamp	59.99
5	314	oak	rocking	chair	153.99
6	315	oak	pedestal	table	178.99
7	316	glass	table	lamp	49.99
8	317	maple	pedestal	table	169.99
9	318	maple	rocking	chair	199.99

StockNum	Finish	Style	Item	TotalPrice
310	oak	pedestal	table	229.99
311	maple	pedestal	table	599.98
312	brass	floor	lamp	679.97
313	glass	table	lamp	739.96
316	glass	table	lamp	789.95
317	maple	pedestal	table	959.94
318	maple	rocking	chair	1159.93

- a. 

```
data test2;
 set sasuser.furniture;
 if finish='oak';
 if price<100 then delete;
 TotalPrice+price;
 drop price;
run;
proc print data=test2 noobs;
run;
```
- b. 

```
data test2;
 set sasuser.furniture;
 if finish='oak' and price<200;
 TotalPrice+price;
 drop price;
run;
proc print data=test2 noobs;
run;
```
- c. 

```
data test2;
 set sasuser.furniture;
 if finish='oak' and price<200 then delete;
 TotalPrice+price;
 drop price;
run;
```

```
proc print data=test2 noobs;
run;
```

- d. data test2;  
     set sasuser.furniture;  
     if finish='oak' and price<100 then do;  
         TotalPrice+price;  
         drop price;  
     end;  
 run;  
 proc print data=test2 noobs;  
 run;

2. How is the variable Amount labeled and formatted in the PROC PRINT output?

```
data credit;
 infile creddata;
 input Account $ 1-5 Name $ 7-25 Type $ 27
 Transact $ 29-35 Amount 37-50;
 label amount='Amount of Loan';
 format amount dollar12.2;
run;
proc print data=credit label;
 label amount='Total Amount Loaned';
 format amount comma10.;
run;
```

- a. label Amount of Loan, format DOLLAR12.2  
 b. label Total Amount Loaned, format COMMA10.  
 c. label Amount, default format  
 d. The PROC PRINT step does not execute because two labels and two formats are assigned to the same variable.
3. Consider the IF-THEN statement shown below. When the statement is executed, which expression is evaluated first?

```
if finlexam>=95
 and (research='A' or
 (project='A' and present='A'))
 then Grade='A+';
```

- a. finlexam>=95  
 b. research='A'  
 c. project='A' and present='A'  
 d. research='A' or  
     (project='A' and present='A')
4. Consider the small raw data file and program shown below. What is the value of Count after the fourth record is read?

```
data work.newnums;
 infile numbers;
 input Tens 2-3;
 Count+tens;
run;
```

1	2	3	4	5	6	7	8	9	10
10									
20									
40									
50									

- a. missing
- b. 0
- c. 30
- d. 70

5. Consider the revised program below. What is the value of Count after the third observation is read?

```
data work.newnums;
 infile numbers;
 input Tens 2-3;
 retain Count 100;
 count+tens;
run;
```

1	2	3	4	5	6	7	8	9	10
10									
20									
40									
50									

- a. missing
- b. 0
- c. 100
- d. 130

6. For the observation shown below, what is the result of the IF-THEN statements?

Status	Type	Count	Action	Control
OK	3	12	E	Go

```
if status='OK' and type=3
 then Count+1;
if status='S' or action='E'
 then Control='Stop';
```

- a. Count = 12      Control = Go
- b. Count = 13      Control =Stop
- c. Count = 12      Control =Stop
- d. Count = 13      Control = Go

7. Which of the following can determine the length of a new variable?

- a. the length of the variable's first reference in the DATA step

- b. the assignment statement
- c. the LENGTH statement
- d. all of the above

8. Which set of statements is equivalent to the code shown below?

```
if code='1' then Type='Fixed';
if code='2' then Type='Variable';
if code^='1' and code^='2' then Type='Unknown';
```

- a. if code='1' then Type='Fixed';  
else if code='2' then Type='Variable';  
else Type='Unknown';
- b. if code='1' then Type='Fixed';  
if code='2' then Type='Variable';  
else Type='Unknown';
- c. if code='1' then type='Fixed';  
else code='2' and type='Variable';  
else type='Unknown';
- d. if code='1' and type='Fixed';  
then code='2' and type='Variable';  
else type='Unknown';

9. What is the length of the variable Type, as created in the DATA step below?

```
data finance.newloan;
 set finance.records;
 TotLoan+payment;
 if code='1' then Type='Fixed';
 else Type='Variable';
 length type $ 10;
run;
```

- a. 5
- b. 8
- c. 10
- d. It depends on the first value of Type.

10. Which program contains an error?

- a. data clinic.stress(drop=timemin timesec);  
infile tests;  
input ID \$ 1-4 Name \$ 6-25 RestHR 27-29 MaxHR 31-33  
RecHR 35-37 TimeMin 39-40 TimeSec 42-43  
Tolerance \$ 45;  
TotalTime=(timemin\*60)+timesec;  
SumSec+totaltime;  
run;
- b. proc print data=clinic.stress;  
label totaltime='Total Duration of Test';  
format timemin 5.2;  
drop sumsec;  
run;
- c. proc print data=clinic.stress(keep=totaltime timemin);  
label totaltime='Total Duration of Test';

```
format timemin 5.2;
run;

d. data clinic.stress;
 infile tests;
 input ID $ 1-4 Name $ 6-25 RestHR 27-29 MaxHR 31-33
 RecHR 35-37 TimeMin 39-40 TimeSec 42-43
 Tolerance $ 45;
 TotalTime=(timemin*60)+timesec;
 keep id totaltime tolerance;
run;
```



## Chapter 12

# Reading SAS Data Sets

---

<b>Reading a Single Data Set to Create Another</b> .....	<b>255</b>
<b>Manipulating Data</b> .....	<b>257</b>
Selected Useful Statements .....	257
Example: Manipulating Data .....	257
Where to Specify the DROP= and KEEP= Data Set Options .....	258
<b>Using BY-Group Processing</b> .....	<b>259</b>
Finding the First and Last Observations in a Group .....	259
Example: Finding the First and Last Observations in a Group .....	259
Finding the First and Last Observations in Subgroups .....	261
Example: Finding the First and Last Observations in Subgroups .....	261
<b>Writing Observations Explicitly</b> .....	<b>263</b>
<b>Detecting the End of a Data Set</b> .....	<b>264</b>
Overview .....	264
Example: The END= Option .....	264
Understanding How Data Sets Are Read .....	266
Compilation Phase .....	266
Execution Phase .....	267
<b>Chapter Quiz</b> .....	<b>273</b>

---

## Reading a Single Data Set to Create Another

The data set Sasuser.Admit contains health information about patients in a clinic, their activity level, height, and weight. Suppose you want to create a subset of the data. Specifically, you want to create a small data set containing data about all the men in the group who are older than 50.

To create the data set, you must first reference the library in which Sasuser.Admit is stored. Then you must specify the name of the library in which you want to store the Males data set. Finally, you write a DATA step to read your data and create a new data set.

Syntax, DATA step for reading a single data set:

```
DATA SAS-data-set;
 SET SAS-data-set;
 <...more SAS statements...>
```

**RUN;**

- *SAS-data-set* in the DATA statement is the name of the SAS data set to be created.
- *SAS-data-set* in the SET statement is the name of the SAS data set to be read.

The DATA step below reads all observations and variables from the existing data set Sasuser.Admit into the new data set Males. The DATA statement creates the permanent SAS data set Males, which is stored in the SAS library Men50. The SET statement reads the permanent SAS data set Sasuser.Admit and subsets the data using a WHERE statement. The new data set, Males, contains all males in Sasuser.Admit who are older than 50.

```
libname sasuser "C:\Users\name\sasuser\";
libname Men50 "C:\Users\name\sasuser\Men50";
data Men50.males;
set sasuser.admit;
 where sex='M' and age>50;
run;
```

When you submit this DATA step, the following messages appear in the log, confirming that the new data set was created:

#### Log 12.1 SAS Log Output

```
9226 data Men50.males;
9227 set sasuser.admit;
9228 where sex='M' and age>50;
9229 run;

NOTE: There were 3 observations read from the data set SASUSER.ADMIT.
 WHERE (sex='M') and (age>50);
NOTE: The data set MEN50.MALES has 3 observations and 9 variables.
NOTE: DATA statement used (Total process time):
 real time 0.00 seconds
 cpu time 0.00 seconds
```

You can add a PROC PRINT statement to this same example to see the output of Men50.Males.

```
proc print data=Men50.males;
 title "Men Over 50";
run;
```

**Figure 12.1** PROC PRINT Output for the Data Set Males**Men Over 50**

Obs	ID	Name	Sex	Age	Date	Height	Weight	ActLevel	Fee
1	2539	LaMance, K	M	51	4	71	158	LOW	124.80
2	2579	Underwood, K	M	60	22	71	191	LOW	149.75
3	2595	Warren, C	M	54	7	71	183	MOD	149.75

## Manipulating Data

### Selected Useful Statements

Here are examples of statements that accomplish specific data-manipulation tasks.

**Table 12.1** Manipulating Data Using the DATA Step

Task	Example Code
Subset data	if resthr<70 then delete; if tolerance='D';
Drop unwanted variables	drop timemin timesec;
Create or modify a variable	TotalTime=(timemin*60)+timesec;
Initialize and retain a variable Accumulate values	retain SumSec 5400; sumsec+totaltime;
Specify a variable's length	length TestLength \$ 6;
Execute statements conditionally	if totaltime>800 then TestLength='Long'; else if 750<=totaltime<=800 then TestLength='Normal'; else if totaltime<750 then TestLength='Short';
Label a variable Format a variable	label sumsec='Cumulative Total Seconds'; format sumsec comma6.;

### Example: Manipulating Data

The following DATA step reads the data set clinic.cltrials, selects observations and variables, and creates new variables.

```
data research.drug1h(drop=placebo uric);
```

```

set clinic.cltrials(drop=triglyc);
 if sex='M' then delete;
 if placebo='YES';
 retain TestDate'22MAY2000'd;
 retain Days 30;
 days+1;
 length Retest $ 5;
 if cholesterol>190 then retest='YES';
 else if 150<=cholesterol<=190 then retest='CHECK';
 else if cholesterol<150 then retest='NO';
 label retest='Perform Cholesterol Test 2?';
run;

```

To view the HTML output of the research.drug1h, use the PROC PRINT procedure.

```

proc print data=research.drug1h;
run;

```

**Figure 12.2** PROC PRINT Output: research.drug1h

Obs	Name	Sex	Cholesterol	TestDate	Days	Retest
1	Johnson,R	F	200	09AUG2000	31	YES
2	Nunnelly,A	F	210	22MAY2000	32	YES

### Where to Specify the DROP= and KEEP= Data Set Options

Recall that you can specify the DROP= and KEEP= data set options anywhere you name a SAS data set. You can specify DROP= and KEEP= in either the DATA statement or the SET statement, depending on whether you want to drop variables from either the output or the source data set:

- If you never reference certain variables and you do not want them to appear in the new data set, use a DROP= option in the SET statement.

In the DATA step shown below, the DROP= or KEEP= option in the SET statement prevents the variables Triglycerides and UricAcid from being read. These variables do not appear in the Lab23.Drug1H data set.

```

data research.drug1h(drop=placebo);
 set clinic.cltrials(drop=triglycerides uricacid);
 if placebo='YES';
run;

```

- If you do need to reference a variable in the original data set (in a subsetting IF statement, for example), you can specify the variable in the DROP= or KEEP= option in the DATA statement. Otherwise, the statement that references the variable uses a missing value for that variable.

This DATA step uses the variable Placebo to select observations. To drop Placebo from the new data set, the DROP= option must appear in the DATA statement.

When used in the DATA statement, the DROP= option simply drops the variables from the new data set. However, they are still read from the original data set and are available within the DATA step.

## Using BY-Group Processing

### *Finding the First and Last Observations in a Group*

Chapter 5, “Creating List Reports,” on page 67 explained how to use a BY statement in PROC SORT to sort observations and in PROC PRINT to group observations for subtotals. You can also use the BY statement in the DATA step to group observations for processing.

```
data temp;
 set salary;
 by dept;
run;
```

When you use the BY statement with the SET statement, here are requirements and results:

- The data sets that are listed in the SET statement must either be sorted by the values of the BY variables, or they must have an appropriate index.
- The DATA step creates two temporary variables for each BY variable. One is named FIRST.variable, where *variable* is the name of the BY variable, and the other is named LAST.variable. Their values are either 1 or 0. The FIRST.variable and LAST.variable identify the first and last observations, respectively, in each BY group.

**Table 12.2** *Finding the First and Last Observations in a Group*

Variable	Equivalent
FIRST.variable	1 for the first observation in a BY group 0 for any other observation in a BY group
LAST.variable	1 for the last observation in a BY group 0 for any other observation in a BY group

### **Example: Finding the First and Last Observations in a Group**

The Sasuser.USA data set contains payroll information for individual employees. Suppose you want to compute the annual payroll by department. Assume 2,000 work hours per year for hourly employees.

Before computing the annual payroll, you need to group observations by values of the variable Dept.

**Figure 12.3** Sample Data Set (partial output)

### The SAS System

Obs	Dept	WageCat	WageRate	Manager	JobType
1	ADM10	S	3392.50	Coxe	3
2	ADM10	S	3420.00	Coxe	50
3	ADM10	S	6862.50	Coxe	50
4	ADM10	H	13.65	Coxe	240
5	ADM20	S	4522.50	Coxe	240
6	ADM20	S	2960.00	Delgado	240
7	ADM20	S	5260.00	Delgado	240
8	ADM20	S	1572.50	Delgado	420
9	ADM30	S	3819.20	Delgado	420
10	ADM30	S	1813.30	Delgado	440
11	CAM10	S	6855.90	Overby	1
12	CAM10	S	4045.80	Overby	5
13	CAM20	S	4480.50	Overby	10
14	ADM10	S	5910.80	Overby	20
15	CAM10	S	9073.80	Overby	20

The following program computes the annual payroll by department. Notice that the variable name Dept has been appended to FIRST. and LAST.

```
proc sort data=sasuser.usa out=work.temp;
 by dept;
run;
data finance.budget(keep=dept payroll);
 set work.temp;
 by dept;
 if wagecat='S' then Yearly=wagerate*12;
 else if wagecat='H' then Yearly=wagerate*2000;
 if first.dept then Payroll=0;
 payroll+yearly;
 if last.dept;
run;
```

If you could look behind the scenes at the program data vector (PDV) as the Finance.Budget data set is created, you would see the following. Notice the values for FIRST.Dept and LAST.Dept.

**Figure 12.4** Program Data Vector

_N_	Dept	Yearly	Payroll	FIRST.Dept	LAST.Dept
1	ADM10	40710.0	40710.0	1	0
2	ADM10	41040.0	81750.0	0	0
3	ADM10	82350.0	164100.0	0	0
4	ADM10	70929.6	235029.6	0	1
6	ADM20	27300.0	27300.0	1	0
7	ADM20	54270.0	81570.0	0	0
8	ADM20	35520.0	117090.0	0	0
9	ADM20	63120.0	180210.0	0	0
10	ADM20	18870.0	199080.0	0	1
11	ADM30	45830.4	45830.4	1	0

When you print the new data set, you can now list and sum the annual payroll by department.

```
proc print data=finance.budget noobs;
 sum payroll;
 format payroll dollar12.2;
run;
```

**Figure 12.5** Payroll Sum

### The SAS System

Dept	Payroll
ADM10	\$262,329.60
ADM20	\$171,780.00
ADM30	\$67,590.00
CAM10	\$239,706.00
CAM20	\$53,766.00
	<b>\$795,171.60</b>

### Finding the First and Last Observations in Subgroups

When you specify multiple BY variables, the following statements are true:

- The FIRST.variable for each variable is set to 1 at the first occurrence of a new value for the primary variable.
- A change in the value of a primary BY variable forces LAST.variable to equal 1 for the secondary BY variables.

### Example: Finding the First and Last Observations in Subgroups

Suppose you now want to compute the annual payroll by job type for each manager. In your program, you specify two BY variables, Manager and JobType.

```
proc sort data=sasuser.usa out=work.temp2;
```

```

 by manager job_type;
data finance.budget2(keep=manager job_type payroll);
 set work.temp2;
 by manager job_type;
 if wagecat='S' then Yearly=wagerate*12;
 else if wagecat='H' then Yearly=wagerate*2000;
 if first.job_type then Payroll=0;
 payroll+yearly;
 if last.job_type;
run;

```

If you could look at the PDV now, you would see the following. Notice that the values for FIRST.JobType and LAST.JobType change according to values of FIRST.Manager and LAST.Manager.

**Figure 12.6** Sample Program Data Vector

_N_	Manager	JobType	Yearly Salary	Payroll	FIRST. Manager	LAST. Manager	FIRST. JobType	LAST. JobType
1	Coxe	3	40710.0	40710.0	1	0	1	1
2	Coxe	50	41040.0	41040.0	0	0	1	0
3	Coxe	50	82350.0	123390.0	0	0	0	1
4	Coxe	240	27300.0	27300.0	0	0	1	0
5	Coxe	240	54270.0	81570.0	0	1	0	1
6	Delgado	240	35520.0	35520.0	1	0	1	0
7	Delgado	240	63120.0	98640.0	0	0	0	1
8	Delgado	420	18870.0	18870.0	0	0	1	0
9	Delgado	420	45830.0	64700.4	0	0	0	1
10	Delgado	440	21759.6	21759.6	0	1	1	1
11	Overby	1	82270.8	82270.8	1	0	1	1
12	Overby	5	48550.2	48549.6	0	0	1	1
13	Overby	10	53766.0	53766.0	0	0	1	1
14	Overby	20	70929.0	70929.0.0	0	0	1	0
15	Overby	20	108885.0	179815.2	0	1	0	1

Now you can sum the annual payroll by job type for each manager. Here, the payroll for only two managers (Coxe and Delgado) is listed.

```

proc print data=finance.budget2 noobs;
 by manager;
 var job_type;
 sum payroll;
 where manager in ('Coxe','Delgado');
 format payroll dollar12.2;
run;

```



**Figure 12.7** Payroll Sum by Job Type and Manager**Manager=Coxe**

Job_Type	Payroll
3	\$40,704.00
50	\$205,734.00
240	\$135,834.00
<b>Manager</b>	<b>\$382,272.00</b>

**Manager=Delgado**

Job_Type	Payroll
240	\$161,760.00
420	\$110,528.40
440	\$21,756.00
<b>Manager</b>	<b>\$294,044.40</b>
	<b>\$676,316.40</b>

---

## Writing Observations Explicitly

To override the default way in which the DATA step writes observations to output, you can use an OUTPUT statement in the DATA step. Placing an explicit OUTPUT statement in a DATA step overrides the automatic output, so that observations are added to a data set only when the explicit OUTPUT statement is executed.

---

Syntax, OUTPUT statement:

**OUTPUT** <SAS-data-set(s)>;

*SAS-data-set(s)* names the data set or data sets to which the observation is written. All data set names that are specified in the OUTPUT statement must also appear in the DATA statement.

Using an OUTPUT statement without a following data set name causes the current observation to be written to all data sets that are specified in the DATA statement.

---

With an OUTPUT statement, your program now writes a single observation to output—observation 5.

```
data work.getobs5;
 obsnum=5;
 set sasuser.usa(keep=manager wage rate) point=obsnum;
 output;
 stop;
run;
proc print data=work.getobs5 noobs;
run;
```

**Figure 12.8** Single Observation

obsnum	WageRate	Manager
5	3392.5	Coxe

Suppose your DATA statement contains two data set names, and you include an OUTPUT statement that references only one of the data sets. The DATA step creates both data sets, but only the data set that is specified in the OUTPUT statement contains output. For example, the program below creates two temporary data sets, Empty and Full. The result of this DATA step is that the data set Empty is created but contains no observations, and the data set Full contains all of the observations from Sasuser.Usa.

```
data empty full;
 set sasuser.usa;
 output full;
run;
```

---

## Detecting the End of a Data Set

### Overview

Instead of reading specific observations, you might want to determine when the last observation in an input data set has been read, so that you can perform specific processing. For example, you might want to write only one observation that contains grand totals for numeric variables.

To create a temporary numeric variable whose value is used to detect the last observation, use the END= option in the SET statement.

---

Syntax, END= option:

**END=***variable*

*variable* creates and names a temporary variable that contains an end-of-file marker. The variable, which is initialized to 0, is set to 1 when the SET statement reads the last observation of the data set.

This variable is not added to the data set.

---

### Example: The END= Option

Suppose you want to sum the number of seconds for treadmill stress tests. If you submit the following program, you produce a new data set that contains cumulative totals for each of the values of TotalTime.

```
data work.addtoend(drop=timemin timesec);
 set sasuser.stress2(keep=timemin timesec);
 TotalMin+timemin;
 TotalSec+timesec;
 TotalTime=totalmin*60+totalsec;
run;
```

```
proc print data=work.addtoend noobs;
run;
```

**Figure 12.9** Data Set with Cumulative Totals for Each of the Values of TotalTime

TotalMin	TotalSec	TotalTime
12	38	758
22	43	1363
33	56	2036
42	98	2618
53	144	3324
65	170	4070
80	211	5011
93	224	5804
103	246	6426
119	295	7435
134	297	8337
146	308	9068
160	317	9917
171	343	10603
185	370	11470
198	389	12269
214	396	13236
231	431	14291
246	472	15232
260	529	16129
272	539	16859

The following program uses the END= variable in the SET statement. The END= variable selects only the last observation of the data set.

```
data work.addtoend(drop=timemin timesec);
 set sasuser.stress2(keep=timemin timesec) end=last;
 TotalMin+timemin;
 TotalSec+timesec;
 TotalTime=totalmin*60+totalsec;
 if last;
run;
proc print data=work.addtoend noobs;
run;
```

Now the new data set has one observation:

**Figure 12.10** Data Set with One Observation

TotalMin	TotalSec	TotalTime
272	539	16859

### Understanding How Data Sets Are Read

DATA step processing for reading existing SAS data sets is very similar to the compilation and execution phases of the DATA step. The main difference is that while reading an existing data set with the SET statement, SAS retains the values of existing variables from one observation to the next.

In the following example, the DATA step reads the data set Finance.Loans, creates the variable Interest, and creates the new data set Finance.DueJan.

```
data finance.duejan;
 set finance.loans;
 Interest=amount*(rate/12);
run;
```

**Figure 12.11** New Data Set Finance.DueJan

Obs	Account	Amount	Rate	Months	Payment	Interest
1	101-1092	\$22,000	10.00%	60	\$467.43	183.333
2	101-1731	\$114,000	9.50%	360	\$958.57	902.500
3	101-1289	\$10,000	10.50%	36	\$325.02	87.500
4	101-3144	\$3,500	10.50%	12	\$308.52	30.625
5	103-1135	\$8,700	10.50%	24	\$403.47	76.125
6	103-1994	\$18,500	10.00%	60	\$393.07	154.167
7	103-2335	\$5,000	10.50%	48	\$128.02	43.750
8	103-3864	\$87,500	9.50%	360	\$735.75	692.708
9	103-3891	\$30,000	9.75%	360	\$257.75	243.750

### Compilation Phase

1. The program data vector is created and contains the automatic variables `_N_` and `_ERROR_`.

Program Data Vector

<u>N</u>	<u>ERROR</u>	
1	0	

2. SAS also scans each statement in the DATA step, looking for syntax errors.

- When the SET statement is compiled, a slot is added to the program data vector for each variable in the input data set. The input data set supplies the variable names, as well as attributes such as type and length.

Program Data Vector

<b>_N_</b>	<b>_ERROR_</b>	<b>Account</b>	<b>Amount</b>	<b>Rate</b>	<b>Months</b>	<b>Payment</b>

- Any variables that are created in the DATA step are also added to the program data vector. The attributes of each of these variables are determined by the expression in the statement.

Program Data Vector

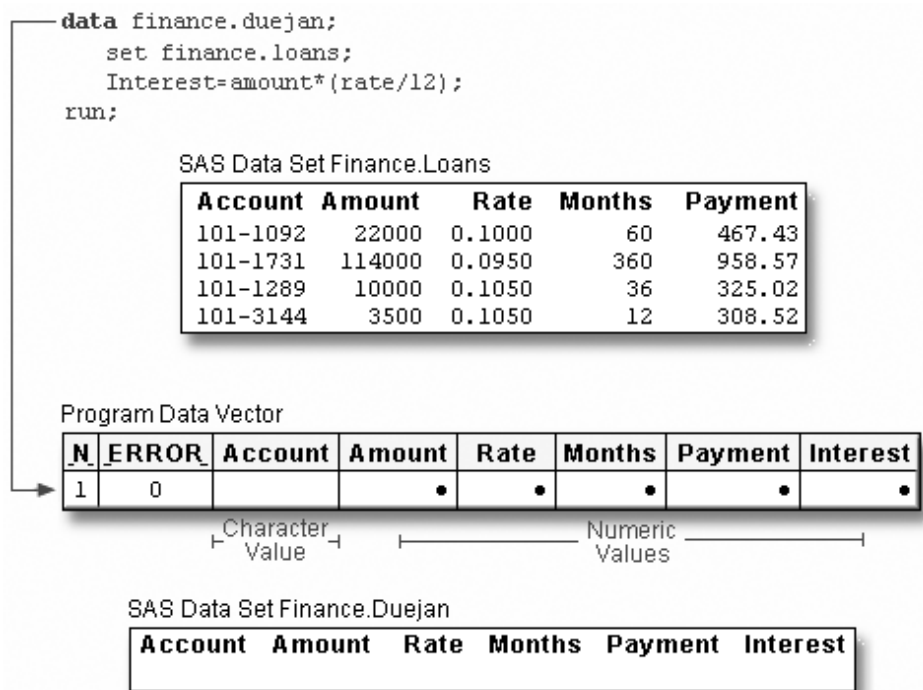
<b>_N_</b>	<b>_ERROR_</b>	<b>Account</b>	<b>Amount</b>	<b>Rate</b>	<b>Months</b>	<b>Payment</b>	<b>Interest</b>

- At the bottom of the DATA step, the compilation phase is complete, and the descriptor portion of the new SAS data set is created. There are no observations because the DATA step has not yet executed.

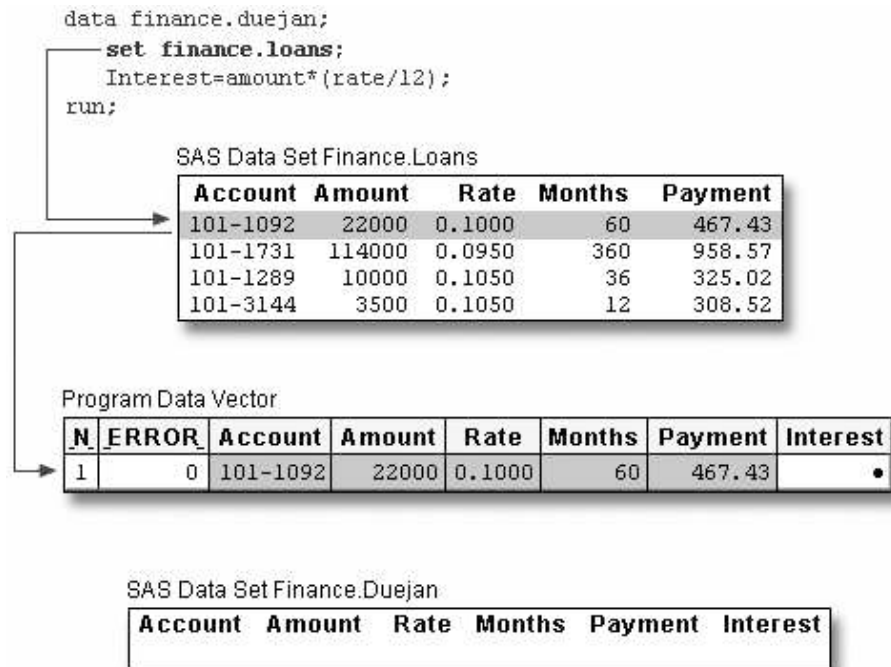
When the compilation phase is complete, the execution phase begins.

## Execution Phase

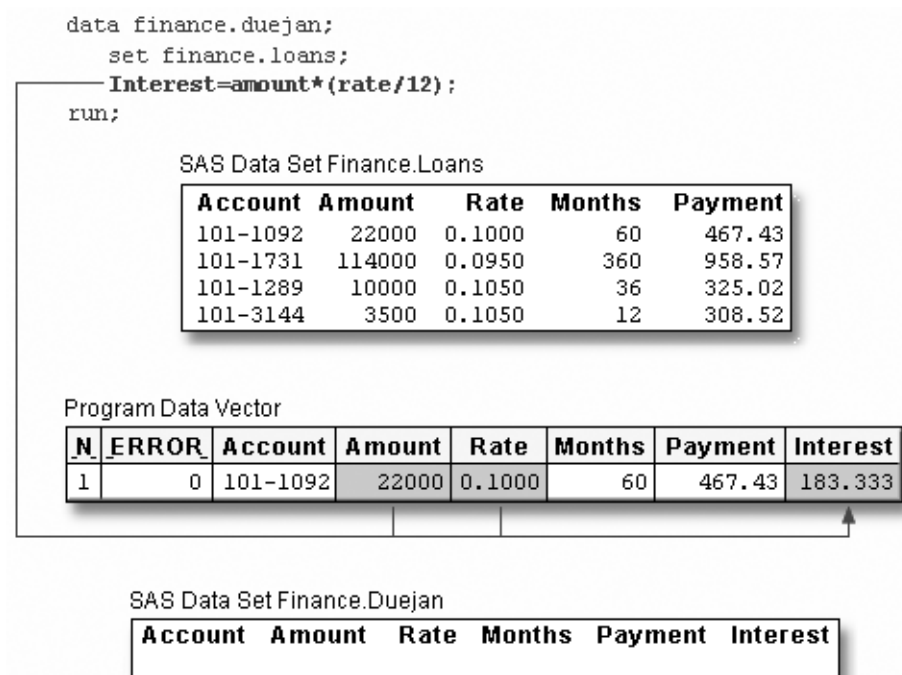
- The DATA step executes once for each observation in the input data set. For example, this DATA step executes four times because there are four observations in the input data set Finance.Loans.
- At the beginning of the execution phase, the value of **\_N\_** is 1. Because there are no data errors, the value of **\_ERROR\_** is 0. The remaining variables are initialized to missing. Missing numeric values are represented by a period, and missing character values are represented by a blank.



3. The SET statement reads the first observation from the input data set into the program data vector.



4. Then, the assignment statement executes to compute the value for Interest.



5. At the end of the first iteration of the DATA step, the values in the program data vector are written to the new data set as the first observation.

```
data finance.duejan;
 set finance.loans;
 Interest=amount*(rate/12);
run;
```

SAS Data Set Finance.Loans

Account	Amount	Rate	Months	Payment
101-1092	22000	0.1000	60	467.43
101-1731	114000	0.0950	360	958.57
101-1289	10000	0.1050	36	325.02
101-3144	3500	0.1050	12	308.52

Program Data Vector

<u>N</u>	<u>ERROR</u>	<u>Account</u>	<u>Amount</u>	<u>Rate</u>	<u>Months</u>	<u>Payment</u>	<u>Interest</u>
1	0	101-1092	22000	0.1000	60	467.43	183.333

SAS Data Set Finance.Duejan

Account	Amount	Rate	Months	Payment	Interest
101-1092	22000	0.1000	60	467.43	183.333

6. The value of N increments from 1 to 2, and control returns to the top of the DATA step. Recall that the automatic variable N keeps track of how many times the DATA step has begun to execute.

```
data finance.duejan;
 set finance.loans;
 Interest=amount*(rate/12);
run;
```

SAS Data Set Finance.Loans

Account	Amount	Rate	Months	Payment
101-1092	22000	0.1000	60	467.43
101-1731	114000	0.0950	360	958.57
101-1289	10000	0.1050	36	325.02
101-3144	3500	0.1050	12	308.52

Program Data Vector

<u>N</u>	<u>ERROR</u>	<u>Account</u>	<u>Amount</u>	<u>Rate</u>	<u>Months</u>	<u>Payment</u>	<u>Interest</u>
2	0	101-1092	22000	0.1000	60	467.43	183.333

SAS Data Set Finance.Duejan

Account	Amount	Rate	Months	Payment	Interest
101-1092	22000	0.1000	60	467.43	183.333

7. SAS retains the values of variables that were read from a SAS data set with the SET statement, or that were created by a sum statement. All other variable values, such as the values of the variable Interest, are set to missing.

```

data finance.duejan;
 set finance.loans;
 Interest=amount*(rate/12);
run;

```

SAS Data Set Finance.Loans

Account	Amount	Rate	Months	Payment
101-1092	22000	0.1000	60	467.43
101-1731	114000	0.0950	360	958.57
101-1289	10000	0.1050	36	325.02
101-3144	3500	0.1050	12	308.52

Program Data Vector

N	ERROR	Account	Amount	Rate	Months	Payment	Interest
2	0	101-1092	22000	0.1000	60	467.43	.

Values Retained      Reset to Missing

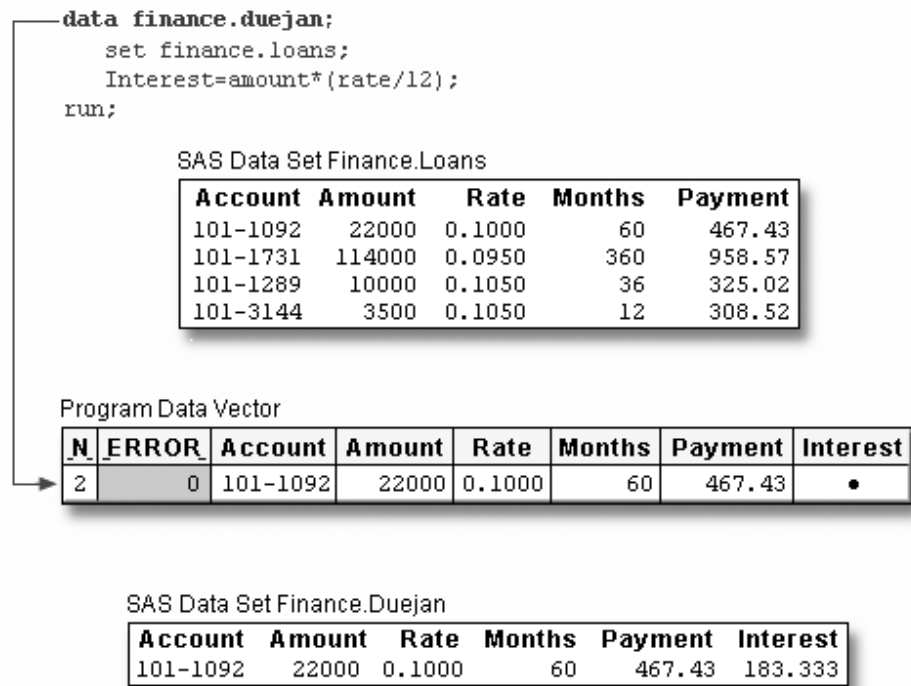
SAS Data Set Finance.Duejan

Account	Amount	Rate	Months	Payment	Interest
101-1092	22000	0.1000	60	467.43	183.333

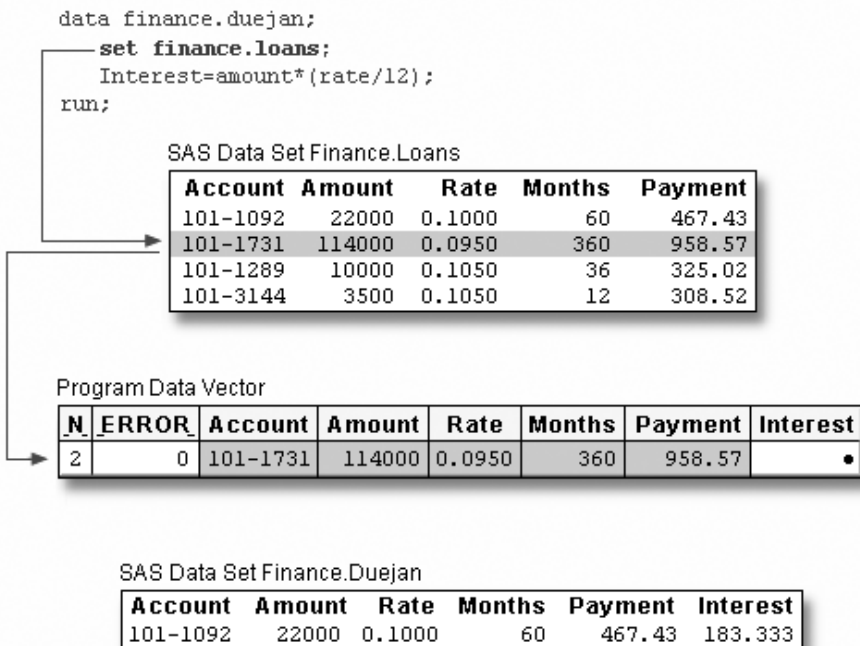
*Note:* When SAS reads raw data, the situation is different. In that case, SAS sets the value of each variable in the DATA step to missing at the beginning of each iteration, with these exceptions.

- variables named in a RETAIN statement
  - variables created in a sum statement
  - data elements in a \_TEMPORARY\_ array
  - any variables created by using options in the FILE or INFILE statements
  - automatic variables
8. At the beginning of the second iteration, the value of \_ERROR\_ is reset to 0.





9. As the SET statement executes, the values from the second observation are read into the program data vector.



10. The assignment statement executes again to compute the value for Interest for the second observation.

```
data finance.duejan;
 set finance.loans;
 Interest=amount*(rate/12);
run;
```

SAS Data Set Finance.Loans

Account	Amount	Rate	Months	Payment
101-1092	22000	0.1000	60	467.43
101-1731	114000	0.0950	360	958.57
101-1289	10000	0.1050	36	325.02
101-3144	3500	0.1050	12	308.52

Program Data Vector

<u>N</u>	<u>ERROR</u>	<u>Account</u>	<u>Amount</u>	<u>Rate</u>	<u>Months</u>	<u>Payment</u>	<u>Interest</u>
2	0	101-1731	114000	0.0950	360	958.57	902.5

SAS Data Set Finance.Duejan

Account	Amount	Rate	Months	Payment	Interest
101-1092	22000	0.1000	60	467.43	183.333

11. At the bottom of the DATA step, the values in the program data vector are written to the data set as the second observation.

```
data finance.duejan;
 set finance.loans;
 Interest=amount*(rate/12);
run;
```

SAS Data Set Finance.Loans

Account	Amount	Rate	Months	Payment
101-1092	22000	0.1000	60	467.43
101-1731	114000	0.0950	360	958.57
101-1289	10000	0.1050	36	325.02
101-3144	3500	0.1050	12	308.52

Program Data Vector

<u>N</u>	<u>ERROR</u>	<u>Account</u>	<u>Amount</u>	<u>Rate</u>	<u>Months</u>	<u>Payment</u>	<u>Interest</u>
2	0	101-1731	114000	0.0950	360	958.57	902.5

SAS Data Set Finance.Duejan

Account	Amount	Rate	Months	Payment	Interest
101-1092	22000	0.1000	60	467.43	183.333
101-1731	114000	0.0950	360	958.57	902.5

12. The value of N increments from 2 to 3, and control returns to the top of the DATA step. SAS retains the values of variables that were read from a SAS data set with the SET statement, or that were created by a sum statement. All other variable values, such as the values of the variable Interest, are set to missing.

```

data finance.duejan;
 set finance.loans;
 Interest=amount*(rate/12);
run;

```

SAS Data Set Finance.Loans

Account	Amount	Rate	Months	Payment
101-1092	22000	0.1000	60	467.43
101-1731	114000	0.0950	360	958.57
101-1289	10000	0.1050	36	325.02
101-3144	3500	0.1050	12	308.52

Program Data Vector

N	ERROR	Account	Amount	Rate	Months	Payment	Interest
3	0	101-1731	114000	0.0950	360	958.57	.

└──────────────────┘
Values Retained
└────────┘
Reset to Missing

SAS Data Set Finance.Duejan

Account	Amount	Rate	Months	Payment	Interest
101-1092	22000	0.1000	60	467.43	183.333
101-1731	114000	0.0950	360	958.57	902.5

This process continues until all of the observations are read.

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- If you submit the following program, which variables appear in the new data set?
 

```

data work.cardiac(drop=age group);
 set clinic.fitness(keep=age weight group);
 if group=2 and age>40;
run;

```

  - none
  - Weight
  - Age, Group
  - Age, Weight, Group
- Which of the following programs correctly reads the data set Orders and creates the data set FastOrdr?
  - ```

data catalog.fastordr(drop=ordertime);
  set july.orders(keep=product units price);
  if ordertime<4;
  Total=units*price;
run;

```
 - ```

data catalog.orders(drop=ordertime);
 set july.fastordr(keep=product units price);

```

```

 if ordertime<4;
 Total=units*price;
run;

c. data catalog.fastordr(drop=ordertime);
 set july.orders(keep=product units price
 ordertime);
 if ordertime<4;
 Total=units*price;
run;

```

d. none of the above

3. Which of the following statements is false when you use the BY statement with the SET statement?
- The data sets listed in the SET statement must be indexed or sorted by the values of the BY variable or variables.
  - The DATA step automatically creates two variables, FIRST. and LAST., for each variable in the BY statement.
  - FIRST. and LAST. identify the first and last observation in each BY group, respectively.
  - FIRST. and LAST. are stored in the data set.
4. There are 500 observations in the data set Usa. What is the result of submitting the following program?

```

data work.getobs5;
 obsnum=5;
 set sasuser.usa(keep=manager payroll);
 stop;
run;

```

- an error
  - an empty data set
  - continuous loop
  - a data set that contains one observation
5. The data set clinic.heart contains 20 observations. What is the result of submitting the following program?
- ```

data work.subset;
    set clinic.heart (firstobs=5 obs=4);
run;

```
- The program executes with no WARNING or ERROR messages. The data set work.subset contains 4 observations.
 - The program executes with no WARNING or ERROR messages. The data set work.subset contains 0 observations.
 - The program executes with a WARNING message. The data set work.subset contains 4 observations.
 - The program terminates execution because of an ERROR message. The data set work.subset contains 0 observations.
6. Assuming that the data set Sasuer.Usa has five or more observations, what is the result of submitting the following program?

```
data work.getobs5;  
  obsnum=5;  
  set sasuser.usa(keep=manager payroll) point=obsnum;  
  output;  
  stop;  
run;
```

- a. an error
 - b. an empty data set
 - c. a continuous loop
 - d. a data set that contains one observation
7. What is the result of submitting the following program?
- ```
data work.addtoend;
 set clinic.stress2 end=last;
 if last;
run;
```
- a. an error
  - b. an empty data set
  - c. a continuous loop
  - d. a data set that contains the last observation from clinic.stress2
8. At the start of DATA step processing, during the compilation phase, variables are created in the program data vector (PDV), and observations are set to which of the following:
- a. blank
  - b. missing
  - c. 0
  - d. there are no observations.
9. To override the DATA step default behavior that writes observations to output, what should you use in the DATA step?
- a. DROP= and KEEP= data set options
  - b. OUTPUT statement
  - c. END= option
  - d. BY statement



## Chapter 13

# Combining SAS Data Sets

---

<b>How to Prepare Your Data Sets</b> .....	<b>278</b>
Determining the Structure and Contents of the Data Sets .....	278
Testing Your Program .....	278
Looking at Sources of Common Problems .....	278
<b>Methods of Combining SAS Data Sets: The Basics</b> .....	<b>278</b>
<b>One-to-One Reading: Details</b> .....	<b>280</b>
One-to-One Reading Syntax .....	280
How One-to-One Reading Selects Data .....	281
How One-to-One Reading Works .....	281
Example: Using One-to-One Reading to Combine Data Sets .....	283
<b>Concatenating: Details</b> .....	<b>284</b>
Concatenating Syntax .....	284
How Concatenating Selects Data .....	284
Example: Using Concatenating to Combine Data Sets .....	285
<b>Appending: Details</b> .....	<b>286</b>
The APPEND Procedure .....	286
Requirements for the APPEND Procedure .....	287
Example: Using the PROC APPEND Statement to Combine Data Sets .....	287
Using the FORCE Option with Unlike-Structured Data Sets .....	288
Example: Appending with FORCE Option .....	288
<b>Interleaving: Details</b> .....	<b>289</b>
Interleaving Syntax .....	289
How Interleaving Selects Data .....	290
Example: Using Interleaving to Combine Data Sets .....	290
<b>Match-Merging: Details</b> .....	<b>291</b>
Match-Merging Syntax .....	291
How Match-Merging Selects Data .....	292
Example: Using Match-Merging to Combine Data Sets .....	292
Example: Merge in Descending Order .....	294
<b>Match-Merge Processing</b> .....	<b>295</b>
The Basics of Match-Merge Processing .....	295
The Compilation Phase: Setting Up a New Data Set .....	296
The Execution Phase: Match-Merging Observations .....	297
Handling Unmatched Observations and Missing Values .....	298
<b>Renaming Variables</b> .....	<b>300</b>
The Basics of Renaming Variables .....	300

RENAME Statement Syntax .....	301
Example: Renaming Variables .....	301
<b>Excluding Unmatched Observations .....</b>	<b>302</b>
Overview .....	302
Identifying Observation in Both Data Sets .....	302
Selecting Matching Observations .....	303
<b>Selecting Variables .....</b>	<b>304</b>
Where to Specify DROP= and KEEP= Data Set Options .....	304
<b>Chapter Quiz .....</b>	<b>305</b>

---

## How to Prepare Your Data Sets

### *Determining the Structure and Contents of the Data Sets*

To help determine how your data is related, look at the structure of the data sets. To see the structure, use the DATASETS procedure. To display the descriptor information, use the CONTENTS procedure. Descriptor information includes the number of rows in each data set, the name and attributes of each column, and an alphabetic list of extended attributes (including data set and column extended attributes). To print a sample of the rows, use the PRINT procedure or the REPORT procedure.

### *Testing Your Program*

Create small temporary data sets that contain a sample of rows that test all of your program's logic. If your logic is faulty and you get unexpected output, you can debug your program.

### *Looking at Sources of Common Problems*

If your program does not run correctly, review your input data for the following errors:

- columns that have the same name but that represent different data  
 To correct the error, you can rename columns before you combine the data sets by using the RENAME= table option in the SET or MERGE statement. As an alternative, use the DATASETS procedure to display all library management functions for all member types (except catalogs).
- common columns that have the same data but different attributes

---

## Methods of Combining SAS Data Sets: The Basics

A common task in SAS programming is to combine observations from two or more data sets into a new data set. Using the DATA step, you can combine data sets in several ways.



**Table 13.1** Quick-Reference Overview of Data-Combining Methods

Method of Combining	Illustration																																						
One-to-one reading   Creates observations that contain all of the variables from each contributing data set.   Combines observations based on their relative position in each data set.   Statement: SET	SAS Data Set C <table><thead><tr><th>Num</th><th>VarA</th></tr></thead><tbody><tr><td>1</td><td>A1</td></tr><tr><td>3</td><td>A2</td></tr><tr><td>5</td><td>A3</td></tr></tbody></table>    SAS Data Set D <table><thead><tr><th>Num</th><th>VarB</th></tr></thead><tbody><tr><td>2</td><td>B1</td></tr><tr><td>4</td><td>B2</td></tr></tbody></table>    Merge  Combined SAS Data Set <table><thead><tr><th>Num</th><th>VarA</th><th>VarB</th></tr></thead><tbody><tr><td>2</td><td>A1</td><td>B1</td></tr><tr><td>4</td><td>A2</td><td>B2</td></tr></tbody></table>	Num	VarA	1	A1	3	A2	5	A3	Num	VarB	2	B1	4	B2	Num	VarA	VarB	2	A1	B1	4	A2	B2															
Num	VarA																																						
1	A1																																						
3	A2																																						
5	A3																																						
Num	VarB																																						
2	B1																																						
4	B2																																						
Num	VarA	VarB																																					
2	A1	B1																																					
4	A2	B2																																					
Concatenating   Appends the observations from one data set to another.   Statement: SET	SAS Data Set A <table><thead><tr><th>Num</th><th>VarA</th></tr></thead><tbody><tr><td>1</td><td>A1</td></tr><tr><td>2</td><td>A2</td></tr><tr><td>3</td><td>A3</td></tr></tbody></table>    SAS Data Set C <table><thead><tr><th>Num</th><th>VarB</th></tr></thead><tbody><tr><td>1</td><td>B1</td></tr><tr><td>2</td><td>B2</td></tr><tr><td>4</td><td>B3</td></tr></tbody></table>    Concatenate  Combined SAS Data Set <table><thead><tr><th>Num</th><th>VarA</th><th>VarB</th></tr></thead><tbody><tr><td>1</td><td>A1</td><td></td></tr><tr><td>2</td><td>A2</td><td></td></tr><tr><td>3</td><td>A3</td><td></td></tr><tr><td>1</td><td></td><td>B1</td></tr><tr><td>2</td><td></td><td>B2</td></tr><tr><td>4</td><td></td><td>B3</td></tr></tbody></table>	Num	VarA	1	A1	2	A2	3	A3	Num	VarB	1	B1	2	B2	4	B3	Num	VarA	VarB	1	A1		2	A2		3	A3		1		B1	2		B2	4		B3	
Num	VarA																																						
1	A1																																						
2	A2																																						
3	A3																																						
Num	VarB																																						
1	B1																																						
2	B2																																						
4	B3																																						
Num	VarA	VarB																																					
1	A1																																						
2	A2																																						
3	A3																																						
1		B1																																					
2		B2																																					
4		B3																																					
Appending   Adds the observations in the second data set directly to the end of the original data set.   Procedure: APPEND	BASE= data set A <table><thead><tr><th>Num</th><th>Sex</th></tr></thead><tbody><tr><td>1</td><td>M</td></tr><tr><td>2</td><td>F</td></tr><tr><td>3</td><td>F</td></tr><tr><td>4</td><td>M</td></tr></tbody></table>    DATA= data set B <table><thead><tr><th>Num</th><th>Sex</th></tr></thead><tbody><tr><td>3</td><td>F</td></tr><tr><td>4</td><td>F</td></tr><tr><td>5</td><td>F</td></tr><tr><td>6</td><td>M</td></tr></tbody></table>    Append  BASE= data set A <table><thead><tr><th>Num</th><th>Sex</th></tr></thead><tbody><tr><td>1</td><td>M</td></tr><tr><td>2</td><td>F</td></tr><tr><td>3</td><td>F</td></tr><tr><td>4</td><td>M</td></tr><tr><td>3</td><td>F</td></tr><tr><td>4</td><td>F</td></tr><tr><td>5</td><td>F</td></tr><tr><td>6</td><td>M</td></tr></tbody></table> with data set B observations appended	Num	Sex	1	M	2	F	3	F	4	M	Num	Sex	3	F	4	F	5	F	6	M	Num	Sex	1	M	2	F	3	F	4	M	3	F	4	F	5	F	6	M
Num	Sex																																						
1	M																																						
2	F																																						
3	F																																						
4	M																																						
Num	Sex																																						
3	F																																						
4	F																																						
5	F																																						
6	M																																						
Num	Sex																																						
1	M																																						
2	F																																						
3	F																																						
4	M																																						
3	F																																						
4	F																																						
5	F																																						
6	M																																						

**Method of Combining****Illustration****Interleaving**

Intersperses observations in order from two or more data sets, based on one or more common variables.

Statements: SET, BY

SAS Data Set C

Num	Var
1	C1
2	C2
2	C3
3	C4

SAS Data Set D

Num	Var
2	D1
3	D2
3	D3

Interleave

Combined SAS Data Set

Num	Var
1	C1
2	C2
2	C3
2	D1
3	C4
3	D2
3	D3

**Match-merging**

Matches observations from two or more data sets into a single observation in a new data set according to the values of a common variable.

Statements: MERGE, BY

SAS Data Set A

Num	VarA
1	A1
2	A2
3	A3

SAS Data Set B

Num	VarB
1	B1
2	B2
4	B3

Match-Merge

Combined SAS Data Set

Num	VarA	VarB
1	A1	B1
2	A2	B2
3	A3	
4		B3

**TIP** You can also use PROC SQL to join data sets according to common values.

## One-to-One Reading: Details

### One-to-One Reading Syntax

Use multiple SET statements in a DATA step to combine data sets. One-to-one reading combines rows from two or more data sets by creating rows that contain all of the columns from each contributing data set. Rows are combined based on their relative position in each data set. That is, the first row in one data set is combined with the first in the other, and so on. The data program stops after it has read the last row from the smallest data set.

Syntax, DATA step for one-to-one reading:

```
DATA output-SAS-data-set;
 SET SAS-data-set-1;
 SET SAS-data-set-2;
RUN;
```

- *output-SAS-data-set* names the data set to be created.
- *SAS-data-set-1* and *SAS-data-set-2* specify the data sets to be read.

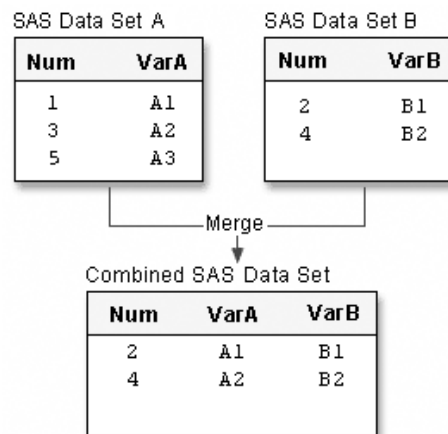
## How One-to-One Reading Selects Data

The following statements are true when you perform one-to-one reading:

- The new data set contains all the variables from all the input data sets. If the data sets contain variables that have the same names, the values that are read from the last data set overwrite the values that were read from earlier data sets.
- The number of observations in the new data set is the number of observations in the smallest original data set. Observations are combined based on their relative position in each data set. That is, the first observation in one data set is joined with the first observation in the other, and so on. The DATA step stops after it has read the last observation from the smallest data set.

```
data one2one;
 set a;
 set b;
run;
```

**Figure 13.1** One-to-One Reading



## How One-to-One Reading Works

Here is a simple example of one-to-one reading.

```
data one2one;
 set a;
 set b;
run;
```

1. The first SET statement reads the first observation from data set A into the PDV.

---

Num	VarA	VarB
1	A1	

---

2. The second SET statement reads the first observation from data set B into the PDV, and SAS writes the contents of the PDV to the new data set. The value for Num from data set B overwrites the value for Num from data set A.

---

Num	VarA	VarB
2	A1	B1

SAS Data Set

Num	VarA	VarB
2	A1	B1

---

3. The second SET statement reads from data set A into the PDV.

---

Num	VarA	VarB
3	A2	

---

4. The second SET statement reads the second observation from data set B, and SAS writes the contents of the PDV to the new data set. The value for Num from data set B overwrites the value for Num from data set A.

---

Num	VarA	VarB
4	A2	B2

SAS Data Set

Num	VarA	VarB
2	A1	B1
4	A2	B2

---

5. The first SET statement reads the third observation from data set A into the PDV.

---

Program Data Vector		
Num	VarA	VarB
5	A3	

---

6. The second SET statement reads the end of file in data set B, which stops the DATA step processing with no further output written to the data set. The last observation in data set A is read into the PDV, but it is not written to the output data set.

Num	VarA	VarB
2	A1	B1
4	A2	B2

### Example: Using One-to-One Reading to Combine Data Sets

Suppose you have basic patient data (ID, sex, and age) in the data set Clinic.Patients and want to combine it with other patient data (height and weight) for patients under age 60. The height and weight data is stored in the data set Clinic.Measure. Both data sets are sorted by the variable ID.

Notice that Clinic.Patients contains seven observations in which the patient age is less than 60, and Clinic.Measure contains six observations.

**Figure 13.2** Example: One-to-One Reading

**SAS Data Set Clinic.Patients**

	ID	Sex	Age
1	1129	F	48
2	1837	F	57
3	2304	F	16
4	2486	F	63
5	4759	F	60
6	5438	F	42
7	6488	F	59
8	9012	F	39
9	9125	F	56

**SAS Data Set Clinic. Measure**

	ID	Height	Weight
1	1129	61	137
2	1387	64	142
3	2304	61	102
4	5438	62	168
5	6488	64	154
6	9012	63	157

To subset observations from the first data set and combine them with observations from the second data set, you can submit the following program:

```
data clinic.one2one;
 set clinic.patients;
 if age<60;
 set clinic.measure;
run;
```

The resulting data set, Clinic.One2one, contains six observations (the number of observations read from the smallest data set, which is Clinic.Measure). The last

observation in Clinic.Patients is not written to the data set because the second SET statement reaches an end-of-file, which stops the DATA step processing.

**Figure 13.3** The Resulting Data Set for One-to-One Reading Example

	ID	Sex	Age	Height	Weight
1	1129	F	48	61	137
2	1387	F	57	64	142
3	2304	F	16	61	102
4	5438	F	42	62	168
5	6488	F	59	64	154
6	9012	F	39	63	157

---

## Concatenating: Details

### Concatenating Syntax

Another way to combine SAS data sets with the SET statement is concatenating, which appends the observations from one data set to another data set. To concatenate SAS data sets, you specify a list of data set names in the SET statement.

---

Syntax, DATA step for concatenating:

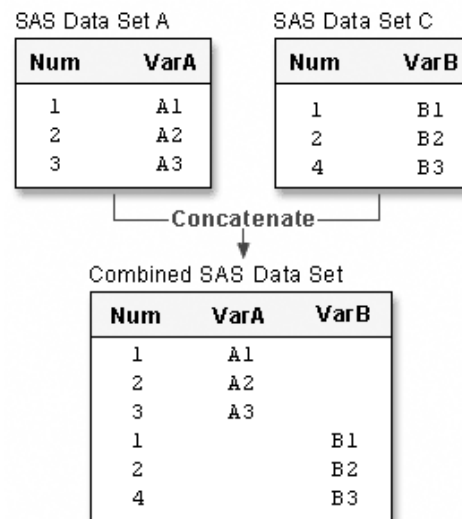
```
DATA output-SAS-data-set;
 SET SAS-data-set-1 SAS-data-set-2;
RUN;
```

- *output-SAS-data-set* names the data set to be created.
  - *SAS-data-set-1* and *SAS-data-set-2* specify the data sets to be read.
- 

### How Concatenating Selects Data

When a program concatenates data sets, all of the observations are read from the first data set listed in the SET statement. Then all of the observations are read from the second data set listed, and so on, until all of the listed data sets have been read. The new data set contains all of the variables and observations from all of the input data sets.

```
data concat;
 set a c;
run;
```

**Figure 13.4** How Concatenating Selects

Notice that A and C contain a common variable named Num:

- Both instances of Num (or any common variable) must have the same type attribute, or SAS stops processing the DATA step and issues an error message stating that the variables are incompatible.
- However, if the length attribute is different, SAS takes the length from the first data set that contains the variable. In this case, the length of Num in A determines the length of Num in Concat.
- The same is true for the label, format, and informat attributes: If any of these attributes are different, SAS takes the attribute from the first data set that contains the variable with that attribute.

### **Example: Using Concatenating to Combine Data Sets**

The following DATA step creates Clinic.Concat by concatenating Clinic.Therapy2012 and Clinic.Therapy2013. Each data set contains 12 observations.

```
data clinic.concat;
 set clinic.therapy2012 clinic.therapy2013;
run;
```

The first 12 observations in the new output data set Clinic.Concat were read from Clinic.Therapy2012, and the last 12 observations were read from Clinic.Therapy2013.

**Figure 13.5** Example: Concatenating (partial output)

Obs	Month	Year	AerClass	WalkJogRun	Swim
1	01	2012	26	78	14
2	02	2012	32	109	19
3	03	2012	15	106	22
4	04	2012	47	115	24
5	05	2012	95	121	31
20	08	2013	63	65	53
21	09	2013	60	49	68
22	10	2013	78	70	41
23	11	2013	82	44	58
24	12	2013	93	57	47

---

## Appending: Details

### The APPEND Procedure

Another way to combine SAS data sets is to append one data set to another using the APPEND procedure. Although appending and concatenating are similar, there are some important differences between the two methods. Whereas the DATA step creates an entirely new data set when concatenating, PROC APPEND simply adds the observations of one data set to the end of a “master” (or BASE) data set. SAS does not create a new data set nor does it read the base data set when executing the APPEND procedure.

To append SAS data sets, you specify a BASE= data set, which is the data set to which observations are added and then specify a DATA= data set, which is the data set containing the observations that are added to the base data set. The data set specified with DATA= is the only one of the two data sets that SAS actually reads.

---

Syntax, APPEND procedure:

**PROC APPEND** BASE=SAS-data-set

DATA=SAS-data-set;

**RUN;**

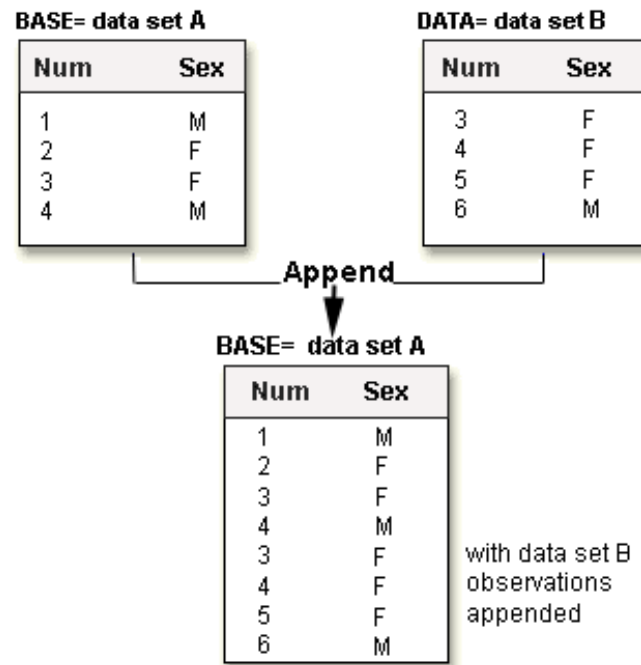
- BASE= names the data set to which observations are added.
  - DATA= names the data set containing observations that are added to the base data set.
- 

For example, the following PROC APPEND statement appends the observations in data set B to the end of data set A:

```
proc append base=A
 data=B;
run;
```



Figure 13.6 How PROC APPEND Appends Data



### Requirements for the APPEND Procedure

Here are the requirements for appending one data set to another:

- Only two data sets can be used at a time in one step.
- The variable information in the descriptor portion of the base data set cannot change.

*Note:* The final data set is the original data set with appended observations. The observations in the BASE= data set are not read.

### Example: Using the PROC APPEND Statement to Combine Data Sets

The following PROC APPEND statement appends the data set Totals2011 to the base data set Totals2005. The two data sets are like-structured data sets: that is, both data sets have the same variable information.

```
proc append base=totals2005 data=totals2011;
run;
```

Below is the listing of Totals2005. The first four observations already existed in totals2005, and the last four observations were read from totals2011 and added to totals2005.

**Figure 13.7** Example: PROC APPEND

Obs	Month	Therapy	NewAdmit	Treadmill
1	01	220	27	11
2	02	209	63	43
3	03	189	11	2
4	04	211	52	36
5	01	210	27	13
6	02	219	63	23
7	03	199	11	21
8	04	251	52	30

### Using the FORCE Option with Unlike-Structured Data Sets

In order to use PROC APPEND with data sets that have unmatching variable definitions, you can use the FORCE option in the PROC APPEND statement.

---

Syntax, APPEND procedure with the FORCE option:

```
PROC APPEND BASE=SAS-data-set
 DATA=SAS-data-set FORCE;
RUN;
```

---

The FORCE option is needed when the DATA= data set contains variables that meet any one of the following criteria:

- They are not in the BASE= data set.
- They are variables of a different type (for example, character or numeric).
- They are longer than the variables in the BASE= data set.

If the length of a variable is longer in the DATA= data set than in the BASE= data set, SAS truncates values from the DATA= data set to fit them into the length that is specified in the BASE= data set.

If the type of a variable in the DATA= data set is different from that in the BASE= data set, SAS replaces all values for the variable in the DATA= data set with missing values and keeps the variable type of the variable specified in the BASE= data set.

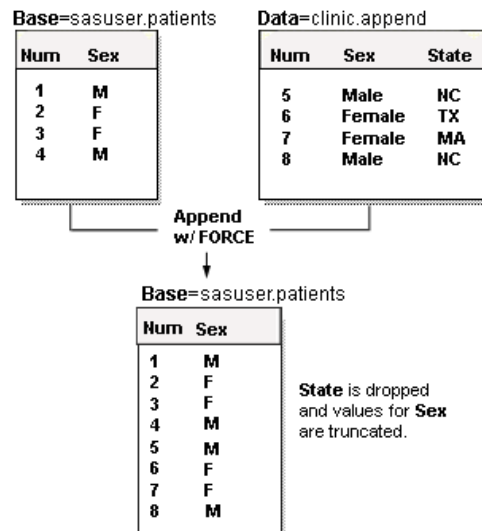
If the BASE= data set contains a variable that is not in the DATA= data set, the observations are appended, but the observations from the DATA= data set have a missing value for the variable that was not present in the DATA= data set. If the DATA= data set contains a variable that is not in the BASE= data set, the variable is dropped from the output.

### Example: Appending with FORCE Option

```
proc append base=sasuser.patients
 data=clinic.append force;
```

```
run;
```

**Figure 13.8** Example: PROC APPEND with FORCE Option



## Interleaving: Details

### Interleaving Syntax

When you use a BY statement to concatenate data sets, the result is interleaving. Interleaving intersperses observations from two or more data sets, based on one or more common variables.

To interleave SAS data sets, specify a list of data set names in the SET statement, and specify one or more BY variables in the BY statement.

Syntax, DATA step for interleaving:

```
DATA output-SAS-data-set;
 SET SAS-data-set-1 SAS-data-set-2;
 BY variable(s);
RUN;
```

- *output-SAS-data-set* names the data set to be created.
- *SAS-data-set-1* and *SAS-data-set-2* specify the data sets to be read.
- *variable(s)* specifies one or more variables that are used to interleave observations.

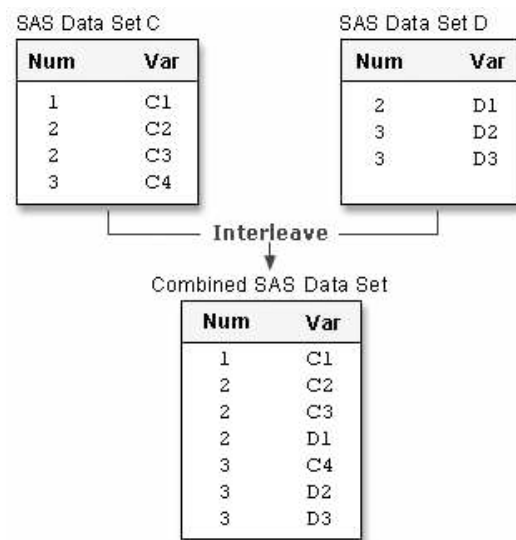
**TIP** Specify any number of data sets in the SET statement. Each input data set must be sorted or indexed in ascending order based on the BY variable or variables.

### How Interleaving Selects Data

When SAS interleaves data sets, observations in each BY group in each data set in the SET statement are read sequentially, in the order in which the data sets and BY variables are listed, until all observations have been processed. The new data set includes all the variables from all the input data sets, and it contains the total number of observations from all input data sets.

```
data interlv;
 set c d;
 by num;
run;
```

**Figure 13.9** How Interleaving Selects Data



### Example: Using Interleaving to Combine Data Sets

The following DATA step creates Clinic.Interlv by interleaving Clinic.Therapy2012 and Clinic.Therapy2013.

```
data clinic.interlv;
 set clinic.therapy2012 clinic.therapy2013;
 by month;
run;
```

Below is the listing of Clinic.Interlv. Notice that, unlike the previous example, observations are interleaved by month instead of being concatenated.

**Figure 13.10** Example: How Interleaving Selects Data (partial output)

Obs	Month	Year	AerClass	WalkJogRun	Swim
1	01	2012	26	78	14
2	01	2013	37	91	83
3	02	2012	32	109	19
4	02	2013	41	102	27
20	10	2013	78	70	41
21	11	2012	84	31	52
22	11	2013	82	44	58
23	12	2012	92	44	55
24	12	2013	93	57	47

---

## Match-Merging: Details

### Match-Merging Syntax

Match-merging combines observations from two or more data sets into a single observation in a new data set according to the values of a common variable.

When match-merging, use the MERGE statement rather than the SET statement to combine data sets.

---

Syntax, DATA step for match-merging:

```
DATA output-SAS-data-set;
 MERGE SAS-data-set-1 SAS-data-set-2;
 BY <DESCENDING> variable(s);
RUN;
```

- *output-SAS-data-set* names the data set to be created.
- *SAS-data-set-1* and *SAS-data-set-2* specify the data sets to be read.
- *variable(s)* in the BY statement specifies one or more variables whose values are used to match observations.
- DESCENDING indicates that the input data sets are sorted in descending order (largest to smallest numerically, or reverse alphabetical for character variables) by the variable that is specified. If you have more than one variable in the BY statement, DESCENDING applies only to the variable that immediately follows it. The default sort order is ASCENDING.

---

**TIP** Each input data set in the MERGE statement must be sorted in order of the values of the BY variable or variables, or it must have an appropriate index. Each BY variable must have the same type in all data sets to be merged.

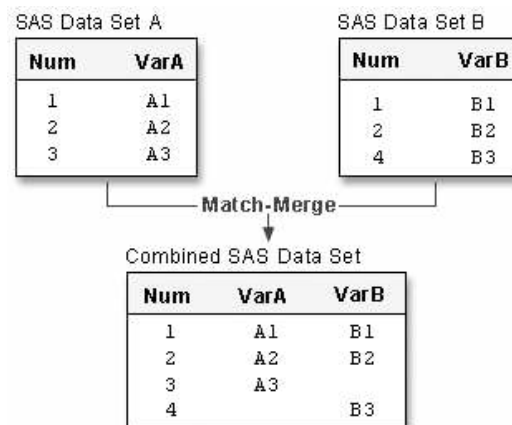
**TIP** You cannot use the DESCENDING option with indexed data sets because indexes are always stored in ascending order.

### How Match-Merging Selects Data

During match-merging SAS sequentially checks each observation of each data set to see whether the BY values match and then writes the combined observation to the new data set.

```
data merged;
 merge a b;
 by num;
run;
```

**Figure 13.11** How Match-Merging Selects Data



Basic DATA step match-merging produces an output data set that contains values from all observations in all input data sets. You can add statements and options to select only matching observations.

If an input data set does not have any observations for a particular value of the BY-variable, then the observation in the output data set contains missing values for the variables that are unique to that input data set.

**TIP** In match-merging, often one data set contains unique values for the BY-variable and other data sets contain multiple values for the BY-variable.

### Example: Using Match-Merging to Combine Data Sets

The data sets Clinic.Demog and Clinic.Visit have been sorted as follows:

```
proc sort data=clinic.demog;
 by id;
run;
proc print data=clinic.demog;
run;
```

**Figure 13.12** HTML Output: Sorting Clinic.Demog

Obs	ID	Age	Sex	Date
1	A001	21	m	05/22/07
2	A002	32	m	06/15/06
3	A003	24	f	08/17/07
4	A004	.		01/27/06
5	A005	44	f	02/24/05
6	A007	39	m	11/11/05

```
proc sort data=clinic.visit;
 by id;
run;
proc print data=clinic.visit;
run;
```

**Figure 13.13** HTML Output: Sorting Clinic.Visit

Obs	ID	Visit	SysBP	DiasBP	Weight	Date
1	A001	1	140	85	195	11/05/09
2	A001	2	138	90	198	10/13/09
3	A001	3	145	95	200	07/04/09
4	A002	1	121	75	168	04/14/09
5	A003	1	118	68	125	08/12/09
6	A003	2	112	65	123	08/21/09
7	A004	1	143	86	204	03/30/09
8	A005	1	132	76	174	02/27/09
9	A005	2	132	78	175	07/11/09
10	A005	3	134	78	176	04/16/09
11	A008	1	126	80	182	05/22/09

You can then submit this DATA step to create Clinic.Merged by merging Clinic.Demog and Clinic.Visit according to values of the variable ID.

```
data clinic.merged;
 merge clinic.demog clinic.visit;
 by id;
run;
proc print data=clinic.merged;
run;
```

*Note:* All observations, including unmatched observations and observations that have missing data, are written to the output data set.

**Figure 13.14** HTML Output: Match-Merging Output

Obs	ID	Age	Sex	Date	Visit	SysBP	DiasBP	Weight
1	A001	21	m	11/05/09	1	140	85	195
2	A001	21	m	10/13/09	2	138	90	198
3	A001	21	m	07/04/09	3	145	95	200
4	A002	32	m	04/14/09	1	121	75	168
5	A003	24	f	08/12/09	1	118	68	125
6	A003	24	f	08/21/09	2	112	65	123
7	A004	.	.	03/30/09	1	143	86	204
8	A005	44	f	02/27/09	1	132	76	174
9	A005	44	f	07/11/09	2	132	78	175
10	A005	44	f	04/16/09	3	134	78	176
11	A007	39	m	11/11/05	.	.	.	.
12	A008	.	.	05/22/09	1	126	80	182

**Example: Merge in Descending Order**

The example above illustrates merging two data sets that are sorted in ascending order of the BY variable ID. To sort the data sets in descending order and then merge them, you can submit the following program.

```
proc sort data=clinic.demog;
 by descending id;
run;
proc sort data=clinic.visit;
 by descending id;
run;
data clinic.merged;
 merge clinic.demog clinic.visit;
 by descending id;
run;
proc print data=clinic.merged;
run;
```

*Note:* Specify the DESCENDING option in the BY statements in both the PROC SORT steps and the DATA step. If you omit the DESCENDING option in the DATA step, you generate error messages about improperly sorted BY variables.



**Figure 13.15** HTML Output: Merge in Descending Order

Obs	ID	Age	Sex	Date	Visit	SysBP	DiasBP	Weight
1	A008	.	.	05/22/09	1	126	80	182
2	A007	39	m	11/11/05	.	.	.	.
3	A005	44	f	02/27/09	1	132	76	174
4	A005	44	f	07/11/09	2	132	78	175
5	A005	44	f	04/16/09	3	134	78	176
6	A004	.	.	03/30/09	1	143	86	204
7	A003	24	f	08/12/09	1	118	68	125
8	A003	24	f	08/21/09	2	112	65	123
9	A002	32	m	04/14/09	1	121	75	168
10	A001	21	m	11/05/09	1	140	85	195
11	A001	21	m	10/13/09	2	138	90	198
12	A001	21	m	07/04/09	3	145	95	200

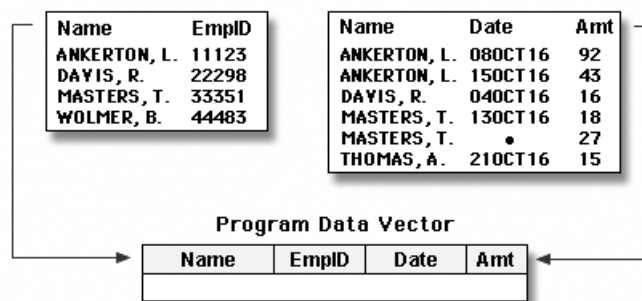
## Match-Merge Processing

### The Basics of Match-Merge Processing

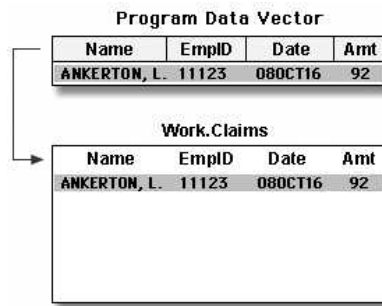
The match-merging examples in this book are straightforward. However, match-merging can be more complex, depending on your data and on the output data set that you want to create. To predict the results of match-merges correctly, you need to understand how the DATA step performs match-merges.

When you submit a DATA step, it is processed in two phases:

- the compilation phase, in which SAS checks the syntax of the SAS statements and compiles them (translates them into machine code). During this phase, SAS also sets up descriptor information for the output data set and creates the PDV, an area of memory where SAS holds one observation at a time.



- the execution phase in which the DATA step reads data and executes any subsequent programming statements. When the DATA step executes, data values are read into the appropriate variables in the PDV. From here, the variables are written to the output data set as a single observation.



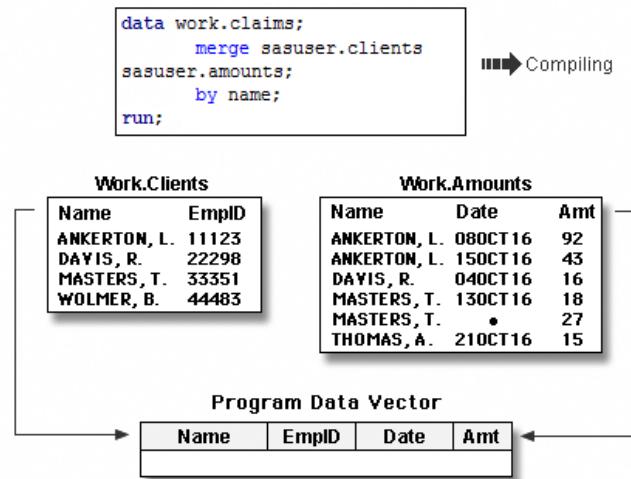
### The Compilation Phase: Setting Up a New Data Set

To prepare to merge data sets, SAS does the following:

- reads the descriptor portions of the data sets that are listed in the MERGE statement
- reads the rest of the DATA step program
- creates the PDV for the merged data set
- assigns a tracking pointer to each data set that is listed in the MERGE statement

If variables that have the same name appear in more than one data set, the variable from the first data set that contains the variable (in the order listed in the MERGE statement) determines the length of the variable.

**Figure 13.16** The Compilation Phase: Setting Up the New Data Set



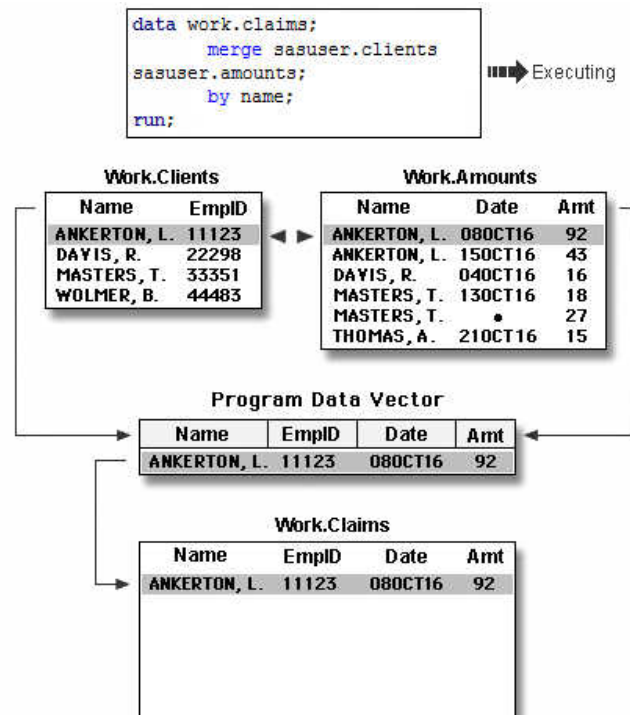
After reading the descriptor portions of the data sets Clients and Amounts, SAS does the following:

1. creates a PDV for the new Claims data set. The PDV contains all variables from the two data sets. Note that although Name appears in both input data sets, it appears in the PDV only once.
2. assigns tracking pointers to Clients and Amounts.

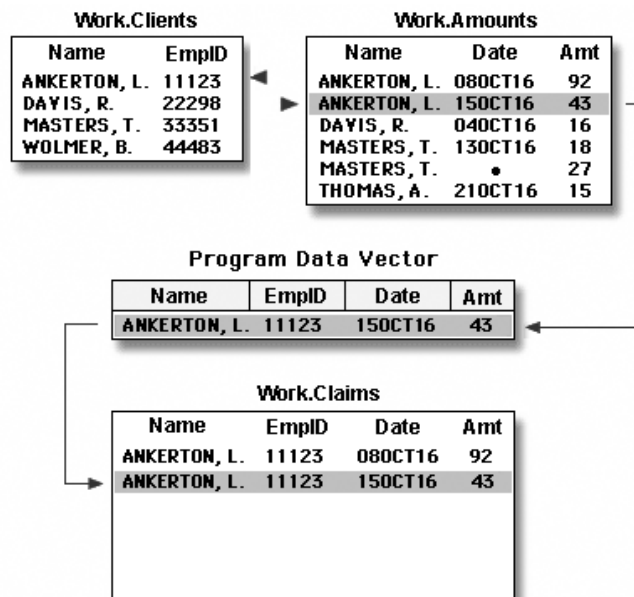
### The Execution Phase: Match-Merging Observations

After compiling the DATA step, SAS sequentially match-merges observations by moving the pointers down each observation of each data set and checking to see whether the BY values match.

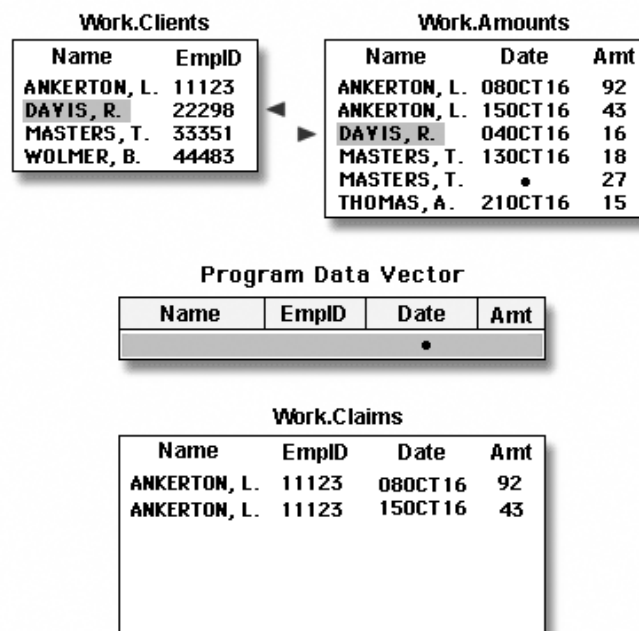
- If the BY values match, the observations are read into the PDV in the order in which the data sets appear in the MERGE statement. Values of any same-named variable are overwritten by values of the same-named variable in subsequent data sets. SAS writes the combined observation to the new data set and retains the values in the PDV until the BY value changes in all the data sets.



- If the BY values do not match, SAS determines which BY value comes first and reads the observation that contains this value into the PDV. Then the contents of the PDV are written.



When the BY value changes in all the input data sets, the PDV is initialized to missing.



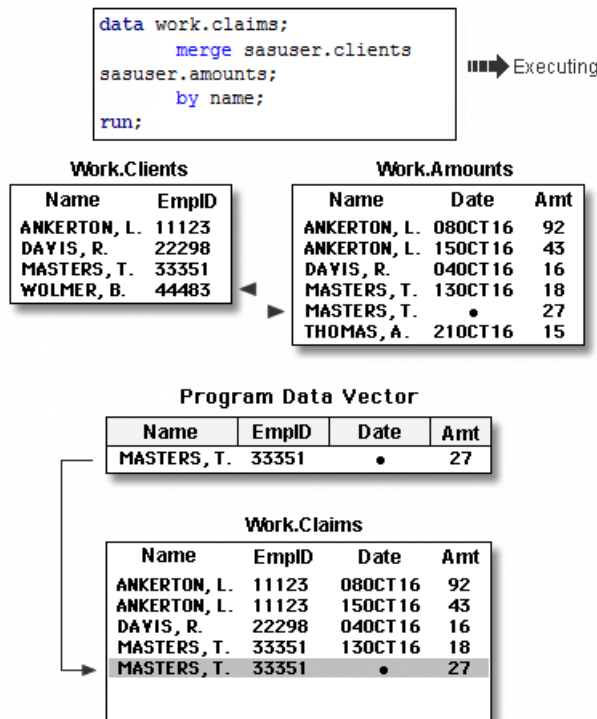
The DATA step merge continues to process every observation in each data set until it has processed all observations in all data sets.

### Handling Unmatched Observations and Missing Values

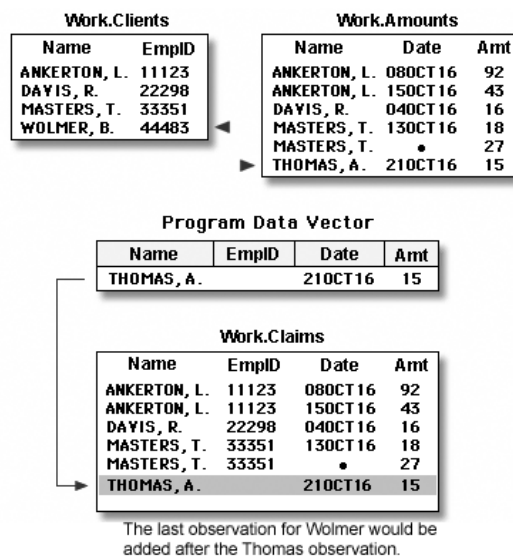
By default, all observations that are read into the PDV, including observations that have missing data and no matching BY values, are written to the output data set. If you specify a subsetting IF statement to select observations, then only those that meet the IF condition are written.

- If an observation contains missing values for a variable, then the observation in the output data set contains the missing values as well. Observations that have missing

values for the BY variable appear at the top of the output data set because missing values sort first in ascending order.



- If an input data set does not have a matching BY value, then the observation in the output data set contains missing values for the variables that are unique to that input data set.



The PROC PRINT output is displayed below. Remember to use the FORMAT statement for the date variable in the PRINT procedure.

```
proc print data=work.claims noobs;
 format Date date9.;
run;
```

**Figure 13.17** PROC PRINT Output of Merged Data

Name	EmplID	Date	Amount
ANKERTON, L.	11123	08OCT2016	92
ANKERTON, L.	11123	15OCT2016	43
DAVIS, R.	22298	04OCT2016	16
MASTERS, T.	33351	13OCT2016	18
MASTERS, T.	33351	.	27
THOMAS, A.	.	21OCT2016	15
WOLMER, B.	44483	.	.

---

## Renaming Variables

### *The Basics of Renaming Variables*

DATA step match-merging overwrites values of the like-named variable in the first data set in which it appears with values of the like-named variable in subsequent data sets.

Consider Clinic.Demog, which contains the variable Date (date of birth), and Clinic.Visit, which also contains Date (date of the clinic visit in 1998). The DATA step below overwrites the date of birth with the date of the clinic visit.

```
data clinic.merged;
 merge clinic.demog clinic.visit;
 by id;
run;
proc print data=clinic.merged;
run;
```

The following output shows the effects of overwriting the values of a variable in the Clinic.Merged data set. In most observations, the date is now the date of the clinic visit. In observation 11, the date is still the birthdate because Clinic.Visit did not contain a matching ID value and did not contribute to the observation.

**Figure 13.18** Renaming Variables

Obs	ID	Age	Sex	Date	Visit	SysBP	DiasBP	Weight
1	A001	23	M	11/05/09	1	140	85	195
2	A001	38	M	10/13/09	2	138	90	198
3	A001	35	M	07/04/09	3	145	95	200
4	A002	22	M	04/14/09	1	121	75	168
5	A003	38	F	08/12/09	1	118	68	125
6	A003	41	F	08/21/09	2	112	65	123
7	A004	26		03/30/09	1	143	86	204
8	A005	33	F	02/27/09	1	136	76	174
9	A005	31	F	07/11/09	2	132	78	175
10	A005	29	F	04/16/09	3	134	78	176
11	A007	39	M	11/11/66	.	.	.	.
12	A008	26		05/22/09	1	126	80	182

### RENAME Statement Syntax

To prevent overwriting, you can rename variables by using the RENAME= data set option in the MERGE statement.

---

Syntax, RENAME= data set option:

**(RENAME=(old-variable-name=new-variable-name))**

- the RENAME= option, in parentheses, follows the name of each data set that contains one or more variables to be renamed
  - *old-variable-name* specifies the variable to be renamed.
  - *new-variable-name* specifies the new name for the variable.
- 

**TIP** Use RENAME= to rename variables in the SET statement or in the output data set that is specified in the DATA statement.

### Example: Renaming Variables

In the following example, the RENAME= option renames the variable Date in Clinic.Demog to BirthDate, and it renames the variable Date in Clinic.Visit to VisitDate.

```
data clinic.merged;
 merge clinic.demog (rename=(date=BirthDate))
 clinic.visit (rename=(date=VisitDate));
 by id;
run;
proc print data=clinic.merged;
run;
```

The following output shows the effect of the RENAME= option.

**Figure 13.19** Output for RENAME= Option

Obs	ID	Age	Sex	BirthDate	Visit	SysBP	DiasBP	Weight	VisitDate
1	A001	23	M	05/22/86	1	140	85	195	11/05/09
2	A001	38	M	05/22/86	2	138	90	198	10/13/09
3	A001	35	M	05/22/86	3	145	95	200	07/04/09
4	A002	22	M	06/15/74	1	121	75	168	04/14/09
5	A003	38	F	08/17/83	1	118	68	125	08/12/09
6	A003	41	F	08/17/83	2	112	65	123	08/21/09
7	A004	26		01/27/70	1	143	86	204	03/30/09
8	A005	33	F	02/24/61	1	136	76	174	02/27/09
9	A005	31	F	02/24/61	2	132	78	175	07/11/09
10	A005	29	F	02/24/61	3	134	78	176	04/16/09
11	A007	39	M	11/11/66	.	.	.	.	.
12	A008	26			1	126	80	182	05/22/09

---

## Excluding Unmatched Observations

### Overview

By default, DATA step match-merging combines all observations in all input data sets.

To exclude unmatched observations from your output data set, use the following in your DATA step:

- Use the IN= data set option to create and name a variable that indicates whether the data set contributed data to the current observation.
- Use the subsetting IF statement to check the IN= values and write to the merged data set only matching observations

### Identifying Observation in Both Data Sets

To match-merge the data sets Clinic.Demog and Clinic.Visit and select only observations that appear in both data sets, use IN= to create two temporary variables, Indemog and Invisit. The IN= variable is a temporary variable that is available to program statements during the DATA step, but it is not included in the SAS data set that is being created.



Syntax, IN= data set option:

(IN= *variable*)

- The IN= option, in parentheses, follows the data set name.
- *variable* names the variable to be created.

Within the DATA step, the value of the variable is 1 if the data set contributed data to the current observation. Otherwise, its value is 0.

The DATA step that contains the IN= options appears below. The first IN= creates the temporary variable indemog, which is set to 1 when an observation from Clinic.Demog contributes to the current observation. Otherwise, it is set to 0. Likewise, the value of Invisit depends on whether Clinic.Visit contributes to an observation or not.

```
data clinic.merged;
 merge clinic.demog(in=indemog)
 clinic.visit(in=invisit
 rename=(date=BirthDate));
 by id;
run;
```

**TIP** To specify multiple data set options for a given data set, enclose the options in a single set of parentheses.

## Selecting Matching Observations

To select only observations that appear in both Clinic.Demog and Clinic.Visit, specify a subsetting IF statement in the DATA step.

The subsetting IF statement checks the values of Indemog and Invisit and continues processing only those observations that meet the condition of the expression. The condition is that both Clinic.Demog and Clinic.Visit contribute to the observation. If the condition is met, the new observation is written to Clinic.Merged. Otherwise, the observation is deleted.

```
data clinic.merged;
 merge clinic.demog(in=indemog
 rename=(date=BirthDate))
 clinic.visit(in=invisit
 rename=(date=VisitDate));
 by id;
 if indemog=1 and invisit=1;
run;
proc print data=clinic.merged;
run;
```

In previous examples, Clinic.Merged contained 12 observations. In the output below, notice that only 10 observations met the condition in the IF expression.

**Figure 13.20** Selecting Matching Observations

Obs	ID	Age	Sex	BirthDate	Visit	SysBP	DiasBP	Weight	VisitDate
1	A001	23	M	05/22/86	1	140	85	195	11/05/09
2	A001	38	M	05/22/86	2	138	90	198	10/13/09
3	A001	35	M	05/22/86	3	145	95	200	07/04/09
4	A002	22	M	06/15/74	1	121	75	168	04/14/09
5	A003	38	F	08/17/83	1	118	68	125	08/12/09
6	A003	41	F	08/17/83	2	112	65	123	08/21/09
7	A004	26		01/27/70	1	143	86	204	03/30/09
8	A005	33	F	02/24/61	1	136	76	174	02/27/09
9	A005	31	F	02/24/61	2	132	78	175	07/11/09
10	A005	29	F	02/24/61	3	134	78	176	04/16/09

SAS evaluates the expression within an IF statement to produce a result that is either nonzero, zero, or missing. A nonzero and nonmissing result causes the expression to be true; a zero or missing result causes the expression to be false.

It is possible to specify the subsetting IF statement from the previous example in either of the following ways. The first IF statement checks specifically for a value of 1. The second IF statement checks for a value that is neither missing nor 0 (which for IN= variables is always 1).

```
if indemog=1 and invisit=1;
```

```
if indemog and invisit;
```

---

## Selecting Variables

### Where to Specify DROP= and KEEP= Data Set Options

When match-merging, specify these options in either the DATA statement or the MERGE statement, depending on whether you want to reference the variables in that DATA step:

- To reference a variable in the original data set (in a subsetting IF statement, for example) specify the variable in the DROP= option in the DATA statement. Otherwise, you might get unexpected results and your variable would be uninitialized.

```
data clinic.merged (drop=id);
```

- To not reference certain variables and exclude them from the new data set, specify them in the DROP= option in the MERGE statement.

```
merge clinic.demog(in=indemog
 rename=(date=BirthDate))
 clinic.visit(drop=weight in=invisit
 rename=(date=VisitDate));
```

When used in the DATA statement, the DROP= option simply drops the variables from the new data set. However, the variables are still read from the original data set and are available for processing within the DATA step.

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

1. Which program combines Brothers.One and Brothers.Two to produce Brothers.Three?

Work.One		+	Work.Two		=	Work.Three		
VarX	VarY		VarX	VarZ		VarX	VarY	VarZ
1	Groucho		2	Chico		2	Groucho	Chico
3	Harpo		4	Zeppo		4	Harpo	Zeppo
5	Karl							

- ```
data work.three;
    set work.one;
    set work.two;
run;
proc print data=work.three;
run;
```
 - ```
data work.three;
 set work.one work.two;
run;
proc print data=work.three;
run;
```
  - ```
data work.three;
    set work.one work.two;
    by varx;
run;
proc print data=work.three;
run;
```
 - ```
data work.three;
 merge work.one work.two;
 by varx;
run;
proc print data=work.three;
run;
```
2. Which program combines Actors.Props1 and Actors.Props2 to produce Actors.Props3?

Actors.Props1		+	Actors.Props2		=	Actors.Props3	
Actor	Prop		Actor	Prop		Actor	Prop
Curly	Anvil		Curly	Ladder		Curly	Anvil
Larry	Ladder		Moe	Pliers		Larry	Ladder
Moe	Poker					Moe	Poker
						Curly	Ladder
						Moe	Pliers

- ```

data actors.props3;
  set actors.props1;
  set actors.props2;
run;

```
- ```

data actors.props3;
 set actors.props1 actors.props2;
run;

```
- ```

data actors.props3;
  set actors.props1 actors.props2;
  by actor;
run;

```
- ```

data actors.props3;
 merge actors.props1 actors.props2;
 by actor;
run;

```

3. If you submit the following program, which new data set is created?

Work.Dataone			Work.Datatwo		
Career	Supervis	Finance	Variety	Feedback	Autonomy
72	26	9	10	11	70
63	76	7	85	22	93
96	31	7	83	63	73
96	98	6	82	75	97
84	94	6	36	77	97

```

data work.jobsatis;
 set work.dataone work.datatwo;
run;
proc print data=work.jobsatis noobs;
run;

```

a.

Career	Supervis	Finance	Variety	Feedback	Autonomy
72	26	9	.	.	.
63	76	7	.	.	.
96	31	7	.	.	.
96	98	6	.	.	.
84	94	6	.	.	.
.	.	.	10	11	70
.	.	.	85	22	93
.	.	.	83	63	73
.	.	.	82	75	97
.	.	.	36	77	97

b.

Career	Supervis	Finance	Variety	Feedback	Autonomy
72	26	9	10	11	70
63	76	7	85	22	93
96	31	7	83	63	73
96	98	6	82	75	97
84	94	6	36	77	97

c.

Career	Supervis	Finance
72	26	9
63	76	7
96	31	7
96	98	6
84	94	6
10	11	70
85	22	93
83	63	73
82	75	97
36	77	97

d. none of the above

4. If you concatenate the data sets below in the order shown, what is the value of Sale in observation 2 of the new data set?

Work.Reps		Work.Close		Work.Bonus	
ID	Name	ID	Sale	ID	Bonus
1	Nay Rong	1	\$28,000	1	\$2,000
2	Kelly Windsor	2	\$30,000	2	\$4,000
3	Julio Meraz	2	\$40,000	3	\$3,000
4	Richard Krabill	3	\$15,000	4	\$2,500
		3	\$20,000		
		3	\$25,000		
		4	\$35,000		

- missing
  - \$30,000
  - \$40,000
  - You cannot concatenate these data sets.
5. What happens if you merge the following data sets by the variable SSN?

1st		2nd		
SSN	Age	SSN	Age	Date
029-46-9261	39	029-46-9261	37	02/15/95
074-53-9892	34	074-53-9892	32	05/22/97
228-88-9649	32	228-88-9649	30	03/04/96
442-21-8075	12	442-21-8075	10	11/22/95
446-93-2122	36	446-93-2122	34	07/08/96
776-84-5391	28	776-84-5391	26	12/15/96
929-75-0218	27	929-75-0218	25	04/30/97

- The values of Age in the data set 1st overwrite the values of Age in the data set 2nd.
  - The values of Age in the data set 2nd overwrite the values of Age in the data set 1st.
  - The DATA step fails because the two data sets contain same-named variables that have different values.
  - The values of Age in the data set 2nd are set to missing.
6. Suppose you merge data sets Health.Set1 and Health.Set2 below:

Health.Set1			Health.Set2		
ID	Sex	Age	ID	Height	Weight
1128	F	48	1129	61	137
1274	F	50	1387	64	142
1387	F	57	2304	61	102
2304	F	16	5438	62	168
2486	F	63	6488	64	154
4425	F	48	9012	63	157
4759	F	60	9125	64	159
5438	F	42			
6488	F	59			
9012	F	39			
9125	F	56			

Which output does the following program create?

```
data work.merged;
 merge health.set1(in=in1) health.set2(in=in2);
 by id;
 if in1 and in2;
run;
proc print data=work.merged;
run;
```

a.

Obs	ID	Sex	Age	Height	Weight
1	1387	F	57	64	142
2	2304	F	16	61	102
3	5438	F	42	62	168
4	6488	F	59	64	154
5	9012	F	39	63	157
6	9125	F	56	64	159

b.

Obs	ID	Sex	Age	Height	Weight
1	1128	F	48	.	.
2	1129		.	61	137
3	1274	F	50	.	.
4	1387	F	57	.	.
5	1387		.	64	142
6	2304	F	16	.	.
7	2304		.	61	102
8	2486	F	63	.	.
9	4425	F	48	.	.
10	4759	F	60	.	.
11	5438	F	42	.	.
12	5438		.	62	168
13	6488	F	59	.	.
14	6488		.	64	154
15	9012	F	39	.	.
16	9012		.	63	157
17	9125	F	56	.	.
18	9125		.	64	159

c.

Obs	ID	Sex	Age	Height	Weight
1	1129	F	48	61	137
2	1387	F	50	64	142
3	2304	F	57	61	102
4	5438	F	16	62	168
5	6488	F	63	64	154
6	9012	F	48	63	157
7	9125	F	60	64	159

d. none of the above

7. The data sets Ensemble.Spring and Ensemble.Sum both contain a variable named Blue. How do you prevent the values of the variable Blue from being overwritten when you merge the two data sets?

a. `data ensemble.merged;`  
`merge ensemble.spring(in=blue)`  
`ensemble.summer;`  
`by fabric;`  
`run;`



- b. 

```
data ensemble.merged;
 merge ensemble.spring(out=blue)
 ensemble.summer;
 by fabric;
run;
```
- c. 

```
data ensemble.merged;
 merge ensemble.spring(blue=navy)
 ensemble.summer;
 by fabric;
run;
```
- d. 

```
data ensemble.merged;
 merge ensemble.spring(rename=(blue=navy))
 ensemble.summer;
 by fabric;
run;
```

8. What happens if you submit the following program to merge Blood.Donors1 and Blood.Donors2, shown below?

```
data work.merged;
 merge blood.donors1 blood.donors2;
 by id;
run;
```

Blood.Donors1			Blood.Donors2		
ID	Type	Units	ID	Code	Units
2304	O	16	6488	65	27
1129	A	48	1129	63	32
1129	A	50	5438	62	39
1129	A	57	2304	61	45
2486	B	63	1387	64	67

- a. The merged data set contains some missing values because not all observations have matching observations in the other data set.
  - b. The merged data set contains eight observations.
  - c. The DATA step produces errors.
  - d. Values for Units in Blood.Donors2 overwrite values of Units in Blood.Donors1.
9. If you merge Company.Staff1 and Company.Staff2 below by ID, how many observations does the new data set contain?

Company.Staff1

ID	Name	Dept	Project
000	Miguel	A12	Document
111	Fred	B45	Survey
222	Diane	B45	Document
888	Monique	A12	Document
999	Vien	D03	Survey

Company.Staff2

ID	Name	Hours
111	Fred	35
222	Diana	40
777	Steve	0
888	Monique	37

- a. 4
  - b. 5
  - c. 6
  - d. 9
10. If you merge data sets Sales.Reps, Sales.Close, and Sales.Bonus by ID, what is the value of Bonus in the third observation in the new data set?

Work.Reps

ID	Name
1	Nay Rong
2	Kelly Windsor
3	Julio Meraz
4	Richard Krabill

Work.Close

ID	Sale
1	\$28,000
2	\$30,000
2	\$40,000
3	\$15,000
3	\$20,000
3	\$25,000
4	\$35,000

Work.Bonus

ID	Bonus
1	\$2,000
2	\$4,000
3	\$3,000
4	\$2,500

- a. \$4,000
- b. \$3,000
- c. missing
- d. You cannot tell from the information given.

## Chapter 14

# Using Functions to Manipulate Data

---

<b>The Basics of SAS Functions</b>	<b>314</b>
Definition	314
Uses of SAS Functions	314
SAS Functions Categories	314
<b>SAS Functions Syntax</b>	<b>315</b>
Arguments, Variable Lists, Arrays	315
Example: Multiple Arguments	316
Target Variables	316
<b>Converting Data with Functions</b>	<b>316</b>
A Word About Converting Data	316
Potential Problems of Omitting INPUT or PUT	317
Automatic Character-to-Numeric Conversion	317
When Automatic Conversion Occurs	317
Restriction for WHERE Expressions	318
Explicit Character-to-Numeric Conversion	319
Automatic Numeric-to-Character Conversion	321
Explicit Numeric-to-Character Conversion	321
<b>Manipulating SAS Date Values with Functions</b>	<b>323</b>
SAS Date and Time Values	323
SAS Date Functions	325
YEAR, QTR, MONTH, and DAY Functions	326
WEEKDAY Function	329
MDY Function	330
DATE and TODAY Functions	332
INTCK Function	334
INTNX Function	337
DATDIF and YRDIF Functions	338
<b>Modifying Character Values with Functions</b>	<b>340</b>
SCAN Function	340
SUBSTR Function	342
SCAN versus SUBSTR Functions	346
LEFT and RIGHT Functions	347
Concatenation Operator	347
TRIM Function	348
CATX Function	349
INDEX Function	350
Finding a String Regardless of Case	351
FIND Function	351

UPCASE Function .....	353
LOWCASE Function .....	354
PROPCASE Function .....	355
TRANWRD Function .....	356
COMPBL Function .....	357
COMPRESS Function .....	358
<b>Modifying Numeric Values with Functions .....</b>	<b>360</b>
CEIL and FLOOR Functions .....	360
INT Function .....	360
ROUND Function .....	361
<b>Nesting SAS Functions .....</b>	<b>362</b>
<b>Chapter Quiz .....</b>	<b>362</b>

---

## The Basics of SAS Functions

### Definition

SAS *functions* are pre-written routines that perform computations or system manipulations on arguments and return a value. Functions can return either numeric or character results. The value that is returned can be used in an assignment statement or elsewhere in expressions.

### Uses of SAS Functions

You can use SAS functions in DATA step programming statements, in WHERE expressions, in macro language statements, in the REPORT procedure, and in Structured Query Language (SQL). They enable you to do the following:

- calculate sample statistics
- create SAS date values
- convert U.S. ZIP codes to state postal codes
- round values
- generate random numbers
- extract a portion of a character value
- convert data from one data type to another

### SAS Functions Categories

SAS functions provide programming shortcuts. The following table shows you all of the SAS function categories. This book covers selected functions that convert data, manipulate SAS date values, and modify values of character variables.

**Table 14.1** SAS Function Categories

Functions by Category			
Arithmetic	Descriptive Statistics*	Numeric*	State and ZIP Code*
Array	Distance	Probability	Trigonometric
Bitwise Logical Operations	External Files	Quantile	Truncation*
CAS	External Routines	Random Number	Variable Control
Character*	Financial	SAS File I/O	Variable Information
Character String Matching	Hyperbolic	Search	Web Services
Combinatorial	Macro	Sort	Web Tools
Date and Time*	Mathematical*	Special*	

\* Denotes the functions that are covered in this chapter.

## SAS Functions Syntax

### Arguments, Variable Lists, Arrays

To use a SAS function, specify the function name followed by the function arguments, which are enclosed in parentheses.

Syntax, SAS function:

**function-name**(*argument-1*<,*argument-n*>);

Each of the following are *arguments*.

- variables, mean(*x,y,z*)
- constants, mean(456,502,612,498)
- expressions, mean(37\*2,192/5,mean(22,34,56))

*Note:* Even if the function does not require arguments, the function name must still be followed by parentheses (for example, *function-name()*).

When a function contains more than one argument, the arguments are usually separated by commas.

`function-name (argument-1, argument-2, argument-n)`

In some functions, variable lists and arrays can also be used as arguments, as long as the list or the array is preceded by the word *of*. For more information about arrays, see [Chapter 16, “Processing Data with Arrays,” on page 383](#).

### Example: Multiple Arguments

Here is an example of a function that contains multiple arguments. Notice that the arguments are separated by commas.

```
mean(x1,x2,x3)
```

The arguments for this function can also be written as a variable list.

```
mean(of x1-x3)
```

As an alternative, the variables can be referenced by an array.

```
mean(of newarray{*})
```

### Target Variables

A target variable is the variable to which the result of a function is assigned. For example, in the statement below, the variable AvgScore is the target variable.

```
AvgScore=mean(exam1,exam2,exam3);
```

Unless the length of the target variable has been previously defined, a default length is assigned. The default length depends on the function; the default for character functions can be as long as 200.

**TIP** Default lengths can cause character variables to use more space than necessary in your data set. So, when using SAS functions, consider the appropriate length for any character target variables. If necessary, add a LENGTH statement to specify a length for the character target variable before the statement that creates the values of that variable.

---

## Converting Data with Functions

### A Word About Converting Data

The following code automatically converts the variable PayRate from character to numeric.

```
data hrd.newtemp;
 set hrd.temp;
 Salary=payrate*hours;
run;
```

You can also use the INPUT function before performing a calculation. The INPUT function converts character data values to numeric values.

You can use the PUT function to convert numeric data values to character values.

### Potential Problems of Omitting INPUT or PUT

If you skip INPUT or PUT function when converting data, SAS detects the mismatched variables and tries an automatic character-to-numeric or numeric-to-character conversion. However, this action is not always successful. Suppose each value of PayRate begins with a dollar sign (\$). When SAS tries to automatically convert the values of PayRate to numeric values, the dollar sign blocks the process. The values cannot be converted to numeric values. Similar problems can occur with automatic numeric-to-character conversion.

Therefore, it is a recommended best practice to include INPUT and PUT functions in your programs to avoid data type mismatches and automatic conversion.

### Automatic Character-to-Numeric Conversion

By default, if you reference a character variable in a numeric context such as an arithmetic operation, SAS tries to convert the variable values to numeric. For example, in the DATA step below, the character variable PayRate appears in a numeric context. It is multiplied by the numeric variable Hours to create a new variable named Salary.

```
data hrd.newtemp;
 set hrd.temp;
 Salary=payrate*hours;
run;
```

When this step executes, SAS automatically attempts to convert the character values of PayRate to numeric values so that the calculation can occur. This conversion is completed by creating a temporary numeric value for each character value of PayRate. This temporary value is used in the calculation. The character values of PayRate are not replaced by numeric values.

Whenever data is automatically converted, a message is written to the SAS log stating that the conversion has occurred.

#### Log 14.1 SAS Log

```
9246 data hrd.temp;
9247 set hrd.temp;
9248 salary=payrate*hours;
9249 run;

NOTE: Character values have been converted to numeric values at the places given
by:
 (Line):(Column) .
 9248:8
NOTE: There were 10 observations read from the data set HRD.TEMP.
NOTE: The data set HRD.TEMP has 10 observations and 16 variables.
NOTE: DATA statement used (Total process time):
 real time 0.00 seconds
 cpu time 0.00 seconds
```

### When Automatic Conversion Occurs

Automatic character-to-numeric conversion occurs in the following circumstances:

- A character value is assigned to a previously defined numeric variable, such as the numeric variable `Rate`.

```
Rate=payrate;
```

- A character value is used in an arithmetic operation.

```
Salary=payrate*hours;
```

- A character value is compared to a numeric value, using a comparison operator.

```
if payrate>=rate;
```

- A character value is specified in a function that requires numeric arguments.

```
NewRate=sum(payrate,raise);
```

The following statements are true about automatic conversion.

- It uses the `w.` informat, where `w` is the width of the character value that is being converted.
- It produces a numeric missing value from any character value that does not conform to standard numeric notation (digits with an optional decimal point, leading sign, or scientific notation).

**Table 14.2** Automatic Conversion of Character Variables

Character Value	Automatic Conversion	Numeric Value
12.47	→	12.47
-8.96	→	-8.96
1.243E1	→	12.43
1,742.64	→	.

### Restriction for WHERE Expressions

The WHERE statement does not perform automatic conversions in comparisons. The simple program below demonstrates what happens when a WHERE expression encounters the wrong data type. The variable `Number` contains a numeric value, and the variable `Character` contains a character value, but the two WHERE statements specify the wrong data type.

```
data work.convtest;
 Number=4;
 Character='4';
run;
proc print data=work.convtest;
 where character=4;
run;
proc print data=work.convtest;
 where number='4';
run;
```



This mismatch of character and numeric variables and values prevents the program from processing the WHERE statements. Automatic conversion is not performed. Instead, the program stops, and error messages are written to the SAS log.

#### Log 14.2 SAS Log

```

9254 data work.convtest;
9255 Number=4;
9256 Character='4';
9257 run;

NOTE: The data set WORK.CONVTEST has 1 observations and 2 variables.
NOTE: DATA statement used (Total process time):
 real time 0.01 seconds
 cpu time 0.01 seconds

9258 proc print data=work.convtest;
9259 where character=4;
ERROR: WHERE clause operator requires compatible variables.
9260 run;

NOTE: The SAS System stopped processing this step because of errors.
NOTE: PROCEDURE PRINT used (Total process time):
 real time 0.00 seconds
 cpu time 0.00 seconds

9261 proc print data=work.convtest;
9262 where number='4';
ERROR: WHERE clause operator requires compatible variables.
9263 run;

NOTE: The SAS System stopped processing this step because of errors.

```

## Explicit Character-to-Numeric Conversion

### Using the INPUT Function

Use the INPUT function to convert character data values to numeric values. You can explicitly convert the character values of PayRate to numeric values by using the INPUT function.

---

Syntax, INPUT function:

**INPUT**(*source*, *informat*)

- *source* indicates the character variable, constant, or expression to be converted to a numeric value.
- a numeric *informat* must also be specified, as in this example:

```
input (payrate, 2.)
```

---

When choosing the informat, be sure to select a numeric informat that can read the form of the values.

**Table 14.3** Character Values and Associated Informats

Character Value	Informat
2115233	7.
2,115,233	COMMA9.

**Example: INPUT Function**

Here is an example of the INPUT function:

```
Test=input(saletest,comma9.);
```

The function uses the numeric informat COMMA9. to read the values of the character variable SaleTest. Then the resulting numeric values are stored in the variable Test.

You can use the INPUT function to convert the character values of PayRate to numeric values.

Because PayRate has a length of 2, the numeric informat 2. is used to read the values of the variable.

```
input(payrate,2.)
```

In the following program, the function is added to the assignment statement in the DATA step.

```
data hrd.newtemp;
 set hrd.temp;
 Salary=input(payrate,2.)*hours;
run;
```

After the DATA step is executed, the new data set, which contains the variable Salary, is created.

**Figure 14.1** SAS Data Set Hrd.Newtemp (partial data set)

	City	State	Zip	Payrate	Days	Hours	Birthdate	Salary
1	CARY	NC	27513	10	11	88	25APR1979	880
2	CARY	NC	27513	8	25	200	06OCT1975	1600
3	CHAPEL HILL	NC	27514	40	26	208	22AUG1968	8320
4	RALEIGH	NC	27612	15	10	80	10SEP1976	1200
5	DURHAM	NC	27713	12	9	72	06NOV1971	864
6	CARRBORO	NC	27510	15	7	64	15JUN1975	960
7	CHAPEL HILL	NC	27514	12	12	96	15JUN1975	1152
8	DURHAM	NC	27713	10	5	40	26NOV1971	400
9	CARY	NC	27513	10	5	30	06NOV1971	300
10	RALEIGH	NC	27612	15	5	25	24MAY1972	375

Notice that no conversion messages appear in the SAS log when the INPUT function is used.

**Log 14.3 SAS Log**

```

9272 data hrd.newtemp;
9273 set hrd.temp;
9274 Salary=input (payrate,2.)*hours;
9275 run;

```

NOTE: There were 10 observations read from the data set HRD.TEMP.

The syntax of the INPUT function is very similar to the syntax of the PUT function (which performs numeric-to-character conversions).

**INPUT**(source, informat)

**PUT**(source, format)

However, note that the INPUT function requires an informat, whereas the PUT function requires a format. To remember which function requires a format versus an informat, note that the INPUT function requires an informat.

**Automatic Numeric-to-Character Conversion**

The automatic conversion of numeric data to character data is very similar to character-to-numeric conversion. Numeric data values are converted to character values whenever they are used in a character context.

For example, the numeric values of the variable Site are converted to character values if you do the following:

- assign the numeric value to a previously defined character variable, such as the character variable SiteCode: SiteCode=site;
- use the numeric value with an operator that requires a character value, such as the concatenation operator: SiteCode=site||dept;
- specify the numeric value in a function that requires character arguments, such as the SUBSTR function: Region=substr(site,1,4);

Specifically, SAS writes the numeric value with the BEST12. format, and the resulting character value is right-aligned. This conversion occurs before the value is assigned or used with any operator or function. However, automatic numeric-to-character conversion can cause unexpected results. For example, suppose the original numeric value has fewer than 12 digits. The resulting character value has leading blanks, which might cause problems when you perform an operation or function.

Automatic numeric-to-character conversion also causes a message to be written to the SAS log indicating that the conversion has occurred.

**Explicit Numeric-to-Character Conversion**

Use the PUT function to explicitly convert numeric data values to character data values.

Suppose you want to create a new character variable named Assignment that concatenates the values of the numeric variable Site and the character variable Dept. The new variable values must contain the value of Site followed by a slash (/) and then the value of Dept (for example, 26/DP).

**Figure 14.2** SAS Data Set *Hrd.Temp* (partial data set)

	Payrate	Days	Hours	Dept	Site	salary
1	10	11	88	DP	26	880
2	8	25	200	PUR	57	1600
3	40	26	208	PER	34	8320
4	15	10	80	BK	57	1200
5	12	9	72	DP	95	864
6	15	7	64	BK	44	960
7	12	12	96	DP	59	1152
8	10	5	40	PUB	38	400
9	10	5	30	DP	44	300
10	15	5	25	DP	90	375

Here is an assignment statement that contains the concatenation operator (||) to indicate that Site should be concatenated with Dept, using a slash as a separator.

```
data hrd.newtemp;
 set hrd.temp;
 Assignment=site||'/'||dept;
run;
```

*Note:* The slash is enclosed in quotation marks. All character constants must be enclosed in quotation marks.

Submitting this DATA step causes SAS to automatically convert the numeric values of Site to character values because Site is used in a character context. The variable Site appears with the concatenation operator, which requires character values. To explicitly convert the numeric values of Site to character values, you must add the PUT function to your assignment statement.

---

Syntax, PUT function:

**PUT**(source, format)

- *source* indicates the numeric variable, constant, or expression to be converted to a character value
- a *format* matching the data type of the source must also be specified, as in this example:

```
put(site,2.)
```

---

Here are several facts about the PUT function.

- The PUT function always returns a character string.
- The PUT function returns the source written with a format.
- The format must agree with the source in type.
- Numeric formats right-align the result; character formats left-align the result.
- When you use the PUT function to create a variable that has not been previously identified, it creates a character variable whose length is equal to the format width.

When you use a numeric variable as the source, you must specify a numeric format.

To explicitly convert the numeric values of Site to character values, use the PUT function in an assignment statement, where Site is the source variable. Because Site has a length of 2, choose 2. as the numeric format. The DATA step adds the new variable from the assignment statement to the data set.

```
data hrd.newtemp;
```

```

set hrd.temp;
Assignment=put(site,2.)||'/'||dept;
run;
proc print data=hrd.newtemp;
run;

```

**Figure 14.3** HTML Output: PROC PRINT (partial output)

Obs	City	State	Zip	Payrate	Days	Hours	Dept	Site	salary	Assignment
1	CARY	NC	27513	10	11	88	DP	26	880	26/DP
2	CARY	NC	27513	8	25	200	PURH	57	1600	57/PURH
3	CHAPEL HILL	NC	27514	40	26	208	PERS	34	8320	34/PERS
4	RALEIGH	NC	27612	15	10	80	BK	57	1200	57/BK
5	DURHAM	NC	27713	12	9	72	DP	95	864	95/DP
6	CARRBORO	NC	27510	15	7	64	BK	44	960	44/BK
7	CHAPEL HILL	NC	27514	12	12	96	DP	59	1152	59/DP
8	DURHAM	NC	27713	10	5	40	PUB	38	400	38/PUB
9	CARY	NC	27513	10	5	30	DP	44	300	44/DP
10	RALEIGH	NC	27612	15	5	25	DP	90	375	90/DP

Notice that no conversion messages appear in the SAS log when you use the PUT function.

**Log 14.4** SAS Log

```

9355 data hrd.newtemp;
9356 set hrd.temp;
9357 Assignment=put(site,2.)||'/'||dept;
9358 run;

```

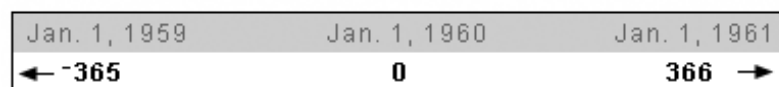
NOTE: There were 10 observations read from the data set HRD.TEMP.  
NOTE: The data set HRD.NEWTEMP has 10 observations and 17 variables.

## Manipulating SAS Date Values with Functions

### SAS Date and Time Values

SAS includes a variety of functions that enable you to work with SAS date values. SAS stores a date value as the number of days from January 1, 1960, to a given date.

**Figure 14.4** SAS Date Values



A SAS time value is stored as the number of seconds since midnight.

**Figure 14.5** SAS Time Values

(12:00 am) midnight	12:15 pm	17:00 (or 5:00 pm)
<b>0</b>	<b>44100</b>	<b>61200 →</b>

A SAS datetime value is stored as the number of seconds between midnight on January 1, 1960, and a given date and time.

**Figure 14.6** SAS Datetime Values

July 4, 1776 11:30:23	Jan. 1, 1960 midnight	July 4, 1994 16:10:45
<b>← -5790400177</b>	<b>0</b>	<b>1088957445 →</b>

SAS stores date values as numbers so that you can easily sort the values or perform arithmetic computations. You can use SAS date values as you use any other numeric values.

```
data test;
 set hrd.temp;
 TotDay=enddate-startdate;
run;
```

When you execute the program, TotDay has a value of 54 based on the StartDate and EndDate values in the Hrd.Temp data set.

To display SAS date values in a variety of forms, associate a SAS format with the values. The format affects only the way the dates are displayed, not the date values in the data set. For example, the FORMAT statement below associates the DATE9. format with the variables StartDate and EndDate. A portion of the output created by this PROC PRINT step appears below.

```
proc print data=hrd.temp;
 format startdate enddate date9.;
run;
```

**Figure 14.7** HTML Output: PROC PRINT (partial output)

Obs	City	State	Zip	Phone	Startdate	Enddate	Payrate	Days	Hours
1	CARY	NC	27513	6224549	19NOV1999	12JAN2000	10	11	88
2	CARY	NC	27513	6223251	07OCT1999	17NOV1999	8	25	200
3	CHAPEL HILL	NC	27514	9974749	22NOV1999	30DEC1999	40	26	208
4	RALEIGH	NC	27612	6970540	29SEP1999	10OCT1999	15	10	80
5	DURHAM	NC	27713	3633618	02NOV1999	13NOV1999	12	9	72
6	CARRBORO	NC	27510	9976732	16NOV1999	04JAN2000	15	7	64
7	CHAPEL HILL	NC	27514	9972070	02AUG1998	17AUG1998	12	12	96
8	DURHAM	NC	27713	3633020	06OCT1998	10OCT1998	10	5	40
9	CARY	NC	27513	6565303	07OCT1999	20OCT1998	10	5	30
10	RALEIGH	NC	27612	4341557	29JUN1999	30OCT1998	15	5	25

SAS date values are valid for dates that are based on the Gregorian calendar from 1582 C.E. through 20,000 C.E.

**Figure 14.8** SAS Date Values

## SAS Date Functions

SAS stores date, time, and datetime values as numeric values. You can use several functions to create these values.

**Table 14.4** Typical Use of SAS Date Functions

Function	Example Code	Result
MDY	<code>date=mdy(mon, day, yr);</code>	SAS date
TODAYDATE	<code>now=today();</code> <code>now=date();</code>	today's date as a SAS date
TIME	<code>curtime=time();</code>	current time as a SAS time

Use other functions to extract months, quarters, days, and years from SAS date values.

**Table 14.5** Selected Functions to Use with SAS Date Values

Function	Example Code	Result
DAY	<code>day=day(date);</code>	day of month (1-31)
QTR	<code>quarter=qtr(date);</code>	quarter (1-4)
WEEKDAY	<code>wkday=weekday(date);</code>	day of week (1-7)
MONTH	<code>month=month(date);</code>	month (1-12)
YEAR	<code>yr=year(date);</code>	year (4 digits)
INTCK	<code>x=intck('day',d1,d2);</code>	days from D1 to D2
	<code>x=intck('week',d1,d2);</code>	weeks from D1 to D2
	<code>x=intck('month',d1,d2);</code>	months from D1 to D2
	<code>x=intck('qtr',d1,d2);</code>	quarters from D1 to D2
	<code>x=intck('year',d1,d2);</code>	years from D1 to D2
INTNX	<code>x=intnx('interval', start-from,increment);</code>	date, time, or datetime value
DATDIF	<code>x=datdif(date1,date2,'ACT/ACT');</code>	days between date1 and date2
YRDIF	<code>x=yrdif(date1,date2,'ACT/ACT');</code>	years between date1 and date2

## YEAR, QTR, MONTH, and DAY Functions

### Overview of YEAR, QTR, MONTH, and DAY Functions

Every SAS date value can be queried for the values of its year, quarter, month, and day. You extract these values by using the functions YEAR, QTR, MONTH, and DAY. They each work the same way.

Syntax, YEAR, QTR, MONTH, and DAY functions:

**YEAR**(date)

**QTR**(date)

**MONTH**(date)

**DAY**(date)

*date* is a SAS date value that is specified either as a variable or as a SAS date constant.

The YEAR function returns a four-digit numeric value that represents the year (for example, 2002). The QTR function returns a value of 1, 2, 3, or 4 from a SAS date value to indicate the quarter of the year in which a date value falls. The MONTH function returns a numeric value that ranges from 1 to 12, representing the month of the year. The value 1 represents January, 2 represents February, and so on. The DAY function returns a numeric value from 1 to 31, representing the day of the month.



**Table 14.6** Selected Date Functions and Their Uses

Function	Description	Form	Sample Value
YEAR	Extracts the year value from a SAS date value.	YEAR( <i>date</i> )	2002
QTR	Extracts the quarter value from a SAS date value	QTR( <i>date</i> )	1
MONTH	Extracts the month value from a SAS date value.	MONTH( <i>date</i> )	12
DAY	Extracts the day value from a SAS date value	DAY( <i>date</i> )	5

**Example: Finding the Year and Month**

Suppose you want to create a subset of the data set Hrd.Temp that contains information about all temporary employees who were hired in November 1999. The data set Hrd.Temp contains the beginning and ending dates for staff employment, but there are no month or year variables in the data set. To determine the year in which employees were hired, you can apply the YEAR function to the variable that contains the employee start date, StartDate. Here is a way to write the YEAR function:

```
year(startdate)
```

Likewise, to determine the month in which employees were hired, you apply the MONTH function to StartDate.

```
month(startdate)
```

To create the new data set, you include these functions in a subsetting IF statement within a DATA step. The subsetting IF statement specifies that the new data set include only observations in which the YEAR function extracts a value of 1999 and the MONTH function extracts a value of 11 (for November).

```
data hrd.nov99;
 set hrd.temp;
 where year(startdate)=1999 and month(startdate)=11;
run;
```

When you add a PROC PRINT step to the program, you can view the new data set. Notice that the PROC PRINT step includes a FORMAT statement to display the variables StartDate and EndDate with the DATE9. format.

```
data hrd.nov99;
 set hrd.temp;
 where year(startdate)=1999 and month(startdate)=11;
proc print data=hrd.nov99;
 format startdate enddate date9.;
run;
```

The new data set contains information about only those employees who were hired in November 1999.

**Figure 14.9** HTML Output: PROC PRINT (partial output)

Obs	City	State	Zip	Phone	Startdate	Enddate	Payrate	Days	Hours
1	CARY	NC	27513	6224549	19NOV1999	12JAN2000	10	11	88
2	CHAPEL HILL	NC	27514	9974749	22NOV1999	30DEC1999	40	26	208
3	DURHAM	NC	27713	3633618	02NOV1999	13NOV1999	12	9	72
4	CARRBORO	NC	27510	9976732	16NOV1999	04JAN2000	15	7	64

**Example: Finding the Year**

Suppose you want to create a subset of the data set Hrd.Temp that contains information about all temporary employees who were hired during a specific year, such as 1998. Hrd.Temp contains the dates on which employees began work with the company and their ending dates, but there is no year variable.

To determine the year in which employees were hired, you can apply the YEAR function to the variable that contains the employee start date, StartDate. You write the YEAR function as follows:

```
year(startdate)
```

To create the new data set, you include this function in a subsetting IF statement within a DATA step. This subsetting IF statement specifies that only observations in which the YEAR function extracts a value of 1998 are placed in the new data set.

```
data hrd.temp98;
 set hrd.temp;
 where year(startdate)=1998;
run;
```

When you add a PROC PRINT step to the program, you can view the new data set. Notice that the PROC PRINT step includes a FORMAT statement to display the variables StartDate and EndDate with the DATE9. format.

```
data hrd.temp98;
 set hrd.temp;
 where year(startdate)=1998;
run;
proc print data=hrd.temp98;
 format startdate enddate date9.;
run;
```

The new data set contains information for only those employees who were hired in 1998.

**Figure 14.10** HTML Output: PROC PRINT (partial output)

Obs	City	State	Zip	Phone	Startdate	Enddate	Payrate	Days	Hours
1	CHAPEL HILL	NC	27514	9972070	02AUG1998	17AUG1998	12	12	96
2	DURHAM	NC	27713	3633020	06OCT1998	10OCT1998	10	5	40

## WEEKDAY Function

### Overview of the WEEKDAY Function

The WEEKDAY function enables you to extract the day of the week from a SAS date value.

---

Syntax, WEEKDAY function:

**WEEKDAY**(date)

*date* is a SAS date value that is specified either as a variable or as a SAS date constant.

---

The WEEKDAY function returns a numeric value from 1 to 7. The values represent the days of the week.

**Table 14.7** Values for the WEEKDAY Function

Value	Equals	Day of the Week
1	=	Sunday
2	=	Monday
3	=	Tuesday
4	=	Wednesday
5	=	Thursday
6	=	Friday
7	=	Saturday

### Example: WEEKDAY Function

For example, suppose the data set Radio.Sch contains a broadcast schedule. The variable AirDate contains SAS date values. To create a data set that contains only weekend broadcasts, you use the WEEKDAY function in a subsetting IF statement. You include only observations in which the value of AirDate corresponds to a Saturday or Sunday.

```
data work.schwkend;
 set sasuser.sch;
 where weekday(airdate) in(1,7);
run;
proc print data=work.schwkend;
run;
```

**Output 14.1** HTML Output: Weekday Function

Obs	Program	Producer	AirDate
1	River to River	NPR	04/01/00
2	World Cafe	WXPB	04/08/00
3	Classical Music	NPR	04/08/00
4	Symphony Live	NPR	04/01/00
5	Symphony Live	NPR	04/16/00
6	World Cafe	WXPB	04/08/00

*Note:* In the example above, the statement `if weekday(airdate) in (1,7);` is the same as `if weekday(airdate)=7 or weekday(airdate)=1;`

**MDY Function****Overview of the MDY Function**

The MDY function returns a SAS date value from month, day, and year values.

---

Syntax, MDY function:

**MDY** (month, day, year)

- *month* specifies a numeric constant, variable, or expression that represents an integer from 1 through 12.
  - *day* specifies a numeric constant, variable, or expression that represents an integer from 1 through 31.
  - *year* specifies a numeric constant, variable, or expression with a value of a two-digit or four-digit integer that represents that year. The YEARCUTOFF=system option defines the year value from two-digit dates.
- 

**Details**

In the data set `Hrd.Temp`, the values for month, day, and year are stored in the numeric variables `Month`, `Day`, and `Year`. It is possible to write the following MDY function to create the SAS date values:

```
mdy(month, day, year)
```

To create a new variable to contain the SAS date values, place this function in an assignment statement.

```
data hrd.newtemp(drop=month day year);
 set hrd.temp;
 Date=mdy(month, day, year);
run;
```

To display SAS date values in a more readable form, you can associate a SAS format with the values. For example, the `FORMAT` statement below associates the `DATE9.` format with the variable `Date`.

```
proc print data=hrd.newtemp;
```

```
format date date9.;
run;
```

The MDY function can also add the same SAS date to every observation. This might be useful if you want to compare a fixed beginning date with different end dates. Just use numbers instead of data set variables when providing values to the MDY function.

```
data hrd.newtemp;
 set hrd.temp;
 DateCons=mdy(6,17,2002);
proc print data=hrd.newtemp;
 format datecons date9.;
run;
```

**Figure 14.11** HTML Output: PROC PRINT (partial output)

Obs	City	State	Zip	Phone	Payrate	Days	Hours	DateCons
1	CARY	NC	27513	6224549	10	11	88	17JUN2002
2	CARY	NC	27513	6223251	8	25	200	17JUN2002
3	CHAPEL HILL	NC	27514	9974749	40	26	208	17JUN2002
4	RALEIGH	NC	27612	6970540	15	10	80	17JUN2002
5	DURHAM	NC	27713	3633618	12	9	72	17JUN2002
6	CARRBORO	NC	27510	9976732	15	7	64	17JUN2002
7	CHAPEL HILL	NC	27514	9972070	12	12	96	17JUN2002
8	DURHAM	NC	27713	3633020	10	5	40	17JUN2002
9	CARY	NC	27513	6565303	10	5	30	17JUN2002
10	RALEIGH	NC	27612	4341557	15	5	25	17JUN2002

*Note:* The MDY function accepts two-digit values for the year, but SAS interprets two-digit values according to the 100-year span that is set by the YEARCUTOFF= system option. The default value of YEARCUTOFF= is 1926. For details, see [Chapter 19, “SAS Date and Time Values,”](#) on page 447.

The use of four-digit year values in the MDY function is recommended:

- **MDY(5,10,20)** = May 10, 1920
- **MDY(5,10,2020)** = May 10, 2020

To display the years clearly, format SAS dates with the DATE9. format. This forces the year to appear with four digits, as shown above in the Date and DateCons variables of the Hrd.Newtemp output.

### **Example: Finding the Date**

The data set Dec.Review contains a variable named Day. This variable contains the day of the month for each employee's performance appraisal. The appraisals were all completed in December of 2010.

The following DATA step uses the MDY function to create a new variable named ReviewDate. This variable contains the SAS date value for the date of each performance appraisal.

```
data sasuser.review2010;
```

```

set sasuser.review;
ReviewDate=mdy(12,day,2010);
run;
proc print data=sasuser.review2010 noobs;
run;

```

**Figure 14.12** SAS Data Set Dec.Review2010

Site	Day	Rate	Name	ReviewDate
Westin	12	A2	Mitchell, K	18608
Stockton	4	A5	Worton, M	18600
Center City	17	B1	Smith, A	18613

*Note:* If you specify an invalid date in the MDY function, SAS assigns a missing value to the target variable.

```

data dec.review2010;
set dec.review;
ReviewDate=mdy(15,day,2010);
run;

```

**Figure 14.13** SAS Data Set Dec.Review2010

Site	Day	Rate	Name	ReviewDate
Westin	12	A2	Mitchell, K	.
Stockton	4	A5	Worton, M	.
Center City	17	B1	Smith, A	.

## DATE and TODAY Functions

### Overview of the DATE Function

The DATE function returns the current date as a numeric SAS date value.

*Note:* If the value of the TIMEZONE= system option is set to a time zone name or time zone ID, the date and time values that are returned for this function are determined by the time zone.

---

Syntax, DATE function:

**DATE ()**

The DATE function does not require any arguments, but it must be followed by parentheses.

---

The DATE function produces the current date in the form of a SAS date value, which is the number of days since January 1, 1960.

### Overview of the TODAY Function

The TODAY function returns the current date as a numeric SAS date value.

*Note:* If the value of the TIMEZONE= system option is set to a time zone name or time zone ID, the date and time values that are returned for this function are determined by the time zone.

---

Syntax, TODAY function:

**TODAY ()**

The TODAY function does not require any arguments, but it must be followed by parentheses.

---

The TODAY function produces the current date in the form of a SAS date value, which is the number of days since January 1, 1960.

### Example: The DATE and TODAY Functions

The DATE and TODAY functions have the same form and can be used interchangeably. To add a new variable, which contains the current date, to the data set Hrd.Temp. To create this variable, write an assignment statement such as the following:

```
EditDate=date();
```

After this statement is added to a DATA step and the step is submitted, the data set that contains EditDate is created.

```
data hrd.newtemp;
 set hrd.temp;
 EditDate=date();
run;
```

*Note:* For this example, the SAS date values shown below were created by submitting this program on June 29, 2017.

To display these SAS date values in a different form, you can associate a SAS format with the values. For example, the FORMAT statement below associates the DATE9. format with the variable EditDate. A portion of the output that is created by this PROC PRINT step appears below.

```
proc print data=hrd.newtemp;
 format editdate date9.;
run;
```

**Figure 14.14** SAS Output: SAS Data Set Hrd.Newtemp (partial output)

Obs	City	State	Phone	Startdate	Enddate	EditDate
1	CARY	NC	6224549	19NOV1999	12JAN2000	29JUN2017
2	CARY	NC	6223251	07OCT1999	17NOV1999	29JUN2017
3	CHAPEL HILL	NC	9974749	22NOV1999	30DEC1999	29JUN2017
4	RALEIGH	NC	6970540	29SEP1999	10OCT1999	29JUN2017
5	DURHAM	NC	3633618	02NOV1999	13NOV1999	29JUN2017
6	CARRBORO	NC	9976732	16NOV1999	04JAN2000	29JUN2017
7	CHAPEL HILL	NC	9972070	02AUG1998	17AUG1998	29JUN2017
8	DURHAM	NC	3633020	06OCT1998	10OCT1998	29JUN2017
9	CARY	NC	6565303	07OCT1999	20OCT1998	29JUN2017
10	RALEIGH	NC	4341557	29JUN1999	30OCT1998	29JUN2017

## INTCK Function

### Overview of the INTCK Function

The INTCK function returns the number of interval boundaries of a given kind that lie between two dates, times, or datetime values. You can use it to count the passage of days, weeks, months, and so on.



Syntax, INTCK function:

**INTCK** (*interval* <multiple> <.shift-index>, *start-date*, *end-date*, <'method'>)

**INTCK**(*custom-interval*, *start-date*, *end-date*, <'method'>)

- *interval* specifies a character constant, a variable, or an expression that contains an interval name. Interval can appear in uppercase or lowercase. The type of interval (date, datetime, or time) must match the type of value in start-date. Multipliers and shift indexes can be used with the basic interval names to construct more complex interval specifications.
  - *interval*<multiple>.shift-index> the parts of the interval name are listed below:
    - *interval* specifies the name of the basic interval type. For example, YEAR specifies yearly intervals.
    - *multiple* specifies an optional multiplier that sets the interval equal to a multiple of the period of the basic interval type. For example, the interval YEAR2 consists of two-year, or biennial, periods.
    - *custom-interval* specifies a user-defined interval that is defined by a SAS data set. Each observation contains two variables, begin and end.
    - *shift-index* specifies an optional shift index that shifts the interval to start at a specified subperiod starting point. For example, YEAR.3 specifies yearly periods shifted to start on the first of March of each calendar year and to end in February of the following year.
- Note:* The shift index cannot be greater than the number of subperiods in the entire interval. For example, you could use YEAR2.24, but YEAR2.25 would be an error because there is no 25th month in a two-year interval. If the default shift period is the same as the interval type, then only multiperiod intervals can be shifted with the optional shift index. For example, MONTH type intervals shift by MONTH subperiods by default. Thus, monthly intervals cannot be shifted with the shift index. However, bimonthly intervals can be shifted with the shift index, because there are two MONTH intervals in each MONTH2 interval. For example, the interval name MONTH2.2 specifies bimonthly periods starting on the first day of even-numbered months.
- *start-date* specifies a SAS expression that represents the starting SAS date, time, or datetime value.
  - *end-date* specifies a SAS expression that represents the ending SAS date, time, or datetime value.
  - <'method'> specifies that intervals are counted using either a discrete or a continuous method. You must enclose method in quotation marks. Method can be one of these values:
    - CONTINUOUS specifies that continuous time is measured. The interval is shifted based on the starting date.
    - DISCRETE specifies that discrete time is measured. The discrete method counts interval boundaries (for example, end of month).

## Details

The INTCK function counts intervals from fixed interval beginnings, not in multiples of an interval unit from the *from* value. Partial intervals are not counted. Intervals can be one of the following values:

- DAY
- MONTH
- QTR
- SEMIMONTH
- SEMIYEAR

- TENDAY
- WEEK
- WEEKDAY
- YEAR

For example, WEEK intervals are counted by Sundays rather than seven-day multiples from the *from* argument. MONTH intervals are counted by day 1 of each month, and YEAR intervals are counted from 01JAN, not in 365-day multiples.

Consider the results in the following table. The values that are assigned to the variables Weeks, Months, and Years are based on consecutive days.

**Table 14.8** Examples of SAS Statements and Their Values

Example Code	Value
Weeks=intck('week','31dec2000'd,'01jan2001'd);	0
Months=intck('month','31dec2000'd,'01jan2001'd);	1
Years=intck('year','31dec2000'd,'01jan2001'd);	1

Because December 31, 2000, is a Sunday, no WEEK interval is crossed between that day and January 1, 2001. However, both MONTH and YEAR intervals are crossed.

### Examples: INTCK Function

The following statement creates the variable Years and assigns it a value of 2. The INTCK function determines that two years have elapsed between June 15, 1999, and June 15, 2001.

```
Years=intck('year','15jun1999'd,'15jun2001'd);
```

*Note:* As shown here, the *from* and *to* dates are often specified as date constants.

Likewise, the following statement assigns the value 24 to the variable Months.

```
Months=intck('month','15jun1999'd,'15jun2001'd);
```

However, the following statement assigns 0 to the variable Years, even though 364 days have elapsed. In this case, the YEAR boundary (01JAN) is not crossed.

```
Years=intck('year','01jan2002'd,'31dec2002'd);
```

### Example: The INTCK Function and Periodic Events

A common use of the INTCK function is to identify periodic events such as due dates and anniversaries.

The following program identifies mechanics whose 20th year of employment occurs in the current month. It uses the INTCK function to compare the value of the variable Hired to the date on which the program is run.

```
data work.anniv30;
 set flights.mechanics(keep=id lastname firstname hired);
 Years=intck('year',hired,today());
 if years=30 and month(hired)=month(today());
run;
proc print data=work.anniv30;
```

```

title '30-Year Anniversaries This Month';
run;

```

The following output is created when the program is run in June 2017.

**Figure 14.15** 30-Year Anniversaries This Month

30-Year Anniversaries This Month					
Obs	ID	LastName	FirstName	Hired	Years
1	1653	ALEXANDER	SUSAN	05JUN87	30
2	1400	APPLE	TROY	07JUN87	30
3	1200	OVERMAN	MICHELLE	15JUN87	30

## INTNX Function

### Overview of the INTNX Function

The INTNX function is similar to the INTCK function. The INTNX function applies multiples of a given interval to a date, time, or datetime value and returns the resulting value. You can use the INTNX function to identify past or future days, weeks, months, and so on.

---

Syntax, INTNX function:

**INTNX**(*interval*,*start-from*,*increment*<,*alignment*>)

- *interval* specifies a character constant or variable.
- *start-from* specifies a starting SAS date, time, or datetime value.
- *increment* specifies a negative or positive integer that represents time intervals toward the past or future.
- *alignment* (optional) forces the alignment of the returned date to the beginning, middle, or end of the interval.

*Note:* The type of interval (date, time, or datetime) must match the type of value in *start-from* and *increment*.

---

### Details

When you specify date intervals, the value of the character constant or variable that is used in *interval* can be one of the following:

- DATETIME
- DAY
- QTR
- MONTH
- SEMIMONTH
- SEMIYEAR
- TENDAY

- TIME
- WEEK
- WEEKDAY
- YEAR

### **Example: INTNX Function**

For example, the following statement creates the variable `TargetYear` and assigns it a SAS date value of **13515**, which corresponds to January 1, 1997.

```
TargetYear=intnx('year','05feb94'd,3);
```

Likewise, the following statement assigns the value for the date July 1, 2001, to the variable `TargetMonth`.

```
TargetMonth=intnx('semiyear','01jan2001'd,1);
```

SAS date values are based on the number of days since January 1, 1960. Yet the INTNX function can use intervals of weeks, months, years, and so on.

The purpose of the optional alignment argument is to specify whether the returned value should be at the beginning, middle, or end of the interval. When specifying date alignment in the INTNX function, use the following values or their corresponding aliases:

- BEGINNING Alias: B
- MIDDLE Alias: M
- END Alias: E
- SAME Alias: SAMEDAY or S

The best way to understand the alignment argument is to see its effect on identical statements. The following table shows the results of three INTNX statements that differ only in the value of alignment.

**Table 14.9** Alignment Values for the INTNX Function

Example Code	Date Value
MonthX=intnx('month','01jan1995'd,5,'b');	<b>12935</b> (June 1, 1995)
MonthX=intnx('month','01jan1995'd,5,'m');	<b>12949</b> (June 15, 1995)
MonthX=intnx('month','01jan1995'd,5,'e');	<b>12964</b> (June 30, 1995)

These INTNX statements count five months from January, but the returned value depends on whether alignment specifies the beginning, middle, or end day of the resulting month. If alignment is not specified, the beginning day is returned by default.

### **DATDIF and YRDIF Functions**

The DATDIF and YRDIF functions calculate the difference in days and years between two SAS dates, respectively. Both functions accept start dates and end dates that are specified as SAS date values. Also, both functions use a basis argument that describes how SAS calculates the date difference.

Syntax, DATDIF, and YRDIF functions:

**DATDIF**(start\_date,end\_date,basis))

**YRDIF**(start\_date,end\_date,basis))

- *start\_date* specifies the starting date as a SAS date value.
- *end\_date* specifies the ending date as a SAS date value.
- *basis* specifies a character constant or variable that describes how SAS calculates the date difference.

There are two character strings that are valid for basis in the DATDIF function, and four character strings that are valid for basis in the YRDIF function. These character strings and their meanings are listed in the table below.

**Table 14.10** Character Strings in the DATDIF Function

Character String	Meaning	Valid in DATDIF	Valid in YRDIF
'30/360'	specifies a 30-day month and a 360-day year	yes	yes
'ACT/ACT'	uses the actual number of days or years between dates	yes	yes
'ACT/360'	uses the actual number of days between dates in calculating the number of years (calculated by the number of days divided by 360)	no	yes
'ACT/365'	uses the actual number of days between dates in calculating the number of years (calculated by the number of days divided by 365)	no	yes

The best way to understand the different options for the basis argument is to see the different effects that they have on the value that the function returns. The table below lists four YRDIF functions that use the same start date and end date. Each function uses one of the possible values for basis, and each one returns a different value.

**Table 14.11** Examples of the YRDIF Function

Example Code	Returned Value
yrdif('16oct1998'd,'16feb2003'd,'30/360')	4.333333333
yrdif('16oct1998'd,'16feb2003'd,'ACT/ACT')	4.3369863014
yrdif('16oct1998'd,'16feb2003'd,'ACT/360')	4.4
yrdif('16oct1998'd,'16feb2003'd,'ACT/365')	4.3397260274

---

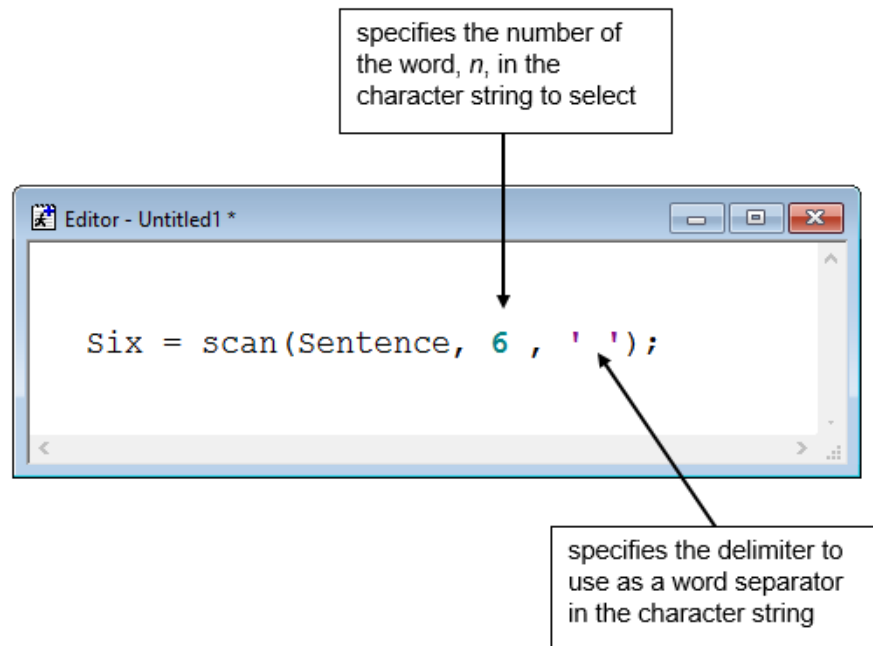
## Modifying Character Values with Functions

### SCAN Function

#### Overview of the SCAN Function

The SCAN function returns the  $n$ th word from a character string. The SCAN function enables you to separate a character value into words and to return a specified word.

**Figure 14.16** SCAN Function



*Note:* In SAS 9.4 or later, in a DATA step, the SCAN function returns a value to a variable that has not yet been given a length, that variable is given the length of the first argument. In SAS 9.3 or earlier, by default, the variable is assigned a length of 200.

---

Syntax, SCAN function:

**SCAN**(*argument*,*n*<,<*delimiters*>>)

- *argument* specifies the character variable or expression to scan.
  - *n* specifies which word to return.
  - *delimiters* are special characters that must be enclosed in single quotation marks (' '). If you do not specify *delimiters*, default delimiters are used.
- 

#### Details

- Leading delimiters before the first word in the character string do not affect the SCAN function.

- If there are two or more contiguous delimiters, the SCAN function treats them as one.
- If  $n$  is greater than the number of words in the character string, the SCAN function returns a blank value.
- If  $n$  is negative, the SCAN function selects the word in the character string starting from the end of the string.

### **Example: Create New Name Variables**

Use the SCAN function to create your new name variables for Hrd.Temp. First, examine the values of the existing Name variable to determine which characters separate the names in the values.

```
LastName=scan(name,1);
```

Notice that blanks and commas appear between the names and that the employee's last name appears first, then the first name, and then the middle name.

To create the LastName variable to store the employee's last name, you write an assignment statement that contains the following SCAN function:

```
LastName=scan(name,1,' ','');
```

Note that a blank and a comma are specified as delimiters. You can also write the function without listing delimiters, because the blank and comma are default delimiters.

```
LastName=scan(name,1);
```

Here is the complete DATA step that is needed to create LastName, FirstName, and MiddleName. Notice that the original Name variable is dropped from the new data set.

```
data hrd.newtemp(drop=name);
 set hrd.temp;
 LastName=scan(name,1);
 FirstName=scan(name,2);
 MiddleName=scan(name,3);
run;
```

### **Specifying Delimiters**

The SCAN function uses delimiters, which are characters that are specified as word separators, to separate a character string into words. For example, if you are working with the character string below and you specify the comma as a delimiter, the SCAN function separates the string into three words.

```
LOW,MODERATE,HIGH
↑ ↑ ↑
1 2 3
```

Then the function returns the word that you specify. In this example, if you specify the third word, the SCAN function returns the word HIGH.

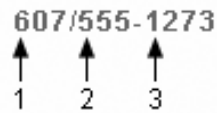
Here is another example that uses the comma as a delimiter, and specifies that the third word be returned.

```
209 RADCLIFFE ROAD, CENTER CITY, NY, 92716
```

In this example, if you specify the third word, the word returned by the SCAN function is NY ( NY contains a leading blank).

### Specifying Multiple Delimiters

When using the SCAN function, you can specify as many delimiters as needed to correctly separate the character expression. When you specify multiple delimiters, SAS uses any of the delimiters, singly or in any combination, as word separators. For example, if you specify both the slash and the hyphen as delimiters, the SCAN function separates the following text string into three words:

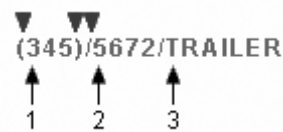


607/555-1273

↑    ↑    ↑

1    2    3

The SCAN function treats two or more contiguous delimiters, such as the parenthesis and slash below, as one delimiter. Also, leading delimiters have no effect.



▼    ▼

(345)/5672/TRAILER

↑    ↑    ↑

1    2    3

### Default Delimiters

If you do not specify delimiters when using the SCAN function, default delimiters are used. Here are the default delimiters:

blank . < ( + | & ! \$ \* ) ; ^ - / , %

### Specifying Variable Length

If a variable is not assigned a length before it is specified in the SCAN function, the variable is given the length of the first argument. This length could be too small or too large for the remaining variables.

You can add a LENGTH statement to the DATA step, and specify an appropriate length for all three variables. Because SAS sets the length of a new character variable the first time it is encountered in the DATA step, the LENGTH statement is placed before the assignment statements that contain the SCAN function.

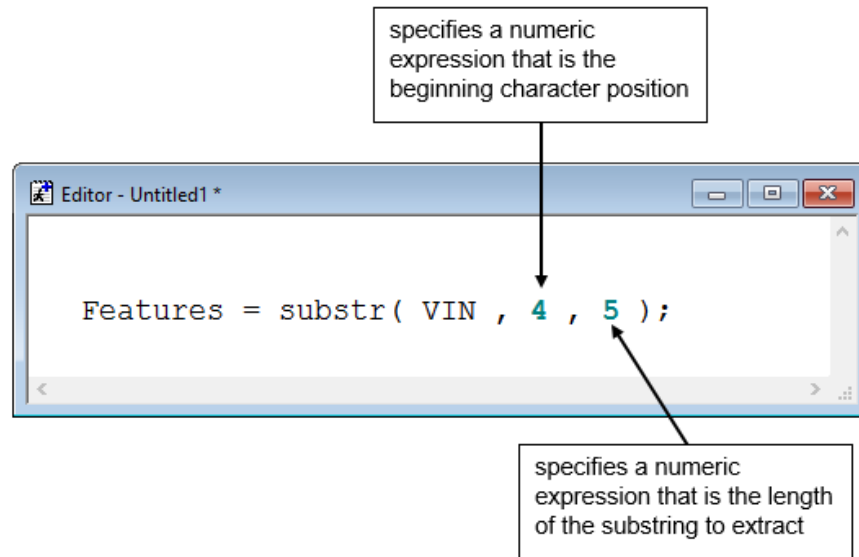
```
data hrd.newtemp(drop=name);
 set hrd.temp;
 length LastName FirstName MiddleName $ 10;
 lastname=scan(name,1);
 firstname=scan(name,2);
 middlename=scan(name,3);
run;
```

## SUBSTR Function

### Overview of the SUBSTR Function

The SUBSTR function extracts a substring from an argument, starting at a specific position in the string.



**Figure 14.17** SUBSTR Function

The SUBSTR function can be used on either the right or left of the equal sign to replace character value constants.

---

Syntax, SUBSTR function:

**SUBSTR**(*argument*, *position* <,n>)

- *argument* specifies the character variable or expression to scan.
  - *position* is the character position to start from.
  - *n* specifies the number of characters to extract. If *n* is omitted, all remaining characters are included in the substring.
- 

### **Example: SUBSTR Function**

This example begins with the task of extracting a portion of a value. In the data set Hrd.Newtemp, the names of temporary employees are stored in three name variables: LastName, FirstName, and MiddleName.

Agency	ID	LastName	FirstName	MiddleName
Administrative Support, Inc.	F274	CICHOCK	ELIZABETH	MARIE
Administrative Support, Inc.	F101	BENINCASA	HANNAH	LEE
OD Consulting, Inc.	F054	SHERE	BRIAN	THOMAS
New Time Temps Agency	F077	HODNOFF	RICHARD	LEE

However, suppose you want to modify the data set to store only the middle initial instead of the full middle name. To do so, you must extract the first letter of the middle name values and assign these values to the new variable MiddleInitial.

Agency	ID	LastName	FirstName	MiddleInitial
Administrative Support, Inc.	F274	CICHOCK	ELIZABETH	M
Administrative Support, Inc.	F101	BENINCASA	HANNAH	L
OD Consulting, Inc.	F054	SHERE	BRIAN	T
New Time Temps Agency	F077	HODNOFF	RICHARD	L

Using the SUBSTR function, you can extract the first letter of the MiddleName value to create the new variable MiddleInitial.

Agency	ID	LastName	FirstName	MiddleName
Administrative Support, Inc.	F274	CICHOCK	ELIZABETH	MARIE
Administrative Support, Inc.	F101	BENINCASA	HANNAH	LEE
OD Consulting, Inc.	F054	SHERE	BRIAN	THOMAS
New Time Temps Agency	F077	HODNOFF	RICHARD	LEE

You write the SUBSTR function as the following:

```
substr(middlename,1,1)
```

This function extracts a character string from the value of MiddleName. The string to be extracted begins in position 1 and contains one character. This function is placed in an assignment statement in the DATA step.

```
data work.newtemp(drop=middlename);
 set hrd.newtemp;
 length MiddleInitial $ 1;
 MiddleInitial=substr(middlename,1,1);
run;
```

The new MiddleInitial variable is given the same length as MiddleName. The MiddleName variable is then dropped from the new data set.

Agency	ID	LastName	FirstName	MiddleInitial
Administrative Support, Inc.	F274	CICHOCK	ELIZABETH	M
Administrative Support, Inc.	F101	BENINCASA	HANNAH	L
OD Consulting, Inc.	F054	SHERE	BRIAN	T
New Time Temps Agency	F077	HODNOFF	RICHARD	L

You can use the SUBSTR function to extract a substring from any character value if you know the position of the value.

### Replacing Text Using SUBSTR

There is a second use for the SUBSTR function. This function can also be used to replace the contents of a character variable. For example, suppose the local phone exchange 622 was replaced by the exchange 433. You need to update the character variable Phone in Hrd.Temp to reflect this change.

Address	City	State	Zip	Phone
65 ELM DR	CARY	NC	27513	6224549
11 SUN DR	CARY	NC	27513	6223251

You can use the SUBSTR function to complete this modification. The syntax of the SUBSTR function, when used to replace a variable's values, is identical to the syntax for extracting a substring.

```
SUBSTR(argument, position, n)
```

However, in this case, note the following:

- The first argument specifies the character variable whose values are to be modified.
- The second argument specifies the position at which the replacement is to begin.
- The third argument specifies the number of characters to replace. If *n* is omitted, all remaining characters are replaced.

### Positioning the SUBSTR Function

SAS uses the SUBSTR function to extract a substring or to modify a variable's values, depending on the position of the function in the assignment statement.

When the function is on the right side of an assignment statement, the function returns the requested string.

```
MiddleInitial=substr(middlename,1,1);
```

But if you place the SUBSTR function on the left side of an assignment statement, the function is used to modify variable values.

```
substr(region,1,3)='NNW';
```

When the SUBSTR function modifies variable values, the right side of the assignment statement must specify the value to place into the variable. For example, to replace the fourth and fifth characters of a variable named Test with the value 92, you write the following assignment statement:

```
substr(test,4,2)='92';
```

Test		Test
S7381K2	→	S7392K2
S7381K7	→	S7392K7

It is possible to use the SUBSTR function to replace the 622 exchange in the variable Phone. First, write this assignment statement:

```
data hrd.temp2;
 set hrd.temp;
 substr(phone,1,3)='433';
run;
```

This statement specifies that the new exchange 433 should be placed in the variable Phone, starting at character position 1 and replacing three characters.

However, executing this DATA step places the value 433 into all values of Phone. You need to replace only the values of Phone that contain the 622 exchange. To extract the exchange from Phone, add an assignment statement to the DATA step. Notice that the SUBSTR function is used on the right side of the assignment statement.

```
data hrd.temp2(drop=exchange);
 set hrd.temp;
 Exchange=substr(phone,1,3);
 substr(phone,1,3)='433';
run;
```

Now the DATA step needs an IF-THEN statement to verify the value of the variable Exchange. If the exchange is 622, the assignment statement executes to replace the value of Phone.

```
data hrd.temp2(drop=exchange);
 set hrd.temp;
 Exchange=substr(phone,1,3);
 if exchange='622' then substr(phone,1,3)='433';
run;
```

After the DATA step is executed, the appropriate values of Phone contain the new exchange.

Obs	Address	City	State	Zip	Phone	Startdate	Enddate
1	65 ELM DR	CARY	NC	27513	4334549	19NOV1999	12JAN2000
2	11 SUN DR	CARY	NC	27513	4333251	07OCT1999	17NOV1999
3	712 HARDWICK STREET	CHAPEL HILL	NC	27514	9974749	22NOV1999	30DEC1999

Recall that when the SUBSTR function is on the right side of an assignment statement, the function extracts a substring.

```
MiddleInitial=substr(middlename,1,1);
```

When the SUBSTR function is on the left side of an assignment statement, the function replaces the contents of a character variable.

```
substr(region,1,3)='NNW';
```

## SCAN versus SUBSTR Functions

The SUBSTR function is similar to the SCAN function. Here is a brief comparison. Both the SCAN and SUBSTR functions can extract a substring from a character value:

- SCAN extracts words within a value that is marked by delimiters.
- SUBSTR extracts a portion of a value by starting at a specified location.

The SUBSTR function is best used when you know the exact position of the string that you want to extract from the character value. It is unnecessary to mark the string by delimiters. For example, the first two characters of the variable ID identify the class level of college students. The position of these characters does not vary within the values of ID.

The SUBSTR function is the best choice to extract class level information from ID. By contrast, the SCAN function is best used during the following actions:

- You know the order of the words in the character value.
- The starting position of the words varies.
- The words are marked by some delimiter.

## LEFT and RIGHT Functions

### Overview of the LEFT and RIGHT Functions

- The LEFT function left-aligns a character expression.  
LEFT returns an argument with leading blanks moved to the end of the value.
- The RIGHT function right-aligns a character expression.  
RIGHT returns an argument with trailing blanks moved to the start of the value.

---

Syntax, LEFT and RIGHT function:

**LEFT**(*argument*)

**RIGHT**(*argument*)

*argument* specifies a character constant, variable, or expression.

---

### Example: LEFT Function

The following SAS statements produce these results:

SAS Statement	Result
	-----1-----
<pre>a=' DUE DATE'; b=left(a); put b;</pre>	<pre>DUE DATE</pre>

---

### Example: RIGHT Function

The following SAS statements produce these results:

SAS Statement	Result
	-----1-----
<pre>a='Due Date  '; b=right(a); put a \$10.; put b \$10.;</pre>	<pre>Due Date Due Date</pre>

---

## Concatenation Operator

The concatenation operator concatenates character values. The operator can be expressed as || (two vertical bars), || (two broken vertical bars), or !(two exclamation points).

```
FullName = First || Middle || Last;
```

The length of the resulting variable is the sum of the lengths of each variable or constant in the concatenation operation, unless you use a LENGTH statement to specify a different length for the new variable.

The concatenation operator does not trim leading or trailing blanks. If variables are padded with trailing blanks, use the TRIM function to trim trailing blanks from values before concatenating them.

## TRIM Function

### Overview of the TRIM Function

The TRIM function removes trailing blanks from character expressions and returns one blank if the expression contains missing values.

```
FullName = trim(First) || trim(Middle) || Last;
```

The TRIM function is useful for concatenating because the concatenation operator does not remove trailing blanks.

If the TRIM function returns a value to a variable that was not yet assigned a length, by default, the variable length is determined by the length of the argument.

---

Syntax, TRIM function:

**TRIM**(*argument*)

*argument* can be any character expression. Here are examples:

- a character variable: trim(address)
  - another character function: trim(left(id))
- 

### Example: TRIM Function

```
data work.nametrim;
 length Name $ 20 First Middle Last $ 10;
 Name= 'Jones, Mary Ann, Sue';
 First = left(scan(Name, 2, ','));
 Middle = left(scan(Name, 3, ','));
 Last = scan(name, 1, ',');
 FullName = trim(First) || trim(Middle) || Last;
 drop Name;
run;

proc print data=work.nametrim;
run;
```

**Figure 14.18** TRIM Function

Obs	First	Middle	Last	FullName
1	Mary Ann	Sue	Jones	Mary AnnSueJones

10 bytes

30 bytes

## CATX Function

### Overview of the CATX Function

The CATX function enables you to concatenate character strings, remove leading and trailing blanks, and insert separators. The CATX function returns a value to a variable, or returns a value to a temporary buffer. The results of the CATX function are usually equivalent to those that are produced by a combination of the concatenation operator and the TRIM and LEFT functions.

In the DATA step, if the CATX function returns a value to a variable that has not previously been assigned a length, then the variable is given the length of 32,767. To save storage space, you can add a LENGTH statement to your DATA step, and specify an appropriate length for your variable. Because SAS sets the length of a new character variable the first time it is encountered in the DATA step, be sure to place the LENGTH statement before the assignment statements that contain the CATX function.

If the concatenation operator (||) returns a value to a variable that has not previously been assigned a length, then that variable is given a length that is the sum of the lengths of the values that are being concatenated.

Remember that you learned to use the TRIM function along with the concatenation operator to create one address variable that contains the values of the three variables Address, City, and Zip, and to remove extra blanks from the new values. You used the DATA step shown below.

```
data hrd.newtemp(drop=address city state zip);
 set hrd.temp;
 NewAddress=trim(address)||', '||trim(city)||', '||zip;
run;
```

You can accomplish the same concatenation using only the CATX function.

---

Syntax, CATX function:

**CATX(separator,string-1 <,...string-n>)**

- *separator* specifies the character string that is used as a separator between concatenated strings
  - *string* specifies a SAS character string.
- 

### Example: Create New Variable Using CATX Function

You want to create the new variable NewAddress by concatenating the values of the Address, City, and Zip variables from the data set Hrd.Temp. You want to strip excess blanks from the old variables' values and separate the variable values with a comma and a space. The DATA step below uses the CATX function to create NewAddress.

```
data hrd.newtemp(drop=address city state zip);
 set hrd.temp;
 NewAddress=catx(', ',address,city,zip);
run;
```

The revised DATA step creates the values that you would expect for NewAddress.

NewAddress
65 ELM DR,CARY,27513
11 SUN DR,CARY,27513
712 HARDWICK STREET,CHAPEL HILL,27514
5372 WHITEBUD ROAD,RALEIGH,27612
11 TALYN COURT,DURHAM,27713
101 HYNERIAN DR,CARRBORO,27510
11 RYGEL ROAD,CHAPEL HILL,27514
121 E. MOYA STREET,DURHAM,27713
1905 DOCK STREET,CARY,27513
1304 CRESCENT AVE,RALEIGH,27612

## INDEX Function

### Overview of the INDEX Function

The INDEX function enables you to search a character value for a specified string. The INDEX function searches values from left to right, looking for the first occurrence of the string. It returns the position of the string's first character. If the string is not found, it returns a value of 0.

---

Syntax, INDEX function:

**INDEX**(*source*,*excerpt*)

- *source* specifies the character variable or expression to search.
  - *excerpt* specifies a character string that is enclosed in quotation marks (").
- 

### Example: Search for Occurrences of a Phrase

Suppose you want to search the values of the variable Job, which lists job skills. You want to create a data set that contains the names of all temporary employees who have word processing experience. The following figure shows a partial list of observations in the Hrd.Temp data set.

To search for the occurrences of the phrase “word processing” in the values of the variable Job, you write the INDEX function as shown below. Note that the character string is enclosed in quotation marks.

```
index(job, 'word processing')
```

To create the new data set, include the INDEX function in a subsetting IF statement. Only those observations in which the function locates the string and returns a value greater than 0 are written to the data set.

```
data hrd.datapool;
 set hrd.temp;
 where index(job, 'word processing') > 0;
run;
```

Here is the data set that shows the temporary employees who have word processing experience. The program processed all of the observations in the Hrd.Temp data set.



**Figure 14.19** SAS Data Set Hrd.Datapool (partial output)

Job	Contact
word processing	WORD PROCESSOR
Bookkeeping word processing	BOOKKEEPER ASST.
word processing sec. work	WORD PROCESSOR
Bookkeeping word processing	BOOKKEEPER ASST.
word processing	WORD PROCESSOR
word processing sec. work	WORD PROCESSOR
word processing	WORD PROCESSOR
word processing	WORD PROCESSOR

Note that the INDEX function is case sensitive, so the character string that you search for must be specified exactly as it is recorded in the data set. For example, the INDEX function shown below would not locate any employees who have word-processing experience.

```
index(job, 'WORD PROCESSING')
```

### **Finding a String Regardless of Case**

To ensure that all occurrences of a character string are found, you can use the UPCASE or LOWCASE function with the INDEX function. The UPCASE and LOWCASE functions enable you to convert variable values to uppercase or lowercase letters. You can then specify the character string in the INDEX function accordingly.

```
index(upcase(job), 'WORD PROCESSING')
```

```
index(lowcase(job), 'word processing')
```

## **FIND Function**

### **Overview of the FIND Function**

The FIND function enables you to search for a specific substring of characters within a specified character string.

- The FIND function searches the string, from left to right, for the first occurrence of the substring, and returns the position in the string of the substring's first character.
- If the substring is not found in the string, the FIND function returns a value of 0.
- If there are multiple occurrences of the substring, the FIND function returns only the position of the first occurrence.

---

Syntax, FIND function:

**FIND**(*string*,*substring*<,<*modifiers*><,<*startpos*> )

- *string* specifies a character constant, variable, or expression that is searched for substrings.
- *substring* is a character constant, variable, or expression that specifies the substring of characters to search for in *string*.
- *modifiers* is a character constant, variable, or expression that specifies one or more modifiers.
- *startpos* is an integer that specifies the position at which the search should start and the direction of the search. The default value for *startpos* is 1.

*Note:* If *string* or *substring* is a character literal, you must enclose it in quotation marks.

---

### Details

The modifiers argument enables you to specify one or more modifiers for the function, as listed below.

- The modifier *i* causes the FIND function to ignore character case during the search. If this modifier is not specified, FIND searches for character substrings with the same case as the characters in substring.
- The modifier *t* trims trailing blanks from string and substring.

Here are several facts about modifiers and constants.

- If the modifier is a constant, enclose it in quotation marks.
- Specify multiple constants in a single set of quotation marks.
- Modifier values are not case sensitive.

If *startpos* is not specified, FIND starts the search at the beginning of the string and searches the string from left to right. If *startpos* is specified, the absolute value of *startpos* determines the position at which to start the search. The sign of *startpos* determines the direction of the search. That is, when *startpos* is positive, FIND searches from *startpos* to the right; when *startpos* is negative, FIND searches from *startpos* to the left.

### Example: Find Word Processing Jobs in a Data Set

The values of the variable *Job* are all lowercase. Therefore, to search for the occurrence of word processing in the values of the variable *Job*, you write the FIND function as shown below. Note that the character substring is enclosed in quotation marks.

```
find(job, 'word processing')
```

To create the new data set, include the FIND function in a subsetting IF statement. Only those observations in which the function locates the string and returns a value greater than 0 are written to the data set.

```
data hrd.datapool;
 set hrd.temp;
 where find(job, 'word processing') > 0;
run;
proc print data=hrd.datapool;
run;
```

**Figure 14.20** HTML Output: FIND function (partial output)

Obs	Address	Payrate	Days	Hours	Birthdate	Job	Contact	Dept	Site
1	65 ELM DR	10	11	88	25APR1979	word processing	WORD PROCESSOR	DP	26
2	5372 WHITEBUD ROAD	15	10	80	10SEP1976	Bookkeeping word processing	BOOKKEEPER ASST.	BK	57
3	11 TALYN COURT	12	9	72	06NOV1971	word processing sec. work	WORD PROCESSOR	DP	95
4	101 HYNERIAN DR	15	7	64	15JUN1975	Bookkeeping word processing	BOOKKEEPER ASST.	BK	44
5	11 RYGEL ROAD	12	12	96	15JUN1975	word processing	WORD PROCESSOR	DP	59
6	121 E. MOYA STREET	10	5	40	26NOV1971	word processing sec. work	WORD PROCESSOR	PUB	38
7	1905 DOCK STREET	10	5	30	06NOV1971	word processing	WORD PROCESSOR	DP	44
8	1304 CRESCENT AVE	15	5	25	24MAY1972	word processing	WORD PROCESSOR	DP	90

## UPCASE Function

The UPCASE function converts all letters in a character expression to uppercase.

---

Syntax, UPCASE function:

**UPCASE(argument)**

*argument* can be any SAS character expression, such as a character variable or constant.

---

In this example, the function is placed in an assignment statement in a DATA step. You can change the values of the variable Job in place.

```
data hrd.newtemp;
 set hrd.temp;
 Job=upcase(job);
run;
proc print data=hrd.newtemp;
run;
```

The new data set contains the converted values of Job.

**Figure 14.21** HTML Output: UPCASE Function (partial output)

Obs	Address	Payrate	Days	Hours	Birthdate	Job	Contact
1	65 ELM DR	10	11	88	25APR1979	WORD PROCESSING	WORD PROCESSOR
2	11 SUN DR	8	25	200	06OCT1975	FILING ADMIN.DUTIES	ADMIN. ASST.
3	712 HARDWICK STREET	40	26	208	22AUG1968	ORGANIZATIONAL DEV. SPECIALIS	CONSULTANT
4	5372 WHITEBUD ROAD	15	10	80	10SEP1976	BOOKKEEPING WORD PROCESSING	BOOKKEEPER ASST.
5	11 TALYN COURT	12	9	72	06NOV1971	WORD PROCESSING SEC. WORK	WORD PROCESSOR
6	101 HYNERIAN DR	15	7	64	15JUN1975	BOOKKEEPING WORD PROCESSING	BOOKKEEPER ASST.
7	11 RYSEL ROAD	12	12	96	15JUN1975	WORD PROCESSING	WORD PROCESSOR
8	121 E. MOYA STREET	10	5	40	26NOV1971	WORD PROCESSING SEC. WORK	WORD PROCESSOR
9	1905 DOCK STREET	10	5	30	06NOV1971	WORD PROCESSING	WORD PROCESSOR
10	1304 CRESCENT AVE	15	5	25	24MAY1972	WORD PROCESSING	WORD PROCESSOR

## LOWCASE Function

The LOWCASE function converts all letters in a character expression to lowercase.

---

Syntax, LOWCASE function:

**LOWCASE(argument)**

*argument* can be any SAS character expression, such as a character variable or constant.

---

In this example, the function converts the values of the variable Contact to lowercase letters.

```
data hrd.newtemp;
 set hrd.temp;
 Contact=lowcase(contact);
run;
proc print data=hrd.newtemp;
run;
```

The new data set is created.

**Figure 14.22** HTML Output: LOWCASE Function (partial output)

Obs	Address	Payrate	Days	Hours	Birthdate	Job	Contact
1	65 ELM DR	10	11	88	25APR1979	word processing	word processor
2	11 SUN DR	8	25	200	06OCT1975	Filing Admin.Duties	admin. asst.
3	712 HARDWICK STREET	40	26	208	22AUG1968	Organizational Dev. Specialis	consultant
4	5372 WHITEBUD ROAD	15	10	80	10SEP1976	Bookkeeping word processing	bookkeeper asst.
5	11 TALYN COURT	12	9	72	06NOV1971	word processing sec. work	word processor
6	101 HYNERIAN DR	15	7	64	15JUN1975	Bookkeeping word processing	bookkeeper asst.
7	11 RYSEL ROAD	12	12	96	15JUN1975	word processing	word processor
8	121 E. MOYA STREET	10	5	40	26NOV1971	word processing sec. work	word processor
9	1905 DOCK STREET	10	5	30	06NOV1971	word processing	word processor
10	1304 CRESCENT AVE	15	5	25	24MAY1972	word processing	word processor

## PROPCASE Function

The PROPCASE function converts all words in an argument to proper case (so that the first letter in each word is capitalized).

---

Syntax, PROPCASE function:

**PROPCASE**(*argument*<, *delimiter(s)*>)

- *argument* can be any SAS expression, such as a character variable or constant.
- *delimiter(s)* specifies one or more delimiters that are enclosed in quotation marks. The default delimiters are blank, forward slash, hyphen, open parenthesis, period, and tab.

*Note:* If you specify *delimiter(s)*, then the default delimiters are no longer in effect.

---

- The PROPCASE function first converts all letters to lowercase letters and then converts the first character of words to uppercase.
- The first character of a word is the first letter of a string or any letter preceded by a default list of delimiters.

Default delimiter List: blank / — ( . tab

**TIP** Delimiters can be specified as a second argument, instead of using the default list.

In this example, the function converts the values of the variable named Contact to proper case and uses the default delimiters.

```
data hrd.newtemp;
 set hrd.temp;
 Contact=propcase(contact);
run;
proc print data=hrd.newtemp;
run;
```

After the DATA step executes, the new data set is created.

Figure 14.23 HTML Output: PROPCASE Function

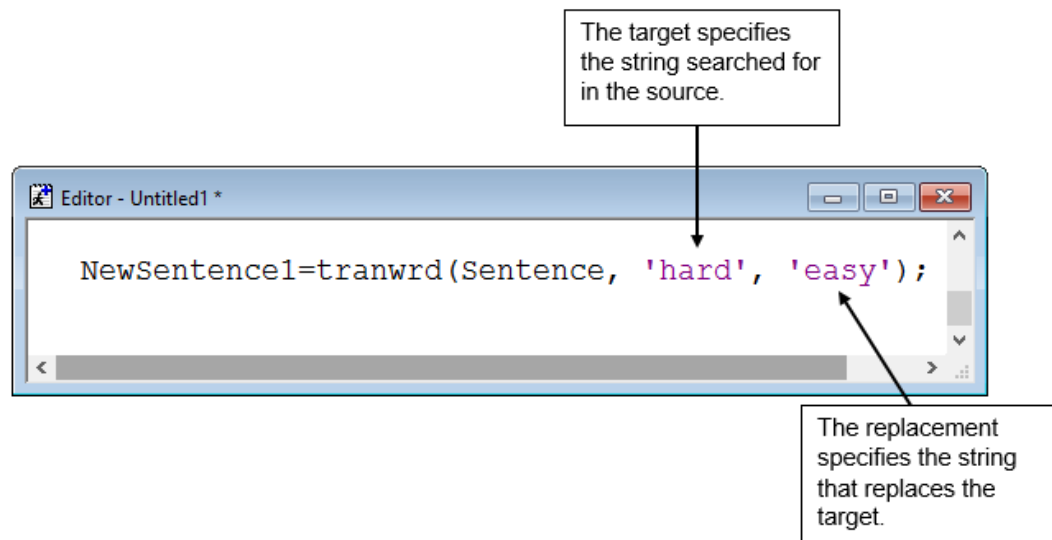
Obs	Address	Payrate	Days	Hours	Birthdate	Job	Contact
1	65 ELM DR	10	11	88	25APR1979	word processing	Word Processor
2	11 SUN DR	8	25	200	06OCT1975	Filing Admin.Duties	Admin. Asst.
3	712 HARDWICK STREET	40	26	208	22AUG1968	Organizational Dev. Specialis	Consultant
4	5372 WHITEBUD ROAD	15	10	80	10SEP1976	Bookkeeping word processing	Bookkeeper Asst.
5	11 TALYN COURT	12	9	72	06NOV1971	word processing sec. work	Word Processor
6	101 HYNERIAN DR	15	7	64	15JUN1975	Bookkeeping word processing	Bookkeeper Asst.
7	11 RYGEL ROAD	12	12	96	15JUN1975	word processing	Word Processor
8	121 E. MOYA STREET	10	5	40	26NOV1971	word processing sec. work	Word Processor
9	1905 DOCK STREET	10	5	30	06NOV1971	word processing	Word Processor
10	1304 CRESCENT AVE	15	5	25	24MAY1972	word processing	Word Processor

## TRANWRD Function

### Overview of the TRANWRD Function

The TRANWRD function replaces or removes all occurrences of a word in a character string. The translated characters can be located anywhere in the string.

Figure 14.24 TRANWRD Function



Syntax, TRANWRD function:

**TRANWRD**(*source*,*target*,*replacement*)

- *source* specifies the source string that you want to translate.
- *target* specifies the string that SAS searches for in *source*.
- *replacement* specifies the string that replaces *target*.

*Note:* *target* and *replacement* can be specified as variables or as character strings. If you specify character strings, be sure to enclose the strings in quotation marks (' ' or " ").

In a DATA step, if the TRANWRD function returns a value to a variable that has not previously been assigned a length, then that variable is given a length of 200 bytes. To save storage space, you can add a LENGTH statement to the DATA step and specify an appropriate length for the variable. SAS sets the length of a new character variable the first time it is encountered in the DATA step. Be sure to place the LENGTH statement before the assignment statements that contain the TRANWRD function.

### **Example: Update Variables in Place Using TRANWRD Function**

You can use TRANWRD function to update variables in place. In this example, the function updates the values of Name by changing every occurrence of the string Monroe to Manson.

```
name=tranwrđ(name, 'Monroe', 'Manson')
```

Another example of the TRANWRD function is shown below. In this case, two assignment statements use the TRANWRD function to change all occurrences of Miss or Mrs. to Ms.

```
data hrd.after;
 set hrd.before;
 name=tranwrđ(name, 'Miss', 'Ms. ');
 name=tranwrđ(name, 'Mrs.', 'Ms. ');
run;
proc print data=work.after;
run;
```

The new data set is created. The TRANWRD function changes all occurrences of Miss or Mrs. to Ms.

**Figure 14.25** HTML Output: TRANWRD Function

Obs	name
1	Ms. Millicent Garrett Fawcett
2	Ms. Charlotte Despard
3	Ms. Emmeline Pankhurst
4	Ms. Sylvia Pankhurst

## **COMPBL Function**

The COMPBL function removes multiple blanks from a character string by translating each occurrence of two or more consecutive blanks into a single blank.

---

Syntax, COMPBL function:

**COMPBL**(source)

- *source* specifies a character constant, variable, or expression to compress.
- 

In a DATA step, if the COMPBL function returns a value to a variable that has not previously been assigned a length, then the length of that variable defaults to the length of the first argument.

The following SAS statements produce these results:

SAS Statement	Result
	-----1-----2--
<pre>string='Hey   Diddle Diddle'; string=compbl(string); put string;</pre>	Hey Diddle Diddle
<pre>string='125    E Main St'; length address \$10; address=compbl(string); put address;</pre>	125 E Main

## COMPRESS Function

### Overview of the COMPRESS Function

The COMPRESS function returns a character string with specified characters removed from the original string. Null arguments are allowed and treated as a string with a length of zero.

The COMPRESS function returns a character string with specified characters removed from the original string.

Syntax, COMPRESS function:

**COMPRESS**(source<, characters> <, modifier(s)>)

- *source* specifies a character constant, variable, or expression from which specified characters are removed.
- *characters* specifies a character constant, variable, or expression that initializes a list of characters.

By default, the characters in this list are removed from the source argument. If you specify the K modifier in the third argument, then only the characters in this list are kept in the result.

*Note:* You can add more characters to this list by using other modifiers in the third argument.

*Note:* Enclose a literal string of characters in quotation marks.

- *modifier* specifies a character constant, variable, or expression in which each non-blank character modifies the action of the COMPRESS function. Blanks are ignored.

Based on the number of arguments, the COMPRESS functions works as follows:



Number of Arguments	Result
only the first argument, <i>source</i>	All blanks have been removed from the argument. If the argument is completely blank, then the result is a string with a length of zero. If you assign the result to a character variable with a fixed length, then the value of that variable will be padded with blanks to fill its defined length.
the first two arguments, <i>source</i> and <i>chars</i>	All characters that appear in the second argument are removed from the result.
three arguments, <i>source</i> , <i>chars</i> , and <i>modifier(s)</i>	The K modifier (specified in the third argument) determines whether the characters in the second argument are kept or removed from the result.

The COMPRESS function compiles a list of characters to keep or remove, comprising the characters in the second argument plus any types of characters that are specified by the modifiers. For example, the D modifier specifies digits. Both of the following function calls remove digits from the result:

```
compress(source, "1234567890");
compress(source, , "d");
```

To remove digits and plus or minus signs, you can use either of the following function calls:

```
compress(source, "1234567890+-");
compress(source, "+-", "d");
```

### Example: Compress a Character String

```
data _null_;
 a='A B C D';
 b=compress(a);
 put b=;
run;
```

The following SAS statements produce this result:

**Table 14.12** Compressing Blanks

SAS Statement	Result
	----+----1
a='A B C D ';	
b=compress(a);	ABCD
put b;	

---

## Modifying Numeric Values with Functions

SAS provides additional functions to create or modify numeric values. These include arithmetic, financial, and probability functions. This book covers the following selected functions.

### ***CEIL and FLOOR Functions***

To return integers that are greater than or equal to the argument, use these functions:

- The CEIL function returns the smallest integer that is greater than or equal to the argument.
- The FLOOR function returns the largest integer that is less than or equal to the argument.

---

Syntax, CEIL and FLOOR function:

**CEIL**(*argument*)

**FLOOR**(*argument*)

*argument* is a numeric variable, constant, or expression.

---

If the argument is within 1E-12 of an integer, the function returns that integer.

### ***INT Function***

To return the integer portion of a numeric value, use the INT function. Any decimal portion of the INT function argument is discarded.

---

Syntax, INT function:

**INT**(*argument*)

*argument* is a numeric variable, constant, or expression.

---

The two data sets shown below give before-and-after views of values that are truncated by the INT function.

```
data work.creditx;
 set sasuser.credit;
 Transaction=int(transaction);
run;
proc print data=work.creditx;
run;
```

**Figure 14.26** HTML Output Comparison: INT Function (partial output)

Before INT function		After INT function	
Type	Transaction	Type	Transaction
D	57.69	D	57
D	145.89	D	145
C	45.00	C	45
A	304.45	A	304
C	234.76	C	234
A	64.98	A	64
D	70.82	D	70
C	188.23	C	188
C	78.90	C	78
D	150.55	D	150

## ROUND Function

To round values to the nearest specified unit, use the ROUND function.

---

Syntax, ROUND function:

**ROUND**(*argument*,*round-off-unit*)

- *argument* is a numeric variable, constant, or expression.
  - *round-off-unit* is numeric and nonnegative.
- 

If a rounding unit is not provided, a default value of 1 is used, and the argument is rounded to the nearest integer. The two data sets shown below give before-and-after views of values that are modified by the ROUND function.

```
data work.creditx;
 set sasuser.credit;
 Transaction=round(transaction, .2);
run;
proc print data=work.creditx;
run;
```

**Figure 14.27** HTML Output Comparison: ROUND Function (partial output)

Before ROUND function		AFTER ROUND function	
Type	Transaction	Type	Transaction
D	57.69	D	57.6
D	145.89	D	145.8
C	45.00	C	45
A	304.45	A	304.4
C	234.76	C	234.8
A	64.98	A	65
D	70.82	D	70.8
C	188.23	C	188.2
C	78.90	C	79
D	150.55	D	150.6

---

## Nesting SAS Functions

To write more efficient programs you can nest functions as appropriate. You can nest any functions as long as the function that is used as the argument meets the requirements for the argument. For example, you can nest the SCAN function within the SUBSTR function in an assignment statement to compute the value for MiddleInitial:

```
MiddleInitial=substr(scan(name,3),1,1);
```

This example of nested numeric functions determines the number of years between June 15, 1999, and today:

```
Years=intck('year','15jun1999'd,today());
```

---

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- Which function calculates the average of the variables Var1, Var2, Var3, and Var4?
  - mean(var1,var4)
  - mean(var1-var4)
  - mean(of var1,var4)
  - mean(of var1-var4)

2. Within the data set `Hrd.Temp`, `PayRate` is a character variable and `Hours` is a numeric variable. What happens when the following program is run?

```
data work.temp;
 set hrd.temp;
 Salary=payrate*hours;
run;
```

- SAS converts the values of `PayRate` to numeric values. No message is written to the log.
  - SAS converts the values of `PayRate` to numeric values. A message is written to the log.
  - SAS converts the values of `Hours` to character values. No message is written to the log.
  - SAS converts the values of `Hours` to character values. A message is written to the log.
3. A typical value for the character variable `Target` is `123,456`. Which statement correctly converts the values of `Target` to numeric values when creating the variable `TargetNo`?
- `TargetNo=input(target,comma6.);`
  - `TargetNo=input(target,comma7.);`
  - `TargetNo=put(target,comma6.);`
  - `TargetNo=put(target,comma7.);`
4. A typical value for the numeric variable `SiteNum` is `12.3`. Which statement correctly converts the values of `SiteNum` to character values when creating the variable `Location`?
- `Location=dept||'/'||input(sitenum,3.1);`
  - `Location=dept||'/'||input(sitenum,4.1);`
  - `Location=dept||'/'||put(sitenum,3.1);`
  - `Location=dept||'/'||put(sitenum,4.1);`
5. Suppose the `YEARCUTOFF=` system option is set to 1920. Which `MDY` function creates the date value for January 3, 2020?
- `MDY(1,3,20)`
  - `MDY(3,1,20)`
  - `MDY(1,3,2020)`
  - `MDY(3,1,2020)`
6. The variable `Address2` contains values such as **Piscataway, NJ**. How do you assign the two-letter state abbreviations to a new variable named `State`?
- `State=scan(address2,2);`
  - `State=scan(address2,13,2);`
  - `State=substr(address2,2);`
  - `State=substr(address2,13,2);`
7. The variable `IDCode` contains values such as **123FA** and **321MB**. The fourth character identifies sex. How do you assign these character codes to a new variable named `Sex`?

- a. `Sex=scan(idcode,4);`
  - b. `Sex=scan(idcode,4,1);`
  - c. `Sex=substr(idcode,4);`
  - d. `Sex=substr(idcode,4,1);`
8. Because of the growth within the 919 area code, the telephone exchange 555 is being reassigned to the 920 area code. The data set `Clients.Piedmont` includes the variable `Phone`, which contains telephone numbers in the form **919-555-1234**. Which of the following programs correctly changes the values of `Phone`?
- a. 

```
data work.piedmont(drop=areacode exchange);
 set clients.piedmont;
 Areacode=substr(phone,1,3);
 Exchange=substr(phone,5,3);
 if areacode='919' and exchange='555'
 then scan(phone,1,3)='920';
run;
```
  - b. 

```
data work.piedmont(drop=areacode exchange);
 set clients.piedmont;
 Areacode=substr(phone,1,3);
 Exchange=substr(phone,5,3);
 if areacode='919' and exchange='555'
 then phone=scan('920',1,3);
run;
```
  - c. 

```
data work.piedmont(drop=areacode exchange);
 set clients.piedmont;
 Areacode=substr(phone,1,3);
 Exchange=substr(phone,5,3);
 if areacode='919' and exchange='555'
 then substr(phone,1,3)='920';
run;
```
  - d. 

```
data work.piedmont(drop=areacode exchange);
 set clients.piedmont;
 Areacode=substr(phone,1,3);
 Exchange=substr(phone,5,3);
 if areacode='919' and exchange='555'
 then phone=substr('920',1,3);
run;
```
9. Suppose you need to create the variable `FullName` by concatenating the values of `FirstName`, which contains first names, and `LastName`, which contains last names. What is the best way to remove extra blanks between first names and last names?
- a. 

```
data work.maillist;
 set retail.maillist;
 length FullName $ 40;
 fullname=trim(firstname)||' '||lastname;
run;
```
  - b. 

```
data work.maillist;
 set retail.maillist;
 length FullName $ 40;
 fullname=trim(firstname)||' '||lastname;
run;
```
  - c. 

```
data work.maillist;
```

```

 set retail.maillist;
 length FullName $ 40;
 fullname=trim(firstname)||' '||trim(lastname);
run;

```

```

d. data work.maillist;
 set retail.maillist;
 length FullName $ 40;
 fullname=trim(firstname)||' '||lastname);
run;

```

10. Within the data set `Furnitur.Bookcase`, the variable `Finish` contains values such as **ash, cherry, teak, matte-black**. Which of the following creates a subset of the data in which the values of `Finish` contain the string **walnut**? Make the search for the string case-insensitive.

```

a. data work.bookcase;
 set furnitur.bookcase;
 if index(finish,walnut) = 0;
run;

b. data work.bookcase;
 set furnitur.bookcase;
 if index(finish,'walnut') > 0;
run;

c. data work.bookcase;
 set furnitur.bookcase;
 if index(lowercase(finish),walnut) = 0;
run;

d. data work.bookcase;
 set furnitur.bookcase;
 if index(lowercase(finish),'walnut') > 0;
run;

```





## Chapter 15

# Processing Data with DO Loops

---

<b>The Basics of DO Loops</b> .....	<b>367</b>
DO Statement, Iterative Syntax .....	367
Details .....	368
<b>Constructing DO Loops</b> .....	<b>370</b>
DO Loop Execution .....	370
Counting Iterations of DO Loops .....	371
Using Explicit OUTPUT Statements .....	371
Decrementing DO Loops .....	372
Specifying a Series of Items .....	372
<b>Nesting DO Loops</b> .....	<b>373</b>
<b>Iteratively Processing Observations from a Data Set</b> .....	<b>375</b>
<b>Conditionally Executing DO Loops</b> .....	<b>376</b>
Overview .....	376
Using the DO UNTIL Statement .....	376
Using the DO WHILE Statement .....	377
<b>Using Conditional Clauses with the Iterative DO Statement</b> .....	<b>377</b>
<b>Chapter Quiz</b> .....	<b>379</b>

---

## The Basics of DO Loops

### *DO Statement, Iterative Syntax*

The *iterative DO statement* executes statements between the DO and END statements repetitively, based on the value of an index variable.

---

Syntax, DO statement, iterative:

**DO** *index-variable=specification-1* <, ...*specification-n*>;  
 ...*more SAS statements*...

**END;**

- *index-variable* names a variable whose value governs execution of the DO group.

*Note:* Unless you specify to drop it, the index variable is included in the data set that is being created.

**CAUTION:**

**Avoid changing the index variable within the DO group.** If you modify the index variable within the iterative DO group, you might cause infinite looping.

- *specification* denotes an expression or series of expressions such as these:

*start* <TO *stop*> <BY *increment*> <WHILE(*expression*) | UNTIL(*expression*)>

The DO group is executed first with *index-variable* equal to *start*. The value of *start* is evaluated before the first execution of the loop.

- *start* specifies the initial value of the index variable.
- TO *stop* specifies the ending value of the index variable.

**TIP** Any changes to *stop* made within the DO group do not affect the number of iterations. To stop iteration of a loop before it finishes processing, change the value of *index-variable* so that it passes the value of *stop*, or use a LEAVE statement to go to a statement outside the loop.

- BY *increment* specifies a positive or negative number (or an expression that yields a number) to control the incrementing of *index-variable*.

The value of *increment* is evaluated before the execution of the loop. If no increment is specified, the index variable is increased by 1. When *increment* is positive, *start* must be the lower bound, and *stop*, if present, must be the upper bound for the loop. If *increment* is negative, *start* must be the upper bound, and *stop*, if present, must be the lower bound for the loop.

- WHILE(*expression*) | UNTIL(*expression*) evaluates, either before or after execution of the DO group, any SAS expression that you specify. Enclose the expression in parentheses.

A WHILE expression is evaluated before each execution of the loop, so that the statements inside the group are executed repetitively while the expression is true. An UNTIL expression is evaluated after each execution of the loop, so that the statements inside the group are executed repetitively until the expression is true.

*Note:* The order of the optional TO and BY clauses can be reversed.

*Note:* When you use more than one specification, each one is evaluated before its execution.

---

## Details

DO loops process a group of statements repeatedly rather than once. This can greatly reduce the number of statements required for a repetitive calculation. For example, these 12 sum statements compute a company's annual earnings from investments. Notice that all 12 statements are identical.

```
data finance.earnings;
 set finance.master;
```

```

Earned=0;
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
earned+(amount+earned)*(rate/12);
run;

```

In this program, each sum statement accumulates the calculated interest that is earned for an investment for one month. The variable `Earned` is created in the DATA step to store the earned interest. The investment is compounded monthly, meaning that the value of the earned interest is cumulative.

By contrast, a DO loop enables you to achieve the same results with fewer statements. In this case, the sum statement executes 12 times within the DO loop during each iteration of the DATA step.

```

data finance.earnings;
 set finance.master;
 Earned=0;
 do month=1 to 12;
 earned+(amount+earned)*(rate/12);
 end;
run;

```

When creating a DO loop with the iterative DO statement, you must specify an index variable. The index variable stores the value of the current iteration of the DO loop. You can use any valid SAS name. The following brief examples show you the syntax.

```

DO index-variable=start TO stop BY increment;
 ...more SAS statements...
END;

```

Next, specify the conditions that execute the DO loop. A simple specification contains a start value, a stop value, and an increment value for the DO loop.

```

Do index-variable=start TO stop BY increment;
 ...more SAS statements...
END;

```

The start value specifies the initial value of the index variable.

```

DO index-variable=start TO stop BY increment;
 ...more SAS statements...
END;

```

The TO clause specifies the stop value. The stop value is the last index value that executes the DO loop.

```

DO index-variable=start TO stop BY increment;
 ...more SAS statements...
END;

```

The optional BY clause specifies an increment value for the index variable. Typically, you want the DO loop to increment by 1 for each iteration. If you do not specify a BY clause, the default increment value is 1.

```
DO index-variable=start TO stop BY increment;
 ...more SAS statements...
END;
```

For example, the specification below increments the index variable by 1, resulting in quiz values of 1, 2, 3, 4, and 5:

```
do quiz=1 to 5;
```

By contrast, the following specification increments the index variable by 2, resulting in rows values of 2, 4, 6, 8, 10, and 12:

```
do rows=2 to 12 by 2;
```

## Constructing DO Loops

### DO Loop Execution

Here is how the DO loop executes in the DATA step. This example sums the interest that was earned each month for a one-year investment.

```
data finance.earnings;
 Amount=1000;
 Rate=.075/12;
 do month=1 to 12;
 Earned+(amount+earned)*rate;
 end;
run;
```

This DATA step does not read data from an external source. When submitted, it compiles and then executes only once to generate data. During compilation, the program data vector is created for the Finance.Earnings data set.

Program Data Vector

<u>N</u>	Amount	Rate	month	Earned
•	•	•	•	•

When the DATA step executes, the values of Amount and Rate are assigned.

Program Data Vector

<u>N</u>	Amount	Rate	month	Earned
1	1000	0.0625	•	0

Next, the DO loop executes. During each execution of the DO loop, the value of Earned is calculated and is added to its previous value. Then the value of month is incremented. On the 12th execution of the DO loop, the value of month is incremented to 12 and the value of Earned is 1069.839.

Program Data Vector

N	Amount	Rate	month	Earned
1	1000	0.075	12	1069.83

After the 12th execution of the DO loop, the value of Month is incremented to 13. Because 13 exceeds the stop value of the iterative DO statement, the DO loop stops executing, and processing continues to the next DATA step statement. The end of the DATA step is reached, the values are written to the Finance.Earnings data set, and in this example, the DATA step ends. Only one observation is written to the data set.

	Amount	Rate	month	Earned
1	1000	0.0625	13	1069.8899918

Notice that the index variable Month is also stored in the data set. In most cases, the index variable is needed only for processing the DO loop and can be dropped from the data set.

### Counting Iterations of DO Loops

In some cases, it is useful to create an index variable to count and store the number of iterations in the DO loop. Then you can drop the index variable from the data set.

```
data work.earn (drop=counter);
 Value=2000;
 do counter=1 to 20;
 Interest=value*.075;
 value+interest;
 Year+1;
 end;
run;
```

**Figure 15.1** HTML Output: Iteration of DO Loops

Obs	Value	Interest	Year
1	8495.70	592.723	20

The sum statement Year+1 accumulates the number of iterations of the DO loop and stores the total in the new variable Year. The final value of Year is then stored in the data set, whereas the index variable counter is dropped. The data set has one observation.

### Using Explicit OUTPUT Statements

To create an observation for each iteration of the DO loop, place an OUTPUT statement inside the loop. By default, every DATA step contains an implicit OUTPUT statement at the end of the step. But placing an explicit OUTPUT statement in a DATA step overrides automatic output, causing SAS to add an observation to the data set only when the explicit OUTPUT statement is executed.

The previous example created one observation because it used automatic output at the end of the DATA step. In the following example, the OUTPUT statement overrides automatic output, so the DATA step writes 20 observations.

```
data work.earn;
```

```

Value=2000;
do Year=1 to 20;
 Interest=value*.075;
 value+interest;
 output;
end;
run;

```

**Figure 15.2** HTML Output: OUTPUT Statement Inside Each DO Loop (Partial Output)

Obs	Value	Year	Interest
1	2150.00	1	150.000
2	2311.25	2	161.250
3	2484.59	3	173.344
4	2670.94	4	186.345
17	6838.71	17	477.119
18	7351.61	18	512.903
19	7902.98	19	551.371
20	8495.70	20	592.723

### Decrementing DO Loops

You can decrement a DO loop's index variable by specifying a negative value for the BY clause. For example, the specification in this iterative DO statement decreases the index variable by 1, resulting in values of 5, 4, 3, 2, and 1. The following brief examples show you the syntax.

```

DO index-variable=5 to 1 by -1;
 ...more SAS statements...
END;

```

When you use a negative BY clause value, the start value must always be greater than the stop value in order to decrease the index variable during each iteration.

```

DO index-variable=5 to 1 by -1;
 ...more SAS statements...
END;

```

### Specifying a Series of Items

You can also specify how many times a DO loop executes by listing items in a series.

---

Syntax, DO loop with a variable list:

```
DO index-variable=value1, value2, value3... ;
 ...more SAS statements...
```

```
END;
```

*values* can be character or numeric.

---

When the DO loop executes, it executes once for each item in the series. The index variable equals the value of the current item. You must use commas to separate items in the series.

To list items in a series, you must specify one of the following, as shown in the syntax:

- all numeric values.

```
DO index-variable=2,5,9,13,27;
 ...more SAS statements...
END;
```

- all character values, which are enclosed in quotation marks.

```
DO index-variable='MON', 'TUE', 'WED', 'THR', 'FRI';
 ...more SAS statements...
END;
```

- all variable names. The index variable takes on the values of the specified variables.

```
DO index-variable=win,place,show;
 ...more SAS statements...
END;
```

Variable names must represent either all numeric or all character values. Do not enclose variable names in quotation marks.

---

## Nesting DO Loops

Iterative DO statements can be executed within a DO loop. Putting a DO loop within a DO loop is called nesting.

```
do i=1 to 20;
 ...more SAS statements...
 do j=1 to 10;
 ...more SAS statements...
 end;
 ...more SAS statements...
end;
```

The DATA step below computes the value of a one-year investment that earns 7.5% annual interest, compounded monthly.

```
data work.earn;
 Capital=2000;
 do month=1 to 12;
 Interest=capital*(.075/12);
 capital+interest;
 end;
run;
```

Assume that the same amount of capital is to be added to the investment each year for 20 years. The new program must perform the calculation for each month during each of the 20 years. To do this, you can include the monthly calculations within another DO loop that executes 20 times.

```
data work.earn;
 do year=1 to 20;
 Capital+2000;
 do month=1 to 12;
 Interest=capital*(.075/12);
 capital+interest;
 end;
 end;
run;
```

During each iteration of the outside DO loop, an additional 2,000 is added to the capital, and the nested DO loop executes 12 times.

```
data work.earn;
 do year=1 to 20;
 Capital+2000;
 do month=1 to 12;
 Interest=capital*(.075/12);
 capital+interest;
 end;
 end;
run;
```

Remember, in order for nested DO loops to execute correctly, you must do the following:

- Assign a unique index-variable name in each iterative DO statement.

```
data work.earn;
 do year=1 to 20;
 Capital+2000;
 do month=1 to 12;
 Interest=capital*(.075/12);
 capital+interest;
 end;
 end;
run;
```

- End each DO loop with an END statement.

```
data work.earn;
 do year=1 to 20;
 Capital+2000;
 do month=1 to 12;
 Interest=capital*(.075/12);
 capital+interest;
 end;
 end;
run;
```

It is easier to manage nested DO loops if you indent the statements in each DO loop as shown above.



## Iteratively Processing Observations from a Data Set

Previous examples of DATA steps used DO loops to generate one or more observations from one iteration of the DATA step. It is also possible to write a DATA step that reads a data set to compute the value of a new variable.

The SAS data set `Finance.CDRates` contains interest rates for certificates of deposit (CDs) that are available from several institutions.

Suppose you want to compare how much each CD earns at maturity with an investment of \$5,000. The DATA step below creates a new data set, `Work.Compare`, that contains the added variable, `Investment`.

```
data work.compare(drop=i);
 set finance.cdRates;
 Investment=5000;
 do i=1 to years;
 investment+rate*investment;
 end;
run;
proc print data=work.compare noobs;
run;
```

The index variable is used only to execute the DO loop, so it is dropped from the new data set. Notice that the data set variable `Years` is used as the stop value in the iterative DO statement. As a result, the DO loop executes the number of times specified by the current value of `Years`.

Here is what happens during each iteration of the DATA step:

- An observation is read from `Finance.CDRates`.
- The value **5000** is assigned to the variable `Investment`.
- The DO loop executes, based on the current value of `Years`.
- The value of `Investment` is incremented (each time that the DO loop executes), using the current value of `Rate`.

At the bottom of the DATA step, the first observation is written to the `Work.Compare` data set. Control returns to the top of the DATA step, and the next observation is read from `Finance.CDRates`. These steps are repeated for each observation in `Finance.CDRates`. The resulting data set contains the computed values of `Investment` for all observations that have been read from `Finance.CDRates`.

**Figure 15.3** HTML Output: *Work.Compare* Data Set

Institution	Rate	Years	Investment
MBNA America	0.0817	5	7404.64
Metropolitan Bank	0.0814	3	6323.09
Standard Pacific	0.0806	4	6817.57

---

## Conditionally Executing DO Loops

### Overview

The iterative DO statement specifies a fixed number of iterations for the DO loop. However, there are times when you want to execute a DO loop until a condition is reached or while a condition exists, but you do not know how many iterations are needed.

Suppose you want to calculate the number of years required for an investment to reach \$50,000. In the DATA step below, using an iterative DO statement is inappropriate because you are trying to determine the number of iterations required for Capital to reach \$50,000.

```
data work.invest;
 do year=1 to ? ;
 Capital+2000;
 capital+capital*.10;
 end;
run;
```

The DO WHILE and DO UNTIL statements enable you to execute DO loops based on whether a condition is true or false.

### Using the DO UNTIL Statement

The DO UNTIL statement executes a DO loop until the expression becomes true.

---

Syntax, DO UNTIL statement:

```
DO UNTIL(expression);
 ...more SAS statements...
END;
```

*expression* is a valid SAS expression enclosed in parentheses.

---

The expression is not evaluated until the bottom of the loop. Therefore, a DO UNTIL loop always executes at least once. When the expression is evaluated as true, the DO loop stops.

Assume you want to know how many years it takes to earn \$50,000 if you deposit \$2,000 each year into an account that earns 10% interest. The DATA step below uses a DO UNTIL statement to perform the calculation until \$50,000 is reached. Each iteration of the DO loop represents one year.

```
data work.invest;
 do until (Capital>=50000);
 capital+2000;
 capital+capital*.10;
 Year+1;
 end;
run;
```

Here is what happens during each iteration of the DO loop:

- 2000 is added to the value of Capital to reflect the annual deposit of \$2,000.
- 10% interest is added to Capital.
- The value of Year is incremented by 1.

Because there is no index variable in the DO UNTIL statement, the variable Year is created in a sum statement to count the number of iterations of the DO loop. This program produces a data set that contains the single observation shown below. To accumulate more than \$50,000 in capital requires 13 years (and 13 iterations of the DO loop).

**Figure 15.4** SAS Data Set *Work.Invest*: Accumulation of More Than \$50,000

	Capital	Year
1	53949.97	13

## Using the DO WHILE Statement

Like the DO UNTIL statement, the DO WHILE statement executes DO loops conditionally. You can use the DO WHILE statement to execute a DO loop while the expression is true.

---

Syntax, DO WHILE statement:

```
DO WHILE(expression);
 ...more SAS statements...
END;
```

*expression* is a valid SAS expression enclosed in parentheses.

---

An important difference between the DO UNTIL and DO WHILE statements is that the DO WHILE expression is evaluated at the top of the DO loop. If the expression is false the first time it is evaluated, the DO loop never executes. For example, in the following program, because the value of Capital is initially zero, which is less than 50,000, the DO loop does not execute.

```
data work.invest;
 do while(Capital>=50000);
 capital+2000;
 capital+capital*.10;
 Year+1;
 end;
run;
```

---

## Using Conditional Clauses with the Iterative DO Statement

Previous examples showed that the DO WHILE and DO UNTIL statements enable you to execute statements conditionally and that the iterative DO statement enables you to execute statements a set number of times, unconditionally.

```
DO WHILE(expression);
```

```
DO UNTIL(expression);
DO index-variable=start TO stop BY increment;
```

In this DATA step, the DO UNTIL statement determines how many years it takes (13) for an investment to reach \$50,000.

```
data work.invest;
 do until(Capital>=50000);
 Year+1;
 capital+2000;
 capital+capital*.10;
 end;
run;
```

**Figure 15.5** SAS Data Set Work.Invest: Number of Years for an Investment to Reach \$50,000

	Capital	Year
1	53949.97	13

Suppose you also want to limit the number of years you invest your capital to 10 years. You can add the UNTIL or WHILE expression to an iterative DO statement to further control the number of iterations. This iterative DO statement enables you to execute the DO loop until Capital is greater than or equal to 50000 or until the DO loop executes 10 times, whichever occurs first.

```
data work.invest;
 do year=1 to 10 until(Capital>=50000);
 capital+2000;
 capital+capital*.10;
 end;
run;
```

**Figure 15.6** SAS Data Set Work.Invest: Executing DO Loop until Capital >=\$50,000

	Year	Capital
1	10	35062.33

In this case, the DO loop stops executing after 10 iterations, and the value of Capital never reaches 50000. If you increase the amount added to Capital each year to 4000, the DO loop stops executing after the eighth iteration when the value of Capital exceeds 50000.

```
data work.invest;
 do year=1 to 10 until(Capital>=50000);
 capital+4000;
 capital+capital*.10;
 end;
run;
```

**Figure 15.7** SAS Data Set Work.Invest: Increase Amount Added to Capital Using a DO Loop

	year	Capital
1	8	50317.91

The UNTIL and WHILE expressions in an iterative DO statement function similarly to the DO UNTIL and DO WHILE statements. As shown in the following syntax, both statements require a valid SAS expression that is enclosed in parentheses.

```
DO index-variable=start TO stop BY increment UNTIL(expression);
```

```
DO index-variable=start TO stop BY increment WHILE(expression);
```

The UNTIL expression is evaluated at the bottom of the DO loop. Therefore, the DO loop always executes at least once. The WHILE expression is evaluated before the execution of the DO loop. As a result, if the condition is initially false, the DO loop never executes.

---

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

1. Which statement is false regarding the use of DO loops?
  - a. They can contain conditional clauses.
  - b. They can generate multiple observations.
  - c. They can be used to combine DATA and PROC steps.
  - d. They can be used to read data.
2. During each execution of the following DO loop, the value of Earned is calculated and is added to its previous value. How many times does this DO loop execute?

```
data finance.earnings;
 Amount=1000;
 Rate=.075/12;
 do month=1 to 12;
 Earned+(amount+earned)*rate;
 end;
run;
```

- a. 0
  - b. 1
  - c. 12
  - d. 13
3. On January 1 of each year, \$5000 is invested in an account. Complete the DATA step below to determine the value of the account after 15 years if a constant interest rate of 10% is expected.

```
data work.invest;
 ...
 Capital+5000;
 capital+(capital*.10);
end;
run;
```

- a. do count=1 to 15;
  - b. do count=1 to 15 by 10%;
  - c. do count=1 to capital;

d. `do count=capital to (capital*.10);`

4. In the data set `Work.Invest`, what would be the stored value for `Year`?

```
data work.invest;
 do year=1990 to 2004;
 Capital+5000;
 capital+(capital*.10);
 end;
run;
```

- a. missing
  - b. 1990
  - c. 2004
  - d. 2005
5. Which of the following statements is false regarding the program shown below?

```
data work.invest;
 do year=1990 to 2004;
 Capital+5000;
 capital+(capital*.10);
 output;
 end;
run;
```

- a. The `OUTPUT` statement writes current values to the data set immediately.
  - b. The last value for `Year` in the new data set is **2005**.
  - c. The `OUTPUT` statement overrides the automatic output at the end of the `DATA` step.
  - d. The `DO` loop performs 15 iterations.
6. How many observations will the data set `Work.Earn` contain?

```
data work.earn;
 Value=2000;
 do year=1 to 20;
 Interest=value*.075;
 value+interest;
 output;
 end;
run;
```

- a. 0
  - b. 1
  - c. 19
  - d. 20
7. Which of the following would you use to compare the result of investing \$4,000 a year for five years in three different banks that compound interest monthly? Assume a fixed rate for the five-year period.
- a. `DO WHILE` statement
  - b. nested `DO` loops
  - c. `DO UNTIL` statement

- d. a DO group
8. Which statement is false regarding DO UNTIL statements?
- a. The condition is evaluated at the top of the loop, before the enclosed statements are executed.
  - b. The enclosed statements are always executed at least once.
  - c. SAS statements in the DO loop are executed until the specified condition is true.
  - d. The DO loop must have a closing END statement.
9. Select the DO WHILE statement that would generate the same result as the program below.

```
data work.invest;
capital=100000;
 do until(Capital gt 500000);
 Year+1;
 capital+(capital*.10);
 end;
run;
```

- a. do while(Capital ge 500000);
  - b. do while(Capital=500000);
  - c. do while(Capital le 500000);
  - d. do while(Capital>500000);
10. In the following program, complete the statement so that the program stops generating observations when Distance reaches 250 miles or when 10 gallons of fuel have been used.

```
data work.go250;
 set perm.cars;
 do gallons=1 to 10 ... ;
 Distance=gallons*mpg;
 output;
 end;
run;
```

- a. while(Distance<250)
- b. when(Distance>250)
- c. over(Distance le 250)
- d. until(Distance=250)





## Chapter 16

## Processing Data with Arrays

---

<b>The Basics of SAS Arrays</b> .....	<b>383</b>
<b>Creating One-Dimensional Arrays</b> .....	<b>384</b>
ARRAY Statement Syntax .....	384
Specifying an Array Name .....	385
Specifying the Dimension .....	385
Specifying Array Elements .....	385
Specifying Variable Lists as Array Elements .....	385
Specifying a Numbered Range of Variables .....	386
Specifying All Numeric Variables .....	386
Specifying All Character Variables .....	386
Specifying All Character or All Numeric Variables .....	386
Examples: Referencing Elements of an Array .....	387
Compilation and Execution .....	389
Using the DIM Function in an Iterative DO Statement .....	390
<b>Expanding Your Use of Arrays</b> .....	<b>391</b>
Creating Variables in an ARRAY Statement .....	391
Default Variable Names .....	392
Creating Arrays of Character Variables .....	392
Assigning Initial Values to Arrays .....	394
Creating Temporary Array Elements .....	396
<b>Chapter Quiz</b> .....	<b>397</b>

---

## The Basics of SAS Arrays

An *array* is a temporary grouping of SAS variables that are arranged in a particular order and identified by an array name. Here are several facts about arrays.

- Arrays exist only for the duration of the current DATA step.
- Arrays are referenced by the array name and a subscript.
- The array name is not a variable.

An array is a convenient way of temporarily identifying a group of variables. Arrays are often referenced in DO loops because more than one element in an array must be processed.

One reason for using an array is to reduce the number of statements that are required for processing variables. For example, in the DATA step below, the values of seven data set variables are converted from Fahrenheit to Celsius.

```
data work.report;
 set master.temps;
 mon=5*(mon-32)/9;
 tue=5*(tue-32)/9;
 wed=5*(wed-32)/9;
 thr=5*(thr-32)/9;
 fri=5*(fri-32)/9;
 sat=5*(sat-32)/9;
 sun=5*(sun-32)/9;
run;
```

The assignment statements perform the same calculation on each variable in this series of statements. Only the name of the variable changes in each statement.

By grouping the variables into a one-dimensional array, you can process the variables in a DO loop. You use fewer statements, and the DATA step program is more easily modified or corrected.

```
data work.report(drop=day);
 set master.temps;
 array wkday{7} mon tue wed thr fri sat sun;
 do day=1 to 7;
 wkday{day}=5*(wkday{day}-32)/9;
 end;
run;
```

---

## Creating One-Dimensional Arrays

### **ARRAY Statement Syntax**

An ARRAY statement groups data set variables into an array.

---

Syntax, ARRAY statement:

**ARRAY** *array-name*{*dimension*} <*elements*>;

- *array-name* specifies the name of the array.
  - *dimension* describes the number and arrangement of array elements.
  - *elements* lists the variables to include in the array. Array elements must be either all numeric or all character. If no elements are listed, new variables are created with default names.
- 

*Note:* Do not give an array the same name as a variable in the same DATA step. Also, avoid using the name of a SAS function; the array is correct, but you will not be able to use the function in the same DATA step, and a warning message appears in the SAS log.

*Note:* You cannot use array names in LABEL, FORMAT, DROP, KEEP, or LENGTH statements. Arrays exist only for the duration of the DATA step.

## Specifying an Array Name

To group the variables in the array, first give the array a name. This example makes an array named Sales.

```
array sales{4} qtr1 qtr2 qtr3 qtr4;
```

## Specifying the Dimension

Following the array name, specify the dimension of the array. The dimension describes the number and arrangement of elements in the array. There are several ways to specify the dimension:

- In a one-dimensional array, you can simply specify the number of array elements. The array elements are the variables that you want to reference and process elsewhere in the DATA step.

```
array sales{4} qtr1 qtr2 qtr3 qtr4;
```

- The dimension of an array does not have to be the number of array elements. You can specify a range of values for the dimension when you define the array. For example, you can define the array Sales as follows:

```
array sales{96:99} totals96 totals97 totals98 totals99;
```

- You can also indicate the dimension of a one-dimensional array by using an asterisk (\*). In this way, SAS determines the dimension of the array by counting the number of elements.

```
array sales{*} qtr1 qtr2 qtr3 qtr4;
```

- Enclose the dimension in either parentheses, braces, or brackets.

```
()
array sales{4} qtr1 qtr2 qtr3 qtr4;
[]
```

*Note:* It is a recommended best practice to use brackets instead of braces or parentheses.

## Specifying Array Elements

When specifying the elements of an array, list each variable name that you want to include in the array. When listing elements, separate each element with a space. As with all SAS statements, you end the ARRAY statement with a semicolon (;).

```
array sales{4} qtr1 qtr2 qtr3 qtr4;
```

You can also specify array elements as a variable list. Here is an example of an ARRAY statement that groups the variables Qtr1 through Qtr4 into a one-dimensional array, using a variable list.

```
array sales{4} qtr1-qtr4;
```

## Specifying Variable Lists as Array Elements

You can specify variable lists in the forms shown below. Each type of variable list is explained in more detail following the table.

**Table 16.1** Variables and Their Forms

Variables	Form
a numbered range of variables	Var1-Var <i>n</i>
all numeric variables	<code>_NUMERIC_</code>
all character variables	<code>_CHARACTER_</code>
all character or all numeric variables	<code>_ALL_</code>

### Specifying a Numbered Range of Variables

```
Qtr1 Qtr2 Qtr3 Qtr4 → Qtr1-Qtr4
```

- The variables must have the same name except for the last character or characters.
- The last character of each variable must be numeric.
- The variables must be numbered consecutively.

```
array sales{4} qtr1-qtr4;
```

In the preceding example, you would use `sales{4}` to reference `Qtr4`. However, the index of an array does not have to range from one to the number of array elements. You can specify a range of values for the index when you define the array. For example, you can define the array `sales` as follows:

```
array sales{96:99} totals96-totals99;
```

### Specifying All Numeric Variables

```
Amount Rate Term → _NUMERIC_
```

`_NUMERIC_` specifies all numeric variables that have already been defined in the current DATA step.

```
array sales{*} _numeric_;
```

### Specifying All Character Variables

```
FrstName LastName Address → _CHARACTER_
```

`_CHARACTER_` specifies all character variables that have already been defined in the current DATA step.

```
array sales{*} _character_;
```

### Specifying All Character or All Numeric Variables

```
FrstName LastName Address → _ALL_
```

```
Amount Rate Term → _ALL_
```

`_ALL_` specifies all variables that have already been defined in the current DATA step. The variables must all be of the same type: either all character or all numeric.

```
array sales{*} _all_;
```

### Examples: Referencing Elements of an Array

There are several ways to use arrays to process variables in the DATA step.

You can group variables in a one-dimensional array.

```
data work.report(drop=day);
 set master.temps;
 array wkday[7] mon tue wed thr fri sat sun;
 do day=1 to 7;
 if wkday{day}>95 then output;
 end;
run;
```

You can specify array elements.

```
data work.weights(drop=day);
 set master.class;
 array wt[6] w1-w6;
 do day=1 to 6;
 wt{day}=wt{day}*2.2;
 end;
run;
```

You can create numbered ranged variables.

```
data work.new(drop=day);
 set master.synms;
 array term[9] also1-also9;
 do day=1 to 9;
 if term{day} ne ' ' then output;
 end;
run;
```

Arrays are powerful because you can use them to reference the elements of an array by an index value. Typically, arrays are used with DO loops to process multiple variables and to perform repetitive calculations.

```
array quarter[4] jan apr jul oct;
do qtr=1 to 4;
 YearGoal=quarter{qtr}*1.2;
end;
```

When you define an array in a DATA step, a subscript value is assigned to each array element. The subscript values are assigned in the order of the array elements.

```
 1 2 3 4
array quarter[4] jan apr jul oct;
do qtr=1 to 4;
 YearGoal=quarter{qtr}*1.2;
end;
```

You use an array reference to perform an action on an array element during execution. To reference an array element in the DATA step, specify the name of the array, followed by a subscript value enclosed in brackets.

---

Syntax, ARRAY reference:

*array-name*[*subscript*]

*subscript*

- is enclosed in parentheses, braces, or brackets.
  - specifies a variable, a SAS expression, or an integer.
  - is within the lower and upper bounds of the dimension of the array.
- 

When used in a DO loop, the index variable of the iterative DO statement can reference each element of the array.

```
array quarter{4} jan apr jul oct;
do i=1 to 4;
 YearGoal=quarter{i}*1.2;
end;
```

For example, the DO loop above increments the index variable *i* from the lower bound of the quarter array, 1, to the upper bound, 4. The following sequence illustrates this process:

1

```
array quarter{4} jan apr jul oct;
do i=1 to 4;
 YearGoal=quarter{1}*1.2;
end;
```

2

```
array quarter{4} jan apr jul oct;
do i=1 to 4;
 YearGoal=quarter{2}*1.2;
end;
```

3

```
array quarter{4} jan apr jul oct;
do i=1 to 4;
 YearGoal=quarter{3}*1.2;
end;
```

4

```
array quarter{4} jan apr jul oct;
do i=1 to 4;
 YearGoal=quarter{4}*1.2;
end;
```

During each iteration of the DO loop, `quarter{i}` refers to an element of the array `quarter` in the order listed.

The Health Center of a company conducts a fitness class for its employees. Each week, participants are weighed so that they can monitor their progress. The weight data, currently stored in kilograms, needs to be converted to pounds.

**Figure 16.1** SAS Data Set Hrd.Fitclass

Name	weight1	weight2	weight3	weight4	weight5	weight6
Alicia	69.6	68.9	68.8	67.4	66.0	66.2
Betsy	52.6	52.6	51.7	50.4	49.8	49.1
Brenda	68.6	67.6	67.0	66.4	65.8	65.2
Carl	67.6	66.6	66.0	65.4	64.8	64.2
Carmela	63.6	62.5	61.9	61.4	60.8	58.2
David	70.6	69.8	69.2	68.4	67.8	67.0

You can use a DO loop to update the variables Weight1 through Weight6 for each observation in the Hrd.Fitclass data set.

```
data hrd.convert(drop=i);
 set hrd.fitclass;
 array wt{6} weight1-weight6;
 do i=1 to 6;
 wt{i}=wt{i}*2.2046;
 end;
run;
```

### Compilation and Execution

To understand how the DO loop processes the array elements, examine the compilation and execution phases of this DATA step.

During compilation, the program data vector is created.

Program Data Vector

_N_	Name	Weight1	Weight2	Weight3	Weight4	Weight5	Weight6	i

The DATA step is scanned for syntax errors.

The index values of the array elements are assigned. Note that the array name is not included in the program data vector. The array exists only for the duration of the DATA step.

During the first iteration of the DATA step, the first observation in Hrd.Fitclass is read into the program data vector.

```
data hrd.convert;
 set hrd.fitclass;
 array wt{6} weight1-weight6;
 do i=1 to 6;
 wt{i}=wt{i}*2.2046;
 end;
run;
```

Program Data Vector

		wt{1}	wt{2}	wt{3}	wt{4}	wt{5}	wt{6}	
<b>N</b>	<b>Name</b>	<b>Weight1</b>	<b>Weight2</b>	<b>Weight3</b>	<b>Weight4</b>	<b>Weight5</b>	<b>Weight6</b>	<b>i</b>
1	Alicia	69.6	68.9	68.8	67.4	66.0	66.2	

Because the ARRAY statement is a compile-time only statement, it is ignored during execution. The DO statement is executed next.

During the first iteration of the DO loop, the index variable *i* is set to 1. As a result, the array reference *wt{*i*}* becomes *wt{1}*. Because *wt{1}* refers to the first array element, Weight1, the value of Weight1 is converted from kilograms to pounds.

```
data hrd.convert;
 set hrd.fitclass;
 array wt{6} weight1-weight6;
 do i=1 to 6;
 wt{1}=wt{1}*2.2046;
 end;
run;
```

Program Data Vector

		wt{1}	wt{2}	wt{3}	wt{4}	wt{5}	wt{6}	
<b>N</b>	<b>Name</b>	<b>Weight1</b>	<b>Weight2</b>	<b>Weight3</b>	<b>Weight4</b>	<b>Weight5</b>	<b>Weight6</b>	<b>i</b>
1	Alicia	153.4	68.9	68.8	67.4	66.0	66.2	1

As the DATA step continues its DO loop iterations, the index variable *i* is changed from 1 to 2, 3, 4, 5, and 6, causing Weight1 through Weight6 to receive new values in the program data vector.

```
data hrd.convert;
 set hrd.fitclass;
 array wt{6} weight1-weight6;
 do i=1 to 6;
 wt{i}=wt{i}*2.2046;
 end;
run;
```

### Using the DIM Function in an Iterative DO Statement

When using DO loops to process arrays, you can also use the DIM function to specify the TO clause of the iterative DO statement. For a one-dimensional array, specify the array name as the argument for the DIM function. The function returns the number of elements in the array.

---

Syntax, DIM function:

**DIM**(*array-name*)

*array-name* specifies the array.

---

In this example, `dim(wt)` returns a value of 6.

```
data hrd.convert;
 set hrd.fitclass;
 array wt{*} weight1-weight6;
```



```

do i=1 to dim(wt);
 wt{i}=wt{i}*2.2046;
end;
run;

```

When you use the DIM function, you do not have to re-specify the stop value of an iterative DO statement if you change the dimension of the array.

<pre> data hrd.convert;     set hrd.fitclass;     array wt{*} weight1-weight6;     do i=1 to dim(wt);         wt{i}=wt{i}*2.2046;     end; run; </pre>	<pre> data hrd.convert;     set hrd.fitclass;     array wt{*} weight1-weight10;     do i=1 to dim(wt);         wt{i}=wt{i}*2.2046;     end; run; </pre>
--------------------------------------------------------------------------------------------------------------------------------------------------------	---------------------------------------------------------------------------------------------------------------------------------------------------------

## Expanding Your Use of Arrays

### Creating Variables in an ARRAY Statement

Previous examples showed several ways to reference existing variables in an ARRAY statement. You can also create variables in an ARRAY statement by omitting the array elements from the statement. If you do not reference existing variables, SAS automatically creates new variables for you and assigns default names to them. The default name is the array-name followed by consecutive numbers 1 to the dimension of the array.

Syntax, ARRAY statement to create new variables:

**ARRAY** *array-name*{*dimension*};

- *array-name* specifies the name of the array.
- *dimension* describes the number and arrangement of array elements.

For example, suppose you need to calculate the weight gain or loss from week to week for each member of a fitness class, shown below.

**Figure 16.2** SAS Data Set Hrd.Convert

Name	weight1	weight2	weight3	weight4	weight5	weight6
Alicia	153.440	151.897	151.676	148.590	145.504	145.945
Betsy	115.962	115.962	113.978	111.112	109.789	108.246
Brenda	151.236	149.031	147.708	146.385	145.063	143.740
Carl	149.031	146.826	145.504	144.181	142.858	141.535
Carmela	140.213	137.788	136.465	135.362	134.040	128.308
David	155.645	153.881	152.558	150.795	149.472	147.708

You would like to create variables that contain this weekly difference. To perform the calculation, you first group the variables Weight1 through Weight6 into an array.

```
data hrd.diff;
 set hrd.convert;
 array wt{6} weight1-weight6;
```

Next, you want to create the new variables to store the differences between the six recorded weights. You can use an additional ARRAY statement without elements to create the new variables, WgtDiff1 to WgtDiff5.

```
data hrd.diff;
 set hrd.convert;
 array wt{6} weight1-weight6;
 array WgtDiff{5};
run;
proc print data=hrd.convert;
run;
```

Remember, when creating variables in an ARRAY statement, you do not need to specify array elements as long as you specify how many elements are in the array.

```
array WgtDiff{5};
```

### Default Variable Names

The default variable names are created by concatenating the array name and the numbers 1, 2, 3, and so on, up to the array dimension.

```
array WgtDiff{5};

WgtDiff1 WgtDiff2 WgtDiff3 WgtDiff4 WgtDiff5
```

**TIP** If you prefer, you can specify individual variable names. To specify variable names, you list each name as an element of the array. The following ARRAY statement creates the numeric variables Oct12, Oct19, Oct26, Nov02, and Nov09.

```
array WgtDiff{5} Oct12 Oct19 Oct26 Nov02 Nov09;
 array WgtDiff{5};

 Oct12 Oct19 Oct26 Nov02 Nov09
```

### Creating Arrays of Character Variables

To create an array of character variables, add a dollar sign (\$) after the array dimension.

```
array firstname{5} $;
```

By default, all character variables that are created in an ARRAY statement are assigned a length of 8. You can assign your own length by specifying the length after the dollar sign.

```
array firstname{5} $ 24;
```

The length that you specify is automatically assigned to all variables that are created by the ARRAY statement.

During the compilation of the DATA step, the variables that this ARRAY statement creates are added to the program data vector and are stored in the resulting data set.

```
data hrd.diff;
 set hrd.convert;
 array wt{6} Weight1-Weight6;
 array WgtDiff{5};
```

Program Data Vector

<b>_N_</b>	<b>Name</b>	<b>Weight1</b>	<b>Weight2</b>	<b>Weight3</b>	<b>Weight4</b>	<b>Weight5</b>	<b>Weight6</b>

<b>WgtDiff1</b>	<b>WgtDiff2</b>	<b>WgtDiff3</b>	<b>WgtDiff4</b>	<b>WgtDiff5</b>

*Note:* When referencing the array elements, be careful not to confuse the array references `WgtDiff{1}` through `WgtDiff{5}` (note the braces) with the variable names `WgtDiff1` through `WgtDiff5`. The program data vector below shows the relationship between the array references and the corresponding variable names.

Program Data Vector

<b>_N_</b>	<b>Name</b>	<b>Weight1</b>	<b>Weight2</b>	<b>Weight3</b>	<b>Weight4</b>	<b>Weight5</b>	<b>Weight6</b>

`WgtDiff{1}` `WgtDiff{2}` `WgtDiff{3}` `WgtDiff{4}` `WgtDiff{5}`

<b>WgtDiff1</b>	<b>WgtDiff2</b>	<b>WgtDiff3</b>	<b>WgtDiff4</b>	<b>WgtDiff5</b>

Now you can use a DO loop to calculate the differences between each of the recorded weights. Notice that each value of `WgtDiff{i}` is calculated by subtracting `wt{i}` from `wt{i+1}`. By manipulating the index variable, you can easily reference any array element.

```
data hrd.diff;
 set hrd.convert;
 array wt{6} weight1-weight6;
 array WgtDiff{5};
 do i=1 to 5;
 wgtdiff{i}=wt{i+1}-wt{i};
 end;
run;
proc print data=hrd.diff;
run;
```

A portion of the resulting data set is shown below.

**Figure 16.3** HTML Output: SAS Data Set Hrd.Diff (partial output)

Obs	Name	WgtDiff1	WgtDiff2	WgtDiff3	WgtDiff4	WgtDiff5	i
1	Alicia	-1.54322	-0.22046	-3.08644	-3.08644	0.44092	6
2	Betsy	0.00000	-1.98414	-2.86598	-1.32276	-1.54322	6
3	Brenda	-2.20460	-1.32276	-1.32276	-1.32276	-1.32276	6
4	Carl	-2.20460	-1.32276	-1.32276	-1.32276	-1.32276	6
5	Carmela	-2.42506	-1.32276	-1.10230	-1.32276	-5.73196	6
6	David	-1.76368	-1.32276	-1.76368	-1.32276	-1.76368	6

### Assigning Initial Values to Arrays

It is useful to assign initial values to elements of an array when you define the array.

```
array goal{4} g1 g2 g3 g4 (initial values);
```

To assign initial values in an ARRAY statement:

1. Place the values after the array elements.

```
array goal{4} g1 g2 g3 g4 (9000 9300 9600 9900);
```

2. Specify one initial value for each corresponding array element.

```

 ↓ ↓ ↓ ↓
array goal{4} g1 g2 g3 g4 (9000 9300 9600 9900);
 ↑ ↑ ↑ ↑

```

3. Separate each value with a comma or blank.

```

 ↓ ↓ ↓
array goal{4} g1 g2 g3 g4 (9000 9300 9600 9900);

```

4. Enclose the initial values in parentheses.

```

 ↓ ↓
array goal{4} g1 g2 g3 g4 (9000 9300 9600 9900);

```

5. Enclose each character value in quotation marks.

```

 ↓ ↓ ↓ ↓ ↓ ↓
array col{3} $ color1-color3 ('red', 'green', 'blue');

```

It is also possible to assign initial values to an array without specifying each array element. The following statement creates the variables Var1, Var2, Var3, and Var4, and assigns them initial values of 1, 2, 3, and 4:

```
array Var{4} (1 2 3 4);
```

For this example, assume that you have the task of comparing the actual sales figures in the Finance.Qsales data set to the sales goals for each sales representative at the beginning of the year. The sales goals are not recorded in Finance.Qsales.

**Figure 16.4** Description of Finance.Qsales

Alphabetic List of Variables and Attributes			
#	Variable	Type	Len
2	Sales1	Num	8
3	Sales2	Num	8
4	Sales3	Num	8
5	Sales4	Num	8
1	SalesRep	Char	8

The DATA step below reads the Finance.Qsales data set to create the Finance.Report data set. The ARRAY statement creates an array to process sales data for each quarter.

```
data finance.report;
 set finance.qsales;
 array sale{4} sales1-sales4;
```

To compare the actual sales to the sales goals, you must create the variables for the sales goals and assign values to them.

```
data finance.report;
 set finance.qsales;
 array sale{4} sales1-sales4;
 array Goal{4} (9000 9300 9600 9900);
```

A third ARRAY statement creates the variables Achieved1 through Achieved4 to store the comparison of actual sales versus sales goals.

```
data finance.report;
 set finance.qsales;
 array sale{4} sales1-sales4;
 array Goal{4} (9000 9300 9600 9900);
 array Achieved{4};
 do i=1 to 4;
 achieved{i}=100*sale{i}/goal{i};
 end;
run;
```

A DO loop executes four times to calculate the value of each element of the achieved array (expressed as a percentage).

```
data finance.report;
 set finance.qsales;
 array sale{4} sales1-sales4;
 array Goal{4} (9000 9300 9600 9900);
 array Achieved{4};
 do i=1 to 4;
 achieved{i}=100*sale{i}/goal{i};
 end;
run;
```

Before submitting this DATA step, you can drop the index variable from the new data set by adding a DROP= option to the DATA statement.

```
data finance.report(drop=i);
 set finance.qsales;
 array sale{4} sales1-sales4;
```

```

array Goal{4} (9000 9300 9600 9900);
array Achieved{4};
do i=1 to 4;
 achieved{i}=100*sale{i}/goal{i};
end;
run;
proc print data=finance.report;
run;

```

This is an example of a simple table-lookup program. The resulting data set contains the variables that were read from `Finance.Qsales`, plus the eight variables that were created with `ARRAY` statements.

**Figure 16.5** HTML Output: *SAS Finance.Report*

SalesRep	Sales1	Sales2	Sales3	Sales4	Goal1	Goal2
Britt	8400	8800	9300	9800	9000	9300
Fruchten	9500	9300	9800	8900	9000	9300
Goodyear	9150	9200	9650	11000	9000	9300

Goal3	Goal4	Achieved1	Achieved2	Achieved3	Achieved4
9600	9900	93.333	94.624	96.875	98.990
9600	9900	105.556	100.000	102.083	89.899
9600	9900	101.667	98.925	100.521	111.111

*Note:* Variables to which initial values are assigned in an `ARRAY` statement are automatically retained.

The variables `Goal1` through `Goal4` should not be stored in the data set because they are needed only to calculate the values of `Achieved1` through `Achieved4`.

### Creating Temporary Array Elements

To create temporary array elements for `DATA` step processing without creating new variables, specify `_TEMPORARY_` after the array name and dimension.

```

data finance.report(drop=i);
 set finance.qsales;
 array sale{4} sales1-sales4;
 array goal{4} _temporary_ (9000 9300 9600 9900);
 array Achieved{4};
 do i=1 to 4;
 achieved{i}=100*sale{i}/goal{i};
 end;
run;
proc print data=finance.report noobs;
run;

```

Temporary array elements do not appear in the resulting data set.

**Figure 16.6** SAS Data Set *Finance.Report*

SalesRep	Sales1	Sales2	Sales3	Sales4	Achieved1	Achieved2	Achieved3	Achieved4
Britt	8400	8800	9300	9800	93.333	94.624	96.875	98.990
Fruchten	9500	9300	9800	8900	105.556	100.000	102.083	89.899
Goodyear	9150	9200	9650	11000	101.667	98.925	100.521	111.111

**TIP** Temporary array elements are useful when the array is needed only to perform a calculation. Use them to improve performance time.

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- Which statement is false regarding an ARRAY statement?
  - It is an executable statement.
  - It can be used to create variables.
  - It must contain either all numeric or all character elements.
  - It must be used to define an array before the array name can be referenced.
- What belongs within the braces of this ARRAY statement?
 

```
array contrib{?} qtr1-qtr4;
```

  - quarter
  - quarter\*
  - 1-4
  - 4
- For the program below, select an iterative DO statement to process all elements in the contrib array.
 

```
data work.contrib;
 array contrib{4} qtr1-qtr4;
 ...
 contrib{i}=contrib{i}*1.25;
end;
run;
```

  - do i=4;
  - do i=1 to 4;
  - do until i=4;
  - do while i le 4;
- What is the value of the index variable that references Jul in the statements below?
 

```
array quarter{4} Jan Apr Jul Oct;
do i=1 to 4;
 yeargoal=quarter{i}*1.2;
```

end;

- a. 1
- b. 2
- c. 3
- d. 4

5. Which DO statement would not process all the elements in the factors array shown below?

```
array factors{*} age height weight bloodpr;
```

- a. do i=1 to dim(factors);
- b. do i=1 to dim(\*);
- c. do i=1,2,3,4;
- d. do i=1 to 4;

6. Which statement below is false regarding the use of arrays to create variables?

- a. The variables are added to the program data vector during the compilation of the DATA step.
- b. You do not need to specify the array elements in the ARRAY statement.
- c. By default, all character variables are assigned a length of eight.
- d. Only character variables can be created.

7. For the first observation, what is the value of diff{i} at the end of the second iteration of the DO loop?

Weight1	Weight2	Weight3
192	200	215
137	130	125
220	120	213

```
array wt{*} weight1-weight10;
array diff{9};
do i=1 to 9;
 diff{i}=wt{i+1}-wt{i};
end;
```

- a. 15
- b. 10
- c. 8
- d. -7

8. Finish the ARRAY statement below to create temporary array elements that have initial values of 9000, 9300, 9600, and 9900.

```
array goal{4} ... ;
```

- a. \_temporary\_ (9000 9300 9600 9900)
- b. temporary (9000 9300 9600 9900)
- c. \_temporary\_ 9000 9300 9600 9900



d. (temporary) 9000 9300 9600 9900



## Chapter 17

# Reading Raw Data in Fixed Fields

<b>Identifying the Type of Numeric Data</b> . . . . .	<b>401</b>
Overview . . . . .	401
Standard Numeric Data . . . . .	402
Nonstandard Numeric Data . . . . .	402
<b>Using Formatted Input</b> . . . . .	<b>403</b>
INPUT Statement Syntax . . . . .	403
Using the @n Column Pointer Control . . . . .	403
Examples: Reading Columns in Any Order . . . . .	404
The +n Pointer Control . . . . .	405
Example: How the +n Pointer Control Works . . . . .	405
<b>Using Informats</b> . . . . .	<b>406</b>
The Basics of Using Informats . . . . .	406
Reading Character Values with the \$w. Informat . . . . .	407
Reading Standard Numeric Data with the w.d Informat . . . . .	407
Reading Nonstandard Numeric Data with the COMMA w.d Informat . . . . .	408
DATA Step Processing of Informats . . . . .	409
<b>Using Record Formats</b> . . . . .	<b>410</b>
Fixed-Length Records . . . . .	410
Variable-Length Records . . . . .	410
Reading Variable-Length Records . . . . .	411
The PAD Option . . . . .	412
The TRUNCOVER Option . . . . .	412
<b>Chapter Quiz</b> . . . . .	<b>413</b>

## Identifying the Type of Numeric Data

### Overview

Raw data can be organized in several ways. External files can contain data that is arranged in columns or fixed fields. It is possible to specify a beginning and ending column for each field.

However, the following file contains nonstandard data, because one of the variable's values includes a special character, the dollar sign (\$).

**Figure 17.1** Raw Data File (Nonstandard)

```

1---+----10---+----20---+----
BIRD FEEDER LG088 3 $29.95
GLASS MUGS SB082 6 $25.00
GLASS TRAY BQ049 12 $39.95
PADDED HANGRS MN256 15 $9.95
JEWELRY BOX AJ498 23 $45.00
RED APRON AQ072 9 $6.50
CRYSTAL VASE AQ672 27 $29.95
PICNIC BASKET Ls930 21 $15.00

```

The following external file contains no special characters, but its data is free-format, meaning that it is not arranged in columns. Notice that the values for a particular field do not begin and end in the same column.

**Figure 17.2** Raw Data File (Free-Format)

```

1---+----10---+----20---+--
BARNES NORTHWEST 36098.45
FARLSON SOUTHWEST 24394.09
LAWRENCE NORTHEAST 19504.26
NELSON SOUTHEAST 16930.84
STEWART MIDWEST 23845.13
TAYLOR MIDWEST 12354.42
TREADWAY SOUTHWEST 41092.84
WALSTON SOUTHEAST 28938.71

```

How your data is organized and what type of data you have determine which input style you should use to read the data. SAS provides you three primary input styles: column input, formatted input, and list input. Recall from [Chapter 6, “Creating SAS Data Sets from External Files,”](#) that column input can be used to read raw data that is stored in an external file. This chapter covers how to use column input and formatted input to read standard and nonstandard data arranged in fixed fields.

### Standard Numeric Data

Standard numeric data values can contain only the following:

- numbers
- decimal points
- numbers in scientific, or E, notation (23E4)
- minus signs and plus signs

Some examples of standard numeric data are 15, -15, 15.4, +.05, 1.54E3, and -1.54E-3.

### Nonstandard Numeric Data

Here are some types of nonstandard numeric data:

- values that contain special characters, such as percent signs (%), dollar signs (\$), and commas (,)
- date and time values
- data in fraction, integer binary, real binary, and hexadecimal forms

---

## Using Formatted Input

### **INPUT Statement Syntax**

*Formatted input* is a powerful method for reading both standard and nonstandard data in fixed fields.

---

Syntax, INPUT statement using formatted input:

**INPUT** <pointer-control> variable informat.;

- *pointer-control* positions the input pointer on a specified column.
  - *variable* is the name of the variable.
  - *informat.* is the special instruction that specifies how SAS reads the raw data.
- 

### **Using the @n Column Pointer Control**

The @*n* is a pointer control that moves the input pointer to a specific column number. The @ moves the pointer to column *n*, which is the first column of the field that is being read.

---

Syntax, INPUT statement using formatted input and the @*n* pointer control:

**INPUT** @*n* variable informat.;

- *variable* is the name of the variable.
  - *informat.* is a special instruction that specifies how SAS reads the raw data.
- 

In this example, you use the @*n* pointer control to locate variable values in the external file Empdata. As you can see, the values for LastName begin in column 1. The example could start with the @1 pointer control.

```
input @1 LastName $7.
```

However, the default column pointer location is column 1, so no column pointer control is necessary to read the first field.

```
input LastName $7.
```

V		10	20	
EVANS	DONNY	112	29,996.63	
HELMS	LISA	105	18,567.23	
HIGGINS	JOHN	111	25,309.00	
LARSON	AMY	113	32,696.78	
MOORE	MARY	112	28,945.89	
POWELL	JASON	103	35,099.50	
RILEY	JUDY	111	25,309.00	

The values for FirstName begin in column 9. To point to column 9, use an @ sign and the column number in the INPUT statement:

```
input LastName $7. @9 FirstName $5.
```

1	V	10	20	
EVANS	DONNY	112	29,996.63	
HELMS	LISA	105	18,567.23	
HIGGINS	JOHN	111	25,309.00	
LARSON	AMY	113	32,696.78	
MOORE	MARY	112	28,945.89	
POWELL	JASON	103	35,099.50	
RILEY	JUDY	111	25,309.00	

### Examples: Reading Columns in Any Order

*Column pointer controls* can also be useful. For example, you can use the @n to move a pointer forward or backward when reading a record.

In this INPUT statement, the value for FirstName is read first, starting in column 9.

```
input @9 FirstName $5.
```

1	V	10	20	
EVANS	DONNY	112	29,996.63	
HELMS	LISA	105	18,567.23	
HIGGINS	JOHN	111	25,309.00	

Read the values for LastName, which begin in the first column. Here you must use the @n pointer control to move the pointer back to column 1.

```
input @9 FirstName $5. @1 LastName $7.
```

V	10	20	
EVANS	DONNY	112	29,996.63
HELMS	LISA	105	18,567.23
HIGGINS	JOHN	111	25,309.00

The rest of the INPUT statement specifies the column locations of the raw data value for JobTitle and Salary.

```
input @9 FirstName $5. @1 LastName $7. @15 JobTitle 3.
 @19 Salary comma9.;
```

1---+---10---v---v20---+---				
EVANS	DONNY	112	29,996.63	
HELMS	LISA	105	18,567.23	
HIGGINS	JOHN	111	25,309.00	

### The +n Pointer Control

The +n pointer control moves the input pointer forward to a column number that is relative to the current position. The + moves the pointer forward *n* columns.

---

Syntax, INPUT statement using formatted input and the +n pointer control:

**INPUT** +n variable informat.;

- *variable* is the name of the variable.
  - *informat.* is a special instruction that specifies how SAS reads the raw data.
- 

In order to count correctly, it is important to understand where the column pointer is located after each data value is read.

### Example: How the +n Pointer Control Works

Suppose you want to read the data from Empdata in the following order: LastName, FirstName, Salary, JobTitle. Because the values for LastName begin in column 1, a column pointer control is not needed.

```
input LastName $7.
```

v---+---10---+---20---+---				
EVANS	DONNY	112	29,996.63	
HELMS	LISA	105	18,567.23	
HIGGINS	JOHN	111	25,309.00	

With formatted input, the column pointer moves to the first column following the field that was just read. In this example, after LastName is read, the pointer moves to column 8.

1---+---v-10---+---20---+---				
EVANS	DONNY	112	29,996.63	
HELMS	LISA	105	18,567.23	
HIGGINS	JOHN	111	25,309.00	

To start reading FirstName, which begins in column 9, the column pointer moves ahead one column with +1.

```
input LastName $7. +1 FirstName $5.
```

1---+---v10---+---20---+---				
EVANS	DONNY	112	29,996.63	
HELMS	LISA	105	18,567.23	
HIGGINS	JOHN	111	25,309.00	

After reading `FirstName`, the column pointer moves to column 14. To skip over the values for `JobTitle` and read the values for `Salary`, which begin in column 19, move the column pointer ahead five columns from column 14.

```
input LastName $7. +1 FirstName $5. +5 Salary comma9.
```

				12345	
1	---	+	----	10	---V---V20---+---
EVANS		DONNY	112	29,996.63	
HELMS		LISA	105	18,567.23	
HIGGINS		JOHN	111	25,309.00	

The last field to be read contains the values for `JobTitle`. You can use the `@n` column pointer control to return to column 15.

```
input LastName $7. +1 FirstName $5. +5 Salary comma9.
 @15 JobTitle 3.;
```

1	---	+	----	10	---V---20---+---
EVANS		DONNY	112	29,996.63	
HELMS		LISA	105	18,567.23	
HIGGINS		JOHN	111	25,309.00	

*Note:* You can use the notation `+(-n)` to move the `+n` pointer backward.

## Using Informats

### The Basics of Using Informats

An *informat* is an instruction that tells SAS how to read raw data. SAS provides many informats for reading standard and nonstandard data values. Here is a small sample.

**Table 17.1** Selected Informats for Reading Data

PERCENT $w.d$	DATE $w.$	NENGOW $w.$
\$BINARY $w.$	DATETIME $w.$	PD $w.d$
\$VARYING $w.$	HEX $w.$	PERCENT $w.$
\$ $w.$	JULIAN $w.$	TIME $w.$
COMMA $w.d$	MMDDYY $w.$	$w.d$

Here are some facts about informats:

- Each informat contains a  $w$  value to indicate the width of the raw data field.
- Each informat also contains a period, which is a required delimiter.



- For some informats, the optional *d* value specifies the number of implied decimal places.
- Informats for reading character data always begin with a dollar sign (\$).

### Reading Character Values with the \$w. Informat

The \$w. informat enables you to read character data. The *w* represents the field width of the data value (the total number of columns that contain the raw data field).

In the example below, the \$ indicates that FirstName is a character variable, the 5 indicates a field width of five columns, and a period ends the informat.

```
input @9 FirstName $5.;
```

	12345			
1---+---v10---+---20---+---				
EVANS	DONNY	112	29,996.63	
HELMS	LISA	105	18,567.23	
HIGGINS	JOHN	111	25,309.00	
LARSON	AMY	113	32,696.78	
MOORE	MARY	112	28,945.89	
POWELL	JASON	103	35,099.50	
RILEY	JUDY	111	25,309.00	
RYAN	NEAL	112	28,180.00	
WILSON	HENRY	113	31,875.46	
WOODS	CHIP	105	17,098.71	

### Reading Standard Numeric Data with the w.d Informat

The informat for reading standard numeric data is the *w.d* informat.

The *w* specifies the field width of the raw data value, the period serves as a delimiter, and the *d* specifies the number of implied decimal places for the value. The *w.d* informat ignores any specified *d* value if the data already contains a decimal point.

For example, the raw data value shown below contains six digits (four are decimals) and one decimal point. Therefore, the *w.* informat requires a field width of only 7 to correctly read the raw data value.

Raw Data Value		w. Informat		Variable Value
34.0008	►	7.	►	34.0008

In the example shown below, the values for JobTitle in columns 15-17 contain only numbers. Remember that standard numeric data values can contain only numbers, decimal points, scientific notation, and plus and minus signs.

A *d* value is not necessary to read the values for JobTitle. Simply move the column pointer forward seven spaces to column 15, name the variable, and specify a field width of 3.

```
input @9 FirstName $5. @1 LastName $7. +7 JobTitle 3.;
```

123

1	---	+	---	10	---	V	---	20	---	+	---
EVANS		DONNY		112				29,996.63			
HELMS		LISA		105				18,567.23			
HIGGINS		JOHN		111				25,309.00			
LARSON		AMY		113				32,696.78			
MOORE		MARY		112				28,945.89			
POWELL		JASON		103				35,099.50			
RILEY		JUDY		111				25,309.00			

*Note:* Be certain to specify the period in the informat name.

### Reading Nonstandard Numeric Data with the **COMMA w.d** Informat

The **COMMA w.d** informat is used to read numeric values and to remove embedded items such as these:

- blanks
- commas
- hyphens
- dollar signs
- percent signs
- close parentheses
- open parentheses, which are interpreted as minus signs

The **COMMA w.d** informat has three parts:

1.	the informat name	COMMA
2.	a value that specifies the width of the field to be read (including dollar signs, decimal places, or other special characters), followed by a period	w.
3.	an optional value that specifies the number of implied decimal places for a value (not necessary if the value already contains decimal places)	d

In the example below, the values for Salary contain commas, which means that they are nonstandard numeric values.

The values for Salary begin in column 19, so the **@n** or **+n** pointer control is used to point to column 19, and the variable is then named.

The following code adds the **COMMA w.d** informat and specifies the field width. The values end in column 27, so the field width is nine columns.

```
data sasuser.empinfo;
 infile empdata;
 input @9 FirstName $5. @1 LastName $7. +7 JobTitle 3.
 @19 Salary comma9.;
run;
```

123456789

1	----	10	----	V20	----	+	V-	----
EVANS	DONNY	112		29,996.63				
HELMS	LISA	105		18,567.23				
HIGGINS	JOHN	111		25,309.00				
LARSON	AMY	113		32,696.78				
MOORE	MARY	112		28,945.89				
POWELL	JASON	103		35,099.50				
RILEY	JUDY	111		25,309.00				

If you use PROC PRINT to display the data set, the commas are removed from the values for Salary in the resulting output.

```
data sasuser.empinfo;
 infile empdata;
 input @9 FirstName $5. @1 LastName $7. +7 JobTitle 3.
 @19 Salary comma9.;
run;
proc print data=sasuser.empinfo;
run;
```

**Figure 17.3** Output from the PRINT Procedure

FirstName	LastName	JobTitle	Salary
DONNY	EVANS	112	29996.63
LISA	HELMS	105	18567.23
JOHN	HIGGINS	111	25309.00
AMY	LARSON	113	32696.78
MARY	MOORE	112	28945.89
JASON	POWELL	103	35099.50
JUDY	RILEY	111	25309.00

Thus, the COMMAw.d informat does more than simply read the raw data values. It removes special characters such as commas from numeric data and stores only numeric values in a SAS data set.

## DATA Step Processing of Informats

Remember that after the DATA step is submitted, it is compiled and then executed.

```
data sasuser.empinfo;
 infile empdata;
 input @9 FirstName $5. @1 LastName $7. +7 JobTitle 3.
 @19 Salary comma9.;
run;
```

During the compilation phase, the character variables in the PDV are defined with the exact length specified by the informat. But notice that the lengths that are defined for JobTitle and Salary in the PDV are different from the lengths that are specified by their informats.

		123	123456789
1	---+---	10	---V-V-V20---+V-
EVANS	DONNY	112	29,996.63
HELMS	LISA	105	18,567.23
HIGGINS	JOHN	111	25,309.00

Program Data Vector

<u>N</u>	<u>ERROR</u>	<u>FirstName</u>	<u>LastName</u>	<u>JobTitle</u>	<u>Salary</u>
		\$5.	\$7.	8.	8.
		DONNY	EVANS	112	29996.6

By default, SAS stores numeric values (no matter how many digits the value contains) as floating-point numbers in 8 bytes of storage. The length of a stored numeric variable is not affected by an informat's width nor by other column specifications in an INPUT statement.

## Using Record Formats

The record format of an external file might affect how data is read with column input and formatted input. A *record format* specifies how records are organized in a file. Two common record formats are fixed-length records and variable-length records.

### ***Fixed-Length Records***

External files that have a fixed-length record format have an end-of-record marker after a predetermined number of columns. A fixed-length format ends at the same ending point (for example, at 80).

123456789.....80	< field1 > field2 > field3 >..unused space	record 1
123456789.....80	< field1 > field2 > field3 > field4 >....	record 2
123456789.....80	< field1 > field2 >.... unused space .....	record 3

## Variable-Length Records

Files that have a variable-length record format have an end-of-record marker after the last field in each record.

As you can see, the length of each record varies.

123456789.....*	
< field1 > < field2 > < field3 >	record 1
123456789.....*	
< field1 > < field2 > < field3 > < field4 >	record 2
123456789.....*	
< field1 > < field2 >	record 3

### Reading Variable-Length Records

When you work with variable-length records that contain fixed-field data, you might have values that are shorter than others or that are missing. This can cause problems when you try to read the raw data into your SAS data set.

For example, notice that the following INPUT statement specifies a field width of 8 columns for Receipts. In the third record, the input pointer encounters an end-of-record marker before the eighth column.

```
input Dept $ 1-11 @13 Receipts comma8.;
```

*Note:* The asterisk symbolizes the end-of-record marker and is not part of the data.

	12345678
1---+-----10---+---V20--	
BED/BATH	1,354.93*
HOUSEWARES	2,464.05*
GARDEN	923.34*
GRILL	598.34*
SHOES	1,345.82*
SPORTS*	
TOYS	6,536.53*

The input pointer moves down to the next record in an attempt to complete the value for Receipts. However, **GRILL** is a character value, and Receipts is a numeric variable. Thus, an invalid data error occurs, and the value for Receipts is set to missing.

V---+-----10---+-----20--	
BED/BATH	1,354.93*
HOUSEWARES	2,464.05*
GARDEN	923.34*
<b>GRILL</b>	598.34*
SHOES	1,345.82*
SPORTS*	
TOYS	6,536.53*

### The PAD Option

When reading variable-length records that contain fixed-field data, you can avoid problems by using the PAD option in the INFILE statement. The PAD option pads each record with blanks so that all data lines have the same length.

```
infile receipts pad;
```

1---+-----10---+-----20---	
BED/BATH	1,354.93
HOUSEWARES	2,464.05
GARDEN	923.34
GRILL	598.34
SHOES	1,345.82
SPORTS	
TOYS	6,536.53

When you use column input or formatted input to read fixed-field data in variable-length records, remember to determine whether you need to use the PAD option.

*Note:* Use the PAD option only when missing data occurs at the end of a record.

The default value of the maximum record length is determined by your operating system. If you encounter unexpected results when reading many variables, you might need to change the maximum record length by specifying the LRECL= option in the INFILE statement.

### The TRUNCOVER Option

Use the TRUNCOVER option when you are reading data using column or formatted input. If a variable's field extends past the end of the data line, then, by default, SAS goes to the next line to start reading the variable's value. This option tells SAS to read data for the variable until it reaches the end of the data line, or until it reaches the last column that is specified in the format or column range, whichever comes first. The next file contains addresses and must be read using column or formatted input because the street names have embedded blanks. Note that the data lines are all different lengths:

```
John Garcia 114 Maple Ave.
Sylvia Chung 1302 Washington Drive
Martha Newton 45 S.E. 14th St.
```

The program uses column input to read the address file. Because some of the addresses stop before the end of the variable Street's field (columns 22 through 37), the TRUNCOVER option is necessary. Without the TRUNCOVER option, SAS would try to go to the next line to read the data for Street on the first and third records.

```
DATA homeaddress;
 INFILE 'c:\MyRawData\Address.dat' TRUNCOVER;
 INPUT Name $1-15 Number 16-19 Street $22-37;
run;
```

TRUNCOVER is similar to MISCOVER. Both assign missing values to variables if the data line ends before the variable's field starts. But when the data line ends in the middle of a variable field, TRUNCOVER covers the first 80 characters of the line, whereas MISCOVER assigns the variable a missing value.

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- Which SAS statement correctly uses column input to read the values in the raw data file below in this order: Address (fourth field), SquareFeet (second field), Style (first field), Bedrooms (third field)?

1	---	+	---	10	---	+	---	20	---	+	---	30
2	STORY	1810	4	SHEPPARD AVENUE								
CONDO	1200	2	RAND STREET									
RANCH	1550	3	MARKET STREET									

- input Address 15-29 SquareFeet 8-11 Style 1-6  
Bedrooms 13;
  - input \$ 15-29 Address 8-11 SquareFeet \$ 1-6 Style  
13 Bedrooms;
  - input Address \$ 15-29 SquareFeet 8-11 Style \$ 1-6  
Bedrooms 13;
  - input Address 15-29 \$ SquareFeet 8-11 Style 1-6  
\$ Bedrooms 13;
- Which is not an advantage of column input?
    - It can be used to read character variables that contain embedded blanks.
    - No placeholder is required for missing data.
    - Standard as well as nonstandard data values can be read.
    - Fields do not have to be separated by blanks or other delimiters.
  - Which is an example of standard numeric data?
    - 34.245
    - \$24,234.25
    - 1/2
    - 50%
  - Formatted input can be used to read which of the following?
    - standard free-format data
    - standard data in fixed fields
    - nonstandard data in fixed fields
    - both standard and nonstandard data in fixed fields
  - Which informat should you use to read the values in columns 1-5?

```

1---+---10---+---20---+---30
2STORY 1810 4 SHEPPARD AVENUE
CONDO 1200 2 RAND STREET
RANCH 1550 3 MARKET STREET

```

- a. *w*.
  - b. *\$w*.
  - c. *w.d*
  - d. *COMMAw.d*
6. The *COMMAw.d* informat can be used to read which of the following values?
    - a. 12,805
    - b. \$177.95
    - c. 18%
    - d. all of the above
  7. Which INPUT statement correctly reads the values for ModelNumber (first field) after the values for Item (second field)? Both Item and ModelNumber are character variables.

```

1---+---10---+---20---+---30
DG345 CD PLAYER $174.99
HJ756 VCR $298.99
AS658 CAMCORDER $1,195.99

```

- a. input +7 Item \$9. @1 ModelNumber \$5.;
  - b. input +6 Item \$9. @1 ModelNumber \$5.;
  - c. input @7 Item \$9. +1 ModelNumber \$5.;
  - d. input @7 Item \$9 @1 ModelNumber 5.;
8. Which INPUT statement correctly reads the numeric values for Cost (third field)?

```

1---+---10---+---20---+---30
DG345 CD PLAYER $174.99
HJ756 VCR $298.99
AS658 CAMCORDER $1,195.99

```

- a. input @17 Cost 7.2;
  - b. input @17 Cost 9.2.;
  - c. input @17 Cost comma7.;
  - d. input @17 Cost comma9.;
9. Which SAS statement correctly uses formatted input to read the values in this order: Item (first field), UnitCost (second field), Quantity (third field)?



1---+---10---+---20---+
ENVELOPE \$13.25 500
DISKETTES \$29.50 10
BANDS \$2.50 600
RIBBON \$94.20 12

- input @1 Item \$9. +1 UnitCost comma6.  
@18 Quantity 3.;
  - input Item \$9. @11 UnitCost comma6.  
@18 Quantity 3.;
  - input Item \$9. +1 UnitCost comma6.  
@18 Quantity 3.;
  - all of the above
10. Which raw data file requires the PAD option in the INFILE statement in order to correctly read the data using either column input or formatted input?

a.

1---+---10---+---20---+
JONES M 48 128.6
LAVERNE M 58 158
JAFFE F 33 115.5
WILSON M 28 130

b.

1---+---10---+---20---+
JONES M 48 128.6
LAVERNE M 58 158.0
JAFFE F 33 115.5
WILSON M 28 130.0

c.

1---+---10---+---20---+
JONES M 48 128.6
LAVERNE M 58 158
JAFFE F 33 115.5
WILSON M 28 130

d.

1---+---10---+---20---+
JONES M 48 128.6
LAVERNE M 58 158.0
JAFFE F 33 115.5
WILSON M 28 130.0



## Chapter 18

# Reading and Creating Raw Data in Free-Format

---

<b>Free-Format Data</b> .....	<b>417</b>
<b>Reading Standard Data with List Input</b> .....	<b>418</b>
The Basics of List Input .....	418
Processing List Input .....	419
Reading a Range of Variables .....	420
<b>Specifying the Length of Character Variables</b> .....	<b>421</b>
The Basics of Specifying Length .....	421
The LENGTH Statement .....	423
<b>Reading Free-Format Data with Non-Blank Delimiters and Missing Values</b> . . .	<b>424</b>
Working with Delimiters .....	424
Example: DLM = Option .....	424
Reading Values That Contain Delimiters within a Quoted String .....	426
Reading Missing Values at the End of a Record .....	427
Reading Missing Values at the Beginning or Middle of a Record .....	428
<b>Reading Nonstandard Data with List Input</b> .....	<b>431</b>
The Basics of Modifying List Input .....	431
Reading Values That Contain Embedded Blanks .....	431
Using the & Modifier with a LENGTH Statement .....	432
Using the & Modifier with an Informat .....	432
Reading Nonstandard Values .....	433
Processing the DATA Step .....	433
Comparing Formatted Input and Modified List Input .....	435
<b>Creating Free-Format Data</b> .....	<b>436</b>
The Basics of Creating Free-Format Data .....	436
Specifying a Delimiter .....	437
Using the DSD Option .....	438
<b>Write a Comma-Delimited File Using Formats</b> .....	<b>439</b>
<b>Chapter Quiz</b> .....	<b>441</b>

---

## Free-Format Data

*Free-format data* is data that is not arranged in fixed fields. The fields are often separated by blanks or other delimiters. Column input and formatted input that you might have used before to read standard and nonstandard data in fixed fields does not

enable you to read all of the values in the raw data file. However, list input enables you to read both formatted and free-format data.

**Figure 18.1** Raw Data in Free-Format That Is Separated by Blanks

```
1---+----10---+----20---+----30
ABRAMS L. MARKETING $8,209
BARCLAY M. MARKETING $8,435
COURTNEY W. MARKETING $9,006
FARLEY J. PUBLICATIONS $8,305
HEINS W. PUBLICATIONS $9,539
```

## Reading Standard Data with List Input

### The Basics of List Input

Syntax, INPUT statement using list input:

**INPUT** *variable* <\$>;

- *variable* specifies the variable whose value the INPUT statement is to read.
- \$ specifies that the variable is a character variable.

Suppose you have an external data file like the one shown below. The file, which is referenced by the fileref *Creditc*, contains the results of a survey on the use of credit cards by males and females in the 18-39 age range.

**Figure 18.2** Raw Data File *Creditc*

Raw Data File

```
1---+----10---+----20
MALE 27 1 8 0 0
FEMALE 29 3 14 5 10
FEMALE 34 2 10 3 3
MALE 35 2 12 4 8
FEMALE 36 4 16 3 7
MALE 21 1 5 0 0
MALE 25 2 9 2 1
FEMALE 21 1 4 2 6
MALE 38 3 11 4 3
FEMALE 30 3 5 1 0
```

You need to read the data values for the following:

- gender
- age
- number of bank credit cards

- bank card use per month
- number of department store credit cards
- department store card use per month

List input might be the easiest input style to use because, as shown in the INPUT statement below, you simply list the variable names in the same order as the corresponding raw data fields. Remember to distinguish character variables from numeric variables with dollar signs.

```
input Gender $ Age Bankcard FreqBank Deptcard FreqDept;
```

By default this input style does not specify column locations. Therefore, in order to read data using list input, the following must be true:

- All fields must be separated by at least one blank or other delimiter.
- Fields must be read in order from left to right.
- Fields cannot be skipped or re-read.

*Note:* Blank is the default delimiter in list input.

## Processing List Input

It is important to remember that list input causes SAS to scan the input lines for values rather than reading from specific columns. When the INPUT statement is submitted for processing, the input pointer is positioned at column 1 of the raw data file.

```
data sasuser.creditsurvey;
 infile creditc;
 input Gender $ Age Bankcard FreqBank Deptcard
 FreqDept;
run;
```

SAS reads the first field until it encounters a blank space. The blank space indicates the end of the field, and the data value is assigned to the program data vector for the first variable in the INPUT statement.

Next, SAS scans the record until the next nonblank space is found, and the second value is read until another blank is encountered. Then the value is assigned to its corresponding variable in the program data vector.

This process of scanning ahead to the next nonblank column, reading the data value until a blank is encountered, and assigning the value to a variable in the program data vector continues until all of the fields have been read and values have been assigned to variables in the program data vector.

Program Data Vector

<u>N</u>	Gender	Age	Bankcard	FreqBank	Deptcard	FreqDept
1	MALE	27	1	8	0	0

When the DATA step has finished executing, you can display the data set with the PRINT procedure. The following code produces the output below.

```
proc print data=sasuser.creditsurvey;
run;
```

**Figure 18.3** Data Set Displayed by PROC PRINT

Gender	Age	Bankcard	FreqBank	Deptcard	FreqDept
MALE	27	1	8	0	0
FEMALE	29	3	14	5	10
FEMALE	34	2	10	3	3
MALE	35	2	12	4	8
FEMALE	36	4	16	3	7
MALE	21	1	5	0	0
MALE	25	2	9	2	1

### Reading a Range of Variables

When the variable values in the raw data file are sequential and are separated by a blank (or by another delimiter), you can specify a range of variables in the INPUT statement. This is especially useful if your data contains similar variables, such as the answers to a questionnaire.

For example, the following INPUT statement creates five new numeric variables and assigns them the names Ques1, Ques2, Ques3, and so on. You can also specify a range in the VAR statement with the PROC PRINT step to list a range of specific variables.

```
data sasuser.phonesurvey;
 infile phonsurv;
 input IDnum $ Ques1-Ques5;
run;
proc print data=sasuser.phonesurvey;
 var ques1-ques3;
run;
```

**Figure 18.4** Raw Data File with Sequential Variables

Raw Data File Survey						
1----	10----	20				
1000	23 94 56 85 99					
1001	26 55 49 87 85					
1002	33 99 54 82 94					
1003	71 33 22 44 92					
1004	88 49 29 57 83					

**Figure 18.5** Output Using Sequential Variable Names

Ques1	Ques2	Ques3
23	94	56
26	55	49
33	99	54
71	33	22
88	49	29

If you specify a range of character variables, both the variable list and the \$ sign must be enclosed in parentheses.

```
data sasuser.stores;
 infile stordata;
 input Age (Store1-Store3) ($);
run;
proc print data=sasuser.stores;
run;
```

*Note:* You can also specify a range of variables using formatted input. If you specify a range of variables using formatted input, both the variable list and the informat must be enclosed in parentheses, regardless of the variable's type.

```
data sasuser.scores;
 infile group3;
 input Age (Score1-Score4) (6.);
run;
```

---

## Specifying the Length of Character Variables

### *The Basics of Specifying Length*

Remember that when you use list input to read raw data, character variables are assigned a default length of 8. In this example, see what happens when list input is used to read character variables whose values are longer than 8.

The raw data file referenced by the fileref Citydata contains 2000 and 2010 population figures for several large U.S. cities. Notice that some city names are rather long.

**Figure 18.6** Raw Data File with Character Values That Are Longer Than 8

```

1---+----10---+----20---+----
ANCHORAGE 48081 174431
ATLANTA 495039 425022
BOSTON 641071 562994
CHARLOTTE 241420 314447
CHICAGO 3369357 3005072
DALLAS 844401 904078
DENVER 514678 492365
DETROIT 1514063 1203339
MIAMI 334859 346865
PHILADELPHIA 1949996 1688210
SACRAMENTO 257105 275741

```

The longer character values are truncated when they are read into the program data vector.

Program Data Vector

<u>N</u>	Rank	City	Pop70	Pop80
1	1	ANCHORAG	48081	174431

PROC PRINT output shows the truncated values for City.

```

data sasuser.growth;
 infile citydata;
 input City $ Pop70 Pop80;
run;
proc print data=sasuser.growth;
run;

```

**Figure 18.7** Output with Truncated Values

City	Pop2000	Pop2010
ANCHORAG	260586	293405
ATLANTA	421323	422919
BOSTON	590433	620701
CHARLOTT	569858	738561
CHICAGO	2896000	2698000
DALLAS	1191000	1201000
DENVER	556094	603300
DETROIT	945297	711088
MIAMI	363177	400949
PHILADEL	1514000	1528000
SACRAMEN	409230	467597



### The LENGTH Statement

Remember, variable attributes are defined when the variable is first encountered in the DATA step. In the program below, the LENGTH statement precedes the INPUT statement and defines both the length and type of the variable City. A length of 12 has been assigned to accommodate **PHILADELPHIA**, which is the longest value for City.

```
data sasuser.growth;
 infile citydata;
 length City $ 12;
 input city $ Pop70 Pop80;
run;
proc print data=sasuser.growth;
run;
```

**Figure 18.8** Raw Data File with Character Values That Are Longer Than 8

```
1---+---10---+---20---+---
ANCHORAGE 260586 293405
ATLANTA 421323 422919
BOSTON 590433 620701
CHARLOTTE 569858 738561
CHICAGO 26896000 2698000
DALLAS 1191000 1201000
DENVER 556094 603300
DETROIT 945297 711088
MIAMI 363177 400949
PHILADELPHIA 1514000 1528000
SACRAMENTO 409230 467597
```

Using this method, you do not need to specify City's type in the INPUT statement. However, leaving the \$ in the INPUT statement does not produce an error. Your output should now display the complete values for City.

**Figure 18.9** Output Using Length Statement

City	Pop2000	Pop2010
ANCHORAGE	260586	293405
ATLANTA	421323	422919
BOSTON	590433	620701
CHARLOTTE	569858	738561
CHICAGO	2896000	2698000
DALLAS	1191000	1201000
DENVER	556094	603300
DETROIT	945297	711088
MIAMI	363177	400949
PHILADELPHIA	1514000	1528000
SACRAMENTO	409230	467597

*Note:* Variable attributes are defined when the variable is first encountered in the DATA step, a variable that is defined in a LENGTH statement (if it precedes an INPUT statement) appears first in the data set, regardless of the order of the variables in the INPUT statement.

## Reading Free-Format Data with Non-Blank Delimiters and Missing Values

### Working with Delimiters

Most free-format data fields are clearly separated by blanks and are easy to imagine as variables and observations. But fields can also be separated by other delimiters, such as commas, as shown below.

**Figure 18.10** Raw Data File with Comma Delimiters

```

Raw Data File Credit
1---+-----10---+-----20
MALE,27,1,8,0,0
FEMALE,29,3,14,5,10
FEMALE,34,2,10,3,3
MALE,35,2,12,4,8
FEMALE,36,4,16,3,7
MALE,21,1,5,0,0
MALE,25,2,9,2,1
FEMALE,21,1,4,2,6
MALE,38,3,11,4,3
FEMALE,30,3,5,1,0

```

When characters other than blanks are used to separate the data values, you can tell SAS which field delimiter to use. Use the DLM= option in the INFILE statement to specify a delimiter other than a blank (the default).

Syntax, DLM= option:

**DLM=delimiter(s)**

*delimiter(s)* specifies a delimiter for list input in either of the following forms:

- *'list-of-delimiting-characters'* specifies one or more characters (up to 200) to read as delimiters. The list of characters must be enclosed in quotation marks.
- *character-variable* specifies a character variable whose value becomes the delimiter.
- DLM is an alias for the DELIMITER= option.

### Example: DLM = Option

The following program creates the output shown below.

```

data sasuser.creditsurvey;
 infile cccomma dlm=',';

```

```

input Gender $ Age Bankcard FreqBank
 Deptcard FreqDept;
run;
proc print data=sasuser.creditsurvey;
run;

```

**Figure 18.11** Output from the DLM= Option

Gender	Age	Bankcard	FreqBank	Deptcard	FreqDept
MALE	27	1	8	0	0
FEMALE	29	3	14	5	10
FEMALE	34	2	10	3	3
MALE	35	2	12	4	8
FEMALE	36	4	16	3	7
MALE	21	1	5	0	0
MALE	25	2	9	2	1
FEMALE	21	1	4	2	6
MALE	38	3	11	4	3
FEMALE	30	3	5	1	0

*Note:* The field delimiter must not be a character that occurs in a data value. For example, the following raw data file contains values for LastName and Salary. Notice that the values for Salary contain commas.

**Figure 18.12** Raw Data File with Commas in Values

```

1---+-----10---+-----20
BROWN 24,456.09
JOHNSON 25,467.17
MCABE 21,766.36

```

If the field delimiter is also a comma, the fields are identified incorrectly, as shown below.

**Figure 18.13** Raw Data File with Incorrect Use of Commas

```

1---+-----10---+-----20
BROWN,24,456.09
JOHNSON,25,467.17
MCABE,21,766.36

```

**Figure 18.14** Output When Commas Are Used Incorrectly

SAS Data Set		
Obs	LastName	Salary
1	BROWN	24
2	JOHNSON	25
3	McABE	21

### Reading Values That Contain Delimiters within a Quoted String

You can also use the DSD option in an INFILE statement to read values that contain delimiters within a quoted string. As shown in the following PROC PRINT output, the INPUT statement correctly interprets the commas within the values for Salary, does not interpret them as delimiters, and removes the quotation marks from the character strings before the value is stored.

```
data work.finance2;
 filename find 'c:\data\findat2';
 infile find dsd;
 length SSN $ 11 Name $ 9;
 input ssn name Salary : comma. Date : date9.;
run;
proc print data=work.finance2;
 format date date9.;
run;
```

**Figure 18.15** Raw Data File Findat2

Raw Data File Findat2				
1----	10----	20----	30----	40----
074-53-9892,	Vincent,	"35,000",	22MAY1997	
776-84-5391,	Phillipon,	"29,750",	15DEC1996	
929-75-0218,	Gunter,	"27,500",	30APR1997	
446-93-2122,	Harbinger,	"33,900",	08JUL1996	
228-88-9649,	Benito,	"28,000",	04MAR1996	
029-46-9261,	Rudelich,	"35,000",	15FEB1995	
442-21-8075,	Sirignano,	"5,000",	22NOV1995	

**Figure 18.16** Output Created with PROC PRINT

Obs	SSN	Name	Salary	Date
1	074-53-9892	Vincent	35000	22MAY1997
2	776-84-5391	Phillipon	29750	15DEC1996
3	929-75-0218	Gunter	27500	30APR1997
4	446-93-2122	Harbinger	33900	08JUL1996
5	228-88-9649	Benito	28000	04MAR1996
6	029-46-9261	Rudelich	35000	15FEB1995
7	442-21-8075	Sirignano	5000	22NOV1995

### Reading Missing Values at the End of a Record

Suppose that the third person who is represented in the raw data file below did not answer the questions about how many department store credit cards she has and about how often she uses them.

**Figure 18.17** Raw Data File with Missing Values at the End of a Record

```

1---+----10---+----20
MALE 27 1 8 0 0
FEMALE 3 14 5 10
FEMALE 34 2 10
MALE 35 2 12 4 8
FEMALE 36 4 16 3 7
MALE 21 1 5 0 0
MALE 25 2 9 2 1
FEMALE 21 1 4 2 6
MALE 38 3 11 4 3
FEMALE 30 3 5 1 0

```

The missing values occur at the end of the record. Therefore, you can use the `MISSOVER` option in the `INFILE` statement to assign the missing values to variables with missing data at the end of a record. The `MISSOVER` option prevents SAS from reading the next record if, when you are using list input, it does not find values in the current line for all the `INPUT` statement variables. At the end of the current record, values that are expected, but not found, are set to missing.

For the raw data file shown above, the `MISSOVER` option prevents the fields in the fourth record from being read as values for `Deptcard` and `FreqDept` in the third observation. Note that values for `Deptcard` and `FreqDept` are set to missing.

```

data sasuser.creditsurvey;
 infile creditcr missover;
 input Gender $ Age Bankcard FreqBank
 Deptcard FreqDept;
run;
proc print data=sasuser.creditsurvey;
run;

```

**Figure 18.18** Output Showing Missing Values

Gender	Age	Bankcard	FreqBank	Deptcard	FreqDept
MALE	27	1	8	0	0
FEMALE	29	3	14	5	10
FEMALE	34	2	10	.	.
MALE	35	2	12	4	8
FEMALE	36	4	16	3	7
MALE	21	1	5	0	0

*Note:* The `MISSOVER` option works only for missing values that occur at the end of the record.

### Reading Missing Values at the Beginning or Middle of a Record

Remember that the MISSEVER option works only for missing values that occur at the end of the record. A different method is required when you use list input to read raw data that contains missing values at the beginning or middle of a record. In this example, see what happens when a missing value occurs at the beginning or middle of a record.

Suppose the value for Age is missing in the first record.

**Figure 18.19** Raw Data File with Missing Values at the Beginning or Middle of a Record

```
Raw Data File Credit2
1---+---10---+---20
MALE, , 1, 8, 0, 0
FEMALE, 29, 3, 14, 5, 10
FEMALE, 34, 2, 10, 3, 3
MALE, 35, 2, 12, 4, 8
FEMALE, 36, 4, 16, 3, 7
```

When the program below executes, each field in the raw data file is read one by one. The INPUT statement tells SAS to read six data values from each record. However, the first record contains only five values.

```
data sasuser.creditsurvey;
 infile credit2 dlm=',';
 input Gender $ Age Bankcard FreqBank
 Deptcard FreqDept;
run;
proc print data=sasuser.creditsurvey;
run;
```

The two commas in the first record are interpreted as one delimiter. The incorrect value 1 is read for Age. The program continues to read subsequent incorrect values for Bankcard 8, FreqBank 0, and Deptcard 0. The program then attempts to read the character field FEMALE, at the beginning of the second record, as the value for the numeric variable FreqDept. This causes the value of FreqDept in the first observation to be interpreted as missing. The input pointer then moves down to the third record to begin reading values for the second observation. Therefore, the first observation in the data set contains incorrect values, and values from the second record in the raw data file are not included.

**Figure 18.20** Output with Missing Data Records

Gender	Age	Bankcard	FreqBank	Deptcard	FreqDept
MALE	1	8	0	0	.
FEMALE	34	2	10	3	3
MALE	35	2	12	4	8
FEMALE	36	4	16	3	7

You can use the Delimiter Sensitive Data (DSD) option in the INFILE statement to correctly read the raw data. The DSD option changes how SAS treats delimiters when list input is used. Specifically, the DSD option does the following:

- sets the default delimiter to a comma
- treats two consecutive delimiters as a missing value
- removes quotation marks from values

When the following program reads the raw data file, the DSD option sets the default delimiter to a comma and treats the two consecutive delimiters as a missing value. Therefore, the data is read correctly.

```
data sasuser.creditsurvey;
 infile credit2 dsd;
 input Gender $ Age Bankcard FreqBank
 Deptcard FreqDept;
run;
proc print data=sasuser.creditsurvey;
run;
```

**Figure 18.21** DSD Raw Data and Output

Raw Data File Credit2					
1---+---10---+---20					
MALE, , 1, 8, 0, 0					
FEMALE, 29, 3, 14, 5, 10					
FEMALE, 34, 2, 10, 3, 3					
MALE, 35, 2, 12, 4, 8					
FEMALE, 36, 4, 16, 3, 7					

Gender	Age	Bankcard	FreqBank	Deptcard	FreqDept
MALE	.	1	8	0	0
FEMALE	29	3	14	5	10
FEMALE	34	2	10	3	3
MALE	35	2	12	4	8
FEMALE	36	4	16	3	7

If the data uses multiple delimiters or a single delimiter other than a comma, simply specify the delimiter value or values with the DLM= option. In the following example, an asterisk (\*) is used as a delimiter. However, the data is still read correctly because of the DSD option.

```
data sasuser.creditsurvey;
 infile credit3 dsd dlm='*';
 input Gender $ Age Bankcard FreqBank
 Deptcard FreqDept;
run;
proc print data=sasuser.creditsurvey;
run;
```

**Figure 18.22** Raw Data with Multiple Delimiters and Output

Raw Data File Credit3					
1---+-----10---+-----20					
MALE**1*8*0*0					
FEMALE*29*3*14*5*10					
FEMALE*34*2*10*3*3					
MALE*35*2*12*4*8					
FEMALE*36*4*16*3*7					

Gender	Age	Bankcard	FreqBank	Deptcard	FreqDept
MALE	.	1	8	0	0
FEMALE	29	3	14	5	10
FEMALE	34	2	10	3	3
MALE	35	2	12	4	8
FEMALE	36	4	16	3	7

The DSD option can also be used to read raw data when there is a missing value at the beginning of a record, as long as a delimiter precedes the first value in the record.

```
data sasuser.creditsurvey;
 infile credit4 dsd;
 input Gender $ Age Bankcard FreqBank
 Deptcard FreqDept;
run;
proc print data=sasuser.creditsurvey;
run;
```

**Figure 18.23** Raw Data with Missing Data and Output

Raw Data File Credit4					
1---+-----10---+-----20					
,27,1,8,0,0					
FEMALE,29,3,14,5,10					
FEMALE,34,2,10,3,3					
MALE,35,2,12,4,8					
FEMALE,36,4,16,3,7					

Gender	Age	Bankcard	FreqBank	Deptcard	FreqDept
	27	1	8	0	0
FEMALE	29	3	14	5	10
FEMALE	34	2	10	3	3
MALE	35	2	12	4	8
FEMALE	36	4	16	3	7

Because DSD uses a comma as the delimiter, you can use DSD and DLM= to specify a different delimiter. You can also use the DSD and DLM= options to read fields that are delimited by blanks.

```
data sasuser.creditsurvey;
```



```
infile credit5.dat dsd dlm=' ';
input Gender $ Age Bankcard FreqBank
 Deptcard FreqDept;
run;
proc print data=sasuser.creditsurvey;
run;
```

## Reading Nonstandard Data with List Input

### The Basics of Modifying List Input

You can make list input more versatile by using modified list input. There are two modifiers that can be used with list input.

- The ampersand (&) modifier is used to read character values that contain embedded blanks.
- The colon (:) modifier is used to read nonstandard data values and character values that are longer than eight characters, but which contain no embedded blanks.

You can use modified list input to read the file shown below. This file contains the names of the 10 largest U.S. cities ranked in order based on their 2016 estimated population figures.

Notice that some of the values for city names contain embedded blanks and are followed by two blanks. Also, note that the values representing the population of each city are nonstandard numeric values (they contain commas).

**Figure 18.24** Raw Data File Topten

```
1---+----10---+----20---+--
1 NEW YORK 8,538,000
2 LOS ANGELES 3,976,000
3 CHICAGO 2,705,000
4 HOUSTON 2,303,000
5 PHILADELPHIA 1,568,000
6 DETROIT 672,795
7 SAN DIEGO 1,407,000
8 DALLAS 1,318,000
9 SAN ANTONIO 1,493,000
10 PHOENIX 1,615,000
```

### Reading Values That Contain Embedded Blanks

The ampersand (&) modifier enables you to read character values that contain single embedded blanks. The & indicates that a character value that is read with list input might contain one or more single embedded blanks. The value is read until two or more consecutive blanks are encountered. The & modifier precedes a specified informat if one is used.

```
input Rank City &;
```

### Using the & Modifier with a LENGTH Statement

As shown below, you can use a LENGTH statement to define the length of City, and then add an & modifier to the INPUT statement to indicate that the values contain embedded blanks.

```
data sasuser.cityrank;
 infile topten;
 length City $ 12;
 input Rank city &;
```

**Figure 18.25** Raw Data File Topten

```
1---+----10---+----20---+--
1 NEW YORK 8,538,000
2 LOS ANGELES 3,976,000
3 CHICAGO 2,705,000
4 HOUSTON 2,303,000
5 PHILADELPHIA 1,568,000
6 DETROIT 672,795
7 SAN DIEGO 1,407,000
8 DALLAS 1,318,000
9 SAN ANTONIO 1,493,000
10 PHOENIX 1,615,000
```

### Using the & Modifier with an Informat

You can also read the values for City with the & modifier followed by the \$w. informat, which reads standard character values, as shown below. When you do this, the w value in the informat determines the variable's length and should be large enough to accommodate the longest value.

*Note:* SAS reads until it encounters two consecutive blanks, the defined length of the variable, or the end of the input line, whichever comes first.

**Figure 18.26** Raw Data File Topten

```
Raw Data File Topten
1---+----10---+----20---+--
1 NEW YORK 7,262,700
2 LOS ANGELES 3,259,340
3 CHICAGO 3,009,530
4 HOUSTON 1,728,910
5 PHILADELPHIA 1,642,900
6 DETROIT 1,086,220
7 SAN DIEGO 1,015,190
8 DALLAS 1,003,520
9 SAN ANTONIO 914,350
10 PHOENIX 894,070
```

**TIP** Use two consecutive blanks as delimiters when you use the & modifier. You cannot use any other delimiter to indicate the end of each field.

## Reading Nonstandard Values

The colon (:) modifier enables you to read nonstandard data values and character values that are longer than eight characters, but which contain no embedded blanks. The : indicates that values are read until a blank (or other delimiter) is encountered, and then an informat is applied. If an informat for reading character values is specified, the *w* value specifies the variable's length, overriding the default length.

Notice the values representing the 2016 population of each city in the raw data file below. Because they contain commas, these values are nonstandard numeric values.

**Figure 18.27** Raw Data File *Topten*

```
1---+-----10---+-----20---+---
1 NEW YORK 8,538,000
2 LOS ANGELES 3,976,000
3 CHICAGO 2,705,000
4 HOUSTON 2,303,000
5 PHILADELPHIA 1,568,000
6 DETROIT 672,795
7 SAN DIEGO 1,407,000
8 DALLAS 1,318,000
9 SAN ANTONIO 1,493,000
10 PHOENIX 1,615,000
```

In order to read these values, you can modify list input with the colon (:) modifier, followed by the `COMMAw.d` informat, as shown in the program below. Notice that the `COMMAw.d` informat does not specify a *w* value.

```
data sasuser.cityrank;
 infile topten;
 input Rank City & $12.
 Pop86 : comma.;
```

Remember that list input reads each value until the next blank is detected. The default length of numeric variables is 8, so you do not need to specify a *w* value to indicate the length of a numeric variable.

This is different from using a numeric informat with formatted input. In that case, you must specify a *w* value in order to indicate the number of columns to be read.

## Processing the DATA Step

At compile time, the informat `$12.` in the example below sets the length of `City` to 12 and stores this information in the descriptor portion of the data set. During the execution phase, however, the *w* value of 12 does not determine the number of columns that are read. This is different from the function of informats in the formatted input style.

```
data sasuser.cityrank;
 infile topten;
 input Rank City & $12.
 Pop86 : comma.;
```

```
run;
```

**Figure 18.28** Reading Raw Data File with Character Values That Are Longer Than 8

```
1---+----10---+----20---+--
1 NEW YORK 8,538,000
2 LOS ANGELES 3,976,000
3 CHICAGO 2,705,000
4 HOUSTON 2,303,000
5 PHILADELPHIA 1,568,000
```

The & modifier indicates that the values for City should be read until two consecutive blanks are encountered. Therefore, the value **NEW YORK** is read from column 4 to column 11, a total of only 8 columns. When blanks are encountered in both columns 12 and 13, the value **NEW YORK** is written to the program data vector.

```
data sasuser.cityrank;
 infile topten;
 input Rank City & $12.
 Pop86 : comma.;
run;
```

**Figure 18.29** Reading City Value from Raw Data File

```
12345678
1---+----10V---+----20---+--
1 NEW YORK 8,538,000
2 LOS ANGELES 3,976,000
3 CHICAGO 2,705,000
4 HOUSTON 2,303,000
5 PHILADELPHIA 1,568,000
```

Program Data Vector

<u>N</u>	Rank	City	Pop16
1	1	New York	.

The input pointer moves forward to the next nonblank column, which is column 14 in the first record. Now the values for Pop86 are read from column 14 until the next blank is encountered. The `COMMAw.d` informat removes the commas, and the value is written to the program data vector.

```
data sasuser.cityrank;
 infile topten input Rank City & $12.
 Pop86 : comma.;
run;
```

**Figure 18.30** Reading Raw Data File POP16 Value

```

1---+-----10---+-----20-V-+---
1 NEW YORK 8,538,000
2 LOS ANGELES 3,976,000
3 CHICAGO 2,705,000
4 HOUSTON 2,303,000
5 PHILADELPHIA 1,568,000

```

Program Data Vector

<u>N</u>	Rank	City	Pop16
1	1	New York	8538000

Notice that the character values for City and Pop16 are stored correctly in the data set.

**Figure 18.31** SAS Data Set Cityrank

Rank	City	Pop16
1	NEW YORK	8538000
2	LOS ANGELES	3976000
3	CHICAGO	2705000
4	HOUSTON	2303000
5	PHILADELPHIA	1568000
6	DETROIT	672795
7	SAN DIEGO	1407000
8	DALLAS	1318000
9	SAN ANTONIO	1493000
10	PHOENIX	1615000

### Comparing Formatted Input and Modified List Input

As you have seen, informats work differently in modified list input than they do in formatted input. With formatted input, the informat determines both the length of character variables and the number of columns that are read. The same number of columns are read from each record.

```
input @3 City $12.;
```

Figure 18.32 Raw Data Showing That the Same Number of Columns Are Read from Each Record

123456789--12											
1	2	3	4	5	6	7	8	9	10	11	12
1	NEW	YORK							8,538,000		
2	LOS	ANGELES							3,976,000		
3	CHICAGO								2,705,000		
4	HOUSTON								2,303,000		
5	PHILADELPHIA								1,568,000		

The informat in modified list input determines only the length of the variable, not the number of columns that are read. Here, the raw data values are read until two consecutive blanks are encountered.

input City & \$12.;

Figure 18.33 Raw Data Showing That Values Are Read until Two Consecutive Blanks Are Encountered

123456789--12											
1	2	3	4	5	6	7	8	9	10	11	12
1	NEW	YORK							8,538,000		
2	LOS	ANGELES							3,976,000		
3	CHICAGO								2,705,000		
4	HOUSTON								2,303,000		
5	PHILADELPHIA								1,568,000		

# Creating Free-Format Data

## The Basics of Creating Free-Format Data

Chapter 6, “Creating SAS Data Sets from External Files,” explained how the PUT statement can be used with column output to write observations from a SAS data set to a raw data file. The PUT statement can also be used with list output to create free-format raw data files.

List output is similar to list input. With list output, you simply list the names of the variables whose values you want to write. The PUT statement writes a value, leaves a blank, and then writes the next value.

Syntax, PUT statement using list output:

**PUT** variable <: format>;

- *variable* specifies the variable whose value you want to write.
- *:* precedes a format.
- *format* specifies a format to use for writing the data values.

The following program creates the raw data file Findat, using the SAS data set sasuser.Finance. The DATEw. format is used to write the value of Date in the form DDMMYYYY.

```
data _null_;
 set sasuser.finance;
 file 'c:\data\findat.txt';
 put ssn name salary date : date9.;
run;
proc print data=sasuser.finance noobs;
run;
```

**Figure 18.34** SAS Data Set Finance

SSN	Name	Salary	Date
029-46-9261	Rudelich	35000	02/15/95
074-53-9892	Vincent	35000	05/22/97
228-88-9649	Benito	28000	03/04/96
442-21-8075	Sirignano	5000	11/22/95
446-93-2122	Harbinger	33900	07/08/96
776-84-5391	Phillipon	29750	12/15/96
929-75-0218	Gunter	27500	04/30/97

**Figure 18.35** Raw Data File Findat

```
Raw Data File Findat
1---+---10---+---20---+---30---+---40
074-53-9892 Vincent 35000 22MAY1997
776-84-5391 Phillipon 29750 15DEC1996
929-75-0218 Gunter 27500 30APR1997
446-93-2122 Harbinger 33900 08JUL1996
228-88-9649 Benito 28000 04MAR1996
029-46-9261 Rudelich 35000 15FEB1995
442-21-8075 sirignano 5000 22NOV1995
```

## Specifying a Delimiter

You can use the DLM= option with a FILE statement to create a character-delimited raw data file.

```
data _null_;
 set sasuser.finance;
 file 'c:\data\findat2' dlm=' ';
 put ssn name salary date : date9.;
run;
```

**Figure 18.36** SAS Data Set Finance

SSN	Name	Salary	Date
029-46-9261	Rudelich	35000	02/15/95
074-53-9892	Vincent	35000	05/22/97
228-88-9649	Benito	28000	03/04/96
442-21-8075	Sirignano	5000	11/22/95
446-93-2122	Harbinger	33900	07/08/96
776-84-5391	Phillipon	29750	12/15/96
929-75-0218	Gunter	27500	04/30/97

**Figure 18.37** Raw Data File Created by the DLM= Option

```

Raw Data File Findat2
1---+---10---+---20---+---30---+---40
074-53-9892,Vincent,35000,22MAY1997
776-84-5391,Phillipon,29750,15DEC1996
929-75-0218,Gunter,27500,30APR1997
446-93-2122,Harbinger,33900,08JUL1996
228-88-9649,Benito,28000,04MAR1996
029-46-9261,Rudelich,35000,15FEB1995
442-21-8075,Sirignano,5000,22NOV1995

```

**TIP** To create a simple raw data file, you can use the EXPORT procedure as an alternative to the DATA step.

Syntax, PROC EXPORT:

```

PROC EXPORT DATA=SAS-data-set;
 OUTFILE=filename <DELIMITER='delimiter'>;
RUN;

```

- *SAS-data-set* names the input SAS data set.
- *filename* specifies the complete path and filename of the output.
- *delimiter* specifies the delimiter to separate columns of data in the output file.

### Using the DSD Option

What happens if you need to create a comma-delimited file that requires the use of a format that writes out values using commas?

If you used the following program, the resulting raw data file would contain five fields rather than four.

```

data _null_;
 set sasuser.finance;
 file 'c:\data\findat2' dlm=',';
 put ssn name salary : comma6. date : date9.;
run;

```



**Figure 18.38** SAS Data Set Finance

SSN	Name	Salary	Date
029-46-9261	Rudelich	35000	02/15/95
074-53-9892	Vincent	35000	05/22/97
228-88-9649	Benito	28000	03/04/96
442-21-8075	Sirignano	5000	11/22/95
446-93-2122	Harbinger	33900	07/08/96
776-84-5391	Phillipon	29750	12/15/96
929-75-0218	Gunter	27500	04/30/97

**Figure 18.39** Raw Data Created with the DLM Option

Raw Data File Findat2

```

1---+---10---+---20---+---30---+---40
074-53-9892,Vincent,35,000,22MAY1997
776-84-5391,Phillipon,29,750,15DEC1996
929-75-0218,Gunter,27,500,30APR1997
446-93-2122,Harbinger,33,900,08JUL1996
228-88-9649,Benito,28,000,04MAR1996
029-46-9261,Rudelich,35,000,15FEB1995
442-21-8075,Sirignano,5,000,22NOV1995

```

You can use the DSD option in the FILE statement to specify that data values that contain commas should be enclosed in quotation marks. Remember that the DSD option uses a comma as a delimiter, so a DLM= option is not necessary here.

```

data _null_;
 set sasuser.finance;
 file 'c:\data\findat2' dsd;
 put ssn name salary : comma. date : date9.;
run;

```

**Figure 18.40** Raw Data Created with the DSD Option

Raw Data File Findat2

```

1---+---10---+---20---+---30---+---40
074-53-9892,Vincent,"35,000",22MAY1997
776-84-5391,Phillipon,"29,750",15DEC1996
929-75-0218,Gunter,"27,500",30APR1997
446-93-2122,Harbinger,"33,900",08JUL1996
228-88-9649,Benito,"28,000",04MAR1996
029-46-9261,Rudelich,"35,000",15FEB1995
442-21-8075,Sirignano,"5,000",22NOV1995

```

With some file layouts, you might need to mix input styles in the same INPUT statement in order to read the data correctly.

**Table 18.1** *Input Styles and the Types of Information They Read*

Input Style	What It Reads
Column	standard data values in fixed fields
Formatted	standard and nonstandard data values in fixed fields
List	data values that are not arranged in fixed fields, but are separated by blanks or other delimiters

Look at the raw data file below and think about how to combine input styles to read these values.

**Figure 18.41** *Raw Data Showing Mixed Input Styles*

```
1---+----10---+----20---+----30---+----40-
209-20-3721 07JAN78 41,983 SALES 2896
223-96-8933 03MAY86 27,356 EDUCATION 2344
232-18-3485 17AUG81 33,167 MARKETING 2674
251-25-9392 08SEP84 34,033 RESEARCH 2956
```

- Column input is an appropriate choice for the first field because the values can be read as standard character values and are located in fixed columns.
- The next two fields are also located in fixed columns, but the values require an informat. So, formatted input is a good choice here.
- Values in the fourth field begin in column 28 but do not end in the same column. List input is appropriate here, but notice that some values are longer than eight characters. You must use the : format modifier with an informat to read these values.
- The last field does not always begin or end in the same column, so list input is the best input style for those values.

**Table 18.2** *Input Styles to Read the Raw Data*

Field Description	Starting Column	Field Width	Data Type	Input Style
Social Security Number	1	11	character	column
Date of Hire	13	7	date	formatted
Annual Salary	21	6	numeric	formatted
Department	28	5 to 9	character	list
Phone Extension	??	4	character	list

The INPUT statement to read the data should look like this:

```
data sasuser.mixedstyles;
 infile rawdata;
 input SSN $ 1-11 @13 HireDate date7.
 @21 Salary comma6. Department : $9. Phone $;
run;
proc print data=sasuser.mixedstyles;
run;
```

When you submit the PRINT procedure, the output displays values for each variable.

**Figure 18.42** Output Created by PROC PRINT

Obs	SSN	HireDate	Salary	Department	Phone
1	209-20-3721	12060	41983	SALES	2896
2	223-96-8933	11445	27356	EDUCATION	2344
3	232-18-3485	12647	33167	MARKETING	2674
4	251-25-9392	11208	34033	RESEARCH	2956

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

1. What kind of data does the raw data file that is referenced by the fileref Students contain?

Raw Data File Students

```
1---+----10---+----20---+
FRED JOHNSON 18 USC 1
ASHLEY FERRIS 20 NCSU 3
BETH ROSEMONT 21 UNC 4
```

- a. arranged in fixed fields
  - b. free-format
  - c. mixed-format
  - d. arranged in columns
2. Which input style should be used to read the values in the raw data file that is referenced by the fileref Students?

Raw Data File Students

```
1---+----10---+----20---+
FRED JOHNSON 18 USC 1
ASHLEY FERRIS 20 NCSU 3
BETH ROSEMONT 21 UNC 4
```

- a. column

- b. formatted
  - c. list
  - d. mixed
3. Which SAS program was used to create the raw data file Teamdat from the SAS data set Work.Scores?

SAS Data Set Work.Scores

Obs	Name	HighScore	Team
1	Joe	87	Blue Beetles, Durham
2	Dani	79	Raleigh Racers, Raleigh
3	Lisa	85	Sand Sharks, Cary
4	Matthew	76	Blue Beetles, Durham

Raw Data File Teamdat

```
1---+-----10---+-----20---+-----30---+
Joe,87,"Blue Beetles, Durham"
Dani,79,"Raleigh Racers, Raleigh"
Lisa,85,"Sand Sharks, Cary"
Matthew,76,"Blue Beetles, Durham"
```

- a. 

```
data _null_;
 set work.scores;
 file 'c:\data\teamdat' dlm=',';
 put name highscore team;
run;
```
  - b. 

```
data _null_;
 set work.scores;
 file 'c:\data\teamdat' dlm=' ';
 put name highscore team;
run;
```
  - c. 

```
data _null_;
 set work.scores;
 file 'c:\data\teamdat' dsd;
 put name highscore team;
run;
```
  - d. 

```
data _null_;
 set work.scores;
 file 'c:\data\teamdat';
 put name highscore team;
run;
```
4. Which SAS statement reads the raw data values in order and assigns them to the variables that are shown below?

Variables: FirstName (character), LastName (character), Age (numeric), School (character), Class (numeric)

Raw Data File Students

```
1---+-----10---+-----20---+
FRED JOHNSON 18 USC 1
ASHLEY FERRIS 20 NCSU 3
BETH ROSEMONT 21 UNC 4
```

- a. `input FirstName $ LastName $ Age School $ Class;`
  - b. `input FirstName LastName Age School Class;`
  - c. `input FirstName $ 1-4 LastName $ 6-12 Age 14-15  
School $ 17-19 Class 21;`
  - d. `input FirstName 1-4 LastName 6-12 Age 14-15  
School 17-19 Class 21;`
5. Which SAS statement should be used to read the raw data file that is referenced by the `fileref Salesrep`?

Raw Data File Salesrep

```
1---+----10---+----20---+----30
ELAINE:FRIEDMAN:WILMINGTON:2102
JIM:LLOYD:20:RALEIGH:38392
JENNIFER:WU:21:GREENSBORO:1436
```

- a. `infile salesrep;`
  - b. `infile salesrep ':';`
  - c. `infile salesrep dlm;`
  - d. `infile salesrep dlm=':';`
6. Which of the following raw data files can be read by using the `MISSOVER` option in the `INFILE` statement? Spaces for missing values are highlighted with shaded blocks.

a.

```
1---+----10---+----20---+----
ORANGE SUNNYDALE 20 10
PINEAPPLE ALOHA 7 10
GRAPE FARMFRESH 3
APPLE FARMFRESH 16 5
GRAPEFRUIT SUNNYDALE 12 8
```

b.

```
1---+----10---+----20---+----
ORANGE SUNNYDALE 20 10
PINEAPPLE ALOHA 7 10
GRAPE FARMFRESH 17
APPLE FARMFRESH 16 5
GRAPEFRUIT SUNNYDALE 12 8
```

c.

```
1---+----10---+----20---+----
ORANGE SUNNYDALE 20 10
PINEAPPLE ALOHA 7 10
GRAPE 3 17
APPLE FARMFRESH 16 5
GRAPEFRUIT SUNNYDALE 12 8
```

d.

```

1---+----10---+----20---+----
ORANGE SUNNYDALE 20 10
PINEAPPLE ALOHA 7 10
 FARMFRESH 3
APPLE FARMFRESH 16 5
GRAPEFRUIT SUNNYDALE 12 8

```

7. Which SAS program correctly reads the data in the raw data file that is referenced by the fileref Volunteer?

Raw Data File Volunteer

```

1---+----10---+----20---+----30
ARLENE BIGGERSTAFF 19 UNC 2
JOSEPH CONSTANTINO 21 CLEM 2
MARTIN FIELDS 18 UNCG 1

```

- ```

data sasuser.contest;
  infile volunteer;
  input FirstName $ LastName $ Age
        School $ Class;
run;

```
 - ```

data sasuser.contest;
 infile volunteer;
 length LastName $ 11;
 input FirstName $ lastname $ Age
 School $ Class;
run;

```
  - ```

data sasuser.contest;
  infile volunteer;
  input FirstName $ lastname $ Age
        School $ Class; length LastName $ 11;
run;

```
 - ```

data sasuser.contest;
 infile volunteer;
 input FirstName $ LastName $ 11. Age
 School $ Class;
run;

```
8. Which type of input should be used to read the values in the raw data file that is referenced by the fileref University?

Raw Data File University

```

1---+----10---+----20---+----30
UNC ASHEVILLE 2,712
UNC CHAPEL HILL 24,189
UNC CHARLOTTE 15,031
UNC GREENSBORO 12,323

```

- column
- formatted
- list
- modified list

9. Which SAS statement correctly reads the values for Flavor and Quantity? Make sure the length of each variable can accommodate the values that are shown.

Raw Data File Cookies

1----	+-----10----	+-----20----	+-----30
CHOCOLATE	CHIP	10,453	
OATMEAL	12,187		
PEANUT	BUTTER	11,546	
SUGAR	12,331		

- input Flavor & \$9. Quantity : comma.;
  - input Flavor & \$14. Quantity : comma.;
  - input Flavor : \$14. Quantity & comma.;
  - input Flavor \$14. Quantity : comma.;
10. Which SAS statement correctly reads the raw data values in order and assigns them to these corresponding variables: Year (numeric), School (character), Enrolled (numeric)?

Raw Data File Founding

1----	+-----10----	+-----20----	+-----30----	+-----40
1868	U OF CALIFORNIA	BERKELEY	31,612	
1906	U OF CALIFORNIA	DAVIS	21,838	
1965	U OF CALIFORNIA	IRVINE	15,874	
1919	U OF CALIFORNIA	LOS ANGELES	35,730	

- input Year School & \$27.  
Enrolled : comma.;
- input Year 1-4 School & \$27.  
Enrolled : comma.;
- input @1 Year 4. +1 School & \$27.  
Enrolled : comma.;
- all of the above





## Chapter 19

## SAS Date and Time Values

---

<b>Reading Dates and Times with Informats</b> .....	<b>447</b>
Overview .....	447
INPUT Statement Syntax .....	448
The MMDDYYw. Informat .....	448
The DATEw. Informat .....	449
The TIMEw. Informat .....	449
The DATETIMEw. Informat .....	450
The YEARCUTOFF= System Option .....	451
<b>Using Dates and Times in Calculations</b> .....	<b>451</b>
<b>Displaying Date and Time Values with Formats</b> .....	<b>456</b>
The WEEKDATEw. Format .....	456
The WORDDATEw. Format .....	457
<b>Chapter Quiz</b> .....	<b>458</b>

---

## Reading Dates and Times with Informats

**Overview**

SAS *date and time informats* read date and time expressions and convert them to SAS date and time values. Like other SAS informats, date and time informats have several parts:

- an informat name
- a field width
- a period delimiter

SAS informat names indicate the form of date expression that can be read using that particular informat. This chapter covers commonly used date and time informats such as these:

- DATEw.
- DATETIMEw.
- MMDDYYw.
- TIMEw.

There are several ways to represent a date. For example, all the following expressions represent the date October 15, 2009. Each of these common date expressions can be read using the appropriate SAS date informat.

**Table 19.1** Date Expressions and Corresponding SAS Date Informats

Date Expression	SAS Date Informat
10/15/09	MMDDYY <sub>w</sub> .
15Oct09	DATE <sub>w</sub> .
10-15-09	MMDDYY <sub>w</sub> .
09/10/15	YYMMDD <sub>w</sub> .

## INPUT Statement Syntax

Using the INPUT statement with an informat after a variable name is the simplest way to read date and time values into a variable.

Syntax, INPUT statement with an informat:

**INPUT** <pointer-control> variable informat.;

- *pointer-control* specifies the absolute or relative position to move the pointer.
- *variable* is the name of the variable being read.
- *informat.* is any valid SAS informat. Note that the informat includes a final period.

For example, the following INPUT statement uses two informats:

```
input @15 Style $3. @21 Price 5;
```

The \$<sub>w</sub>. character informat (\$3.) reads values, starting at column 15 of the raw data, into the variable Style. The *w.d* numeric informat (5) reads values, starting at column 21, into the variable Price.

## The MMDDYY<sub>w</sub>. Informat

The informat MMDDYY<sub>w</sub>. reads date values in the form 10/15/99.

Syntax, values read with MMDDYY<sub>w</sub>. informat:

*mmddy* or *mmddy*

- *mm* is an integer between 01 and 12, representing the month.
- *dd* is an integer between 01 and 31, representing the day.
- *yy* or *yyyy* is an integer that represents the year.

In the MMDDYY<sub>w</sub>. informat, the month, day, and year fields can be separated by blanks or delimiters such as - or /. If delimiters are present, they must occur between all fields in the values. Remember to specify a field width that includes not only the month, day, and

year values, but any delimiters as well. Here are some date expressions that you can read using the MMDDYY $w$ . informat:

**Table 19.2** Date Expressions and Corresponding SAS Date Informats

Date Expression	SAS Date Informat
101509	MMDDYY6.
10/15/09	MMDDYY8.
10 15 09	MMDDYY8.
10-15-2009	MMDDYY10.

### The DATE $w$ . Informat

The DATE $w$ . informat reads date values in the form 30May2000.

Syntax, values read with DATE $w$ . informat:

*ddmmmyy* or *ddmmmyyyy*

- *dd* is an integer from 01 to 31, representing the day.
- *mmm* is the first three letters of the month's name.
- *yy* or *yyyy* is an integer that represents the year.

Blanks or other special characters can appear between the day, month, and year, as long as you increase the width of the informat to include these delimiters. Here are some date expressions that you can read using the DATE $w$ . informat:

**Table 19.3** Date Expressions and Corresponding SAS Date Informats

Date Expression	SAS Date Informat
30May10	DATE7.
30May2010	DATE9.
30-May-2010	DATE11.

### The TIME $w$ . Informat

The TIME $w$ . informat reads values in the form *hh:mm:ss.ss*.

---

Syntax, values read with TIME $w$ . informat:

*hh:mm:ss.ss*

- *hh* is an integer from 00 to 23, representing the hour.
  - *mm* is an integer from 00 to 59, representing the minute.
  - *ss.ss* is an optional field that represents seconds and hundredths of seconds.
- 

If you do not enter a value for *ss.ss*, a value of zero is assumed. Here are some examples of time expressions that you can read using the TIME $w$ . informat:

**Table 19.4** Time Expressions and Corresponding SAS Time Informats

Time Expression	SAS Time Informat
17:00:01.34	TIME11.
17:00	TIME5.
2:34	TIME5.

---

*Note:* Five is the minimum acceptable field width for the TIME $w$ . informat. If you specify a  $w$  value less than 5, you will receive an error message in the SAS log.

### The DATETIME $w$ . Informat

The DATETIME $w$ . informat reads expressions that consist of two parts, a date value and a time value, in the form: *ddmmyy hh:mm:ss.ss*.

---

Syntax, values read with DATETIME $w$ . informat:

*ddmmyy hh:mm:ss.ss*

- *ddmmyy* is the date value, the same form as for the DATE $w$ . informat
  - The time value must be in the form *hh:mm:ss.ss*.
  - *hh* is an integer from 00 to 23, representing the hour.
  - *mm* is an integer from 00 to 59, representing the minute.
  - *ss.ss* is an optional field that represents seconds and hundredths of seconds.
  - The date value and time value are separated by a blank or other delimiter.
- 

If you do not enter a value for *ss.ss*, a value of zero is assumed.

*Note:* In the time value, you must use delimiters to separate the values for hour, minutes, and seconds.

**Table 19.5** Date and Time Expressions and Corresponding SAS Datetime Informats

Date and Time Expression	SAS Datetime Informat
30May2010:10:03:17.2	DATETIME20.

---

Date and Time Expression	SAS Datetime Informat
30May10 10:03:17.2	DATETIME18.
30May2010/10:03	DATETIME15.

### The YEARCUTOFF= System Option

A date value that contains a four-digit year value is interpreted correctly even if it does not fall within the 100-year span that is set by the YEARCUTOFF= system option.

**Table 19.6** Date Expressions

Date Expression	SAS Date Informat	Result
06Oct09	date7.	06Oct2009
17Mar2013	date9.	17Mar2013

If you specify an inappropriate field width, you receive incorrect results. The date expression in the table below contains a four-digit year value. The informat specifies a width value that is too small to read the entire value, so the last two digits of the year are truncated.

**Table 19.7** Date Expressions (Wrong Field Width)

Date Expression	SAS Date Informat	Result
17Mar1783	date7.	17Mar2017

If you use the wrong informat to read a date or time expression, the SAS log displays an invalid data message, and the variable's values are set to missing.

When you work with date and time values, remember to do the following:

- Check the default value of the YEARCUTOFF= system option, and change it if necessary. The default YEARCUTOFF= value is 1926.
- Specify the proper informat for reading a date value.
- Specify the correct field width so that the entire date value is read.

## Using Dates and Times in Calculations

Suppose you work in the billing department of a small community hospital. It is your job to create a SAS data set from the raw data file that is referenced by the fileref Aprdata. A portion of the raw data file below shows the following patient data:

- last name

- date checked in
- date checked out
- daily room rate
- equipment cost

**Figure 19.1** Raw Data File Aprdata

Raw Data File Aprdata				
1---+---10---+---20---+---30---+---40				
Akron	04/05/99	04/09/99	175.00	298.45
Brown	04/12/99	05/01/99	125.00	326.78
Carnes	04/27/99	04/29/99	125.00	174.24
Denison	04/11/99	04/12/99	175.00	87.41
Fields	04/15/99	04/22/99	175.00	378.96
Jamison	04/16/99	04/23/99	125.00	346.28

The data set that you create must also include variable values that represent how many days each person stayed in the hospital, the total room charges, and the total of all expenses that each patient incurred. To create the SAS program, you must first name the data set, identify the raw data file Aprdata, and use formatted input to read the data.

The following example is shown with the YEARCUTOFF= system option. When you work with two-digit year data, remember to check the default value of the YEARCUTOFF= option and change it if necessary.

```
options yearcutoff=1926;
data perm.aprbills;
 infile aprdata;
 input LastName $8.
```

Notice that the values in the second and third fields are in the form *mmddyy*. To complete the INPUT statement, add instructions to read the values for RoomRate (fourth field) and EquipCost (fifth field), and add a semicolon.

```
options yearcutoff=1926;
data perm.aprbills;
 infile aprdata;
 input LastName $8. @10 DateIn mmddyy8. +1 DateOut
 mmddyy8. RoomRate 6. @34 EquipCost 6.;
```

Now that the INPUT statement is complete, calculate how many days each patient was hospitalized. Because DateIn and DateOut are numeric variables, you can simply subtract to find the difference. But because the dates should be inclusive (patients are charged for both the first and last days), you must add 1 to the difference. Call this new variable Days.

```
options yearcutoff=1926;
data perm.aprbills;
 infile aprdata;
 input LastName $8. @10 DateIn mmddyy8. +1 DateOut
 mmddyy8. RoomRate 6. @34 EquipCost 6.;
 Days=dateout-datein+1;
```

You can calculate a total room charge by multiplying Days times RoomRate.

```
options yearcutoff=1926;
data perm.aprbills;
```

```

infile aprdata;
input LastName $8. @10 DateIn mmddyy8. +1 DateOut
 mmddyy8. RoomRate 6. @34 EquipCost 6.;
Days=dateout-datein+1;
RoomCharge=days*roomrate;

```

To calculate the total cost for each patient, create a variable named Total whose value is the sum of RoomCharge and EquipCost. Then add a PROC PRINT step and a RUN statement to view the new data.

```

options yearcutoff=1926;
data perm.aprbills;
 infile aprdata;
 input LastName $8. @10 DateIn mmddyy8. +1 DateOut
 mmddyy8. RoomRate 6. @34 EquipCost 6.;
 Days=dateout-datein+1;
 RoomCharge=days*roomrate;
 Total=roomcharge+equipcost;
run;
proc print data=perm.aprbills;
run;

```

**Figure 19.2** Table Created with PROC PRINT

Obs	LastName	DateIn	DateOut	RoomRate	EquipCost	Days	RoomCharge	Total
1	Akron	17992	17996	175	298.45	5	875	1173.45
2	Brown	17999	18018	125	326.78	20	2500	2826.78
3	Carnes	18014	18016	125	174.24	3	375	549.24
4	Denison	17998	17999	175	87.41	2	350	437.41
5	Fields	18002	18009	175	378.96	8	1400	1778.96
6	Jamison	18003	18010	125	346.28	8	1000	1346.28

If the values for DateIn and DateOut look odd to you, remember that these are SAS date values. Applying a format such as MMDDYY displays them as they appeared in Aprdata.

When the DATA step executes the following program, the values for DateIn and DateOut are converted to SAS date values.

```

options yearcutoff=1926;
data perm.aprbills;
 infile aprdata;
 input LastName $8. @10 DateIn mmddyy8. +1 DateOut
 mmddyy8. RoomRate 6. @35 EquipCost 6.;
 Days=dateout-datein+1;
 RoomCharge=days*roomrate;
 Total=roomcharge+equipcost;
run;

```

Here is how the data moves through the program data vector in order to create the report.

**Figure 19.3** Raw Data File Aprdata and Program Data Vector

Raw Data File Aprdata							
1----	10----	20----	30----	40			
Akron	04/05/09	04/09/09	175.00	298.45			
Brown	04/12/09	05/01/09	125.00	326.78			
Carnes	04/27/09	04/29/09	125.00	174.24			
Denison	04/11/09	04/12/09	175.00	87.41			
Fields	04/15/09	04/22/09	175.00	378.96			
Jamison	04/16/09	04/23/09	125.00	346.28			

Program Data Vector							
LastName	DateIn	DateOut	RoomRate	EquipCost	Days	RoomCharge	Total
Akron	17992	17996	.	.	.	.	.

Now, the variable Days is created by subtracting DateIn from DateOut and adding 1.

```
options yearcutoff=1926;
data perm.aprbills;
 infile aprdata;
 input LastName $8. @10 DateIn mmddyy8. +1 DateOut
 mmddyy8. RoomRate 6. @34 EquipCost 6.;
 Days=dateout-datein+1;
 RoomCharge=days*roomrate;
 Total=roomcharge+equipcost;
run;
```

**Figure 19.4** Program Data Vector

Program Data Vector							
LastName	DateIn	DateOut	RoomRate	EquipCost	Days	RoomCharge	Total
Akron	17992	17996	175.00	298.45	5	.	.

The value for RoomCharge is calculated next. RoomCharge is the product of Days and RoomRate.

```
options yearcutoff=1926;
data perm.aprbills;
 infile aprdata;
 input LastName $8. @10 DateIn mmddyy8. +1 DateOut
 mmddyy8. +1 RoomRate 6. @35 EquipCost 6.;
 Days=dateout-datein+1;
 RoomCharge=days*roomrate;
 Total=roomcharge+equipcost;
run;
```



**Figure 19.5** Raw Data File Aprdata and Program Data Vector

Raw Data File Aprdata

1---+---	10---+---	20---+---	30---+---	40
Akron	04/05/09	04/09/09	175.00	298.45
Brown	04/12/09	05/01/09	125.00	326.78
Carnes	04/27/09	04/29/09	125.00	174.24
Denison	04/11/09	04/12/09	175.00	87.41
Fields	04/15/09	04/22/09	175.00	378.96
Jamison	04/16/09	04/23/09	125.00	346.28

Program Data Vector

LastName	DateIn	DateOut	RoomRate	EquipCost	Days	RoomCharge	Total
Akron	17992	17996	175.00	298.45	5	875.00	•

\*

The value for Total is the final calculation. Total is the sum of EquipCost and RoomCharge.

```
options yearcutoff=1926;
data perm.aprbills;
 infile aprdata;
 input LastName $8. @10 DateIn mmddyy8. +1 DateOut
 mmddyy8. +1 RoomRate 6. @35 EquipCost 6.;
 Days=dateout-datein+1;
 RoomCharge=days*roomrate;
 Total=roomcharge+equipcost;
run;
```

**Figure 19.6** Raw Data File Aprdata and Program Data Vector

Raw Data File Aprdata

1	---	+	---	10	---	+	---	20	---	+	---	30	---	+	---	40
Akron	04/05/09	04/09/09	175.00	298.45												
Brown	04/12/09	05/01/09	125.00	326.78												
Carnes	04/27/09	04/29/09	125.00	174.24												
Denison	04/11/09	04/12/09	175.00	87.41												
Fields	04/15/09	04/22/09	175.00	378.96												
Jamison	04/16/09	04/23/09	125.00	346.28												

Program Data Vector

LastName	DateIn	DateOut	RoomRate	EquipCost	Days	RoomCharge	Total
Akron	17992	17996	175.00	298.45	5	875.00	1173.45

+

↑

## Displaying Date and Time Values with Formats

SAS stores date and time values as numeric variables. Formats should be applied to the data so that meaningful date and time values are displayed in reports.

### The WEEKDATEw. Format

Use the WEEKDATEw. format to write date values in a format that displays the day of the week, month, day, and year.

Syntax, WEEKDATEw. format:

**WEEKDATEw.**

The WEEKDATEw. format writes date values in the form *day-of-week, month-name dd, yy* (or *yyyy*).

- *dd* is an integer between 01 and 31, representing the day.
- *yy* or *yyyy* is an integer that represents the year.

*Note:* If the *w* value is too small to write the complete day of the week and month, SAS abbreviates as needed.

```
proc print data=perm.aprbills;
 format datein dateout weekdate17.;
run;
```

**Figure 19.7** PROC PRINT Output for the above Example

Obs	LastName	DateIn	DateOut	RoomRate	EquipCost	Days	RoomCharge	Total
1	Akron	Sun, Apr 5, 2009	Thu, Apr 9, 2009	175	298.45	5	875	1173.45
2	Brown	Sun, Apr 12, 2009	Fri, May 1, 2009	125	326.78	20	2500	2826.78
3	Carnes	Mon, Apr 27, 2009	Wed, Apr 29, 2009	125	174.24	3	375	549.24
4	Denison	Sat, Apr 11, 2009	Sun, Apr 12, 2009	175	87.41	2	350	437.41
5	Fields	Wed, Apr 15, 2009	Wed, Apr 22, 2009	175	378.96	8	1400	1778.96
6	Jamison	Thu, Apr 16, 2009	Thu, Apr 23, 2009	125	346.28	8	1000	1346.28

You can vary the results by changing the *w* value in the format.

FORMAT Statement	Result
format datein weekdate3.;	Mon
format datein weekdate6.;	Monday
format datein weekdate17.;	Mon, Apr 5, 2009
format datein weekdate21.;	Monday, April 5, 2009

### The WORDDATEw. Format

The WORDDATEw. format is similar to the WEEKDATEw. format, but it does not display the day of the week or the two-digit year values.

Syntax, WORDDATEw. format:

#### WORDDATEw.

The WORDDATEw. format writes date values in the form *month-name dd, yyyy*.

- *dd* is an integer between 01 and 31, representing the day.
- *yyyy* is an integer that represents the year.

*Note:* If the *w* value is too small to write the complete month, SAS abbreviates as needed.

```
proc print data=perm.aprbills;
 format datein dateout worddate12.;
run;
```

**Figure 19.8** PROC PRINT Output for the Example

Obs	LastName	DateIn	DateOut	RoomRate	EquipCost	Days	RoomCharge	Total
1	Akron	Apr 5, 2009	Apr 9, 2009	175	298.45	5	875	1173.45
2	Brown	Apr 12, 2009	May 1, 2009	125	326.78	20	2500	2826.78
3	Carnes	Apr 27, 2009	Apr 29, 2009	125	174.24	3	375	549.24
4	Denison	Apr 11, 2009	Apr 12, 2009	175	87.41	2	350	437.41
5	Fields	Apr 15, 2009	Apr 22, 2009	175	378.96	8	1400	1778.96
6	Jamison	Apr 16, 2009	Apr 23, 2009	125	346.28	8	1000	1346.28

You can vary the results by changing the *w* value in the format.

**Table 19.8** FORMAT Statements and Corresponding Results

FORMAT Statement	Result
format datein worddate3.;	Apr
format datein worddate5.;	April
format datein worddate14.;	April 15, 1999

You can permanently assign a format to variable values by including a FORMAT statement in the DATA step.

```
options yearcutoff=1926;
data work.aprbills;
 infile aprdata;
 input LastName $8. @10 DateIn mmddyy8. +1 DateOut
 mmddyy8. RoomRate 6. @34 EquipCost 6.;
 Days=dateout-datein+1;
 RoomCharge=days*roomrate;
```

```
Total=roomcharge+equipcost;
format datein dateout worddate12.;
run;
proc print data=work.aprbills;
run;
```

---

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

1. SAS date values are the number of days since which date?
  - a. January 1, 1900
  - b. January 1, 1950
  - c. January 1, 1960
  - d. January 1, 1970
2. What is an advantage of storing dates and times as SAS numeric date and time values?
  - a. They can easily be edited.
  - b. They can easily be read and understood.
  - c. They can be used in text strings like other character values.
  - d. They can be used in calculations like other numeric values.
3. SAS does not automatically make adjustments for daylight saving time, but it does make adjustments for which one of the following?
  - a. leap seconds
  - b. leap years
  - c. Julian dates
  - d. time zones
4. An input data file has date expressions in the form 10222001. Which SAS informat should you use to read these dates?
  - a. DATE6.
  - b. DATE8.
  - c. MMDDYY6.
  - d. MMDDYY8.
5. What is the minimum width of the TIMEw. informat?
  - a. 4
  - b. 5
  - c. 6
  - d. 7

6. Shown below are date and time expressions and corresponding SAS datetime informats. Which date and time expression cannot be read by the informat that is shown beside it?
- a. 30May2000:10:03:17.2 **DATETIME20.**
  - b. 30May00 10:03:17.2 **DATETIME18.**
  - c. 30May2000/10:03 **DATETIME15.**
  - d. 30May2000/1003 **DATETIME14.**
7. Suppose your input data file contains the date expression 13APR2009. The YEARCUTOFF= system option is set to 1910. How does SAS read the date?
- a. 13APR1909
  - b. 13APR1920
  - c. 13APR2009
  - d. 13APR2020
8. Suppose the YEARCUTOFF= system option is set to 1920. An input file contains the date expression 12/08/1925, which is being read with the MMDDYY8. informat. Which date appears in your data?
- a. 08DEC1920
  - b. 08DEC1925
  - c. 08DEC2019
  - d. 08DEC2025
9. Suppose your program creates two variables from an input file. Both variables are stored as SAS date values: FirstDay records the start of a billing cycle, and LastDay records the end of that cycle. What would be the code for calculating the total number of days in the cycle?
- a. TotDays=lastday-firstday;
  - b. TotDays=lastday-firstday+1;
  - c. TotDays=lastday/firstday;
  - d. You cannot use date values in calculations.



## Chapter 20

# Creating a Single Observation from Multiple Raw Data Records

---

<b>A Word About Creating a Single Observation</b> .....	<b>461</b>
<b>Using Line Pointer Controls</b> .....	<b>462</b>
<b>Reading Multiple Records Sequentially</b> .....	<b>462</b>
The Forward Slash (/) Line Pointer Control .....	462
Using the / Line Pointer Control .....	463
Sequential Processing of Multiple Records in the DATA Step .....	465
The Number of Records per Observation .....	468
<b>Reading Multiple Records Non-Sequentially</b> .....	<b>469</b>
The #n Line Pointer Control .....	469
Using the #n Line Pointer Control .....	469
Execution of the DATA Step .....	472
<b>Combining Line Pointer Controls</b> .....	<b>473</b>
<b>Chapter Quiz</b> .....	<b>474</b>

---

## A Word About Creating a Single Observation

Information for one observation can be spread out over several raw data file records. You can write multiple INPUT statements when multiple input records comprise a single observation, as shown below.

```
input Lname $ 1-8 Fname $ 10-15;
input Department $ 1-12 JobCode $ 15-18;
input Salary comma10.;
```

**Figure 20.1** Multiple Records Comprising Each Observation

1----	-----10----	-----
ABRAMS	THOMAS	
MARKETING	SR01	
\$25,209.03		
BARCLAY	ROBERT	
EDUCATION	IN01	
\$24,435.71		
COURTNEY	MARK	
PUBLICATIONS	TW01	
\$24,006.16		

---

## Using Line Pointer Controls

SAS reads raw data values and it keeps track of its position with an input pointer. Use column pointer controls and column specifications to determine the column placement with the line pointer tools.

---

## Reading Multiple Records Sequentially

### *The Forward Slash (/) Line Pointer Control*

Use the forward slash (/) line pointer control to read multiple records sequentially. The / advances the input pointer to the next record. The / line pointer control moves the input pointer forward only. Note that it must be specified after the instructions for reading the values in the current record.

The single INPUT statement below reads the values for Lname and Fname in the first record, followed by the values for Department and JobCode in the second record. Then the value for Salary is read in the third record.

```
input Lname $ 1-8 Fname $ 10-15 /
 Department $ 1-12 JobCode $ 15-19 /
 Salary comma10.;
```

**Figure 20.2** Multiple Records Comprising Each Observation

1----	-----10----	-----
ABRAMS	THOMAS	
MARKETING	SR01	
\$25,209.03		
BARCLAY	ROBERT	
EDUCATION	IN01	
\$24,435.71		
COURTNEY	MARK	
PUBLICATIONS	TW01	
\$24,006.16		



### Using the / Line Pointer Control

The raw data file Memdata contains the mailing list of a professional organization. Here are the steps for combining the information for each member into a single observation.

1. As you write the instructions to read the values for Fname and Lname, notice that not all of the values for Lname begin in the same column. Use standard list input to read these values.

```
data perm.members;
 infile memdata trunccover;
 input Fname $ Lname $
```

**TIP** Use the TRUNCOVER option in your INFILE statement so that SAS can read data for the variables until it reaches the end of the data line. Otherwise, your data will not be produced with the correct variables in the correct columns.

**Figure 20.3** Raw Data File Memdata

```
1---+----10---+----20
LEE ATHNOS
1215 RAINTREE CIRCLE
PHOENIX AZ 85044
HEIDIE BAKER
1751 DIEHL ROAD
VIENNA VA 22124
MYRON BARKER
131 DONERAIL DRIVE
ATLANTA GA 30363
JOYCE BENEFIT
85 MAPLE AVENUE
MENLO PARK CA 94025
```

2. Now read the values for Address from the second record. The / line pointer control advances the input pointer to the next record. At this point the INPUT statement is incomplete, so you should not place a semicolon after the line pointer control.

```
data perm.members;
 infile memdata trunccover;
 input Fname $ Lname $ /
```

**Figure 20.4** Raw Data File Memdata

```
1---+----10---+----20
LEE ATHNOS
1215 RAINTREE CIRCLE
PHOENIX AZ 85044
HEIDIE BAKER
1751 DIEHL ROAD
VIENNA VA 22124
MYRON BARKER
131 DONERAIL DRIVE
ATLANTA GA 30363
JOYCE BENEFIT
85 MAPLE AVENUE
MENLO PARK CA 94025
```

3. You can use column input to read the values in the next record as one variable that is named Address. Then add a line pointer control to move the input pointer to the next record.

```
data perm.members;
 infile memdata trunccover;
 input Fname $ Lname $ /
 Address $ 1-20 /
```

**Figure 20.5** Line Pointer Control Advanced One Record

```
1---+----10---+----20
LEE ATHNOS
1215 RAINTREE CIRCLE
PHOENIX AZ 85044
HEIDIE BAKER
1751 DIEHL ROAD
VIENNA VA 22124
MYRON BARKER
131 DONERAIL DRIVE
ATLANTA GA 30363
JOYCE BENEFIT
85 MAPLE AVENUE
MENLO PARK CA 94025
```

4. As you write the statements to read the values for City, notice that some of the values are longer than eight characters and contain embedded blanks. Also note that each value is followed by two consecutive blanks. To read these values, you should use modified list input with the ampersand (&) modifier.

The values for State and the values for Zip do not begin in the same column. Therefore, you should use list input to read these values.

```
data perm.members;
 infile memdata trunccover;
 input Fname $ Lname $ /
 Address $ 1-20 /
 City :& $10. State $ Zip $;

run;
```

**Figure 20.6** Line Pointer Control Advanced Another Record

```
1---+----10---+----20
LEE ATHNOS
1215 RAINTREE CIRCLE
PHOENIX AZ 85044
HEIDIE BAKER
1751 DIEHL ROAD
VIENNA VA 22124
MYRON BARKER
131 DONERAIL DRIVE
ATLANTA GA 30363
JOYCE BENEFIT
85 MAPLE AVENUE
MENLO PARK CA 94025
```

### Sequential Processing of Multiple Records in the DATA Step

The values in the first record are read, and the / line pointer control moves the input pointer to the second record.

**Figure 20.7** Reading the First Record of the First Observation

```
data perm.members;
 infile memdata truncover;
 input FName $ LName $ /
 Address $ 1-20 /
 City :& $10.
 State $ Zip $;
run;
```

```
>-----+-----10---+-----20
LEE ATHNOS
1215 RAINTREE CIRCLE
PHOENIX AZ 85044
HEIDIE BAKER
1751 DIEHL ROAD
VIENNA VA 22124
```

Program Data Vector

N	Fname	Lname	Address	City	State	Zip
1	LEE	ATHNOS				

The values for Address are read, and the second / line pointer control advances the input pointer to the third record.

**Figure 20.8** Reading the Second Record of the First Observation

```
data perm.members;
 infile memdata truncover;
 input FName $ LName $ /
 Address $ 1-20 /
 City :& $10.
 State $ Zip $;
run;
```

```
>-----+-----10---+-----20
LEE ATHNOS
1215 RAINTREE CIRCLE
PHOENIX AZ 85044
HEIDIE BAKER
1751 DIEHL ROAD
VIENNA VA 22124
```

Program Data Vector

N	Fname	Lname	Address	City	State	Zip
1	LEE	ATHNOS	1215 RAINTREE CIRCLE			

The values for City, State, and Zip are read. The INPUT statement is complete.

**Figure 20.9** Reading the Third Record of the First Observation

```
data perm.members;
 infile memdata truncover;
 input FName $ LName $ /
 Address $ 1-20 /
 City :& $10.
 State $ Zip $;
run;
```

```
>-----+-----10---+-----20
LEE ATHNOS
1215 RAINTREE CIRCLE
PHOENIX AZ 85044
HEIDIE BAKER
1751 DIEHL ROAD
VIENNA VA 22124
```

Program Data Vector

N	Fname	Lname	Address	City	State	Zip
1	LEE	ATHNOS	1215 RAINTREE CIRCLE	PHOENIX	AZ	85044

The values in the program data vector are written to the data set as the first observation.

**Figure 20.10** The Program Data Vector and the SAS Data Set Perm.Members

Program Data Vector

N	Fname	Lname	Address	City	State	Zip
1	LEE	ATHNOS	1215 RAINTREE CIRCLE	PHOENIX	AZ	85044

SAS Data Set Perm.Members

Fname	Lname	Address	City	State	Zip
LEE	ATHNOS	1215 RAINTREE CIRCLE	PHOENIX	AZ	85044

Control returns to the top of the DATA step. The variable values are reinitialized to missing.

**Figure 20.11** Reinitializing the Variable Values to Missing

```
data perm.members;
 infile memdata truncover;
 input Fname $ Lname $ /
 Address $ 1-20 /
 City :& $10.
 State $ Zip $;
run;
```

```
>-----+-----10-----+-----20
LEE ATHNOS
1215 RAINTREE CIRCLE
PHOENIX AZ 85044
HEIDIE BAKER
1751 DIEHL ROAD
VIENNA VA 22124
```

Program Data Vector

N	Fname	Lname	Address	City	State	Zip
2						

During the second iteration, values for Fname and Lname are read beginning in column 1 of the fourth record.

**Figure 20.12** Reading the First Record of the Second Observation

```
data perm.members;
 infile memdata truncover;
 input Fname $ Lname $ /
 Address $ 1-20 /
 City :& $10.
 State $ Zip $;
run;
```

```
>-----+-----10-----+-----20
LEE ATHNOS
1215 RAINTREE CIRCLE
PHOENIX AZ 85044
HEIDIE BAKER
1751 DIEHL ROAD
VIENNA VA 22124
```

Program Data Vector

N	Fname	Lname	Address	City	State	Zip
2	HEIDIE	BAKER				

The values for Address are read. The / line pointer control advances the input pointer to the fifth record.

**Figure 20.13** Reading the Second Record of the Second Observation

```

data perm.members;
 infile memdata trunccover;
 input FName $ LName $ /
 Address $ 1-20 /
 City :& $10.
 State $ Zip $;

run;

```

N	Fname	Lname	Address	City	State	Zip
2	HEIDIE	BAKER	1751 DIEHL ROAD			

The values for City, State, and Zip are read. The INPUT statement is complete.

**Figure 20.14** Reading the Third Record of the First Observation

```

data perm.members;
 infile memdata trunccover;
 input FName $ LName $ /
 Address $ 1-20 /
 City :& $10.
 State $ Zip $;

run;

```

N	Fname	Lname	Address	City	State	Zip
2	HEIDIE	BAKER	1751 DIEHL ROAD	VIENNA	VA	22124

The values in the program data vector are written to the data set as the second observation.

**Figure 20.15** The Program Data Vector and the SAS Data Set Perm.Members

Program Data Vector						
N	Fname	Lname	Address	City	State	Zip
2	HEIDIE	BAKER	1751 DIEHL ROAD	VIENNA	VA	22124

SAS Data Set Perm.Members						
Fname	Lname	Address	City	State	Zip	
LEE	ATHNOS	1215 RAINTREE CIRCLE	PHOENIX	AZ	85044	
HEIDIE	BAKER	1751 DIEHL ROAD	VIENNA	VA	22124	

After the data set is complete, PROC PRINT output for Perm.Members shows that each observation contains the complete information for one member.

```

proc print data=perm.members;
run;

```

**Figure 20.16** PROC PRINT Output of the Complete Perm.Members Data Set

Obs	FName	LName	Address	City	State	Zip
1	LEE	ATHNOS	1215 RAINTREE CIRCLE	PHOENIX	AZ	85044
2	HEIDIE	BAKER	1751 DIEHL ROAD	VIENNA	VA	22124
3	MYRON	BARKER	131 DONERAIL DRIVE	ATLANTA	GA	30383
4	JOYCE	BENEFIT	85 MAPLE AVENUE	MENLO PARK	CA	94025

### The Number of Records per Observation

*Note:* A raw data file must contain the same number of records for each observation.

Suppose there are only two records for the second member. Remember, the INPUT statement is set up to read three records.

```
data perm.members;
 infile memdata truncover;
 input FName $ Lname $ /
 Address $ 1-20 /
 City :& $10. State $ Zip $;
```

**Figure 20.17** Raw Data File Memdata

```
1---+---10---+---20
LEE ATHNOS
1215 RAINTREE CIRCLE
PHOENIX AZ 85044
HEIDIE BAKER
1751 DIEHL ROAD
MYRON BARKER
131 DONERAIL DRIVE
ATLANTA GA 30363
JOYCE BENEFIT
85 MAPLE AVENUE
MENLO PARK CA 94025
```

In this case, the second member's name and address are read and assigned to corresponding variables. Then the input pointer advances to the next record, as directed by the INPUT statement, and the third member's name is read as a value for City.

The INPUT statement looks for a value for State and Zip, so the input pointer advances to the next record and reads the member's address.

Before you write the INPUT statement, check to see whether the raw data file contains the same number of records for each observation. In this raw data file there are now three records for each observation.

**Figure 20.18** Verifying the Number of Records for Each Observation

```

1---+-----10---+-----20
1 LEE ATHNOS
2 1215 RAINTREE CIRCLE
3 PHOENIX AZ 85044
1 HEIDIE BAKER
2 1751 DIEHL ROAD
3 VIENNA VA 22124
1 MYRON BARKER
2 131 DONERAIL DRIVE
3 ATLANTA GA 30363
1 JOYCE BENEFIT
2 85 MAPLE AVENUE
3 MENLO PARK CA 94025

```

## Reading Multiple Records Non-Sequentially

### The #n Line Pointer Control

The #n specifies the absolute number of the line where you want to move the input pointer. The #n pointer control can read records in any order. Therefore, it must be specified before the instructions for reading values in a specific record.

The INPUT statement below first reads the values for Department and JobCode in the second record, and then reads the values for Lname and Fname in the first record. Finally, it reads the value for Salary in the third record.

```

input #2 Department $ 1-12 JobCode $ 15-19
 #1 Lname $ Fname $
 #3 Salary comma10.;

```

**Figure 20.19** The Records of the First Observation

```

1---+-----10---+-----
ABRAMS THOMAS
MARKETING SR01
$25,209.03
BARCLAY ROBERT
EDUCATION IN01
$24,435.71
COURTNEY MARK
PUBLICATIONS TW01
$24,006.16

```

### Using the #n Line Pointer Control

The raw data file Patdata contains information about the patients of a small group of general surgeons. The first three records contain a patient's name, address, city, state, and

ZIP code. The fourth record contains the patient's ID number followed by the name of the primary physician.

**Figure 20.20** Observation Records in a Raw Data File

```

1---+---10---+---20---
1 ALEX BEDWAN
2 609 WILTON MEADOW DRIVE
3 GARNER NC 27529
4 XM034 FLOYD
 ALISON BEYER
 8521 HOLLY SPRINGS ROAD
 APEX NC 27502
 XF124 LAWSON

```

Suppose you want to read each patient's information in the following order:

1. ID number (ID)
  2. first name (Fname)
  3. last name (Lname)
  4. address (Address)
  5. city (City)
  6. state (State)
  7. zip (Zip)
  8. doctor (Doctor)
- To read the values for ID in the fourth record, specify #4 before naming the variable and defining its attributes.

```

data perm.patients;
 infile patdata;
 input #4 ID $5.

```

**Figure 20.21** Specifying the ID Value in the Fourth Record of an Observation

```

1---+---10---+---20---
ALEX BEDWAN
609 WILTON MEADOW DRIVE
GARNER NC 27529
XM034 FLOYD

```

- To read the values for Fname and Lname in the first record, specify #1 before naming the variables and defining their attributes.

```

data perm.patients;
 infile patdata;
 input #4 ID $5.
 #1 Fname $ Lname $

```



**Figure 20.22** Specifying the First Record of an Observation

```
1---+----10---+----20---
ALEX BEDWAN
609 WILTON MEADOW DRIVE
GARNER NC 27529
XM034 FLOYD
```

- Use the *#n* line pointer control to move the input pointer to the second record and read the value for Address.

```
data perm.patients;
 infile patdata;
 input #4 ID $5.
 #1 Fname $ Lname $
 #2 Address $23.
```

**Figure 20.23** Specifying the Second Record of an Observation

```
1---+----10---+----20---
ALEX BEDWAN
609 WILTON MEADOW DRIVE
GARNER NC 27529
XM034 FLOYD
```

- Now move the input pointer to the third record and read the values for City, State, and Zip, in that order. In this raw data file, the values for City contain eight characters or fewer and do not contain embedded blanks. Therefore, you can use standard list input to read these values.

```
data perm.patients;
 infile patdata;
 input #4 ID $5.
 #1 Fname $ Lname $
 #2 Address $23.
 #3 City $ State $ Zip $
```

**Figure 20.24** Specifying the Third Record of an Observation

```
1---+----10---+----20---
ALEX BEDWAN
609 WILTON MEADOW DRIVE
GARNER NC 27529
XM034 FLOYD
```

- Now you need to move the input pointer down to the fourth record to read the values for Doctor, which begin in column 7. Be sure to add a semicolon at the end of the INPUT statement. A RUN statement completes the program.

```
data perm.patients;
 infile patdata;
 input #4 ID $5.
 #1 Fname $ Lname $
 #2 Address $23.
 #3 City $ State $ Zip $
 #4 @7 Doctor $6.;
run;
```

**Figure 20.25** Specifying the Doctor Value in the Fourth Record of an Observation

```

1---+---10---+---20---
ALEX BEDWAN
609 WILTON MEADOW DRIVE
GARNER NC 27529
XM034 FLOYD

```

### Execution of the DATA Step

The *#n* pointer controls in the program below cause four records to be read for each execution of the DATA step.

```

data perm.patients;
 infile patdata;
 input #4 ID $5.
 #1 Fname $ Lname $
 #2 Address $23.
 #3 City $ State $ Zip $
 #4 @7 Doctor $6.;
run;

```

The first time the DATA step executes, the first four records are read, and an observation is written to the data set.

**Figure 20.26** Raw Data File with the First Four Records Highlighted

```

1---+---10---+---20---
ALEX BEDWAN
609 WILTON MEADOW DRIVE
GARNER NC 27529
XM034 FLOYD
ALISON BEYER
8521 HOLLY SPRINGS ROAD
APEX NC 27502
XF124 LAWSON

```

During the second iteration, the next four records are read, and the second observation is written to the data set, and so on.

**Figure 20.27** Raw Data File with the Next Four Records Highlighted

```

1---+---10---+---20---
ALEX BEDWAN
609 WILTON MEADOW DRIVE
GARNER NC 27529
XM034 FLOYD
ALISON BEYER
8521 HOLLY SPRINGS ROAD
APEX NC 27502
XF124 LAWSON

```

The PROC PRINT output of the data set shows how information that was spread over several records has been condensed into one observation.

```
proc print data=perm.patients noobs;
run;
```

**Figure 20.28** PROC PRINT Output of Data Set Perm.Patients

ID	Fname	Lname	Address	City	State	Zip	Doctor
XM034	ALEX	BEDWAN	609 WILTON MEADOW DRIVE	GARNER	NC	27529	FLOYD
XF124	ALISON	BEYER	8521 HOLLY SPRINGS ROAD	APEX	NC	27502	LAWSON
XF232	LISA	BONNER	109 BRAMPTON AVENUE	CARY	NC	27511	LAWSON
XM065	GEORGE	CHESSON	3801 WOODSIDE COURT	GARNER	NC	27529	FLOYD

## Combining Line Pointer Controls

The forward slash (/) line pointer control and the #*n* line pointer control can be used together in a SAS program to read multiple records both sequentially and non-sequentially.

For example, you could use both the / line pointer control and the #*n* line pointer control to read the variables in the raw data file Patdata in the following order:

- |            |           |
|------------|-----------|
| 1. ID      | 5. City   |
| 2. Fname   | 6. State  |
| 3. Lname   | 7. Zip    |
| 4. Address | 8. Doctor |

```
data perm.patients;
 infile patdata;
 input #4 ID $5.
 #1 Fname $ Lname $ /
 Address $23. /
 City $ State $ Zip $ /
 @7 Doctor $6.;
run;
```

**Figure 20.29** Raw Data File with the First Four Records Highlighted

```
1---+---10---+---20---
ALEX BEDWAN
609 WILTON MEADOW DRIVE
GARNER NC 27529
XM034 FLOYD
ALISON BEYER
8521 HOLLY SPRINGS ROAD
APEX NC 27502
XF124 LAWSON
```

- To read the values for ID in the fourth record, specify #4 before naming the variable and defining its attributes.
- Specify #1 to move the input pointer back to the first record, where the values for Fname and Lname are read.
- Because the next record to be read is sequential, you can use the / line pointer control after the variable Lname to move the input pointer to the second record, where the value for Address is read.
- The / line pointer control in the next line directs the input pointer to the third record, where the values for City, State, and Zip are read.
- The final / line pointer control moves the input pointer back to the fourth record, where the value for Doctor is read.

**TIP** Alternatively, you can use just the #*n* line pointer control (as shown earlier in this chapter and below) to read the variables in the order shown above.

```
data perm.patients;
 infile patdata;
 input #4 ID $5.
 #1 Fname $ Lname $
 #2 Address $23.
 #3 City $ State $ Zip $
 #4 @7 Doctor $6.;
run;
```

**Figure 20.30** Raw Data File with the First Four Records Highlighted

```
1---+---10---+---20---
ALEX BEDWAN
609 WILTON MEADOW DRIVE
GARNER NC 27529
XM034 FLOYD
ALISON BEYER
8521 HOLLY SPRINGS ROAD
APEX NC 27502
XF124 LAWSON
```

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

1. Which of the following can you use to position the input pointer on a specific record?
  - a. column pointer controls
  - b. column specifications
  - c. line pointer controls
  - d. line hold specifiers
2. Which pointer control is used to read multiple records sequentially?

- a. *@n*
  - b. *+n*
  - c. */*
  - d. all of the above
3. Which pointer control can be used to read records non-sequentially?
- a. *@n*
  - b. *#n*
  - c. *+n*
  - d. */*
4. Which SAS statement correctly reads the values for Fname, Lname, Address, City, State, and Zip in order?

```
1---+-----10---+-----20---
LAWRENCE CALDWELL
1010 LAKE STREET
ANAHEIM CA 94122
RACHEL CHEVONT
3719 OLIVE VIEW ROAD
HARTFORD CT 06183
```

- a. `input Fname $ Lname $ /  
Address $20. /  
City $ State $ Zip $;`
  - b. `input Fname $ Lname $ /;  
Address $20. /;  
City $ State $ Zip $;`
  - c. `input / Fname $ Lname $  
/ Address $20.  
City $ State $ Zip $;`
  - d. `input / Fname $ Lname $;  
/ Address $20. ;  
City $ State $ Zip $;`
5. Which INPUT statement correctly reads the values for ID in the fourth record, and then returns to the first record to read the values for Fname and Lname?

```
1---+-----10---+-----20---
GEORGE CHESSON
3801 WOODSIDE COURT
GARNER NC 27529
XM065 FLOYD
JAMES COLDWELL
123-A TARBERT
APEX NC 27529
XM065 LAWSON
```

- a. `input #4 ID $5.  
#1 Fname $ Lname $;`
- b. `input #4 ID $ 1-5  
#1 Fname $ Lname $;`

c. input #4 ID \$  
#1 Fname \$ Lname \$;

d. all of the above

6. How many records are read for each execution of the following DATA step?

```
1---+-----10---+-----20---
SKIRT BLACK
COTTON
036499 $44.98
SKIRT NAVY
LINEN
036899 $51.50
DRESS RED
SILK
037299 $76.98
```

```
data spring.sportswr;
 infile newitems;
 input #1 Item $ Color $
 #3 @8 Price comma6.
 #2 Fabric $
 #3 SKU $ 1-6;

run;
```

a. one

b. two

c. three

d. four

7. Which INPUT statement correctly reads the values for City, State, and Zip?

```
1---+-----10---+-----20---
DINA FIELDS
904 MAPLE CIRCLE
DURHAM NC 27713
ELIZABETH GARRISON
1293 OAK AVENUE
CHAPEL HILL NC 27614
DAVID HARRINGTON
2426 ELMWOOD LANE
RALEIGH NC 27803
```

a. input #3 City \$ State \$ Zip \$;

b. input #3 City & \$11. State \$ Zip \$;

c. input #3 City \$11. +2 State \$2. + 2 Zip \$5.;

d. all of the above

8. Which program does not read the values in the first record as a variable named Item and the values in the second record as two variables named Inventory and Type?

```

1---+----10---+----20---
COLORED PENCILS
12 BOXES
WATERCOLOR PAINT
8 PALETTES
DRAWING PAPER
15 PADS

```

- a. 

```
data perm.supplies;
 infile instock pad;
 input Item & $16. /
 Inventory 2. Type $8.;
run;
```
  - b. 

```
data perm.supplies;
 infile instock pad;
 input Item & $16.
 / Inventory 2. Type $8.;
run;
```
  - c. 

```
data perm.supplies;
 infile instock pad;
 input #1 Item & $16.
 Inventory 2. Type $8.;
run;
```
  - d. 

```
data perm.supplies;
 infile instock pad;
 input Item & $16.
 #2 Inventory 2. Type $8.;
run;
```
9. Which INPUT statement reads the values for Lname, Fname, Department, and Salary (in that order)?

```

1---+----10---+----20---
ABRAMS THOMAS
SALES $25,209.03
BARCLAY ROBERT
MARKETING $29,180.36
COURTNEY MARK
PUBLICATIONS $24,006.16

```

- a. 

```
input #1 Lname $ Fname $ /
 Department $12. Salary comma10.;
```
  - b. 

```
input #1 Lname $ Fname $ /
 Department : $12. Salary : comma.;
```
  - c. 

```
input #1 Lname $ Fname $
 #2 Department : $12. Salary : comma.;
```
  - d. both b and c
10. Which raw data file poses potential problems when you are reading multiple records for each observation?
- a.

```
1---+----10---+----20---
LAWRENCE CALDWELL
1010 LAKE STREET
ANAHEIM CA 94122
RACHEL CHEVONT
3719 OLIVE VIEW ROAD
HARTFORD CT 06183
```

b.

```
1---+----10---+----20---
SHIRT LT BLUE SOLID
SKU 128699
$38.99
SHIRT DK BLUE STRIPE
SKU 128799
$41.99
```

c.

```
1---+----10---+----20---
MARCUS JONES
SR01 $26,134.00
MARY ROBERTSON
COURTNEY NEILS
TWO1 $28,342.00
```

d.

```
1---+----10---+----20---
CROCUS MIX
10 CASES
DAFFODIL
12 CASES
HYACINTH BLUE
8 BAGS
```



## Chapter 21

# Creating Multiple Observations from a Single Raw Data Record

---

<b>A Word About Creating Multiple Observations</b> . . . . .	<b>479</b>
<b>Reading Repeating Blocks of Data</b> . . . . .	<b>480</b>
The Basics of Reading Repeating Blocks of Data . . . . .	480
Holding the Current Record with a Line-Hold Specifier . . . . .	481
Using the Double Trailing At-Sign (@@) to Hold the Current Record . . . . .	481
Completing the DATA Step . . . . .	482
DATA Step Processing of Repeating Blocks of Data . . . . .	482
<b>Reading the Same Number of Repeating Fields</b> . . . . .	<b>487</b>
The Basics of Reading the Same Number of Repeating Fields . . . . .	487
Using the Single Trailing At-Sign (@) to Hold the Current Record . . . . .	488
Tips for More Efficient Programming . . . . .	490
Processing a DATA Step That Contains an Iterative DO Loop . . . . .	490
<b>Reading a Varying Number of Repeating Fields</b> . . . . .	<b>495</b>
The Basics of Reading a Varying Number of Repeating Fields . . . . .	495
Using the MISSEVER Option . . . . .	496
Executing SAS Statements While a Condition Is True . . . . .	496
Creating a Counter Variable . . . . .	497
Completing the DO WHILE Loop . . . . .	497
Processing a DATA Step That Has a Varying Number of Repeating Fields . . . . .	497
<b>Chapter Quiz</b> . . . . .	<b>501</b>

---

## A Word About Creating Multiple Observations

Sometimes raw data files contain data for several observations in one record. To reduce the size of an entire data file, data can be stored in this manner. This chapter covers several techniques for handling raw data, based on how the data is arranged.

## Reading Repeating Blocks of Data

### The Basics of Reading Repeating Blocks of Data

Consider this example data. Each record in the file Tempdata contains three blocks of data. Each block contains a date followed by the day's high temperature in a small city located in the southern United States.

**Figure 21.1** Raw Data File Tempdata

Raw Data File Tempdata					
1----	----	10----	----	20----	----
01APR90	68	02APR90	67	03APR90	70
04APR90	74	05APR90	72	06APR90	73
07APR90	71	08APR90	75	09APR90	76
10APR90	78	11APR90	70	12APR90	69
13APR90	71	14APR90	72	15APR90	74
16APR90	73	17APR90	71	18APR90	75
19APR90	75	20APR90	73	21APR90	75
22APR90	77	23APR90	78	24APR90	80
25APR90	78	26APR90	77	27APR90	79
28APR90	81	29APR90	81	30APR90	84

You could write a DATA step that reads each record and creates three different Date and Temp variables.

**Figure 21.2** Three Date and Temp Variables

SAS Data Set					
Date1	Temp1	Date2	Temp2	Date3	Temp3
11048	68	11049	67	11050	70

Alternatively, you could create a separate observation for each block of data in a record. This data set is better structured for analysis and reporting with SAS procedures.

**Figure 21.3** Separate Observations for Each Block of Data in a Record

SAS Data Set	
Date	HighTemp
11048	68
11049	67
11050	70

### Holding the Current Record with a Line-Hold Specifier

You need to hold the current record so that the INPUT statement can read, and SAS can generate output from, repeated blocks of data in the same record. This is easily accomplished by using a line-hold specifier in the INPUT statement.

SAS provides two line-hold specifiers.

- The trailing at-sign (@) holds the input record for the execution of the next INPUT statement.
- The double trailing at-sign (@@) holds the input record for the execution of the next INPUT statement, even across iterations of the DATA step.

The term *trailing* indicates that the @ or @@ must be the last item that is specified in the INPUT statement. Here is an example.

```
input Name $20. @; or input Name $20. @@;
```

### Using the Double Trailing At-Sign (@@) to Hold the Current Record

Normally, each time a DATA step executes, the INPUT statement reads the next record. But when you use the trailing @@, the INPUT statement continues reading from the same record.

Here are several facts about the double trailing at-sign (@@):

- It works like the trailing @ except it holds the data line in the input buffer across multiple executions of the DATA step.
- It typically is used to read multiple SAS observations from a single data line.
- It should not be used with the @ pointer control, with column input, or with the MISSOVER option.

A record that is held by the double trailing at-sign (@@) is not released until either of the following events occurs:

- The input pointer moves past the end of the record. Then the input pointer moves down to the next record.
- An INPUT statement that has no trailing @ executes.

```
input ID $ @@;
.
.
input Department 5.;
```

The following example requires only one INPUT statement to read the values for Date and HighTemp, but the INPUT statement must execute three times for each record.

The INPUT statement reads a block of values for Date and HighTemp, and holds the current record by using the trailing @@. The values in the program data vector are written to the data set as an observation, and control returns to the top of the DATA step.

```
data perm.april10;
 infile tempdata;
 input Date : date. HighTemp @@;
```



**Example: How the DATA Step Processes Repeated Blocks of Data**

As the execution phase begins, the input pointer rests on column 1 of record 1.

**Figure 21.7** Input Pointer on Column 1 of Record 1

Raw Data File Tempdata		
>V-----10-----20-----30--		
01APR10 68	02APR10 67	03APR10 70
04APR10 74	05APR10 72	06APR10 73
07APR10 71	08APR10 75	09APR10 76
10APR10 78	11APR10 70	12APR10 69
13APR10 71	14APR10 72	15APR10 74

Program Data Vector		
N	Date	HighTemp
1	.	.

During the first iteration of the DATA step, the first block of values for Date and HighTemp are read into the program data vector.

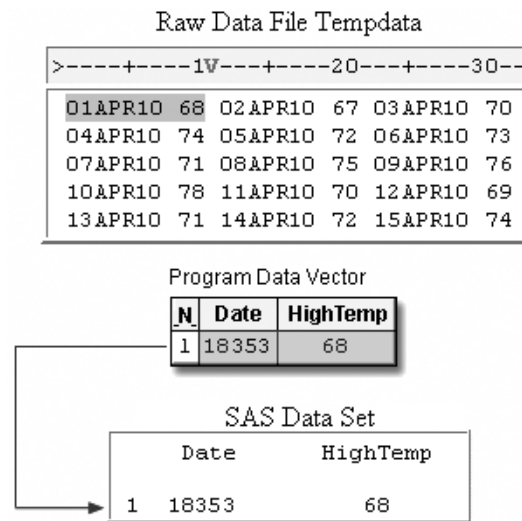
**Figure 21.8** Reading the First Block of Values

Raw Data File Tempdata		
>-----1V-----20-----30--		
01APR10 68	02APR10 67	03APR10 70
04APR10 74	05APR10 72	06APR10 73
07APR10 71	08APR10 75	09APR10 76
10APR10 78	11APR10 70	12APR10 69
13APR10 71	14APR10 72	15APR10 74

Program Data Vector		
N	Date	HighTemp
1	18353	68

The first observation is written to the data set.

**Figure 21.9** Control Returns to the Top of the DATA Step

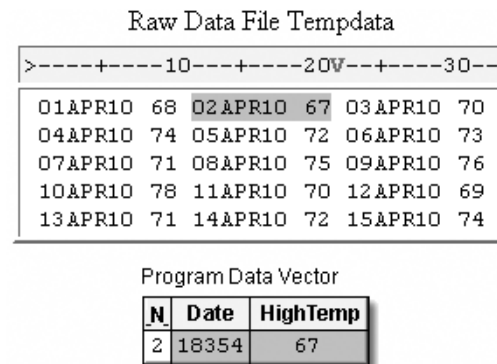
Control returns to the top of the DATA step, and the values are reset to missing.

**Figure 21.10** Reset Values

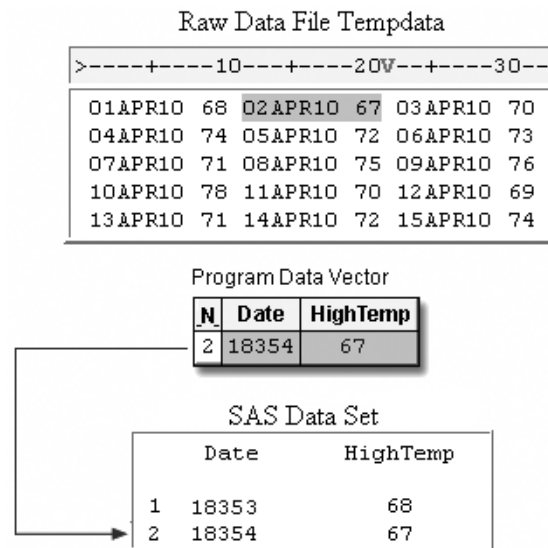
Program Data Vector

N	Date	HighTemp
2	.	.

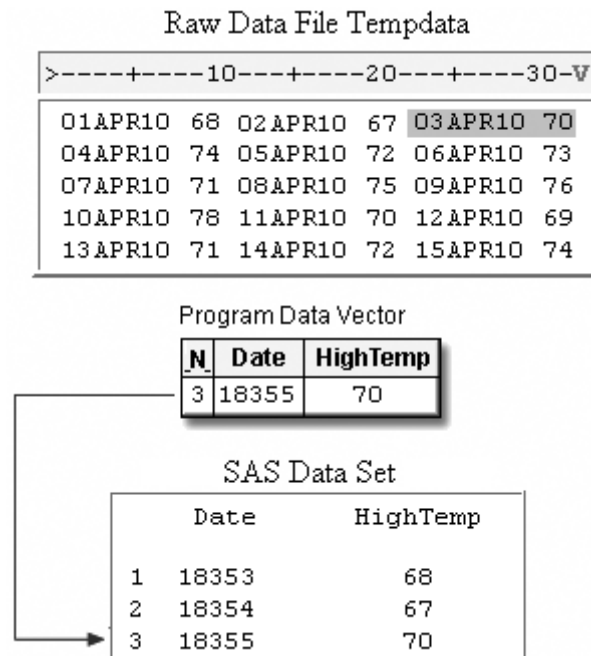
During the second iteration, the @@ prevents the input pointer from moving down to the next record. Instead, the INPUT statement reads the second block of values for Date and HighTemp from the first record.

**Figure 21.11** Reading the Second Block of Values from the First Record

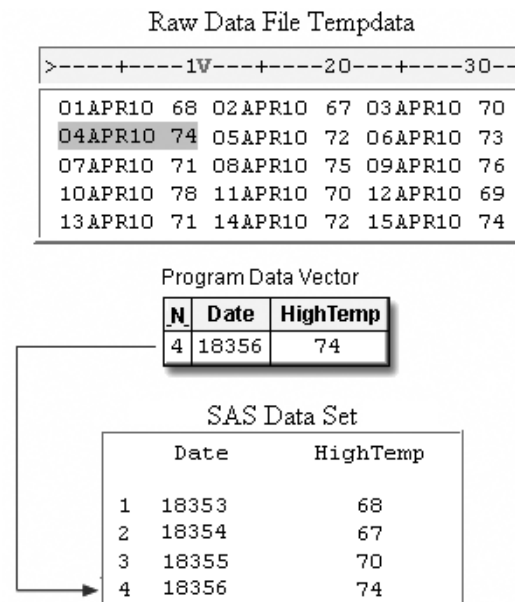
The second observation is written to the data set, and control returns to the top of the DATA step.

**Figure 21.12** Writing the Second Observation to the Data Set

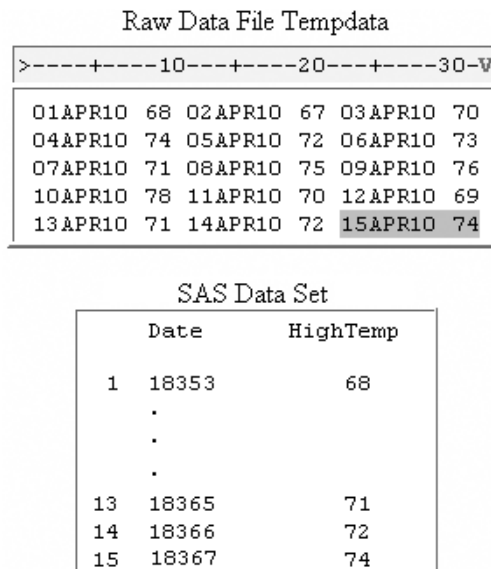
During the third iteration, the last block of values is read and written to the data set as the third observation.

**Figure 21.13** Writing the Third Observation to the Data Set

During the fourth iteration, the first block of values in the second record is read and written as the fourth observation.

**Figure 21.14** Writing the Fourth Observation to the Data Set

The execution phase continues until the last block of data is read.

**Figure 21.15** Writing the Last Observation to the Data Set

You can display the data set with the PRINT procedure.

```
proc print data=perm.april10;
run;
```



**Figure 21.16** PROC PRINT Output of the Data Set.

Date	High Temp
01APR2010	68
02APR2010	67
03APR2010	70
04APR2010	74
05APR2010	72
06APR2010	73
07APR2010	71
08APR2010	75
09APR2010	76
10APR2010	78
11APR2010	70
12APR2010	69
13APR2010	71
14APR2010	72
15APR2010	74

---

## Reading the Same Number of Repeating Fields

### *The Basics of Reading the Same Number of Repeating Fields*

Each record in the file Data97 contains a sales representative's ID number, followed by four repeating fields that represent his or her quarterly sales totals for 1997. You want to pair each employee ID number with one quarterly sales total to produce a single observation. Four observations are generated from each record.

**Figure 21.17** Multiple Fields for the Same ID

Raw Data File

1	---	+	---	10	---	+	---	20	---	+	---	30	---	+	---	40
0734	1,323.34	2,472.85	3,276.65	5,345.52												
0943	1,908.34	2,560.38	3,472.09	5,290.86												
1009	2,934.12	3,308.41	4,176.18	7,581.81												
1043	1,295.38	5,980.28	8,876.84	6,345.94												
1190	2,189.84	5,023.57	2,794.67	4,243.35												
1382	3,456.34	2,065.83	3,139.08	6,503.49												
1734	2,345.83	3,423.32	1,034.43	1,942.28												

ID	Quarter	Sales
0734	1	1323.34
0734	2	2472.85
0734	3	3276.65
0734	4	5345.52
0943	1	1908.34
0943	2	2560.38
0943	3	3472.09
0943	4	5290.86

To accomplish this, you must execute the DATA step once for each record, repetitively reading and writing values in one iteration.

This means that a DATA step must do the following:

- read the value for ID and hold the current record
- create a new variable named Quarter to identify the fiscal quarter for each sales figure
- read a new value for Sales and write the values to the data set as an observation
- continue reading a new value for Sales and writing values to the data set three more times

### Using the Single Trailing At-Sign (@) to Hold the Current Record

To read the value for ID and hold the record so that subsequent values for Sales can be read, submit the following code.

```
data perm.sales07;
 infile data07;
 input ID $
```

**Figure 21.18** Holding a Record

Raw Data File					
1---V---10---+---20---+---30---+---40					
0734	1,323.34	2,472.85	3,276.65	5,345.52	
0943	1,908.34	2,560.38	3,472.09	5,290.86	
1009	2,934.12	3,308.41	4,176.18	7,581.81	

Recall that, in general, the trailing @@ holds the current record across multiple iterations of the DATA step.

However, in this case, you want to hold the record with the trailing @, so that a second INPUT statement can read the multiple sales values from a single record within the same iteration of the DATA step. Like the trailing @@, the single trailing @ can do the following:

- It enables the next INPUT statement to continue reading from the same record.
- It releases the current record when a subsequent INPUT statement executes without a line-hold specifier.

To distinguish between the trailing @@ and the trailing @, remember the following:

- The double trailing at-sign (@@) holds a record across multiple iterations of the DATA step until the end of the record is reached.
- The single trailing at-sign (@) releases a record when control returns to the top of the DATA step.

In this example, the first INPUT statement reads the value for ID and uses the trailing @ to hold the current record for the next INPUT statement in the DATA step.

```
data perm.sales07;
 infile data07;
 input ID $ @;
 input Sales : comma. @;
output;
```

**Figure 21.19** Reading the Value for ID

Raw Data File					
1---V---10---+---20---+---30---+---40					
0734	1,323.34	2,472.85	3,276.65	5,345.52	
0943	1,908.34	2,560.38	3,472.09	5,290.86	
1009	2,934.12	3,308.41	4,176.18	7,581.81	

The second INPUT statement reads a value for Sales and holds the record. The COMMAw.d informat in the INPUT statement reads the numeric value for Sales and removes the embedded commas. An OUTPUT statement writes the observation to the SAS data set, and the DATA step continues processing.

*Note:* The COMMAw.d informat does not specify a w value. Remember that list input reads values until the next blank is detected. The default length of numeric variables is 8 bytes, so you do not need to specify a w value to determine the length of a numeric variable.

When all of the repeating fields have been read and written to output, control returns to the top of the DATA step, and the record is released.

```

data perm.sales07;
 infile data07;
 input ID $ @;
 input Sales : comma. @;
 output;
 input Sales : comma. @;
 output;
 input Sales : comma. @;
 output;
 input Sales : comma. @;
 output;
run;

```

**Figure 21.20** Reading the Value for Sales

Raw Data File

1	10	20	30	40
0734	1,323.34	2,472.85	3,276.65	5,345.52
0943	1,908.34	2,560.38	3,472.09	5,290.86
1009	2,934.12	3,308.41	4,176.18	7,581.81

### Tips for More Efficient Programming

Each record contains four different values for the variable Sales. Therefore, the INPUT statement must execute four times. Rather than writing four INPUT statements, you can execute one INPUT statement repeatedly in an iterative DO loop.

Each time the loop executes, you need to write the values for ID, Quarter, and Sales as an observation to the data set. This is easily accomplished by using the OUTPUT statement.

```

data perm.sales07;
 infile data07;
 input ID $ @;
 do Quarter=1 to 4;
 input Sales : comma. @;
 output;
 end;
run;

```

By default, every DATA step contains an implicit OUTPUT statement at the end of the step. Placing an explicit OUTPUT statement in a DATA step overrides the automatic output, and SAS adds an observation to a data set only when the explicit OUTPUT statement is executed.

### Processing a DATA Step That Contains an Iterative DO Loop

Now that the program is complete, here is how SAS processes a DATA step that contains an iterative DO loop.

```

data perm.sales07;
 infile data07;
 input ID $ @;
 do Quarter=1 to 4;

```

```

input Sales : comma. @;
output;
end;
run;

```

During the first iteration, the value for ID is read and Quarter is initialized to 1 as the loop begins to execute.

**Figure 21.21** Reading the Value for ID and Initializing Quarter

Raw Data File				
>-----V-----10-----+-----20-----+-----30-----+-----40-----				
0734	1,323.34	2,472.85	3,276.65	5,345.52
0943	1,908.34	2,560.38	3,472.09	5,290.86
1009	2,934.12	3,308.41	4,176.18	7,581.81

Program Data Vector			
N	ID	Quarter	Sales
1	0734	1	.

The INPUT statement reads the first repeating field and assigns the value to Sales in the program data vector. The @ holds the current record.

**Figure 21.22** Results of the INPUT Statement

Raw Data File				
>-----+-----10-----V+-----20-----+-----30-----+-----40-----				
0734	1,323.34	2,472.85	3,276.65	5,345.52
0943	1,908.34	2,560.38	3,472.09	5,290.86
1009	2,934.12	3,308.41	4,176.18	7,581.81

Program Data Vector			
N	ID	Quarter	Sales
1	0734	1	1323.34

The OUTPUT statement writes the values in the program data vector to the data set as the first observation.

**Figure 21.23** Results of the OUTPUT Statement

Raw Data File				
>-----+-----10-----V+-----20-----+-----30-----+-----40-----				
0734	1,323.34	2,472.85	3,276.65	5,345.52
0943	1,908.34	2,560.38	3,472.09	5,290.86
1009	2,934.12	3,308.41	4,176.18	7,581.81

Program Data Vector			
N	ID	Quarter	Sales
1	0734	1	1323.34

SAS Data Set			
	ID	Quarter	Sales
1	0734	1	1323.34

The END statement indicates the bottom of the loop, but control returns to the DO statement, not to the top of the DATA step. Now the value of Quarter is incremented to 2.

**Figure 21.24** Results of the END Statement

Raw Data File				
>-----10-----V-----20-----30-----40-----				
0734	1,323.34	2,472.85	3,276.65	5,345.52
0943	1,908.34	2,560.38	3,472.09	5,290.86
1009	2,934.12	3,308.41	4,176.18	7,581.81

Program Data Vector			
N	ID	Quarter	Sales
1	0734	2	1323.34

SAS Data Set			
ID	Quarter	Sales	
1 0734	1	1323.34	

The INPUT statement executes again, reading the second repeating field and storing the value for Sales in the program data vector.

**Figure 21.25** Results of the Second Reading of the INPUT Statement

Raw Data File				
>-----10-----+-----20-V-----30-----40-----				
0734	1,323.34	2,472.85	3,276.65	5,345.52
0943	1,908.34	2,560.38	3,472.09	5,290.86
1009	2,934.12	3,308.41	4,176.18	7,581.81

Program Data Vector			
N	ID	Quarter	Sales
1	0734	2	2472.85

SAS Data Set			
ID	Quarter	Sales	
1 0734	1	1323.34	

The OUTPUT statement writes the values in the program data vector as the second observation.

**Figure 21.26** Results of the Second Reading of the OUTPUT Statement

Raw Data File

>-----10-----20-V-----30-----40-				
0734	1,323.34	2,472.85	3,276.65	5,345.52
0943	1,908.34	2,560.38	3,472.09	5,290.86
1009	2,934.12	3,308.41	4,176.18	7,581.81

Program Data Vector

N	ID	Quarter	Sales
1	0734	2	2472.85

SAS Data Set

	ID	Quarter	Sales
1	0734	1	1323.34
2	0734	2	2472.85

The loop continues executing while the value for Quarter is 3, and then 4. In the process, the third and fourth observations are written.

**Figure 21.27** Writing the Third and Fourth Observations

Raw Data File

>-----10-----20-----30-----4V-				
0734	1,323.34	2,472.85	3,276.65	5,345.52
0943	1,908.34	2,560.38	3,472.09	5,290.86
1009	2,934.12	3,308.41	4,176.18	7,581.81

Program Data Vector

N	ID	Quarter	Sales
1	0734	4	5345.52

SAS Data Set

	ID	Quarter	Sales
1	0734	1	1323.34
2	0734	2	2472.85
3	0734	3	3276.65
4	0734	4	5345.52

After the fourth observation is written, Quarter is incremented to 5 at the bottom of the DO loop and control returns to the top of the loop. The loop does not execute again because the value of Quarter is now greater than 4.

Raw Data File				
>-----10-----20-----30-----4V-				
0734	1,323.34	2,472.85	3,276.65	5,345.52
0943	1,908.34	2,560.38	3,472.09	5,290.86
1009	2,934.12	3,308.41	4,176.18	7,581.81

Program Data Vector

N	ID	Quarter	Sales
1	0734	5	5345.52

SAS Data Set

	ID	Quarter	Sales
1	0734	1	1323.34
2	0734	2	2472.85
3	0734	3	3276.65
4	0734	4	5345.52

Control returns to the top of the DATA step, and the input pointer moves to column 1 of the next record. The variable values in the program data vector are reset to missing.

**Figure 21.28** Returning Control to the Top of the DATA Step

Raw Data File				
>V---+-----10-----20-----30-----40-				
0734	1,323.34	2,472.85	3,276.65	5,345.52
0943	1,908.34	2,560.38	3,472.09	5,290.86
1009	2,934.12	3,308.41	4,176.18	7,581.81

Program Data Vector

N	ID	Quarter	Sales
2	.	.	.

SAS Data Set

	ID	Quarter	Sales
1	0734	1	1323.34
2	0734	2	2472.85
3	0734	3	3276.65
4	0734	4	5345.52

You can display the data set with the PRINT procedure.

```
proc print data=perm.sales07;
run;
```



**Figure 21.29** Partial Output of PROC PRINT

Obs	ID	Quarter	Sales
1	0734	1	1323.34
2	0734	2	2472.85
3	0734	3	3276.65
4	0734	4	5345.52
5	0943	1	1908.34
6	0943	2	2560.38
7	0943	3	3472.09
8	0943	4	5290.86
9	1009	1	2934.12
10	1009	2	3308.41
11	1009	3	4176.18
12	1009	4	7581.81

---

## Reading a Varying Number of Repeating Fields

### *The Basics of Reading a Varying Number of Repeating Fields*

Suppose that some of the employees quit at various times. Their records might not contain sales totals for the second, third, or fourth quarters. These records contain a variable number of repeating fields.

**Figure 21.30** Raw Data File Data97 Showing Empty Records

```

Raw Data File Data97
1---+---10---+---20---+---30---+---40
1824 1,323.34 2,472.85
1943 1,908.34
2046 1,423.52 1,673.46 3,276.65
2063 2,345.34 2,452.45 3,523.52 2,983.01

```

The DATA step that you just wrote does not work with a variable number of repeating fields because now the value of Quarter is not constant for every record.

```

data perm.sales07;
 infile data07;
 input ID $ @;
 do Quarter=1 to 4;
 input Sales : comma. @;
 output;

```

```
end;
run;
```

### Using the MISSOVER Option

You can adapt the DATA step to accommodate a varying number of Sales values.

Like the previous example with the same number of repeating fields, your DATA step must read the same record repeatedly. However, you need to prevent the input pointer from moving to the next record when there are missing Sales values.

You can use the MISSOVER option in an INFILE statement to prevent SAS from reading the next record when missing values are encountered at the end of a record. Essentially, records that have a varying number of repeating fields are records that contain missing values. Therefore, you need to specify the MISSOVER option here as well.

Because there is at least one value for the repeating field, Sales, in each record, the first INPUT statement reads both the value for ID and the first Sales value for each record. The trailing @ holds the record so that any subsequent repeating fields can be read.

```
data perm.sales07;
 infile data07 missover;
 input ID $ Sales : comma. @;
```

**Figure 21.31** Holding a Record

Raw Data File Data97

1----	10--	20----	30----	40
1824	1,323.34	2,472.85		
1943	1,908.34			
2046	1,423.52	1,673.46	3,276.65	
2063	2,345.34	2,452.45	3,523.52	2,983.01

*Note:* SAS provides several options to control reading past the end of a line. Recall that the MISSOVER option can be used to set the remaining INPUT statement variables to missing values if the pointer reaches the end of a record. You can also use other options such as the TRUNCOVER option, which reads column or formatted input when the last variable that is read by the INPUT statement contains varying-length data. The TRUNCOVER option assigns the contents of the input buffer to a variable when the field is shorter than expected.

Other related options include FLOWOVER (the default), STOPOVER, and SCANOVER.

### Executing SAS Statements While a Condition Is True

To execute SAS statements while a condition is true, use a DO WHILE statement instead of the iterative DO statement, enclosing the expression in parentheses. In the example below, the DO WHILE statement executes while the value of Sales is not equal to a missing value (which is represented by a period).

```
data perm.sales07;
 infile data07 missover;
 input ID $ Sales : comma. @;
 do while (sales ne .);
```

### Creating a Counter Variable

The DO WHILE statement does not create an index variable, but you can create your own “counter” variable. You can use a sum statement to increment the value of the counter variable each time the DO WHILE loop executes.

In the example below, the assignment statement that precedes the loop creates the counter variable Quarter and assigns it an initial value of zero. Each time the DO WHILE loop executes, the sum statement increments the value of Quarter by one.

```
data perm.sales08;
 infile data08 missover;
 input ID $ Sales : comma. @;
 Quarter=0;
 do while (sales ne .);
 quarter+1;
```

### Completing the DO WHILE Loop

You need an OUTPUT statement to write the current observation to the data set. Then, another INPUT statement reads the next value for Sales and holds the record. You complete the DO WHILE loop with an END statement.

```
data perm.sales08;
 infile data08 missover;
 input ID $ Sales : comma. @;
 Quarter=0;
 do while (sales ne .);
 quarter+1;
 output;
 input sales : comma. @;
 end;
run;
```

### Processing a DATA Step That Has a Varying Number of Repeating Fields

This example uses the following DATA step:

```
data perm.sales08;
 infile data08 missover;
 input ID $ Sales : comma. @;
 Quarter=0;
 do while (sales ne .);
 quarter+1;
 output;
 input sales : comma. @;
 end;
run;
```

During the first iteration of the DATA step, values for ID and Sales are read. Quarter is initialized to 0.

**Figure 21.32** Initializing the Value of Quarter to 0

Raw Data File				
>-----+-----10---+-----20---+-----30---+-----40-				
1824	1,323.34	2,472.85		
1943	1,908.34			
2046	1,423.52	1,673.46	3,276.65	
2063	2,345.34	2,452.45	3,523.52	2,983.01

Program Data Vector			
N	ID	Sales	Quarter
1	1824	1323.34	0

The DO WHILE statement checks to see whether Sales has a value, which it does, so the other statements in the DO loop execute. The Value of Quarter is incremented by 1 and the current observation is written to the data set.

**Figure 21.33** Executing the DO Loop

Raw Data File				
>-----+-----10---+-----20---+-----30---+-----40-				
1824	1,323.34	2,472.85		
1943	1,908.34			
2046	1,423.52	1,673.46	3,276.65	
2063	2,345.34	2,452.45	3,523.52	2,983.01

Program Data Vector			
N	ID	Sales	Quarter
1	1824	1323.34	1

SAS Data Set			
	ID	Sales	Quarter
1	1824	1323.34	1

The INPUT statement reads the next value for Sales, the end of the loop is reached, and control returns to the DO WHILE statement.

**Figure 21.34** Returning Control to the DO WHILE Statement

Raw Data File				
>-----+-----10---+-----20---+-----30---+-----40-				
1824	1,323.34	2,472.85		
1943	1,908.34			
2046	1,423.52	1,673.46	3,276.65	
2063	2,345.34	2,452.45	3,523.52	2,983.01

Program Data Vector			
N	ID	Sales	Quarter
1	1824	2472.85	1

SAS Data Set			
	ID	Sales	Quarter
1	1824	1323.34	1

The condition is checked and Sales still has a value, so the loop executes again.

Figure 21.35 Executing the Loop Again

Raw Data File				
>-----10-----+-----20-V-----30-----+-----40-----				
1824	1,323.34	2,472.85		
1943	1,908.34			
2046	1,423.52	1,673.46	3,276.65	
2063	2,345.34	2,452.45	3,523.52	2,983.01

Program Data Vector			
N	ID	Sales	Quarter
1	1824	2472.85	1

SAS Data Set			
	ID	Sales	Quarter
1	1824	1323.34	1

Quarter is incremented to 2, and the values in the program data vector are written as the second observation.

Raw Data File					
>-----10-----+-----20-V-----30-----+-----40-----					
1824	1,323.34	2,472.85			
1943	1,908.34				
2046	1,423.52	1,673.46	3,276.65		
2063	2,345.34	2,452.45	3,523.52	2,983.01	

Program Data Vector			
N	ID	Sales	Quarter
1	1824	2472.85	2

SAS Data Set			
	ID	Sales	Quarter
1	1824	1323.34	1
2	1824	2472.85	2

The INPUT statement executes again. The MISOVER option prevents the input pointer from moving to the next record in search of another value for Sales. Therefore, Sales receives a missing value.

Raw Data File				
>-----+-----10-----+-----20-V-----30-----+-----40-----				
1824	1,323.34	2,472.85		
1943	1,908.34			
2046	1,423.52	1,673.46	3,276.65	
2063	2,345.34	2,452.45	3,523.52	2,983.01

Program Data Vector			
N	ID	Sales	Quarter
1	1824	.	2

SAS Data Set			
	ID	Sales	Quarter
1	1824	1323.34	1
2	1824	2472.85	2

The end of the loop is reached, and control returns to the DO WHILE statement. Because the condition is now false, the statements in the loop are not executed and the values in the PDV are not written to output.

**Figure 21.36** Returning Control to the DO WHILE Statement

Raw Data File				
>-----+-----10-----+-----20-V-----30-----+-----40-----				
1824	1,323.34	2,472.85		
1943	1,908.34			
2046	1,423.52	1,673.46	3,276.65	
2063	2,345.34	2,452.45	3,523.52	2,983.01

Program Data Vector

N	ID	Sales	Quarter
1	1824	.	2

SAS Data Set

	ID	Sales	Quarter
1	1824	1323.34	1
2	1824	2472.85	2

Instead, control returns to the top of the DATA step, the values in the program data vector are reset to missing, and the INPUT statement reads the next record. The DATA step continues executing until the end of the file.

**Figure 21.37** Returning Control to the Top of the DATA Step

Raw Data File				
>V-----10-----20-----30-----40-----				
1824	1,323.34	2,472.85		
1943	1,908.34			
2046	1,423.52	1,673.46	3,276.65	
2063	2,345.34	2,452.45	3,523.52	2,983.01

Program Data Vector

N	ID	Sales	Quarter
2	.	.	.

SAS Data Set

	ID	Sales	Quarter
1	1824	1323.34	1
2	1824	2472.85	2

PROC PRINT output for the data set shows a varying number of observations for each employee.

```
proc print data=perm.sales08;
run;
```

Figure 21.38 PROC PRINT Output

Obs	ID	Sales	Quarter
1	1824	1323.34	1
2	1824	2472.85	2
3	1943	1908.34	1
4	2046	1423.52	1
5	2046	1673.46	2
6	2046	3276.65	3
7	2063	2345.34	1
8	2063	2452.45	2
9	2063	3523.52	3
10	2063	2983.01	4

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

- What is true for the double trailing at-sign (@@)?
  - It enables the next INPUT statement to read from the current record across multiple iterations of the DATA step.
  - It must be the last item specified in the INPUT statement.
  - It is released when the input pointer moves past the end of the record.
  - All of the above.
- A record that is being held by a single trailing at-sign (@) is automatically released when one of the following actions occurs.
  - The input pointer moves past the end of the record.
  - The next iteration of the DATA step begins.
  - Another INPUT statement that has an @ executes.
  - Another value is read from the observation.
- Which SAS program correctly creates a separate observation for each block of data?

```
1---+---10---+---20---+---30---+---40---+
1001 apple 1002 banana 1003 cherry
1004 guava 1005 kiwi 1006 papaya
1007 pineapple 1008 raspberry 1009 strawberry
```

- data perm.produce;
   
infile fruit;

- ```

input Item $ Variety : $10.;
run;

b. data perm.produce;
   infile fruit;
   input Item $ Variety : $10. @;
run;

c. data perm.produce;
   infile fruit;
   input Item $ Variety : $10. @@;
run;

d. data perm.produce;
   infile fruit @@;
   input Item $ Variety : $10.;
run;

```

4. Which SAS program reads the values for ID and holds the record for each value of Quantity, so that three observations are created for each record?

| | | | | | | |
|------|--------|--------|--------|----|-----|----|
| 1 | --- | 10 | --- | 20 | --- | 30 |
| 2101 | 21,208 | 19,047 | 22,890 | | | |
| 2102 | 18,775 | 20,214 | 22,654 | | | |
| 2103 | 19,763 | 22,927 | 21,862 | | | |

- ```

a. data work.sales;
 infile unitsold;
 input ID $;
 do week=1 to 3;
 input Quantity : comma.;
 output;
 end;
run;

b. data work.sales;
 infile unitsold;
 input ID $ @@;
 do week=1 to 3;
 input Quantity : comma.;
 output;
 end;
run;

c. data work.sales;
 infile unitsold;
 input ID $ @;
 do week=1 to 3;
 input Quantity : comma.;
 output;
 end;
run;

d. data work.sales;
 infile unitsold;
 input ID $ @;
 do week=1 to 3;
 input Quantity : comma. @;
 output;
 end;
run;

```



```
end;
run;
```

5. Which SAS statement repetitively executes several statements when the value of an index variable named Count ranges from 1 to 50, incremented by 5?
  - a. `do count=1 to 50 by 5;`
  - b. `do while count=1 to 50 by 5;`
  - c. `do count=1 to 50 + 5;`
  - d. `do while (count=1 to 50 + 5);`
6. Which option below, when used in a DATA step, writes an observation to the data set after each value for Activity has been read?
  - a. 

```
do choice=1 to 3;
 input Activity : $10. @;
 output;
end;
run;
```
  - b. 

```
do choice=1 to 3;
 input Activity : $10. @;
end;
output;
run;
```
  - c. 

```
do choice=1 to 3;
 input Activity : $10. @;
end;
run;
```
  - d. both a and b
7. Which SAS statement repetitively executes several statements while the value of Cholesterol is greater than 200?
  - a. `do cholesterol > 200;`
  - b. `do cholesterol gt 200;`
  - c. `do while (cholesterol > 200);`
  - d. `do while cholesterol > 200;`
8. Which choice below is an example of a sum statement?
  - a. `totalpay=1;`
  - b. `totalpay+1;`
  - c. `totalpay*1;`
  - d. `totalpay by 1;`
9. Which program creates the SAS data set Perm.Topstore from the raw data file shown below?

	1---+---10---+---20---+---30---+
1001	77,163.19 76,804.75 74,384.27
1002	76,612.93 81,456.34 82,063.97
1003	82,185.16 79,742.33

**Figure 21.39** Output from PROC PRINT

Store	Sales	Month
1001	77163.19	1
1001	76804.75	2
1001	74384.27	3
1002	76612.93	1
1002	81456.34	2
1002	82063.97	3
1003	82185.16	1
1003	79742.33	2

- a. 

```
data perm.topstores;
 infile sales98 missover;
 input Store Sales : comma. @;
 do while (sales ne .);
 month + 1;
 output;
 input sales : comma. @;
 end;
run;
```
- b. 

```
data perm.topstores;
 infile sales98 missover;
 input Store Sales : comma. @;
 do while (sales ne .);
 Month=0;
 month + 1;
 output;
 input sales : comma. @;
 end;
run;
```
- c. 

```
data perm.topstores;
 infile sales98 missover;
 input Store Sales : comma.
 Month @;
 do while (sales ne .);
 month + 1;
 input sales : comma. @;
 end;
 output;
run;
```
- d. 

```
data perm.topstores;
 infile sales98 missover;
 input Store Sales : comma. @;
 Month=0;
 do while (sales ne .);
 month + 1;
 output;
 input sales : comma. @;
```

```

end;
run;

```

10. How many observations are produced by the DATA step that reads this external file?

```

1---+---10---+---20---+---30---+---40
01 CHOCOLATE VANILLA RASPBERRY
02 VANILLA PEACH
03 CHOCOLATE
04 RASPBERRY PEACH CHOCOLATE
05 STRAWBERRY VANILLA CHOCOLATE

```

```

data perm.choices;
 infile icecream missover;
 input Day $ Flavor : $10. @;
 do while (flavor ne ' ');
 output; input flavor : $10. @;
 end;
run;

```

- a. 3
- b. 5
- c. 12
- d. 15



## Chapter 22

# Conditioning Raw Data Files

---

<b>A Word About Conditioning Raw Data Files</b> . . . . .	<b>507</b>
<b>Creating One Observation per Detail Record</b> . . . . .	<b>508</b>
The Basics of Creating One Observation per Detail Record . . . . .	508
Retaining the Values of Variables . . . . .	509
Conditionally Executing SAS Statements . . . . .	509
Reading a Detail Record . . . . .	510
Dropping Variables . . . . .	511
Processing a DATA Step That Creates One Observation per Detail Record . . . .	511
Displaying Your Results . . . . .	514
<b>Creating One Observation per Header Record</b> . . . . .	<b>515</b>
The Basics of Creating One Observation per Header Record . . . . .	515
DO Group Actions for Header Records . . . . .	516
Reading Detail Records . . . . .	517
Determining the End of the External File . . . . .	518
The INFILE Statement . . . . .	519
<b>Processing a DATA Step That Creates One Observation per Header Record</b> . . .	<b>520</b>
<b>Chapter Quiz</b> . . . . .	<b>522</b>

---

## A Word About Conditioning Raw Data Files

Raw data files can be hierarchical in structure, consisting of a header record and one or more detail records. Typically, each record contains a field that identifies the record type.

Raw Data File			
1	----	10	-----20
P	1095		
C	01-08-89	\$45.0	
C	01-17-89	\$37.5	
P	1096		
C	01-09-89	\$156.5	
P	1097		
C	01-02-89	\$109.0	
P	1099		
C	01-03-89	\$45.0	
C	01-05-89	\$45.0	
P	1201		
C	01-05-89	\$37.0	
C	01-10-89	\$45.0	

In the raw data file above, the P indicates a header record that contains a patient's ID number. The C indicates a detail record that contains the date of the patient's appointment and the charges that the patient has incurred. This book covers two methods of building a SAS data set from a hierarchical file. You can build a SAS data set from a hierarchical file by creating one observation per detail record, retaining the patient ID from the header record. You can also build a SAS data set from a hierarchical file by creating one observation per header record and combining the information from detail records into summary variables. These are powerful features of SAS that enable complex raw data files to be converted into SAS data sets for statistical analysis.

---

## Creating One Observation per Detail Record

### *The Basics of Creating One Observation per Detail Record*

In order to create one observation per detail record, it is necessary to distinguish between header and detail records. Having a field that identifies the type of the record makes this task easier.

In the partial raw data file Census shown below, H indicates a header record that contains a street address, and P indicates a detail record that contains information about a person who lives at that address.

Raw Data File

```

1---+---10---+---
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST
P THOMAS H 79 M
P WALTER S 46 M
P ALICE A 42 F
P MARYANN A 20 F
P JOHN S 16 M
H 325A S. MAIN ST

```

### Retaining the Values of Variables

While writing the DATA step to read this file, remember that you want to keep the header record as a part of each observation until the next header record is encountered. To do this, you need to use a RETAIN statement to retain the values for Address across iterations of the DATA step.

```

data perm.people;
 infile census;
 retain Address;

```

```

1---+---10---+---
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F

```

Next, read the first field in each record, which identifies the record's type. Use the @ line-hold specifier to hold the current record so that the other values in the record can be read later.

```

data perm.people;
 infile census;
 retain Address;
 input type $1. @;

```

```

V---+---10---+---
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F

```

### Conditionally Executing SAS Statements

You can use the value of type to identify each record. If Type is **H**, use an INPUT statement to read the values for Address. However, if Type is **P**, then use an INPUT statement to read the values for Name, Age, and Gender.

You can tell SAS to perform a given task based on a specific condition by using an IF-THEN statement.

```
data perm.people;
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then
 input @3 address $15.;
```

```
V---+-----10---+-----
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
```

Expressions in conditional statements usually involve some type of comparison. In the example shown above, a variable is compared to a constant. When the condition is met, the expression is evaluated as true, and the statement that follows the keyword THEN is executed.

The expression defines a condition so that when the value of type is **H**, the INPUT statement reads the values for Address. However, when the value of type is not **H**, the expression is evaluated as false, and the INPUT statement is not executed. Notice that the value is enclosed in quotation marks because it is a character value.

**TIP** When you compare values, be sure to express the values exactly as they appear in the data. For example, the expression below would evaluate to false because the values in the data are stored in uppercase letters.

```
if type='h' then ... ;
```

## Reading a Detail Record

You can use a subsetting IF statement to check for the condition that type is **P**. The remaining DATA step statements execute only when the condition is true. If type is not **P**, then the values for Name, Age, and Gender are not read, the values in the program data vector are not written to the data set as an observation, and control returns to the top of the DATA step. However, Address is retained.

If type is **P**, Name, Age, and Gender are read, and an observation is written to the data set. Remember that you want to create an observation for detail records only.

```
data perm.people;
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then input @3 address $15.;
 if type='P';
 input @3 Name $10. @13 Age 3. @16 Gender $1.;
run;
```



## Dropping Variables

Type is useful only for identifying a record's type. Therefore, drop the variable from the data set. The DROP= option in the DATA statement shown here prevents the variable Type from being written to the data set.

```
data perm.people (drop=type);
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then input @3 address $15.;
 if type='P';
 input @3 Name $10. @13 Age 3. @16 Gender $1.;
run;
```

## Processing a DATA Step That Creates One Observation per Detail Record

At compile time, the variable Type is flagged so that its values are not written to the data set. Address is flagged so that its value is retained across iterations of the DATA step.

```
data perm.people (drop=type);
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then input @3 address $15.;
 if type='P';
 input @3 Name $10. @13 Age 3. @16 Gender $1.;
run;
```

```
>-----+-----10-----+-----
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST
```

Retain		Drop			
N	Address	type	Name	Age	Gender
•				•	

As the DATA step begins to execute, the INPUT statement reads the value for Type and holds the first record.

```

>-----+-----10-----+-----
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST

```

Retain		Drop			
N	Address	type	Name	Age	Gender
1		H		•	

The condition Type='H' is checked and found to be true. Therefore, the INPUT statement reads the value for Address in the first record.

```

>-----+-----10-----+-----
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST

```

Retain		Drop			
N	Address	type	Name	Age	Gender
1	321 S. MAIN ST	H		•	

Next, the subsetting IF statement checks for the condition Type='P'. Because the condition is not true, the remaining statements are not executed and control returns to the top of the DATA step. The PDV is initialized but Address is retained.

```

>-----+-----10-----+-----
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST

```

Retain		Drop			
N	Address	type	Name	Age	Gender
2	321 S. MAIN ST			•	

As the second iteration begins, the input pointer moves to the next record and a new value for Type is read. The condition expressed in the IF-THEN statement is not true, so the statement following the THEN keyword is not executed.

```

>-----+-----10-----+-----
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST

```

Retain		Drop			
N	Address	type	Name	Age	Gender
2	321 S. MAIN ST	P		•	

Now the subsetting IF statement checks for the condition Type='P'. In this iteration, the condition is true, so the final INPUT statement reads the values for Name, Age, and Gender.

```

>-----+-----10-----+-----
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST

```

Retain		Drop			
N	Address	type	Name	Age	Gender
2	321 S. MAIN ST	P	MARY E	21	F

Next, the values in the program data vector are written as the first observation, and control returns to the top of the DATA step. Notice that the values for Type are not included.

```

>-----+-----10-----+-----
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST

```

Retain		Drop			
N	Address	type	Name	Age	Gender
2	321 S. MAIN ST	P	MARY E	21	F

SAS Data Set Perm.People					
Obs	Address		Name	Age	Gender
1	321 S. MAIN ST		MARY E	21	F

As execution continues, observations are produced from the third and fourth records. However, notice that the fifth record is a header record. During the fifth iteration, the condition Type='H' is true, so a new address is read into the program data vector, overwriting the previous value.

>-----+-----10-----+-----					
H 321 S. MAIN ST					
P MARY E 21 F					
P WILLIAM M 23 M					
P SUSAN K 3 F					
H 324 S. MAIN ST					

	Retain		Drop		
N	Address	type	Name	Age	Gender
5	324 S. MAIN ST	H		.	

SAS Data Set Perm.People					
Obs	Address		Name	Age	Gender
1	321 S. MAIN ST		MARY E	21	F
2	321 S. MAIN ST		WILLIAM M	23	M
3	321 S. MAIN ST		SUSAN K	3	F

## Displaying Your Results

When the execution phase is complete, you can display the data set by using the PRINT procedure. The first 10 observations are displayed.

**Figure 22.1** Output from the PRINT Procedure

Obs	Address	Name	Age	Gender
1	321 S. MAIN ST	MARY E	21	F
2	321 S. MAIN ST	WILLIAM M	23	M
3	321 S. MAIN ST	SUSAN K	3	F
4	324 S. MAIN ST	THOMAS H	79	M
5	324 S. MAIN ST	WALTER S	46	M
6	324 S. MAIN ST	ALICE A	42	F
7	324 S. MAIN ST	MARYANN A	20	F
8	324 S. MAIN ST	JOHN S	16	M
9	325A S. MAIN ST	JAMES L	34	M
10	325A S. MAIN ST	LIZA A	31	F
11	325B S. MAIN ST	MARGO K	27	F
12	325B S. MAIN ST	WILLIAM R	27	M
13	325B S. MAIN ST	ROBERT W	1	M

## Creating One Observation per Header Record

### The Basics of Creating One Observation per Header Record

This example uses the data set below to create a summary variable, Total.

Raw Data File

```

1---+---10---+---20
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST
P THOMAS H 79 M
P WALTER S 46 M
P ALICE A 42 F
P MARYANN A 20 F
P JOHN S 16 M
H 325A S. MAIN ST
P JAMES L 34 M
P LIZA A 31 F
H 325B S. MAIN ST
P MARGO K 27 F
P WILLIAM R 27 M
P ROBERT W 1 M

```

Obs	Address	Total
1	321 S. MAIN ST	3
2	324 S. MAIN ST	5
3	325A S. MAIN ST	2
4	325B S. MAIN ST	3

As you write the DATA step to read this file, think about performing several tasks. First, the value of Address must be retained as detail records are read and summarized.

```

data perm.residents;
 infile census;
 retain Address;

```

```

1---+---10---+---
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F

```

Next, the value of Type must be read in order to determine whether the current record is a header record or a detail record. You can add an @ to hold the record so that another INPUT statement can read the remaining values.

```
data perm.residents;
 infile census;
 retain Address;
 input type $1. @;
```

```
V---+-----10---+-----
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
```

When the value of Type indicates a header record, several statements need to be executed. When the value of Type indicates a detail record, you need to define an alternative set of actions.

### DO Group Actions for Header Records

To execute multiple SAS statements based on the value of a variable, you can use a simple DO group with an IF-THEN statement. When the condition Type='H' is true, several statements need to be executed.

```
data perm.residents;
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then do;
```

- First, determine whether this is the first header record in the external file. You do not want the first header record to be written as an observation until the related detail records have been read and summarized.

`_N_` is an automatic variable whose value is the number of times the DATA step has begun to execute. The expression `_N_ > 1` defines a condition where the DATA step has executed more than once. Use this expression in conjunction with the previous IF-THEN statement to check for these two conditions:

1. The current record is a header record.
2. The DATA step has executed more than once.

```
data perm.residents;
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then do;
 if _N_ > 1
```

- When the conditions Type='H' and `_N_ > 1` are true, an OUTPUT statement is executed. Thus, each header record except for the first one causes an observation to be written to the data set.

```
data perm.residents;
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then do;
 if _N_ > 1 then output;
```

- An assignment statement creates the summary variable Total and sets its value to 0.

```
data perm.residents;
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then do;
 if _n_ > 1 then output;
 Total=0;
```

- An INPUT statement reads the values for Address.

```
data perm.residents;
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then do;
 if _n_ > 1 then output;
 Total=0;
 input address $ 3-17;
```

- An END statement closes the DO group.

```
data perm.residents;
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then do;
 if _n_ > 1 then output;
 Total=0;
 input address $ 3-17;
 end;
```

## Reading Detail Records

When the value of type is not **H**, you need to define an alternative action. You can do this by adding an ELSE statement after the DO group.

Remember that the IF-THEN statement executes a SAS statement when the condition that is specified in the IF clause is true. By adding an ELSE statement after the IF-THEN statement, you define an alternative action to be performed when the IF condition is false.

```
data perm.residents;
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then do;
 if _n_ > 1 then output;
 Total=0;
 input address $ 3-17;
 end;
 else
```

The only other type of record is a detail record, represented by a P. You want to count each person who is represented by a detail record and store the accumulated value in the summary variable Total. You do not need to read the values for Name, Age, and Gender.

```
data perm.residents;
```

```

infile census;
retain Address;
input type $1. @;
if type='H' then do;
 if _n_ > 1 then output;
 Total=0;
 input address $ 3-17;
end;
else if type='P' then

```

```

1---+-----10---+-----20
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST
P THOMAS H 79 M
P WALTER S 46 M
P ALICE A 42 F
P MARYANN A 20 F
P JOHN S 16 M

```

At this point, the value of `Total` has been initialized each time a header record is read and set to 0. Now, as each detail record is read, you can increment the value of `Total` by using a `sum` statement. In this example, you are counting the number of detail records for each header record. Therefore, you increment the value of `Total` by 1 when the value of `type` is `P`.

```

data perm.residents;
 infile census;
 retain Address;
 input type $1. @;
 if type='H' then do;
 if _n_ > 1 then output;
 Total=0;
 input address $ 3-17;
 end;
 else if type='P' then total+1;

```

*Note:* A `sum` statement enables you to add any valid SAS expression to an accumulator variable.

```

else if type='B' then total+cost;

```

The value generated by a `sum` statement is automatically retained throughout the `DATA` step. That is why it is important to set the value of `Total` to 0 each time a header record is read.

### Determining the End of the External File

The program writes an observation to the data set only when another header record is read and the `DATA` step has executed more than once. But after the last detail record is read, there are no more header records to cause the last observation to be written to the data set.

```

data perm.residents;

```



```

infile census;
retain Address;
input type $1. @;
if type='H' then do;
 if _n_ > 1 then
 output;
 Total=0;
 input address $ 3-17;
end;
else if type='P'
 then total+1;

```

```

1---+-----10---+-----20
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST
P THOMAS H 79 M
P WALTER S 46 M
P ALICE A 42 F
P MARYANN A 20 F
P JOHN S 16 M
H 325A S. MAIN ST
P JAMES L 34 M
P LIZA A 31 F
H 325B S. MAIN ST
P MARGO K 27 F
P WILLIAM R 27 M
P ROBERT W 1 M

```

## The INFILE Statement

It is necessary to determine when the last record in the file is read so that you can then execute another explicit OUTPUT statement. You can determine when the current record is the last record in an external file by specifying the END= option in the INFILE statement.

---

Syntax, INFILE statement with the END= option:

**INFILE** *file-specification* **END=***variable*;

*variable* is a temporary numeric variable whose value is 0 until the last line is read and 1 after the last line is read.

*Note:* Like automatic variables, the END= variable is not written to the data set.

---

In the following example, the END= variable is defined in the INFILE statement as Last. When Last has a value other than 0, the OUTPUT statement writes the final observation to the data set.

```

data perm.residents;
 infile census end=last;
 retain Address;

```

```

input type $1. @;
if type='H' then do;
 if _n_ > 1 then output;
 Total=0;
 input address $ 3-17;
end;
else if type='P' then total+1;
if last then output;

```

A DROP= option in the DATA statement drops the variable type from the data set, and a RUN statement completes the DATA step.

```

data perm.residents (drop=type);
 infile census end=last;
 retain Address;
 input type $1. @;
 if type='H' then do;
 if _n_ > 1 then output;
 Total=0;
 input address $ 3-17;
 end;
 else if type='P' then total+1;
 if last then output;
run;

```

## Processing a DATA Step That Creates One Observation per Header Record

1. During the compilation phase, the variable type is flagged so that later it can be dropped. The value for Address and Total (sum statement) are retained.

```

data perm.residents (drop=type);
 infile census end=last;
 retain Address;
 input type $1. @;
 if type='H' then do;
 if _n_ > 1 then output;
 Total=0;
 input address $3-17;
 end;
 else if type='P' then total +1;
 if last then output;
run;

```

```

>-----+-----10-----+-----
H 321 S. MAIN ST
P MARY E 21 F
P WILLIAM M 23 M
P SUSAN K 3 F
H 324 S. MAIN ST

```

		Retain	Drop	Retain
<b>_N_</b>	<b>last</b>	<b>Address</b>	<b>type</b>	<b>Total</b>
•	•			•

2. As the execution begins, \_N\_ is 1 and Last is 0. Total is 0 because of the sum statement.

		Retain	Drop	Retain
<b>_N_</b>	<b>last</b>	<b>Address</b>	<b>type</b>	<b>Total</b>
1	0			0

3. The value for Type is read, the condition `type='H'` is true, and therefore the statements in the DO group execute.
4. The condition `N>1` is not true, so the OUTPUT statement is not executed. However, Total is assigned the value of 0 and the value for Address is read.

		Retain	Drop	Retain
<u>N</u>	last	Address	type	Total
1	0	321 S. MAIN ST	H	0

5. The END statement closes the DO group. The alternative condition expressed in the ELSE statement is not checked because the first condition, `Type='H'`, was true.
6. The value of Last is still 0, so the OUTPUT statement is not executed. Control returns to the top of the DATA step.
7. During the second iteration, the value of type is 'P' and Total is incremented by 1. Again, the value of Last is 0, so control returns to the top of the DATA step.

		Retain	Drop	Retain
<u>N</u>	last	Address	type	Total
2	0	321 S. MAIN ST	P	1

8. During the fifth iteration, the value of Type is 'H' and `_N_` is greater than 1, so the values for Address and Total are written to the data set as the first observation.
9. As the last record in the file is read, the variable Last is set to 1. Now that the condition for Last is true, the values in the program data vector are written to the data set as the final observation.

```
data perm.residents (drop=type);
 infile census end=last;
 retain Address;
 input type $1. @;
 if type='H' then do;
 if _n_ > 1 then output;
 Total=0;
 input address $3-17;
end;
 else if type='P' then total +1;
 if last then output;
run;
```

```
>-V---+----10---+----
P LIZA A 31 F
H 325B S. MAIN ST
P MARGO K 27 F
P WILLIAM R 27 F
P ROBERT W 1 M
```

		Retain	Drop	Retain
<u>N</u>	last	Address	type	Total
13	1	325B S. MAIN ST	P	3

SAS Data Set Perm.People

Obs	Address	Total
1	321 S. MAIN ST	3
13	325B S. MAIN ST	3

## Chapter Quiz

Select the best answer for each question. Check your answers using the answer key in the appendix.

1. When you write a DATA step to create one observation per detail record, what do you need to do?
  - a. Distinguish between header and detail records.
  - b. Keep the header record as a part of each observation until the next header record is encountered.
  - c. Hold the current value of each record type so that the other values in the record can be read.
  - d. All of the above.
2. Which SAS statement reads the value for code (in the first field), and then holds the value until an INPUT statement reads the remaining value in each observation in the same iteration of the DATA step?

```
1---+----10---+----20---+----30
01 Office Supplies
02 Paper Clips
02 Tape
01 Art Supplies
02 Crayons
02 Finger Paint
02 Construction Paper
```

- a. input code \$2. @;
  - b. input code \$2. @@;
  - c. retain code;
  - d. none of the above
3. Which SAS statement checks for the condition that Record equals C and executes a single statement to read the values for Amount?
  - a. if record=c then input @3 Amount comma7.;
  - b. if record='C' then input @3 Amount comma7.;
  - c. if record='C' then do input @3 Amount comma7.;
  - d. if record=C then do input @3 Amount comma7.;
4. After the value for code is read in the sixth iteration, which illustration of the program data vector is correct?

```

1---+---10---+---20---+---30
H Lettuce
P Green Leaf Quality Growers
P Iceberg Pleasant Farm
P Romaine Quality Growers
H Squash
P Yellow Tasty Acres
P Zucchini Pleasant Farm

```

```

data perm.produce (drop=code);
 infile orders;
 retain Vegetable;
 input code $1. @;
 if code='H' then input @3 vegetable $6.;
 if code='P';
 input @3 Variety : $10. @15 Supplier : $15.;
run;
proc print data=perm.produce;
run;

```

a.

_N_	Vegetable	code	Variety	Supplier
6		P		

b.

_N_	Vegetable	code	Variety	Supplier
6	Squash	P		

c.

_N_	Vegetable	code	Variety	Supplier
6	Squash	H		

d.

_N_	Vegetable	code	Variety	Supplier
6	Squash	H	Yellow	

5. What happens when the fourth iteration of the DATA step is complete?

```

1---+---10---+---20---+---30
F Apples
V Gala $2.50
V Golden Delicious $1.99
V Rome $2.35
F Oranges
V Navel $2.79
V Temple $2.99

```

```

data perm.orders (drop=type);
 infile produce;

```

```

retain Fruit;
input type $1. @;
if type='F' then input @3 fruit $7.;
if type='V';
input @3 Variety : $16. @20 Price comma5.;
run;

```

- All of the values in the program data vector are written to the data set as the third observation.
  - All of the values in the program data vector are written to the data set as the fourth observation.
  - The values for Fruit, Variety, and Price are written to the data set as the third observation.
  - The values for Fruit, Variety, and Price are written to the data set as the fourth observation.
6. Which SAS statement indicates that several other statements should be executed when Record has a value of **A**?

```

1---+----10---+----20---+----30
A 124153-01
C $153.02
D $20.00
D $20.00

```

- if record='A' then do;
  - if record=A then do;
  - if record='A' then;
  - if record=A then;
7. Which is true for the following statements (X indicates a header record)?
- ```

if code='X' then do;
  if _n_ > 1 then output;
  Total=0;
  input Name $ 3-20;
end;

```
- `_N_` equals the number of times the DATA step has begun to execute.
 - When `code='X'` and `_n_ > 1` are true, an OUTPUT statement is executed.
 - Each header record causes an observation to be written to the data set.
 - Both a and b.
8. What happens when the condition `Type='P'` is false?
- ```

if type='P' then input @3 ID $5. @9 Address $20.;
else if type='V' then input @3 Charge 6.;

```
- The values for ID and Address are read.
  - The values for Charge are read.
  - Type is assigned the value of **V**.
  - The ELSE statement is executed.
9. What happens when Last has a value other than zero?

```

data perm.househld (drop=code);
 infile citydata end=last;
 retain Address;
 input type $1. @;
 if code='A' then do;
 if _n_ > 1 then output;
 Total=0;
 input address $ 3-17;
 end;
 else if code='N' then total+1;
 if last then output;
run;

```

- Last has a value of 1.
  - The OUTPUT statement writes the last observation to the data set.
  - The current value of last is written to the DATA set.
  - Both a and b.
10. Based on the values in the program data vector, what happens next?

_N_	last	Department	Extension	type	Total	Amount
5	0	Accounting	x3808	D	16.50	.

1	---	10	---	20	---	30
D	Accounting		x3808			
S	Paper Clips		\$2.50			
S	Paper		\$4.95			
S	Binders		\$9.05			
D	Personnel		x3810			
S	Markers		\$8.98			
S	Pencils		\$3.35			

```

data work.supplies (drop=type amount);
 infile orders end=last;
 retain Department Extension;
 input type $1. @;
 if type='D' then do;
 if _n_ > 1 then output;
 Total=0;
 input @3 department $10. @16 extension $5.;
 end;
 else if type='S' then do;
 input @16 Amount comma5.;
 total+amount;
 if last then output; end;
run;

```

- All the values in the program data vector are written to the data set as the first observation.
- The values for Department, Total, and Extension are written to the data set as the first observation.
- The values for Department, Total, and Extension are written to the data set as the fourth observation.
- The value of Last changes to 1.





## Appendix 1

# Quiz Answer Keys

---

Chapter 2: Basic Concepts .....	527
Chapter 3: Setting Up Your SAS Session .....	528
Chapter 4: Identifying and Correcting Errors .....	529
Chapter 5: Creating List Reports .....	531
Chapter 6: Creating SAS Data Sets from External Files .....	532
Chapter 7: Understanding DATA Step Processing .....	533
Chapter 8: Creating and Applying User-Defined Formats .....	534
Chapter 9: Producing Descriptive Statistics .....	535
Chapter 10: Creating Output with ODS .....	536
Chapter 11: Creating and Managing Variables .....	537
Chapter 12: Reading SAS Data Sets .....	538
Chapter 13: Combining SAS Data Sets .....	539
Chapter 14: Using Functions to Manipulate Data .....	541
Chapter 15: Processing Data with DO Loops .....	542
Chapter 16: Processing Data with Arrays .....	542
Chapter 17: Reading Raw Data in Fixed Fields .....	543
Chapter 18: Reading and Creating Raw Data in Free-Format .....	544
Chapter 19: SAS Date and Time Values .....	545
Chapter 20: Creating a Single Observation from Multiple Raw Data Records ..	546
Chapter 21: Creating Multiple Observations from a Single Raw Data Record ..	547
Chapter 22: Conditioning Raw Data Files .....	548

---

## Chapter 2: Basic Concepts

1. Correct answer: c

Rows in the data set are called observations, and columns are called variables.  
Missing values do not affect the structure of the data set.

2. Correct answer: a

When it encounters a DATA, PROC, or RUN statement, SAS stops reading statements and executes the previous step in the program. This program contains one DATA step and two PROC steps, for a total of three program steps.

3. Correct answer: b

It must be a character variable, because the values contain letters and underscores, which are not valid characters for numeric values.

4. Correct answer: a

It must be a numeric variable, because the missing value is indicated by a period rather than by a blank.

5. Correct answer: c

Variable names follow the same rules as SAS data set names. They can be 1 to 32 characters long, must begin with a letter (A-Z, either uppercase or lowercase) or an underscore, and can continue with any combination of numbers, letters, or underscores.

6. Correct answer: d

To store a file permanently in a SAS data library, you assign it a libref other than the default Work. For example, by assigning the libref Profits to a SAS data library, you specify that files within the library are to be stored until you delete them. Therefore, SAS files in the Sashelp, Sasuser, and Profits libraries are permanent files.

7. Correct answer: d

To reference a temporary SAS file in a DATA step or PROC step, you can specify the one-level name of the file (for example, Forecast) or the two-level name using the libref Work (for example, Work.Forecast).

8. Correct answer: d

The numeric variable Balance has a default length of 8. Numeric values (no matter how many digits they contain) are stored in 8 bytes of storage unless you specify a different length.

9. Correct answer: c

The five statements are 1) the PROC PRINT statement (two lines long); 2) the VAR statement; 3) the WHERE statement (on the same line as the VAR statement); 4) the LABEL statement; and 5) the RUN statement (on the same line as the LABEL statement).

10. Correct answer: d

Every SAS file is stored in a SAS library, which is a collection of SAS files, such as SAS data sets and catalogs. In some operating environments, a SAS library is a physical collection of files. In others, the files are only logically related. In the Windows and UNIX environments, a SAS library is typically a group of SAS files in the same folder or directory.

---

## Chapter 3: Setting Up Your SAS Session

1. Correct answer: b

You can create HTML output using programming statements on any SAS platform. In addition, on all except mainframe platforms, you can use SAS windows to specify HTML as a result format.

2. Correct answer: d

As long as you specify an informat with the correct field width for reading the entire date value, the YEARCUTOFF= option does not affect date values that have four-digit years.

3. Correct answer: a

A SAS engine is a set of internal instructions that SAS uses for writing to and reading from files in a SAS library. Each engine specifies the file format for files that are stored in the library, which in turn enables SAS to access files with a particular format. Some engines access SAS files, and other engines support access to other vendors' files.

4. Correct answer: a

To print a summary of library contents with the CONTENTS procedure, use a period to append the `_ALL_` option to the libref. Adding the NODS option suppresses detailed information about the files.

5. Correct answer: b

Librefs must be 1 to 8 characters long, must begin with a letter or underscore, and can contain only numbers, letters, or underscores. After you assign a libref, you specify it as the first level in the two-level name for a SAS file.

6. Correct answer: a

The YEARCUTOFF= option specifies which 100-year span is used to interpret two-digit year values. The default value of YEARCUTOFF= is **1920**. However, you can override the default and change the value of YEARCUTOFF= to the first year of another 100-year span. If you specify YEARCUTOFF=**1950**, then the 100-year span will be from **1950** to **2049**.

7. Correct answer: c

The LIBNAME statement is global, which means that librefs stay in effect until changed or canceled, or until the SAS session ends. Therefore, the LIBNAME statement assigns the libref for the current SAS session only. You must assign a libref before accessing SAS files that are stored in a permanent SAS data library.

8. Correct answer: b

In the LIBNAME statement, you specify the library name before the engine name. Both are followed by the path.

---

## Chapter 4: Identifying and Correcting Errors

1. Correct answer: d

Although you can write SAS statements in almost any format, a consistent layout enhances readability and enables you to understand the program's purpose. It is a good idea to begin DATA and PROC steps in column one, to indent statements within a step, to begin RUN statements in column one, and to include a RUN statement after every DATA step or PROC step.

2. Correct answer: a

To correct errors in programs when you use the Editor window, you usually need to recall the submitted statements from the recall buffer to the Editor window. After correcting the errors, you can resubmit the revised program. However, before doing so, it is a good idea to clear the messages from the Log window so that you do not confuse the old error messages with the new messages. Remember to check the Log window again to verify that your program ran correctly.

3. Correct answer: d

The missing quotation mark in the LABEL statement causes SAS to misinterpret the statements in the program. When you submit the program, SAS is unable to resolve the PROC step, and a PROC PRINT running message appears at the top of the active window.

4. Correct answer: c

Syntax errors generally cause SAS to stop processing the step in which the error occurred. When a program that contains an error is submitted, messages regarding the problem also appear in the Log window. When a syntax error is detected, the Log window displays the word ERROR, identifies the possible location of the error, and gives an explanation of the error.

5. Correct answer: c

Syntax errors are common types of errors. Some SAS system options and features of the code editing window can help you identify syntax errors. Other types of errors include data errors and logic errors.

6. Correct answer: b

When you submit a SAS statement that contains an invalid option, a log message notifies you that the option is not valid or not recognized. You should recall the program, remove or replace the invalid option, check your statement syntax as needed, and resubmit the corrected program.

7. Correct answer: b

The DATA step contains a misspelled keyword (dat instead of data). However, this is such a common (and easily interpretable) error that SAS produces only a warning message, not an error.

8. Correct answer: c

This program contains an unbalanced quotation mark. When you have an unbalanced quotation mark, SAS is often unable to detect the end of the statement in which it occurs. Simply adding a quotation mark and resubmitting your program usually does not solve the problem. SAS still considers the quotation marks to be unbalanced. To correct the error, you need to resolve the unbalanced quotation mark before you recall, correct, and resubmit the program.

9. Correct answer: b

Without a RUN statement (or a following DATA or PROC step), the DATA step does not execute, so it continues to run. Unbalanced quotation marks can also cause the DATA step running message if relatively little code follows the unbalanced quotation mark. The other three problems that the quiz question mentions also generate errors or warnings in the Log window.

10. Correct answer: c

A logic error occurs when the program statements follow the rules and execute, but the results are not correct. You can use the PUTLOG statement in the DATA step to help identify logic errors.

---

## Chapter 5: Creating List Reports

1. Correct answer: c

The DATA= option specifies the data set that you are listing, and the ID statement replaces the Obs column with the specified variable. The VAR statement specifies variables and controls the order in which they appear, and the WHERE statement selects rows based on a condition. The LABEL option in the PROC PRINT statement causes the labels that are specified in the LABEL statement to be displayed.

2. Correct answer: a

You use the DATA= option to specify the data set to be printed. The LABEL option specifies that variable labels appear in output instead of in variable names.

3. Correct answer: d

In the WHERE statement, the IN operator enables you to select observations based on several values. You specify values in parentheses and separated by spaces or commas. Character values must be enclosed in quotation marks and must be in the same case as in the data set.

4. Correct answer: c

In a PROC SORT step, you specify the DATA= option to specify the data set to sort. The OUT= option specifies an output data set. The required BY statement specifies the variable or variables to use in sorting the data.

5. Correct answer: b

The DATE and NONUMBER system options cause the output to appear with the date but without page numbers. In the PROC PRINT step, the DOUBLE option specifies double spacing, and the NOOBS option removes the default Obs column.

6. Correct answer: d

You do not need to name the variables in a VAR statement if you specify them in the SUM statement, but you can. If you choose not to name the variables in the VAR statement as well, then the SUM statement determines their order in the output.

7. Correct answer: c

The BY statement is required in PROC SORT. Without it, the PROC SORT step fails. However, the PROC PRINT step prints the original data set as requested.

8. Correct answer: a

Column totals appear at the end of the report in the same format as the values of the variables, so b is incorrect. Work.Loans is sorted by Month and Amount, so c is incorrect. The program sums both Amount and Payment, so d is incorrect.

9. Correct answer: c

To ensure that the compound expression is evaluated correctly, you can use parentheses to group the observations:

```
account='101-1092' or rate eq 0.095
```

OBS	Account	Amount	Rate	Months	Payment
1	101-1092	\$22,000	10.00%	60	\$467.43
2	101-1731	\$114,000	9.50%	360	\$958.57
3	101-1289	\$10,000	10.50%	36	\$325.02
4	101-3144	\$3,500	10.50%	12	\$308.52
5	103-1135	\$8,700	10.50%	24	\$403.47
6	103-1994	\$18,500	10.00%	60	\$393.07
7	103-2335	\$5,000	10.50%	48	\$128.02
8	103-3864	\$87,500	9.50%	360	\$735.75
9	103-3891	\$30,000	9.75%	360	\$257.75

For example, from the data set above, a and b select observations 2 and 8 (those that have a rate of 0.095); c selects no observations; and d selects observations 4 and 7 (those that have an amount less than or equal to 5000).

10. Correct answer: d

You can remove the column for observation numbers. You can also specify the variables you want, and you can select observations according to conditions.

---

## Chapter 6: Creating SAS Data Sets from External Files

1. Correct answer: a

You assign a fileref by using a FILENAME statement in the same way that you assign a libref by using a LIBNAME statement.

2. Correct answer: d

Like LIBNAME statements, FILENAME statements are global; they remain in effect until changed or canceled, or until the SAS session ends.

3. Correct answer: b

You use an INFILE statement to specify the raw data file to be read. You can specify a fileref or an actual filename (in quotation marks). The OBS= option in the INFILE statement enables you to process only records 1 through *n*.

4. Correct answer: d

The keyword `_NULL_` in the DATA statement enables you to use the power of the DATA step without actually creating a SAS data set. You use FILE and PUT statements to write observations from a SAS data set to a raw data file. The FILE statement specifies the raw data file, and the PUT statement describes the lines to write to the raw data file. The filename and location that are specified in the FILE statement must be enclosed in quotation marks.

5. Correct answer: b

Column input is appropriate only in some situations. When you use column input, your data must be standard character or numeric values, and they must be in fixed fields. That is, values for a particular variable must be in the same location in all records.

6. Correct answer: a

The INPUT statement creates a variable using the name that you assign to each field. Therefore, when you write an INPUT statement, you need to specify the variable names exactly as you want them to appear in the SAS data set.

7. Correct answer: b

You can use column input to read fields in any order. You must specify the variable name, identify character variables with a \$, and specify the correct starting and ending column for each field.

8. Correct answer: c

To redefine the values of the variable Income in an assignment statement, you specify the variable name on the left side of the equal sign and an appropriate expression including the variable name on the right side of the equal sign.

9. Correct answer: d

To read instream data, you specify a DATALINES statement and data lines, followed by a null statement (single semicolon) to indicate the end of the input data. Program a. contains no DATALINES statement, and the INPUT statement does not specify the fields to read. Program b. contains no data lines, and the INPUT statement in Program c. does not specify the necessary dollar signs for the character variables Country and Vehicle.

10. Correct answer: c

To subset data, you can use a subsetting IF statement in any DATA step to process only those observations that meet a specified condition. Because Sex is a character variable, the value F must be enclosed in quotation marks and must be in the same case as in the data set.

---

## Chapter 7: Understanding DATA Step Processing

1. Correct answer: b

During the compilation phase, the program data vector is created. The program data vector includes the two automatic variables \_N\_ and \_ERROR\_. The descriptor portion of the new SAS data set is created at the end of the compilation phase. The descriptor portion includes the name of the data set, the number of observations and variables, and the names and attributes of the variables. Observations are not written until the execution phase.

2. Correct answer: a

Syntax checking can detect many common errors, but it cannot verify the values of variables or the correctness of formats.

3. Correct answer: c

The DATA step executes once for each record in the input file, unless otherwise directed.

4. Correct answer: d

The remaining variables are initialized to missing. Missing numeric values are represented by periods, and missing character values are represented by blanks.

5. Correct answer: b

The default value of `_ERROR_` is 0, which means there is no data error. When an error occurs, whether one error or multiple errors, the value is set to 1.

6. Correct answer: d

By default, at the end of the DATA step, the values in the program data vector are written to the data set as an observation, control returns to the top of the DATA step, the value of the automatic variable `_N_` is incremented by one, and the values of variables that were created in programming statements are reset to missing. The automatic variable `_ERROR_` is reset to 0 if necessary.

7. Correct answer: a

The order in which variables are defined in the DATA step determines the order in which the variables are stored in the data set.

8. Correct answer: c

When SAS cannot detect syntax errors, the DATA step compiles, but it does not execute.

9. Correct answer: a

A semicolon is missing from the second line. It will cause an error because the INPUT statement will be interpreted as an invalid INFILE statement option.

10. Correct answer: d

The third line of the log displays the value for IDnum, which is clearly a character value. The fourth line displays the values in the program data vector and shows a period, the symbol for a missing numeric value, for IDnum. The other values are correctly assigned. Thus, it appears that numeric values were expected for IDnum. A dollar sign, to indicate character values, is missing from the INPUT statement.

---

## Chapter 8: Creating and Applying User-Defined Formats

1. Correct answer: c

If you do not specify the `LIBRARY=` option, formats are stored in a default format catalog named `Work.Formats`. The libref `Work` signifies that any format that is stored in `Work.Formats` is a temporary format; it exists only for the current SAS session.

2. Correct answer: a

To store formats in a permanent catalog, you first write a `LIBNAME` statement to associate the libref with the SAS data library in which the catalog will be stored. Then add the `LIB=` (or `LIBRARY=`) option to the `PROC FORMAT` statement, specifying the name of the catalog.

3. Correct answer: d

The name of a format that is created with a `VALUE` statement must begin with a dollar sign (\$) if it applies to a character variable.

4. Correct answer: b

A semicolon is needed after the `PROC FORMAT` statement. The `VALUE` statement begins with the keyword `VALUE` and ends with a semicolon after all the labels have been defined.



5. Correct answer: d

You can list values separated by commas, but the list must contain either all numeric values or all character values. Data set variables are either numeric or character.

6. Correct answer: d

When specifying a label, enclose it in quotation marks and limit the label to 32,767 characters.

7. Correct answer: d

MISS and MISSING are invalid keywords, and LOW does not include missing numeric values. The keyword OTHER can be used in the VALUE statement to label missing values as well as any values that are not specifically included in a range.

8. Correct answer: b

By placing the FORMAT statement in a DATA step, you permanently associate the defined format with variables.

9. Correct answer: b

To associate a user-defined format with a variable, place a period at the end of the format name when it is used in the FORMAT statement.

10. Correct answer: d

Adding the keyword FMTLIB to the PROC FORMAT statement displays a list of all the formats in your catalog, along with descriptions of their values.

---

## Chapter 9: Producing Descriptive Statistics

1. Correct answer: c

By default, the MEANS procedure produces the n, mean, minimum, maximum, and standard deviation.

2. Correct answer: d

To specify the variables that PROC MEANS analyzes, add a VAR statement and list the variable names.

3. Correct answer: a

Unlike Age, Height, or Weight, the values of IDnum are unlikely to yield any useful statistics.

4. Correct answer: a

Unlike CLASS processing, BY-group processing requires that your data already be indexed or sorted in the order of the BY variables. You might need to run the SORT procedure before using PROC MEANS with a BY group.

5. Correct answer: b

A CLASS statement produces a single large table, whereas BY-group processing creates a series of small tables. The order of the variables in the CLASS statement determines their order in the output table.

6. Correct answer: d

You can use either PROC MEANS or PROC SUMMARY to create the table. Adding a PRINT option to the PROC SUMMARY statement produces the same report as if you used PROC MEANS.

7. Correct answer: c

By default, PROC FREQ creates a table for all variables in a data set.

8. Correct answer: c

Both continuous values and unique values can result in lengthy, meaningless tables. Frequency distributions work best with categorical values.

9. Correct answer: d

An asterisk is used to join the variables in a two-way TABLES statement. The first variable forms the table rows. The second variable forms the table columns.

10. Correct answer: d

An asterisk is used to join the variables in crosstabulation tables. The only results shown in this table are cell percentages. The NOFREQ option suppresses cell frequencies, the NOROW option suppresses row percentages, and the NOCOL option suppresses column percentages.

---

## Chapter 10: Creating Output with ODS

1. Correct answer: d

You can generate any number of output types as long as you open the ODS destination for each type of output you want to create.

2. Correct answer: a

HTML output is created by default in the SAS windowing environment for the Windows operating environment and UNIX, so these statements create HTML and PDF output.

3. Correct answer: a

By default, in the SAS windowing environment for the Windows operating environment and UNIX, SAS programs produce HTML output. If you want only RTF output, it is a good idea to close the HTML destination before creating RTF output, as an open destination uses system resources.

4. Correct answer: c

When multiple procedures are run while HTML output is open, procedure output is appended to the same body file.

5. Correct answer: a

The CONTENTS= option creates a table of contents containing links to the body file, **D:\Output\body.html**.

6. Correct answer: b

The table of contents contains a numbered heading for each procedure that creates output.

7. Correct answer: c

The FRAME= option creates an HTML file that integrates the table of contents and the body file.

8. Correct answer: b

Specifying the URL= suboption in the file specification provides a URL that ODS uses in the links it creates. Specifying a simple (one name) URL creates a relative link address to the file.

9. Correct answer: c

You can change the appearance of HTML output by using the STYLE= option in the ODS HTML statement. The style name does not need quotation marks.

10. Correct answer: d

You use the PATH= option to specify the location for HTML files to be stored. When you use the PATH= option, you do not need to specify the full pathname for the body, contents, or frame files.

---

## Chapter 11: Creating and Managing Variables

1. Correct answer: c

Program c correctly deletes the observation in which the value of Finish is oak and the value of Price is less than 200. It also creates TotalPrice by summing the variable Price down observations, and then drops Price by using the DROP= data set option in the DATA statement.

2. Correct answer: b

The PROC PRINT output displays the label Total Amount Loaned for the variable Amount and formats this variable using the COMMA10. format. Temporary labels or formats that are assigned in a PROC step override permanent labels or formats that are assigned in a DATA step.

3. Correct answer: c

Logical comparisons that are enclosed in parentheses are evaluated as true or false before they are compared to other expressions. In the example, the AND comparison within the nested parentheses is evaluated before being compared to the OR comparison.

4. Correct answer: d

The sum statement adds the result of the expression that is on the right side of the plus sign to the numeric variable that is on the left side. The new value is then retained for subsequent observations. The sum statement ignores the missing value, so the value of Count in the fourth observation would be 10+20+0+40, or 70.

5. Correct answer: d

The RETAIN statement assigns an initial value of 100 to the variable Count, so the value of Count in the third observation would be 100+10+20+0, or 130.

6. Correct answer: b

You must enclose character values in quotation marks, and you must specify them in the same case in which they appear in the data set. The value **OK** is not identical to Ok, so the value of Count is not changed by the IF-THEN statement.

7. Correct answer: d

The length of a variable is determined by its first reference in the DATA step. When creating a new character variable, SAS allocates as many bytes of storage space as there are characters in the reference to that variable. The first reference to a new variable can also be made with a LENGTH statement or an assignment statement.

8. Correct answer: a

You can write multiple ELSE statements to specify a series of mutually exclusive conditions. The ELSE statement must immediately follow the IF-THEN statement in your program. An ELSE statement executes only if the previous IF-THEN/ELSE statement is false.

9. Correct answer: a

The length of a new variable is determined by the first reference in the DATA step, not by data values. In this case, the length of Type is determined by the value Fixed. The LENGTH statement is in the wrong place; it must occur before any other reference to the variable in the DATA step.

10. Correct answer: b

To select variables, you can use a DROP or KEEP statement in any DATA step. You can also use the DROP= or KEEP= data set options following a data set name in any DATA or PROC step. However, you cannot use DROP or KEEP statements in PROC steps.

---

## Chapter 12: Reading SAS Data Sets

1. Correct answer: b

The variables Age, Weight, and Group are specified using the KEEP= option in the SET statement. After processing, Age and Group are dropped in the DATA statement.

2. Correct answer: c

You specify the data set to be created in the DATA statement. The DROP= data set option prevents variables from being written to the data set. Because you use the variable OrdTime when processing your data, you cannot drop OrdTime in the SET statement. If you use the KEEP= option in the SET statement, then you must list OrdTime as one of the variables to be kept.

3. Correct answer: d

When you use the BY statement with the SET statement, the DATA step creates the temporary variables FIRST. and LAST. They are not stored in the data set.

4. Correct answer: b

The DATA step writes out observations at the end of the DATA step. However, in this program, the STOP statement stops processing before the end of the DATA step. An explicit OUTPUT statement is needed in order to produce an observation.

5. Correct answer: d

The program is attempting to read the first 4 observations and write the fifth observation to the data set Work.Subset. The number of observations that the

FIRSTOBS option returns is greater than the number of observations that the OBS option returns. Therefore, the result is an error.

6. Correct answer: a

When you use the KEEP= data set option and the POINT= option with the OUTPUT and STOP statements, your program cannot write out a single observation.

7. Correct answer: d

This program uses the END= option to name a temporary variable that contains an end-of-file marker. That variable — last — is set to 1 when the SET statement reads the last observation of the data set.

8. Correct answer: d

At the bottom of the DATA step, the compilation phase is complete, and the descriptor portion of the new SAS data set is created. There are no observations because the DATA step has not yet executed.

9. Correct answer: c

The END= option enables you to write only one observation to the data set. The END= option overrides the DATA step default behavior of writing observations to the output.

---

## Chapter 13: Combining SAS Data Sets

1. Correct answer: a

This example is a case of one-to-one matching, which requires multiple SET statements. Where same-named variables occur, values that are read from the second data set replace those that are read from the first data set. Also, the number of observations in the new data set is the number of observations in the smallest original data set.

2. Correct answer: c

This is a case of interleaving, which requires a list of data set names in the SET statement and one or more BY variables in the BY statement. Notice that observations in each BY group are read sequentially, in the order in which the data sets and BY variables are listed. The new data set contains all the variables from all the input data sets, as well as the total number of records from all input data sets.

3. Correct answer: a

Concatenating appends the observations from one data set to another data set. The new data set contains the total number of records from all input data sets, so b is incorrect. All the variables from all the input data sets appear in the new data set, so c is incorrect.

4. Correct answer: a

The concatenated data sets are read sequentially, in the order in which they are listed in the SET statement. The second observation in Sales.Reps does not contain a value for Sale, so a missing value appears for this variable. (Note that if you merge the data sets, the value of Sale for the second observation is \$30,000.)

5. Correct answer: b

If you have variables with the same name in more than one input data set, values of the same-named variable in the first data set in which it appears are overwritten by values of the same-named variable in subsequent data sets.

6. Correct answer: c

The DATA step uses the IN= data set option and the subsetting IF statement to exclude unmatched observations from the output data set. So a and b, which contain unmatched observations, are incorrect.

7. Correct answer: d

Match-merging overwrites same-named variables in the first data set with same-named variables in subsequent data sets. To prevent overwriting, rename variables by using the RENAME= data set option in the MERGE statement.

8. Correct answer: c

The two input data sets are not sorted by values of the BY variable, so the DATA step produces errors and stops processing.

9. Correct answer: c

In this example, the new data set contains one observation for each unique value of ID. The merged data set is shown below.

ID	Name	Dept	Project	Hours
000	Miguel	A12	Document	.
111	Fred	B45	Survey	35
222	Diana	B45	Document	40
777	Steve			0
888	Monique	A12	Document	37
999	Vien	D03	Survey	.

10. Correct answer: a

In the new data set, the third observation is the second observation for ID number 2 (Kelly Windsor). The value for Bonus is retained from the previous observation because the BY variable value did not change. The new data set is shown below.

ID	Name	Sale	Bonus
1	Nay Rong	\$28,000	\$2,000
2	Kelly Windsor	\$30,000	\$4,000
2	Kelly Windsor	\$40,000	\$4,000
3	Julio Meraz	\$15,000	\$3,000
3	Julio Meraz	\$20,000	\$3,000
3	Julio Meraz	\$25,000	\$3,000
4	Richard Krabill	\$35,000	\$2,500

---

## Chapter 14: Using Functions to Manipulate Data

1. Correct answer: d

Use a variable list to specify a range of variables as the function argument. When specifying a variable list, be sure to precede the list with the word *of*. If you omit the word *of*, the function argument might not be interpreted as expected.

2. Correct answer: b

When this DATA step is executed, SAS automatically converts the character values of PayRate to numeric values so that the calculation can occur. Whenever data is automatically converted, a message is written to the SAS log stating that the conversion has occurred.

3. Correct answer: b

You explicitly convert character values to numeric values by using the INPUT function. Be sure to select an informat that can read the form of the values.

4. Correct answer: d

You explicitly convert numeric values to character values by using the PUT function. Be sure to select a format that can read the form of the values.

5. Correct answer: c

Because the YEARCUTOFF= system option is set to 1920, SAS sees the two-digit year value 20 as 1920. Four-digit year values are always read correctly.

6. Correct answer: a

The SCAN function is used to extract words from a character value when you know the order of the words, when their position varies, and when the words are marked by some delimiter. In this case, you do not need to specify delimiters, because the blank and the comma are default delimiters.

7. Correct answer: d

The SUBSTR function is best used when you know the exact position of the substring to extract from the character value. You specify the position to start from and the number of characters to extract.

8. Correct answer: c

The SUBSTR function replaces variable values if it is placed on the left side of an assignment statement. When placed on the right side (as in Question 7), the function extracts a substring.

9. Correct answer: b

The TRIM function removes trailing blanks from character values. In this case, extra blanks must be removed from the values of FirstName. Although answer c also works, the extra TRIM function for the variable LastName is unnecessary. Because of the LENGTH statement, all values of FullName are padded to 40 characters.

10. Correct answer: d

Use the INDEX function in a subsetting IF statement, enclosing the character string in quotation marks. Only those observations in which the function locates the string and returns a value greater than 0 are written to the data set.

---

## Chapter 15: Processing Data with DO Loops

1. Correct answer: c

DO loops are DATA step statements and cannot be used in conjunction with PROC steps.

2. Correct answer: c

The number of iterations is determined by the DO statement's stop value, which in this case is 12.

3. Correct answer: a

Use a DO loop to perform repetitive calculations starting at 1 and looping 15 times.

4. Correct answer: d

At the end of the 15th iteration of the DO loop, the value for Year is incremented to **2005**. Because this value exceeds the stop value, the DO loop ends. At the bottom of the DATA step, the current values are written to the data set.

5. Correct answer: b

The OUTPUT statement overrides the automatic output at the end of the DATA step. On the last iteration of the DO loop, the value of Year, **2004**, is written to the data set.

6. Correct answer: d

The number of observations is based on the number of times the OUTPUT statement executes. The new data set has 20 observations, one for each iteration of the DO loop.

7. Correct answer: b

Place the monthly calculation in a DO loop within a DO loop that iterates once for each year. The DO WHILE and DO UNTIL statements are not used here because the number of required iterations is fixed. A non-iterative DO group would not be useful.

8. Correct answer: a

The DO UNTIL condition is evaluated at the bottom of the loop, so the enclosed statements are always executed at least once.

9. Correct answer: c

Because the DO WHILE loop is evaluated at the top of the loop, you specify the condition that must exist in order to execute the enclosed statements.

10. Correct answer: a

The WHILE expression causes the DO loop to stop executing when the value of Distance becomes equal to or greater than **250**.

---

## Chapter 16: Processing Data with Arrays

1. Correct answer: a



An ARRAY statement is not an executable statement; it merely defines an array.

2. Correct answer: d

The value in parentheses indicates the number of elements in the array. In this case, there are four elements.

3. Correct answer: b

In the DO statement, you specify the index variable that represents the values of the array elements. Then specify the start and stop positions of the array elements.

4. Correct answer: c

The index value represents the position of the array element. In this case, the third element is Jul.

5. Correct answer: b

To process all the elements in an array, you can either specify the array dimension or use the DIM function with the array name as the argument.

6. Correct answer: d

Either numeric or character variables can be created by an ARRAY statement.

7. Correct answer: a

At the end of the second iteration, `diff{i}` resolves as follows:

```
diff{2}=wt{2+1}-wt{2};
diff{2}=215-200
```

8. Correct answer: a

To create temporary array elements, specify `_TEMPORARY_` after the array name and dimension. Specify an initial value for each element, separated by either blanks or commas, and enclose the values in parentheses.

---

## Chapter 17: Reading Raw Data in Fixed Fields

1. Correct answer: c

Column input specifies the variable's name, followed by a dollar (\$) sign if the values are character values, and the beginning and ending column locations of the raw data values.

2. Correct answer: c

Column input is useful for reading standard values only.

3. Correct answer: a

A standard numeric value can contain numbers, scientific notation, decimal points, and plus and minus signs. Nonstandard numeric data includes values that contain fractions or special characters such as commas, dollar signs, and percent signs.

4. Correct answer: d

Formatted input can be used to read both standard and nonstandard data in fixed fields.

5. Correct answer: b

The \$w. informat enables you to read character data. The w represents the field width of the data value or the total number of columns that contain the raw data field.

6. Correct answer: d

The COMMAw.d informat strips out special characters, such as commas, dollar signs, and percent signs, from numeric data and stores only numeric values in a SAS data set.

7. Correct answer: b

The +6 pointer control moves the input pointer to the beginning column of Item, and the values are read. Then the @1 pointer control returns to column 1, where the values for ModelNumber are located.

8. Correct answer: d

The values for Cost contain dollar signs and commas, so you must use the COMMAw.d informat. Counting the numbers, dollar sign, comma, and decimal point, the field width is 9 columns. Because the data value contains decimal places, a d value is not needed.

9. Correct answer: d

The default location of the column pointer control is column 1, so a column pointer control is optional for reading the first field. You can use the @n or +n pointer controls to specify the beginning column of the other fields. You can use the \$w. informat to read the values for Item, the COMMAw.d informat for UnitCost, and the w.d informat for Quantity.

10. Correct answer: a

Use the PAD option in the INFILE statement to read variable-length records that contain fixed-field data. The PAD option pads each record with blanks so that all data lines have the same length.

---

## Chapter 18: Reading and Creating Raw Data in Free-Format

1. Correct answer: b

The raw data file contains data that is free-format, meaning that the data is not arranged in columns or fixed fields.

2. Correct answer: c

List input should be used to read data that is free-format because you do not need to specify the column locations of the data.

3. Correct answer: c

You can use the DSD option in the FILE statement to specify that data values containing commas should be enclosed in quotation marks. The DSD option uses a comma as the delimiter by default.

4. Correct answer: a

Because the data is free-format, list input is used to read the values. With list input, you simply name each variable and identify its type.

5. Correct answer: d

The INFILE statement identifies the location of the external data file. The DLM= option specifies the colon (:) as the delimiter that separates each field.

6. Correct answer: a

You can use the MISSEVER option in the INFILE statement to read the missing values at the end of a record. The MISSEVER option prevents SAS from moving to the next record if values are missing in the current record.

7. Correct answer: b

The LENGTH statement extends the length of the character variable LastName so that it is large enough to accommodate the data. Variable attributes such as length are defined the first time a variable is named in a DATA step. The LENGTH statement should precede the INPUT statement so that the correct length is defined.

8. Correct answer: d

Notice that the values for School contain embedded blanks, and the values for Enrolled are nonstandard numeric values. Modified list input can be used to read the values that contain embedded blanks and nonstandard values.

9. Correct answer: b

The INPUT statement uses list input with format modifiers and informats to read the values for each variable. The ampersand (&) modifier enables you to read character values that contain single embedded blanks. The colon (:) modifier enables you to read nonstandard data values and character values that are longer than eight characters, but which contain no embedded blanks.

10. Correct answer: d

The values for Year can be read with column, formatted, or list input. However, the values for School and Enrolled are free-format data that contain embedded blanks or nonstandard values. Therefore, these last two variables must be read with modified list input.

---

## Chapter 19: SAS Date and Time Values

1. Correct answer: c

A SAS date value is the number of days from January 1, 1960, to the given date.

2. Correct answer: d

In addition to tracking time intervals, SAS date and time values can be used in calculations like other numeric values. This lets you calculate values that involve dates much more easily than in other programming languages.

3. Correct answer: b

SAS automatically makes adjustments for leap years.

4. Correct answer: d

The SAS informat MMDDYYw. reads dates such as 10222001, 10/22/01, or 10-22-01. In this case, the field width is eight.

5. Correct answer: b

The minimum acceptable field width for the TIMEw. informat is five. If you specify a w value less than five, you will receive an error message in the SAS log.

6. Correct answer: d

In the time value of a date and time expression, you must use delimiters to separate the values for hour, minutes, and seconds.

7. Correct answer: c

The value of the YEARCUTOFF= system option does not affect four-digit year values. Four-digit values are always read correctly.

8. Correct answer: c

The *w* value of the informat MMDDYY8. is too small to read the entire value, so the last two digits of the year are truncated. The last two digits thus become 19 instead of 25. Because the YEARCUTOFF= system option is set to 1920, SAS interprets this year as 2019. To avoid such errors, be sure to specify an informat that is wide enough for your date expressions.

9. Correct answer: b

To find the number of days spanned by two dates, subtract the first day from the last day and add one. Because SAS date values are numeric values, they can easily be used in calculations.

---

## Chapter 20: Creating a Single Observation from Multiple Raw Data Records

1. Correct answer: c

Information for one observation can be spread out over several records. You can write one INPUT statement that contains line pointer controls to specify the record or records from which values are read.

2. Correct answer: c

The forward slash (/) line pointer control is used to read multiple records sequentially. Each time a / pointer is encountered, the input pointer advances to the next line. @*n* and +*n* are column pointer controls.

3. Correct answer: b

The #*n* line pointer control is used to read records non-sequentially. The #*n* specifies the absolute number of the line to which you want to move the pointer.

4. Correct answer: a

The INPUT statement uses the / line pointer control to move the input pointer forward from the first record to the second record, and from the second record to the third record. The / line pointer control only moves the input pointer forward and must be specified after the instructions for reading the values in the current record. You should place a semicolon only at the end of a complete INPUT statement.

5. Correct answer: d

The first #*n* line pointer control enables you to read the values for ID from the fourth record. The second #*n* line pointer control moves back to the first record and reads the values for Fname and Lname. You can use formatted input, column input, or list input to read the values for ID.

6. Correct answer: c

The first time the DATA step executes, the first three records are read, and an observation is written to the data set. During the second iteration, the next three records are read, and the second observation is written to the data set. During the third iteration, the last three records are read, and the final observation is written to the data set.

7. Correct answer: b

A combination of modified and simple list input can be used to read the values for City, State, and Zip. You need to use modified list input to read the values for City, because one of the values is longer than eight characters and contains an embedded blank. You cannot use formatted input, because the values do not begin and end in the same column in each record.

8. Correct answer: c

The values for Item in the first record are read, and then the following / or #n line pointer control advances the input pointer to the second record to read the values for Inventory and Type.

9. Correct answer: d

You can use either the / or #n line pointer control to advance the input pointer to the second line, in order to read the values for Department and Salary. The colon (:) modifier is used to read the character values that are longer than eight characters (Department) and the nonstandard data values (Salary).

10. Correct answer: c

The third raw data file does not contain the same number of records for each observation, so the output from this data set will show invalid data for the ID and salary information in the fourth line.

---

## Chapter 21: Creating Multiple Observations from a Single Raw Data Record

1. Correct answer: d

The double trailing at-sign (@@) enables the next INPUT statement to read from the current record across multiple iterations of the DATA step. It must be the last item specified in the INPUT statement. A record that is being held by the double trailing at-sign (@@) is not released until the input pointer moves past the end of the record, or until an INPUT statement that has no line-hold specifier executes.

2. Correct answer: b

Unlike the double trailing at-sign (@@), the single trailing at-sign (@) is automatically released when control returns to the top of the DATA step for the next iteration. The trailing @ does not toggle on and off. If another INPUT statement that has a trailing @ executes, the holding effect is still on.

3. Correct answer: c

Each record in this file contains three repeating blocks of data values for Item and Variety. The INPUT statement reads a block of values for Item and Variety, and then holds the current record by using the double trailing at-sign (@@). The values in the program data vector are written to the data set as the first observation. In the next

iteration, the INPUT statement reads the next block of values for Item and Variety from the same record.

4. Correct answer: d

This raw data file contains an ID field followed by repeating fields. The first INPUT statement reads the values for ID and uses the @ line-hold specifier to hold the current record for the next INPUT statement in the DATA step. The second INPUT statement reads the values for Quantity. When all of the repeating fields have been read, control returns to the top of the DATA step, and the record is released.

5. Correct answer: a

The iterative DO statement begins the execution of a loop based on the value of an index variable. Here, the loop executes when the value of count ranges from 1 to 50, incremented by 5.

6. Correct answer: a

The OUTPUT statement must be in the loop so that each time a value for Activity is read, an observation is immediately written to the data set.

7. Correct answer: c

The DO WHILE statement checks for the condition that Cholesterol is greater than 200. The expression must be enclosed in parentheses. The expression is evaluated at the top of the loop before the loop executes. If the condition is true, the DO WHILE loop executes. If the expression is false the first time it is evaluated, the loop does not execute.

8. Correct answer: b

The sum statement adds the result of an expression to an accumulator variable. The + sign is an essential part of the sum statement. Here, the value of TotalPay is incremented by 1.

9. Correct answer: d

The first input statement reads STORE and SALES and holds the record. Month is initialized to 0. The DO loop executes while SALES is not missing. Inside the loop, month increments, an observation is written to output, the next SALES value is read, and the record is held. The loop stops when a missing value of SALES is read. Control returns to the top of the DATA step, the held record is released, and the input statement reads the next record.

10. Correct answer: c

This DATA step produces one observation for each repeating field. The MISSEVER option in the INFILE statement prevents SAS from reading the next record when missing values occur at the end of a record. Every observation contains one value for Flavor, paired with the corresponding value for ID. Because there are 12 values for Flavor, there are 12 observations in the data set.

---

## Chapter 22: Conditioning Raw Data Files

1. Correct answer: d

In order to create one observation per detail record, it is necessary to distinguish between header and detail records. Use a RETAIN statement to keep header data as part of each observation until the next header record is encountered. You also need to

use the @ line-hold specifier to hold the current record so that other values in the record can be read.

2. Correct answer: a

An INPUT statement is used to read the value for code. The single @ sign at the end of the INPUT statement holds the current record for a later INPUT statement in the same iteration of the DATA step.

3. Correct answer: b

The IF-THEN statement defines the condition that Record equals C and executes an INPUT statement to read the value for Amount when the condition is true. C must be enclosed in quotation marks and must be specified exactly as shown because it is a character value.

4. Correct answer: b

The value of Vegetable is retained across iterations of the DATA step. As the sixth iteration begins, the INPUT statement reads the value for code and holds the record, so that the values for Variety and Supplier can be read with an additional INPUT statement.

5. Correct answer: c

This program creates one observation for each detail record. The RETAIN statement retains the value for Fruit as part of each observation until the values for Variety and Price can be read. The DROP= option in the DATA statement prevents the variable for type from being written to the data set.

6. Correct answer: a

The IF-THEN statement defines the condition that Record equals A and specifies a simple DO group. The keyword DO indicates that several executable statements follow until the DO group is closed by an END statement. The value A must be enclosed in quotation marks and specified exactly as shown because it is a character value.

7. Correct answer: d

\_N\_ is an automatic variable whose value is the number of times the DATA step has begun to execute. The expression `_n_ > 1` defines a condition where the DATA step has executed more than once. When the conditions `code='X'` and `_n_ > 1` are true, an OUTPUT statement is executed, and Total is initialized to zero. Thus, each header record except the first one causes an observation to be written to the data set.

8. Correct answer: d

The condition is false, so the values for ID and Address are not read. Instead, the ELSE statement is executed and defines another condition that might or might not be true.

9. Correct answer: d

You can determine when the current record is the last record in an external file by specifying the END= option in the INFILE statement. last is a temporary numeric variable whose value is zero until the last line is read. Last has a value of 1 after the last line is read. Like automatic variables, the END= variable is not written to the data set.

10. Correct answer: b

This program creates one observation for each header record and combines information from each detail record into the summary variable, Total. When the value of Type is D and the value of \_N\_ is greater than 1, the OUTPUT statement

executes, and the values for Department, Total and Extension are written to the data set as the first observation. The variables `_N_`, Last, Type, and Amount are not written to the data set.



# Index

---

## Special Characters

\_ALL\_ keyword 127, 385, 387  
 \_CHARACTER\_ keyword 385, 386  
 \_ERROR\_ automatic variable  
     DATA step iterations and 148, 266, 270  
     functionality 141  
     initializing variables 144  
     PUT statement and 160  
 \_N\_ automatic variable  
     DATA step processing and 147, 266, 269, 272  
     functionality 141, 516  
     PUT statement and 160  
 \_NULL\_ keyword 118  
 \_NUMERIC\_ keyword 385, 386  
 \_TEMPORARY\_ keyword 396  
 @ (trailing at-sign) 481, 488, 509  
 @@ (double trailing at-sign) 481  
 @n pointer control 403  
 \$BINARYw. informat 406  
 \$VARYINGw. informat 406  
 \$w. format 91  
 \$w. informat  
     ampersand modifier 432  
     functionality 406, 407  
 #n pointer control  
     combining pointer controls 473  
     reading records non-sequentially 469  
 +n pointer control 405

## A

ampersand modifier 431, 434  
 AND operator  
     examples 236  
     in logical operations 235  
     in SAS expressions 113  
     in WHERE statement, PRINT  
         procedure 73  
 APPEND procedure  
     examples 287  
     FORCE option 288  
     functionality 278, 286  
     requirements 287

appending data sets 278, 286  
 arguments in functions 315  
 arithmetic operators in SAS expressions 112  
 array elements  
     assigning initial values 394  
     creating temporary 396  
     referencing 387  
     specifying 385  
     variables lists as 385  
 ARRAY statement  
     arrays of character variables 393  
     assigning initial values to arrays 394  
     creating temporary array elements 396  
     creating variables 391  
     defining one-dimensional arrays 384  
     DO loop processing 389, 390  
     referencing array elements 387  
     specifying array name 385  
     specifying dimensions 385  
     specifying elements 385  
     syntax 384  
     variable lists as elements 385  
 arrays 384  
     concatenating names 392  
     creating one-dimensional 383  
     creating variables 391  
     defining 384  
     DO loop processing 389, 390  
     function arguments and 316  
     specifying dimensions 385  
     specifying names 385  
 assignment statements 113  
     conditional processing 235  
     DATA step processing 146, 268, 271  
     date constants 115  
     examples 113  
     positioning SUBSTR function 345  
     SAS expressions in 112  
 asterisk (\*) 191  
 attributes  
     *See* variable attributes  
 automatic variables  
     DATA step processing 270

PUT statement 160  
 testing programs 160

## B

BEST12. format 321  
 BESTw. format 180  
 blanks  
   compressing 359  
 browsers 37  
   preferred web browser 38  
 BY clause, DO statement 370, 372  
 BY statement  
   DESCENDING option 294  
   group processing with 183, 259  
   interleaving data sets 278, 289  
   match-merging data sets 278, 291  
   PRINT procedure 77, 79  
   SORT procedure 75, 294  
   syntax 183  
 BY variables  
   finding observations in subgroups 261  
   interleaving data sets 289

## C

calculations  
   dates and times in 451  
 case sensitivity  
   format values 168  
 CATX function  
   functionality 349  
   syntax 349  
 CEIL function 360  
 character strings 159  
   PUT statement 159  
   searching 350, 351  
   specifying delimiters 341  
   testing programs 159  
 character variables 407  
   array processing and 386  
   creating arrays of 392  
   embedded blanks 431  
   informat support 407  
   reading range of 421  
   removing trailing blanks 347, 348  
   replacing 344  
   searching for strings 350  
   specifying value length 421  
 character-to-numeric conversions 316, 317  
 CLASS statement, MEANS procedure 182  
 cleaning data 155  
 CLM statistic 178  
 code editing windows

  resubmitting revised programs 54  
 colon (:) modifier 431, 433  
 column input  
   mixing styles 439  
   nonstandard data values and 402  
 column pointer controls  
   @n 403  
   +n 405  
   functionality 404  
 columns  
   See *variables*  
 combining data sets 278  
   by appending 278, 286  
   by concatenating 278, 284  
   by interleaving 278, 289  
   by match-merging 278, 291  
   by one-to-one reading 278, 280  
   excluding unmatched observations 302  
   methods for 278  
   renaming variables 300  
 comma (,) 21, 394  
 COMMA10. informat 21  
 COMMA12.2 format 93  
 COMMA6. format 244  
 COMMA8.2 format 21  
 COMMA9. informat 320  
 COMMAw.d format 91, 92  
 COMMAw.d informat  
   example 406  
   functionality 408, 434  
   modify list input and 433  
 common errors  
   missing RUN statement 52  
   missing semicolon (;) 52  
   unbalanced quotation mark 52  
 comparison operators 235  
 compilation phase (DATA step)  
   data set variables 142  
   descriptor portion of data sets 143  
   diagnosing errors in 152  
   input buffer 141  
   match-merge processing 296  
   program data vector 141  
   reading data sets 266  
   syntax checking 142  
 COMPRESS function  
   compressing blanks 359  
   examples 359  
 compressing character strings  
   blanks 359  
 concatenating array names 392  
 concatenating data sets 278, 284, 285  
 concatenation operator (||) 321, 322  
 conditional processing  
   assigning variable values 234, 245  
   assignment statements 235

- comparison operators 235
- DO groups 248
- DO loops 376
- iterative DO statement 377
- logical operators 235
- providing alternative actions 237
- PUT statement and 161
- SELECT groups 245
- single observations per detail records 509
- testing programs 161, 238
- CONTAINS operator 73
- CONTENTS procedure
  - reading Excel data 122, 127
  - VARNUM option 35
  - viewing library contents 31
- CONTENTS statement, DATASETS procedure 35
- converting data
  - See [data conversion](#)
- counter variables 497
- CROSSLIST option, TABLES statement (FREQ) 195
- CSS statistic
  - MEANS procedure 178
- current record, holding with line-hold specifier 481, 488
- CV statistic
  - MEANS procedure 178

## D

- data cleaning 155
- data conversion 316
  - character-to-numeric 317
  - lower case 354
  - numeric-to-character 316
  - numeric-to-character conversion 321
  - upper case 353
- data sets
  - See also [combining data sets](#)
  - See also [match-merging data sets](#)
  - BY-group processing 259
  - data portion 16
  - descriptor portion 15, 143
  - detecting end of 264
  - dropping and keeping variables 241
  - iteratively processing data 375
  - listing 109
  - manipulating data 257
  - missing values 17
  - naming 104
  - naming conventions 14
  - observations (rows) 16
  - reading 255, 266

- specifying observations via system options 46
- summarized using MEANS procedure 184
- summarized using SUMMARY procedure 186
- testing programs 159
- variable attributes 17, 142
- variables and 17
- viewing descriptor information 35
- DATA step
  - characteristics 7
  - checking processing 108
  - column input 105
  - compilation phase 141, 266, 296
  - creating data sets from raw data files 101
  - creating raw data files 118
  - creating/modifying variables 111
  - debugging 152
  - describing data in programs 106
  - execution phase 267, 297
  - functionality 139
  - invalid data 109
  - iterations of 148, 490
  - listing data sets 109
  - manipulating data 257
  - naming data sets 104
  - nonstandard numeric data in 105
  - PROC IMPORT 121
  - processing informats 409
  - reading Excel data 121, 122, 125
  - reading raw data files 109
  - repeating blocks of data 482
  - single observations per detail records 511
  - single observations per header records 520
  - specifying raw data files 104
  - specifying variable names 107
  - standard numeric data in 105
  - submitting 108
  - subsetting data 115
  - syntax 104
  - varying number of repeating fields 497
  - verifying data 108
  - writing 104, 125
- data validation 155
- DATALINES statement 116, 117
- DATASETS procedure 33, 35
- DATDIF function 325, 338
- DATE function 325, 332
- DATE7. format 93
- DATE9. format 93, 327, 328
- dates and times
  - displaying 482

- handling four-digit years 44
  - handling two-digit years 42
  - in calculations 451
  - informat support 447
  - manipulating for output 42
  - manipulating with functions 323
- DATETIMEw. format 447
- DATETIMEw. informat 406, 450
- DATEw. format
  - examples 91, 447, 482
  - functionality 437
- DATEw. informat
  - examples 406
  - functionality 449
  - syntax 449
- DAY function
  - functionality 314
  - manipulating date values 326
  - syntax 326
  - typical use 325
- debugging
  - cleaning data 155
  - diagnosing errors in compilation phase 152
  - diagnosing errors in execution phase 152
  - validating data 155
- decimal places, limiting 180
- DELETE statement
  - example 240
  - in IF-THEN statement 240
- delimiters
  - DLM= option, FILE statement 437
  - DLM= option, INFILE statement 424, 429
  - DSD option, FILE statement 438
  - DSD option, INFILE statement 428
  - for SCAN function 341
  - free-format data 424, 429
  - specifying multiple 342
  - within quoted strings 426
- DESCENDING option
  - BY statement, SORT procedure 75
- descriptive statistics 176
  - creating summarized data sets 184, 186
  - creating tables in list format 194
  - group processing with BY statement 183
  - group processing with CLASS statement 182
  - limiting decimal places 180
  - procedure syntax 176
  - producing frequency tables 187, 191
  - selecting statistics 177
  - specifying variables in FREQ procedure 189
  - specifying variables in MEANS procedure 181
- detail record, single observation per
  - conditional processing 509
  - DATA step processing 511
  - displaying results 514
  - dropping variables 511
  - overview 508
  - reading detail records 510
  - retaining variable values 509
- DIM function 390
- DLM= option
  - FILE statement 437
  - INFILE statement 424, 429
- DO groups
  - indenting 249
  - iteratively processing data 373
  - nesting 249
  - single observations per header records 516
- DO loops
  - conditionally executing 376
  - constructing 368
  - counting iterations 371
  - decrementing 372
  - functionality 370
  - index variables 371, 372, 374, 388
  - INPUT statement example 490
  - iterative DO statement 377
  - nesting 373
  - processing array elements 389, 390
  - specifying series of items 372
- DO statement
  - See also* iterative DO statements
  - BY clause 370, 372
  - grouping statements 248
  - TO clause 369
- DO UNTIL statement 249, 376, 377
- DO WHILE statement
  - counter variables 497
  - executing 497
  - functionality 249, 376, 377, 496
  - syntax 377
- Document destination 204
- dollar sign (\$)
  - arrays of character variables 392
  - in format names 168
  - informat and 21, 407
  - name literals and 124, 125
- DOLLAR12.2 format 93
- DOLLAR8.2 format 20, 93
- DOLLAR9.2 format 93
- DOLLARw.d format 91
- DOUBLE option, PRINT procedure 82
- double spacing LISTING output 82
- double trailing at-sign (@@) 481

DROP statement 242, 384  
 DROP= data set option  
   assigning initial values to arrays 395  
   determining when to specify 258  
   examples 511, 520  
   selecting variables 241  
 DSD option  
   FILE statement 438  
   INFILE statement 428

## E

END statement  
   assigning values conditionally 245  
   grouping statements 248  
 end-of-file marker 150  
 END= option, SET statement 264  
 engines  
   See [SAS engines](#)  
 eq (=) operator 235  
 error handling 53  
   correcting common errors 52  
   error types 51  
   IF-THEN/ELSE statement 161  
   in DATA step compilation phase 152  
   in DATA step execution phase 152  
   interpreting messages 51  
   invalid data 109  
   invalid option 61  
   resubmitting revised programs 53  
   semicolon errors 57  
   unbalanced quotation marks 59, 60  
   validating or cleaning data 155  
 Excel data  
   See [Microsoft Excel data](#)  
 execution phase (DATA step)  
   diagnosing errors in 152  
   end of processing actions 146, 151  
   end-of-file marker 150  
   initializing variables 144  
   input data 144  
   input pointer 144  
   iterations of DATA step 148  
   match-merge processing 297  
   reading data sets 267  
 expressions  
   See [SAS expressions](#)  
 external files  
   See [raw data files](#)

## F

FILE statement  
   DATA step processing 270  
   DLM= option 437  
   DSD option 438

  functionality 118  
   syntax 118  
 FILENAME statement  
   creating data sets from raw data files 101, 102  
   defining aggregate storage location 103  
   describing data in programs 106  
   listing data sets 109  
   naming data sets 104  
   specifying raw data files 104  
   syntax 102  
 filerefs  
   associating with external files 102  
   fully qualified filenames in 102, 103  
   in aggregate storage locations 103  
 files  
   See [raw data files](#)  
   See [SAS files](#)  
 FIND function  
   examples 352  
   functionality 351  
   syntax 351  
 FIRST.variable  
   examples 259  
   finding observations in subgroups 261  
   values supported 259  
 FIRSTOBS= system option 44  
 fixed-length records 410  
 FLOOR function 360  
 FLOWOVER option, INFILE statement 496  
 FMTLIB keyword 171  
 FOOTNOTE statement  
   canceling 85  
   examples 84  
   modifying 85  
   quotation marks in 59  
   specifying in list reports 82  
 FOOTNOTES command 83  
 FOOTNOTES window 83  
 FORCE option, APPEND procedure 288  
 FORMAT procedure  
   FMTLIB keyword 171  
   functionality 165, 166  
   invoking 166  
   LIBRARY= option 166  
   syntax 166  
   VALUE statement 167, 168, 170  
 FORMAT statement  
   array name restrictions 384  
   assigning formats to variables 170, 243, 457  
   displaying date/time values 482  
   formatting dates 327, 328  
   functionality 90  
   syntax 90

formats 20  
   assigning permanent 243, 457  
   assigning to variables 170, 457  
   decimal places 92  
   defining unique 167  
   examples 93  
   field widths 92  
   for variables 90  
   functionality 90  
   permanently assigned 93  
   specifying 91  
   storing 166  
   storing permanently 166  
   writing numeric values 180

formatted input  
   @n pointer control 403  
   +n pointer control 405  
   mixing styles 439  
   modified list input comparison 435  
   reading columns in any order 404  
   syntax 403

forward slash (/)  
   specifying multiple delimiters 342

forward slash (/) pointer control  
   combining pointer controls 473  
   reading records sequentially 462

FRAME= option  
   syntax 210

free-format data  
   *See* raw data files

FREQ procedure  
   *See also* TABLES statement, FREQ procedure  
   detecting invalid data 155  
   producing frequency tables 187, 191  
   specifying variables 189  
   suppressing table information 196  
   syntax 155, 187

frequency tables  
   creating in list format 194  
   n-way 187, 192  
   one-way 187  
   suppressing information 196  
   two-way 191

functions  
   arguments 315  
   arrays and 315  
   character-to-numeric conversions 317  
   converting data 316  
   manipulating date/time values 323  
   syntax 315  
   target variables and 316  
   variable lists 315

**G**

ge (>=) operator 235

group processing  
   finding observations in subgroups 261  
   with BY statement 183, 259  
   with CLASS statement 182

gt (>) operator 235

**H**

header record, single observation per  
   DATA step processing 520  
   determining end of external file 518  
   DO group actions 516  
   overview 515  
   reading detail records 517

HEXw. informat 406

hierarchical files 507  
   creating observation per detail record 508  
   creating observation per header record 515

HIGH keyword 168

HTML destination 204

HTML Link and Path Options  
   URL= suboption 211

HTML output 36  
   creating 36  
   ODS overview 204  
   overview 206  
   viewing 37

HTML Output  
   Appearance of HTML 214  
   Frame Files 210  
   Link and Path Options 211, 212  
   Specify Link and Path 211  
   Table of Contents 208, 210

HTML Table of Contents  
   CONTENTS= option 210

hyphen (-) 342

hypothesis testing 178

**I**

ID statement, PRINT procedure  
   BY statement and 79  
   VAR statement and 71

IF-THEN statement  
   assigning values conditionally 234  
   cleaning data 157  
   conditionally executing statements 510  
   DELETE statement in 240  
   DO groups 248  
   ELSE statement 237  
   examples 234  
   for flagging errors 161

- SELECT groups 245
  - single observations per header records 516, 517
  - syntax 234
  - testing programs 158
  - in operator 235
  - IN= data set option 302
  - indenting DO groups 249
  - INDEX function
    - functionality 350
    - syntax 350
  - index variables in DO loops 371, 372, 374, 388
  - INFILE statement
    - column input 105
    - creating data sets from raw data files 101
    - DATA step processing 144, 270
    - describing data in programs 106
    - DLM= option 424, 429
    - DSD option 428
    - FLOWOVER option 496
    - listing data sets 109
    - MISCOVER option 427, 496
    - OBS= option 108, 109
    - PAD option 412
    - SCANOVER option 496
    - single observations per header records 519
    - specifying raw data files 104
    - STOVER option 496
    - syntax 104, 519
    - TRUNCOVER option 496
  - informats 21
    - ampersand modifier 432
    - components 447
    - DATA step processing 409
    - modifying list input 435
    - reading character values 407
    - reading dates and times 447
    - reading numeric data 407
    - specifying 448
  - initializing variables 144, 233
  - input buffer 141, 148
  - INPUT function
    - character-to-numeric conversion 316, 319
    - examples 320
    - syntax 319
  - INPUT statement
    - creating data sets from raw data files 101
    - DATA step processing 144, 148
    - describing data in programs 106
    - DO loop example 490
    - examples 106
    - line-hold specifier 481, 489
    - listing data sets 109
    - mixing input styles 441
    - pointer controls 403, 405, 462, 469
    - processing informats 409
    - processing list input 419
    - specifying informats 448
    - syntax 106, 403, 405, 418, 448
    - variable-length records 411
  - INT function 360
  - INTCK function
    - examples 336
    - functionality 325, 335
    - syntax 335
  - interface library engines 30
  - interleaving data sets
    - example 290
    - functionality 278, 289
  - Internal Browser option 37
  - INTNX function 325, 337
  - invalid data 109, 155
  - invalid option 61
  - iterative DO statements 376
    - conditional clauses 377
    - conditional executing 376
    - DATA step example 490
    - DIM function 390
    - nesting DO loops 373
- J**
- JULIANw. informat 406
- K**
- KEEP statement 242, 384
  - KEEP= data set option
    - determining when to specify 258
    - selecting variables 241
  - KURTOSIS statistic, MEANS procedure 178
- L**
- LABEL option, PRINT procedure 88, 244
  - LABEL statement 243
    - array name restrictions 384
    - assigning labels in multiple 89
    - assigning labels in single 89
    - example 88
    - functionality 88
    - syntax 88
  - labels
    - assigning descriptive 88
    - assigning for variables 21



- assigning permanent 93, 243
  - LAST.variable
    - examples 259
    - finding observations in subgroups 261
    - values supported 259
  - LCLM statistic, MEANS procedure 178
  - le (<=) operator 235
  - leading blanks, removing 349
  - LEFT function 347
  - LENGTH statement
    - ampersand modifier 432
    - array name restrictions 384
    - examples 239
    - functionality 238, 342
    - specifying character value length 423
  - length, variable 20, 342
  - LIBNAME statement
    - assigning librefs 29
    - creating data sets from raw data files 101
    - defining SAS libraries 28
    - describing data in programs 106
    - listing data sets 109
    - naming data sets 104
    - reading Excel data 121
    - referencing files in other formats 30
    - specifying raw data files 104
    - syntax 28
  - libraries
    - See SAS libraries
  - LIBRARY= option, FORMAT procedure 166
  - librefs
    - assigning 28
    - defined 13
    - lifespan of 29
    - verifying 29
  - line pointer controls
    - / (forward slash) 462
    - #n 469
    - combining 473
    - functionality 462
    - reading records non-sequentially 469
    - reading records sequentially 462
  - line-hold specifiers
    - double trailing at-sign 481
    - overview 481
    - trailing at-sign 481, 488, 509
  - list input
    - See modified list input
  - LIST option, TABLES statement (FREQ) 194
  - list reports
    - creating 178
    - creating tables for 194
    - double spacing output 82
    - formatting data values 90
    - generating column totals 76
    - identifying observations 70
    - selecting observations 179
    - selecting variables 69
    - sorting data 74
    - specifying footnotes 82
    - specifying titles 82
  - LISTING destination 204
  - log messages 8
  - Log window
    - clearing 53
    - resubmitting revised programs 54
  - logic errors
    - PUTLOG statement 55
  - logical operators
    - conditional processing 235
    - in SAS expressions 113
  - LOW keyword 168
  - LOWCASE function 351, 354
  - lt (<) operator 235
- M**
- Markup Languages Family destination 204
  - match-merging data sets 295
    - compilation phase 296
    - examples 292
    - execution phase 297
    - functionality 278, 291
    - handling missing values 298
    - handling unmatched observations 298
    - selecting data 292
  - MAX statistic
    - MEANS procedure 178, 185
  - MAXDEC= option, MEANS procedure 180
  - MDY function
    - examples 331
    - functionality 325
    - missing values 332
  - MEAN statistic
    - MEANS procedure 178, 185
  - MEANS procedure
    - BY statement 183
    - CLASS statement 182
    - creating summarized data sets 184
    - descriptive statistics 178
    - detecting invalid data 155, 156
    - functionality 176
    - hypothesis testing 178
    - keywords supported 177
    - limiting decimal places 180
    - MAXDEC= option 180
    - OUTPUT statement 184



- quantile statistics 178
- selecting statistics 177
- specifying variables 181
- syntax 156, 176
- VAR statement 156, 181
- MEDIAN statistic, MEANS procedure 178
- MERGE statement
  - match-merging data sets 278, 291
  - RENAME= data set option 301
  - RETAIN statement and 233
  - syntax 291
- merging data sets 289
- Microsoft Excel data 130
  - CONTENTS procedure 122, 127
  - creating worksheets 130
  - DATA statement 122, 125
  - LIBNAME statement 121
  - name literals 125
  - named ranges 126
  - overview 121
  - PRINT procedure 122, 125, 126
  - PROC IMPORT 121
  - referencing workbooks 123
  - RUN statement 122, 125
  - SAS/ACCESS LIBNAME statement 121, 122, 129
  - SET statement 122
  - steps for reading 122
  - WHERE statement, DATA step 125
  - writing the DATA step 125
- MIN statistic
  - MEANS procedure 178, 185
- missing values
  - at beginning/middle of record 428
  - at end of record 427
  - in match-merge processing 298
  - MDY function and 332
  - overview 17
- MISSEVER option, INFILE statement 427, 496
- MMDDYY10. format 93
- MMDDYY8. format 93
- MMDDYYw. format 91
- MMDDYYw. informat
  - examples 406, 447
  - functionality 448
  - syntax 448
- MODE statistic
  - MEANS procedure 178
- modified list input 418
  - ampersand modifier 431
  - colon modifier 431, 433
  - formatted input comparison 435
  - mixing styles 439
  - processing 418
- MONTH function
  - examples 327
  - functionality 314
  - manipulating date values 326
  - syntax 326
  - typical use 325
- multiple observations from single record
  - overview 479
  - reading repeating blocks of data 480
  - reading same number of repeating fields 487
  - reading varying number of repeating fields 495
- multiple records, single observation from
  - line pointer controls 462, 473
  - overview 461
  - reading records non-sequentially 469
  - reading records sequentially 462
- N**
- N statistic
  - MEANS procedure 178, 185
- n-way frequency tables 187, 192
- naming conventions
  - for variables 18, 107
  - SAS data sets 14
- ne (^=) operator 235
- NENGOW. informat 406
- nesting
  - DO groups 249
  - DO loops 373
- NMISS statistic
  - MEANS procedure 178
- NOCOL option, TABLES statement (FREQ) 197
- NOCUM option, TABLES statement (FREQ) 191
- NOFREQ option, TABLES statement (FREQ) 197
- NOOBS option, PRINT procedure 70
- NOPERCENT option, TABLES statement (FREQ) 197
- NOROW option, TABLES statement (FREQ) 197
- NOT operator 235, 236
- numeric variables 402
  - array processing and 386
  - identifying nonstandard data 402
  - reading nonstandard data 408
  - reading range of 420
  - reading standard data 407
- numeric-to-character conversion 316, 321

**O**

- OBS= option, INFILE statement 108, 109
- OBS= system option 44
- observations 16
  - See also* combining data sets
  - combining from multiple data sets 278
  - creating for DO loop iterations 371
  - creating from multiple records 462
  - creating from single record 480
  - creating per detail record 508
  - creating per header record 515
  - deleting 240
  - detecting end of data sets 264
  - finding in subgroups 261
  - identifying 70
  - limiting when testing programs 158
  - selecting in list reports 71
  - selecting matching 303
  - specifying via system options 44
  - unmatched 298, 302
  - writing explicitly 263
- ODS \_ALL\_ CLOSE statement 205
- ODS (Output Delivery System)
  - advantages 204
  - HTML support 206, 208
  - opening and closing destinations 204
- ODS destinations 204
- ODS EXCEL destination
  - TAGATTR= style 223
- ODS EXCEL Statement
  - Syntax 222
- ODS HTML CLOSE statement
  - syntax 205
- ODS HTML statement
  - Table of Contents syntax 208
- ODS HTML Statement
  - syntax 206
- ODS LISTING CLOSE statement 205
- ODS PDF Destinations
  - Open and Close Statements 216
  - Table of Contents 216
- ODS PDF Statement
  - Statements 216
  - syntax 215
- ODS RTF
  - RTF Formats 221
  - RTF Graphics 221
- ODS RTF Destinations
  - Open and Close Statements 221
- ODS RTF Statement
  - syntax 220
- ODS statements 204
- one-to-one reading of data sets
  - example 283
  - functionality 278, 280, 281
  - selecting data 281
- one-way frequency tables 187
- operands 112
- operators
  - comparison 235
  - concatenation 321, 322
  - defined 112
  - in SAS expressions 112
  - logical 113, 235
- OPTIONS statement 41
- Options window
  - Results tab 38, 40
- OR operator
  - examples 236
  - in logical operations 235
  - in SAS expressions 113
  - in WHERE statement, PRINT procedure 73
- OTHER keyword 168
- OTHERWISE statement 245
- output
  - See also* HTML output
  - double spacing 82
  - manipulating date and times 42
  - setting system options 41
- Output Delivery System (ODS)
  - EXCEL 222, 223
  - PDF 215, 216, 219
  - RTF 220, 221
- Output destination 204
- OUTPUT statement
  - creating for DO loop iterations 371
  - functionality 263
  - single observations per header records 519
  - syntax 263
- OUTPUT statement (MEANS) 185
- OUTPUT statement, MEANS procedure
  - functionality 184
  - syntax 184

**P**

- P1 statistic, MEANS procedure 178
- P10 statistic, MEANS procedure 178
- P25 statistic, MEANS procedure 178
- P5 statistic, MEANS procedure 178
- P50 statistic, MEANS procedure 178
- P75 statistic, MEANS procedure 178
- P90 statistic, MEANS procedure 178
- P95 statistic, MEANS procedure 178
- P99 statistic, MEANS procedure 178
- PAD option, INFILE statement 412
- parentheses ()
  - for function arguments 315
  - logical comparisons in 236
- PATH= Option

- syntax 212
  - PDV
    - See [program data vector](#)
  - PDw.d informat 406
  - PERCENTw. informat 406
  - PERCENTw.d informat 406
  - period (.)
    - in SAS filenames 13
  - pointer controls
    - / (forward slash) 463
    - @n 403
    - #n 469
    - +n 405
    - combining 473
    - functionality 404, 462
    - reading records non-sequentially 469
    - reading records sequentially 462
  - Preferences window
    - depicted 36, 39, 40
    - opening 36
    - Preferred Web Browser option 38
    - Results tab 36
  - preferred web browser 38
  - PRINT option, SUMMARY procedure 186
  - PRINT procedure
    - BY statement 77, 79
    - creating data sets from raw data files 101
    - creating list reports 68
    - DOUBLE option 82
    - ID statement 71, 79
    - LABEL option 88, 244
    - listing data sets 109
    - NOOBS option 70
    - reading Excel data 122, 125, 126
    - SUM statement 76, 77
    - VAR statement 69, 71
    - WHERE statement 71
  - Printer Family destination 204
  - PROBT statistic, MEANS procedure 178
  - PROC IMPORT
    - reading Excel data 121
  - PROC step
    - characteristics 7
    - missing RUN statement 57
    - OPTIONS statement and 41
  - program data vector 141
    - DATA step processing 141, 148, 149
    - match-merge processing 295, 296, 297, 298
    - reading data sets 266, 268, 272
  - programming workspace
    - SAS libraries 11
  - PROPCASE function 355
  - PUT function
    - numeric-to-character conversion 316, 321
    - syntax 322
  - PUT statement
    - automatic variables 160
    - character strings 159
    - conditional processing and 161
    - data set variables 159
    - functionality 118, 436
    - syntax 119, 158, 436
    - testing programs 158, 159, 238
  - PUTLOG statement 55
- Q**
- Q1 statistic, MEANS procedure 178
  - Q3 statistic, MEANS procedure 178
  - QRANGE statistic, MEANS procedure 178
  - QTR function
    - functionality 314
    - manipulating date values 326
    - syntax 326
    - typical use 325
  - quantile statistics 178
  - quotation marks
    - common errors 58
    - delimiters within strings 426
    - format names and 168
    - logical operations 236
    - numeric-to-character conversion 322
    - reading Excel data 124
- R**
- RANGE statistic
    - MEANS procedure 178
  - raw data files 100
    - See also [hierarchical files](#)
    - creating 118
    - describing the data 106, 119
    - formatted input 403
    - free-format data 417, 436
    - invalid data 109, 110
    - mixing input styles 439
    - modifying list input 431
    - nonstandard numeric data 402
    - processing list input 418
    - reading entire files 109
    - reading missing values 427
    - record formats 410
    - sample record layout 100
    - specifying 104, 118
    - specifying character value length 421
  - records
    - fixed-length 410

- multiple observations from single record 480
  - single observation from multiple records 462
  - single observation per detail record 508
  - single observation per header record 515
  - variable-length 410
  - RENAME= data set option 301
  - renaming variables 300
  - repeating blocks of data, reading 480
  - repeating fields
    - reading same number of 487
    - reading varying number of 495
  - Results Viewer window 37
  - RETAIN statement
    - DATA step processing 270
    - initializing sum variables 233
    - single observations per detail records 509
    - syntax 233
  - RIGHT function 347
  - ROUND function 361
  - rows
    - See* observations
  - RTF destination 204
  - RUN statement
    - creating data sets from raw data files 101
    - DATA step processing 148
    - describing data in programs 106
    - listing data sets 109
    - reading Excel data 122, 125
- S**
- SAS engines 30
    - interface library engines 30
  - SAS expressions 112
    - accumulating totals 232
    - arithmetic operators in 112
    - logical operators in 113
    - SELECT statements 247
    - specifying compound 73
    - specifying in list reports 72
  - SAS files 13
    - naming conventions 14
    - referencing 13
    - referencing in other formats 30
    - storing 12
    - temporary 14
    - two-level names 13, 28, 30
  - SAS formats
    - See* formats
  - SAS informats
    - See* informats
  - SAS libraries 11
    - creating 11
    - DATASETS procedure 33
    - defining 11, 28
    - deleting 12
    - storing SAS files 12
    - viewing 31
    - viewing library contents 31
  - SAS log 321
  - SAS programs
    - DATA step processing 139
    - error handling 51
    - log messages 8
    - processing 8
    - resubmitting revised 53
    - results of processing 9
  - SAS sessions
    - libref lifespan and 29
    - OPTIONS statement and 42
  - SAS statements
    - See also* assignment statements
    - conditionally executing 509
    - DO groups 248
    - executing repeatedly 368
    - executing when true 496
    - SELECT groups 245
  - SAS System Options window
    - depicted 47
    - finding options quickly 47
    - functionality 47
    - SAS Options Environment pane 47
  - SAS/ACCESS engines 30
  - SAS/ACCESS LIBNAME statement
    - reading Excel data 121, 122
    - syntax 123
  - Sashelp library 11
  - Sasuser library 11
  - SCAN function
    - functionality 340
    - specifying delimiters 341
    - specifying variable length 342
    - SUBSTR function versus 346
    - syntax 340
  - SCANOVER option, INFILE statement 496
  - SELECT groups 245
  - SELECT statement 245
  - semantic errors 51
  - semicolon (;)
    - common errors 57
  - SET statement
    - BY statement and 181
    - concatenating data sets 278, 284
    - END= option 264
    - interleaving data sets 278, 289
    - one-to-one reading 278, 280

- reading data sets 255, 266
  - reading Excel data 122, 125
  - RETAIN statement and 233
  - syntax 255
  - single observation from multiple records
    - line pointer controls 462, 473
    - overview 461
    - reading records non-sequentially 469
    - reading records sequentially 462
  - single observation per detail record
    - conditional processing 509
    - DATA step processing 511
    - displaying results 514
    - dropping variables 511
    - overview 508
    - reading detail records 510
    - retaining variable values 509
  - single observation per header record
    - DATA step processing 520
    - determining end of external file 518
    - DO group actions 516
    - overview 515
    - reading detail records 517
  - single record, multiple observations from
    - overview 479
    - reading repeating blocks of data 480
    - reading same number of repeating fields 487
    - reading varying number of repeating fields 495
  - SKEWNESS statistic, MEANS procedure 178
  - SORT procedure
    - BY statement 75, 294
    - examples 75
    - sorting data in list reports 74
    - syntax 74
  - sorting data in list reports 74
  - statistics
    - quantile 178
    - summary 185
  - STD statistic
    - MEANS procedure 178, 185
  - STDDEV statistic, MEANS procedure 178
  - STDERR statistic
    - MEANS procedure 178
  - STEP statement 125
  - STOPOVER option, INFILE statement 496
  - storing
    - formats 166
    - SAS files 12, 103
  - strings
    - See [character strings](#)
  - STYLE= option
    - syntax 214, 219
  - subgroups, finding observations in 261
  - subsetting data 240
  - subsetting IF statement
    - examples 116, 264
    - finding year 328
    - functionality 115, 258
    - selecting matching observations 303
    - syntax 115
  - SUBSTR function
    - functionality 321, 342
    - positioning 345
    - replacing text 344
    - SCAN function versus 346
    - syntax 343
  - subtotaling variables 77, 189
  - sum statement
    - accumulating totals 232
    - DO loops and 369
    - reading data sets 269, 270
    - syntax 232
  - SUM statement, PRINT procedure
    - creating customized layouts 79
    - generating column totals 76
    - requesting subtotals 77
    - syntax 76
  - SUM statistic
    - MEANS procedure 178
  - sum variables, initializing 233
  - SUMMARY procedure
    - creating summarized data sets 186
    - PRINT option 186
  - summary statistics 185
  - SUMWGT statistic
    - MEANS procedure 178
  - syntax errors 51
  - system options
    - changing 47
    - finding quickly 47
    - modifying via OPTIONS statement 41
    - setting 41
    - specifying observations 44
- ## T
- T statistic
    - MEANS procedure 178
  - TABLES statement, FREQ procedure 155
    - creating n-way tables 192
    - creating two-way tables 191
    - CROSSLIST option 195
    - examples 189
    - LIST option 194
    - NOCOL option 197
    - NOCUM option 191
    - NOFREQ option 197

- NOPERCENT option 197
- NOROW option 197
- syntax 189, 191
- target variables
  - defined 316
  - missing values and 332
- temporary array elements 396
- temporary variables 181, 261, 302
- testing programs
  - automatic variables 160
  - character strings 159
  - conditional processing 161, 238
  - data set variables 159
  - hypothesis testing 178
  - limiting observations 158
  - PUT statement 158
- TIME function 325
- TIMEw. format 447
- TIMEw. informat 406, 449
- TITLE statement
  - canceling 85
  - examples 83
  - modifying 85
  - quotation marks in 59
  - specifying in list reports 82
- TITLES command 83
- TITLES window 83
- TO clause, DO statement 369
- TODAY function 325
- trailing at-sign (@) 481, 488, 509
- trailing blanks, removing 347, 348, 349
- TRANWRD function 356
- TRIM function 348
- TRUNCOVER option, INFILE statement 496
- two-way frequency tables 191

## U

- UCLM statistic, MEANS procedure 178
- underscore ( \_ ) 18
- UNIX environment
  - SAS library implementation 12, 29
  - storing files 12
  - unbalanced quotation marks 59, 60
- unmatched observations
  - excluding 302
  - handling 298
- UPCASE function 351, 353
- UPDATE statement 233
- USS statistic
  - MEANS procedure 178

## V

- validating data 155

- VALUE statement, FORMAT procedure
  - assigning formats to variables 170
  - functionality 167
  - HIGH keyword 168
  - LOW keyword 168
  - OTHER keyword 168
  - specifying value ranges 168
  - syntax 167
- VAR statement
  - MEANS procedure 156, 178, 181, 185
  - PRINT procedure 69, 71
- VAR statistic
  - MEANS procedure 178
- variable attributes 17
  - data sets 17, 142
  - format considerations 20
- variable lists
  - as array elements 385
  - function arguments and 316
- variable-length records 410
- variables 17
  - See also* arrays
  - See also* character variables
  - See also* numeric variables
  - See also* variable attributes
  - accumulating totals 232
  - assigning formats 170, 243, 457
  - assigning labels 21, 197, 243
  - assigning values conditionally 234, 245
  - attributes 17, 20
  - counter 497
  - creating or modifying 111
  - creating/modifying 232
  - default names 392
  - DO groups 248
  - dropping 511
  - format overview 20
  - functionality 17
  - generating totals 76
  - index 371, 372, 374, 388
  - informat overview 21
  - initializing 144, 233
  - labels for 21
  - length of 20, 342
  - missing values 17
  - naming conventions 18, 107
  - reading ranges 420
  - renaming 300
  - requesting subtotals 77, 189
  - retaining values 509
  - SELECT groups 245
  - selecting in list reports 69
  - selecting to drop and keep 241
  - specifying in FREQ procedure 189
  - specifying in MEANS procedure 181
  - specifying lengths 238

- specifying names in programs 107
- subsetting data 240
- sum 233
- target 316, 332
- temporary 181, 261, 302
- testing programs 159
- types of 19
- VARNUM option
  - CONTENTS procedure 35
  - CONTENTS statement, DATASETS procedure 35

## W

- w. format 91
- w.d format 91
- w.d informat 318, 406, 407
- web browsers
  - See [browsers](#)
- WEEKDATEw. format 456
- WEEKDAY function 325, 329
- WHEN statement 245
- WHERE statement, DATA step
  - automatic conversions and 318
  - reading Excel data 125
- WHERE statement, PRINT procedure
  - CONTAINS operator 73
  - examples 73
  - specifying compound expressions 73

- specifying expressions 72
- syntax 71
- Windows environment
  - SAS library implementation 12, 29
  - storing files 12
  - unbalanced quotation marks 59
- WORDDATEw. format 457
- Work library 11
- writing observations explicitly 263

## Y

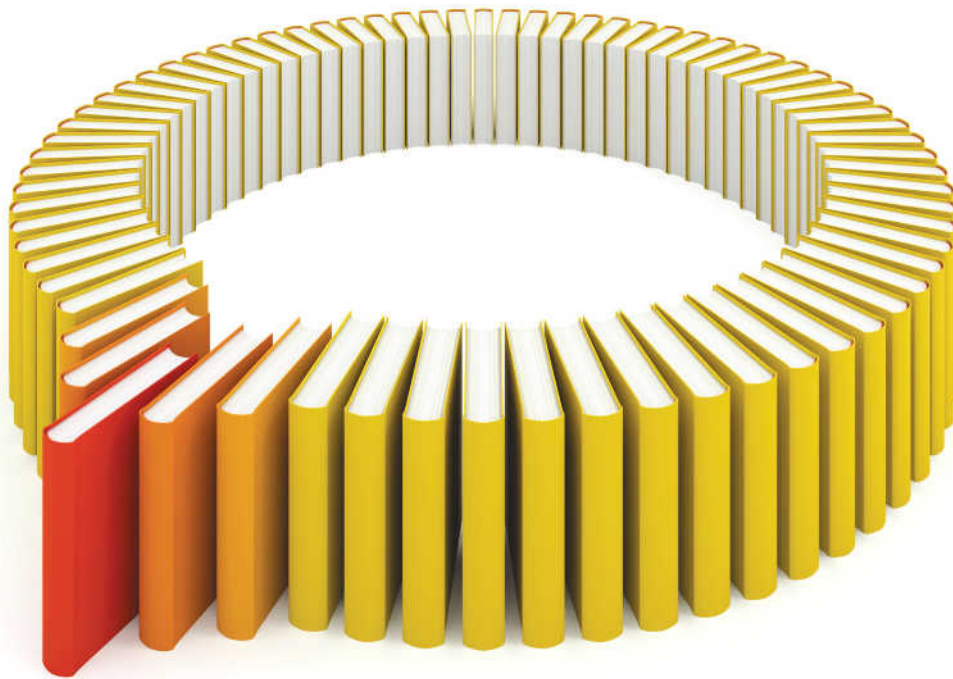
- YEAR function
  - examples 327
  - functionality 314
  - manipulating date values 326
  - syntax 326
  - typical use 325
- YEARCUTOFF= system option
  - functionality 451
  - handling four-digit years 44
  - handling two-digit years 42, 452
- YRDIF function 325, 338

## Z

- z/OS environment
  - SAS library implementation 12, 29
  - unbalanced quotation marks 59, 60







# Gain Greater Insight into Your SAS® Software with SAS Books.

Discover all that you need on your journey to knowledge and empowerment.



SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration. Other brand and product names are trademarks of their respective companies. © 2013 SAS Institute Inc. All rights reserved. S107969US.0613

